Swift 6.4

Swift 프로그래밍 언어

한국어 번역본

The Swift Programming Language

언어 가이드 · 언어 참조 · Swift 6.4 번역자 보충

한국어 번역 bbiguduk

영문 원본 The Swift Project

2026년 10월 14일 · PDF 판

이 PDF에 대하여

이 책은 The Swift Programming Language의 한국어 번역을 PDF로 편집한 문서입니다. Swift 6.4 기준의 수정 내용과 번역자 보충을 포함하며, 웹 문서의 목차 순서대로 전체 본문과 코드 예제, 삽화를 수록했습니다.

목차의 항목을 누르면 해당 장으로 이동합니다. PDF 뷰어의 책갈피에서 장과 절을 탐색할 수 있으며, 본문의 문서 참조와 외부 링크도 사용할 수 있습니다.

영문 원본: The Swift Project · 한국어 번역: bbiguduk. 한국어 번역, 기술적 정정, 보충 설명과 PDF 편집을 적용했습니다. 원본의 Apache License 2.0을 문서 끝에 수록했습니다.

한국어 문서 저장소

이 PDF의 한국어 소스 (29dcb02)

영문 원본 소스 (1c059843)

Copyright (c) 2014 - 2023 Apple Inc. and the Swift project authors

목차

Swift 시작하기 **5**

Swift 에 대해 (About Swift) 5

버전 호환성 (Version Compatibility) 6

Swift 둘러보기 (A Swift Tour) 7

언어 가이드 **25**

기본 (The Basics) 25

기본 연산자 (Basic Operators) 45

문자열과 문자 (Strings and Characters) 56

컬렉션 타입 (Collection Types) 72

제어 흐름 (Control Flow) 87

함수 (Functions) 112

클로저 (Closures) 124

열거형 (Enumerations) 136

구조체와 클래스 (Structures and Classes) 144

프로퍼티 (Properties) 151

메서드 (Methods) 168

서브스크립트 (Subscripts) 174

상속 (Inheritance) 179

초기화 (Initialization) 185

소멸 (Deinitialization) 214

옵셔널 체이닝 (Optional Chaining) 217

오류 처리 (Error Handling) 226

동시성 (Concurrency) 235

매크로 (Macros) 249

타입 캐스팅 (Type Casting) 259

중첩 타입 (Nested Types) 264

확장 (Extensions) 266
프로토콜 (Protocols) 272
제네릭 (Generics) 295
불투명 타입과 박싱 프로토콜 타입 (Opaque and Boxed Protocol Types) 313
자동 참조 카운팅 (Automatic Reference Counting) 321
메모리 안전성 (Memory Safety) 338
접근 제어 (Access Control) 344
고급 연산자 (Advanced Operators) 354

언어 참조 370

언어 참조에 대해 (About the Language Reference) 370
어휘 구조 (Lexical Structure) 371
타입 (Types) 383
표현식 (Expressions) 397
구문 (Statements) 426
선언 (Declarations) 443
속성 (Attributes) 484
패턴 (Patterns) 509
제네릭 매개변수와 인자 (Generic Parameters and Arguments) 514
문법 요약 (Summary of the Grammar) 518

개정 이력 537

Document Revision History 537

번역자 보충 542

Swift 6.4 번역자 보충 (Swift 6.4 Translation Notes) 542

연락처 545

연락처 545

원본 라이선스 546

Swift 시작하기

Swift 에 대해 (About Swift)

언어의 높은 수준의 목표를 이해합니다.

Swift는 폰, 태블릿, 데스크탑, 서버 또는 코드를 실행할 수 있는 모든 곳에서 작성하는 환상적인 소프트웨어입니다. 현대 언어 사고방식과 오픈소스 커뮤니티의 지혜를 결합한 안전하고 빠른 프로그래밍 언어입니다.

Swift는 숙련된 프로그래머에게 필요한 성능과 유연성을 놓치지 않으면서 새로운 프로그래머에게 친숙합니다. 스크립팅 언어처럼 표현력이 풍부하고 즐거움이 가득한 프로그래밍 언어입니다. 컴파일러는 성능에 최적화 되어있고 언어는 개발에 최적화 되어있으며, 어느쪽도 타협하지 않습니다.

Swift는 최신 프로그래밍 패턴을 채택하여 많은 클래스의 일반적인 프로그래밍 오류를 정의합니다:

- 변수는 항상 사용 전에 초기화됩니다.
- 배열(Array) 인덱스는 범위 초과(out-of-bounds) 여부가 검사됩니다.
- 정수(Integer) 연산은 오버플로우(overflow) 여부가 검사됩니다.
- 옵셔널(Optional)은 `nil` 값이 명시적으로 처리되도록 합니다.
- 메모리는 자동으로 관리됩니다.
- 오류 처리를 통해 예기치 않은 오류를 처리할 수 있습니다.

Swift 코드는 최신 하드웨어를 최대한 활용하기 위해 컴파일되고 최적화 됩니다. 구문과 표준 라이브러리는 코드를 작성하는 방법이 최상의 성능을 발휘해야 한다는 기본원칙에 따라 설계되었습니다. 안전성과 속도의 조합으로 Swift는 “Hello, world!” 로 부터 전체 시스템에 이르기 까지 최고의 선택입니다.

Swift는 강력한 기능으로 타입 추론(type inference)과 패턴 매칭(pattern matching)을 다른 유명한 언어를 개발하는 개발자들에게 친숙한 현대적이고 가벼운 구문으로 결합하여 복잡한 것도 명확하고 간결한 방식으로 표현할 수 있습니다. 그 결과로 코드는 읽고, 쓰고, 유지하기 쉽습니다.

Swift 는 새롭고 강력한 기능으로 발전하고 있습니다. Swift 에 대한 목표는 야심찹니다. Swift 로 무엇을 만들지 기대가 됩니다.

Swift 시작하기

버전 호환성 (Version Compatibility)

이전 언어 모드에서 사용가능한 기능에 대해 배워봅니다.

이 책은 Xcode 27에 포함된 Swift의 기본 버전인, Swift 6.4에 대해 설명합니다. Swift 6.4, Swift 5, Swift 4.2, Swift 4로 작성된 코드를 빌드하기위해 Swift 6.4 컴파일러를 사용할 수 있습니다.

Swift 5 언어 모드를 사용하는 코드를 빌드하기위해 Swift 6.4 컴파일러를 사용하면, Swift 6.4의 새로운 기능을 사용할 수 있습니다 - 새로운 기능은 기본적으로 활성화 되거나 플래그로 인해 활성화 됩니다. 그러나, 엄격한 동시성 검사를 활성화 하려면, Swift 6.4 언어 모드로 업그레이드가 필요합니다.

또한, Swift 4와 Swift 4.2 코드를 빌드하기위해 Xcode 15.3을 사용하면, 대부분의 Swift 5 기능을 사용할 수 있습니다. 다시 말해, 다음의 변경사항은 Swift 5 이상의 언어 모드를 사용하는 코드에서만 사용가능 합니다:

- 불투명 타입(opaque type)을 반환하는 함수는 Swift 5.1 런타임이 필요합니다.
- `try?` 표현식은 이미 옵셔널(optional)을 반환하는 표현식에 추가로 옵셔널 표현식을 도입하지 않습니다.
- 큰 정수(integer) 리터럴 초기화 표현식은 올바른 정수(integer) 타입으로 추론합니다. 예를 들어 `UInt64(0xffff_ffff_ffff_ffff)`는 오버플로우가 아닌 올바른 값입니다.

동시성(Concurrency)은 Swift 5 언어 모드와 동시성 타입을 제공하는 Swift 표준 라이브러리의 버전이 필요합니다. 애플 플랫폼에서 배포 대상(deployment target)을 적어도 iOS 13, macOS 10.15, tvOS 13, watchOS 6, visionOS 1로 설정합니다.

Swift 6.4로 작성된 타겟은 Swift 5, Swift 4.2, Swift 4로 작성된 타겟에 의존할 수 있으며 그 반대의 경우도 마찬가지입니다. 즉, 여러 프레임워크로 분할 된 대규모 프로젝트가 있는 경우 코드를 새로운 언어 버전으로 한번에 하나씩 프레임워크로 마이그레이션 할 수 있습니다.

Swift 시작하기

Swift 둘러보기 (A Swift Tour)

Swift의 기능과 구문을 살펴봅니다.

전통적으로 새로운 언어의 첫번째 프로그램은 “Hello, world!“를 출력해야 한다고 합니다. Swift에서는 이것을 한줄로 표기할 수 있습니다:

```
print("Hello, world!")
// Prints "Hello, world!".
```

이 구문은 다른 언어를 알고 있다면 익숙합니다 - Swift에서 이 코드의 라인은 완벽한 프로그램입니다. 문자 출력 또는 문자열 처리와 같은 기능을 위한 별도의 라이브러리를 가져올 필요는 없습니다. 전역 범위로 작성한 코드는 프로그램의 전체에서 사용되기 때문에 `main()` 함수가 필요하지 않습니다. 모든 구문에 끝에 세미콜론을 작성할 필요도 없습니다.

이 둘러보기를 통해 다양한 프로그래밍 작업을 수행하는 방법을 보여줌으로써 Swift에서 코드 작성을 시작할 수 있는 충분한 정보를 제공합니다. 이해하지 못하는 부분이 있어도 걱정하지 마시기 바랍니다. 이 둘러보기에서 소개된 모든 내용은 이 책의 나머지 부분에 자세히 설명되어 있습니다.

간단한 값 (Simple Values)

상수를 만들기 위해 `let`을 사용하고 변수를 만들기 위해 `var`를 사용합니다. 상수는 컴파일 때 알 필요는 없지만 반드시 한번은 할당 해야 합니다. 이것은 값은 한번만 할당되지만 여러 위치에서 사용할 수 있다는 의미입니다.

```
var myVariable = 42
myVariable = 50
let myConstant = 42
```

상수 또는 변수는 할당하려는 값과 동일한 타입이어야 합니다. 하지만 항상 타입을 명시해야 하는 것은 아닙니다. 상수 또는 변수를 생성할 때 값을 제공하면 컴파일러는 타입을 유추합니다. 위의 예시에서 `myVariable`의 초기값이 정수이므로 컴파일러는 정수로 유추합니다.

초기값이 충분한 정보를 제공하지 않거나 (초기값이 없는 경우), 변수 뒤에 콜론으로 구분하여 타입을 지정해야 합니다.

```
let implicitInteger = 70
let implicitDouble = 70.0
let explicitDouble: Double = 70
```

실습

명시적 타입이 `Float`이고 값이 `4` 인 상수를 만들어 봅시다.

값은 다른 타입의 값으로 절대 변경되지 않습니다. 값을 다른 타입으로 변경해야 한다면 원하는 타입의 인스턴스를 명시적으로 만들어야 합니다.

```
let label = "The width is "
let width = 94
let widthLabel = label + String(width)
```

실습

마지막 줄에서 `String`으로 변환하는 부분을 삭제해 보세요. 어떠한 오류가 발생합니까?

문자열(string)에 값을 포함하는 더 간단한 방법이 있습니다: 소괄호 안에 값을 작성하고 소괄호 전에 역슬래시 (\)를 작성하면 됩니다. 예를 들어:

```
let apples = 3
let oranges = 5
let appleSummary = "I have \(apples) apples."
let fruitSummary = "I have \(apples + oranges) pieces of fruit."
```

실습

`\()`을 사용하여 문자열에 부동소수점(floating-point) 계산을 포함하고 인사말에 누군가의 이름을 포함해 보세요.

여러줄의 문자열에 대해 쌍따옴표 3개(`"""`)를 사용하면 됩니다. 닫는 따옴표의 들여쓰기와 일치하는 한 각 인용된 줄의 시작부분에 있는 들여쓰기는 제거됩니다. 예를 들어:

```
let quotation = """
        Even though there's whitespace to the left,
        the actual lines aren't indented.
            Except for this line.
        Double quotes (") can appear without being escaped.

        I still have \(apples + oranges) pieces of fruit.
        """
```

대괄호(`[]`)를 사용하여 배열(array)과 딕셔너리(dictionary)를 생성하고 대괄호에 인덱스 또는 키를 작성하여 해당 요소에 접근할 수 있습니다. 마지막 요소 뒤에 쉼표도 허용 합니다.

```
var fruits = ["strawberries", "limes", "tangerines"]
fruits[1] = "grapes"

var occupations = [
    "Malcolm": "Captain",
    "Kaylee": "Mechanic",
]
occupations["Jayne"] = "Public Relations"
```

배열은 요소를 추가함에 따라 자동으로 크기가 늘어납니다.

```
fruits.append("blueberries")
print(fruits)
// Prints "["strawberries", "grapes", "tangerines", "blueberries"]".
```

빈 배열 또는 딕셔너리를 작성할 때도 괄호를 사용합니다. 배열을 작성할 때는 `[]`, 딕셔너리는 `[:]`로 작성합니다.

```
fruits = []
occupations = [:]
```

새로운 변수 또는 다른 장소의 타입 정보가 없는 곳에 빈 배열 또는 빈 딕셔너리를 할당하려면 타입을 명시해야 합니다.

```
let emptyArray: [String] = []
let emptyDictionary: [String: Float] = [:]
```

제어 흐름 (Control Flow)

조건문을 만드려면 `if`와 `switch`를 사용하고 루프를 만드려면 `for-in`, `while`, 그리고 `repeat-while`을 사용합니다. 조건문이나 루프 변수를 둘러싼 소괄호는 선택 사항 입니다. 본문을 둘러싼 중괄호는 필수사항 입니다.

```
let individualScores = [75, 43, 103, 87, 12]
var teamScore = 0
for score in individualScores {
    if score > 50 {
        teamScore += 3
    } else {
        teamScore += 1
    }
}
print(teamScore)
// Prints "11".
```

`if` 문에서는 조건부가 반드시 Boolean으로 표현되어야 합니다 - 즉, `if score { ... }`와 같은 코드는 암시적으로 0과 비교하는 것이 아니라 오류를 의미합니다.

조건을 기준으로 값을 선택하기위해 할당의 동등 사인(=) 뒤나 `return` 뒤에 `if` 또는 `switch`를 작성할 수 있습니다.

```
let scoreDecoration = if teamScore > 10 {
    "🎉"
} else {
    ""
}
print("Score:", teamScore, scoreDecoration)
// Prints "Score: 11 🎉".
```

`if`와 `let`을 사용하여 누락될 수 있는 값에 대해 사용할 수 있습니다. 이러한 값은 옵셔널(optional)로 표기됩니다. 옵셔널 값은 값을 포함하거나 값이 없음을 나타내는 `nil`을 포함합니다. 옵셔널로 값을 표시하기 위해 값의 타입 뒤에 물음표(`?`)를 작성합니다.

```
var optionalString: String? = "Hello"
print(optionalString == nil)
// Prints "false".

var optionalName: String? = "John Appleseed"
var greeting = "Hello!"
if let name = optionalName {
```

```
    greeting = "Hello, \(name)"
}
print(greeting)
```

실습

`optionalName`을 `nil`로 변경하세요. 어떤 인사말을 얻습니까? `optionalName`이 `nil` 인 경우 다른 인사말을 설정하기 위해 `else` 절을 추가하세요.

옵셔널 값이 `nil`이면 조건은 `false`이고 중괄호 안에 코드는 건너뜁니다. 옵셔널 값이 `nil`이 아니면 옵셔널 값은 언래핑 되고 `let` 뒤의 상수로 할당되어 코드 블록 안에서 언래핑된 값으로 사용할 수 있습니다.

옵셔널 값을 처리하는 다른 방법은 `??` 연산자를 사용하여 기본값을 제공하는 것입니다. 옵셔널 값이 없다면 기본값이 대신 사용됩니다.

```
let nickname: String? = nil
let fullName: String = "John Appleseed"
let informalGreeting = "Hi \(nickname ?? fullName)"
```

더 짧게 같은 이름으로 언래핑된 값을 사용할 수 있습니다.

```
if let nickname {
    print("Hey, \(nickname)")
}
// Doesn't print anything, because nickname is nil.
```

스위치(switch)는 모든 종류의 데이터와 다양한 비교 작업을 지원합니다 - 스위치는 정수(integer) 및 동등성 비교로 제한되지 않습니다.

```
let vegetable = "red pepper"
switch vegetable {
case "celery":
    print("Add some raisins and make ants on a log.")
case "cucumber", "watercress":
    print("That would make a good tea sandwich.")
case let x where x.hasSuffix("pepper"):
    print("Is it a spicy \(x)?")
default:
    print("Everything tastes good in soup.")
}
// Prints "Is it a spicy red pepper?".
```

실습

Default 케이스를 삭제해 보세요. 어떤 오류가 발생합니까?

패턴과 일치하는 값을 상수에 할당하기 위해 패턴에서 `let`을 어떻게 사용하는지 확인 하시기 바랍니다.

일치하는 스위치 케이스(case) 문을 실행하고 프로그램은 스위치 문을 종료합니다. 다음 케이스로 이어서 실행되지 않기 때문에 각 케이스 코드에 명시적으로 스위치 종료를 할 필요가 없습니다.

`for-in`을 사용하여 각 키-값 쌍에 사용할 이름의 쌍을 제공하여 딕셔너리의 항목을 조회합니다. 딕셔너리는 순서가 없는 컬렉션(collection)이므로 키와 값은 임의의 순서로 조회됩니다.

```
let interestingNumbers = [
    "Prime": [2, 3, 5, 7, 11, 13],
    "Fibonacci": [1, 1, 2, 3, 5, 8],
    "Square": [1, 4, 9, 16, 25],
]
var largest = 0
for (_, numbers) in interestingNumbers {
    for number in numbers {
        if number > largest {
            largest = number
        }
    }
}
print(largest)
// Prints "25".
```

실습

_을 변수의 이름으로 변경하고, 어떤 숫자가 가장 큰지 추적하세요.

조건이 바뀔 때까지 코드의 블록을 반복하려면 `while`을 사용해야 합니다. 대신 루프의 조건이 끝에 있을 수 있으므로 적어도 한번은 루프가 실행되도록 합니다.

```
var n = 2
while n < 100 {
    n *= 2
}
print(n)
// Prints "128".

var m = 2
repeat {
    m *= 2
} while m < 100
print(m)
// Prints "128".
```

실습

루프 조건이 이미 거짓일 때 `while`과 `repeat-while`이 어떻게 다른지 살펴보기위해 `m < 100`에서 `m < 0`으로 조건을 변경해 보세요.

인덱스의 범위를 만들기 위해선 `..<`을 사용하여 루프에 인덱스를 만들 수 있습니다.

```
var total = 0
for i in 0..<4 {
    total += i
}
print(total)
// Prints "6".
```

가장 상위 값을 생략하는 범위를 만들기 위해 `..<`을 사용하고 포함하려면 `...`을 사용합니다.

함수와 클로저 (Functions and Closures)

함수를 선언하려면 `func`을 사용합니다. 소괄호 안에 인자의 리스트와 함수의 이름으로 호출합니다. 함수의 반환 타입(return type)에서 매개변수(parameter) 이름과 타입을 구분하기 위해 –> 을 사용합니다.

```
func greet(person: String, day: String) -> String {
    return "Hello \(person), today is \(day)."
}
greet(person: "Bob", day: "Tuesday")
```

실습

day 매개변수를 지워 보세요. 인사말에 오늘의 특별 점심을 포함하기 위해 매개변수를 추가해 보세요.

기본적으로 함수는 매개변수 이름을 인자의 레이블로 사용합니다. 매개변수 이름 전에 인자 레이블을 작성하거나 인자 레이블을 사용하지 않으려면 _을 작성해야 합니다.

```
func greet(_ person: String, on day: String) -> String {
    return "Hello \(person), today is \(day)."
}
greet("John", on: "Wednesday")
```

튜플(tuple)을 사용하여복합값을 만듭니다 - 예를 들어 함수로 부터 여러개의 값을 반환할 때 사용합니다. 튜플의 요소는 이름 또는 번호로 참조할 수 있습니다.

```
func calculateStatistics(scores: [Int]) -> (min: Int, max: Int, sum: Int) {
    var min = scores[0]
    var max = scores[0]
    var sum = 0

    for score in scores {
        if score > max {
            max = score
        } else if score < min {
            min = score
        }
        sum += score
    }

    return (min, max, sum)
}
let statistics = calculateStatistics(scores: [5, 3, 100, 3, 9])
print(statistics.sum)
// Prints "120".
print(statistics.2)
// Prints "120".
```

함수는 중첩될 수 있습니다. 중첩된 함수는 외부 함수에서 선언한 변수에 접근할 수 있습니다. 중첩된 함수를 사용하여 길거나 복잡한 함수에서 코드를 구성할 수 있습니다.

```
func returnFifteen() -> Int {
    var y = 10
    func add() {
        y += 5
    }
    add()
    return y
}
returnFifteen()
```

함수는 1급 타입(first-class type)입니다. 이것은 함수가 다른 함수를 값으로 반환할 수 있다는 의미입니다.

```
func makeIncrementer() -> ((Int) -> Int) {
    func addOne(number: Int) -> Int {
        return 1 + number
    }
    return addOne
}
var increment = makeIncrementer()
increment(7)
```

함수는 다른 함수를 인자(argument) 중 하나로 가질 수 있습니다.

```
func hasAnyMatches(list: [Int], condition: (Int) -> Bool) -> Bool {
    for item in list {
        if condition(item) {
            return true
        }
    }
    return false
}
func lessThanTen(number: Int) -> Bool {
    return number < 10
}
var numbers = [20, 19, 7, 12]
hasAnyMatches(list: numbers, condition: lessThanTen)
```

함수는 클로저(closure)의 특별한 케이스 입니다: 코드의 블록은 나중에 호출될 수 있습니다. 클로저에 있는 코드는 클로저가 실행될 때 다른 범위에 있더라도 클로저가 생성된 범위에서 사용 가능한 변수와 함수와 같은 항목에 접근할 수 있습니다 - 중첩된 함수 예시에서 이미 살펴보았습니다. 중괄호({})로 코드를 묶어 이름없이 클로저를 작성할 수 있습니다. 본문으로 부터 인자와 반환 타입을 분리하기 위해 `in`을 사용합니다.

```
numbers.map({ (number: Int) -> Int in
    let result = 3 * number
    return result
})
```

실습

모든 홀수에 대해 0을 반환하도록 클로저를 다시 작성해보세요.

더 간단하게 클로저를 작성하기 위해 몇가지 선택 사항이 있습니다. 대리자(delegate)에 대한 콜백과 같이 클로저의 타입을 이미 알고 있다면 매개변수의 타입, 반환 타입 또는 둘다 생략할 수 있습니다. 단일 클로저 구문은 암시적으로 구문의 값만 반환합니다.

```
let mappedNumbers = numbers.map({ number in 3 * number })
print(mappedNumbers)
// Prints "[60, 57, 21, 36]".
```

이름 대신 숫자로 매개변수를 참조할 수 있습니다 - 이 방식은 매우 짧은 클로저에 유용합니다. 함수의 마지막 인자로 전달된 클로저는 소괄호 뒤에 바로 나타날 수 있습니다. 클로저가 함수의 유일한 인자일 때 소괄호는 생략할 수 있습니다.

```
let sortedNumbers = numbers.sorted { $0 > $1 }
print(sortedNumbers)
// Prints "[20, 19, 12, 7]".
```

객체와 클래스 (Objects and Classes)

`class` 뒤에 클래스의 이름을 사용하여 클래스를 생성합니다. 클래스에서 프로퍼티 선언은 클래스의 컨텍스트(context) 내에 있다는 점을 제외하고는 상수 또는 변수를 선언하는 방법과 동일합니다. 마찬가지로 메서드와 함수 선언도 동일한 방법으로 작성됩니다.

```
class Shape {
    var numberOfSides = 0
    func simpleDescription() -> String {
        return "A shape with \(numberOfSides) sides."
    }
}
```

실습

`let`을 사용하여 상수 프로퍼티를 추가하고, 인자를 가지는 다른 메서드를 추가해 보세요.

클래스 이름 뒤에 소괄호를 넣어 클래스의 인스턴스를 생성합니다. 인스턴스의 프로퍼티와 메서드에 접근하기 위해 점 문법을 사용합니다.

```
var shape = Shape()
shape.numberOfSides = 7
var shapeDescription = shape.simpleDescription()
```

이 버전의 Shape 클래스는 중요한 사항을 놓쳤습니다: 인스턴스가 생성될 때 클래스를 설정하는 이니셜라이저(initializer)이 누락되었습니다. `init` 을 사용하여 생성합니다.

```
class NamedShape {
    var numberOfSides: Int = 0
    var name: String

    init(name: String) {
        self.name = name
    }
```

```
    func simpleDescription() -> String {
        return "A shape with \(numberOfSides) sides."
    }
}
```

이니셜라이저에서 `name` 인자와 `name` 프로퍼티를 구분하기 위해 `self`가 어떻게 사용되는지 확인 하십시오. 이니셜라이저의 인자는 클래스의 인스턴스를 생성할 때 함수 호출처럼 전달됩니다. 모든 프로퍼티는 값이 할당되어야 합니다 - 선언 시(`numberOfSides`)나 이니셜라이저(`name`)에서 값을 할당해야 합니다.

객체가 할당 해제되기 전에 어떠한 정리작업이 필요하여 디이니셜라이저(deinitializer)을 생성하려면 `deinit`을 사용합니다.

하위 클래스(subclass)는 클래스 이름 뒤에 콜론으로 구분하여 상위 클래스(superclass)를 포함합니다. 클래스가 모든 표준 루트 클래스를 하위 클래스화 할 필요 없으므로 필요에 따라 상위 클래스를 포함하거나 생략할 수 있습니다.

상위 클래스의 구현을 재정의 하는 하위 클래스의 메서드는 `override`로 표시됩니다 -- 실수로 `override` 없이 메서드를 재정의 하면 오류로 컴파일러에 의해 감지됩니다. 컴파일러는 또한 `override`를 사용하지만 실제로는 상위 클래스의 어떤 메서드도 재정의하지 않는 메서드를 감지합니다.

```
class Square: NamedShape {
    var sideLength: Double

    init(sideLength: Double, name: String) {
        self.sideLength = sideLength
        super.init(name: name)
        numberOfSides = 4
    }

    func area() -> Double {
        return sideLength * sideLength
    }

    override func simpleDescription() -> String {
        return "A square with sides of length \(sideLength)."
    }
}
let test = Square(sideLength: 5.2, name: "my test square")
test.area()
test.simpleDescription()
```

실습

이니셜라이저에 반지름과 이름을 인자로 가지는 `Circle`이라는 `NamedShape`의 하위 클래스를 만들어 보세요. `Circle` 클래스에 `area()`와 `simpleDescription()` 메서드를 구현해 보세요.

저장된 단순 프로퍼티 외에도 프로퍼티는 getter와 setter를 가질 수 있습니다.

```
class EquilateralTriangle: NamedShape {
    var sideLength: Double = 0.0

    init(sideLength: Double, name: String) {
        self.sideLength = sideLength
        super.init(name: name)
```

```
        numberOfSides = 3
    }

    var perimeter: Double {
        get {
            return 3.0 * sideLength
        }
        set {
            sideLength = newValue / 3.0
        }
    }

    override func simpleDescription() -> String {
        return "An equilateral triangle with sides of length \(sideLength)."
    }
}
var triangle = EquilateralTriangle(sideLength: 3.1, name: "a triangle")
print(triangle.perimeter)
// Prints "9.3".
triangle.perimeter = 9.9
print(triangle.sideLength)
// Prints "3.3000000000000003".
```

`perimeter`의 setter에서 새로운 값은 암시적으로 `newValue`라는 이름을 가집니다. `set` 이후에 소괄호 안에 명시적으로 이름을 제공할 수 있습니다.

`EquilateralTriangle` 클래스의 이니셜라이저는 세가지 단계가 있습니다:

1. 하위 클래스가 선언한 프로퍼티의 값을 설정합니다.
2. 상위 클래스의 이니셜라이저를 호출합니다.
3. 상위 클래스에 의해 정의된 프로퍼티의 값을 변경합니다. 메서드(method), getter, 또는 setter를 사용하는 추가 설정 작업도 이 시점에서 수행할 수 있습니다.

프로퍼티를 계산할 필요는 없지만 새로운 값을 설정하기 전과 후에 실행되는 코드를 제공하려면 `willSet` 과 `didSet` 을 사용하면 됩니다. 이렇게 제공한 코드는 이니셜라이저의 외부에서 값이 변경될 때마다 실행됩니다. 예를 들어 아래의 클래스는 삼각형의 측면의 길이가 항상 사각형의 측면의 길이와 같다는 것을 확인합니다.

```
class TriangleAndSquare {
    var triangle: EquilateralTriangle {
        willSet {
            square.sideLength = newValue.sideLength
        }
    }
    var square: Square {
        willSet {
            triangle.sideLength = newValue.sideLength
        }
    }
    init(size: Double, name: String) {
        square = Square(sideLength: size, name: name)
        triangle = EquilateralTriangle(sideLength: size, name: name)
    }
}
var triangleAndSquare = TriangleAndSquare(size: 10, name: "another test shape")
print(triangleAndSquare.square.sideLength)
// Prints "10.0".
```

```
print(triangleAndSquare.triangle.sideLength)
// Prints "10.0".
triangleAndSquare.square = Square(sideLength: 50, name: "larger square")
print(triangleAndSquare.triangle.sideLength)
// Prints "50.0".
```

옵셔널 값으로 동작할 때 메서드, 프로퍼티, 그리고 서브스크립트(subscript)와 같은 동작 전에 ? 을 작성할 수 있습니다. ? 전의 값이 nil이면 ? 이후의 모든 것은 무시되고 전체 표현의 값은 nil입니다. 그렇지 않으면 옵셔널 값은 언래핑 되고 ? 후에 모든 동작은 언래핑된 값으로 동작합니다. 이 경우, 전체 표현식의 값은 옵셔널 값입니다.

```
let optionalSquare: Square? = Square(sideLength: 2.5, name: "optional square")
let sideLength = optionalSquare?.sideLength
```

열거형과 구조체 (Enumerations and Structures)

열거형을 생성하기 위해 enum을 사용합니다. 클래스와 다른 명명된 타입과 같이 열거형은 메서드를 가질 수 있습니다.

```
enum Rank: Int {
    case ace = 1
    case two, three, four, five, six, seven, eight, nine, ten
    case jack, queen, king

    func simpleDescription() -> String {
        switch self {
        case .ace:
            return "ace"
        case .jack:
            return "jack"
        case .queen:
            return "queen"
        case .king:
            return "king"
        default:
            return String(self.rawValue)
        }
    }
}
let ace = Rank.ace
let aceRawValue = ace.rawValue
```

실습

원시값으로 2개의 Rank 값을 비교하는 함수를 작성해 보세요.

기본적으로 Swift는 0을 시작으로 매번 증가하는 원시값(raw value)을 할당하지만 특정 값으로 이 동작을 변경할 수 있습니다. 위의 예시에서 Ace는 명시적으로 1의 값이 주어지고 나머지 원시값은 순서대로 할당됩니다. 열거형의 원시 타입으로 문자열 또는 부동소수점도 사용할 수 있습니다. 열거형 케이스의 원시값에 접근하기 위해 rawValue 프로퍼티를 사용합니다.

원시값으로 부터 열거형의 인스턴스를 생성하기 위해 init?(rawValue:) 이니셜라이저를 사용합니다. 원시값이 일치하는 열거형 케이스나 Rank에 일치하는 항목이 없으면 nil을 반환합니다.

```
if let convertedRank = Rank(rawValue: 3) {
    let threeDescription = convertedRank.simpleDescription()
}
```

열거형의 케이스 값은 원시값을 작성하는 다른 방법이 아니라 실제 값입니다. 실제로 의미 없는 원시값의 케이스의 경우에는 제공할 필요가 없습니다.

```
enum Suit {
    case spades, hearts, diamonds, clubs

    func simpleDescription() -> String {
        switch self {
        case .spades:
            return "spades"
        case .hearts:
            return "hearts"
        case .diamonds:
            return "diamonds"
        case .clubs:
            return "clubs"
        }
    }
}
let hearts = Suit.hearts
let heartsDescription = hearts.simpleDescription()
```

실습

스페이드와 클럽의 경우 “black”을 반환하고 하트와 다이아몬드의 경우 “red”를 반환하는 `color()` 메서드를 `Suit`에 추가하십시오.

위에서 열거형의 `hearts` 케이스를 참조하는 두가지 방법에 유의하십시오: `hearts` 상수에 값을 할당할 때 명시적으로 타입을 지정하지 않았으므로 열거형 케이스 `Suit.hearts` 전체 이름으로 참조됩니다. 스위치에서 `self`의 값은 이미 카드(suit)로 알고 있기 때문에 열거형 케이스는 짧은 형식인 `.hearts`로 참조됩니다. 값의 타입을 이미 알고 있다면 언제나 짧은 형식을 사용할 수 있습니다.

열거형이 원시값을 갖는 경우 선언의 부분으로 결정됩니다. 이것은 특정 열거형 케이스의 모든 인스턴스는 항상 같은 원시값을 갖는다는 의미입니다. 열거형 케이스의 또다른 선택은 케이스와 연관값(associated value)을 가지는 것입니다 -- 이러한 값은 인스턴스를 생성할 때 결정되고 열거형 케이스의 각 인스턴스에 대해 다를 수 있습니다. 연관값은 열거형 케이스 인스턴스의 저장 프로퍼티처럼 동작한다고 생각할 수 있습니다. 예를 들어 서버에서 일출과 일몰 시간에 대해 요청한다고 가정해 봅시다. 서버는 요청된 정보에 대한 응답을 하거나 무엇이 잘못되었는지에 대한 설명을 응답합니다.

```
enum ServerResponse {
    case result(String, String)
    case failure(String)
}

let success = ServerResponse.result("6:00 am", "8:09 pm")
let failure = ServerResponse.failure("Out of cheese.")

switch success {
case let .result(sunrise, sunset):
```

```
    print("Sunrise is at \(sunrise) and sunset is at \(sunset).")
case let .failure(message):
    print("Failure...  \(message)")
}
// Prints "Sunrise is at 6:00 am and sunset is at 8:09 pm.".
```

실습

`ServerResponse` 에 세번째 케이스를 추가하십시오.

스위치 케이스에 대한 값이 일치하는 부분으로 `ServerResponse` 값에서 일출과 일몰 시간이 어떻게 추출되는지 확인하시기 바랍니다.

구조체를 생성하기 위해 `struct` 를 사용합니다. 구조체는 메서드와 이니셜라이저를 포함하여 클래스와 동일한 동작을 많이 지원합니다. 구조체와 클래스의 가장 큰 차이점은 구조체는 코드에서 전달될 때 항상 복사되지만 클래스는 참조로 전달됩니다.

```
struct Card {
    var rank: Rank
    var suit: Suit
    func simpleDescription() -> String {
        return "The \(rank.simpleDescription()) of \(suit.simpleDescription())"
    }
}
let threeOfSpades = Card(rank: .three, suit: .spades)
let threeOfSpadesDescription = threeOfSpades.simpleDescription()
```

실습

숫자와 문양의 모든 조합을 가진 카드를 포함하는 배열을 반환하는 함수를 작성하세요.

동시성 (Concurrency)

비동기적으로 실행되는 함수를 나타내기 위해 `async`를 사용합니다.

```
func fetchUserID(from server: String) async -> Int {
    if server == "primary" {
        return 97
    }
    return 501
}
```

앞에 `await`를 작성하여 비동기 함수를 호출하는 것을 나타냅니다.

```
func fetchUsername(from server: String) async -> String {
    let userID = await fetchUserID(from: server)
    if userID == 501 {
        return "John Appleseed"
    }
    return "Guest"
}
```

비동기 함수를 호출하기 위해 `async let`을 사용하여 다른 비동기 코드와 병렬로 실행할 수 있습니다. `await`를 작성하여 반환된 값을 사용합니다.

```
func connectUser(to server: String) async {
    async let userID = fetchUserID(from: server)
    async let username = fetchUsername(from: server)
    let greeting = await "Hello \(username), user ID \(userID)"
    print(greeting)
}
```

비동기 함수의 반환을 기다리지 않고 동기 코드에서 비동기 함수를 호출하려면 `Task`를 사용합니다.

```
Task {
    await connectUser(to: "primary")
}
// Prints "Hello Guest, user ID 97".
```

동시성 코드를 구성 하기위해 태스크 그룹(task group)을 사용합니다.

```
let userIDs = await withTaskGroup(of: Int.self) { group in
    for server in ["primary", "secondary", "development"] {
        group.addTask {
            return await fetchUserID(from: server)
        }
    }

    var results: [Int] = []
    for await result in group {
        results.append(result)
    }
    return results
}
```

액터(Actor)는 비동기 함수가 동시에 동일한 액터의 인스턴스와 안전하게 상호작용 할 수 있다는 것을 제외하고는 클래스와 유사합니다.

```
actor ServerConnection {
    var server: String = "primary"
    private var activeUsers: [Int] = []
    func connect() async -> Int {
        let userID = await fetchUserID(from: server)
        // ... communicate with server ...
        activeUsers.append(userID)
        return userID
    }
}
```

액터의 메서드를 호출하거나 프로퍼티에 접근할 때, 액터에서 이미 실행 중인 다른 코드가 완료될 때까지 기다리고 있는 것을 나타내기 위해 코드에 `await`를 표기합니다.

```
let server = ServerConnection()
let userID = await server.connect()
```

프로토콜과 확장 (Protocols and Extensions)

프로토콜을 선언하려면 `protocol`을 사용해야 합니다.

```
protocol ExampleProtocol {
    var simpleDescription: String { get }
    mutating func adjust()
}
```

클래스, 열거형, 그리고 구조체는 프로토콜을 채택할 수 있습니다.

```
class SimpleClass: ExampleProtocol {
    var simpleDescription: String = "A very simple class."
    var anotherProperty: Int = 69105
    func adjust() {
        simpleDescription += "  Now 100% adjusted."
    }
}
var a = SimpleClass()
a.adjust()
let aDescription = a.simpleDescription

struct SimpleStructure: ExampleProtocol {
    var simpleDescription: String = "A simple structure"
    mutating func adjust() {
        simpleDescription += " (adjusted)"
    }
}
var b = SimpleStructure()
b.adjust()
let bDescription = b.simpleDescription
```

실습

`ExampleProtocol`에 다른 요구사항을 추가하십시오. 프로토콜을 여전히 준수하기 위해 `SimpleClass`와 `SimpleStructure`에 무엇을 바꿔야 할까요?

구조체를 수정하는 메서드를 표시하기위해 `SimpleStructure`의 선언에서 `mutating` 키워드 사용을 주목하시기 바랍니다. 클래스의 메서드는 항상 클래스를 수정할 수 있으므로 `SimpleClass`의 선언에는 `mutating`으로 표시된 메서드가 필요하지 않습니다.

새로운 메서드와 연산 프로퍼티와 같이 존재하는 타입에 기능을 추가하려면 `extension`을 사용합니다. 확장(extension)을 사용하여 다른 곳에 선언된 타입 또는 라이브러리나 프레임워크에서 가져온 타입에 프로토콜 준수를 추가할 수 있습니다.

```
extension Int: ExampleProtocol {
    var simpleDescription: String {
        return "The number \(self)"
    }
    mutating func adjust() {
        self += 42
    }
}
print(7.simpleDescription)
// Prints "The number 7".
```

실습

`Double` 타입에 `absoluteValue` 프로퍼티를 추가하기 위해 확장을 작성하십시오.

다른 타입처럼 프로토콜 이름을 사용할 수 있습니다 -- 예를 들어 타입이 다르지만 모두 단일 프로토콜을 준수하는 객체의 컬렉션을 생성할 수 있습니다. 박싱 프로토콜 타입의 값을 다룰 때, 프로토콜 정의에 없는 메서드는 사용할 수 없습니다.

```
let protocolValue: any ExampleProtocol = a
print(protocolValue.simpleDescription)
// Prints "A very simple class.  Now 100% adjusted.".
// print(protocolValue.anotherProperty)  // Uncomment to see the error
```

`protocolValue` 변수에 `SimpleClass`의 런타임 타입을 가지고 있더라도, 컴파일러는 주어진 `ExampleProtocol`의 타입으로 처리합니다. 즉, 클래스가 프로토콜을 준수하면서 구현한 추가적인 메서드나 프로퍼티에 실수로 접근할 수 없습니다.

오류 처리 (Error Handling)

`Error` 프로토콜을 채택하는 모든 타입을 사용하여 오류를 나타냅니다.

```
enum PrinterError: Error {
    case outOfPaper
    case noToner
    case onFire
}
```

`throw`를 사용하여 오류를 발생시키고 `throws`를 사용하여 오류를 발생시킬 수 있는 함수를 나타냅니다. 함수에서 오류를 발생시키면, 그 함수는 즉시 반환되며 함수를 호출한 코드가 오류를 처리합니다.

```
func send(job: Int, toPrinter printerName: String) throws -> String {
    if printerName == "Never Has Toner" {
        throw PrinterError.noToner
    }
    return "Job sent"
}
```

오류를 처리하는 방법은 몇가지 존재합니다. 한가지 방법은 `do-catch`를 사용하는 것입니다. `do` 블록 내에서 앞에 `try`를 작성하여 오류가 발생할 수 있는 코드임을 표시합니다. `catch` 블록 내에서 오류는 다른 이름으로 지정하기 전까지는 자동으로 `error`라는 이름으로 주어집니다.

```
do {
    let printerResponse = try send(job: 1040, toPrinter: "Bi Sheng")
    print(printerResponse)
} catch {
    print(error)
}
// Prints "Job sent".
```

실습

`send(job:toPrinter:)` 함수에서 오류가 발생하도록 프린터 이름을 `"Never Has Toner"`로 변경합니다.

특정 오류를 처리하는 여러개의 `catch` 블록을 제공할 수 있습니다. `catch` 뒤에 패턴을 작성하는 방법은 switch에서 `case` 뒤에 패턴을 작성하는 방식과 동일합니다.

```
do {
    let printerResponse = try send(job: 1440, toPrinter: "Gutenberg")
    print(printerResponse)
} catch PrinterError.onFire {
    print("I'll just put this over here, with the rest of the fire.")
} catch let printerError as PrinterError {
    print("Printer error: \(printerError).")
} catch {
    print(error)
}
// Prints "Job sent".
```

실습

`do` 블록 내에 오류를 발생시키는 코드를 추가하세요. 첫번째 `catch` 블록에서 오류를 처리하려면 어떤 종류의 오류를 발생시켜야 할까요? 두번째와 세번째 블록에 대해선 무슨 오류를 발생시켜야 할까요?

오류를 처리하는 다른 방법은 결과를 옵셔널로 바꾸는 `try?`를 사용하는 것입니다. 함수에서 오류가 발생하면, 특정 오류는 버려지고 결과는 `nil` 입니다. 그렇지 않으면 결과는 함수가 반환한 옵셔널 값을 포함합니다.

```
let printerSuccess = try? send(job: 1884, toPrinter: "Mergenthaler")
let printerFailure = try? send(job: 1885, toPrinter: "Never Has Toner")
```

`defer`를 사용하여 함수가 반환되기 직전에, 함수 내의 다른 코드가 모두 실행된 후에 실행될 코드를 작성할 수 있습니다. 이 코드는 함수에서 오류가 발생하는지 여부와 관계없이 실행됩니다. 서로 다른 시간에 실행되어야 하는 경우에도 설정 및 정리 코드를 나란히 작성하기 위해 `defer`를 사용할 수 있습니다.

```
var fridgeIsOpen = false
let fridgeContent = ["milk", "eggs", "leftovers"]

func fridgeContains(_ food: String) -> Bool {
    fridgeIsOpen = true
    defer {
        fridgeIsOpen = false
    }

    let result = fridgeContent.contains(food)
    return result
}
if fridgeContains("banana") {
    print("Found a banana")
}
print(fridgeIsOpen)
// Prints "false".
```

제네릭 (Generics)

제네릭(Generic) 함수 또는 타입을 만들기 위해 꺾쇠 괄호 안에 이름을 작성합니다.

```
func makeArray<Item>(repeating item: Item, numberOfTimes: Int) -> [Item] {
    var result = [Item]()
    for _ in 0..<numberOfTimes {
        result.append(item)
    }
    return result
}
makeArray(repeating: "knock", numberOfTimes: 4)
```

함수와 메서드 뿐만 아니라 클래스, 열거형, 구조체도 제네릭 형태로 만들 수 있습니다.

```
// Reimplement the Swift standard library's optional type
enum OptionalValue<Wrapped> {
    case none
    case some(Wrapped)
}
var possibleInteger: OptionalValue<Int> = .none
possibleInteger = .some(100)
```

`where`를 본문 바로 앞에 사용하여 요구사항 목록을 지정할 수 있습니다 - 예를 들어, 타입이 프로토콜을 구현하도록 요구하거나, 두 타입이 동일하도록 요구하거나, 클래스가 특정 상위 클래스를 가져오도록 요구할 수 있습니다.

```
func anyCommonElements<T: Sequence, U: Sequence>(_ lhs: T, _ rhs: U) -> Bool
    where T.Element: Equatable, T.Element == U.Element
{
    for lhsItem in lhs {
        for rhsItem in rhs {
            if lhsItem == rhsItem {
                return true
            }
        }
    }
    return false
}
anyCommonElements([1, 2, 3], [3])
```

실습

`anyCommonElements(_:_:)` 함수를 수정하여 두 시퀀스가 공통으로 가진 요소들을 배열로 반환하는 함수를 작성하세요.

`<T: Equatable>`을 작성하는 것은 `<T> ... where T: Equatable`으로 작성하는 것과 동일합니다.

언어 가이드

기본 (The Basics)

일반적인 종류의 데이터로 동작하고 기본 구문을 작성합니다.

Swift는 정수에 사용되는 `Int`, 부동소수점(floating-point) 값에 사용되는 `Double`, Boolean 값을 나타내는 `Bool`, 그리고 텍스트를 나타내는 `String`과 같은 기본적인 데이터 타입을 제공합니다. Swift는 컬렉션 타입 (Collection Types)에서 설명된 대로 `Array`, `Set`, `Dictionary`라는 세 가지 기본 컬렉션 타입을 제공합니다.

Swift는 변수를 식별 가능한 이름으로 값을 저장하고 참조합니다. Swift는 값을 변경할 수 없는 변수를 광범위하게 사용합니다. 이러한 변수를 상수라고 하며, 값을 변경할 필요가 없을 때 코드를 더 안전하고 명확하게 만들기위해 Swift에서 사용됩니다.

익숙한 타입 외에도, Swift는 튜플(tuple)과 같은 고급 타입도 있습니다. 튜플은 값을 그룹화 하여 생성하거나 전달할 수 있습니다. 함수에서 여러 값을 단일복합값으로 반환할 때 튜플을 사용할 수 있습니다.

Swift는 옵셔널(optional) 타입을 사용하여 값이 존재하지 않는 상태를 처리합니다. 옵셔널은 “값이 있고 x이다” 또는 “값이 없다“를 알려줍니다. 옵셔널은 코드가 항상 값을 사용하기 전에 값이 누락되었는지 확인하는 것을 보장하고, 옵셔널이 아닌 값은 값이 절대로 누락되지 않도록 보장합니다.

Swift는 안전한 언어이며, 이것은 개발 과정에서 가능한 빨리 여러 범주의 버그를 찾고 수정하는 것을 더 쉽게 만들고 특정 종류의 버그가 발생하지 않도록 보장합니다. 타입 안전성(Type safety)은 코드가 다루는 값의 타입에 대해 명확하게 할 수 있도록 해줍니다. `String`을 요구하는 코드에서 타입 안전성은 실수로 `Int`를 전달하는 것을 방지합니다. 메모리 안전성(Memory Safety)은 초기화되지 않은 메모리나 할당 해제된 객체가 아닌 유효한 데이터만 사용하도록 보장하며, 여러 코드가 동시에 실행되는 프로그램에서도 데이터를 안전하게 다루도록 보장합니다. Swift는 대부분의 안전성 검사를 코드를 빌드하는 동안 수행하며, 일부 경우에는 코드가 실행되는 동안 검사를 수행합니다.

상수와 변수 (Constants and Variables)

상수와 변수는 이름 (`maximumNumberOfLoginAttempts` 또는 `welcomeMessage`)과 특정 타입 (숫자 `10` 또는 문자열 `"Hello"`와 같은 타입)의 값을 연결합니다. *상수(Constant)*의 값은 최초 지정 후 변경이 불가능하지만, *변수(Variable)*는 다른 값으로 변경이 가능합니다.

상수와 변수 선언 (Declaring Constants and Variables)

상수와 변수는 사용하기 전에 선언이 되어야 합니다. 상수는 `let` 키워드와 함께 선언하고 변수는 `var` 키워드와 함께 선언합니다. 다음의 예시는 상수와 변수를 사용하여 어떻게 사용자 로그인 시도 횟수를 추적하는지를 보여줍니다:

```
let maximumNumberOfLoginAttempts = 10
var currentLoginAttempt = 0
```

이 코드는 아래와 같이 읽을 수 있습니다:

“`maximumNumberOfLoginAttempts` 인 새로운 상수를 선언하고, `10`이라는 값을 설정합니다. 그리고 `currentLoginAttempt` 인 새로운 변수를 선언하고, `0`으로 초기값을 설정합니다.”

이 예시에서, 최대 로그인 시도 횟수는 최대값은 절대 변경되지 않아야 하므로 상수로 선언했습니다. 현재 로그인 시도 횟수는 로그인 실패 시 값을 증가시켜야 하므로 변수로 선언했습니다.

코드에서 저장된 값이 변경되지 않으면, `let` 키워드를 사용하여 상수로 선언합니다. 변경되는 값을 저장하려면 변수를 사용합니다.

상수 또는 변수로 선언할 때, 위의 예시처럼 선언과 함께 값을 지정할 수 있습니다. 또는 프로그램 내에서 초기값을 나중에 할당할 수 있지만, 처음으로 해당 값을 읽기 전에 반드시 값이 지정되어야 합니다.

```
var environment = "development"
let maximumNumberOfLoginAttempts: Int
// maximumNumberOfLoginAttempts has no value yet.

if environment == "development" {
    maximumNumberOfLoginAttempts = 100
} else {
    maximumNumberOfLoginAttempts = 10
}
// Now maximumNumberOfLoginAttempts has a value, and can be read.
```

이 예시에서, 최대 로그인 시도 횟수는 상수이고, 이 값은 환경에 따라 달라집니다. 개발 환경에서는 100의 값을 가지고; 다른 환경에서는 10을 가집니다. `if` 구문의 각 조건에서 어떠한 값으로 `maximumNumberOfLoginAttempts`을 초기화하고, 이 상수는 항상 값이 있음을 보장합니다. 이 방법으로 초기값을 설정할 때, Swift가 어떻게 코드를 검사하는지 자세한 내용은 상수 선언 (Constant Declaration)을 참고바랍니다.

여러개의 상수 또는 여러개의 변수를 선언할 때 콤마로 구분하여 한줄로 선언이 가능합니다:

```
var x = 0.0, y = 0.0, z = 0.0
```

타입 명시 (Type Annotations)

상수 또는 변수를 선언할 때, 저장할 수 있는 값의 종류를 명확하게 하기위해 *타입 명시(Type Annotation)*를 제공할 수 있습니다. 타입 명시는 상수 또는 변수 이름 뒤에 콜론과 공백 한칸 뒤에 사용할 타입 이름을 적어 사용합니다.

이 예시는 `welcomeMessage`라는 변수에 `String` 값을 저장할 수 있는 변수를 나타내는 타입 명시를 제공합니다:

```
var welcomeMessage: String
```

위 코드의 선언에 있는 콜론은 “…의 타입은…” 을 의미하므로 아래와 같이 읽을 수 있습니다:

“선언한 변수는 `welcomeMessage`라고 하며 이것의 타입은 `String`입니다.”

“…의 타입은 `String`입니다.“라는 의미는 “모든 `String` 값은 저장 가능합니다.” 입니다. 저장할 수 있는 “어떠한 타입”(또는 “어떠한 종류”)라고 생각합니다.

`welcomeMessage` 변수는 아무런 오류 없이 어떠한 문자열 값을 설정할 수 있습니다:

```
welcomeMessage = "Hello"
```

동일한 타입의 연관된 변수들을 한 줄로 콤마로 구분하고, 마지막 변수 이름 뒤에 타입 명시를 통해 선언할 수 있습니다:

```
var red, green, blue: Double
```

참고

실제로 타입 명시가 필요한 경우는 드뭅니다. 상수 또는 변수를 선언할 때 초기값을 지정하면 Swift는 타입 안전성과 타입 추론 (Type Safety and Type Inference)에서 설명한대로 해당 상수 또는 변수에 사용될 타입을 거의 항상 추론할 수 있습니다. 위의 `welcomeMessage` 예시에서 초기값을 지정하지 않았으므로, `welcomeMessage` 변수의 타입은 초기값에서 유추되지 않고 타입을 명시 하였습니다.

상수와 변수의 이름 (Naming Constants and Variables)

상수와 변수 이름은 유니코드(Unicode) 문자를 포함하여 대부분의 문자를 포함할 수 있습니다:

```
let π = 3.14159
let 你好 = "你好世界"
let 🐶🐮 = "dogcow"
```

상수와 변수 이름은 공백, 수학적 기호, 화살표, 내부에서 사용하는 유니코드 스칼라 값, 선과 박스를 그리는 문자를 포함할 수 없습니다. 숫자는 이름의 다른곳에는 포함될 수 있지만 숫자로 시작하는 이름은 선언할 수 없습니다.

특정 타입으로 상수 또는 변수를 선언하면 동일한 이름으로 다시 선언하거나 다른 타입의 값을 저장하도록 변경하여 선언할 수 없습니다. 상수를 변수로 바꾸거나 변수를 상수로 바꿀 수도 없습니다.

참고

Swift 예약어와 동일한 이름의 상수 또는 변수를 제공해야 한다면 이름을 백틱( )으로 감싸야 합니다. 그러나 특별한 경우가 아니라면 예약어를 이름으로 사용하는 것은 피하는 것이 좋습니다.

이미 선언된 변수에 동일한 타입의 다른 값으로 변경할 수 있습니다. 이 예시에서 `friendlyWelcome` 값은 `"Hello!"`에서 `"Bonjour!"`로 변경됩니다:

```
var friendlyWelcome = "Hello!"
friendlyWelcome = "Bonjour!"
// friendlyWelcome is now "Bonjour!"
```

변수와 다르게 상수 값은 지정된 이후에는 변경할 수 없습니다. 값을 변경하려고 하면 코드가 컴파일 될 때 오류가 발생합니다:

```
let languageName = "Swift"
languageName = "Swift++"
// This is a compile-time error: languageName cannot be changed.
```

상수와 변수 출력 (Printing Constants and Variables)

`print(_:separator:terminator:)` 함수로 상수 또는 변수의 현재값을 출력할 수 있습니다:

```
print(friendlyWelcome)
// Prints "Bonjour!".
```

`print(_:separator:terminator:)` 함수는 하나 이상의 값을 출력하는 전역 함수입니다. 예를 들어, Xcode에서 `print(_:separator:terminator:)` 함수는 Xcode “콘솔(console)” 창에 결과를 출력합니다. `separator`와 `terminator` 매개변수는 기본값을 가지고 있으므로 함수를 호출할 때 생략할 수 있습니다. 기본적

으로 이 함수는 줄바꿈을 출력하고 종료됩니다. 줄바꿈 없이 값을 출력하려면, `terminator`로 빈 문자열을 전달하면 됩니다 - 예를 들어, `print(someValue, terminator: "")`처럼 사용합니다. 매개변수 기본값에 대한 자세한 내용은 매개변수 기본값 (Default Parameter Values)을 참고바랍니다.

Swift는 *문자열 보간(String interpolation)*을 사용하여 상수나 변수의 이름을 더 긴 문자열에 포함하고, Swift가 상수나 변수의 현재값으로 대체하도록 합니다. 이름을 소괄호로 감싸고 여는 소괄호 전에 역슬래시를 추가해야 합니다:

```
print("The current value of friendlyWelcome is \(friendlyWelcome)")
// Prints "The current value of friendlyWelcome is Bonjour!".
```

참고

문자열 보간과 함께 사용할 수 있는 모든 옵션은 문자열 삽입 (String Interpolation)에 자세히 설명되어 있습니다.

주석 (Comments)

코드에서 설명이나 기록을 위해 실행되지 않는 문자를 추가할 땐 주석(Comments)을 사용합니다. 주석은 코드가 컴파일 될 때 Swift 컴파일러에 의해 무시됩니다.

Swift에서 주석은 C에서 주석을 다는 방법과 유사합니다. 한줄 주석은 두개의 슬래시(`//`)로 시작합니다:

```
// This is a comment.
```

여러줄 주석은 슬래시 뒤에 애스터리스크(`/*`)로 시작하고 애스터리스크 뒤에 슬래시(`*/`)로 끝납니다:

```
/* This is also a comment
but is written over multiple lines. */
```

C에서 여러줄 주석과 다르게, Swift는 여러줄 주석 안에 다른 여러줄 주석을 통해 중첩시킬 수 있습니다. 여러줄 주석 블록을 시작한 다음 첫번째 블록 내에서 두번째 여러줄 주석을 시작하여 중첩된 주석을 작성합니다. 그런 다음에 두번째 블록을 닫은 다음 첫번째 블록을 닫습니다:

```
/* This is the start of the first multiline comment.
 /* This is the second, nested multiline comment. */
This is the end of the first multiline comment. */
```

중첩된 여러줄 주석을 사용하면 코드에 이미 여러줄 주석이 포함되어 있어도 큰 코드 블록을 빠르고 쉽게 주석 처리할 수 있습니다.

세미콜론 (Semicolons)

많은 다른 언어와 다르게, Swift는 코드의 각 구문 후에 세미콜론(`;`)은 필수가 아닙니다. 그러나 여러 구문을 한줄로 작성할 경우 세미콜론은 필수로 작성되어야 합니다:

```
let cat = "🐱"; print(cat)
// Prints "🐱".
```

정수 (Integers)

*정수(Integers)*는 42와 –23과 같은 분수가 아닌 전체 숫자입니다. 정수는 *부호가 있는 정수(signed) (양수, 0, 음수)*나 *부호가 없는 정수(unsigned) (양수나 0)*가 있고 최대값과 최소값은 정수의 크기에 따라 결정됩니다 (값을 저장하는데 사용하는 비트수). 정수 타입은 이름에 크기와 부호가 포함되어 있습니다 - 예를 들어 8비트 부호 없는 정수는 `UInt8` 타입이고, 32비트 부호 있는 정수는 `Int32` 타입입니다. Swift에서의 타입과 같이 이 정수 타입은 대문자로 시작합니다. 대부분의 경우 정수 크기를 지정할 필요가 없다면, 아래 설명과 같이 `Int` 타입을 사용합니다.

정수 타입은 우리가 손으로 계산하는 산술 연산 방식처럼 동작하고 정수 연산은 근사값이 아닌 정확한 결과를 도출합니다. 이러한 특성은 정수가 개수를 세거나 정확한 값을 표현해야하는 계산에 적합합니다 - 예를 들어 텍스트 파일에서 가장 긴 라인을 찾거나 게임에서 점수에 배율을 적용하거나 영수증의 가격을 합산할 때 유용합니다.

정수에는 소수 부분이 없지만, 소수 부분을 개수로 환산해서 세는 방식으로 소수를 포함한 값을 표현할 수 있습니다. 예를 들어 센트 단위로 세는 정수에서 $1.23을 123으로 저장해서 표현할 수 있습니다. 이러한 접근 방식은 소수점이 숫자에 고정된 위치에 있으므로 *고정 소수점 연산(fixed-point math)*라고 합니다. 위의 예제에서 숫자 123은 마지막 두 자리 앞에 소수점이 있다고 이해됩니다.

> **참고**
>
> 금융이나 건설에서와 같은 곳의 계산이나 정밀한 결과가 요구되는 도메인에서는 해당 요구사항에 맞는 반올림이나 버림과 같은 동작이 구현된 특정 목적의 숫자 타입이 필요할 수 있습니다.

정수 범위 (Integer Bounds)

각 정수 타입의 `min`과 `max` 프로퍼티를 통해 각 정수 타입의 최소값과 최대값에 접근할 수 있습니다:

```
let minValue = UInt8.min  // minValue is equal to 0, and is of type UInt8
let maxValue = UInt8.max  // maxValue is equal to 255, and is of type UInt8
```

이러한 프로퍼티의 값은 적절한 크기의 숫자 타입 (위 예에서 `UInt8`)이므로 동일한 타입의 다른 값과 함께 표현식에 사용될 수 있습니다.

`max` 프로퍼티보다 큰 값처럼 범위를 벗어나는 결과는 유효하지 않은 결과를 저장하는 대신에 프로그램 실행을 중단합니다. 하지만 오버플로우 연산자 (Overflow Operators)에서 설명한대로 연산이 오버플로우되도록 명시적으로 만들 수 있습니다.

Int

대부분의 경우 코드에서 사용할 정수의 특정 사이즈를 결정할 필요는 없습니다. Swift는 현재 플랫폼의 네이티브 사이즈와 같은 정수 타입, `Int`를 제공합니다:

- 32-bit 플랫폼에서 `Int`는 `Int32`와 같은 크기를 가짐
- 64-bit 플랫폼에서 `Int`는 `Int64`와 같은 크기를 가짐

특정 크기의 정수로 작업해야 하는 경우가 아니라면 항상 코드의 정수값을 사용할 때 `Int`를 사용하세요. 이것은 코드 일관성과 상호 운용성을 지원합니다. 32-bit 플랫폼에서도 `Int`는 `–2,147,483,648`과 `2,147,483,647` 사이의 값을 저장할 수 있으며, 일반적인 사용성에 문제가 없습니다.

UInt

Swift는 또한 현재 플랫폼의 네이티브 사이즈와 같은 정수 타입, `UInt`를 제공합니다:

- 32-bit 플랫폼에서 `UInt`는 `UInt32`와 같은 크기를 가짐
- 64-bit 플랫폼에서 `UInt`는 `UInt64`와 같은 크기를 가짐

참고

`UInt`는 플랫폼의 네이티브 사이즈와 같은 크기의 부호 없는 정수 타입이 필요한 경우에만 사용하세요. 저장될 값이 음수가 아니어도 `Int`를 더 선호합니다. 정수값에 `Int`를 일관되게 사용하면 코드 상호 운용성을 지원하고 타입 안전성과 타입 추론 (Type Safety and Type Inference)에서 설명한대로 다른 숫자 형식간에 변환 할 필요가 없습니다.

부동소수점 숫자 (Floating-Point Numbers)

*부동소수점 숫자(Floating-point numbers)*는 `3.14159`, `0.1`, `-273.15`와 같은 분수 성분을 가진 숫자입니다. Swift는 정수에 다양한 크기의 타입이 있는 것처럼 다양한 크기를 지원하는 여러가지 부동소수점 타입을 제공합니다. 크기를 지정할 필요가 없다면, 기본적으로 `Double`을 사용합니다. 그렇지 않으면 `Float16`이나 `Float80`처럼 이름에 필요한 크기를 포함한 타입을 사용합니다. 부동소수점 연산에 일반적인 용어에 따라 `Float`는 32비트를 `Double`은 64비트를 사용합니다. 이 타입을 `Float32`나 `Float64`로 작성할 수도 있습니다. 예를 들어 그래픽 코드는 GPU의 가장 빠른 데이터 타입에 맞추기 위해 `Float`를 주로 사용합니다. 일부 부동소수점 타입은 특정 플랫폼에서나 지원하지만 `Float`와 `Double`은 모든 플랫폼에서 사용 가능합니다.

부동소수점 숫자는 매우 작은 숫자와 매우 큰 숫자를 다룰 수 있지만 범위 안의 모든 값을 표현할 수는 없습니다. 항상 정확한 결과를 나타내는 정수 계산과 다르게 부동소수점 연산은 표현할 수 있는 숫자에 가깝게 결과를 반올림합니다. 예를 들어 숫자 10,000을 `Float`로 저장하면, 표현할 수 있는 바로 다음 값은 10,000.001이고 -- 이 수 사이의 값은 둘 중 하나의 값으로 반올림됩니다. 숫자 사이의 간격도 가변적입니다; 큰 숫자 사이의 간격은 크고 작은 숫자의 간격은 작습니다. 예를 들어 0.001 다음의 `Float` 값은 0.0010000002이고, 10,000 다음의 값보다 간격이 더 작습니다.

부동소수점 숫자는 음의 0, 양의 무한대, 음의 무한대와 같은 값을 가지며 이것은 계산 중의 오버플로우와 언더플로우를 나타냅니다. 또한 숫자가 아닌 값(NaN)을 포함하고 이것은 0을 0으로 나누는 것처럼 유효하지 않거나 정의되지 않은 결과를 나타냅니다. 이것은 결과를 표현할 수 없을 때 프로그램을 중단하는 정수와는 다른 동작을 보여줍니다.

가능한 값 사이의 동일한 간격이 필요하거나 수행하는 계산이 정확한 결과를 요구하고 위에서 언급한 특수한 값이 필요하지 않으면 부동소수점 숫자는 적절한 데이터 타입이 아닐 수 있습니다. 이런 경우에는 정수 (Integers)에서 설명한 고정소수점 숫자를 사용하는걸 추천합니다.

타입 안전성과 타입 추론 (Type Safety and Type Inference)

Swift 프로그램의 모든 값은 타입을 가집니다. 상수, 변수, 프로퍼티를 포함하여 값을 저장하는 모든 곳에도 타입을 가집니다. 타입 주석을 사용하여 명시적으로 타입을 작성하거나 Swift는 초기값으로부터 타입을 추론할 수 있습니다. 코드를 작성하는 모든 곳에서 제공하는 값의 타입은 그 값을 사용하는 곳에서의 타입과 일치해야 합니다. 예를 들어 코드에서 `String`을 요구하면, 실수로 `Int`를 전달할 수 없습니다. 이러한 검사로 인해 Swift는 타입 안전(type-safe) 언어입니다.

타입 안전 언어는 코드에서 다루는 값의 타입에 대해 명확하게 정의하도록 권장합니다. 한 타입의 값은 다른 타입으로 암시적으로 변환되지 않습니다. 그러나 일부 타입은 명시적으로 변환될 수 있습니다. 코드를 빌드할 때, Swift는 타입 안전성에 대해 코드를 검사하고 일치하지 않는 타입을 오류로 표시합니다.

타입 검사는 다른 타입의 값으로 작업할 때 오류를 피하는데 도움이 됩니다. 그러나 이것이 선언하는 모든 상수와 변수의 타입을 지정해야 한다는 것은 아닙니다. 필요한 값의 특정 타입을 지정하지 않으면 Swift는 적절한 타입으로 *

타입 추론(Type Inference)*을 사용합니다. 타입 추론을 통해 컴파일러는 코드를 컴파일 할 때 제공한 값을 검사하여 특정 표현식의 타입을 자동으로 추론할 수 있습니다.

타입 추론 때문에, Swift는 C나 Objective-C와 같은 언어보다 타입 선언을 더 적게 요구합니다. 상수와 변수는 여전히 명시적으로 타입을 지정하지만 타입을 지정하는 대부분의 동작은 자동으로 처리됩니다.

타입 추론은 상수나 변수에 초기값을 선언할 때 아주 유용합니다. 이것은 종종 상수나 변수를 선언할 때 리터럴 값(literal value)(또는 리터럴(literal))을 할당하여 이루어집니다. (리터럴 값은 아래 예에서 `42` 및 `3.14159`와 같은 소스 코드에 직접 표시되는 값입니다.)

예를 들어 어떤 타입인지 선언하지 않고 새로운 상수를 `42`의 리터럴 값으로 지정하면 Swift는 정수처럼 보이는 숫자로 초기화 했기 때문에 상수를 `Int`라고 추론합니다:

```
let meaningOfLife = 42
// meaningOfLife is inferred to be of type Int
```

반대로 어떤 부동소수점 리터럴의 타입인지 선언하지 않으면, Swift는 `Double`이라고 추론합니다:

```
let pi = 3.14159
// pi is inferred to be of type Double
```

Swift는 부동소수점 숫자의 타입을 추론할 때, 항상 `Double`(`Float` 보다)을 선택합니다.

정수 리터럴과 부동소수점 리터럴을 하나의 표현식에서 결합하면, `Double` 타입으로 추론합니다:

```
let anotherPi = 3 + 0.14159
// anotherPi is also inferred to be of type Double
```

`3`의 리터럴 값은 명시적인 타입이 없으므로, 부동소수점 리터럴이 존재하는 것을 기반으로 `Double`이 추론됩니다.

숫자 리터럴 (Numeric Literals)

정수 리터럴은 아래와 같이 쓸 수 있습니다:

- 접두사 없는 10진수
- `0b` 접두사로 2진수
- `0o` 접두사로 8진수
- `0x` 접두사로 16진수

아래의 예에서 모든 정수 리터럴은 10진수 `17`의 값을 가집니다:

```
let decimalInteger = 17
let binaryInteger = 0b10001       // 17 in binary notation
let octalInteger = 0o21           // 17 in octal notation
let hexadecimalInteger = 0x11     // 17 in hexadecimal notation
```

부동소수점 리터럴은 10진수(접두사 없음) 또는 16진수(접두사 `0x`)일 수 있습니다. 소수점 양쪽에 항상 숫자(또는 16진수)가 있어야 합니다. 10진수는 대문자나 소문자 `e`로 표시되는 지수를 가질 수도 있습니다. 16진수는 대문자나 소문자 `p`로 표시되는 지수를 가질 수도 있습니다.

지수가 x인 10진수는 기본 숫자에 10^x가 곱해집니다:

- `1.25e2`는 1.25 x 10^2, 또는 `125.0`.
- `1.25e-2`는 1.25 x 10^{-2} , 또는 `0.0125`.

지수가 x인 16진수는 기본 숫자에 2^x가 곱해집니다:

- `0xFp2`는 15 x 2^2 , 또는 `60.0`.
- `0xFp-2`는 15 x 2^{-2} , 또는 `3.75`.

아래의 예에서 모든 부동소수점 리터럴은 10진수 `12.1875`를 가집니다:

```
let decimalDouble = 12.1875
let exponentDouble = 1.21875e1
let hexadecimalDouble = 0xC.3p0
```

숫자 리터럴은 읽기 쉽게 만드는 추가 포맷을 포함할 수 있습니다. 정수와 부동소수점 모두 추가 0으로 채워질 수 있으며 가독성을 돕기 위해 밑줄을 포함할 수 있습니다. 어떤 형식도 리터럴의 기본값에 영향을 주지 않습니다:

```
let paddedDouble = 000123.456
let oneMillion = 1_000_000
let justOverOneMillion = 1_000_000.000_000_1
```

숫자 타입 변환 (Numeric Type Conversion)

음수를 사용하지 않더라도 코드에서 상수와 변수가 정수로 사용이 되면 `Int` 타입을 사용하세요. 일반적인 상황에서 기본 정수 타입을 사용하면, 정수 상수와 변수가 코드에서 즉시 호환 가능하며 정수 리터럴 값의 추론된 타입과 일치하게 됩니다.

다른 정수 타입은 특정 작업에 필요하거나 외부 소스에서 크기가 명시된 데이터가 필요하거나 성능, 메모리 사용량 또는 다른 최적화가 필요한 경우에만 사용하세요. 이러한 상황에서 명시적으로 크기가 지정된 타입을 사용하면 실수로 인한 값 초과를 방지하고 사용되는 데이터의 특성을 알 수 있습니다.

정수 변환 (Integer Conversion)

정수를 저장할 수 있는 상수나 변수의 숫자 범위는 숫자 타입에 따라 다릅니다. `Int8` 상수나 변수는 `-128`과 `127` 사이의 숫자를 저장할 수 있는 반면에 `UInt8` 상수나 변수는 `0`과 `255` 사이의 숫자를 저장할 수 있습니다. 크기가 지정된 정수 타입의 상수나 변수에 맞지않는 숫자는 컴파일 시에 오류가 발생합니다:

```
let cannotBeNegative: UInt8 = -1
// UInt8 cannot store negative numbers, and so this will report an error
let tooBig: Int8 = Int8.max + 1
// Int8 cannot store a number larger than its maximum value,
// and so this will also report an error
```

숫자 타입은 다른 범위의 값을 저장할 수 있으므로, 경우에 따라 숫자 타입 변환을 타입별로 선택해야 합니다. 이 방식은 숨겨진 변환 오류를 방지하고, 코드에서 타입 변환 의도를 명확하게 만드는데 도움을 줍니다.

특정 숫자 타입을 다른 타입으로 변환하려면, 기존값을 사용하여 원하는 타입의 새로운 숫자를 초기화해야 합니다. 아래의 예시에서 상수 `twoThousand`는 `UInt16` 타입인 반면에 상수 `one`은 `UInt8` 타입입니다. 두 상수는 타입이 다

르기 때문에 직접 더할 수 없습니다. 대신 이 예시에서 `UInt16(one)`을 호출하여 `one`의 값을 새로운 `UInt16`으로 초기화하면 기존값 대신에 이 값을 사용합니다:

```
let twoThousand: UInt16 = 2_000
let one: UInt8 = 1
let twoThousandAndOne = twoThousand + UInt16(one)
```

양쪽의 타입이 `UInt16`이므로, 덧셈은 이제 제대로 동작합니다. 출력 상수(`twoThousandAndOne`)은 두 `UInt16` 값을 더하므로 `UInt16` 타입으로 추론됩니다.

`SomeType(ofInitialValue)`는 Swift 타입의 초기화를 호출하고 초기화 값을 전달하는 기본적인 방법입니다. 내부적으로 `UInt16`은 `UInt8` 값을 허용하는 초기화가 있으므로 초기화는 기존 `UInt8`에서 새 `UInt16`을 만드는데 사용됩니다. 그러나 여기서 아무 타입이나 전달할 수 없습니다 - `UInt16`이 초기화를 제공하는 타입이어야만 합니다. 기존 타입을 확장하여 새로운 타입 (자신이 정의한 새로운 타입) 을 받아들이는 초기화를 제공하는 것은 확장(Extensions)에서 다룹니다.

정수와 부동소수점 변환 (Integer and Floating-Point Conversion)

정수와 부동소수점 숫자 타입의 변환은 명시적으로 변환해야 합니다:

```
let three = 3
let pointOneFourOneFiveNine = 0.14159
let pi = Double(three) + pointOneFourOneFiveNine
// pi equals 3.14159, and is inferred to be of type Double
```

여기서 상수 `three`는 타입 `Double`의 새로운 값으로 생성하는데 사용되어 덧셈의 양쪽이 동일한 타입입니다. 이 변환이 없으면 덧셈이 허용되지 않습니다.

부동소수점을 정수로 변환하는 것 또한 명시적으로 변환해야 합니다. 정수 타입은 `Double`이나 `Float`로 초기화 될 수 있습니다:

```
let integerPi = Int(pi)
// integerPi equals 3, and is inferred to be of type Int
```

부동소수점 값을 이런 방식으로 새로운 정수값으로 초기화할 때 항상 소수점 아래를 버림합니다. 이것은 `4.75`는 `4`, 그리고 `-3.9`는 `-3`이 된다는 의미입니다.

> **참고**
>
> 숫자 상수와 변수를 결합하는 규칙은 숫자 리터럴 규칙과 다릅니다. 리터럴 값 `3`은 숫자 리터럴에 명시적인 타입이 없으므로 리터럴 값 `0.14159`에 직접 추가할 수 있습니다. 이것의 타입은 컴파일러가 실행되는 시점에서만 추론됩니다.

타입 별칭 (Type Aliases)

*타입 별칭(Type aliases)*은 이미 존재하는 타입을 다른 이름으로 정의합니다. 타입 별칭은 `typealias` 키워드를 사용하여 정의할 수 있습니다.

타입 별칭은 외부 소스에서 특정 크기의 데이터로 작업할 때와 같이 상황에 맞는 이름으로 기존 타입을 참조하려는 경우에 유용합니다:

```
typealias AudioSample = UInt16
```

타입 별칭을 정의하면 원래 이름을 사용할 수 있는 모든 위치에서 별칭을 사용할 수 있습니다:

```
var maxAmplitudeFound = AudioSample.min
// maxAmplitudeFound is now 0
```

여기서 `AudioSample`은 `UInt16`의 별칭으로 정의됩니다. 별칭이므로 `AudioSample.min`에 대한 호출은 실제로 `UInt16.min`을 호출하며, `maxAmplitudeFound` 변수에 `0`이라는 초기값을 제공합니다.

Booleans

Swift는 `Bool`이라 불리는 기본 Boolean 타입이 있습니다. Boolean 값은 논리적 값이라고 불리는데, 이는 참 또는 거짓만 가질 수 있기 때문입니다. Swift는 2개의 Boolean 상수 값인 `true`와 `false`를 제공합니다:

```
let orangesAreOrange = true
let turnipsAreDelicious = false
```

`orangesAreOrange`와 `turnipsAreDelicious`의 타입은 Boolean 리터럴 값으로 초기화되었기 때문에 `Bool`로 추론되었습니다. 위의 `Int`와 `Double`에서와 같이 상수나 변수를 초기화 시 `true` 또는 `false`로 선언하면 이를 `Bool` 타입으로 선언할 필요가 없습니다.

Boolean 값은 `if` 구문과 같은 조건문으로 동작할 때 특히 유용합니다:

```
if turnipsAreDelicious {
    print("Mmm, tasty turnips!")
} else {
    print("Eww, turnips are horrible.")
}
// Prints "Eww, turnips are horrible.".
```

`if` 구문 같은 조건문은 제어 흐름 (Control Flow)에서 자세히 다룹니다.

Swift의 타입 안전성은 Boolean이 아닌 값이 `Bool`로 대체되는 것을 방지합니다. 아래 예시는 컴파일 시 오류를 발생시킵니다:

```
let i = 1
if i {
    // this example will not compile, and will report an error
}
```

그러나 아래와 같은 예시는 정상 동작합니다:

```
let i = 1
if i == 1 {
    // this example will compile successfully
}
```

`i == 1` 비교 결과는 `Bool` 타입이므로, 이 두번째 예시는 타입 검사를 정상적으로 수행할 수 있습니다. `i == 1`과 같은 비교는 기본 연산자 (Basic Operators)에서 설명합니다.

Swift의 타입 안전성에 대한 다른 예시처럼 이 방법은 실수로 인한 오류를 방지하고 코드 섹션의 의도가 항상 명확하게 전달되도록 보장합니다.

튜플 (Tuples)

*튜플(Tuples)*은 여러 값을 단일복합값으로 그룹화 합니다. 튜플안에 값은 어떠한 타입도 가능하며 서로 같은 타입일 필요는 없습니다.

이 예시에서 `(404, "Not Found")`은 *HTTP 상태 코드(HTTP status code)*를 나타내는 튜플입니다. HTTP 상태 코드는 웹 페이지를 요청할 때마다 웹 서버가 반환하는 특정 값입니다. `404 Not Found` 상태 코드는 요청한 웹 페이지가 존재하지 않을 때 반환됩니다.

```
let http404Error = (404, "Not Found")
// http404Error is of type (Int, String), and equals (404, "Not Found")
```

`(404, "Not Found")` 튜플은 HTTP 상태 코드에 두 개별 값인 `Int`와 `String`을 함께 그룹화하여 제공합니다: 숫자와 사람이 읽을 수 있는 설명을 제공합니다. 이것은 "튜플의 타입은 `(Int, String)` 입니다"라고 설명할 수 있습니다.

모든 타입의 조합으로 튜플을 만들 수 있으며, 원하는 만큼 다양한 타입을 포함할 수 있습니다. `(Int, Int, Int)` 타입의 튜플이나 `(String, Bool)` 타입의 튜플, 또는 실제로 필요한 다른 어떠한 조합도 만들 수 있습니다.

튜플의 내용을 별도의 상수나 변수로 *분해(decompose)*하여 평소와 같이 접근할 수 있습니다:

```
let (statusCode, statusMessage) = http404Error
print("The status code is \(statusCode)")
// Prints "The status code is 404".
print("The status message is \(statusMessage)")
// Prints "The status message is Not Found".
```

튜플의 값 중 일부만 필요한 경우, 튜플을 분해할 때 밑줄(_)로 튜플의 일부를 무시할 수 있습니다:

```
let (justTheStatusCode, _) = http404Error
print("The status code is \(justTheStatusCode)")
// Prints "The status code is 404".
```

또는, 0으로 시작하는 인덱스를 사용하여 튜플의 개별 요소 값에 접근할 수 있습니다:

```
print("The status code is \(http404Error.0)")
// Prints "The status code is 404".
print("The status message is \(http404Error.1)")
// Prints "The status message is Not Found".
```

튜플을 정의할 때 튜플의 요소에 이름을 정할 수 있습니다:

```
let http200Status = (statusCode: 200, description: "OK")
```

튜플 요소에 이름이 있다면, 요소의 값에 요소 이름을 사용하여 접근이 가능합니다:

```
print("The status code is \(http200Status.statusCode)")
// Prints "The status code is 200".
print("The status message is \(http200Status.description)")
// Prints "The status message is OK".
```

튜플은 특히 함수의 반환값으로 유용합니다. 예를 들어 웹 페이지를 가져오는 함수는 페이지 가져오기의 성공 또는 실패를 설명하기위해 `(Int, String)` 튜플 타입을 반환할 수 있습니다. 이 함수는 각각 다른 타입의 두 개의 값을 반환함으로써, 단일 타입의 단일 값만 반환할 수 있는 경우보다 유용합니다. 자세한 내용은 여러 개의 반환값이 있는 함수 (Functions with Multiple Return Values)를 참조 바랍니다.

> **참고**
>
> 튜플은 관련된 값의 간단한 그룹에 유용합니다. 복잡한 데이터 구조를 생성하는데는 맞지 않습니다. 데이터 구조가 복잡한 경우 튜플이 아닌 클래스(class)나 구조체(structure)를 사용하세요. 자세한 내용은 구조체와 클래스 (Structures and Classes)를 참조 바랍니다.

옵셔널 (Optionals)

값이 없는 경우에 *옵셔널(optional)*을 사용합니다. 옵셔널은 2가지 가능성이 있습니다: 지정된 타입의 값이 있고 옵셔널을 풀어서 값에 접근하거나 값이 없을 수도 있습니다.

누락될 수 있는 값의 예로 Swift의 `Int` 타입은 `String` 값을 `Int` 값으로 변환하는 초기화가 존재합니다. 그러나 일부 문자열만 정수로 변환할 수 있습니다. 문자열 `"123"`은 숫자값 `123`으로 변환될 수 있지만, 문자열 `"hello, world"`는 변환할 숫자값이 없습니다. 아래 예시는 `String`을 `Int`로 초기화하는 것을 보여줍니다:

```
let possibleNumber = "123"
let convertedNumber = Int(possibleNumber)
// The type of convertedNumber is "optional Int"
```

위의 코드에서 초기화가 실패할 수 있으므로, `Int`가 아닌 옵셔널 `Int`를 반환합니다.

옵셔널 타입을 작성하려면, 옵셔널을 포함하는 타입의 이름 다음에 물음표(`?`)를 작성합니다 - 예를 들어, 옵셔널 `Int`의 타입은 `Int?`입니다. 옵셔널 `Int`는 항상 어떠한 `Int` 값 또는 값이 없음을 포함합니다. `Bool`이나 `String` 값과 같은 다른 타입을 포함할 수 없습니다.

nil

옵셔널 변수에 특수한 값 `nil`로 지정하여 값이 없는 상태를 나타낼 수 있습니다:

```
var serverResponseCode: Int? = 404
// serverResponseCode contains an actual Int value of 404
serverResponseCode = nil
// serverResponseCode now contains no value
```

기본값이 없이 옵셔널 변수를 정의하면, 이 변수는 자동적으로 `nil`로 설정됩니다:

```
var surveyAnswer: String?
// surveyAnswer is automatically set to nil
```

`if` 구문은 옵셔널이 값을 포함하고 있는지 여부를 `nil`과 비교하여 확인할 수 있습니다. 이 비교는 "같음" 연산자 (==) 또는 "같지 않음" 연산자(`!=`)를 사용하여 수행됩니다.

옵셔널에 값이 있다면 `nil`과 "같지 않음"으로 간주됩니다:

```
let possibleNumber = "123"
let convertedNumber = Int(possibleNumber)

if convertedNumber != nil {
    print("convertedNumber contains some integer value.")
}
// Prints "convertedNumber contains some integer value.".
```

옵셔널이 아닌 상수나 변수에는 `nil`을 사용할 수 없습니다. 코드에서 상수나 변수가 특정 조건에서 값이 없이 동작하려면, 적절한 타입의 옵셔널 값을 선언해야합니다. 옵셔널이 아닌 값으로 선언된 상수나 변수는 `nil` 값이 포함되지 않도록 보장됩니다. 옵셔널이 아닌 값에 `nil`을 할당하면, 컴파일 오류가 발생합니다.

옵셔널과 옵셔널이 아닌 값의 분리는 누락된 정보를 명시적으로 표시할 수 있고, 누락된 값을 처리하는 코드를 더 쉽게 작성할 수 있습니다. 옵셔널을 옵셔널이 아닌 값으로 처리하는 실수는 방지하기 위해 컴파일 때 오류가 발생합니다. 값을 언래핑한 후에는 `nil`에 대한 값을 검사할 필요가 없으므로 코드의 다른 부분에서 이 값을 동일하게 확인할 필요가 없습니다.

옵셔널 값에 접근할 때 코드에서 `nil`과 `nil`이 아닌 것에 대해 모두 처리해야 합니다. 값이 없을 때 수행할 수 있는 작업이 몇가지 있으며, 이는 다음 섹션에서 설명합니다:

- 값이 `nil`일 때 해당 값에 대한 동작을 건너뜁니다.
- `nil` 값을 전파하는 방법으로, `nil` 을 반환하거나 옵셔널 체이닝 (Optional Chaining)에서 설명한 `?.` 연산자를 사용할 수 있습니다.
- `??` 연산자를 사용해서 대체값을 제공합니다.
- `!` 연산자를 사용해서 프로그램을 멈춥니다.

참고

Objective-C에서 `nil`은 객체가 존재하지 않는 포인터입니다. Swift에서 `nil`은 포인터가 아닙니다 - 특정 타입의 값이 없다는 의미입니다. 모든 타입의 옵셔널은 객체 타입 뿐만 아니라 `nil`로 설정할 수 있습니다.

옵셔널 바인딩 (Optional Binding)

옵셔널 바인딩(optional binding)은 옵셔널이 값을 포함하고 있는지 확인하고 값이 있는 경우 해당 값을 임시로 상수나 변수로 사용할 수 있게 해줍니다. 옵셔널 바인딩은 `if`, `guard`, `while` 구문에서 옵셔널에 값이 있는지 체크하고 단일 동작의 일부로 상수나 변수로 추출할 수 있습니다. `if`, `guard`, `while` 구문은 제어 흐름 (Control Flow)에서 자세히 다룹니다.

`if` 구문에서 옵셔널 바인딩은 아래와 같이 사용합니다:

```
if let <#constantName#> = <#someOptional#> {
   <#statements#>
}
```

강제 언래핑(forced unwrapping)보다 옵셔널 바인딩을 사용하여 옵셔널 (Optionals) 섹션에 있는 예시의 `possibleNumber`를 다시 작성할 수 있습니다:

```
if let actualNumber = Int(possibleNumber) {
    print("The string \"\(possibleNumber)\" has an integer value of \(actualNumber)")
} else {
    print("The string \"\(possibleNumber)\" could not be converted to an integer")
}
// Prints "The string "123" has an integer value of 123".
```

이 코드는 아래와 같이 읽을 수 있습니다:

"`Int(possibleNumber)`에 의해 반환된 옵셔널 `Int`에 값이 포함되어 있으면, `actualNumber`라는 새로운 상수에 옵셔널에 포함된 값을 설정합니다."

변환에 성공하면, `actualNumber` 상수는 `if` 구문의 첫번째 중괄호 내에서 사용할 수 있습니다. 옵셔널에서 포함된 값으로 초기화되었고, 옵셔널이 아닌 타입입니다. 이 경우 `Int(possibleNumber)`의 타입은 `Int?`이므로 `actualNumber`의 타입은 `Int`입니다.

값에 접근한 후에 기존 옵셔널 상수나 변수를 참조할 필요가 없다면 같은 이름으로 새로운 상수 또는 변수를 사용할 수 있습니다:

```
let myNumber = Int(possibleNumber)
// Here, myNumber is an optional integer
if let myNumber = myNumber {
    // Here, myNumber is a non-optional integer
    print("My number is \(myNumber)")
}
// Prints "My number is 123".
```

이 코드는 이전 예시에서의 코드와 마찬가지로 `myNumber`가 값이 있는지 없는지를 먼저 확인합니다. `myNumber`가 값이 있다면, 해당 값이 새로운 `myNumber`라는 상수에 설정됩니다. `if` 구문 안에서 `myNumber`는 새로운 옵셔널이 아닌 상수를 참조합니다. `if` 구문의 앞이나 뒤에서 `myNumber`를 작성하면 원래 옵셔널 정수 상수를 참조합니다.

이러한 코드는 일반적이기 때문에 옵셔널 값을 언래핑 하는데 더 짧게 사용할 수 있습니다: 언래핑할 상수나 변수의 이름만 작성합니다. 언래핑된 새로운 상수나 변수는 암시적으로 같은 이름의 옵셔널 값을 사용합니다.

```
if let myNumber {
    print("My number is \(myNumber)")
}
// Prints "My number is 123".
```

옵셔널 바인딩은 상수와 변수 둘다 사용이 가능합니다. `if` 구문에 첫번째 중괄호에서 `myNumber`의 값을 변경하고 싶다면 `if var myNumber`로 대신 쓸 수 있으며 옵셔널에 포함된 값은 상수가 아닌 변수로 사용할 수 있습니다. `if` 구문내에서 `myNumber`를 바꾸면 이것은 지역 변수에만 적용되며, 기존 옵셔널 상수나 변수에 적용되지 않습니다.

필요한 경우 `if` 구문에 쉼표로 구분하여 옵셔널 바인딩 및 Boolean 조건을 여러개 포함할 수 있습니다. 옵셔널 바인딩 값 중 하나가 `nil`이거나 Boolean 조건이 `false`로 판단되면 전체 `if` 구문의 조건은 `false`로 간주됩니다. 다음의 예는 같은 `if` 구문입니다:

```
if let firstNumber = Int("4"), let secondNumber = Int("42"), firstNumber < secondNumber && sec
  ondNumber < 100 {
    print("\(firstNumber) < \(secondNumber) < 100")
}
// Prints "4 < 42 < 100".

if let firstNumber = Int("4") {
    if let secondNumber = Int("42") {
        if firstNumber < secondNumber && secondNumber < 100 {
            print("\(firstNumber) < \(secondNumber) < 100")
        }
    }
}
// Prints "4 < 42 < 100".
```

`if` 구문에서 옵셔널 바인딩으로 생성된 상수와 변수는 `if` 구문의 본문 내에서만 사용 가능합니다. 반대로 `guard` 구문에서 생성된 상수와 변수는 이른 종료 (Early Exit)에서 설명되어 있듯이 `guard` 구문 다음에 코드에서 사용 가능합니다.

대체값 제공 (Providing a Fallback Value)

누락된 값을 처리하는 다른 방법은 nil-결합 연산자(`??`)를 사용하여 기본값을 제공하는 방법입니다. 옵셔널에서 `??`의 왼편이 `nil`이 아니면, 값은 언래핑되고 사용됩니다. 그렇지 않으면 `??`의 오른편에 값이 사용됩니다. 예를 들어, 아래 코드는 이름이 지정되면 지정된 이름에 인사하고, 이름이 `nil`이면 일반적인 인사를 사용합니다.

```
let name: String? = nil
let greeting = "Hello, " + (name ?? "friend") + "!"
print(greeting)
// Prints "Hello, friend!".
```

대체 값 제공하기 위한 `??` 사용에 대한 더 자세한 내용은 Nil-병합 연산자 (Nil-Coalescing Operator)를 참고바랍니다.

강제 언래핑 (Force Unwrapping)

프로그래머의 오류나 원치않는 상태와 같은 실패를 `nil`로 표현하려면, 옵셔널의 이름 뒤에 느낌표(`!`)를 추가하여 접근할 수 있습니다. 이것을 옵셔널 값의 *강제 언래핑(force unwrapping)*이라고 합니다. `nil`이 아닌 값에 강제 언래핑을 하면, 언래핑된 값을 결과로 얻습니다. `nil` 값을 강제 언래핑하면 런타임 오류가 발생합니다.

`!`은 `fatalError(_:file:line:)`의 짧은 표현입니다. 예를 들어, 아래 코드는 동일한 접근을 보여줍니다:

```
let possibleNumber = "123"
let convertedNumber = Int(possibleNumber)

let number = convertedNumber!

guard let number = convertedNumber else {
    fatalError("The number was invalid")
}
```

위 코드의 두 버전은 값을 항상 가지는 `convertedNumber`에 의존하는 것을 보여줍니다. 위의 접근방식 중 하나를 사용해서 코드에 작성하면, 실행 중에 요구사항이 참인지 확인할 수 있습니다.

데이터 요구사항을 강제하고 실행 중에 가정을 확인하는 방법에 대한 자세한 내용은 어설션과 전제 조건 (Assertions and Preconditions)을 참고바랍니다.

암시적 언래핑 옵셔널 (Implicitly Unwrapped Optionals)

위에서 설명한대로, 옵셔널은 상수나 변수가 "값이 없음"을 허용합니다. 옵셔널은 값이 존재하는지 `if` 구문에서 체크할 수 있고 값이 존재한다면 옵셔널 값에 접근하기위해 옵셔널 바인딩을 이용할 수 있습니다.

때로는 프로그램 구조에서 옵셔널이 값을 처음 설정한 후에는 항상 값을 갖는 것은 분명합니다. 이러한 경우 항상 값이 있다고 가정할 수 있으므로 접근할 때마다 옵셔널의 값을 확인하고 언래핑 할 필요가 없습니다.

이러한 옵셔널은 *암시적 언래핑 옵셔널(implicitly unwrapped optionals)*이라고 합니다. 옵셔널을 만들기 위해 타입뒤에 물음표(`String?`)를 작성하는 대신에 느낌표(`String!`)로 암시적 언래핑 옵셔널을 작성합니다. 사용할 때 옵셔널 이름의 뒤에 느낌표를 위치시키는 것보다 선언할 때 옵셔널 타입 뒤에 느낌표를 위치시키는 것이 더 좋습니다.

암시적 언래핑 옵셔널은 옵셔널의 값이 처음 정의된 직후에 존재하는 것으로 확인되고 그 이후 모든 시점에 존재한다고 가정할 수 있는 경우에 유용합니다. Swift에서 암시적 언래핑 옵셔널은 미소유 참조와 암시적 언래핑 옵셔널 프로퍼티 (Unowned References and Implicitly Unwrapped Optional Properties)에서 설명한 대로 클래스 초기화 중에 주로 사용합니다.

나중에 변수가 `nil`이 될 가능성이 있다면 암시적 언래핑 옵셔널을 사용하면 안됩니다. 변수를 사용하는 동안 `nil` 값을 확인해야 한다면 일반적인 옵셔널 타입을 사용해야 합니다.

암시적 언래핑 옵셔널은 내부적으로 옵셔널이지만, 옵셔널에 접근할 때마다 옵셔널 값을 언래핑할 필요없이 옵셔널이 아닌 값처럼 사용할 수 있습니다. 다음 예시는 명시적 `String`로서 값에 접근할 때 옵셔널 문자열과 암시적 언래핑 옵셔널 문자열의 동작 차이를 보여줍니다:

```
let possibleString: String? = "An optional string."
let forcedString: String = possibleString! // Requires explicit unwrapping

let assumedString: String! = "An implicitly unwrapped optional string."
let implicitString: String = assumedString // Unwrapped automatically
```

암시적 언래핑 옵셔널은 필요할 경우 옵셔널을 강제로 언래핑할 수 있도록 허용한다고 생각할 수 있습니다. 암시적 언래핑 옵셔널을 사용할 때, Swift는 처음에 기존의 옵셔널 값으로 사용하려고 합니다; 사용이 불가능할 경우, Swift는 값을 강제로 언래핑합니다. 위의 코드에서 옵셔널 값 `assumedString`은 `implicitString`이 명시적으로 옵셔널이 아닌 `String` 타입이기 때문에 `implicitString`에 값을 할당하기 전에 강제로 언래핑됩니다. 아래의 코드에서 `optionalString`은 명시적 타입이 없으므로 일반 옵셔널로 취급합니다.

```
let optionalString = assumedString
// The type of optionalString is "String?" and assumedString isn't force-unwrapped.
```

암시적 언래핑 옵셔널이 `nil`이고 값에 접근하려고 하면 런타임 오류가 발생합니다. 결과는 값을 포함하지 않는 일반적인 옵셔널을 강제 언래핑을 하기위해 느낌표를 작성한 것과 같습니다.

암시적 언래핑 옵셔널은 일반 옵셔널과 같은 방법으로 `nil` 체크를 할 수 있습니다:

```
if assumedString != nil {
    print(assumedString!)
}
// Prints "An implicitly unwrapped optional string.".
```

옵셔널 바인딩과 함께 암시적 언래핑 옵셔널은 단일 구문으로 해당 값을 확인하고 언래핑할 수 있습니다.

```
if let definiteString = assumedString {
    print(definiteString)
}
// Prints "An implicitly unwrapped optional string.".
```

메모리 안전성 (Memory Safety)

위의 타입 안전성과 타입 추론 (Type Safety and Type Inference)에서 설명한대로 타입 불일치를 방지하기 위한 검사 외에도 Swift는 유효하지 않은 메모리 작업을 방지하도록 코드를 보호합니다. 이 보호는 *메모리 안전성(memory safety)*라고 하며 다음의 요구사항을 포함합니다:

- 값은 읽기 전에 설정됩니다. 초기화되지 않은 메모리 영역과의 상호작용을 방지하는 이 보호를 *확정 초기화(definite initialization)*라고도 합니다.
- 배열과 버퍼는 유효한 인덱스로만 접근됩니다. 경계를 벗어난 접근을 방지하는 이 보호를 *경계 안전성(bounds safety)*라고도 합니다.
- 메모리는 값의 수명 중에만 접근됩니다. 할당 해제 후 사용(use-after-free) 오류를 방지하는 이 보호를 *수명 안전성(lifetime safety)*라고도 합니다.
- 메모리 접근은 증명 가능한 안전한 방식으로만 겹칩니다. 동시성 코드에서 발생 가능한 데이터 경쟁을 방지하는 이 보호를 *스레드 안전성(thread safety)*라고도 합니다.

이러한 보장을 제공하지 않는 언어에서 작업한다면, 위에 나열된 오류와 버그에 대해 익숙할 것입니다. 이러한 문제를 겪어보지 않았더라도 괜찮습니다; Swift의 안전한 코드는 이러한 문제를 피합니다. Swift가 어떻게 초기값 설정을 보장하는지에 대한 자세한 내용은 초기화 (Initialization)를 참고하고, Swift가 어떻게 동시성 코드에서 메모리 안전성을 검사하는지에 대한 자세한 내용은 동시성 (Concurrency)을 참고하고, Swift가 어떻게 메모리의 겹치는 접근을 검사하는지에 대한 자세한 내용은 메모리 안전성 (Memory Safety)을 참고바랍니다.

때로는 예를 들어 언어나 표준 라이브러리의 한계 때문에 안전성의 경계 밖에서 작업을 해야할 수도 있습니다. 이러한 경우에 Swift는 일부 API의 안전하지 않은 버전을 제공합니다. 이름에 “unsafe”, “unchecked”, “unmanaged”와 같은 단어를 포함하는 타입이나 메서드를 사용할 때는, 안전에 대한 책임을 직접 처리해야 합니다.

Swift의 안전한 코드도 오류와 예기치 않은 실패로 프로그램의 실행이 중단될 수 있습니다. 안전성이 코드의 완벽한 실행을 보장하지 않습니다. Swift는 아래 오류 처리 (Error Handling)와 어설션과 전제 조건 (Assertions and Preconditions)에서 설명한대로 오류를 나타내고 복구하는 여러 방법을 제공합니다. 그러나 일부 경우에 오류를 처리하는 유일한 방법은 실행을 중단하는 것입니다. 서비스가 예기치 않게 중단되지 않아야 한다면, 구성요소 중 하나가 예기치 않게 중단되더라도 복구할 수 있도록 전체 아키텍처에 내결함성을 통합해야 합니다.

오류 처리 (Error Handling)

*오류 처리(error handling)*는 프로그램 실행 중에 프로그램이 만날 수 있는 오류 조건에 대응하기 위해 사용됩니다.

값의 존재 유무를 사용하여 함수의 성공 또는 실패를 전달할 수 있는 옵셔널과 달리 오류 처리는 오류 원인을 판별하고 필요한 경우 오류를 프로그램의 다른 부분으로 전파할 수 있습니다.

함수가 오류 조건을 만나면, 그 함수는 오류를 발생시킵니다. 해당 함수의 호출자는 오류를 포착하고 적절하게 응답할 수 있습니다.

```
func canThrowAnError() throws {
    // this function may or may not throw an error
}
```

함수는 선언에 `throws` 키워드를 포함시켜 오류가 발생할 수 있음을 나타냅니다. 오류를 발생할 수 있는 함수를 호출할 때는 표현식 앞에 `try` 키워드를 붙여야 합니다.

Swift는 `catch` 절에 의해 처리될 때까지 현재 범위에서 오류를 자동으로 전파합니다.

```
do {
    try canThrowAnError()
    // no error was thrown
} catch {
    // an error was thrown
}
```

`do` 구문은 새로운 범위를 생성하여 오류가 하나 이상의 `catch` 절로 전파될 수 있도록 합니다.

다음은 오류 처리로 다양한 오류 조건에 대응하는 방법의 예입니다:

```
func makeASandwich() throws {
    // ...
}

do {
    try makeASandwich()
    eatASandwich()
} catch SandwichError.outOfCleanDishes {
    washDishes()
} catch SandwichError.missingIngredients(let ingredients) {
    buyGroceries(ingredients)
}
```

이 예시에서 `makeASandwich()` 함수는 깨끗한 그릇을 사용할 수 없거나 재료가 없는 경우 오류가 발생할 것입니다. `makeASandwich()` 함수는 오류를 발생할 수 있으므로, `try` 표현식으로 래핑되어 호출합니다. 함수 호출을 `do` 구문으로 래핑하면 모든 오류는 `catch` 절로 전파됩니다.

오류가 발생하지 않으면 `eatASandwich()` 함수는 호출됩니다. `SandwichError.outOfCleanDishes` 오류가 발생하면, `washDishes()` 함수가 호출됩니다. `SandwichError.missingIngredients` 오류가 발생하면, `catch` 패턴에 의해 캡쳐된 `[String]` 값과 함께 `buyGroceries(_:)` 함수가 호출됩니다.

오류 발생, 포착, 전파는 오류 처리 (Error Handling)에서 자세히 다룹니다.

어설션과 전제 조건 (Assertions and Preconditions)

*어설션(Assertion)*과 *전제 조건(Precondition)*은 실행 중에 이루어지는 검사입니다. 추가 코드를 실행하기 전에 이를 사용하여 필수 조건이 충족되는지 확인할 수 있습니다. 어설션이나 전제 조건의 Boolean 조건이 `true`이면 코드는 평소와 같이 실행됩니다. 조건이 `false`로 판단되면, 프로그램의 현재 상태는 유효하지 않습니다; 코드 실행은 종료되고 앱은 종료됩니다.

어설션과 전제 조건은 가정과 기대치를 표현하므로 코드의 일부로 포함할 수 있습니다. 어설션은 개발 과정에서 실수와 잘못된 가정을 찾는데 도움이 되고, 전제 조건은 프로덕션 환경에서 문제를 감지하는데 도움이 됩니다.

실행 중에 예상한 값을 검사하는 것 외에도 어설션과 전제 조건은 코드 내에서 유용한 문서 형식이 됩니다. 위의 오류 처리 (Error Handling)와 다르게 어설션과 전제 조건은 복구 가능하거나 예상되는 오류에 사용되지 않습니다. 실패한 어설션이나 전제 조건은 유효하지 않은 프로그램 상태를 나타내기 때문에 실패한 상태를 처리할 방법은 없습니다. 유효하지 않은 상태에서 복구하는 것은 불가능합니다. 어설션이 실패하면, 프로그램의 데이터 중 하나가 유효하지 않다는 의미입니다 - 그러나 그것이 왜 유효하지 않거나 다른 상태도 유효하지 않은지 알 수 없습니다.

어설션과 전제 조건의 사용이 유효하지 않은 상태가 발생하기 어렵도록 코드를 설계하는 일을 대신하지는 않습니다. 그러나 유효한 데이터 및 상태를 강제하는데 어설션과 전제 조건을 사용하면 잘못된 상태가 발생할 경우 앱이 더 예측 가능하게 종료되고 문제를 디버깅하는데 도움이 됩니다. 가정이 검사되지 않으면, 다른 코드가 실패하기 시작하고, 사용자 데이터가 손상된 후에야 이런 종류의 문제를 알 수 있습니다. 유효하지 않은 상태가 감지되는 즉시 실행을 중지하면 해당 유효하지 않은 상태로 인한 피해를 제한하는데 도움이 됩니다.

어설션과 전제 조건의 차이점은 언제 체크되는지에 있습니다: 어설션은 오직 디버그 빌드에서 체크되지만 전제 조건은 디버그와 프로덕션 빌드에서 체크됩니다. 프로덕션 빌드일 때, 어설션 내부의 조건은 실행되지 않습니다. 이 의미는 프로덕션에서 성능의 영향이 없이 개발 단계에서 많은 양의 어설션을 사용할 수 있다는 뜻입니다.

어설션을 통한 디버깅 (Debugging with Assertions)

Swift 표준 라이브러리에 `assert(_:_:file:line:)` 함수로 어설션을 작성할 수 있습니다. 이 함수에 `true`나 `false`로 판단될 표현식과 조건이 `false`일 경우 출력될 메세지를 전달합니다. 예를 들어:

```
let age = -3
assert(age >= 0, "A person's age can't be less than zero.")
// This assertion fails because -3 is not >= 0.
```

이 예에서 코드는 `age`가 음수가 아니고 `age >= 0`이 `true`일 경우 이어서 실행됩니다. `age`가 음수이면 `age >= 0`은 `false`가 되고, 어설션은 실패되고 애플리케이션은 종료됩니다.

어설션 메세지를 생략할 수 있습니다 - 예를 들어 평범하게 조건만 반복되는 경우에 사용합니다.

```
assert(age >= 0)
```

이미 코드에서 조건을 검사하고 있는 경우, `assertionFailure(_:file:line:)` 함수를 사용하여 어설션이 실패했음을 나타낼 수 있습니다. 예를 들어:

```
if age > 10 {
    print("You can ride the roller-coaster or the ferris wheel.")
} else if age >= 0 {
    print("You can ride the ferris wheel.")
} else {
    assertionFailure("A person's age can't be less than zero.")
}
```

강제 전제 조건 (Enforcing Preconditions)

조건이 거짓일 가능성이 있지만 코드 실행을 계속하려면 반드시 참이어야 하는 경우 전제 조건을 사용합니다. 예를 들어, 어떤 값들이 범위를 벗어나는지 또는 함수에 유효한 값이 전달되는지 체크하기 위해 전제 조건을 사용합니다.

`precondition(_:_:file:line:)` 함수로 전제 조건을 작성할 수 있습니다. 이 함수에 `true`나 `false`로 판단될 표현식과 조건이 `false`일 경우 출력될 메세지를 전달합니다. 예를 들어:

```
// In the implementation of a subscript...
precondition(index > 0, "Index must be greater than zero.")
```

`preconditionFailure(_:file:line:)` 함수를 호출하여 실패가 발생했음을 나타낼 수 있습니다 - 예를 들어, 스위치 문에서 기본 케이스가 선택되었지만, 모든 유효한 입력 데이터는 스위치의 다른 케이스 중 하나에서 처리되어야 하는 경우에 사용할 수 있습니다.

> **참고**
>
> 체크하지 않는 모드(`-Ounchecked`)로 컴파일하면, 전제 조건을 검사하지 않습니다. 컴파일러는 전제 조건이 항상 참이라고 가정하고 코드를 알맞게 최적화 합니다. 그러나 `fatalError(_:file:line:)` 함수는 최적화 설정과 무관하게 항상 중지를 실행합니다.
>
> 프로토타입과 초기 개발 단계에서 아직 구현되지 않은 기능에서 `fatalError(_:file:line:)`을 사용할 수 있으며 `fatalError("Unimplemented")`와 같이 작성할 수 있습니다. 어설션이나 전제 조건과 다르게 치명적인 오류는 절대 최적화 되지 않기 때문에 이 구현을 만나면 항상 중지됩니다.

언어 가이드

기본 연산자 (Basic Operators)

할당, 산술, 비교와 같은 동작을 수행합니다.

*연산자(operator)*는 값을 확인, 변경, 결합하기 위해 사용하는 기호나 구 입니다. 예를 들어 덧셈 연산자(addition operator)(+)는 `let i = 1 + 2`에서처럼 두 숫자를 더하고, 논리 AND 연산자(&&)는 `if enteredDoorCode && passedRetinaScan`에서처럼 두 Boolean 값을 결합합니다.

Swift는 C와 같은 언어에서 알고 있는 연산자를 지원하고, 일반적인 코딩 오류를 방지하기 위해 여러 기능을 개선했습니다. 할당 연산자(assignment operator)(=)는 등가 연산자(==)를 의도했을 때 실수로 사용되지 않도록 값을 반환하지 않습니다. 산술 연산자(+, –, *, /, % 등)는 타입이 수용할 수 있는 값의 범위보다 크거나 작은 숫자로 작업을 수행하려할 때 예기치 않은 결과를 피하기 위해 값 오버플로우(overflow)를 감지하고 허용하지 않습니다. Swift의 오버플로우 연산자를 사용하여 값 오버플로우 동작을 선택적으로 활성화할 수 있으며, 이에 대한 자세한 내용은 오버플로우 연산자 (Overflow Operators)에서 확인할 수 있습니다.

Swift는 C에서는 없는 `a..<b` 및 `a...b`와 같은 값의 범위를 나타내는 범위 연산자를 제공합니다.

이 챕터에서는 Swift의 기본 연산자에 대해 설명합니다. 고급 연산자 (Advanced Operators)에서는 Swift의 고급 연산자에 대해 다루고 커스텀 연산자를 어떻게 정의하고 커스텀 타입을 위한 표준 연산자의 구현은 어떻게 하는지 설명합니다.

용어 (Terminology)

연산자는 단항(unary), 이항(binary), 삼항(ternary)으로 구분됩니다:

- 단항(Unary) 연산자는 단일 항목에 동작합니다(예: –a). 단항 접두 연산자는 항목 바로 직전에 위치하고(예: `!b`), 단항 접미 연산자는 항목 바로 다음에 위치합니다(예: `c!`).
- 이항(Binary) 연산자는 두 항목에 동작하고(예: `2 + 3`) 두 항목 사이에 위치하므로 위치는 중위 연산자라고 합니다.
- 삼항(Ternary) 연산자는 세 항목에 동작합니다. C처럼 Swift는 하나의 삼항 연산자만 있으며, 삼항 조건 연산자입니다(`a ? b : c`).

연산자가 작용하는 값을 피연산자라고 합니다. `1 + 2` 표현식에서 +기호는 중위 연산자(infix operator)이고 값 1과 2는 피연산자 입니다.

할당 연산자 (Assignment Operator)

할당 연산자(assignment operator)(`a = b`)는 a가 b로 초기화 되거나 업데이트 됩니다:

```
let b = 10
var a = 5
a = b
// a is now equal to 10
```

할당의 우항이 여러개의 값이 있는 튜플이라면, 튜플의 요소는 여러개의 상수나 변수로 한 번에 분해될 수 있습니다:

```
let (x, y) = (1, 2)
// x is equal to 1, and y is equal to 2
```

C와 Objective-C에서의 할당 연산자와 다르게 Swift의 할당 연산자는 값을 반환하지 않습니다. 아래 예문은 유효하지 않습니다:

```
if x = y {
    // This is not valid, because x = y does not return a value.
}
```

이 특징은 등가 연산자(==)가 실제로 사용되어야 할 때, 실수로 할당 연산자(=)가 사용되는 것을 방지합니다. `if x = y`가 유효하지 않는 것처럼, Swift는 코드에서 이런 종류의 오류를 피할 수 있도록 도와줍니다.

산술 연산자 (Arithmetic Operators)

Swift는 모든 숫자 타입에 대해 4개의 기본 *산술 연산자(arithmetic operators)*를 제공합니다:

- 덧셈(+)
- 뺄셈(–)
- 곱셈(*)
- 나눗셈(/)

```
1 + 2       // equals 3
5 - 3       // equals 2
2 * 3       // equals 6
10.0 / 2.5  // equals 4.0
```

C와 Objective-C에서의 산술 연산자와 다르게 Swift 산술 연산자는 기본적으로 값의 오버플로우를 허용하지 않습니다. Swift의 오버플로우 연산자(예: `a &+ b`)를 사용하여 값 오버플로우 동작을 선택적으로 활성화할 수 있습니다. 자세한 내용은 오버플로우 연산자 (Overflow Operators)를 참고바랍니다.

덧셈 연산자는 `String` 연결도 지원합니다:

```
"hello, " + "world"  // equals "hello, world"
```

나머지 연산자 (Remainder Operator)

나머지 연산자(remainder operator)(`a % b`)는 `a` 안에 들어갈 `b`의 배수가 몇 번 들어갈 수 있는지 계산하고, 남는 값을 반환합니다 (이것을 나머지라고 합니다).

> **참고**
>
> 나머지 연산자(%)는 다른 언어에서 *모듈로 연산자(modulo operator)*라고 합니다. 그러나 음수에 대한 Swift의 동작은 모듈로 연산이 아닌 나머지 연산 입니다.

여기 나머지 연산자가 어떻게 동작하는지 살펴봅시다. `9 % 4`를 계산하기위해, 9 안에 얼마나 많은 4가 들어가는지 알아야 합니다:



9에 4가 2번 들어가고 1이 남는 것을 알 수 있습니다(오렌지 색).

Swift에서 이것을 작성해보면:

```
9 % 4    // equals 1
```

a % b의 결과를 구하기 위해 % 연산자는 아래의 방정식을 계산하고 출력으로 remainder를 반환합니다:

a = (b x some multiplier) + remainder

여기서 some multiplier는 a에 들어갈 b의 최대 배수 입니다.

이 방정식에 9와 4를 넣고 계산하면:

9 = (4 x 2) + 1

a에 음수를 이용하여 나머지를 계산할 때도 같은 메서드로 적용합니다:

```
-9 % 4   // equals -1
```

방정식에 –9와 4를 넣고 계산하면:

–9 = (4 x –2) + –1

–1의 나머지를 얻습니다.

b의 음수 값에 대해서는 b의 부호가 무시됩니다. 이것은 a % b와 a % –b는 항상 같은 결과를 반환한다는 의미입니다.

단항 뺄셈 연산자 (Unary Minus Operator)

숫자 값의 부호는 – 접미사를 사용하여 변경할 수 있으며, 이것을 *단항 뺄셈 연산자(unary minus operator)*라고 합니다:

```
let three = 3
let minusThree = -three       // minusThree equals -3
let plusThree = -minusThree   // plusThree equals 3, or "minus minus three"
```

단항 빼기 연산자(–)는 공백없이 작동하는 값 바로 앞에 추가합니다.

단항 덧셈 연산자 (Unary Plus Operator)

단항 덧셈 연산자(+)는 어떠한 변경없이 그 값을 그대로 반환합니다:

```
let minusSix = -6
let alsoMinusSix = +minusSix  // alsoMinusSix equals -6
```

단항 덧셈 연산자는 실제로 아무런 동작을 하지 않지만, 음수에 단항 뺄셈 연산자를 사용할 때 양수에 대칭을 위해 사용할 수 있습니다.

복합 할당 연산자 (Compound Assignment Operators)

C처럼, Swift는 할당(=)과 다른 연산자를 결합한 *복합 할당 연산자(compound assignment operators)*를 제공합니다. 아래 예시는 덧셈 할당 연산자(+=)입니다:

```
var a = 1
a += 2
// a is now equal to 3
```

표현식 `a += 2`는 `a = a + 2`의 짧은 표현입니다. 효과적으로 덧셈과 할당은 동시에 수행되고 하나의 연산자로 결합됩니다.

> **참고**
>
> 복합 할당 연산자는 값을 반환하지 않습니다. 예를 들어 `let b = a += 2`로 작성할 수 없습니다.

Swift 표준 라이브러리에서 제공하는 연산자에 대한 내용은 연산자 선언(Operator Declarations)을 참고바랍니다.

비교 연산자 (Comparison Operators)

Swift는 아래의 비교 연산자(comparison operators)를 제공합니다:

- 같음(`a == b`)
- 다름(`a != b`)
- 보다 큼(`a > b`)
- 보다 작음(`a < b`)
- 보다 크거나 같음(`a >= b`)
- 보다 작거나 같음(`a <= b`)

> **참고**
>
> Swift는 두 객체 참조가 동일한 객체 인스턴스를 참조하는지 판별하는 식별 연산자(identity operators)(===와 !==)를 제공합니다. 자세한 내용은 동일성 연산자 (Identity Operators)를 참고바랍니다.

각 비교 연산자는 구문이 참인지 아닌지 판단하기 위해 `Bool` 값을 반환합니다:

```
1 == 1   // true because 1 is equal to 1
2 != 1   // true because 2 is not equal to 1
2 > 1    // true because 2 is greater than 1
1 < 2    // true because 1 is less than 2
1 >= 1   // true because 1 is greater than or equal to 1
2 <= 1   // false because 2 is not less than or equal to 1
```

비교 연산자는 `if` 구문과 같은 조건 구문에서 사용되기도 합니다:

```
let name = "world"
if name == "world" {
    print("hello, world")
} else {
    print("I'm sorry \(name), but I don't recognize you")
}
// Prints "hello, world", because name is indeed equal to "world"..
```

`if` 구문에 대한 자세한 내용은 제어 흐름 (Control Flow)을 참고바랍니다.

같은 타입과 같은 갯수의 값을 가지고 있는 튜플은 비교할 수 있습니다. 튜플은 값이 다를 때까지 왼쪽에서 오른쪽으로 한 번에 하나씩 비교합니다. 두 값이 비교되고 해당 비교 결과에 따라 튜플 비교의 전체 결과가 결정됩니다. 모든 요소가 동일하면, 튜플은 같습니다. 예를 들어:

```
(1, "zebra") < (2, "apple")   // true because 1 is less than 2; "zebra" and "apple" are not co
  mpared
(3, "apple") < (3, "bird")    // true because 3 is equal to 3, and "apple" is less than "bird"
(4, "dog") == (4, "dog")      // true because 4 is equal to 4, and "dog" is equal to "dog"
```

위 예시의 첫번째 줄에서 왼쪽에서 오른쪽으로 비교하는 동작을 확인할 수 있습니다. 튜플의 다른 어떤 값과 상관없이 `1`이 `2`보다 작기 때문에, `(1, "zebra")`는 `(2, "apple")`보다 작습니다. 튜플의 첫번째 요소에 의해 비교를 이미 완료했기 때문에 `"zebra"`가 `"apple"`보다 더 작은지 여부는 아무런 상관이 없습니다. 그러나 튜플의 첫번째 요소가 같을 때는 두번째 요소를 비교합니다 - 위 예시에서 두 번째와 세 번째 줄의 결과를 살펴보면 알 수 있습니다.

튜플은 해당 튜플의 각 값에 연산자를 적용할 수 있을 때에만 주어진 연산자로 비교할 수 있습니다. 예를 들어 아래의 코드에서 `String`과 `Int` 값은 `<` 연산자를 사용하여 비교가 가능하므로 타입 `(String, Int)`의 두 튜플은 비교할 수 있습니다. 반대로 `<` 연산자는 `Bool` 값에 적용할 수 없기 때문에 타입 `(String, Bool)`의 두 튜플은 `<` 연산자로 비교할 수 없습니다.

```
("blue", -1) < ("purple", 1)        // OK: Evaluates to true.
("blue", false) < ("purple", true)  // Error: Can't use < to compare Boolean values.
```

참고

Swift 표준 라이브러리는 7개 미만의 요소를 가지고 있는 튜플에 대해 튜플 비교 연산자를 제공합니다. 7개 이상의 요소의 튜플을 비교하려면 비교 연산자를 직접 구현해야 합니다.

삼항 조건 연산자 (Ternary Conditional Operator)

*삼항 조건 연산자(ternary conditional operator)*는 `question ? answer1 : answer2`형태의 세 부분으로 이루어진 특별한 연산자입니다. `question`이 참이나 거짓인지에 따라 두 표현식 중 하나를 나타내는 식입니다. `question`이 참이면 `answer1`을 반환하고; 반대면 `answer2`를 반환합니다.

삼항 조건 연산자는 아래의 코드를 줄여서 표현한 것입니다:

```
if question {
    answer1
} else {
    answer2
}
```

테이블 열의 높이를 계산하는 예시가 있습니다. 테이블 열은 헤더를 가지고 있다면 콘텐츠 높이보다 50 포인트 더 커야 하고 헤더가 없다면 20 포인트 더 커야 합니다:

```
let contentHeight = 40
let hasHeader = true
let rowHeight = contentHeight + (hasHeader ? 50 : 20)
// rowHeight is equal to 90
```

위의 예시는 아래의 코드를 짧게 표현한 것입니다:

```
let contentHeight = 40
let hasHeader = true
let rowHeight: Int
if hasHeader {
    rowHeight = contentHeight + 50
} else {
    rowHeight = contentHeight + 20
}
// rowHeight is equal to 90
```

첫번째 예에서 삼항 조건 연산자를 사용하면 `rowHeight`를 한줄의 코드로 올바른 값을 설정할 수 있으며, 두번째 예에서 사용된 코드보다 간결합니다.

삼항 조건 연산자는 두 표현식 중 어느 것을 선택할지 결정하는 효율적인 축약형을 제공합니다. 그러나 삼항 조건 연산자 사용시에는 주의가 필요합니다. 너무 남용하면 읽기 어려운 코드가 될 수 있습니다. 여러 삼항 조건 연산자를 하나의 복합 구문으로 결합하는 것을 피해야 합니다.

Nil-병합 연산자 (Nil-Coalescing Operator)

nil-병합 연산자(nil-coalescing operator)(`a ?? b`)는 옵셔널 `a`에 값이 있으면 `a`를 풀거나, `a`가 `nil`이면 기본값 `b`를 반환합니다. 표현식 `a`는 항상 옵셔널 타입 입니다. 표현식 `b`는 `a`에 저장된 타입과 같아야 합니다.

nil-병합 연산자는 아래 코드를 짧게 표현할 수 있습니다:

```
a != nil ? a! : b
```

위 코드는 삼항 조건 연산자를 사용하고 `a`가 `nil`이 아닐경우 `a` 값을 접근하기 위해 강제 언래핑(`a!`)하며 `a`가 `nil`일 경우 `b`를 반환합니다. nil-병합 연산자는 조건 검사 및 언래핑을 간결하고 읽기 쉬운 형태로 캡슐화 합니다.

> **참고**
>
> `a`의 값이 `nil`이 아닐경우 `b`는 평가되지 않습니다. 이러한 경우를 *단락 평가(short-circuit evaluation)*라 합니다.

아래 예시는 기본 컬러 이름과 사용자가 정의한 옵셔널 컬러 이름 중 선택하기 위해 nil-병합 연산자를 사용합니다:

```
let defaultColorName = "red"
var userDefinedColorName: String?   // defaults to nil

var colorNameToUse = userDefinedColorName ?? defaultColorName
// userDefinedColorName is nil, so colorNameToUse is set to the default of "red"
```

`userDefinedColorName` 변수는 기본값이 `nil` 인 옵셔널 `String`으로 정의됩니다.
`userDefinedColorName`은 옵셔널 타입이기 때문에 값을 선택하기 위해 nil-병합 연산자를 사용할 수 있습니다.
위의 예시에서 연산자는 `colorNameToUse`라는 `String` 변수의 초기값을 결정하는데 사용됩니다.
`userDefinedColorName`이 `nil`이기 때문에, 표현식 `userDefinedColorName ?? defaultColorName`은 `defaultColorName`나 `"red"`의 값을 반환합니다.

`userDefinedColorName`에 `nil`이 아닌 값을 대입하고 nil-병합 연산자로 검사를 다시 수행하면 기본값 대신에 `userDefinedColorName`에 래핑된 값을 사용합니다:

```
userDefinedColorName = "green"
colorNameToUse = userDefinedColorName ?? defaultColorName
// userDefinedColorName is not nil, so colorNameToUse is set to "green"
```

범위 연산자 (Range Operators)

Swift는 값의 범위를 짧게 표현하는 *범위 연산자(range operators)*가 있습니다.

닫힌 범위 연산자 (Closed Range Operator)

닫힌 범위 연산자(closed range operator)(`a...b`)는 값 a와 b가 포함된 a부터 b까지의 범위를 정의합니다. a의 값은 b보다 클 수 없습니다.

닫힌 범위 연산자는 `for-in` 루프와 같이 모든 값의 범위를 반복할 때 유용합니다:

```
for index in 1...5 {
    print("\(index) times 5 is \(index * 5)")
}
// 1 times 5 is 5
// 2 times 5 is 10
// 3 times 5 is 15
// 4 times 5 is 20
// 5 times 5 is 25
```

`for-in` 루프에 대한 자세한 설명은 제어 흐름 (Control Flow)을 참고바랍니다.

반-열림 범위 연산자 (Half-Open Range Operator)

반-열림 범위 연산자(half-open range operator)(`a..<b`)는 b가 포함되지 않은 a부터 b까지의 범위 실행을 정의합니다. 이것은 마지막 값은 포함되지 않지만 처음 값은 포함되므로 *반-열림(half-open)*이라 합니다. 닫힌 범위 연산자와 같이 a의 값은 b보다 클 수 없습니다. a의 값이 b와 같다면 결과 범위는 비어있을 것입니다.

반열린 범위는 리스트의 길이에 해당하는 값은 포함하지 않고 그 직전까지 순회할 때 유용하며, 배열처럼 인덱스가 0부터 시작하는 리스트를 다루기에 적합합니다:

```
let names = ["Anna", "Alex", "Brian", "Jack"]
let count = names.count
for i in 0..<count {
    print("Person \(i + 1) is called \(names[i])")
}
// Person 1 is called Anna
// Person 2 is called Alex
// Person 3 is called Brian
// Person 4 is called Jack
```

배열은 4개의 아이템을 가지고 있지만 `0..<count`는 반-열림 범위이기 때문에 오직 3 (배열의 마지막 아이템의 인덱스)까지 카운트 합니다. 배열에 대한 자세한 설명은 배열 (Arrays)을 참고바랍니다.

단-방향 범위 (One-Sided Ranges)

닫힌 범위 연산자는 한 방향으로 계속되는 범위에 대한 대체 형식입니다 - 인덱스 2에서 배열 끝까지 배열의 모든 요소를 포함하는 범위가 그 예입니다. 이러한 경우 범위 연산자의 한 쪽을 생략할 수 있습니다. 이러한 범위의 종류는 연산자가 오직 한쪽의 값만 가지고 있으므로 *단-방향 범위(one-sided range)*라고 합니다. 예를 들어:

```
for name in names[2...] {
    print(name)
}
// Brian
// Jack

for name in names[...2] {
    print(name)
}
// Anna
// Alex
// Brian
```

반-열림 연산자는 마지막 값만 기입하여 단-방향 형식을 가질 수 있습니다. 보통 형식과 같이 마지막 값은 범위에 포함되지 않습니다. 예를 들어:

```
for name in names[..<2] {
    print(name)
}
// Anna
// Alex
```

단-방향 범위는 서브스크립트 뿐만 아니라 다른 상황에서도 사용할 수 있습니다. 첫 번째 값을 생략한 단-방향 범위는 어디서부터 반복을 시작해야 할지 명확하지 않기 때문에 반복할 수 없습니다. 반복문에서 단-방향 범위의 마지막 값은 생략할 수 있습니다; 그러나 범위가 무한정 계속되므로 루프에 대해 명시적으로 종료 조건을 추가해야 합니다. 아래 코드와 같이 단-방향 범위에 특정 값이 포함되어 있는지 체크할 수 있습니다.

```
let range = ...5
range.contains(7)   // false
range.contains(4)   // true
range.contains(-1)  // true
```

논리 연산자 (Logical Operators)

*논리 연산자(Logical operator)*는 Boolean 로직 값을 `true`와 `false`로 수정하거나 결합합니다. Swift는 C-기반 언어에서 볼 수 있는 세 가지 표준 논리 연산자를 제공합니다:

- 논리적 NOT(`!a`)
- 논리적 AND(`a && b`)
- 논리적 OR(`a || b`)

논리적 NOT 연산자 (Logical NOT Operator)

논리적 NOT 연산자(logical NOT operator)(`!a`)는 Boolean 값에 대해 `true`를 `false`로 `false`를 `true`와 같이 반대로 만듭니다.

논리적 NOT 연산자는 접두사 연산자이며, 동작할 값 바로 앞에 공백없이 위치합니다. 이것은 아래 예시와 같이 "a가 아니다"라고 읽을 수 있습니다:

```
let allowedEntry = false
if !allowedEntry {
    print("ACCESS DENIED")
}
// Prints "ACCESS DENIED".
```

`if !allowedEntry` 구문은 "엔트리가 허락되지 않는다면"이라고 읽을 수 있습니다. 다음 라인은 "엔트리가 허락되지 않음"이 참일 때; 이 말은 `allowedEntry`가 `false` 일 경우에만 실행된다는 의미입니다.

이 예시와 같이 Boolean 상수와 변수 이름을 신중하게 선택하면 중복 부정이나 논리적 상태의 혼동을 피하기 위해 읽기 쉽고 간결함을 유지할 수 있습니다.

논리적 AND 연산자 (Logical AND Operator)

논리적 AND 연산자(logical AND operator)(`a && b`)는 두 값이 모두 `true`여야 `true`를 표현하는 논리적 표현식을 만듭니다.

두 값중 하나라도 `false`이면, 결과는 `false`입니다. 사실 첫 번째 값이 `false`이면, `true`의 결과를 얻을 수 없기 때문에 두번째 값은 살펴보지 않습니다. 이러한 경우를 *단락 평가(short-circuit evaluation)*라 합니다.

이 예시는 두 `Bool` 값이 `true` 일 경우에만 접근을 허용합니다:

```
let enteredDoorCode = true
let passedRetinaScan = false
if enteredDoorCode && passedRetinaScan {
    print("Welcome!")
} else {
    print("ACCESS DENIED")
}
// Prints "ACCESS DENIED".
```

논리적 OR 연산자 (Logical OR Operator)

논리적 OR 연산자(logical OR operator)(`a || b`)는 두 인접한 파이프 문자(|)로 만들어진 중위 연산자(infix operator)입니다. 이것을 사용하여 두 값 중 하나라도 `true`이면 표현식이 `true`가 되는 논리적 표현식을 만듭니다.

위에서 논리적 AND 연산자와 마찬가지로 논리적 OR 연산자는 단락 평가(short-circuit evaluation)를 사용하여 표현식을 고려합니다. 논리적 OR 연산자의 좌변이 `true`이면, 표현식의 결과는 바뀌지 않으므로 우변은 고려하지 않습니다.

아래 예시에서 첫번째 `Bool` 값(`hasDoorKey`)은 `false`이지만, 두번째 값(`knowsOverridePassword`)은 `true`입니다. 하나의 값이 `true`이기 때문에, 표현식은 `true`가 되고 접근이 허용됩니다:

```
let hasDoorKey = false
let knowsOverridePassword = true
if hasDoorKey || knowsOverridePassword {
    print("Welcome!")
} else {
    print("ACCESS DENIED")
}
// Prints "Welcome!".
```

논리적 연산자 결합 (Combining Logical Operators)

긴 복합 표현식을 생성하기 위해 여러 개의 논리적 연산자를 결합할 수 있습니다:

```
if enteredDoorCode && passedRetinaScan || hasDoorKey || knowsOverridePassword {
    print("Welcome!")
} else {
    print("ACCESS DENIED")
}
// Prints "Welcome!".
```

이 예시는 긴 복합 표현식을 생성하기 위해 여러 개의 &&와 || 연산자를 사용하였습니다. 그러나 &&와 || 연산자는 오직 두 값에서만 동작하므로, 이것은 세 개의 작은 표현식이 함께 엮어있는 형태입니다. 이 예시는 아래와 같이 읽을 수 있습니다:

올바른 출입문 코드를 입력하고 망막 스캔을 통과했거나 유효한 키를 가지고 있거나 긴급 비밀번호를 알고 있을 경우에 접근이 허용됩니다.

`enteredDoorCode`, `passedRetinaScan`, `hasDoorKey`의 값을 기반으로 첫 번째 두 표현식은 `false`입니다. 그러나 긴급 비밀번호는 알고 있으므로 이 표현식의 결과는 `true`입니다.

> **참고**
>
> Swift 논리적 연산자 &&와 ||은 왼쪽 우선 결합(left-associative)입니다 그 의미는 여러 개의 논리적 연산자로 이루어진 복합 표현식은 가장 왼쪽부터 판단한다는 뜻입니다.

명시적 소괄호 (Explicit Parentheses)

소괄호를 포함하는 것이 영향이 없을 때 복합 표현식을 읽기 쉽게 하기위해 소괄호를 포함하는 것은 때때로 유용합니다. 위의 출입문 접근 예시에서 의도를 명확하게 하기위해 복합 표현식의 첫 번째 부분을 소괄호를 추가하면 유용합니다:

```
if (enteredDoorCode && passedRetinaScan) || hasDoorKey || knowsOverridePassword {
    print("Welcome!")
} else {
    print("ACCESS DENIED")
}
// Prints "Welcome!".
```

소괄호는 처음 두 값을 전체 논리에서 별도의 부분으로 고려되도록 해줍니다. 복합 표현식의 결과는 바뀌지 않지만 전체적인 의도는 명확해 집니다. 가독성은 항상 간결성보다 선호됩니다; 의도를 명확하게 하는데 도움이 되는 곳에 소괄호를 사용해야 합니다.

언어 가이드

문자열과 문자 (Strings and Characters)

텍스트를 저장하고 다룹니다.

*문자열(string)*은 `"hello, world"`나 `"albatross"`와 같은 문자의 연속입니다. Swift 문자열은 `String` 타입으로 표현됩니다. `String`의 내용은 `Character` 값의 컬렉션으로 접근하는 것을 포함하여 여러 가지 방법으로 접근할 수 있습니다.

Swift의 `String`과 `Character` 타입은 코드에서 텍스트를 처리 하는 빠르고 유니코드에 호환되는 방법을 제공합니다. 문자열 생성과 취급을 위한 구문은 C와 유사한 리터럴 구문을 사용하여 가볍고 읽기 쉽습니다. 문자열 연결은 두 문자열 사이에 + 연산자와 함께 결합하여 간단하게 연결이 가능하고 Swift의 다른 값과 마찬가지로 상수나 변수 중에서 선택 하여 문자열 변경 가능성을 관리합니다. 문자열 삽입이라는 프로세스를 통해 문자열을 사용하여 상수, 변수, 리터럴, 표현식을 긴 문자열에 삽입할 수 있습니다. 이것은 화면에 표시, 저장, 출력을 위한 커스텀 문자열 값을 쉽게 생성할 수 있습니다.

간단한 구문임에도 Swift의 `String` 타입은 빠르고, 최신 문자열 구현입니다. 모든 문자열은 인코딩에 독립적인 유니코드 문자로 구성되어 있으며, 다양한 유니코드 표현의 문자에 접근할 수 있도록 지원합니다.

> **참고**
>
> Swift의 `String` 타입은 Foundation의 `NSString` 클래스와 연결되어 있습니다. Foundation은 또한 `NSString`에 의해 정의된 메서드를 노출시키기 위해 `String`을 확장합니다. 이것은 Foundation을 import 하면, 캐스팅 없이 `String`에서 `NSString` 메서드를 접근할 수 있습니다.
>
> Foundation과 Cocoa에서 `String` 사용에 대한 자세한 내용은 String과 NSString의 연결 (Bridging Between String and NSString)을 참고바랍니다.

문자열 리터럴 (String Literals)

코드 내에 미리 정의된 `String` 값을 *문자열 리터럴(string literals)*로 포함할 수 있습니다. 문자열 리터럴은 쌍따옴표(")로 둘러싸인 문자의 연속입니다.

상수나 변수의 초기값으로 문자열 리터럴을 사용합니다:

```
let someString = "Some string literal value"
```

Swift는 문자열 리터럴 값으로 초기화 되었기 때문에 `someString` 상수를 `String` 타입으로 추론합니다.

여러줄 문자열 리터럴 (Multiline String Literals)

여러 줄의 문자열이 필요하다면, 여러 줄 문자열 리터럴을 사용하면 됩니다 - 연속된 문자에 세 개의 쌍따옴표로 둘러싸 사용합니다:

```
let quotation = """
The White Rabbit put on his spectacles.  "Where shall I begin,
please your Majesty?" he asked.

"Begin at the beginning," the King said gravely, "and go on
till you come to the end; then stop."
"""
```

여러 줄 문자열 리터럴은 열리고 닫힌 따옴표 사이에 있는 모든 라인을 포함합니다. 문자열은 여는 따옴표(""") 다음 줄부터 시작하고 닫는 따옴표의 이전 줄로 끝납니다. 이것은 아래의 문자열은 줄바꿈 없이 시작하고 끝난다는 의미입니다:

```
let singleLineString = "These are the same."
let multilineString = """
These are the same.
"""
```

소스 코드에서 여러 줄 문자열 리터럴에 줄 바꿈을 포함하면 줄 바꿈도 문자열의 값으로 나타납니다. 소스 코드를 쉽게 읽을 수 있게 줄 바꿈을 사용하고 싶지만, 줄 바꿈이 문자열 값의 일부가 되는걸 원치 않을 때는 줄 바꿈을 원하는 줄 끝에 역슬래시(\)를 쓰면 됩니다:

```
let softWrappedQuotation = """
The White Rabbit put on his spectacles.  "Where shall I begin, \
please your Majesty?" he asked.

"Begin at the beginning," the King said gravely, "and go on \
till you come to the end; then stop."
"""
```

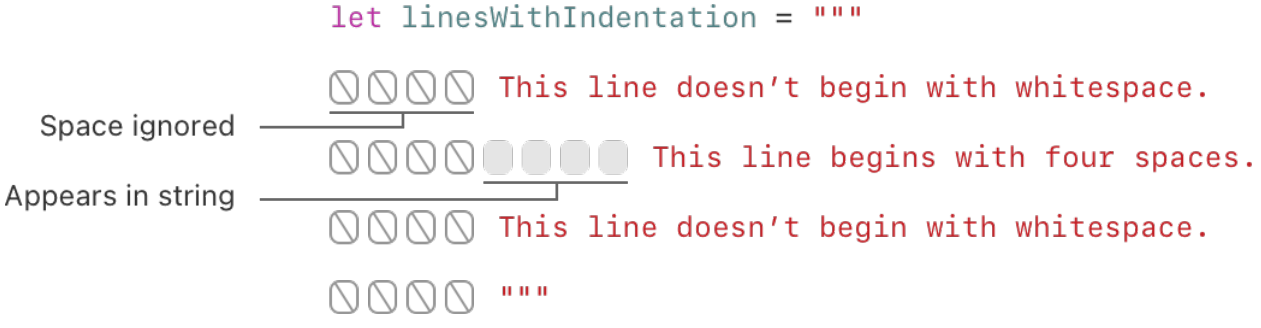
여러 줄 문자열 리터럴의 시작이나 끝에 빈 줄을 추가하고 싶다면, 첫 줄이나 마지막 줄에 빈 줄을 추가하면 됩니다. 예를 들어:

```
let lineBreaks = """

This string starts with a line break.
It also ends with a line break.

"""
```

여러 줄 문자열은 주변 코드와 일치하도록 들여쓸 수 있습니다. 닫는 따옴표(""") 앞의 공백은 Swift가 다른 모든 줄 앞에 있는 공백을 무시한다는 것을 말합니다. 그러나 닫는 따옴표 앞의 공백 외에도 줄 시작 부분에 공백을 추가하면, 그 공백은 추가됩니다.



위의 예시에서 여러 줄 문자열 리터럴 전체가 들여쓰기 되어있지만, 문자열에서 첫번째와 마지막 줄은 어떠한 공백없이 시작합니다. 중간 줄은 닫는 따옴표보다 들여쓰기가 더 많으므로, 4칸 들여쓰기로 시작합니다.

문자열 리터럴에 특수 문자 (Special Characters in String Literals)

문자열 리터럴은 아래와 같은 특수 문자를 포함할 수 있습니다:

- 이스케이프된 문자 `\0`(null 문자), `\\`(역슬래시), `\t`(수평 탭), `\n`(개행), `\r`(캐리지 리턴), `\"`(쌍따옴표)와 `\'`(홑따옴표)
- 임의의 유니코드 스칼라 값은 `\u{n}` 형식으로 작성되며, 여기서 n은 1-8 자리의 16진수 숫자 (유니코드는 아래 유니코드 (Unicode)에서 설명합니다.)

아래 코드는 특수 문자의 네 가지 예를 보여줍니다. `wiseWords` 상수는 두 이스케이프된 쌍따옴표를 포함합니다. `dollarSign`, `blackHeart`, `sparklingHeart` 상수는 유니코드 스칼라 형식을 가집니다:

```
let wiseWords = "\"Imagination is more important than knowledge\" - Einstein"
// "Imagination is more important than knowledge" - Einstein
let dollarSign = "\u{24}"        // $,  Unicode scalar U+0024
let blackHeart = "\u{2665}"      // ♥,  Unicode scalar U+2665
let sparklingHeart = "\u{1F496}" // 💖, Unicode scalar U+1F496
```

여러 줄 문자열 리터럴은 세 쌍따옴표를 사용하기 때문에 이스케이프 없이 여러 줄 문자열 리터럴 안에 쌍따옴표(")를 포함할 수 있습니다. 여러 줄 문자열 텍스트에 `"""`를 포함하려면, 따옴표에 적어도 하나의 이스케이프를 포함해야 합니다. 예를 들어:

```
let threeDoubleQuotationMarks = """
Escaping the first quotation mark \"""
Escaping all three quotation marks \"\"\"
"""
```

확장된 문자열 구분기호 (Extended String Delimiters)

아무런 영향없이 문자열에 특수 문자를 포함하기 위해 확장된 구분기호(extended delimiters) 안에 문자열 리터럴을 위치할 수 있습니다. 문자열을 따옴표(")로 둘러싸고 숫자 기호(#)로 둘러쌉니다. 예를 들어 문자열 리터럴 `#"Line 1\nLine 2"#`을 출력하면 2줄로 출력되지 않고 개행 문자(`\n`)가 출력됩니다.

문자열 리터럴에서 문자의 특수 영향이 필요하다면, 이스케이프 문자(\) 다음에 선언된 숫자 기호의 개수를 문자열 안에서 일치시켜야 합니다. 예를 들어 문자열 `#"Line 1\nLine 2"#`에 개행을 수행하고 싶다면, `#"Line 1\#nLine 2"#`로 작성하면 됩니다. 비슷하게 `###"Line1\###nLine2"###`도 개행이 이루어집니다.

확장된 구분기호를 사용하여 생성된 문자열 리터럴은 여러 줄 문자열 리터럴로 사용할 수도 있습니다. 여러 줄 문자열에 텍스트 `"""`를 포함하기 위해 리터럴을 종료하는 기본 동작을 재정의 하여 확장된 구분기호를 사용할 수 있습니다. 예를 들어:

```
let threeMoreDoubleQuotationMarks = #"""
Here are three more double quotes: """
"""#
```

빈 문자열 초기화 (Initializing an Empty String)

긴 문자열을 만들기 위한 시작점으로 빈 `String` 값을 생성하려면, 빈 문자열 리터럴을 변수에 할당하거나 이니셜라이저으로 새로운 `String` 인스턴스를 초기화합니다:

```
var emptyString = ""               // empty string literal
var anotherEmptyString = String()  // initializer syntax
// these two strings are both empty, and are equivalent to each other
```

`String` 값이 비어있는지 확인하기 위해서는 Boolean `isEmpty` 프로퍼티로 확인할 수 있습니다:

```
if emptyString.isEmpty {
    print("Nothing to see here")
}
// Prints "Nothing to see here".
```

문자열 변경 (String Mutability)

특정 `String`을 변수에 할당하면(이 경우 수정 가능합니다) 수정될 수 있고(또는 변형될 수 있고) 상수에 할당하면(이 경우 수정할 수 없습니다) 수정할 수 없습니다:

```
var variableString = "Horse"
variableString += " and carriage"
// variableString is now "Horse and carriage"

let constantString = "Highlander"
constantString += " and another Highlander"
// this reports a compile-time error - a constant string cannot be modified
```

> **참고**
>
> 이 접근방식은 Objective-C와 Cocoa의 문자열 변형과 다릅니다. 여기서 두 클래스(`NSString` 및 `NSMutableString`) 중에서 선택하여 문자열 변형이 가능한지 여부를 나타냅니다.

문자열은 값 타입 (Strings Are Value Types)

Swift의 `String` 타입은 *값 타입(value type)*입니다. 새로운 `String` 값을 생성하면, `String` 값은 함수나 메서드에 전달될 때나 상수나 변수에 할당될 때 복사됩니다. 각 경우에 존재하는 `String` 값의 새로운 복사본이 생성되고, 원본이 아닌 새로운 복사본은 전달되거나 할당됩니다. 값 타입은 구조체와 열거형은 값 타입 (Structures and Enumerations Are Value Types)에서 설명합니다.

Swift의 기본 복사 방식인 `String` 동작은 함수나 메서드가 `String` 값을 전달할 때, 출처에 관계없이 `String` 값은 정확하다고 보장합니다. 전달된 문자열은 직접 수정하지 않는 한 수정되지 않습니다.

Swift의 컴파일러는 꼭 필요할 때 실제 복사가 이뤄지도록 문자열 사용을 최적화합니다. 이것은 문자열을 값 타입으로 사용할 때는 항상 뛰어난 성능을 얻을 수 있다는 의미입니다.

문자 작업 (Working with Characters)

문자열과 `for-in` 루프로 `String`의 각각의 `Character` 값에 접근할 수 있습니다:

```
for character in "Dog!🐶" {
    print(character)
}
// D
// o
// g
// !
// 🐶
```

`for-in` 루프는 For-In 루프 (For-In Loops)에서 설명합니다.

하나의 문자 문자열 리터럴을 `Character` 타입을 명시하여 단독의 `Character` 상수나 변수를 생성할 수 있습니다:

```
let exclamationMark: Character = "!"
```

`String` 값은 초기화 인자로 `Character` 값의 배열을 전달해 생성할 수 있습니다:

```
let catCharacters: [Character] = ["C", "a", "t", "!", "🐱"]
let catString = String(catCharacters)
print(catString)
// Prints "Cat!🐱".
```

문자열과 문자 연결 (Concatenating Strings and Characters)

`String` 값은 덧셈 연산자(+)로 추가(또는 연결)하여 새로운 `String` 값을 생성할 수 있습니다:

```
let string1 = "hello"
let string2 = " there"
var welcome = string1 + string2
// welcome now equals "hello there"
```

존재하는 `String` 변수에 덧셈 할당 연산자(+=)로 `String` 값을 연결할 수도 있습니다:

```
var instruction = "look over"
instruction += string2
// instruction now equals "look over there"
```

`String` 타입의 `append()` 메서드를 이용하여 `String` 변수에 `Character` 값을 추가할 수 있습니다:

```
let exclamationMark: Character = "!"
welcome.append(exclamationMark)
// welcome now equals "hello there!"
```

참고

`Character` 값은 반드시 하나의 문자만 포함해야 하므로, `Character` 변수에 추가로 `String`나 `Character`를 추가할 수 없습니다.

긴 문자열을 만들기 위해 여러 줄 문자열 리터럴을 이용한다면, 마지막 줄을 포함하여 문자열의 모든 줄이 줄 바꿈으로 끝나도록 해야 합니다. 예를 들어:

```
let badStart = """
one
two
"""
let end = """
three
"""
print(badStart + end)
// Prints two lines:
// one
// twothree

let goodStart = """
one
two

"""
print(goodStart + end)
// Prints three lines:
// one
// two
// three
```

위의 코드에서 `badStart`와 `end`를 출력하면 원하는 결과와 다르게 2줄 문자열로 출력됩니다. `badStart`의 마지막 줄에 공백이 포함되지 않았기 때문에 `end`의 첫 번째 줄과 결합된 결과를 얻게 됩니다. 반면에 `goodStart` 마지막 줄에 개행이 포함되므로, `end`와 결합이 되면 기대했던 결과와 같이 3줄로 출력됩니다.

문자열 삽입 (String Interpolation)

*문자열 삽입(String interpolation)*은 상수, 변수, 리터럴, 문자열 리터럴에 값이 포함된 표현식을 혼합해 새로운 `String` 값을 생성하는 방법입니다. 문자열 삽입은 한 줄과 여러 줄 문자열 리터럴에서 사용할 수 있습니다. 문자열 리터럴에 추가하는 방법은 역슬래시(`\`)를 접두사로 소괄호를 감싸서 추가합니다:

```
let multiplier = 3
let message = "\(multiplier) times 2.5 is \(Double(multiplier) * 2.5)"
// message is "3 times 2.5 is 7.5"
```

위의 예시에서 `multiplier`의 값은 `\(multiplier)`로 문자열 리터럴 안에 삽입됩니다. 문자열 삽입이 실제 문자열이 생성될 때, `multiplier`의 실제 값으로 대체됩니다.

`multiplier`의 값은 문자열 내에서 표현식의 일부로 사용됩니다. 이 표현식은 `Double(multiplier) * 2.5`의 값을 계산하고 문자열에 결과 `7.5`를 삽입합니다. 이 경우 표현식은 문자열 리터럴에 포함될 때 `\(Double(multiplier) * 2.5)`로 작성합니다.

확장된 문자열 구분기호를 사용하여 문자열 삽입으로 사용할 문자를 포함하는 문자열을 생성할 수 있습니다. 예를 들어:

```
print(#"Write an interpolated string in Swift using \(multiplier)."#)
// Prints "Write an interpolated string in Swift using \(multiplier).".
```

확장된 구분기호를 사용하는 문자열에서 문자열 삽입을 사용하기 위해 문자열의 시작과 끝에 숫자 기호의 개수만큼 역슬래시 다음에 숫자 기호를 넣어주면 됩니다. 예를 들어:

```
print(#"6 times 7 is \#(6 * 7)."#)
// Prints "6 times 7 is 42.".
```

참고

소괄호 안에 작성한 표현식에 삽입된 문자열은 역슬래시(\), 캐리지 리턴, 개행을 포함할 수 없습니다. 그러나 다른 문자열 리터럴은 포함할 수 있습니다.

유니코드 (Unicode)

*유니코드(Unicode)*는 인코딩, 표기, 다른 쓰기 시스템에서의 텍스트 프로세싱을 위한 국제 표준입니다. 거의 모든 언어의 문자를 표준화 된 형식으로 표현하고, 텍스트 파일이나 웹 페이지와 같은 외부 소스에서 해당 문자를 읽고 쓸 수 있습니다. Swift의 `String`과 `Character` 타입은 이 섹션에서 설명한대로 유니코드를 완벽하게 지원합니다.

유니코드 스칼라 값 (Unicode Scalar Values)

Swift의 기본 `String` 타입은 *유니코드 스칼라 값(Unicode scalar values)*으로부터 생성됩니다. 유니코드 스칼라 값은 `LATIN SMALL LETTER A`(`"a"`)의 경우 `U+0061` 또는 `FRONT-FACING BABY CHICK`(`"🐥"`)의 경우 `U+1F425`와 같은 문자를 위한 유니크 한 21-bit 숫자나 수정자입니다.

모든 21-bit 유니코드 스칼라 값이 한 문자에 할당되는 것은 아닙니다 - 어떤 스칼라는 나중에 할당되거나 UTF-16 인코딩에 사용하기 위해 지정되어 있습니다. 문자에 할당된 스칼라 값은 일반적으로 위의 예에서 `LATIN SMALL LETTER A`와 `FRONT-FACING BABY CHICK`같이 이름을 가지고 있습니다.

확장된 그래프임 클러스터 (Extended Grapheme Clusters)

Swift의 `Character` 타입의 모든 인스턴스는 하나의 *확장된 그래프임 클러스터(extended grapheme cluster)*를 나타냅니다. 확장된 그래프임 클러스터는 하나 이상의 유니코드 스칼라 값의 시퀀스로, 이 값들이 결합되어 하나의 사람이 읽을 수 있는 문자를 생성합니다.

여기 예시가 있습니다. 문자 é은 단일 유니코드 스칼라 é (`LATIN SMALL LETTER E WITH ACUTE`나 `U+00E9`)로 표기 가능합니다. 그러나 동일한 문자를 스칼라 쌍으로 표시될 수도 있습니다 - 표준 문자 e(`LATIN SMALL LETTER E`나 `U+0065`) 뒤에 `COMBINING ACUTE ACCENT` 스칼라(`U+0301`)가 옵니다. `COMBINING ACUTE ACCENT` 스칼라는 앞에 있는 스칼라에 그래픽적으로 적용되어, 유니코드를 인식하는 텍스트 렌더링 시스템에서 e를 é로 변환합니다.

두 경우 모두 문자 é는 확장된 그래프임 클러스터를 나타내는 단일 Swift `Character` 값으로 표시됩니다. 첫 번째 경우, 클러스터에는 단일 스칼라가 포함됩니다; 두 번째 경우, 두 스칼라의 클러스터 입니다:

```
let eAcute: Character = "\u{E9}"                         // é
let combinedEAcute: Character = "\u{65}\u{301}"          // e followed by ´
// eAcute is é, combinedEAcute is é
```

확장된 그래프임 클러스터는 많은 복잡한 스크립트 문자를 하나의 `Character` 값으로 표기하는 방법입니다. 예를 들어 한글 음절은 단일 시퀀스 또는 분해된 시퀀스로 표시될 수 있습니다. 이 두 표현은 모두 Swift에서 단일 `Character` 값으로 간주됩니다:

```
let precomposed: Character = "\u{D55C}"                  // 한
let decomposed: Character = "\u{1112}\u{1161}\u{11AB}"   // ㅎ, ㅏ, ㄴ
// precomposed is 한, decomposed is 한
```

확장된 그래프임 클러스터는 동그라미(`COMBINING ENCLOSING CIRCLE` 또는 `U+20DD`)를 둘러싸는 스칼라를 사용하여 다른 유니코드 스칼라를 단일 문자값의 일부로 묶을 수 있습니다:

```
let enclosedEAcute: Character = "\u{E9}\u{20DD}"
// enclosedEAcute is é⃝
```

국가 표시 기호에 대한 유니코드 스칼라를 쌍으로 결합하여 단일 `Character` 값으로 만들 수 있습니다. 예를 들어 `REGIONAL INDICATOR SYMBOL LETTER U`(`U+1F1FA`)와 `REGIONAL INDICATOR SYMBOL LETTER S`(`U+1F1F8`)의 결합이 있습니다:

```
let regionalIndicatorForUS: Character = "\u{1F1FA}\u{1F1F8}"
// regionalIndicatorForUS is 🇺🇸
```

문자 수 (Counting Characters)

문자열에서 `Character` 값의 카운트를 구하려면 문자열에서 `count` 프로퍼티를 사용합니다:

```
let unusualMenagerie = "Koala 🐨, Snail 🐌, Penguin 🐧, Dromedary 🐪"
print("unusualMenagerie has \(unusualMenagerie.count) characters")
// Prints "unusualMenagerie has 40 characters".
```

Swift에서 `Character` 값에 확장된 그래프임 클러스터를 사용하는 것은 문자열을 연결하거나 수정할 때 문자열의 문자 수가 항상 변경되지 않을 수 있음을 의미합니다.

예를 들어 네 문자 단어인 `cafe`로 새로운 문자열을 초기화 하고, 문자열 끝에 `COMBINING ACUTE ACCENT`(`U+0301`)을 추가하면, 문자열은 네번째 문자가 `e`가 아닌 `e´`가 되고 여전히 문자 수는 `4`입니다:

```
var word = "cafe"
print("the number of characters in \(word) is \(word.count)")
// Prints "the number of characters in cafe is 4".

word += "\u{301}"    // COMBINING ACUTE ACCENT, U+0301

print("the number of characters in \(word) is \(word.count)")
// Prints "the number of characters in café is 4".
```

참고

확장된 그래프임 클러스터는 여러 유니코드 스칼라로 구성될 수 있습니다. 이것은 다른 문자와 - 같은 문자의 다른 표기법은 - 저장할 때 메모리 사용량이 다르게 요구될 수 있다는 의미입니다. 이 때문에 Swift의 문자는 문자열 내에서 각기 다른 메모리 사용량을 요구할 수 있습니다. 그 결과 문자열의 문자 수는 확장된 그래프임 클러스터 경계를 결정하기 위해 문자열을 반복하지 않고는 계산할 수 없습니다. 특히 긴 문자열 값으로 작업하는 경우에 해당 문자열의 문자를 결정하려면 `count` 프로퍼티가 전체 문자열의 유니코드 스칼라를 반복해야 합니다.

`count` 프로퍼티로 반환된 문자의 개수는 같은 문자여도 `NSString`의 `length` 프로퍼티와 항상 같지는 않습니다. `NSString`의 길이는 문자열 내에 유니코드 확장된 그래프임 클러스터 수가 아니라 문자열의 UTF-16 표현 내의 16-bit 코드 단위 수를 기반으로 합니다.

문자열 접근과 수정 (Accessing and Modifying a String)

메서드와 프로퍼티 또는 서브스크립트 구문으로 문자열에 접근, 수정할 수 있습니다.

문자열 인덱스 (String Indices)

각 `String` 값은 문자열에 각 `Character`의 위치에 해당하는 `String.Index` 인, *인덱스 타입(index type)*을 가지고 있습니다.

위에서 언급했듯이, 문자마다 저장할 메모리 양이 다를 수 있으므로, 특정 위치에 있는 `Character`를 확인하려면 해당 `String`의 시작이나 끝에서 각 유니코드 스칼라를 반복해야 합니다. 이러한 이유로 Swift 문자열은 정수값으로 인덱스를 생성할 수 없습니다.

`String`의 첫번째 `Character`에 접근하기 위해 `startIndex` 프로퍼티를 사용합니다. `endIndex` 프로퍼티는 `String`에 마지막 문자의 다음 위치입니다. 그 결과 `endIndex` 프로퍼티는 문자열의 서브스크립트에 유효한 인자가 아닙니다. `String`이 비어있다면, `startIndex`와 `endIndex`는 같습니다.

`String`의 메서드 `index(before:)`와 `index(after:)`를 사용하여 주어진 인덱스의 전과 후에 접근할 수 있습니다. 주어진 인덱스에서 먼 인덱스에 접근하려면 이러한 메서드를 여러번 호출하는 대신 `index(_:offsetBy:)` 메서드를 사용할 수 있습니다.

특정 `String` 인덱스의 `Character`에 접근하기 위해 서브스크립트 구문을 사용할 수 있습니다.

```
let greeting = "Guten Tag!"
greeting[greeting.startIndex]
// G
greeting[greeting.index(before: greeting.endIndex)]
// !
greeting[greeting.index(after: greeting.startIndex)]
// u
let index = greeting.index(greeting.startIndex, offsetBy: 7)
greeting[index]
// a
```

문자열 범위에 벗어나는 인덱스로 접근하거나 문자열 범위에 벗어나는 인덱스의 `Character`를 접근하려고 하면 런타임 오류가 발생합니다.

```
greeting[greeting.endIndex] // Error
greeting.index(after: greeting.endIndex) // Error
```

`indices` 프로퍼티를 사용하여 문자열에 있는 개별 문자의 모든 인덱스에 접근합니다.

```
for index in greeting.indices {
    print("\(greeting[index]) ", terminator: "")
}
// Prints "G u t e n   T a g ! ".
```

참고

`Collection` 프로토콜을 준수하는 모든 타입에서 `startIndex`와 `endIndex` 프로퍼티와 `index(before:)`, `index(after:)`, `index(_:offsetBy:)` 메서드를 사용할 수 있습니다. 이것은 여기서 봤듯이 `String` 뿐만 아니라 `Array`, `Dictionary`, `Set`과 같은 컬렉션 타입도 포함됩니다.

삽입과 삭제 (Inserting and Removing)

문자열에 특정 인덱스로 하나의 문자를 삽입하려면 `insert(_:at:)` 메서드를 사용하고, 다른 문자열의 콘텐츠를 특정 인덱스로 삽입하려면 `insert(contentsOf:at:)` 메서드를 사용합니다.

```
var welcome = "hello"
welcome.insert("!", at: welcome.endIndex)
// welcome now equals "hello!"

welcome.insert(contentsOf: " there", at: welcome.index(before: welcome.endIndex))
// welcome now equals "hello there!"
```

문자열에서 특정 인덱스에 있는 하나의 문자를 삭제하려면 `remove(at:)` 메서드를 사용하고, 특정 범위의 문자열을 삭제하려면 `removeSubrange(_:)` 메서드를 사용합니다:

```
welcome.remove(at: welcome.index(before: welcome.endIndex))
// welcome now equals "hello there"

let range = welcome.index(welcome.endIndex, offsetBy: -6)..<welcome.endIndex
welcome.removeSubrange(range)
// welcome now equals "hello"
```

참고

`RangeReplaceableCollection` 프로토콜을 준수하는 모든 타입은 `insert(_:at:)`, `insert(contentsOf:at:)`, `remove(at:)`, `removeSubrange(_:)` 메서드를 사용할 수 있습니다. 이것은 여기서 봤듯이 `String` 뿐만 아니라 `Array`, `Dictionary`, `Set`과 같은 컬렉션 타입도 포함됩니다.

부분 문자열 (Substrings)

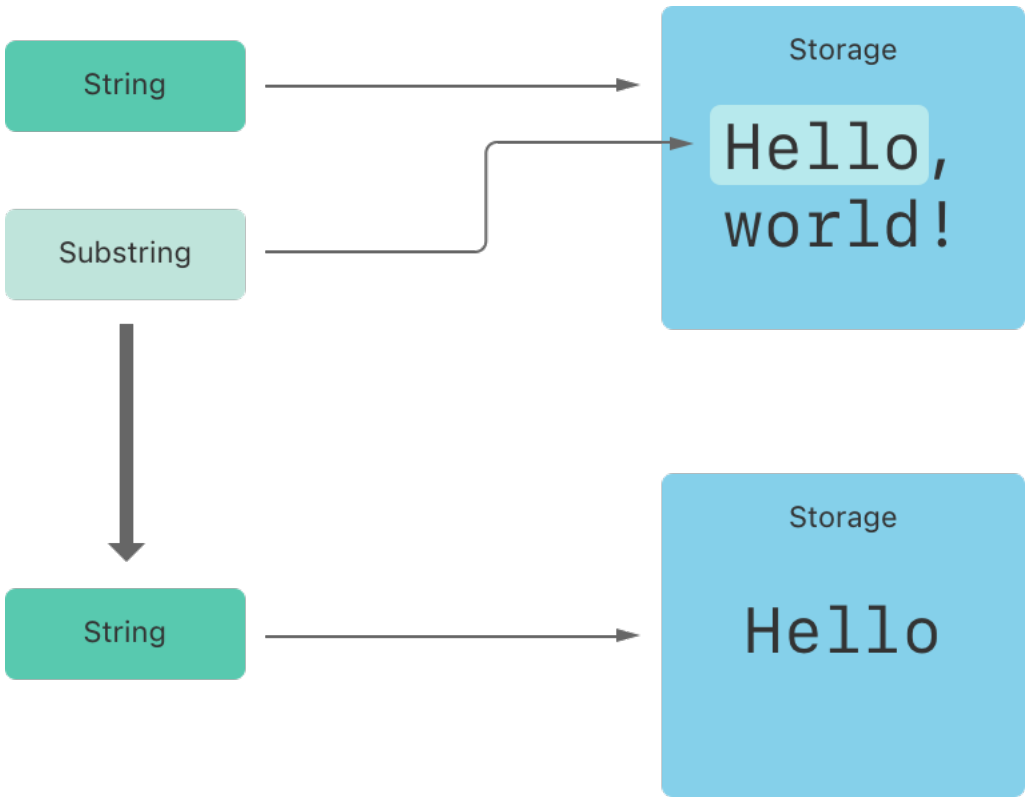
문자열에서 부분 문자열(substring)을 얻기위해 - 예를 들어 서브스크립트나 `prefix(_:)`와 같은 메서드를 사용하면 - 결과는 다른 문자열이 아닌 `Substring` 인스턴스 입니다. Swift의 부분 문자열은 문자열과 거의 동일한 메서드를 가지고 있기 때문에 문자열과 동일하게 부분 문자열을 사용할 수 있습니다. 그러나 문자열과 다르게, 문자열에 대한 작업을 수행하는 동안 짧은 시간 동안 부분 문자열을 사용합니다. 결과를 저장할 준비가 되었을 때, 부분 문자열을 `String` 인스턴스로 변환합니다. 예를 들어:

```
let greeting = "Hello, world!"
let index = greeting.firstIndex(of: ",") ?? greeting.endIndex
let beginning = greeting[..<index]
// beginning is "Hello"

// Convert the result to a String for long-term storage.
let newString = String(beginning)
```

문자열과 마찬가지로 각 부분 문자열에는 부분 문자열을 구성하는 문자가 저장되는 메모리 영역이 있습니다. 문자열과 부분 문자열의 차이점은 성능 최적화를 위해 부분 문자열이 원래 문자열을 저장하는데 사용된 메모리의 일부나 다른 부분 문자열을 저장하는데 사용되는 메모리의 일부를 재사용할 수 있다는 것입니다 (문자열은 비슷한 최적화를 갖지만, 두 문자열이 메모리를 공유하면 두 문자열은 같습니다). 이 성능 최적화는 문자열이나 부분 문자열을 수정할 때까지 메모리 복사에 대한 비용을 지불할 필요가 없음을 의미합니다. 위에서 언급했듯이 부분 문자열은 장기 저장에 적합하지 않습니다 - 이것은 원래 문자열의 저장소를 재사용하기 때문에 부분 문자열이 사용되는 한 전체 원본 문자열을 메모리에 보관해야 하기 때문입니다.

위의 예시에서 `greeting`은 문자열을 구성하는 문자가 저장되는 메모리 영역이 있는 문자열입니다. `beginning`은 `greeting`의 부분 문자열이기 때문에 `greeting`이 사용하는 메모리는 재사용됩니다. 반대로 `newString`은 문자열입니다 - 즉 이것은 부분 문자열에서 생성될 때 자신만의 저장소를 가집니다. 아래는 이러한 관계를 보여줍니다:



참고

`String`과 `Substring`은 모두 `StringProtocol`을 준수합니다. 이것은 문자열 조작 함수가 `StringProtocol` 값을 받아들이는 것이 편리한 경우가 많다는 것을 의미합니다. 이러한 함수는 `String`이나 `Substring` 값으로 호출될 수 있습니다.

문자열 비교 (Comparing Strings)

Swift는 텍스트 값을 비교하는 세 가지 방법이 있습니다; 문자열과 문자 같음, 접두사 같음, 접미사 같음이 있습니다.

문자열과 문자 동등성 (String and Character Equality)

비교 연산자 (Comparison Operators)에서 설명한대로 "같음" 연산자(==)와 "같지 않음" 연산자(!=)로 문자열과 문자가 같은지 확인합니다:

```
let quotation = "We're a lot alike, you and I."
let sameQuotation = "We're a lot alike, you and I."
if quotation == sameQuotation {
    print("These two strings are considered equal")
}
// Prints "These two strings are considered equal".
```

두 `String` 값(또는 두 `Character` 값)은 확장된 그래프임 클러스터가 정규적으로 동일한 경우 두 값이 같다고 합니다. 확장된 그래프임 클러스터는 다른 유니코드 스칼라로 구성되어 있더라도 언어적 의미와 모양이 같다면 정규적으로 같다고 합니다.

예를 들어 `LATIN SMALL LETTER E WITH ACUTE`(`U+00E9`)은 `LATIN SMALL LETTER E`(`U+0065`) 뒤에 `COMBINING ACUTE ACCENT`(`U+0301`)가 오는 것은 정규적으로 같습니다. 이러한 확장된 그래프임 클러스터는 문자 é을 표시하는 유효한 방법이므로 동일하다고 간주합니다:

```
// "Voulez-vous un café?" using LATIN SMALL LETTER E WITH ACUTE
let eAcuteQuestion = "Voulez-vous un caf\u{E9}?"

// "Voulez-vous un café?" using LATIN SMALL LETTER E and COMBINING ACUTE ACCENT
let combinedEAcuteQuestion = "Voulez-vous un caf\u{65}\u{301}?"

if eAcuteQuestion == combinedEAcuteQuestion {
    print("These two strings are considered equal")
}
// Prints "These two strings are considered equal".
```

반대로 영어로 사용된 `LATIN CAPITAL LETTER A`(`U+0041` 또는 `"A"`)와 러시아어로 사용된 `CYRILLIC CAPITAL LETTER A`(`U+0410` 또는 `"А"`)은 같지 않습니다. 이 문자는 모양은 같지만, 언어적 의미가 같지 않습니다:

```
let latinCapitalLetterA: Character = "\u{41}"

let cyrillicCapitalLetterA: Character = "\u{0410}"

if latinCapitalLetterA != cyrillicCapitalLetterA {
    print("These two characters are not equivalent.")
}
// Prints "These two characters are not equivalent.".
```

참고

Swift의 문자열과 문자 비교는 지역을 구분하지 않습니다.

접두사와 접미사 동등성 (Prefix and Suffix Equality)

문자열이 특정 문자열 접두사나 접미사를 가지고 있는지 확인하기 위해 문자열의 `hasPrefix(_:)`와 `hasSuffix(_:)` 메서드를 호출하면 됩니다. 두 메서드는 하나의 `String` 타입 인자를 받고 Boolean 값을 반환합니다.

아래 예시에서는 셰익스피어의 로미오와 줄리엣에 처음 두막의 장면을 나타내는 문자열 배열을 나타냅니다:

```
let romeoAndJuliet = [
    "Act 1 Scene 1: Verona, A public place",
    "Act 1 Scene 2: Capulet's mansion",
    "Act 1 Scene 3: A room in Capulet's mansion",
    "Act 1 Scene 4: A street outside Capulet's mansion",
    "Act 1 Scene 5: The Great Hall in Capulet's mansion",
    "Act 2 Scene 1: Outside Capulet's mansion",
    "Act 2 Scene 2: Capulet's orchard",
    "Act 2 Scene 3: Outside Friar Lawrence's cell",
    "Act 2 Scene 4: A street in Verona",
    "Act 2 Scene 5: Capulet's mansion",
    "Act 2 Scene 6: Friar Lawrence's cell"
]
```

`hasPrefix(_:)` 메서드를 이용하여 `romeoAndJuliet` 배열의 연극 Act 1의 장면의 수를 알 수 있습니다:

```
var act1SceneCount = 0
for scene in romeoAndJuliet {
    if scene.hasPrefix("Act 1 ") {
        act1SceneCount += 1
    }
}
print("There are \(act1SceneCount) scenes in Act 1")
// Prints "There are 5 scenes in Act 1".
```

마찬가지로 `hasSuffix(_:)` 메서드를 사용하여 Capulet's mansion과 Friar Lawrence's cell의 장면의 수를 구할 수 있습니다:

```
var mansionCount = 0
var cellCount = 0
for scene in romeoAndJuliet {
    if scene.hasSuffix("Capulet's mansion") {
        mansionCount += 1
    } else if scene.hasSuffix("Friar Lawrence's cell") {
        cellCount += 1
    }
}
print("\(mansionCount) mansion scenes; \(cellCount) cell scenes")
// Prints "6 mansion scenes; 2 cell scenes".
```

참고

`hasPrefix(_:)`와 `hasSuffix(_:)` 메서드는 문자열과 문자 동등성 (String and Character Equality)에서 설명했듯이 각 문자열에 확장된 그래프임 클러스터 간의 문자 단위로 동등한지 수행합니다.

문자열의 유니코드 표현 (Unicode Representations of Strings)

유니코드 문자열이 텍스트 파일이나 다른 저장소에 쓰여질 때, 해당 문자열의 유니코드 스칼라는 유니코드에서 정의된 인코딩 형식 중 하나로 인코딩됩니다. 각 형식은 문자열을 코드 단위라고 불리는 작은 조각들로 인코딩합니다. 이것은 UTF-8 인코딩 형식(8-bit 코드 단위로 문자열을 인코딩), UTF-16 인코딩 형식(16-bit 코드 단위로 문자열을 인코딩), UTF-32 인코딩 형식(32-bit 코드 단위로 문자열을 인코딩)을 포함합니다.

Swift는 문자열의 유니코드 표현에 접근하는 여러 가지 방법을 제공합니다. `for-in` 구문으로 문자열을 반복하여 유니코드 확장된 그래프임 클러스터인 `Character` 값에 접근할 수 있습니다. 문자 작업 (Working with Characters)에 자세히 설명되어 있습니다.

세 가지 다른 유니코드 호환 표현 중 하나로 `String` 값에 접근할 수 있습니다:

- UTF-8 코드 단위(문자열의 `utf8` 프로퍼티로 접근)
- UTF-16 코드 단위(문자열의 `utf16` 프로퍼티로 접근)
- 문자열의 UTF-32 인코딩 형식에 해당하는 21-bit 유니코드 스칼라 값 (문자열의 `unicodeScalars` 프로퍼티로 접근)

아래의 각 예시는 `D`, `o`, `g`, `!!` 문자(`DOUBLE EXCLAMATION MARK`, 또는 유니코드 스칼라 `U+203C`)와 🐶 문자(`DOG FACE`, 또는 유니코드 스칼라 `U+1F436`)로 구성된 문자열에 다른 표현을 보여줍니다:

```
let dogString = "Dog!!🐶"
```

UTF-8 표현 (UTF-8 Representation)

`utf8` 프로퍼티 반복을 통해 `String`의 UTF-8 표현에 접근할 수 있습니다. 이 프로퍼티는 문자열의 UTF-8 표현에서 각 바이트에 대해 하나씩 부호가 없는 8-bit(`UInt8`) 값의 모음인 `String.UTF8View` 타입 입니다:

Character	D U+0044	o U+006F	g U+0067	!! U+203C			🐶 U+1F436			
UTF-8 Code Unit	68	111	103	226	128	188	240	159	144	182
Position	0	1	2	3	4	5	6	7	8	9

```
for codeUnit in dogString.utf8 {
    print("\(codeUnit) ", terminator: "")
}
print("")
// Prints "68 111 103 226 128 188 240 159 144 182 ".
```

위 예시에서 처음 세 개의 `codeUnit` 값 (`68`, `111`, `103`)은 ASCII와 같은 UTF-8 표현인 문자 `D`, `o`, `g`를 나타냅니다. 다음 세 개의 `codeUnit` 값 (`226`, `128`, `188`)은 `DOUBLE EXCLAMATION MARK` 문자의 3-바이트 UTF-8 표현입니다. 마지막 네 개의 `codeUnit` 값(`240`, `159`, `144`, `182`)은 `DOG FACE` 문자의 4-바이트 UTF-8 표현입니다.

UTF-16 표현 (UTF-16 Representation)

`utf16` 프로퍼티 반복을 통해 `String`의 UTF-16 표현에 접근할 수 있습니다. 이 프로퍼티는 문자열의 UTF-16 표현에서 각 16-bit 코드 단위에 대해 하나씩 부호가 없는 16-bit(`UInt16`) 값의 모음인 `String.UTF16View` 타입 입니다:

Character	D U+0044	o U+006F	g U+0067	!! U+203C	🐶 U+1F436	
UTF-16 Code Unit	68	111	103	8252	55357	56374
Position	0	1	2	3	4	5

```
for codeUnit in dogString.utf16 {
    print("\(codeUnit) ", terminator: "")
}
print("")
// Prints "68 111 103 8252 55357 56374 ".
```

다시 처음 세 개의 `codeUnit` 값 (`68`, `111`, `103`)은 UTF-16 코드 단위는 문자열의 UTF-8 표현과 같은 값을 가지므로 문자 `D`, `o`, `g`입니다 (유니코드 스칼라는 ASCII 문자를 표현하기 때문).

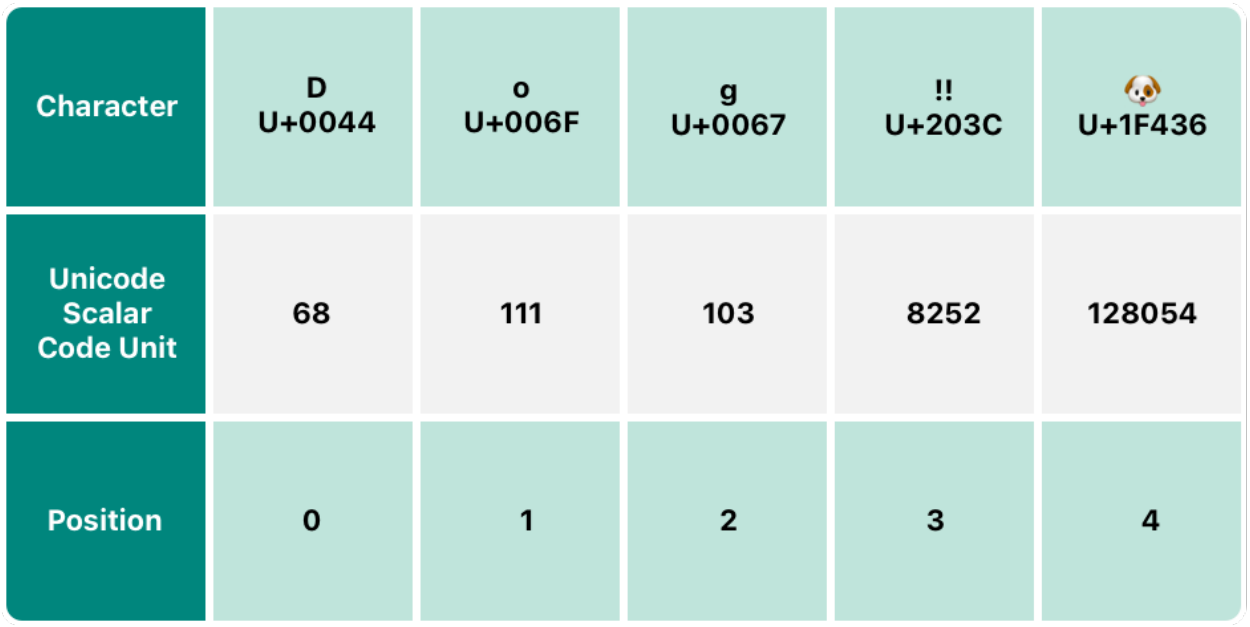
네 번째 `codeUnit` 값(`8252`)은 `DOUBLE EXCLAMATION MARK` 문자에 해당하는 유니코드 스칼라 `U+203C`에서 16진법 `203C` 값을 나타내는 10진법 수 입니다. 이 문자는 UTF-16에서 하나의 코드 단위로 표현할 수 있습니다.

5번째와 6번째 `codeUnit` 값(`55357`과 `56374`)은 `DOG FACE` 문자의 UTF-16 대리 쌍 표현입니다. 이 값은 높은 대리 값 `U+D83D`(10진법 값 `55357`)와 낮은 대리 값 `U+DC36`(10진법 값 `56374`)입니다.

유니코드 스칼라 표현 (Unicode Scalar Representation)

`unicodeScalars` 프로퍼티 반복을 통해 `String` 값의 유니코드 스칼라 표현에 접근할 수 있습니다. 이 프로퍼티는 `UnicodeScalar` 타입의 값의 모음인 `UnicodeScalarView` 타입입니다.

각 `UnicodeScalar`는 스칼라의 21-bit 값을 반환하는 `value` 프로퍼티를 가지며, `UInt32` 값으로 표현됩니다.




```
for scalar in dogString.unicodeScalars {
    print("\(scalar.value) ", terminator: "")
}
print("")
// Prints "68 111 103 8252 128054 ".
```

`value` 프로퍼티를 통해 얻은 처음 세 개의 `UnicodeScalar` 값 (`68`, `111`, `103`)은 문자 `D`, `o`, `g`를 표현합니다.

네 번째 `codeUnit` 값(`8252`)은 `DOUBLE EXCLAMATION MARK` 문자에 해당하는 유니코드 스칼라 `U+203C`에서 16진법 `203C`와 동일한 10진법 수 입니다.

다섯 번째이자 마지막 `UnicodeScalar`의 `value` 프로퍼티인 `128054`는 `DOG FACE` 문자에 해당하는 유니코드 스칼라 `U+1F436`에서 16진법 `1F436`의 10진법 수 입니다.

`value` 프로퍼티 대신에 각 `UnicodeScalar` 값은 문자열 삽입을 사용하는 것과 같이 새로운 `String` 값을 생성하는데 사용할 수 있습니다:

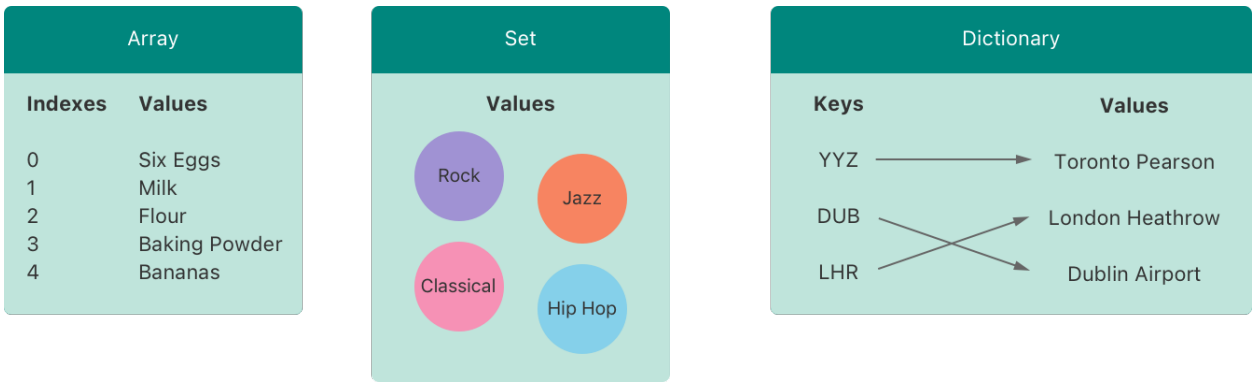
```
for scalar in dogString.unicodeScalars {
    print("\(scalar) ")
}
// D
// o
// g
// !!
// 🐶
```

언어 가이드

컬렉션 타입 (Collection Types)

배열, 집합, 딕셔너리를 사용하여 데이터를 구성합니다.

Swift는 컬렉션의 값을 저장하기 위한 배열(array), 집합(set), 딕셔너리(dictionary)와 같은 세 가지 *컬렉션 타입(collection types)*을 제공합니다. 배열은 컬렉션 값에 순서를 가지고 있습니다. 집합은 반복되지 않는 값에 순서가 없는 컬렉션 타입입니다. 딕셔너리는 키-값 쌍의 순서가 없는 컬렉션 타입입니다.



Swift의 배열, 집합, 딕셔너리는 저장할 수 있는 값의 타입과 키의 타입에 대해 명확합니다. 이것은 실수로 컬렉션에 잘못된 타입을 추가할 수 없다는 의미입니다. 또한 컬렉션에서 값을 조회할 때 타입이 명확하다는 것을 의미합니다.

> **참고**
>
> Swift의 배열, 집합, 딕셔너리 타입은 *제네릭 컬렉션(generic collections)*으로 구현됩니다. 제네릭 타입과 컬렉션에 대한 자세한 내용은 제네릭 (Generics)을 참고바랍니다.

컬렉션의 가변성 (Mutability of Collections)

배열, 집합, 딕셔너리를 생성하고 변수에 할당하면, 생성된 컬렉션은 *변경 가능(mutable)*합니다. 이것은 컬렉션이 생성된 후에 컬렉션의 아이템을 추가, 삭제, 변경할 수 있다는 뜻입니다. 배열, 집합, 딕셔너리를 상수에 할당하면 이 컬렉션은 불가변성이며, 크기와 콘텐츠를 변경할 수 없습니다.

> **참고**
>
> 컬렉션을 변경할 필요가 없는 경우에 변경 불가능한 컬렉션을 만드는 것이 좋습니다. 이렇게 하면 코드에 대해 더 쉽게 추론할 수 있고, Swift 컴파일러가 생성한 컬렉션의 성능을 최적화 할 수 있습니다.

배열 (Arrays)

*배열(array)*은 순서대로 같은 타입의 값을 저장합니다. 같은 값은 배열에 다른 순서로 존재할 수 있습니다.

> **참고**
>
> Swift의 `Array` 타입은 Foundation의 `NSArray` 클래스와 연결됩니다.

Foundation과 Cocoa의 `Array` 사용에 대한 자세한 내용은 Array와 NSArray 사이의 연결 (Bridging Between Array and NSArray)을 참고바랍니다.

배열 타입 축약 구문 (Array Type Shorthand Syntax)

Swift 배열의 타입은 `Array<Element>`로 작성되고, `Element`는 저장할 배열 값의 타입을 나타냅니다. 배열의 타입을 `[Element]`와 같은 축약형으로도 작성할 수 있습니다. 두 형식이 기능적으로 동일하지만, 축약형이 선호되며 이 가이드에서는 배열의 타입을 언급할 때 축약형을 사용합니다.

빈 배열 생성 (Creating an Empty Array)

Swift에서 두 가지 접근방식으로 빈 배열을 생성할 수 있습니다. 컨텍스트에서 함수 인자나 타입 변수나 상수와 같이 타입 정보를 이미 제공하고 있다면, `[]`로 작성하는 빈 배열 리터럴을 사용할 수 있습니다 (빈 대괄호 쌍):

```
var someInts: [Int] = []
print("someInts is of type [Int] with \(someInts.count) items.")
// Prints "someInts is of type [Int] with 0 items.".
```

또한 명시적 이니셜라이저 문법을 사용하여 특정 타입의 빈 배열을 생성할 수 있으며, 이것은 대괄호 안에 요소 타입을 작성하고 다음에 소괄호를 작성합니다 - 예를 들어 다음의 `[Int]()`입니다:

```
var someInts = [Int]()
print("someInts is of type [Int] with \(someInts.count) items.")
// Prints "someInts is of type [Int] with 0 items.".
```

두 접근방식 모두 동일한 결과를 생성합니다. 그러나 빈 배열 리터럴이 더 짧고 읽기 쉽습니다.

두 경우 모두 빈 배열 리터럴(`[]`)을 사용하여 기존 변수에 빈 배열을 다시 할당할 수 있습니다:

```
someInts.append(3)
// someInts now contains 1 value of type Int
someInts = []
// someInts is now an empty array, but is still of type [Int]
```

기본값 배열 생성 (Creating an Array with a Default Value)

Swift의 `Array` 타입은 같은 기본값으로 설정하고 크기를 고정하여 배열을 생성하는 초기화도 제공합니다. 적절한 타입(매개변수 명 `repeating`)의 기본값과 새로운 배열에 반복될 값의 횟수(매개변수 명 `count`)를 초기화에 전달합니다:

```
var threeDoubles = Array(repeating: 0.0, count: 3)
// threeDoubles is of type [Double], and equals [0.0, 0.0, 0.0]
```

배열을 더해 생성 (Creating an Array by Adding Two Arrays Together)

동등한 타입의 두 배열을 덧셈 연산자(+)를 통해 합쳐서 새로운 배열을 생성할 수 있습니다. 새로운 배열의 타입은 합쳐진 두 배열의 타입으로 추론됩니다:

```
var anotherThreeDoubles = Array(repeating: 2.5, count: 3)
// anotherThreeDoubles is of type [Double], and equals [2.5, 2.5, 2.5]

var sixDoubles = threeDoubles + anotherThreeDoubles
// sixDoubles is inferred as [Double], and equals [0.0, 0.0, 0.0, 2.5, 2.5, 2.5]
```

배열 리터럴로 생성 (Creating an Array with an Array Literal)

배열 컬렉션으로 하나 이상의 값을 *배열 리터럴(array literal)*에 작성하여 배열을 생성할 수 있습니다. 배열 리터럴은 값을 리스트로 작성하고 콤마로 구분하며, 대괄호로 둘러싸서 작성합니다:

```
[<#value 1#>, <#value 2#>, <#value 3#>]
```

아래 예시는 `String` 값을 저장하는 `shoppingList` 배열을 생성하는 예시입니다:

```
var shoppingList: [String] = ["Eggs", "Milk"]
// shoppingList has been initialized with two initial items
```

`shoppingList` 변수는 `[String]`으로 쓰고 "문자열 값의 배열"로 선언됩니다. 이 배열은 `String` 타입을 가지고 있기 때문에 `String` 값만 저장이 가능합니다. 여기서 `shoppingList` 배열은 배열 리터럴 안에 쓰여진 두 `String` 값 (`"Eggs"`와 `"Milk"`)으로 초기화됩니다.

> **참고**
>
> 아래 예시에서 쇼핑 리스트에 더 많은 아이템이 추가되기 때문에 `shoppingList` 배열은 상수(`let`으로 선언)가 아닌 변수(`var`로 선언)로 선언되었습니다.

이 경우 배열 리터럴은 두 `String` 값만 포함합니다. 이것은 `shoppingList` 변수의 선언 타입 (`String` 값만 포함될 수 있는 배열)과 일치하므로, 배열 리터럴을 할당하여 `shoppingList`를 두 개의 초기 항목으로 초기화하는 것이 허용됩니다.

Swift의 타입 추론에 덕분에 같은 타입의 값을 포함하는 배열 리터럴로 초기화하면 배열의 타입을 명시할 필요가 없습니다. `shoppingList`의 초기화는 아래와 같이 간단하게 작성될 수 있습니다:

```
var shoppingList = ["Eggs", "Milk"]
```

배열 리터럴 안에 모든 값은 같은 타입이기 때문에, Swift는 `[String]`가 `shoppingList` 변수를 사용하는데 올바른 타입이라는 것을 유추할 수 있습니다.

배열 리터럴의 마지막 값 뒤에 콤파를 포함할 수 있으며, 이것을 *후행 콤마(trailing comma)*라고 합니다:

```
var shoppingList = [
    "Eggs",
    "Milk",
]
```

후행 콤마는 모든 줄을 동일한 형태로 끝나게 만들어주기 때문에 위와 같이 한 줄에 하나의 값을 가지는 배열을 작성할 때 유용합니다. 값을 추가, 제거, 순서를 바꾸는 것과 같이 배열을 변경할 때, 콤마를 추가하거나 제거하지 않아도 됩니다.

배열 접근과 수정 (Accessing and Modifying an Array)

메서드와 프로퍼티 또는 서브스크립트 구문을 사용하여 배열에 접근 및 수정이 가능합니다.

배열에 아이템 수를 알려면 읽기 전용 `count` 프로퍼티로 확인하면 됩니다:

```
print("The shopping list contains \(shoppingList.count) items.")
// Prints "The shopping list contains 2 items.".
```

Boolean `isEmpty` 프로퍼티를 사용하여 배열의 `count` 프로퍼티 값이 `0`인지 아닌지 빠르게 판단할 수 있습니다:

```
if shoppingList.isEmpty {
    print("The shopping list is empty.")
} else {
    print("The shopping list is not empty.")
}
// Prints "The shopping list is not empty.".
```

배열의 `append(_:)` 메서드를 호출하여 배열의 끝에 새로운 아이템을 추가할 수 있습니다:

```
shoppingList.append("Flour")
// shoppingList now contains 3 items, and someone is making pancakes
```

하나 이상의 아이템의 배열은 덧셈 할당 연산자(+=)를 통해서도 추가할 수 있습니다:

```
shoppingList += ["Baking Powder"]
// shoppingList now contains 4 items
shoppingList += ["Chocolate Spread", "Cheese", "Butter"]
// shoppingList now contains 7 items
```

*서브스크립트 구문(subscript syntax)*을 사용하여 배열의 값을 가져올 수 있습니다. 배열의 이름 뒤에 바로 대괄호를 붙이고 가져올 값의 인덱스를 넣어 해당 값을 가져올 수 있습니다:

```
var firstItem = shoppingList[0]
// firstItem is equal to "Eggs"
```

> **참고**
>
> 배열 내의 첫 번째 아이템은 인덱스가 `1`이 아닌 `0`입니다. Swift에 배열은 항상 제로-인덱스(zero-indexed)입니다.

서브스크립트 구문을 사용하여 존재하는 값을 변경할 수 있습니다:

```
shoppingList[0] = "Six eggs"
// the first item in the list is now equal to "Six eggs" rather than "Eggs"
```

서브스크립트 구문을 사용할 때, 유효한 인덱스를 지정해야 합니다. 예를 들어 `shoppingList[shoppingList.count] = "Salt"`으로 배열 끝에 추가하면 런타임 오류가 발생합니다.

변경할 값들이 변경할 범위와 다른 길이를 가지고 있더라도 서브스크립트 구문으로 범위 안에 값을 한 번에 변경할 수 있습니다. 아래 예시는 `"Chocolate Spread"`, `"Cheese"`, `"Butter"`를 `"Bananas"`와 `"Apples"`로 대체합니다:

```
shoppingList[4...6] = ["Bananas", "Apples"]
// shoppingList now contains 6 items
```

배열에 특정 인덱스에 아이템을 추가하려면, 배열의 `insert(_:at:)` 메서드를 사용합니다:

```
shoppingList.insert("Maple Syrup", at: 0)
// shoppingList now contains 7 items
// "Maple Syrup" is now the first item in the list
```

`insert(_:at:)` 메서드를 호출하면 인덱스가 `0`인 쇼핑 리스트에 가장 맨 앞에 `"Maple Syrup"`의 값을 가진 새로운 아이템을 추가합니다.

비슷하게 `remove(at:)` 메서드를 통해 배열의 아이템을 삭제할 수 있습니다. 이 메서드는 해당 인덱스의 아이템을 삭제하고 삭제한 아이템을 반환합니다 (반환된 값이 필요 없으면 무시해도 됩니다):

```
let mapleSyrup = shoppingList.remove(at: 0)
// the item that was at index 0 has just been removed
// shoppingList now contains 6 items, and no Maple Syrup
// the mapleSyrup constant is now equal to the removed "Maple Syrup" string
```

참고

배열의 범위를 넘는 인덱스로 값에 접근이나 수정할 경우, 런타임 오류가 발생합니다. 배열의 `count` 프로퍼티를 사용하여 인덱스가 유효한지 확인할 수 있습니다. 배열은 0부터 인덱스되기 때문에 배열에서 가장 큰 유효한 인덱스는 `count - 1`입니다 - 그러나 `count`가 `0`이면(배열이 비어있음), 유효한 인덱스가 없다는 의미입니다.

아이템이 삭제되면 배열의 공간도 삭제되므로, 인덱스 `0`인 값은 `"Six eggs"`와 같습니다:

```
firstItem = shoppingList[0]
// firstItem is now equal to "Six eggs"
```

배열의 마지막 아이템을 삭제하고 싶다면, 배열의 `count` 프로퍼티의 사용을 피하기 위해 `remove(at:)` 메서드 보다 `removeLast()` 메서드를 사용하세요. `remove(at:)` 메서드와 같이 `removeLast()` 메서드는 삭제된 아이템을 반환합니다:

```
let apples = shoppingList.removeLast()
// the last item in the array has just been removed
// shoppingList now contains 5 items, and no apples
// the apples constant is now equal to the removed "Apples" string
```

배열 반복 (Iterating Over an Array)

`for-in` 루프를 사용하여 배열의 전체 값을 조회할 수 있습니다:

```
for item in shoppingList {
    print(item)
}
// Six eggs
// Milk
// Flour
// Baking Powder
// Bananas
```

각 아이템의 인덱스 뿐만 아니라 값도 필요하다면, 배열을 반복할 때 `enumerated()` 메서드를 사용합니다. 배열의 각 아이템에 대해 `enumerated()` 메서드는 정수와 아이템을 조합하여 튜플을 반환합니다. 정수는 0부터 시작하여 각 아이템에 대해 1씩 증가합니다; 전체 배열을 열거하는 경우 이 정수는 아이템의 인덱스와 일치합니다. 튜플을 임시의 상수나 변수로 분리할 수 있습니다:

```
for (index, value) in shoppingList.enumerated() {
    print("Item \(index + 1): \(value)")
}
// Item 1: Six eggs
// Item 2: Milk
// Item 3: Flour
// Item 4: Baking Powder
// Item 5: Bananas
```

`for-in` 루프에 대한 자세한 내용은 For-In 루프 (For-In Loops)를 참고바랍니다.

집합 (Sets)

*집합(set)*은 컬렉션에 순서와 상관없이 같은 타입의 다른 값을 저장합니다. 아이템의 순서가 중요하지 않거나 아이템이 반복되면 안될 때 배열 대신에 집합을 사용할 수 있습니다.

> **참고**
>
> Swift의 `Set` 타입은 Foundation의 `NSSet` 클래스와 연결됩니다.
>
> Foundation과 Cocoa의 `Set`에 대해 더 자세한 내용은 Set과 NSSet 사이의 연결 (Bridging Between Set and NSSet)을 참고바랍니다.

집합 타입을 위한 해시값 (Hash Values for Set Types)

집합(set)에 저장하기 위해 타입은 반드시 hashable해야 합니다 - 즉, *해시값(hash value)*을 계산할 수 있는 방법을 타입은 제공해야 합니다. 해시값은 `Int` 값으로 동일하게 비교되는 모든 객체에 대해 동일하며, `a == b`이면, `a`의 해시값은 `b`의 해시값과 같아야 합니다.

Swift의 모든 기본 타입(`String`, `Int`, `Double`, `Bool`)은 기본적으로 hashable이고, 집합의 값 타입이나 딕셔너리의 키 타입으로 사용할 수 있습니다. 연관값이 없는 열거형(Enumeration) 케이스 값 (열거형 (Enumerations)에서 설명한대로)은 기본적으로 hashable입니다.

> **참고**
>
> 집합 값 타입이나 딕셔너리 키 타입으로 사용자화 타입을 사용하려면 Swift 표준 라이브러리의 `Hashable` 프로토콜을 준수해야 사용할 수 있습니다. 필수인 `hash(into:)` 메서드를 구현하는 것에 대한 자세한 내용은 `Hashable`을 참고바랍니다. 프로토콜 준수에 대한 자세한 내용은 프로토콜 (Protocols)을 참고바랍니다.

집합 타입 구문 (Set Type Syntax)

Swift 집합의 타입은 `Element`가 저장할 값의 타입을 나타내는 `Set<Element>`로 작성합니다. 배열과 다르게, 집합은 축약형 형식이 없습니다.

빈 집합 생성과 초기화 (Creating and Initializing an Empty Set)

이니셜라이저를 사용하여 타입을 포함한 빈 집합을 생성할 수 있습니다:

```
var letters = Set<Character>()
print("letters is of type Set<Character> with \(letters.count) items.")
// Prints "letters is of type Set<Character> with 0 items.".
```

> **참고**
>
> `letters` 변수의 타입은 이니셜라이저를 통해 `Set<Character>`로 추론됩니다.

또한 함수 인자나 타입이 명시된 변수나 상수로 타입 정보를 이미 제공한다면 빈 배열 리터럴을 사용하여 빈 집합을 생성할 수 있습니다:

```
letters.insert("a")
// letters now contains 1 value of type Character
letters = []
// letters is now an empty set, but is still of type Set<Character>
```

배열 리터럴로 집합 생성 (Creating a Set with an Array Literal)

집합 컬렉션으로 하나 이상의 값으로 축약형 방법과 같이 배열 리터럴을 사용하여 집합을 초기화 할 수도 있습니다.

아래 예시는 `String` 값을 저장하는 `favoriteGenres`라는 집합을 생성합니다:

```
var favoriteGenres: Set<String> = ["Rock", "Classical", "Hip hop"]
// favoriteGenres has been initialized with three initial items
```

`favoriteGenres` 변수는 `Set<String>`라고 쓰고 "`String` 값의 집합"으로 선언됩니다. 이 집합은 특정 `String` 타입의 값을 가지고 있기 때문에 `String` 값만 저장할 수 있습니다. 여기서 `favoriteGenres` 집합은 배열 리터럴 내에 적힌 세 개의 `String` 값 (`"Rock"`, `"Classical"`, `"Hip hop"`)으로 초기화 됩니다.

> **참고**
>
> `favoriteGenres` 집합은 아래 예시에서 아이템을 추가하거나 삭제하기 때문에 상수(`let`)가 아닌 변수(`var`)로 선언되었습니다.

집합 타입은 배열 리터럴에서 추론할 수 없기 때문에 `Set` 타입은 명시적으로 선언해야 합니다. 그러나 Swift의 타입 추론 때문에 배열 리터럴로 초기화할 때 타입의 값이 포함되어 있는 경우 집합 요소의 타입을 명시하지 않아도 됩니다. `favoriteGenres` 초기화는 짧게 쓸 수 있습니다:

```
var favoriteGenres: Set = ["Rock", "Classical", "Hip hop"]
```

배열 리터럴 안에 값은 같은 타입이기 때문에, Swift는 `favoriteGenres` 변수를 사용하는데 `Set<String>`이라고 유추할 수 있습니다.

집합 접근과 수정 (Accessing and Modifying a Set)

메서드와 프로퍼티로 집합에 접근 및 수정할 수 있습니다.

읽기 전용 `count` 프로퍼티로 집합의 아이템 수를 알 수 있습니다:

```
print("I have \(favoriteGenres.count) favorite music genres.")
// Prints "I have 3 favorite music genres.".
```

`count` 프로퍼티가 `0`과 같은지, Boolean `isEmpty` 프로퍼티를 사용해 확인할 수 있습니다:

```
if favoriteGenres.isEmpty {
    print("As far as music goes, I'm not picky.")
} else {
    print("I have particular music preferences.")
}
// Prints "I have particular music preferences.".
```

집합의 `insert(_:)` 메서드를 호출하여 집합에 새로운 아이템을 추가할 수 있습니다:

```
favoriteGenres.insert("Jazz")
// favoriteGenres now contains 4 items
```

집합의 `remove(_:)` 메서드를 호출하여 집합의 아이템을 삭제할 수 있습니다. `remove(_:)` 메서드는 집합에 아이템이 있을 경우 삭제하고, 삭제된 값을 반환하거나 해당 아이템이 없을 경우 `nil`을 반환합니다. 또한 `removeAll()` 메서드를 사용하여 전체 아이템을 삭제할 수 있습니다.

```
if let removedGenre = favoriteGenres.remove("Rock") {
    print("\(removedGenre)? I'm over it.")
} else {
    print("I never much cared for that.")
}
// Prints "Rock? I'm over it.".
```

`contains(_:)` 메서드를 사용하여 집합에 특정 아이템이 포함되어 있는지 알 수 있습니다.

```
if favoriteGenres.contains("Funk") {
    print("I get up on the good foot.")
} else {
    print("It's too funky in here.")
}
// Prints "It's too funky in here.".
```

집합 반복 (Iterating Over a Set)

`for-in` 루프를 사용하여 집합에 값을 조회할 수 있습니다.

```
for genre in favoriteGenres {
    print("\(genre)")
}
// Classical
// Jazz
// Hip hop
```

`for-in` 루프에 대해 자세한 내용은 For-In 루프 (For-In Loops)를 참고바랍니다.

Swift의 `Set` 타입은 정의된 순서를 가지고 있지 않습니다. 특정 순서로 집합의 값을 반복하려면, 집합의 요소를 `<` 연산자를 사용하여 정렬된 배열로 집합의 요소를 반환하는 `sorted()` 메서드를 사용합니다.

```
for genre in favoriteGenres.sorted() {
    print("\(genre)")
}
// Classical
// Hip hop
// Jazz
```

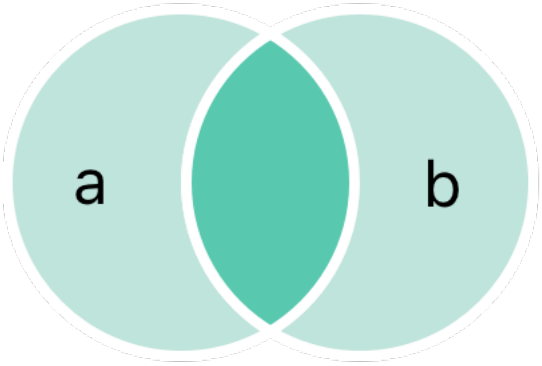
집합 연산 수행 (Performing Set Operations)

두 집합을 합치거나, 두 집합의 공통 값을 구하거나, 두 집합이 모두 같은 값을 갖고 있거나 한 쪽에만 존재하거나 아예 없는 경우와 같은 기본적인 집합 연산을 효율적으로 수행할 수 있습니다.

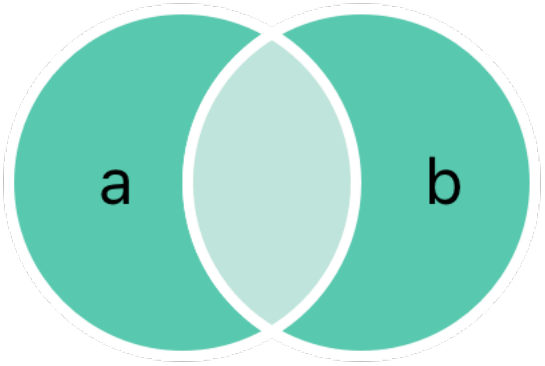
기본 집합 연산 (Fundamental Set Operations)

아래 그림은 두 집합 - `a`와 `b`의 여러 가지 집합 연산에 대한 결과를 나타냅니다.

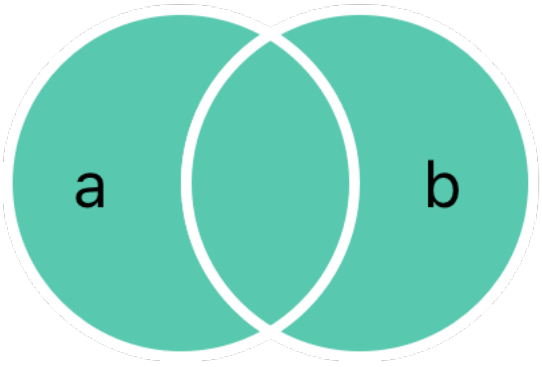
a.intersection(b)



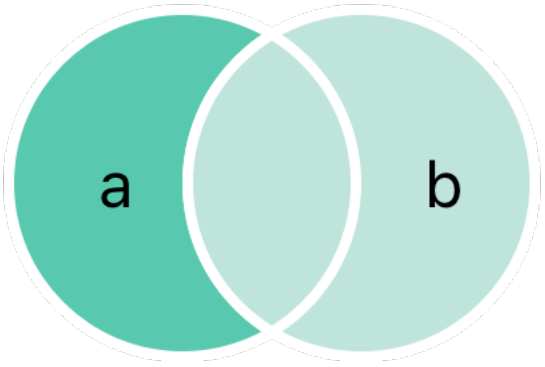
a.symmetricDifference(b)



a.union(b)



a.subtracting(b)



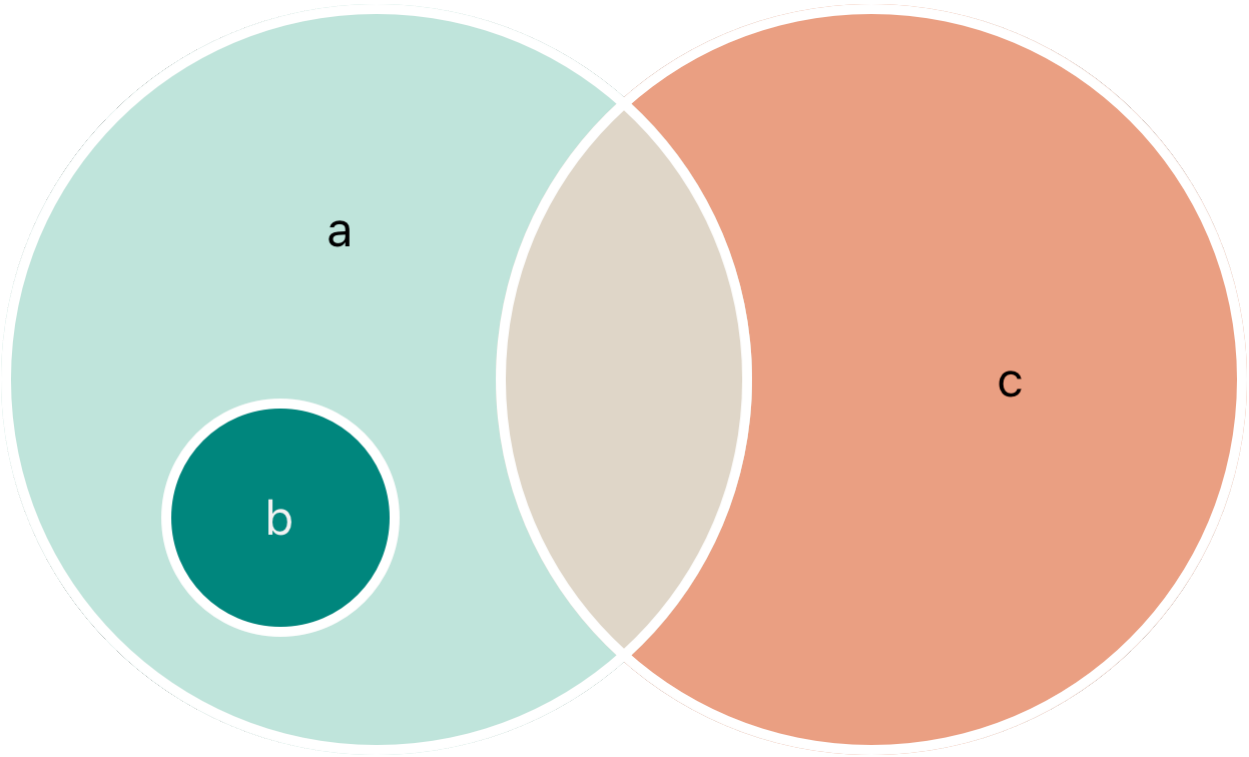
- `intersection(_:)` 메서드를 사용하면 두 집합의 공통 값을 가지는 새로운 집합을 생성합니다.
- `symmetricDifference(_:)` 메서드를 사용하면 두 집합의 공통 값을 제외한 새로운 집합을 생성합니다.
- `union(_:)` 메서드를 사용하면 두 집합의 모든 값으로 새로운 집합을 생성합니다.
- `subtracting(_:)` 메서드를 사용하면 특정 집합의 공통 값을 제외한 새로운 집합을 생성합니다.

```
let oddDigits: Set = [1, 3, 5, 7, 9]
let evenDigits: Set = [0, 2, 4, 6, 8]
let singleDigitPrimeNumbers: Set = [2, 3, 5, 7]

oddDigits.union(evenDigits).sorted()
// [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
oddDigits.intersection(evenDigits).sorted()
// []
oddDigits.subtracting(singleDigitPrimeNumbers).sorted()
// [1, 9]
oddDigits.symmetricDifference(singleDigitPrimeNumbers).sorted()
// [1, 2, 9]
```

집합 멤버십과 동등성 (Set Membership and Equality)

아래 그림은 세 집합 - a, b, c를 - 나타내며 집합의 공통 요소는 겹쳐서 표현하고 있습니다. a는 b의 모든 요소를 포함하므로, 집합 a는 b의 *초집합(superset)*이라 합니다. 반대로 b의 모든 요소가 a에 포함되어 있으므로, 집합 b는 집합 a의 *부분집합(subset)*이라 합니다. 집합 b와 집합 c는 공통 요소가 없으므로 *분리집합(disjoint)*이라 합니다.



- 두 집합이 같은 값을 모두 포함하고 있는지 판단하려면 “같음” 연산자(==)를 사용합니다.
- `isSubset(of:)` 메서드를 사용하면 집합이 특정 집합에 모든 값이 포함되어 있는지 판단합니다.
- `isSuperset(of:)` 메서드를 사용하면 집합에 특정 집합에 모든 값을 포함 하는지 판단합니다.
- `isStrictSubset(of:)`이나 `isStrictSuperset(of:)` 메서드를 사용하면 집합이 특정 집합과 같지 않고 부분집합인지 초집합인지 판단합니다.
- `isDisjoint(with:)` 메서드를 사용하면 두 집합이 공통 값이 없는지 판단합니다.

```
let houseAnimals: Set = ["🐶", "🐱"]
let farmAnimals: Set = ["🐷", "🐔", "🐮", "🐶", "🐱"]
let cityAnimals: Set = ["🐦", "🐭"]

houseAnimals.isSubset(of: farmAnimals)
// true
farmAnimals.isSuperset(of: houseAnimals)
// true
farmAnimals.isDisjoint(with: cityAnimals)
// true
```

딕셔너리 (Dictionaries)

*딕셔너리(dictionary)*는 순서와 상관없이 컬렉션에 같은 타입의 키(key)와 같은 타입의 값(value)을 저장합니다. 각 값은 고유한 키와 연결되어 있으며, 이 키는 딕셔너리 내부에서 해당 값을 식별하는 역할을 합니다. 배열의 아이템과 다르게, 딕셔너리의 아이템은 특정 순서를 가지고 있지 않습니다. 딕셔너리는 특정 단어를 찾기위해 사전을 찾는 방법과 같이 식별자를 기반으로 값을 조회해야 할 때 사용합니다.

> **참고**
>
> Swift의 `Dictionary` 타입은 Foundation의 `NSDictionary` 클래스와 연결됩니다.

Foundation과 Cocoa에서 `Dictionary` 사용에 대한 자세한 내용은 Dictionary와 NSDictionary의 연결 (Bridging Between Dictionary and NSDictionary)을 참고바랍니다.

딕셔너리 타입 축약형 구문 (Dictionary Type Shorthand Syntax)

Swift 딕셔너리의 타입은 `Dictionary<Key, Value>`로 적으며, `Key`는 딕셔너리 키로 사용되는 값의 타입이고, `Value`는 딕셔너리에 저장될 값의 타입 입니다.

> **참고**
>
> 딕셔너리 `Key` 타입은 집합의 값 타입과 같이 반드시 `Hashable` 프로토콜을 준수해야 합니다.

`[Key: Value]`와 같이 축약형으로 딕셔너리 타입을 작성할 수도 있습니다. 두 형식은 기능적으로 동일하지만, 이 가이드에서는 딕셔너리의 타입을 언급할 때, 축약형을 더 선호되어 사용됩니다.

빈 딕셔너리 생성 (Creating an Empty Dictionary)

배열 처럼, 이니셜라이저를 사용하여 타입을 포함한 빈 `Dictionary`를 생성할 수 있습니다:

```
var namesOfIntegers = [Int: String]()
// namesOfIntegers is an empty [Int: String] dictionary
```

이 예시에서 정수값을 사람이 읽을 수 있는 이름을 저장하는 `[Int: String]` 타입의 빈 딕셔너리를 생성합니다. 키는 `Int` 타입이고 값은 `String` 타입 입니다.

이미 타입 정보를 제공한다면, `[:]` (대괄호 안에 콜론을 포함해야 합니다)와 같이 작성하여 빈 딕셔너리를 생성할 수 있습니다:

```
namesOfIntegers[16] = "sixteen"
// namesOfIntegers now contains 1 key-value pair
namesOfIntegers = [:]
// namesOfIntegers is once again an empty dictionary of type [Int: String]
```

딕셔너리 리터럴로 딕셔너리 생성 (Creating a Dictionary with a Dictionary Literal)

앞서 배열 리터럴과 비슷한 구문을 가진 *딕셔너리 리터럴(dictionary literal)*로 딕셔너리를 초기화 할 수 있습니다. 딕셔너리 리터럴은 `Dictionary` 컬렉션으로 하나 이상의 키-값 쌍으로 작성하는 축약형입니다.

키-값 쌍은 키와 값의 결합입니다. 딕셔너리 리터럴에서 각 키-값 쌍의 키와 값은 콜론으로 구분됩니다. 키-값 쌍은 콤마로 구분하고, 대괄호로 둘러싸 리스트 형식으로 작성됩니다:

```
[<#key 1#>: <#value 1#>, <#key 2#>: <#value 2#>, <#key 3#>: <#value 3#>]
```

아래 예시는 국제 공항의 이름을 저장하는 딕셔너리를 생성합니다. 딕셔너리에 키는 세 글자로 된 국제 항공 운송 협회(Air Transport Association) 코드이고, 값은 공항 이름입니다:

```
var airports: [String: String] = ["YYZ": "Toronto Pearson", "DUB": "Dublin"]
```

`airports` 딕셔너리는 `[String: String]`의 타입을 가지도록 선언되고, `Dictionary` 키는 `String` 타입이고 값은 `String` 타입인 `[String: String]` 타입을 갖도록 선언됩니다.

참고

`airports` 딕셔너리는 아래 예시에서 딕셔너리에 더 많은 공항이 추가되므로 상수(`let`)가 아닌 변수(`var`)로 선언됩니다.

`airports` 딕셔너리는 두 개의 키-값 쌍을 포함한 딕셔너리 리터럴로 초기화 됩니다. 첫 번째 쌍은 `"YYZ"`의 키와 `"Toronto Pearson"` 값을 가집니다. 두 번째 쌍은 `"DUB"`의 키와 `"Dublin"` 값을 가집니다.

딕셔너리 리터럴은 두 개의 `String: String` 쌍을 포함합니다. 키-값 타입은 `airports` 변수 선언 (딕셔너리는 `String` 키와 `String` 값만 가능) 타입과 동일하며 `airports` 딕셔너리에 두 아이템을 포함하여 초기화 하는 것은 가능합니다.

배열과 마찬가지로 키와 값이 일관된 타입을 갖는 딕셔너리 리터럴로 초기화하는 경우 딕셔너리 타입을 작성할 필요가 없습니다. `airports`의 초기화는 축약형으로 작성 가능합니다:

```
var airports = ["YYZ": "Toronto Pearson", "DUB": "Dublin"]
```

리터럴 안에 모든 키와 값이 각각 같은 타입이기 때문에 Swift는 `airports` 딕셔너리를 사용하기 위해 `[String: String]`이 올바른 타입이라는 것을 추론할 수 있습니다.

배열 리터럴처럼 딕셔너리 리터럴도 마지막 키-값 쌍에 후행 콤마를 포함할 수 있습니다:

```
var airports = [
    "YYZ": "Toronto Pearson",
    "DUB": "Dublin",
]
```

딕셔너리 접근과 수정 (Accessing and Modifying a Dictionary)

메서드와 프로퍼티 또는 서브스크립트 구문을 사용하여 딕셔너리에 접근과 수정이 가능합니다.

배열과 마찬가지로 읽기 전용 `count` 프로퍼티로 `Dictionary`에 아이템 수를 확인할 수 있습니다:

```
print("The airports dictionary contains \(airports.count) items.")
// Prints "The airports dictionary contains 2 items.".
```

`count` 프로퍼티가 `0`과 같은지 Boolean `isEmpty` 프로퍼티를 이용하여 확인할 수 있습니다:

```
if airports.isEmpty {
    print("The airports dictionary is empty.")
} else {
    print("The airports dictionary is not empty.")
}
// Prints "The airports dictionary is not empty.".
```

서브스크립트 구문을 사용하여 딕셔너리에 새로운 아이템을 추가할 수 있습니다. 적절한 키의 타입을 서브스크립트 인덱스로 사용하고 적절한 값의 타입을 할당 합니다:

```
airports["LHR"] = "London"
// the airports dictionary now contains 3 items
```

특정 키를 서브스크립트 구문으로 사용하여 값을 변경할 수 있습니다:

```
airports["LHR"] = "London Heathrow"
// the value for "LHR" has been changed to "London Heathrow"
```

서브스크립트를 대체할 수 있는 방법으로, 딕셔너리의 `updateValue(_:forKey:)` 메서드를 사용하여 특정 키에 값을 설정하거나 업데이트할 수 있습니다. 위의 서브스크립트 예시와 같이, `updateValue(_:forKey:)` 메서드는 해당 키에 값이 존재하지 않으면 값을 설정 하거나 해당 키에 값이 존재하면 값을 업데이트 합니다. 그러나 서브스크립트와 다르게, `updateValue(_:forKey:)` 메서드는 업데이트 수행 후에 이전 값을 반환합니다. 이를 통해 업데이트가 발생했는지 여부를 알 수 있습니다.

`updateValue(_:forKey:)` 메서드는 딕셔너리의 값 타입의 옵셔널 값을 반환합니다. 예를 들어 딕셔너리에 `String` 값을 저장하면, `String?` 타입이나 "옵셔널 `String`"을 메서드는 반환합니다. 이 옵셔널 값은 해당 키에 존재한 업데이트 전의 값이나 존재한 값이 없었을 때는 `nil`을 포함합니다:

```
if let oldValue = airports.updateValue("Dublin Airport", forKey: "DUB") {
    print("The old value for DUB was \(oldValue).")
}
// Prints "The old value for DUB was Dublin.".
```

특정 키로 딕셔너리에 값을 가져올 때 서브스크립트 구문을 사용할 수도 있습니다. 값이 없는 키로 요청이 가능하기 때문에 딕셔너리의 서브스크립트는 딕셔너리의 값 타입의 옵셔널 값을 반환합니다. 딕셔너리에 요청된 키의 값이 있는 경우, 서브스크립트는 그 값의 옵셔널 값을 반환합니다. 값이 없으면 서브스크립트는 `nil`을 반환합니다:

```
if let airportName = airports["DUB"] {
    print("The name of the airport is \(airportName).")
} else {
    print("That airport is not in the airports dictionary.")
}
// Prints "The name of the airport is Dublin Airport.".
```

딕셔너리의 해당 키에 `nil` 값을 할당하여 키-값 쌍을 서브스크립트 구문을 사용하여 삭제할 수 있습니다:

```
airports["APL"] = "Apple International"
// "Apple International" is not the real airport for APL, so delete it
airports["APL"] = nil
// APL has now been removed from the dictionary
```

또한 딕셔너리에 `removeValue(forKey:)` 메서드를 사용하여 키-값 쌍을 삭제할 수 있습니다. 이 메서드는 키-값 쌍이 존재하면 삭제하고 삭제된 값을 반환하거나 값이 존재하지 않으면 `nil`을 반환합니다:

```
if let removedValue = airports.removeValue(forKey: "DUB") {
    print("The removed airport's name is \(removedValue).")
} else {
    print("The airports dictionary does not contain a value for DUB.")
}
// Prints "The removed airport's name is Dublin Airport.".
```

딕셔너리 반복 (Iterating Over a Dictionary)

`for-in` 루프로 딕셔너리에 키-값 쌍을 반복할 수 있습니다. 딕셔너리의 각 아이템은 `(key, value)` 튜플로 반환되고 튜플의 멤버를 임시 상수나 변수로 분리할 수 있습니다:

```
for (airportCode, airportName) in airports {
    print("\(airportCode): \(airportName)")
}
// LHR: London Heathrow
// YYZ: Toronto Pearson
```

`for-in` 루프에 대한 자세한 설명은 For-In 루프 (For-In Loops)를 참고바랍니다.

딕셔너리의 `keys`와 `values` 프로퍼티로 딕셔너리의 키나 값에 반복 가능한 컬렉션을 가져올 수도 있습니다:

```
for airportCode in airports.keys {
    print("Airport code: \(airportCode)")
}
// Airport code: LHR
// Airport code: YYZ

for airportName in airports.values {
    print("Airport name: \(airportName)")
}
// Airport name: London Heathrow
// Airport name: Toronto Pearson
```

딕셔너리의 키나 값을 `Array` 인스턴스의 API를 사용해야 할 경우, `keys`나 `values` 프로퍼티로 새로운 배열을 초기화하세요:

```
let airportCodes = [String](airports.keys)
// airportCodes is ["LHR", "YYZ"]

let airportNames = [String](airports.values)
// airportNames is ["London Heathrow", "Toronto Pearson"]
```

Swift의 `Dictionary` 타입은 정의된 순서를 가지고 있지 않습니다. 특정 순서로 딕셔너리의 키나 값을 반복하려면, `keys`나 `values` 프로퍼티에 `sorted()` 메서드를 사용 하십시오.

언어 가이드

제어 흐름 (Control Flow)

분기, 루프, 이른 종료로 코드를 구성합니다.

Swift는 다양한 제어 흐름(control flow) 구문을 제공합니다. 이것은 여러 번 작업을 수행하는 `while` 루프를 포함하고; 조건에 따라 다르게 실행되는 `if`, `guard`, `switch` 구문과; 코드에서 다른 포인트로 실행 플로우를 전달하는 `break`와 `continue`를 포함합니다. Swift는 배열, 딕셔너리, 범위, 문자열, 그리고 다른 연속적인 것에 대한 반복을 쉽게 조회하는 `for-in` 루프를 제공합니다. Swift는 또한 현재 범위를 벗어날 때 수행되는 `defer` 구문을 제공합니다.

Swift의 `switch` 구문은 C와 같은 언어들의 구문보다 훨씬 더 강력합니다. 케이스들은 간격 일치, 튜플, 특정 타입으로의 변환을 포함하여 다른 많은 패턴을 비교할 수 있습니다. `switch` 케이스에서 일치된 값은 케이스 문 안에서 사용할 수 있는 임시의 상수나 변수로 사용이 가능하고 복잡한 일치 조건은 각 케이스에 대해 `where` 절로 표현될 수 있습니다.

For-In 루프 (For-In Loops)

배열에 아이템, 범위의 숫자, 문자열에 문자와 같은 연속된 것에 대해 `for-in` 루프를 사용하여 반복할 수 있습니다.

이 예시는 `for-in` 루프를 사용하여 배열의 아이템을 반복합니다:

```
let names = ["Anna", "Alex", "Brian", "Jack"]
for name in names {
    print("Hello, \(name)!")
}
// Hello, Anna!
// Hello, Alex!
// Hello, Brian!
// Hello, Jack!
```

딕셔너리의 키-값 쌍 접근을 위해 반복을 사용할 수도 있습니다. 딕셔너리의 각 아이템은 딕셔너리가 반복될 때, `(key, value)` 튜플로 반환되고 `for-in` 루프 본문 내에서 사용하기 위해 `(key, value)` 튜플의 멤버를 명시적으로 이름을 가진 상수로 분해할 수 있습니다. 아래의 예시에서 딕셔너리의 키는 `animalName` 상수로 분해되고, 딕셔너리의 값은 `legCount` 상수로 분해됩니다.

```
let numberOfLegs = ["spider": 8, "ant": 6, "cat": 4]
for (animalName, legCount) in numberOfLegs {
    print("\(animalName)s have \(legCount) legs")
}
// cats have 4 legs
// ants have 6 legs
// spiders have 8 legs
```

`Dictionary`는 기본적으로 순서가 없으며, 반복으로 가져올 아이템에 대한 순서를 보장하지 않습니다. 특히 아이템을 딕셔너리에 삽입하는 순서는 아이템이 반복되는 순서를 정의하지 않습니다. 배열과 딕셔너리의 자세한 내용은 컬렉션 타입 (Collection Types)을 참고바랍니다.

숫자 범위에 대해 `for-in` 루프를 사용할 수도 있습니다. 이 예시는 5의 배수 항목을 몇 개 출력합니다:

```
for index in 1...5 {
    print("\(index) times 5 is \(index * 5)")
}
// 1 times 5 is 5
// 2 times 5 is 10
// 3 times 5 is 15
// 4 times 5 is 20
// 5 times 5 is 25
```

이 반복되는 시퀀스는 닫힌 범위 연산자(`...`)를 사용하여 표시되는 1부터 5까지의 숫자 범위 입니다. `index`의 값은 범위의 첫 번째 숫자(1)로 설정되고, 루프 안의 구문이 실행됩니다. 이 경우 루프는 `index`의 현재값을 5배 하여 출력하는 하나의 구문만 포함합니다. 구문이 실행된 후에, `index`의 값은 범위의 두 번째 값(2)로 업데이트 되고, `print(_:separator:terminator:)` 함수가 다시 호출됩니다. 이 프로세스는 범위의 끝에 도달할 때까지 계속됩니다.

위의 예시에서 `index`는 루프의 각 반복이 시작할 때 자동으로 설정되는 상수입니다. 따라서 `index`를 사용하기 전에 선언할 필요가 없습니다. `let` 선언 키워드가 필요없이 루프 선언에 포함되어 암시적으로 선언됩니다.

시퀀스로부터 각 값이 필요하지 않으면, 변수 이름의 위치에 언더바를 사용하여 값을 무시할 수 있습니다.

```
let base = 3
let power = 10
var answer = 1
for _ in 1...power {
    answer *= base
}
print("\(base) to the power of \(power) is \(answer)")
// Prints "3 to the power of 10 is 59049".
```

위의 예시는 한 숫자를 다른 숫자의 제곱으로 계산하는 방법을 보여줍니다 (이 경우 3의 10제곱). 1로 시작하고 10으로 끝나는 닫힌 범위를 사용하여 시작값 1 (즉, 3의 0제곱)에 3을 10번 곱합니다. 이 계산에서 루프의 각 카운터 값은 불필요 합니다 - 이 코드는 간단하게 올바른 숫자만큼 실행되기만 하면 됩니다. 루프 변수 위치에 사용된 언더바 문자(_)는 각 값을 무시하고 루프의 각 반복동안 현재값에 접근하는 것을 제공하지 않습니다.

어떤 상황에선 양 끝점을 포함하는 닫힌 범위를 사용하지 않을 수 있습니다. 시계 페이스에 매 분마다 눈금을 그리는 것을 생각해 봅시다. 0분을 시작으로 60개의 눈금을 그려야 합니다. 이럴 경우 반열림 범위 연산자(`..<`)를 사용하여 가장 최소값은 포함하지만 최대값은 포함되지 않게 사용할 수 있습니다. 범위에 대한 자세한 내용은 범위 연산자 (Range Operators)를 참고바랍니다.

```
let minutes = 60
for tickMark in 0..<minutes {
    // render the tick mark each minute (60 times)
}
```

어떤 사용자는 UI에 더 적은 눈금을 원할 수도 있습니다. 매 5분마다 눈금을 그리기 원할 수 있습니다. 원하지 않는 눈금을 건너뛰기 위해 `stride(from:to:by:)` 함수를 사용 하세요.

```
let minuteInterval = 5
for tickMark in stride(from: 0, to: minutes, by: minuteInterval) {
    // render the tick mark every 5 minutes (0, 5, 10, 15 ... 45, 50, 55)
}
```

닫힌 범위에서도 `stride(from:through:by:)`를 사용할 수 있습니다:

```
let hours = 12
let hourInterval = 3
for tickMark in stride(from: 3, through: hours, by: hourInterval) {
    // render the tick mark every 3 hours (3, 6, 9, 12)
}
```

위의 예시는 범위, 배열, 딕셔너리, 문자열을 조회하기 위해 `for-in` 루프를 사용합니다. 그러나 이 구문은 모든 컬렉션을 조회할 수 있으며, 시퀀스 (Sequence) 프로토콜을 준수하는 사용자가 정의한 클래스나 컬렉션 타입도 포함됩니다.

While 루프 (While Loops)

`while` 루프는 조건이 `false`가 될 때까지 구문을 수행합니다. 이러한 루프는 첫 번째 반복이 시작되기 전에 반복 횟수를 알 수 없을 때 가장 잘 사용됩니다. Swift는 두 종류의 `while` 루프를 제공합니다:

- `while`은 루프가 시작할 때마다 조건을 비교합니다.
- `repeat-while`은 루프가 끝날 때 마다 조건을 비교합니다.

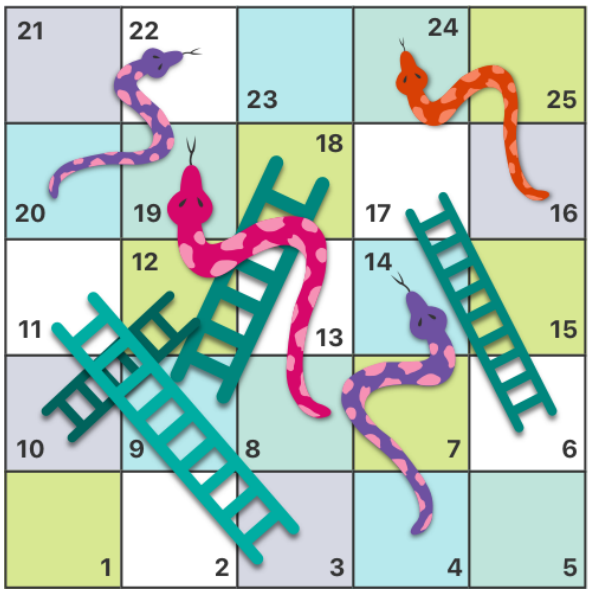
While

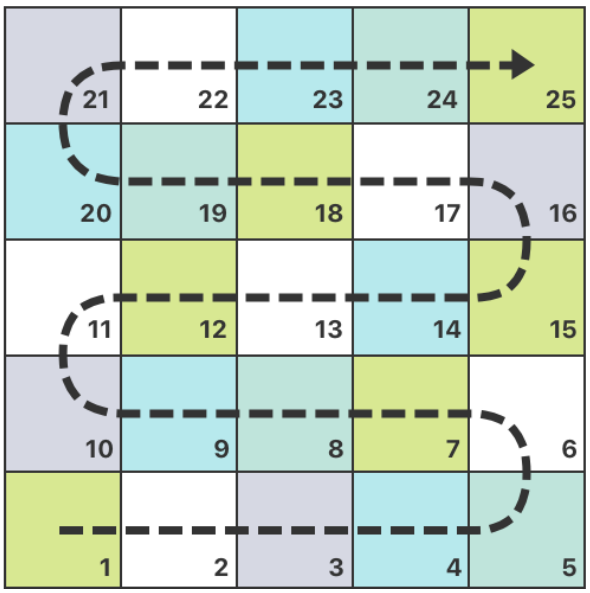
`while` 루프는 단일 조건을 평가하는 것으로 시작합니다. 조건이 `true`이면, 조건이 `false`가 될 때까지 구문은 반복됩니다.

다음은 `while` 루프의 기본 형식 입니다:

```
while <#condition#> {
   <#statements#>
}
```

이 예시는 Snakes and Ladders 게임을 플레이 합니다 (또는 Chutes and Ladders):





게임의 규칙은 다음과 같습니다:

- 이 보드는 25개의 정사각형을 가지고 있고, 25에 착륙하거나 25를 넘는 것이 목표입니다.
- 플레이어의 시작 정사각형을 보드의 좌측 하단의 모서리로 “정사각형 0”이라 합니다.
- 각 턴마다 주사위를 굴리고, 위의 점선 화살표 방향으로 각 숫자만큼 정사각형을 이동합니다.
- 턴이 사다리의 바닥에서 끝나면 사다리를 타고 올라갑니다.
- 턴이 뱀의 머리에서 끝나면 뱀의 꼬리로 내려갑니다.

게임 보드는 `Int` 값의 배열로 표현됩니다. 크기는 `finalSquare`라는 상수를 기반으로 하며 배열을 초기화하고 나중에 예시에서 승리 조건을 확인하는데 사용됩니다. “정사각형 0”에서 플레이어는 시작하기 때문에 보드는 `Int` 값 0을 포함하여 25가 아닌 26의 크기로 초기화 됩니다.

```
let finalSquare = 25
var board = [Int](repeating: 0, count: finalSquare + 1)
```

그런 다음 일부 사각형은 뱀과 사다리에 대해 더 구체적인 값을 갖도록 설정됩니다. 사다리 기반의 정사각형은 보드의 위로 이동하기 때문에 양수를 가지는 반면에, 뱀 머리의 정사각형은 보드의 아래로 이동하기 때문에 음수를 가집니다.

```
board[03] = +08; board[06] = +11; board[09] = +09; board[10] = +02
board[14] = −10; board[19] = −11; board[22] = −02; board[24] = −08
```

정사각형 3은 정사각형 11로 이동하는 사다리 바닥이 포함됩니다. 이것을 표현하기 위해 `board[03]`은 정수값 `8` (`3`과 `11`의 차이)과 동등한 `+08`을 대입합니다. 값과 구문을 정렬하기 위해 단항 덧셈 연산자(`+i`)는 단항 빼기 연산자(`−i`)를 명시적으로 사용하고 `10`보다 작은 숫자의 앞에 `0`을 채웁니다. (두 스타일 모두 꼭 필요한 것은 아니지만 코드가 깔끔해 집니다.)

```
var square = 0
var diceRoll = 0
while square < finalSquare {
    // roll the dice
    diceRoll += 1
    if diceRoll == 7 { diceRoll = 1 }
    // move by the rolled amount
    square += diceRoll
    if square < board.count {
        // if we're still on the board, move up or down for a snake or a ladder
        square += board[square]
    }
}
print("Game over!")
```

위의 예시는 주사위 굴리기에 매우 간단한 접근방식을 사용합니다. 난수를 생성하는 대신에 `diceRoll`은 `0`의 값으로 시작합니다. 매 `while` 루프를 실행할 때 마다 `diceRoll`은 1씩 증가하고 너무 커졌는지 확인합니다. 반환된 값이 `7`과 같을 때마다 주사위 굴림이 너무 커지므로 값을 `1`로 재설정 합니다. 그 결과 `diceRoll` 값은 항상 `1`, `2`, `3`, `4`, `5`, `6`, `1`, `2` 등과 같은 시퀀스 입니다.

주사위를 굴리고 난 후에 플레이어는 `diceRoll` 정사각형으로 앞으로 이동합니다. 주사위 굴림이 플레이어를 정사각형 25를 넘어 이동할 수 있으며, 이럴 경우 게임이 끝납니다. 이 시나리오에 대처하기 위해 `board` 배열의 `count` 프로퍼티가 `square`보다 작은지 확인합니다. `square`가 유효하다면, `board[square]`에 현재 `square` 값을 저장하고 플레이어를 이동합니다.

> **참고**
>
> 확인을 수행하지 않으면, `board[square]`는 `board` 배열의 범위를 넘는 값을 접근하려 시도할 수 있습니다. 이런 경우 런타임 오류가 발생합니다.

현재 `while` 루프의 실행이 끝나면, 루프를 다시 실행해야 되는지 알기 위해 조건을 확인합니다. 플레이어가 정사각형 숫자 25에 위치하거나 넘으면, 루프 조건은 `false`가 되고 게임은 종료됩니다.

`while` 루프가 시작될 때 게임의 길이가 명확하지 않으므로 이러한 경우 `while` 루프가 적절합니다. 대신에 루프는 특정 조건이 만족할 때까지 실행됩니다.

Repeat-While

`while` 루프의 다른 하나는 루프의 조건을 판단하기 전에 루프 블록을 처음에 한번 먼저 통과하는 `repeat-while` 루프가 있습니다. 조건이 `false`가 될 때까지 루프를 반복합니다.

> **참고**
>
> Swift에 `repeat-while` 루프는 다른 언어의 `do-while` 루프와 유사합니다.

다음은 `repeat-while` 루프의 일반적인 형식입니다:

```
repeat {
   <#statements#>
} while <#condition#>
```

Snakes and Ladders 예시에서 `while` 루프보다 `repeat-while` 루프를 쓰는 것이 더 좋습니다. `finalSquare`, `board`, `square`, `diceRoll`의 값은 `while` 루프에서와 같은 방식으로 초기화 됩니다.

```
let finalSquare = 25
var board = [Int](repeating: 0, count: finalSquare + 1)
board[03] = +08; board[06] = +11; board[09] = +09; board[10] = +02
board[14] = −10; board[19] = −11; board[22] = −02; board[24] = −08
var square = 0
var diceRoll = 0
```

이 게임의 버전에서는 루프에서 첫 번째 행동은 사다리인지 뱀인지 확인합니다. 보드에는 플레이러를 바로 정사각형 25로 보내는 사다리가 없으므로, 사다리를 타고 게임에서 이기는 것은 불가능합니다. 따라서 루프의 첫 번째 동작으로 뱀이나 사다리를 확인하는 것이 안전합니다.

게임 시작할 때 플레이어는 “정사각형 0”에 있습니다. `board[0]`은 항상 `0`과 같으며 영향이 없습니다.

```
repeat {
    // move up or down for a snake or ladder
    square += board[square]
    // roll the dice
    diceRoll += 1
    if diceRoll == 7 { diceRoll = 1 }
    // move by the rolled amount
    square += diceRoll
} while square < finalSquare
print("Game over!")
```

뱀과 사다리를 코드는 확인 후에 주사위를 굴리고 플레이어를 `diceRoll` 정사각형에 따라 앞으로 이동합니다. 그러면 현재 루프는 종료됩니다.

루프의 조건(`while square < finalSquare`)은 이전과 같지만, 이번에는 루프가 처음 실행된 후 끝에서 평가됩니다. 이전 예시의 `while` 루프보다 이 게임에서는 `repeat-while` 루프가 더 적합합니다. 위의 `repeat-while` 루프에서 `square += board[square]`는 루프의 `while` 조건이 `square` 가 항상 보드 위에 있는 것을 확인 한 직후에 실행 됩니다. 이 동작은 이전 예시에서 `while` 루프 버전에서 배열의 범위를 체크할 필요가 없습니다.

조건 구문 (Conditional Statements)

특정 조건에 따라 다양한 코드를 실행하는 것은 유용한 경우가 있습니다. 오류가 발생했을 때 추가 코드를 실행하거나, 값이 너무 크거나 작을 때 메세지를 출력하려고 싶을 수 있습니다. 이러한 동작을 위해 코드의 일부를 조건부로 만들어야 합니다.

Swift는 코드에 조건부를 추가하는 두 가지 방법이 있습니다: `if` 구문과 `switch` 구문이 있습니다. 일반적으로 `if` 구문은 몇 가지 가능한 결과만 있는 단순한 조건을 평가할 때 적합합니다. `switch` 구문은 여러 가지 가능한 조합을 가진 더 복잡한 조건에 적합하고, 패턴 일치를 통해 적절한 코드 분기를 선택하는데 유용합니다.

If

가장 간단한 형식으로 `if` 구문은 단일 `if` 조건을 갖습니다. 조건이 `true`일 경우에만 구문을 실행합니다.

```
var temperatureInFahrenheit = 30
if temperatureInFahrenheit <= 32 {
    print("It's very cold. Consider wearing a scarf.")
}
// Prints "It's very cold. Consider wearing a scarf.".
```

위 예시는 기온이 화씨 32도 보다 작거나 같은지 확인합니다 (물이 어는 온도 입니다). 만약 화씨 32도 보다 작거나 같으면 메세지가 출력됩니다. 그렇지 않으면 메세지가 출력되지 않으며, `if` 구문의 닫힘 중괄호 이후에 코드 실행이 계속됩니다.

`if` 구문은 `if` 조건이 `false`일 때, 그 밖의 다른 구문을 제공할 수 있습니다. 이 구문은 `else` 키워드로 표기합니다.

```
temperatureInFahrenheit = 40
if temperatureInFahrenheit <= 32 {
    print("It's very cold. Consider wearing a scarf.")
} else {
    print("It's not that cold. Wear a T-shirt.")
}
// Prints "It's not that cold. Wear a T-shirt.".
```

두 중괄호 중 하나는 항상 실행됩니다. 기온이 증가하여 화씨 `40`도를 가지므로, 더 이상 추워서 스카프를 해야 한다고 충고하지 않으며 `else` 구문이 실행 됩니다.

추가 절을 고려하기 위해 여러 `if` 구문을 연결할 수 있습니다.

```
temperatureInFahrenheit = 90
if temperatureInFahrenheit <= 32 {
    print("It's very cold. Consider wearing a scarf.")
} else if temperatureInFahrenheit >= 86 {
    print("It's really warm. Don't forget to wear sunscreen.")
```

```
} else {
    print("It's not that cold. Wear a T-shirt.")
}
// Prints "It's really warm. Don't forget to wear sunscreen.".
```

여기서 `if` 구문은 특정 따뜻한 기온에 대한 조건이 추가 되었습니다. 마지막 `else` 절은 남아있고 어떤 기온이 너무 따뜻하거나 너무 춥지 않을 경우에 응답을 출력합니다.

그러나 마지막 `else` 절은 옵셔널이고 이 조건이 완벽하게 필요가 없으면 제외할 수 있습니다.

```
temperatureInFahrenheit = 72
if temperatureInFahrenheit <= 32 {
    print("It's very cold. Consider wearing a scarf.")
} else if temperatureInFahrenheit >= 86 {
    print("It's really warm. Don't forget to wear sunscreen.")
}
```

기온이 너무 춥지도 따뜻하지도 않아 `if`나 `else if` 조건에 포함되지 않으므로 아무런 메세지가 출력되지 않습니다.

Swift는 값을 설정할 때 사용할 수 있는 `if`의 축약형을 제공합니다. 예를 들어, 아래 코드와 같습니다:

```
let temperatureInCelsius = 25
let weatherAdvice: String

if temperatureInCelsius <= 0 {
    weatherAdvice = "It's very cold. Consider wearing a scarf."
} else if temperatureInCelsius >= 30 {
    weatherAdvice = "It's really warm. Don't forget to wear sunscreen."
} else {
    weatherAdvice = "It's not that cold. Wear a T-shirt."
}

print(weatherAdvice)
// Prints "It's not that cold. Wear a T-shirt.".
```

여기서, 각 분기는 `if` 구문 이후에 출력되는 `weatherAdvice` 상수에 값을 설정합니다.

`if` 표현식을 다른 표현을 사용하여 더 간결하게 코드를 작성할 수 있습니다:

```
let weatherAdvice = if temperatureInCelsius <= 0 {
    "It's very cold. Consider wearing a scarf."
} else if temperatureInCelsius >= 30 {
    "It's really warm. Don't forget to wear sunscreen."
} else {
    "It's not that cold. Wear a T-shirt."
}

print(weatherAdvice)
// Prints "It's not that cold. Wear a T-shirt.".
```

이 `if` 표현식에서 각 분기는 하나의 값을 포함합니다. 분기의 조건이 참이면, 값은 `weatherAdvice`에 할당되고 `if` 표현식 전체에서 사용됩니다. 모든 `if` 분기는 `else if` 분기나 `else` 분기를 가지고 있어, 분기 중 하나에 항상 일치하고 `if` 표현식이 어떤 조건이 참인지 상관없이 값을 항상 생성합니다.

할당 구문은 `if` 표현식 바깥에서 시작하므로, 각 분기 안에 `weatherAdvice =`를 반복할 필요가 없습니다. 대신에 `if` 표현식의 각 분기는 `weatherAdvice`에 대한 세 가지 가능한 값 중 하나를 생성하고 할당합니다.

`if` 표현식의 모든 분기는 동일한 타입의 값을 포함해야 합니다. Swift는 각 분기의 타입을 개별적으로 확인하기 때문에 여러 타입과 사용할 수 있는 `nil`과 같은 값은 Swift가 `if` 표현식의 타입을 자동으로 결정하는 것을 방지합니다. 대신에 타입을 명시적으로 지정해야 합니다 - 예를 들어:

```
let freezeWarning: String? = if temperatureInCelsius <= 0 {
    "It's below freezing. Watch for ice!"
} else {
    nil
}
```

위의 코드에서 `if` 표현식 중 하나의 분기는 문자열 값을 가지고 다른 분기는 `nil` 값을 가집니다. `nil` 값은 모든 옵셔널 타입의 값으로 사용될 수 있으므로, 타입 명시 (Type Annotations)에서 설명했듯이 `freezeWarning`은 옵셔널 문자열이라고 명시적으로 작성되어야 합니다.

타입 정보를 제공하기 위한 다른 방법은 `freezeWarning`에 대한 타입을 명시적으로 제공하는 대신에 `nil`에 대한 타입을 명시적으로 제공하는 것입니다:

```
let freezeWarning = if temperatureInCelsius <= 0 {
    "It's below freezing. Watch for ice!"
} else {
    nil as String?
}
```

`if` 표현식은 오류를 발생시키거나 반환하지 않는 `fatalError(_:file:line:)`와 같은 함수를 호출하여 예기치 않은 실패에 대응할 수 있습니다. 예를 들어:

```
let weatherAdvice = if temperatureInCelsius > 100 {
    throw TemperatureError.boiling
} else {
    "It's a reasonable temperature."
}
```

이 예시에서 `if` 표현식은 예보된 온도가 100℃ - 물의 끓는점보다 높은지 확인합니다. 이렇게 높은 온도에서는 텍스트의 요약을 반환하는 대신에 `.boiling` 오류를 발생시킵니다. `if` 표현식은 오류를 발생할 수 있지만, `try`를 전에 작성하지 않습니다. 오류에 대한 자세한 내용은 오류 처리 (Error Handling)을 참고바랍니다.

위의 예시에서와 같이 할당의 오른편으로 `if` 표현식을 사용하는 것 외에 함수나 클로저가 반환하는 값으로도 사용할 수 있습니다.

Switch

`switch` 구문은 값을 고려하고 가능한 여러 개의 일치하는 패턴과 비교합니다. 그런 다음 첫 번째로 일치하는 패턴을 기반으로 적절한 코드 블록을 실행합니다. `switch` 구문은 여러 가능한 상태에 응답하기 위해 `if` 구문의 대체 구문으로 제공합니다.

가장 간단한 형식으로 `switch` 구문은 하나 이상의 같은 타입의 값과 비교합니다.

```
switch <#some value to consider#> {
case <#value 1#>:
    <#respond to value 1#>
case <#value 2#>,
    <#value 3#>:
    <#respond to value 2 or 3#>
default:
    <#otherwise, do something else#>
}
```

모든 switch 구문은 각각 case 키워드로 시작하는 여러 개의 케이스로 구성되어 있습니다. 특정 값과 비교하는 것 외에도 Swift는 각 케이스에 대해 더 복잡한 일치 패턴을 지정하는 여러 가지 방법을 제공합니다. 이 옵션은 이 챕터 뒤에서 다루겠습니다.

if 구문의 본문과 같이, 각 case는 코드 실행 부분이 분리되어 있습니다. switch 구문은 실행될 부분을 선택합니다. 이 절차를 값의 *스위칭(switching)*이라 알려져 있습니다.

모든 switch 구문은 완벽해야 합니다. 이것은 고려 중인 타입의 가능한 모든 값은 switch 케이스 중 하나와 일치해야 합니다. 가능한 모든 값에 대한 케이스를 제공하는 것이 적절하지 않은 경우, 명시적으로 해결되지 않은 모든 값을 포함하도록 기본 케이스를 정의할 수 있습니다. 기본 케이스는 default 키워드로 나타내고 항상 마지막에 위치합니다.

이 예시는 switch 구문을 사용하여 someCharacter의 소문자를 고려합니다:

```
let someCharacter: Character = "z"
switch someCharacter {
case "a":
    print("The first letter of the Latin alphabet")
case "z":
    print("The last letter of the Latin alphabet")
default:
    print("Some other character")
}
// Prints "The last letter of the Latin alphabet".
```

switch 구문의 첫번째 케이스는 영어 알파벳의 첫 번째 문자인 a와 일치하고, 두 번째 케이스는 마지막 문자인 z와 일치합니다. switch는 모든 알파벳 문자 뿐만 아니라 모든 가능한 문자에 대한 케이스를 가지고 있어야 하므로, 이 switch 구문은 a와 z을 제외한 다른 모든 문자는 default 케이스를 사용합니다. 이렇게 함으로 switch 구문은 완벽하다는 것을 보장합니다.

if 구문과 같이, switch 구문도 아래와 같은 형식을 가지고 있습니다:

```
let anotherCharacter: Character = "a"
let message = switch anotherCharacter {
case "a":
    "The first letter of the Latin alphabet"
case "z":
    "The last letter of the Latin alphabet"
default:
    "Some other character"
}

print(message)
// Prints "The first letter of the Latin alphabet".
```

이 예시에서 `switch` 표현식에 각 케이스는 `anotherCharacter`와 동일한 케이스일 때 사용될 `message`에 대한 값을 포함합니다. `switch`는 항상 동일한 케이스가 존재하기 때문에, 값은 항상 할당됩니다.

`if` 표현식과 마찬가지로, 주어진 케이스에 대해 값을 제공하는 대신에 오류를 발생시키거나 반환되지 않는 `fatalError(_:file:line:)`와 같은 함수를 호출할 수 있습니다. 위의 예시에서와 같이 할당의 오른편에 있는 `switch` 표현식을 함수나 클로저가 반환하는 값으로 사용할 수 있습니다.

명시적 Fallthrough (No Implicit Fallthrough)

C와 Objective-C의 `switch` 구문과 다르게, Swift의 `switch` 구문은 기본적으로 각 케이스의 바닥에서 다음 케이스로 바로 실행되지 않습니다. 명시적으로 `break` 구문 요청 없이 처음 일치하는 `switch` 케이스가 완료되자마자 `switch` 구문 전체가 끝납니다. 이러한 점은 `switch` 구문을 더 안전하고 C의 `switch` 구문보다 사용하기 쉽게 해주고 실수로 `switch` 케이스가 하나 이상 실행되는 것을 피할 수 있습니다.

> **참고**
>
> Swift에서 `break`는 요구되지 않지만, 특정 케이스를 무시하거나 일치하는 케이스의 실행이 완료되기 전에 빠져나와야 할 경우 `break` 구문을 사용할 수 있습니다. 자세한 내용은 Switch 구문에서 중단 (Break in a Switch Statement)을 참고바랍니다.

각 케이스의 본문은 반드시 적어도 하나의 실행가능한 구문이 포함되어야 합니다. 아래의 코드는 첫번째 케이스가 비어 있으므로 유효하지 않습니다:

```
let anotherCharacter: Character = "a"
switch anotherCharacter {
case "a": // Invalid, the case has an empty body
case "A":
    print("The letter A")
default:
    print("Not the letter A")
}
// This will report a compile-time error.
```

C의 `switch` 구문과 다르게, 이 `switch` 구문은 `"a"`와 `"A"` 둘 다 일치하지 않습니다. `case "a":`에 어떠한 실행가능한 구문이 없기 때문에 컴파일 때 오류가 발생합니다. 이 접근방식은 한 케이스에서 다른 케이스로의 우발적인 실행을 방지하고 의도를 더 명확하게 하고 안전한 코드를 만듭니다.

`"a"`와 `"A"` 모두 일치하는 단일 케이스의 `switch`를 만드려면 두 값을 콤마로 구분하여 하나로 결합하여 구성합니다.

```
let anotherCharacter: Character = "a"
switch anotherCharacter {
case "a", "A":
    print("The letter A")
default:
    print("Not the letter A")
}
// Prints "The letter A".
```

읽기 쉽게 하기위해 합성 케이스는 여러줄로 작성할 수도 있습니다. 합성 케이스에 대한 자세한 내용은 합성 케이스 (Compound Cases)를 참고바랍니다.

참고

특정 `switch` 케이스 끝에서 명시적으로 다음 케이스로 떨어뜨리려면 Fallthrough에서 설명한대로 `fallthrough` 키워드를 사용합니다.

범위 일치 (Interval Matching)

`switch` 케이스에서 값이 특정 범위에 포함되는지 확인할 수 있습니다. 이 예시에서는 숫자 범위를 사용하여 모든 크기의 숫자에도 자연스러운 언어 표현을 제공합니다:

```
let approximateCount = 62
let countedThings = "moons orbiting Saturn"
let naturalCount: String
switch approximateCount {
case 0:
    naturalCount = "no"
case 1..<5:
    naturalCount = "a few"
case 5..<12:
    naturalCount = "several"
case 12..<100:
    naturalCount = "dozens of"
case 100..<1000:
    naturalCount = "hundreds of"
default:
    naturalCount = "many"
}
print("There are \(naturalCount) \(countedThings).")
// Prints "There are dozens of moons orbiting Saturn.".
```

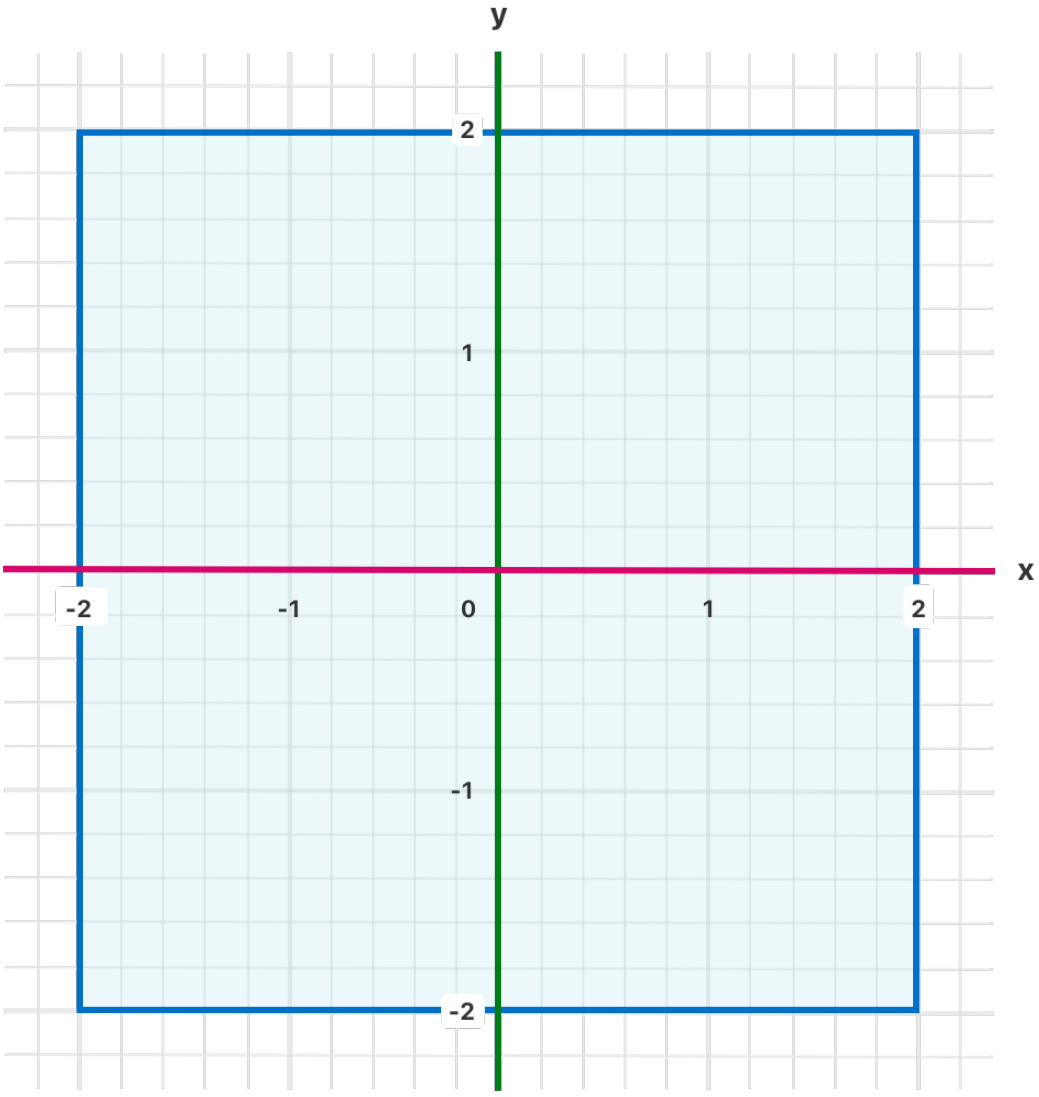
위의 예시에서 `approximateCount`는 `switch` 구문에서 평가됩니다. 각 `case`는 숫자 값이나 범위를 비교합니다. `approximateCount`의 값은 12와 100 사이에 속하므로, `naturalCount`는 `"dozens of"` 값이 할당되고, `switch` 구문을 빠져나옵니다.

튜플 (Tuples)

같은 `switch` 구문에 여러 값인 튜플을 사용할 수 있습니다. 튜플의 각 요소는 다른 값이나 값의 범위를 판단할 수 있습니다. 가능한 어떠한 값도 일치하도록 와일드카드 패턴(wildcard pattern)으로 알려진 언더바 문자(_)를 사용할 수 있습니다.

아래 예시는 타입 `(Int, Int)`의 단순한 튜플로 표현된 (x, y) 좌표를 받아서, 예시 다음에 나오는 그래프에서 해당 좌표를 분류합니다.

```
let somePoint = (1, 1)
switch somePoint {
case (0, 0):
    print("\(somePoint) is at the origin")
case (_, 0):
    print("\(somePoint) is on the x-axis")
case (0, _):
    print("\(somePoint) is on the y-axis")
case (-2...2, -2...2):
    print("\(somePoint) is inside the box")
default:
    print("\(somePoint) is outside of the box")
}
// Prints "(1, 1) is inside the box".
```



`switch` 구문은 해당 점이 원점(0, 0)에 있는지, 빨간색 x축 위에 있는지, 초록색 y축 위에 있는지, 원점을 중심으로 하는 파란색 4 x 4 박스 내부에 있는지 외부에 있는지 판단합니다.

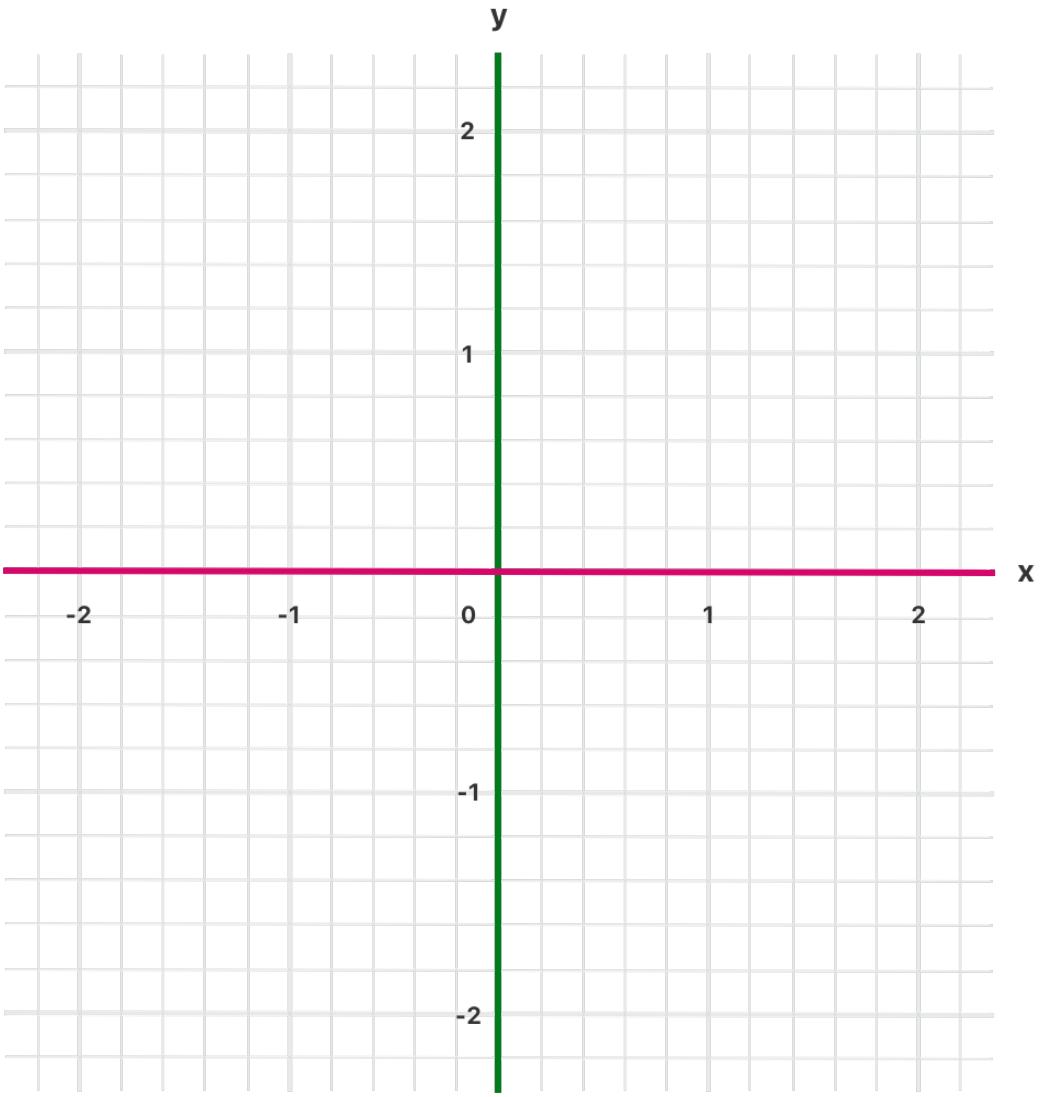
C와 다르게 Swift는 여러 `switch` 케이스가 같은 값인지 고려할 수 있습니다. 사실 좌표(0, 0)는 이 예시의 모든 4개의 케이스에 일치합니다. 그러나 여러 개가 일치할 수 있다면, 첫 번째 일치하는 케이스가 항상 사용됩니다. 좌표(0, 0)는 첫번째 `case (0, 0)`와 일치하고, 다른 모든 케이스는 무시됩니다.

값 바인딩 (Value Bindings)

switch 케이스는 일치하는 값을 임시적으로 상수나 변수로 이름을 가질 수 있으며, 케이스 본문 안에서 사용할 수 있습니다. 값은 케이스의 본문 내부에서 임시적 상수나 변수로 바인드 되기 때문에 이러한 동작을 *값 바인딩(value binding)*이라 합니다.

아래 예시는 튜플의 타입 (Int, Int)로 표현된 좌표 (x, y)를 받아서, 이어지는 그래프에서 해당 좌표를 분류합니다:

```
let anotherPoint = (2, 0)
switch anotherPoint {
case (let x, 0):
    print("on the x-axis with an x value of \(x)")
case (0, let y):
    print("on the y-axis with a y value of \(y)")
case let (x, y):
    print("somewhere else at (\(x), \(y))")
}
// Prints "on the x-axis with an x value of 2".
```



이 switch 구문은 좌표가 빨간색인 x축에 있는지, 초록색인 y축에 있는지, 또는 다른 곳(축 위가 아닌)에 있는지 판단합니다.

이 세 개의 `switch` 케이스는 자리 상수 `x`와 `y`를 선언하며, `anotherPoint`에서 임시적으로 하나나 두 개의 튜플 값을 받습니다. 첫 번째 케이스인 `case (let x, 0)`는 `y` 좌표 값이 `0`이고 `x` 좌표 값은 임시적 상수 `x`에 할당한 것과 일치합니다. 비슷하게 두 번째 케이스 `case (0, let y)`는 `x` 좌표 값이 `0`이고 `y` 좌표 값은 임시적 상수 `y`에 할당한 것과 일치합니다.

임시적 상수가 선언된 후에 케이스의 코드 블록 안에서 사용될 수 있습니다. 여기서는 좌표를 분리해 출력합니다.

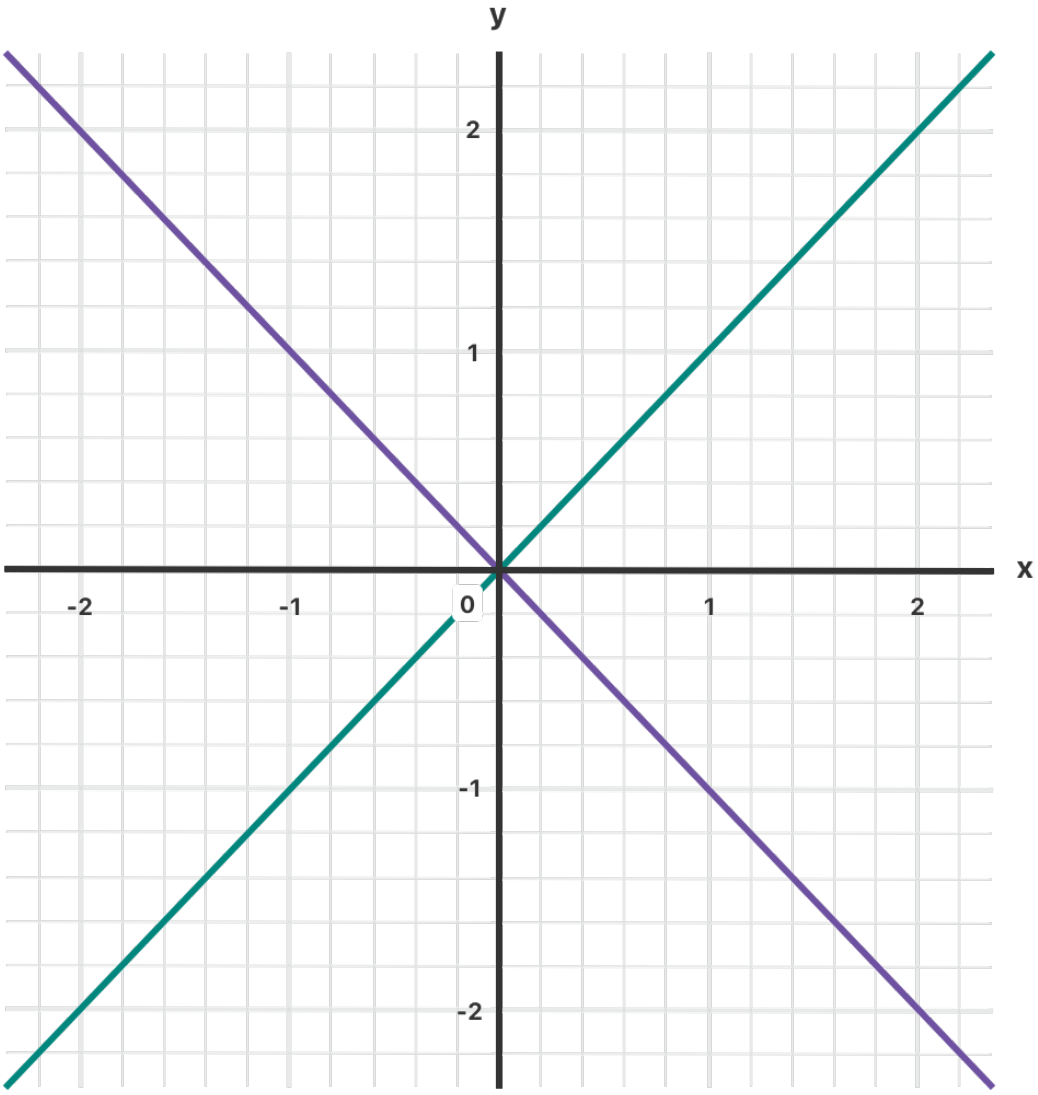
이 `switch` 구문은 `default` 케이스를 가지고 있지 않습니다. 마지막 케이스 `case let (x, y)`는 어떠한 값도 일치할 수 있는 두 개의 상수를 가지는 튜플로 선언합니다. `anotherPoint`는 두 값의 튜플이기 때문에 가능한 남아 있는 모든 케이스와 일치하며, `switch` 구문을 완벽하게 하기위해 `default` 케이스가 필요치 않습니다.

Where

`switch` 케이스는 추가 조건으로 `where` 절을 사용할 수 있습니다.

아래 예시는 그래프에 (x, y)를 분류합니다:

```
let yetAnotherPoint = (1, -1)
switch yetAnotherPoint {
case let (x, y) where x == y:
    print("(\(x), \(y)) is on the line x == y")
case let (x, y) where x == -y:
    print("(\(x), \(y)) is on the line x == -y")
case let (x, y):
    print("(\(x), \(y)) is just some arbitrary point")
}
// Prints "(1, -1) is on the line x == -y".
```



`switch` 구문은 `x == y`인 초록색 대각선 위에 있는지, `x == -y`인 보라색 대각선 위에 있거나 아니면 그 외에 위치하는지 판단합니다.

세 개의 `switch` 케이스는 자리 표시자 상수 `x`와 `y`를 선언하며, 이들은 `yetAnotherPoint` 튜플의 두 값을 임시로 저장합니다. 이 상수는 `where` 절의 일부로 사용되어 동적인 필터를 생성합니다. `switch` 케이스는 `where` 절의 조건이 `true`일 때만 현재 `point` 값을 일치시킵니다.

이전 예시와 마찬가지로 마지막 케이스는 가능한 남아있는 값과 모두 일치하므로, `switch` 구문을 완벽하게 하기 위해 `default` 케이스가 필요치 않습니다.

합성 케이스 (Compound Cases)

같은 본문을 공유하는 여러 개의 스위치 케이스는 `case` 다음에 각 패턴 사이에 콤마로 구분하여 여러 개의 패턴을 결합할 수 있습니다. 여러 패턴 중 어떤 패턴이 일치하면 해당 케이스로 고려됩니다. 패턴이 길면 여러줄로 작성할 수 있습니다. 예를 들어:

```
let someCharacter: Character = "e"
switch someCharacter {
case "a", "e", "i", "o", "u":
    print("\(someCharacter) is a vowel")
case "b", "c", "d", "f", "g", "h", "j", "k", "l", "m",
     "n", "p", "q", "r", "s", "t", "v", "w", "x", "y", "z":
```

```
    print("\(someCharacter) is a consonant")
default:
    print("\(someCharacter) is not a vowel or a consonant")
}
// Prints "e is a vowel".
```

`switch` 구문의 첫 번째 케이스는 영어의 다섯 개의 소문자 모음이 일치합니다. 비슷하게 두 번째 케이스는 모든 영어 소문자 자음이 일치합니다. 마지막으로 `default` 케이스는 다른 어떠한 문자와 일치합니다.

혼합 케이스(Compound cases)는 값 바인딩을 포함할 수도 있습니다. 혼합 케이스의 모든 패턴은 값 바인딩의 같은 집합을 포함해야 하고 각 바인딩은 혼합 케이스에 모든 패턴으로 부터 같은 타입의 값을 얻어야 합니다. 혼합 케이스 부분이 일치하는 것과 상관없이 케이스의 본문에 코드는 바인딩을 위해 값에 항상 접근할 수 있고 값은 항상 같은 타입을 가집니다.

```
let stillAnotherPoint = (9, 0)
switch stillAnotherPoint {
case (let distance, 0), (0, let distance):
    print("On an axis, \(distance) from the origin")
default:
    print("Not on an axis")
}
// Prints "On an axis, 9 from the origin".
```

위의 `case`는 두 개의 패턴을 가지고 있습니다: `(let distance, 0)`은 x축 위의 좌표와 일치하고 `(0, let distance)`는 y축 위의 좌표와 일치합니다. 두 패턴 모두 `distance` 바인딩을 포함하고 `distance`는 두 패턴 모두 정수입니다 - 즉, `case`의 본문 안에 코드는 항상 `distance`로 값에 접근할 수 있다는 의미입니다.

패턴 (Patterns)

이전 예시에서 각 스위치 케이스는 어떤 값이 해당 케이스에 일치하는지 나타내는 패턴을 포함합니다. `if` 구문의 조건으로도 패턴을 사용할 수 있습니다. 다음은 그 예시입니다:

```
let somePoint = (12, 100)
if case (let x, 100) = somePoint {
    print("Found a point on the y=100 line, at \(x)")
}
// Prints "Found a point on the y=100 line, at 12".
```

이 코드에서 `if` 구문의 조건은 `case`로 시작하고 조건이 Boolean 값 대신 패턴으로 나타냅니다. 패턴이 일치하면, `if` 구문의 조건은 참이라고 여겨지며, `if` 구문의 본문 코드를 실행합니다. `if case` 다음에 작성할 수 있는 패턴은 스위치 케이스에 작성할 수 있는 패턴과 동일합니다.

`for-in` 루프에서는 `case`를 작성하지 않고도 값 바인딩 패턴을 사용하여 값의 부분으로 이름을 줄 수 있습니다:

```
let points = [(10, 0), (30, -30), (-20, 0)]

for (x, y) in points {
    if y == 0 {
        print("Found a point on the x-axis at \(x)")
    }
}
// Prints "Found a point on the x-axis at 10".
// Prints "Found a point on the x-axis at -20".
```

위 `for-in` 루프는 튜플의 배열을 반복하며, 튜플의 첫 번째와 두 번째 요소를 `x`와 `y` 상수에 바인딩합니다. 루프 내의 구문은 점이 x축 위에 있는지 확인하는 `if` 구문과 같이 이 상수를 사용할 수 있습니다. 이 코드를 작성하는 더 간결한 방법은 `for-case-in` 루프를 사용하여 값 바인딩과 조건을 결합하는 것입니다. 아래 코드는 위의 `for-in` 루프와 동일한 동작을 하는 코드입니다:

```
for case (let x, 0) in points {
    print("Found a point on the x-axis at \(x)")
}
// Prints "Found a point on the x-axis at 10".
// Prints "Found a point on the x-axis at -20".
```

이 코드에서 조건은 패턴의 일부로 `for-case-in` 루프에 결합됩니다. `for-case-in` 루프 내의 구문은 x축 위에 있는 점일 때만 실행됩니다. 이 코드는 위의 `for-in` 루프와 동일한 결과를 생성하지만, 컬렉션의 특정 요소만 반복하는 더 간결한 방법입니다.

`for-case-in` 루프는 추가 조건을 확인하는 `where` 절을 추가할 수도 있습니다. 루프 내의 구문은 현재 요소가 `where` 절과 일치할 때만 실행됩니다. 예를 들어:

```
for case let (x, y) in points where x == y || x == -y  {
    print("Found (\(x), \(y)) along a line through the origin")
}
// Prints "Found (30, -30) along a line through the origin".
```

이 코드는 튜플의 첫 번째와 두 번째 요소를 `x`와 `y` 상수로 바인딩한 다음에 해당 값을 `where` 절에서 확인합니다. `where` 절이 `true`이면, `for` 루프의 본문 내 코드를 실행합니다; 그렇지 않으면 다음 요소로 반복이 계속됩니다.

패턴은 값을 바인딩할 수 있으므로, `if-case` 구문과 `for-case-in` 루프는 연관값 (Associated Values)에서 설명한대로 연관값을 가진 열거형과 작업하는데 유용합니다.

제어 전송 구문 (Control Transfer Statements)

*제어 전송 구문(Control transfer statements)*은 한 코드에서 다른 코드로 제어를 변경하여 코드가 실행되는 순서를 변경합니다. Swift는 다섯 개의 제어 전송 구문이 있습니다:

- `continue`
- `break`
- `fallthrough`
- `return`
- `throw`

`continue`, `break`, `fallthrough` 구문은 아래 설명되어 있습니다. `return` 구문은 함수 (Functions)에 설명되어 있고, `throw` 구문은 던지는 함수를 이용한 오류 전파 (Propagating Errors Using Throwing Functions)에 설명되어 있습니다.

계속 (Continue)

`continue` 구문은 현재 수행 중인 작업을 중단하고, 루프의 다음 반복을 시작하도록 지시합니다. 이것은 "현재 루프 반복을 끝냈다."라고 선언하지만, 루프 자체를 완전히 종료하지는 않습니다.

아래 예시는 비밀의 퍼즐 구문을 생성하기 위해 소문자 문자열에서 모음과 공백을 삭제합니다:

```
let puzzleInput = "great minds think alike"
var puzzleOutput = ""
let charactersToRemove: [Character] = ["a", "e", "i", "o", "u", " "]
for character in puzzleInput {
    if charactersToRemove.contains(character) {
        continue
    }
    puzzleOutput.append(character)
}
print(puzzleOutput)
// Prints "grtmndsthnklk".
```

위 코드는 모음이나 공백이 일치하면, 현재 반복의 루프를 종료하고 즉시 다음 반복을 시작하기 위해 `continue` 키워드를 호출합니다.

중단 (Break)

`break` 구문은 전체 제어 흐름 구문을 즉시 종료합니다. `break` 구문은 `switch` 내부나 루프 구문에서 `switch`나 루프 구문을 다른 경우보다 일찍 종료시킬 때 사용될 수 있습니다.

루프 구문에서 중단 (Break in a Loop Statement)

루프 구문 내에서 사용할 때, `break`는 루프의 실행을 즉시 종료하고 제어를 루프의 닫기 중괄호(}) 다음으로 이전합니다. 루프의 현재 반복으로 부터 코드는 더이상 실행되지 않고, 루프의 반복은 더 이상 시작되지 않습니다.

Switch 구문에서 중단 (Break in a Switch Statement)

`switch` 구문 내에서 사용할 때, `break`는 `switch` 구문을 즉시 종료하고 제어를 `switch` 구문의 닫힌 중괄호(}) 다음으로 이동시킵니다.

이러한 동작은 일치할 때 사용될 수 있고 `switch` 구문에서 하나 이상의 케이스를 무시할 때 사용됩니다. Swift의 `switch` 구문은 완벽하고 빈 케이스를 허락하지 않기 때문에, 의도를 명시하기 위해 의도적으로 케이스를 일치시키고 무시해야 하는 경우가 있습니다. 무시할 케이스의 전체 본문으로 `break` 구문을 작성하여 이를 수행합니다. 해당 케이스가 `switch` 구문과 일치하면 케이스 내부의 `break` 구문이 `switch` 구문의 실행을 즉시 종료합니다.

> **참고**
>
> `switch` 케이스에 주석만 포함되어 있으면 컴파일 시 오류가 발생합니다. 주석은 구문이 아니며 `switch` 케이스는 무시되지 않습니다. 항상 `switch` 케이스를 무시하려면 `break` 구문을 사용하십시오.

아래 예시는 `Character` 값을 바꾸고 네 언어중 하나의 언어로 숫자 기호를 표기하는지 판단합니다. 간결함을 위해 단일 `switch` 케이스에 여러 값이 포함됩니다.

```
let numberSymbol: Character = "三"  // Chinese symbol for the number 3
var possibleIntegerValue: Int?
switch numberSymbol {
case "1", "١", "一", "๑":
    possibleIntegerValue = 1
case "2", "٢", "二", "๒":
    possibleIntegerValue = 2
case "3", "٣", "三", "๓":
    possibleIntegerValue = 3
case "4", "٤", "四", "๔":
    possibleIntegerValue = 4
default:
    break
}
if let integerValue = possibleIntegerValue {
    print("The integer value of \(numberSymbol) is \(integerValue).")
} else {
    print("An integer value could not be found for \(numberSymbol).")
}
// Prints "The integer value of 三 is 3.".
```

이 예시는 `numberSymbol`이 1부터 4의 숫자 기호가 라틴어, 아랍어, 중국어, 태국어 인지 판단합니다. 일치하는 것을 찾으면, `switch` 구문의 케이스 중에 하나는 적절한 정수값을 `possibleIntegerValue`라 불리는 옵셔널 `Int?` 변수에 할당합니다.

`switch` 구문이 실행을 완료한 후에 이 예시에서는 옵셔널 바인딩을 사용하여 값이 존재하는지 확인합니다. `possibleIntegerValue` 변수는 옵셔널 타입이기 때문에 초기값은 `nil`을 가지며, 옵셔널 바인딩이 성공하려면 `switch` 구문의 케이스 중 하나에서 실제 값이 할당되어야 합니다.

위 예시에서 가능한 모든 `Character` 값을 리스트화 할 수 없기 때문에 `default` 케이스로 일치되지 않는 문자를 처리합니다. `default` 케이스는 어떠한 동작도 수행할 필요가 없으므로, `break` 구문만 적습니다. `default` 케이스에 일치되자마자, `break` 구문은 `switch` 구문의 실행을 종료하고, 이어서 `if let` 구문을 실행합니다.

Fallthrough

Swift에서 `switch` 구문은 각 케이스의 맨 아래에서 다음 케이스로 넘어가지 않습니다. 첫 번째 케이스가 일치하자마자 `switch` 구문의 실행은 완료됩니다. 반대로 C는 다음 케이스로 넘어가는 것을 막기 위해 모든 `switch` 케이스 마지막에 명시적으로 `break` 구문을 명시적으로 넣어야 합니다. 기본적으로 다음 케이스로 넘어가지 않는다는 것은 Swift의 `switch` 구문이 C의 대응문 보다 훨씬 간결하고 예측 가능하다는 것을 의미하므로 실수로 여러 `switch` 케이스를 실행하지 않습니다.

C처럼 다음 케이스로 넘어가려면 `fallthrough` 키워드로 케이스 별로 동작을 선택할 수 있습니다. 아래 예시에서 `fallthrough`를 사용하여 숫자의 설명을 생성합니다.

```
let integerToDescribe = 5
var description = "The number \(integerToDescribe) is"
switch integerToDescribe {
case 2, 3, 5, 7, 11, 13, 17, 19:
    description += " a prime number, and also"
    fallthrough
default:
    description += " an integer."
}
print(description)
// Prints "The number 5 is a prime number, and also an integer.".
```

이 예시는 `description`이라 불리는 새로운 `String` 변수를 선언하고 초기값을 할당합니다. 이 함수는 `switch` 구문을 사용하여 `integerToDescribe`의 값을 고려합니다. `integerToDescribe`의 값이 리스트에서의 소수 중 하나이면 소수라는 것을 나타내는 텍스트를 `description` 뒤에 추가합니다. 그러면 `fallthrough` 키워드를 사용하여 `default` 케이스 또한 동작하게 됩니다. `default` 케이스는 추가 설명을 덧붙이고, `switch` 구문은 완료됩니다.

`integerToDescribe`의 값이 리스트에 없는 소수이면 첫 번째 `switch` 케이스와 전혀 일치하지 않습니다. 특정 케이스가 없기 때문에 `integerToDescribe`는 `default` 케이스와 일치됩니다.

`switch` 구문 실행이 완료된 후에 숫자의 설명은 `print(_:separator:terminator:)` 함수를 사용하여 출력됩니다. 이 예시에서 숫자 5는 소수로써 유효합니다.

> **참고**
>
> `fallthrough` 키워드는 `switch` 케이스 실행을 위한 케이스 조건을 확인하지 않습니다. `fallthrough` 키워드는 간단하게 C의 표준 `switch` 구문 동작처럼 다음 케이스(또는 `default` 케이스)로 코드 실행을 직접적으로 이동시킵니다.

레이블이 있는 구문 (Labeled Statements)

Swift에서는 루프문과 조건문을 다른 루프문이나 조건문 안에 중첩하여 복잡한 제어 흐름 구조를 만들 수 있습니다. 그러나 루프문과 조건문은 모두 `break` 구문을 사용하여 실행을 조기에 종료할 수 있습니다. 따라서 `break` 구문이 종료할 대상이 되는 루프문이나 조건문을 명확하게 지정하는 것이 유용할 때가 있습니다. 마찬가지로 여러 개의 중첩된 루프문을 가지고 있다면, `continue` 구문이 영향을 미칠 루프를 명확하게 지정하는 것도 유용할 수 있습니다.

이러한 목적을 달성하기 위해, 루프문이나 조건문에 *구문 레이블(statement label)*을 사용할 수 있습니다. 조건문에서 레이블이 있는 구문에 실행을 종료하기 위해 `break` 구문과 구문 레이블을 사용할 수 있습니다. 루프문에서 레이블이 있는 구문에 실행을 종료하거나 이어서 진행하기 위해 `break`나 `continue` 구문과 함께 구문 레이블을 사용할 수 있습니다.

레이블이 있는 구문(labeled statement)을 나타내는 키워드와 같은 줄에 위치하고, 바로 다음에 콜론을 붙여 표시합니다. 다음은 `while` 루프에서 이 구문을 사용하는 예시이며, 동일한 원리가 모든 루프 및 `switch` 구문에도 적용됩니다:

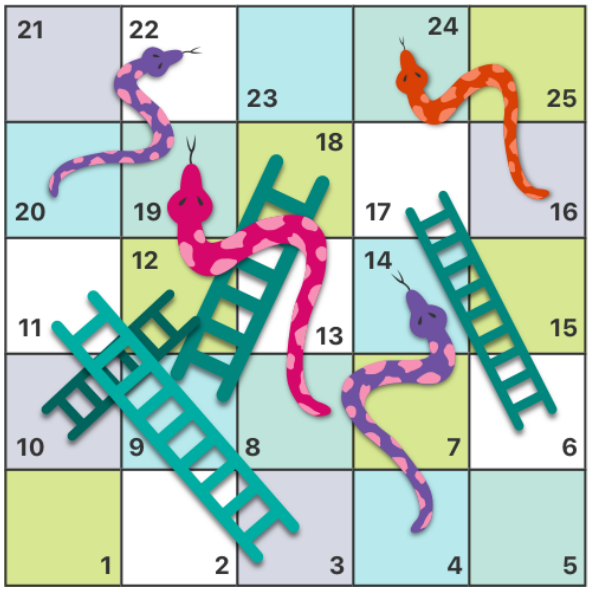
```
<#label name#>: while <#condition#> {
   <#statements#>
}
```

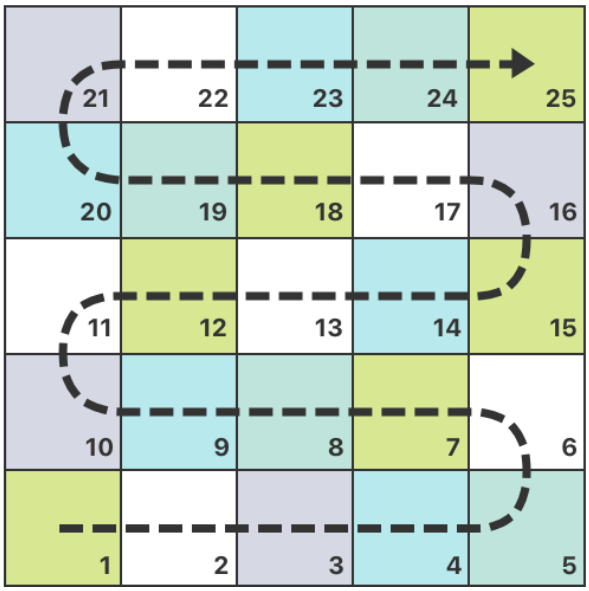
다음 예시는 이 챕터의 이전에 봤던 Snakes and Ladders 게임에 적절한 버전을 위해 레이블이 있는 `while` 루프와 `break`와 `continue` 구문을 사용합니다. 이번에는 게임에 추가 규칙이 있습니다:

- 이기려면 정사각형 25에 정확히 위치해야 합니다.

주사위를 굴려 정사각형 25를 넘어간다면, 정확히 정사각형 25에 위치할 때까지 주사위를 굴려야 합니다.

게임 보드는 이전과 동일합니다.





`finalSquare`, `board`, `square`, `diceRoll`은 이전과 동일하게 초기화 됩니다:

```
let finalSquare = 25
var board = [Int](repeating: 0, count: finalSquare + 1)
board[03] = +08; board[06] = +11; board[09] = +09; board[10] = +02
board[14] = −10; board[19] = −11; board[22] = −02; board[24] = −08
var square = 0
var diceRoll = 0
```

이 버전의 게임은 `while` 루프와 `switch` 구문을 사용하여 게임의 로직을 구현합니다. `while` 루프에는 `gameLoop` 라는 구문 레이블이 붙어 있으며, 이는 해당 루프가 Snakes and Ladders 게임의 메인 게임 루프임을 나타냅니다.

`while` 루프의 조건은 정사각형 25에 정확하게 착지를 위한 조건을 반영한 `while square != finalSquare`입니다.

```
gameLoop: while square != finalSquare {
    diceRoll += 1
    if diceRoll == 7 { diceRoll = 1 }
    switch square + diceRoll {
    case finalSquare:
        // diceRoll will move us to the final square, so the game is over
        break gameLoop
    case let newSquare where newSquare > finalSquare:
        // diceRoll will move us beyond the final square, so roll again
        continue gameLoop
    default:
        // this is a valid move, so find out its effect
        square += diceRoll
        square += board[square]
    }
}
print("Game over!")
```

각 루프가 시작될 때 주사위는 굴려집니다. 즉시 플레이어를 이동하는 것보다, 루프는 이동의 결과를 고려하고 이동이 가능한지 판단하기 위해 `switch` 구문을 사용합니다:

- 주사위 굴림이 플레이어를 마지막 정사각형으로 이동시킨다면 게임은 종료됩니다. `break gameLoop` 구문은 게임 종료를 위해 `while` 루프의 바깥으로 제어를 이동합니다.

- 주사위 굴림이 플레이어를 마지막 정사각형을 넘어 이동시킨다면, 이 이동은 유효하지 않고 플레이어는 주사위를 다시 굴려야 합니다. `continue gameLoop` 구문은 현재 `while` 루프 반복을 종료하고 루프의 다음 반복을 시작합니다.
- 다른 모든 케이스에 해당하는 주사위 굴림은 유효합니다. 플레이어는 `diceRoll`만큼 이동하고, 게임 로직은 뱀과 사다리에 대해 확인합니다. 그러면 루프는 종료되고, 다음 턴이 요구되는지 결정하기 위해 `while` 조건으로 되돌아 갑니다.

참고

위에서 `break` 구문이 `gameLoop` 레이블을 사용하지 않았다면, `while` 구문을 빠져나오는 것이 아니라, `switch` 구문을 빠져나옵니다. `gameLoop` 레이블을 사용하면 어떤 제어문을 종료해야 하는지 명확해집니다.

루프의 다음 반복으로 이동하기위해 `continue gameLoop` 호출할 때 반드시 `gameLoop` 레이블을 사용할 필요는 않습니다. 게임에서 오직 하나의 루프만 존재하므로, `continue` 구문이 영향을 미칠 루프가 명확하기 때문입니다. 그러나 `continue` 구문에서 `gameLoop` 레이블을 사용해도 아무런 문제가 없습니다. 이렇게 하면 `break` 구문과의 일관성을 유지할 수 있으며, 게임의 로직을 더 읽기 쉽고 이해하기 쉽게 만듭니다.

이른 종료 (Early Exit)

`if` 구문처럼 `guard` 구문은 표현식의 Boolean 값에 따라 특정 구문을 실행합니다. `guard` 구문은 특정 조건이 참이어야만 `guard` 구문 이후의 코드가 실행되도록 요구하는 데 사용됩니다. `if` 구문과 다르게, `guard` 구문은 항상 `else` 절을 가지고 있습니다 - `else` 절 안의 코드는 조건이 참이 아닐 경우에 실행됩니다.

```
func greet(person: [String: String]) {
    guard let name = person["name"] else {
        return
    }

    print("Hello \(name)!")

    guard let location = person["location"] else {
        print("I hope the weather is nice near you.")
        return
    }

    print("I hope the weather is nice in \(location).")
}

greet(person: ["name": "John"])
// Prints "Hello John!".
// Prints "I hope the weather is nice near you.".
greet(person: ["name": "Jane", "location": "Cupertino"])
// Prints "Hello Jane!".
// Prints "I hope the weather is nice in Cupertino.".
```

`guard` 구문의 조건이 충족되면, 코드 실행은 `guard` 구문의 닫는 중괄호 다음으로 이어집니다. 조건의 일부로 옵셔널 바인딩을 사용하여 값이 할당된 모든 변수나 상수는 `guard` 문이 존재하는 나머지 코드 블록에서 사용할 수 있습니다.

조건이 충족되지 않으면, `else` 안의 코드가 실행됩니다. 이 분기는 반드시 제어 흐름을 전환하여 `guard` 구문이 포함된 코드 블록을 빠져나가야 합니다. 이를 위해 `return`, `break`, `continue`, `throw`와 같은 제어 전달 문을 사용할 수 있으며, 또는 `fatalError(_:file:line:)`처럼 반환되지 않는 함수나 메서드를 호출할 수도 있습니다.

guard 구문을 사용하면 동일한 조건 검사를 if 구문으로 수행하는 것보다 코드의 가독성이 향상됩니다. 이를 통해 일반적으로 실행되는 코드를 else 블록으로 래핑하지 않고 작성할 수 있으며 요구에 충족하지 않는 부분을 요구사항 근처에서 처리할 수 있도록 해줍니다.

연기된 동작 (Deferred Actions)

if와 while과 같은 제어 흐름 구조는 코드의 일부분이 실행되거나 실행 횟수를 제어할 수 있는 반면에, defer 구문은 특정 코드가 언제 실행될지를 제어합니다. 현재 범위의 마지막에 실행되는 코드 블록을 작성하기 위해 defer 블록을 사용합니다. 예를 들어:

```
var score = 1
if score < 10 {
    defer {
        print(score)
    }
    score += 5
}
// Prints "6".
```

위의 예시에서 defer 블록 안의 코드는 if 구문의 본문이 종료되기 전에 실행됩니다. 먼저 score를 5 증가시키는 if 구문의 코드가 수행됩니다. 그런 다음 if 구문의 범위가 종료되기 전에 score를 출력하는 코드가 수행됩니다.

프로그램이 어떻게 종료되든 defer 안의 코드는 항상 수행됩니다. 여기에는 함수에서 조기에 반환되거나, for 루프를 빠져나가거나, 오류를 발생시키는 경우에도 실행됩니다. 이러한 동작 방식 덕분에 defer는 특정 작업의 쌍을 보장해야 하는 경우에 유용합니다 - 수동으로 메모리를 할당하고 해제하는 경우, 저수준 파일 디스크립터를 열고 닫는 경우, 데이터베이스에서 트랜잭션을 시작하고 종료와 같은 동작에서 활용할 수 있습니다 - 두 동작을 코드에서 나란히 작성할 수 있기 때문입니다. 예를 들어, 다음의 코드는 점수에 100을 더하고 뺌으로써 임시로 보너스를 주는 코드입니다:

```
var score = 3
if score < 100 {
    score += 100
    defer {
        score -= 100
    }
    // Other code that uses the score with its bonus goes here.
    print(score)
}
// Prints "103".
```

같은 범위에 하나 이상의 defer 블록을 작성하면, 첫 번째 defer 블록은 마지막에 수행됩니다.

```
if score < 10 {
    defer {
        print(score)
    }
    defer {
        print("The score is:")
    }
    score += 5
}
// Prints "The score is:".
// Prints "6".
```

프로그램이 실행을 중지하면 - 예를 들어, 런타임 오류 또는 충돌로 인해 - 지연된 코드가 실행되지 않습니다. 그러나 오류가 발생한 후에는 지연된 코드가 실행됩니다; `defer`를 오류 처리와 함께 사용하는 방법에 대한 자세한 내용은 정리 작업 지정 (Specifying Cleanup Actions)을 참고바랍니다.

API 가용성 검사 (Checking API Availability)

Swift는 API 가용성 검사(checking API availability)를 기본적으로 지원하여, 특정 배포 타겟에서 사용할 수 없는 API를 실수로 사용하는 것을 방지합니다.

컴파일러는 SDK의 가용성 정보를 사용하여 코드에서 사용하는 모든 API가 프로젝트에서 지정한 배포 대상에서 사용할 수 있는지 확인합니다. Swift는 사용할 수 없는 API를 사용하려고 하면 컴파일 시 오류를 발생합니다.

`if` 또는 `guard` 구문에서 *가용성 조건(availability condition)*을 사용하여, 런타임에 원하는 API가 사용 가능한지 여부에 따라 코드 블록을 조건부로 실행할 수 있습니다. 컴파일러는 해당 코드 블록 내 API의 가용성을 확인할 때 가용성 조건에서 제공된 정보를 사용합니다.

```
if #available(iOS 10, macOS 10.12, *) {
    // Use iOS 10 APIs on iOS, and use macOS 10.12 APIs on macOS
} else {
    // Fall back to earlier iOS and macOS APIs
}
```

iOS에서 위의 특정 가용성 조건의 `if` 구문의 본문은 iOS 10 이상 버전에서만 실행되고; macOS에서는 macOS 10.12 이상 버전에서만 실행됩니다. 마지막 인자 `*`는 필수이며 다른 플랫폼에서 `if`의 본문이 최소 타겟 버전에서 실행되도록 지정합니다.

일반적인 형식에서 가용성 조건은 플랫폼 이름 및 버전 목록을 포함합니다. `iOS`, `macOS`, `watchOS`, `tvOS`, `visionOS`와 같은 플랫폼 이름을 사용하고 - 자세한 내용은 선언 속성 (Declaration Attributes)을 참고바랍니다. iOS 8 또는 macOS 10.10과 같은 메이저 버전 번호를 지정하는 것 외에 iOS 11.2.6과 macOS 10.13.3과 같은 마이너 버전 번호로 지정할 수 있습니다.

```
if #available(<#platform name#> <#version#>, <#...#>, *) {
    <#statements to execute if the APIs are available#>
} else {
    <#fallback statements to execute if the APIs are unavailable#>
}
```

`guard` 구문과 함께 가용성 조건을 사용하면, 해당 코드 블록 내에서 나머지 코드에 적용되는 가용성 정보가 더 구체화됩니다.

```
@available(macOS 10.12, *)
struct ColorPreference {
    var bestColor = "blue"
}

func chooseBestColor() -> String {
    guard #available(macOS 10.12, *) else {
        return "gray"
    }
    let colors = ColorPreference()
    return colors.bestColor
}
```

위의 예시에서 `ColorPreference` 구조체는 macOS 10.12 이상을 요구합니다. `chooseBestColor()` 함수는 가용성 가드로 시작합니다. 플랫폼 버전이 `ColorPreference`를 사용하기에 너무 예전 버전이면, 항상 사용 가능한 동작으로 대체됩니다. `guard` 구문 이후는 macOS 10.12 이상 버전을 요구하는 API를 사용할 수 있습니다.

`#available` 이외에도 Swift는 비가용성 조건을 사용하여 반대로 확인할 수도 있습니다. 예를 들어, 아래의 내용은 동일한 내용을 확인합니다:

```
if #available(iOS 10, *) {
} else {
    // Fallback code
}

if #unavailable(iOS 10) {
    // Fallback code
}
```

`#unavailable`를 사용하는 형식은 대체 코드만을 포함할 때 더 읽기 쉬운 코드로 만들어 줍니다.

언어 가이드

함수 (Functions)

함수를 정의하고 호출하며, 인자에 레이블을 지정하고 반환값을 사용합니다.

*함수(Functions)*는 특정 작업을 수행하는 독립적인 코드 블록입니다. 무슨 동작을 하는지 함수에 특정 이름을 부여할 수 있으며, 작업을 수행하기 위해 함수를 "호출"할 때 사용합니다.

Swift의 통합된 함수 문법은 매개변수 이름이 없는 단순한 C 스타일 함수에서 각 매개변수에 대한 이름과 인자 레이블이 있는 복잡한 Objective-C 스타일 메서드에 이르기까지 모든 것을 표현할 수 있을 만큼 유연합니다. 매개변수는 함수 호출을 단순화 하기위해 기본값을 제공할 수 있으며 함수가 실행을 완료하면 전달된 변수를 수정하는 in-out 매개변수로 전달할 수도 있습니다.

Swift의 모든 함수는 매개변수 타입과 반환 타입으로 구성된 함수 타입을 가집니다. Swift에서는 함수 타입을 다른 타입처럼 사용할 수 있으므로, 함수를 다른 함수의 매개변수로 전달하거나 함수에서 함수를 반환하는 것이 가능합니다. 함수 내부에 또 다른 함수를 정의하여 특정 기능을 중첩 함수의 형태로 캡슐화할 수도 있습니다.

함수 정의와 호출 (Defining and Calling Functions)

함수를 정의할 때, *매개변수(parameter)*라고 알고 있는 선택적으로 하나 이상의 이름이 지정된 값과 타입이 있는 입력값을 정의할 수 있습니다. 또한 *반환 타입(return type)*이라고 알고 있는 작업이 완료 되었을 때 반환할 값의 타입을 선택적으로 정의할 수 있습니다.

모든 함수는 함수가 수행하는 작업을 설명하는 함수 이름을 가지고 있습니다. 함수를 사용하려면, 함수의 이름으로 "호출"하고 함수의 매개변수 타입과 일치하는 입력값(인자(argument))을 전달해야 합니다. 함수의 인자는 항상 함수의 매개변수 순서와 동일하게 제공해야 합니다.

아래 예시의 함수는 `greet(person:)`이라는 이름을 가지고, 사람의 이름을 입력으로 받아, 해당 사람에게 인사말을 반환하는 역할을 합니다. 이것을 수행하기 위해 `person`이라는 `String` 타입의 매개변수와 그 사람의 인사말을 포함하는 `String` 반환 타입을 정의합니다:

```
func greet(person: String) -> String {
    let greeting = "Hello, " + person + "!"
    return greeting
}
```

이 정보의 모든 것은 `func` 키워드를 앞에 붙여 함수를 정의합니다. 반환 화살표 `->` (하이픈 뒤에 오른쪽 방향 꺾쇠를 붙임) 뒤에 반환 타입의 이름을 붙여 함수의 반환 타입을 나타냅니다.

함수 정의는 함수가 수행하는 작업, 받아야 할 입력값, 완료 후 반환하는 값을 설명합니다. 이러한 정의를 통해 코드의 다른 위치에서도 함수를 명확하게 호출할 수 있습니다.

```
print(greet(person: "Anna"))
// Prints "Hello, Anna!".
print(greet(person: "Brian"))
// Prints "Hello, Brian!".
```

`greet(person: "Anna")`와 같이 `person` 인자 레이블 뒤에 `String` 값을 전달하여 `greet(person:)` 함수를 호출합니다. 함수는 `String` 값을 반환하므로, `greet(person:)`은 위에서와 같이 반환값의 문자열을 출력하기 위해 `print(_:separator:terminator:)` 함수 안에서 호출할 수 있습니다.

> **참고**
>
> `print(_:separator:terminator:)` 함수는 첫 번째 인자가 레이블을 가지고 있지 않고, 다른 인자는 기본값을 가지고 있으므로 생략할 수 있습니다. 이러한 함수 구문의 변형은 아래 함수 인자 레이블과 매개변수 이름 (Function Argument Labels and Parameter Names)과 매개변수 기본값 (Default Parameter Values)에서 자세히 설명합니다.

`greet(person:)` 함수의 본문은 `greeting`이라는 `String` 타입의 새로운 상수를 정의하고, 간단한 인사말 메세지 설정합니다. 이 인사말은 `return` 키워드를 사용하여 함수의 반환값으로 전달됩니다. `return greeting`이라는 코드 줄에서 함수는 실행을 완료하고 `greeting`의 현재값을 반환합니다.

다른 입력값으로 `greet(person:)` 함수를 여러 번 호출할 수 있습니다. 위의 예시에서 `"Anna"`와 `"Brian"`의 입력값으로 호출할 경우 어떤 일이 생기는지 보여줍니다. 이 함수는 각 경우에 맞춰 인사말을 반환합니다.

이 함수의 본문을 더 간결하게 만들기 위해 메세지 생성과 반환 구문을 한 줄로 결합할 수 있습니다:

```
func greetAgain(person: String) -> String {
    return "Hello again, " + person + "!"
}
print(greetAgain(person: "Anna"))
// Prints "Hello again, Anna!".
```

함수 매개변수와 반환값 (Function Parameters and Return Values)

함수 매개변수와 반환값(Function parameters and return values)은 Swift에서 매우 유연합니다. 이름이 지정되지 않은 단일 매개변수가 있는 간단한 유틸리티 함수에서 매개변수 이름과 다른 매개변수 옵션이 있는 복잡한 함수에 이르기까지 모든 것을 정의할 수 있습니다.

매개변수 없는 함수 (Functions Without Parameters)

함수는 입력 매개변수 정의를 요구하지 않습니다. 여기 호출될 때마다 항상 같은 `String` 메세지를 반환하는 입력 매개변수가 없는 함수가 있습니다:

```
func sayHelloWorld() -> String {
    return "hello, world"
}
print(sayHelloWorld())
// Prints "hello, world".
```

함수 정의는 어떠한 매개변수가 없더라도 함수의 이름 뒤에 소괄호가 필요합니다. 함수 이름은 함수가 호출될 때 빈 소괄호가 뒤따릅니다.

여러 개 매개변수가 있는 함수 (Functions With Multiple Parameters)

함수는 함수의 소괄호 내에 콤마로 구분하여 여러 개의 입력 매개변수를 가질 수 있습니다.

이 함수는 사람의 이름과 이미 인사를 받았는지를 입력으로 받아, 그 사람에게 적절한 인사말을 반환합니다:

```
func greet(person: String, alreadyGreeted: Bool) -> String {
    if alreadyGreeted {
        return greetAgain(person: person)
    } else {
        return greet(person: person)
    }
}
print(greet(person: "Tim", alreadyGreeted: true))
// Prints "Hello again, Tim!".
```

`person`이라는 레이블을 가진 `String` 타입의 인자값과 `alreadyGreeted`이라는 레이블을 가진 `Bool` 타입의 인자값을 소괄호 안에 콤마로 구분하여 `greet(person:alreadyGreeted:)` 함수로 전달하여 호출합니다. 이전 섹션에서 본 `greet(person:)` 함수와는 다른 함수인 것을 명심해야 합니다. 두 함수 모두 `greet`으로 시작하는 이름을 가지지만, `greet(person:alreadyGreeted:)` 함수는 두 개의 인자를 가지지만 `greet(person:)` 함수는 오직 하나의 인자를 가집니다.

반환값 없는 함수 (Functions Without Return Values)

함수는 반환 타입 정의를 요구하지 않습니다. 다음은 반환하지 않고 `String` 타입의 값을 출력하는 `greet(person:)` 함수 버전입니다:

```
func greet(person: String) {
    print("Hello, \(person)!")
}
greet(person: "Dave")
// Prints "Hello, Dave!".
```

반환값이 필요하지 않기 때문에, 함수의 정의는 반환 화살표(->)나 반환 타입을 포함하지 않습니다.

> **참고**
>
> 엄밀히 말하면, `greet(person:)` 함수는 반환값을 정의하지 않았지만, 여전히 반환값이 있습니다. 반환 타입이 정의되지 않은 함수는 `Void` 타입의 특별한 값을 반환합니다. 이것은 `()`로 쓰여진 빈 튜플입니다.

함수의 반환값은 호출될 때 무시될 수 있습니다:

```
func printAndCount(string: String) -> Int {
    print(string)
    return string.count
}
func printWithoutCounting(string: String) {
    let _ = printAndCount(string: string)
}
printAndCount(string: "hello, world")
// Prints "hello, world". and returns a value of 12
printWithoutCounting(string: "hello, world")
// Prints "hello, world". but does not return a value
```

첫 번째 함수 `printAndCount(string:)`은 문자열을 출력하고, 문자 갯수를 `Int`로 반환합니다. 두 번째 함수 `printWithoutCounting(string:)`은 첫 번째 함수를 호출하지만 반환값을 무시합니다. 두 번째 함수가 호출될 때, 첫 번째 함수에 의해 메세지는 출력되지만 반환된 값을 사용하지 않습니다.

참고

반환값은 무시할 수 있지만, 함수는 항상 값을 반환할 것입니다. 반환 타입이 있는 함수는 반환되는 값 없이 함수의 끝에서 값을 반환하지 않고 제어가 빠져나가게 해서는 안되며, 그렇게 시도할 경우 컴파일 시 오류가 발생합니다.

여러 개의 반환값이 있는 함수 (Functions with Multiple Return Values)

튜플 타입을 함수의 반환 타입으로 사용하면 여러 값을 하나의 복합 반환값으로 반환할 수 있습니다.

아래 예시는 `Int` 값의 배열에서 가장 작은 값과 가장 큰 값을 찾는 `minMax(array:)` 함수를 정의합니다:

```
func minMax(array: [Int]) -> (min: Int, max: Int) {
    var currentMin = array[0]
    var currentMax = array[0]
    for value in array[1..<array.count] {
        if value < currentMin {
            currentMin = value
        } else if value > currentMax {
            currentMax = value
        }
    }
    return (currentMin, currentMax)
}
```

`minMax(array:)` 함수는 두 개의 `Int` 값을 포함한 튜플을 반환합니다. 이 값들은 함수의 반환값을 조회할 때 이름으로 접근할 수 있도록 `min`과 `max`로 레이블되어 있습니다.

`minMax(array:)` 함수의 본문은 배열의 첫 번째 정수의 값을 `currentMin`과 `currentMax`라 불리는 두 개의 동작 변수에 값을 설정하는 것으로 시작합니다. 그러면 함수는 배열의 나머지 값들을 반복하고 각 값이 `currentMin`과 `currentMax`의 값보다 더 작거나 더 큰지 확인합니다. 마지막으로 가장 작고 가장 큰 값은 두 개의 `Int` 값인 튜플로 반환됩니다.

튜플의 멤버 값은 함수의 반환 타입이므로 가장 작은 값과 가장 큰 값을 찾기 위해 점 문법으로 접근할 수 있습니다:

```
let bounds = minMax(array: [8, -6, 2, 109, 3, 71])
print("min is \(bounds.min) and max is \(bounds.max)")
// Prints "min is -6 and max is 109".
```

함수에서 튜플을 반환할 때, 튜플의 멤버들은 이미 함수의 반환 타입에서 이름이 지정되어 있기 때문에 반환 시점에서 이름을 지정할 필요는 없습니다.

옵셔널 튜플 반환 타입 (Optional Tuple Return Types)

함수에서 반환되는 튜플 타입이 전체 튜플에 대해 "값이 없을" 가능성이 있는 경우, 옵셔널 튜플 반환 타입을 사용하여 전체 튜플이 `nil`일 수 있다는 사실을 반영할 수 있습니다. `(Int, Int)?`나 `(String, Int, Bool)?`와 같이 튜플 타입의 닫는 소괄호 다음에 물음표를 붙여 옵셔널 튜플 반환 타입을 작성합니다.

참고

`(Int, Int)?`와 같은 옵셔널 튜플 타입은 `(Int?, Int?)`와 같이 옵셔널 타입을 가지는 튜플과는 다릅니다. 옵셔널 튜플 타입은 튜플 안에 각각의 값이 옵셔널이 아니라 전체 튜플이 옵셔널이라는 의미입니다.

위에서 `minMax(array:)` 함수는 두 개의 `Int` 값을 포함하는 튜플을 반환합니다. 그러나 이 함수는 전달된 배열에 대한 안전성 검사를 하지 않습니다. `array` 인자가 빈 배열을 포함하면 위에서 정의된 `minMax(array:)` 함수는 `array[0]`을 접근할 때 런타임 오류가 발생합니다.

빈 배열을 안전하게 처리하기 위해선 `minMax(array:)` 함수는 옵셔널 튜플 반환 타입을 가지고 배열이 비어있을 때 `nil`을 반환해야 합니다:

```
func minMax(array: [Int]) -> (min: Int, max: Int)? {
    if array.isEmpty { return nil }
    var currentMin = array[0]
    var currentMax = array[0]
    for value in array[1..<array.count] {
        if value < currentMin {
            currentMin = value
        } else if value > currentMax {
            currentMax = value
        }
    }
    return (currentMin, currentMax)
}
```

옵셔널 바인딩을 통해 이 버전의 `minMax(array:)` 함수가 실제 튜플 값을 반환하는지 `nil`을 반환하는지 확인할 수 있습니다:

```
if let bounds = minMax(array: [8, -6, 2, 109, 3, 71]) {
    print("min is \(bounds.min) and max is \(bounds.max)")
}
// Prints "min is -6 and max is 109".
```

암시적 반환을 가진 함수 (Functions With an Implicit Return)

함수의 전체 본문이 하나의 표현식이면, 함수는 해당 표현식을 암시적으로 반환합니다. 예를 들어 아래의 두 함수는 모두 같은 동작을 합니다:

```
func greeting(for person: String) -> String {
    "Hello, " + person + "!"
}
print(greeting(for: "Dave"))
// Prints "Hello, Dave!".

func anotherGreeting(for person: String) -> String {
    return "Hello, " + person + "!"
}
print(anotherGreeting(for: "Dave"))
// Prints "Hello, Dave!".
```

`greeting(for:)` 함수의 전체 정의는 인사말 메세지를 반환하므로, 이것은 더 짧은 형태로 작성할 수 있습니다. `anotherGreeting(for:)` 함수는 같은 인사말 메세지를 반환하지만, `return` 키워드를 사용하여 더 긴 형식으로 작성합니다. `return` 한 줄로만 작성된 함수는 `return`을 생략할 수 있습니다.

간략한 Getter 선언 (Shorthand Getter Declaration)에서 살펴보겠지만, 프로퍼티 getter 또한 암시적 반환을 사용할 수 있습니다.

참고

암시적 반환값으로 작성하는 코드는 반드시 어떤 값을 반환해야 합니다. 예를 들어 암시적 반환값으로 `print(13)`을 사용할 수 없습니다. 그러나 `fatalError("Oh no!")`와 같이 Swift가 암시적 반환이 일어나지 않는 것을 아는 경우에는 사용할 수 있습니다.

함수 인자 레이블과 매개변수 이름 (Function Argument Labels and Parameter Names)

각 함수 매개변수는 *인자 레이블(argument label)*과 *매개변수 이름(parameter name)*을 가지고 있습니다. 인자 레이블은 함수가 호출될 때 사용되고; 각 인자는 함수 호출 시 인자 레이블 다음에 작성합니다. 매개변수 이름은 함수의 구현 내부에서 사용됩니다. 기본적으로 매개변수는 매개변수 이름을 인자 레이블로 사용합니다.

```
func someFunction(firstParameterName: Int, secondParameterName: Int) {
    // In the function body, firstParameterName and secondParameterName
    // refer to the argument values for the first and second parameters.
}
someFunction(firstParameterName: 1, secondParameterName: 2)
```

모든 매개변수는 유니크한 이름을 가져야 합니다. 여러 매개변수에 동일한 인자 레이블을 가질 수 있지만, 유니크한 인자 레이블은 코드를 더욱 읽기 편하게 해줍니다.

인자 레이블 지정 (Specifying Argument Labels)

공백으로 구분하여 매개변수 이름 앞에 인자 레이블을 작성합니다:

```
func someFunction(argumentLabel parameterName: Int) {
    // In the function body, parameterName refers to the argument value
    // for that parameter.
}
```

다음은 사람의 이름과 고향을 가져와 인사말을 반환하는 `greet(person:)` 함수의 변형입니다:

```
func greet(person: String, from hometown: String) -> String {
    return "Hello \(person)!  Glad you could visit from \(hometown)."
}
print(greet(person: "Bill", from: "Cupertino"))
// Prints "Hello Bill!  Glad you could visit from Cupertino.".
```

인자 레이블을 사용하면 함수를 마치 문장처럼 표현력 있게 호출할 수 있으며, 동시에 함수 본문은 읽기 쉽고 의도를 명확하게 유지할 수 있습니다.

인자 레이블 생략 (Omitting Argument Labels)

매개변수에 인자 레이블을 원치 않으면, 명시적으로 인자 레이블 작성하는 대신에 언더바(_)를 작성합니다.

```
func someFunction(_ firstParameterName: Int, secondParameterName: Int) {
    // In the function body, firstParameterName and secondParameterName
    // refer to the argument values for the first and second parameters.
}
someFunction(1, secondParameterName: 2)
```

매개변수가 인자 레이블을 가지고 있다면, 함수를 호출할 때 인자는 반드시 레이블을 지정해야 합니다.

매개변수 기본값 (Default Parameter Values)

매개변수의 타입 뒤에 매개변수 값을 할당하여 함수의 매개변수에 *기본값(default value)*을 정의할 수 있습니다. 기본값이 정의되어 있다면, 함수를 호출할 때 매개변수를 생략할 수 있습니다.

```
func someFunction(parameterWithoutDefault: Int, parameterWithDefault: Int = 12) {
    // If you omit the second argument when calling this function, then
    // the value of parameterWithDefault is 12 inside the function body.
}
someFunction(parameterWithoutDefault: 3, parameterWithDefault: 6) // parameterWithDefault is 6
someFunction(parameterWithoutDefault: 4) // parameterWithDefault is 12
```

기본값이 없는 매개변수는 가본 값이 있는 매개변수보다 함수 매개변수 목록의 앞 쪽에 배치합니다. 기본값이 없는 매개변수는 일반적으로 함수의 의미에서 더 중요합니다 - 이를 먼저 작성하면 기본값이 있는 매개변수가 생략되더라도 동일한 함수가 호출되고 있음을 쉽게 인식할 수 있습니다.

가변 매개변수 (Variadic Parameters)

*가변 매개변수(variadic parameter)*는 0개 이상의 특정 타입의 값을 허용합니다. 함수가 호출될 때 여러 개의 입력값이 전달될 수 있는 특정 매개변수는 가변 매개변수를 사용합니다. 가변 매개변수는 매개변수의 타입 이름 뒤에 마침표 세 개(`...`)를 추가하여 작성합니다.

가변 매개변수에 전달된 값은 함수 본문 내에서 적절한 타입의 배열로 사용될 수 있습니다. 예를 들어 `numbers`라는 이름과 `Double...` 타입을 가진 가변 매개변수는 함수 본문 내에서 `[Double]` 타입의 `numbers`라 불리는 상수 배열로 사용할 수 있습니다.

아래 예시는 길이에 관계없이 숫자 목록에 대한 산술 평균(평균)을 계산합니다:

```
func arithmeticMean(_ numbers: Double...) -> Double {
    var total: Double = 0
    for number in numbers {
        total += number
    }
    return total / Double(numbers.count)
}
arithmeticMean(1, 2, 3, 4, 5)
// returns 3.0, which is the arithmetic mean of these five numbers
arithmeticMean(3, 8.25, 18.75)
// returns 10.0, which is the arithmetic mean of these three numbers
```

함수는 여러 개의 가변 매개변수를 가질 수 있습니다. 가변 매개변수 뒤에 오는 첫 번째 매개변수는 인자 레이블을 가지고 있어야 합니다. 인자 레이블은 가변 매개변수에 전달되는 인자와 가변 매개변수 뒤에 오는 매개변수에 전달되는 인자를 명확하게 합니다.

In-Out 매개변수 (In-Out Parameters)

함수 매개변수는 기본적으로 상수입니다. 함수 본문 내에서 매개변수 값을 변경하려고 하면 컴파일 시에 오류가 발생합니다. 이것은 실수로 매개변수의 값을 변경하는 것을 방지합니다. 함수가 매개변수 값을 변경하고, 함수 호출이 종료된 후에도 이러한 변경된 값을 유지하고 싶다면, 해당 매개변수를 *in-out 매개변수(in-out parameter)*로 정의해야 합니다.

in-out 매개변수는 매개변수의 타입 바로 전에 `inout` 키워드를 붙여서 작성합니다. in-out 매개변수는 함수에 입력되고, 함수에서 수정되며, 함수가 끝난 후 원래 값 대신 수정된 값이 출력됩니다. In-out 매개변수의 동작 방식과 관련된 컴파일러 최적화에 대한 자세한 설명은 In-Out 매개변수 (In-Out Parameters)를 참고바랍니다.

in-out 매개변수에 전달할 수 있는 인자는 변수만 가능합니다. 상수나 리터럴 값은 수정할 수 없기 때문에 인자로 전달할 수 없습니다. in-out 매개변수에 변수를 인자로 전달할 때는 변수 이름 앞에 앰퍼샌드(&)를 붙여서 해당 변수가 함수에 의해 수정될 수 있음을 나타냅니다.

> **참고**
>
> In-Out 매개변수는 기본값을 가질 수 없고, 가변 매개변수는 `inout`으로 표기할 수 없습니다.

다음 예시는 a와 b라 하는 두 개의 in-out 정수 매개변수를 가지는 `swapTwoInts(_:_:)`라는 함수입니다:

```
func swapTwoInts(_ a: inout Int, _ b: inout Int) {
    let temporaryA = a
    a = b
    b = temporaryA
}
```

`swapTwoInts(_:_:)` 함수는 간단하게 b의 값을 a로 a의 값을 b로 바꿉니다. 이 함수는 a의 값을 `temporaryA`라 하는 임시 상수에 저장하고 b의 값을 a에 할당하고 `temporaryA`의 값을 b에 할당합니다.

`Int` 타입의 바꿀 두 개의 변수를 이용하여 `swapTwoInts(_:_:)` 함수를 호출할 수 있습니다. `swapTwoInts(_:_:)` 함수에 전달할 때, `someInt`와 `anotherInt`의 이름 앞에 앰퍼샌드를 붙여야 합니다:

```
var someInt = 3
var anotherInt = 107
swapTwoInts(&someInt, &anotherInt)
print("someInt is now \(someInt), and anotherInt is now \(anotherInt)")
// Prints "someInt is now 107, and anotherInt is now 3".
```

위의 예시는 `someInt`와 `anotherInt`의 함수의 외부에서 정의되었지만 `swapTwoInts(_:_:)` 함수로 인해 원래 값이 수정되는 것을 보여줍니다.

> **참고**
>
> In-Out 매개변수는 함수에서 값을 반환하는 것과 다릅니다. 위의 `swapTwoInts` 예시는 반환 타입을 정의하거나 값을 반환하지 않지만, `someInt`와 `anotherInt`의 값은 수정됩니다. In-out 매개변수는 함수 본문 범위를 벗어난 곳에서도 영향을 미칠 수 있는 또 다른 방법입니다.

함수 타입 (Function Types)

모든 함수는 매개변수 타입과 반환 타입으로 구성된 특정 *함수 타입(function type)*이 있습니다.

예를 들어:

```
func addTwoInts(_ a: Int, _ b: Int) -> Int {
    return a + b
}
func multiplyTwoInts(_ a: Int, _ b: Int) -> Int {
    return a * b
}
```

이 예시는 `addTwoInts`와 `multiplyTwoInts`라 불리는 간단한 수학 방정식 함수를 정의합니다. 이 함수는 각각 두 개의 `Int` 값을 가지며, 적절한 수학 연산을 수행하여 `Int` 값을 반환합니다.

이 두 함수의 타입은 `(Int, Int) -> Int`입니다. 이것은 아래와 같이 읽을 수 있습니다:

"함수는 모두 `Int` 타입인 두 개의 매개변수를 가지며, `Int` 타입을 반환합니다."

다음의 예시는 매개변수나 반환값이 없는 함수 입니다:

```
func printHelloWorld() {
    print("hello, world")
}
```

이 함수의 타입은 `() -> Void` 또는 "함수는 매개변수가 없고 `Void`를 반환합니다."

함수 타입 사용 (Using Function Types)

Swift에서는 다른 타입처럼 함수 타입을 사용합니다. 예를 들어 함수 타입에 대해 상수나 변수로 정의 할 수 있고, 변수에 적절한 함수 타입을 할당할 수 있습니다:

```
var mathFunction: (Int, Int) -> Int = addTwoInts
```

이것은 이렇게 읽을 수 있습니다:

"'두 개의 `Int` 값과 `Int`를 반환하는 함수'의 타입을 가지는 `mathFunction`이라는 변수를 정의합니다. 그리고 이 새로운 변수에 `addTwoInts`라는 함수에 대한 참조를 할당합니다."

`addTwoInts(_:_:)` 함수는 `mathFunction` 변수와 같은 타입을 가지므로, Swift의 타입 검사기에 의해 이 할당은 허용됩니다.

이제 `mathFunction`이라는 이름으로 할당된 함수를 호출할 수 있습니다:

```
print("Result: \(mathFunction(2, 3))")
// Prints "Result: 5".
```

비함수 타입과 동일한 방식으로 같은 타입으로 일치하는 다른 함수를 같은 변수에 할당할 수 있습니다:

```
mathFunction = multiplyTwoInts
print("Result: \(mathFunction(2, 3))")
// Prints "Result: 6".
```

다른 타입과 마찬가지로, 함수 타입을 상수나 변수에 함수를 할당할 때 Swift가 함수 타입을 추론하도록 맡길 수 있습니다:

```
let anotherMathFunction = addTwoInts
// anotherMathFunction is inferred to be of type (Int, Int) -> Int
```

매개변수 타입으로 함수 타입 (Function Types as Parameter Types)

`(Int, Int) -> Int`와 같은 함수 타입을 다른 함수의 매개변수로 사용할 수 있습니다. 이렇게 하면 함수의 일부 구현을 해당 함수를 호출하는 쪽에서 제공하도록 할 수 있습니다.

다음의 예시는 위 수학 함수의 결과를 출력합니다:

```
func printMathResult(_ mathFunction: (Int, Int) -> Int, _ a: Int, _ b: Int) {
    print("Result: \(mathFunction(a, b))")
}
printMathResult(addTwoInts, 3, 5)
// Prints "Result: 8".
```

이 예시는 세 개의 매개변수를 가지는 `printMathResult(_:_:_:)`라는 함수를 정의합니다. 첫 번째 매개변수는 `mathFunction`이라 불리는 `(Int, Int) -> Int` 타입입니다. 이 첫 번째 매개변수의 인자로 해당 타입의 함수를 전달할 수 있습니다. 두 번째 세 번째 매개변수는 `a`와 `b`라 불리고 둘 다 `Int` 타입입니다. 이들은 제공된 수학 함수의 두 개의 입력값으로 사용됩니다.

`printMathResult(_:_:_:)` 호출할 때, `addTwoInts(_:_:)` 함수와 정수 3과 5가 전달됩니다. 3과 5로 제공된 함수를 호출하고 8의 결과를 출력합니다.

`printMathResult(_:_:_:)`의 역할은 적절한 타입의 수학 함수를 호출하고 값을 출력합니다. 이것은 함수의 실제 구현 내용은 중요하지 않습니다 - 중요한 것은 함수가 올바른 타입을 갖는다는 점입니다. 이렇게 하면 `printMathResult(_:_:_:)`는 자신의 일부 기능을 타입 안전성을 유지하면서 함수 호출자에게 넘길 수 있습니다.

반환 타입으로 함수 타입 (Function Types as Return Types)

다른 함수의 반환 타입으로 함수 타입을 사용할 수 있습니다. 반환하는 함수의 반환 화살표(->) 바로 뒤에 전체 함수 타입을 작성하여 이를 수행합니다.

다음 예시는 `stepForward(_:)`와 `stepBackward(_:)`라 불리는 간단한 두 개의 함수를 정의합니다. `stepForward(_:)` 함수는 입력값에 1을 더해 반환하고, `stepBackward(_:)` 함수는 입력값에 1을 빼고 반환합니다. 두 함수 모두 `(Int) -> Int` 타입입니다:

```
func stepForward(_ input: Int) -> Int {
    return input + 1
}
func stepBackward(_ input: Int) -> Int {
    return input - 1
}
```

다음의 함수는 반환 타입이 `(Int) -> Int` 인 `chooseStepFunction(backward:)`입니다. `chooseStepFunction(backward:)` 함수는 `backward`라 불리는 Boolean 매개변수를 기반으로 `stepForward(_:)` 함수나 `stepBackward(_:)` 함수를 반환합니다:

```
func chooseStepFunction(backward: Bool) -> (Int) -> Int {
    return backward ? stepBackward : stepForward
}
```

이제 `chooseStepFunction(backward:)`를 사용하여 한 방향이나 다른 방향으로 이동할 함수를 얻을 수 있습니다:

```
var currentValue = 3
let moveNearerToZero = chooseStepFunction(backward: currentValue > 0)
// moveNearerToZero now refers to the stepBackward() function
```

위의 예시는 `currentValue`라는 변수를 점차적으로 0에 가깝게 이동하기 위해 양수 또는 음수 단계가 필요한지 판단합니다. `currentValue`는 3의 초기값을 가지며, 이것은 `currentValue > 0`이 `true`를 반환한다는 의미이며, `chooseStepFunction(backward:)`가 `stepBackward(_:)` 함수를 반환하도록 합니다. 반환된 함수의 참조는 `moveNearerToZero`라는 상수에 저장됩니다.

`moveNearerToZero`가 올바른 함수를 참조하므로, 0으로 계산하는데 사용할 수 있습니다:

```
print("Counting to zero:")
// Counting to zero:
while currentValue != 0 {
    print("\(currentValue)... ")
    currentValue = moveNearerToZero(currentValue)
}
print("zero!")
// 3...
// 2...
// 1...
// zero!
```

중첩 함수 (Nested Functions)

이 챕터에서 접한 모든 함수는 전역 범위에서 정의된 *전역 함수(global functions)*의 예입니다. *중첩 함수(nested functions)*라고 하는 다른 함수 내에 함수를 정의할 수도 있습니다.

중첩 함수는 기본적으로 외부에서 보이지 않지만, 중첩 함수를 둘러싼 함수를 통해 호출될 수 있고 사용될 수 있습니다. 중첩 함수를 둘러싼 함수는 중첩 함수 중 하나를 반환하여 중첩 함수를 다른 범위에서 사용할 수도 있습니다.

위의 예시 `chooseStepFunction(backward:)`를 중첩 함수를 사용하고 반환하도록 다시 작성할 수 있습니다:

```
func chooseStepFunction(backward: Bool) -> (Int) -> Int {
    func stepForward(input: Int) -> Int { return input + 1 }
    func stepBackward(input: Int) -> Int { return input - 1 }
    return backward ? stepBackward : stepForward
}
var currentValue = -4
let moveNearerToZero = chooseStepFunction(backward: currentValue > 0)
// moveNearerToZero now refers to the nested stepForward() function
while currentValue != 0 {
    print("\(currentValue)... ")
    currentValue = moveNearerToZero(currentValue)
}
print("zero!")
```

```
// -4...
// -3...
// -2...
// -1...
// zero!
```

언어 가이드

클로저 (Closures)

함수 이름 없이 함께 실행되어야 하는 코드를 그룹화합니다.

*클로저(Closures)*는 코드에서 전달되고 사용할 수 있는 독립적인 기능 블록입니다. Swift의 클로저는 다른 프로그래밍 언어에서 클로저, 익명 함수, 람다, 블록과 유사합니다.

클로저는 정의된 컨텍스트에서 상수나 변수에 대한 참조를 캡처하고 저장할 수 있습니다. 이러한 상수와 변수를 *클로저가 포착한다(closing over)*라고 합니다. Swift는 캡처에 대한 메모리 관리를 자동으로 처리합니다.

> **참고**
>
> 캡처에 대한 개념이 생소하더라도 걱정하지 마세요. 아래 캡처값 (Capturing Values)에서 자세히 다루도록 하겠습니다.

함수 (Functions)에서 소개한 전역 함수와 중첩 함수는 클로저의 특별한 형태입니다. 클로저는 세 가지 형태 중 하나를 취합니다:

- 전역 함수는 이름을 가지고 어떠한 값도 캡처하지 않는 클로저입니다.
- 중첩 함수는 이름을 가지고 외부 함수로부터 값을 캡처할 수 있는 클로저입니다.
- 클로저 표현식은 주변 컨텍스트에서 값을 캡처할 수 있는 간결한 구문으로 작성된 이름이 없는 클로저입니다.

Swift의 클로저 표현식은 명확한 스타일을 가지고 있으며, 일반적인 상황에서 간결하고 복잡하지 않은 문법을 유도하는 여러 최적화를 제공합니다. 이러한 최적화에는 다음이 포함됩니다:

- 컨텍스트에서 매개변수와 반환값 타입 유추
- 단일 표현식 클로저의 암시적 반환
- 축약된 인자 이름
- 후행 클로저 구문

클로저 표현식 (Closure Expressions)

중첩 함수 (Nested Functions)에서 소개된 중첩 함수는 더 큰 함수의 부분으로 독립적인 코드 블록을 이름 붙여 정의할 수 있는 편리한 수단입니다. 그러나 전체 선언과 이름 없이 함수와 유사한 구조의 짧은 형태로 작성하는 것이 때때로 유용합니다. 특히 함수나 메서드가 다른 함수를 하나 이상의 인자로 받을 때 그렇습니다.

*클로저 표현식(Closure expressions)*은 간단하고 집중적인 구문으로 인라인 클로저를 작성하는 방법입니다. 클로저 표현식은 명확성이나 의도를 잃지 않고 더 짧은 형태로 클로저를 작성 하기위한 여러 구문 최적화를 제공합니다. 아래의 클로저 표현식 예시는 `sorted(by:)` 메서드의 동일한 기능을 여러 단계에 걸쳐 점차 간결하게 표현함으로써 이러한 최적화를 보여줍니다.

정렬 메서드 (The Sorted Method)

Swift의 표준 라이브러리는 주어진 정렬 클로저의 결과에 따라, 알려진 타입의 값들로 구성된 배열을 정렬합니다. 정렬이 완료되면, `sorted(by:)` 메서드는 원본 배열과 같은 타입과 같은 크기를 가진 새로운 배열을 반환하며, 그 요소들은 올바른 정렬 순서로 배치되어 있습니다. 기존 배열은 `sorted(by:)` 메서드로 수정되지 않습니다.

아래 예시의 클로저 표현식은 알파벳 역순으로 `String` 값의 배열을 정렬하기 위해 `sorted(by:)` 메서드를 사용합니다. 다음은 정렬할 초기 배열입니다:

```
let names = ["Chris", "Alex", "Ewa", "Barry", "Daniella"]
```

`sorted(by:)` 메서드는 배열의 요소 타입과 동일한 두 개의 인자를 받아, 첫 번째 값이 두 번째 값보다 앞에 와야 하는지를 판단하여 `Bool` 값을 반환하는 클로저를 인자로 받습니다. 정렬 클로저는 첫 번째 값이 두 번째 값보다 먼저 와야 하면 `true`를 반환하고, 그렇지 않으면 `false`를 반환해야 합니다.

이 예시는 `String` 값의 배열을 정렬하고, 정렬 클로저는 `(String, String) -> Bool` 타입의 함수여야 합니다.

정렬 클로저를 제공하는 한가지 방법은 올바른 타입의 일반 함수를 작성하고 `sorted(by:)` 메서드의 인자로 전달하는 것입니다:

```
func backward(_ s1: String, _ s2: String) -> Bool {
    return s1 > s2
}
var reversedNames = names.sorted(by: backward)
// reversedNames is equal to ["Ewa", "Daniella", "Chris", "Barry", "Alex"]
```

첫 번째 문자열(`s1`)이 두 번째 문자열(`s2`)보다 크다면 `backward(_:_:)` 함수는 `true`를 반환하고, 이것은 정렬된 배열에 `s2` 전에 `s1`이 나타납니다. 문자열의 문자가 "더 크다"는 "알파벳 순으로 더 뒤에 나타난다"는 의미입니다. 이것은 문자 `B`는 문자 `A`보다 "더 크다"이고, 문자열 `"Tom"`은 문자열 `"Tim"`보다 더 큽니다. 알파벳 역순으로 정렬될 때 `"Barry"`는 `"Alex"`보다 앞에 위치합니다.

그러나 이 함수는 사실상 단일 표현식 함수(`a > b`)이기 때문에 이렇게 길게 작성하는 것은 다소 번거롭습니다. 이 예시에서는 클로저 표현식 구문을 사용하여 정렬 클로저를 인라인으로 작성하는 것이 좋습니다.

클로저 표현식 문법 (Closure Expression Syntax)

클로저 표현식 문법은 아래와 같은 일반적인 형태를 가지고 있습니다:

```
{ (<#parameters#>) -> <#return type#> in
   <#statements#>
}
```

클로저 표현식 문법에서 매개변수는 in-out 매개변수로 정의할 수 있지만, 기본값을 가질 수는 없습니다. 가변 매개변수의 이름을 지정하면 가변 매개변수를 사용할 수 있습니다. 튜플은 매개변수 타입이나 반환 타입으로 사용될 수도 있습니다.

아래 예시는 위에서 `backward(_:_:)` 함수의 클로저 표현입니다:

```
reversedNames = names.sorted(by: { (s1: String, s2: String) -> Bool in
    return s1 > s2
})
```

이 인라인 클로저에서 매개변수와 반환 타입의 선언은 `backward(_:_:)` 함수에서 선언한 것과 동일합니다. 두 경우 모두 `(s1: String, s2: String) -> Bool`로 작성합니다. 그러나 인라인 클로저 표현식에서는 매개변수와 반환 타입은 중괄호 바깥이 아닌 안에 작성합니다.

클로저 본문의 시작은 `in` 키워드로 시작합니다. 이 키워드는 클로저의 매개변수와 반환 타입 정의가 끝남을 나타내며, 클로저의 본문이 시작함을 나타냅니다.

클로저의 본문이 너무 짧으면, 한 줄로 작성할 수 있습니다:

```
reversedNames = names.sorted(by: { (s1: String, s2: String) -> Bool in return s1 > s2 } )
```

이것은 `sorted(by:)` 메서드에 대한 전체 호출 구조는 동일하게 유지된다는 것을 보여줍니다. 소괄호는 여전히 메서드의 전체 인자를 둘러싸고 있습니다. 그러나 그 인자는 인라인 클로저로 작성되었습니다.

컨텍스트로부터 타입 추론 (Inferring Type From Context)

정렬 클로저가 메서드의 인자로 전달되기 때문에, Swift는 매개변수 타입과 반환되는 값의 타입을 유추할 수 있습니다. `sorted(by:)` 메서드는 문자열 배열에서 호출되므로, 인자는 `(String, String) -> Bool` 타입의 함수여야 합니다. 이는 `(String, String)`과 `Bool` 타입을 클로저 표현식 정의에 일부러 작성할 필요가 없음을 의미합니다. 모든 타입은 유추할 수 있기 때문에, 반환 화살표(`->`)와 매개변수의 이름을 둘러싼 소괄호를 생략할 수 있습니다:

```
reversedNames = names.sorted(by: { s1, s2 in return s1 > s2 } )
```

클로저를 함수나 메서드의 인라인 클로저 표현식으로 전달할 때, 항상 매개변수 타입과 반환 타입을 유추할 수 있습니다. 따라서 클로저가 함수나 메서드 인자로 사용될 때, 완전한 형태로 인라인 클로저를 작성할 필요가 없습니다.

그럼에도 불구하고 원하는 경우 타입을 명시적으로 작성할 수 있으며, 코드의 의미가 모호해지는 것을 방지할 수 있다면 그렇게 하는 것이 좋습니다. `sorted(by:)` 메서드의 경우, 정렬이 발생한다는 사실에서 클로저의 목적이 명확하며, 문자열 배열의 정렬을 지원하기 때문에 코드를 읽는 사람이 클로저가 `String` 값으로 작동할 가능성이 있다고 자연스럽게 추측할 수 있습니다.

단일 표현식 클로저의 암시적 반환 (Implicit Returns from Single-Expression Closures)

단일 표현식 클로저(Single-expression closures)는 이전 예시에서 `return` 키워드를 생략하여 단일 표현식의 결과를 암시적으로 반환할 수 있습니다:

```
reversedNames = names.sorted(by: { s1, s2 in s1 > s2 } )
```

여기서 `sorted(by:)` 메서드의 인자는 함수 타입이므로, 클로저가 `Bool` 값을 반환해야 한다는 것이 명확합니다. 클로저 본문이 단일 표현식(`s1 > s2`)으로 구성되어 있고, 이 표현식이 `Bool` 값을 반환하기 때문에 모호함이 없으므로 `return` 키워드를 생략할 수 있습니다.

축약 인자 이름 (Shorthand Argument Names)

Swift는 인라인 클로저에 `$0`, `$1`, `$2` 등 클로저의 인자값으로 참조하는데 사용할 수 있는 자동으로 축약 인자 이름(shorthand argument names)을 제공합니다.

클로저 표현식에 이런 축약 인자 이름을 사용한다면, 선언에 클로저의 인자 목록을 생략할 수 있습니다. 축약 인자 이름의 타입은 함수 타입에서 추론되고, 사용한 축약 인자 중 가장 큰 번호가 클로저가 받는 인자의 개수를 결정합니다. 클로저 표현식이 오직 본문으로만 이루어져 있다면, `in` 키워드도 생략할 수 있습니다:

```
reversedNames = names.sorted(by: { $0 > $1 } )
```

여기서 `$0`와 `$1`은 클로저의 첫 번째와 두 번째 `String` 인자를 나타냅니다. `$1`이 축약 인자에서 가장 높은 숫자이므로, 클로저는 두 개의 인자가 있는 것으로 간주됩니다. 여기서 `sorted(by:)` 함수는 인자가 모두 문자열인 클로저로 기대하므로, 축약 인자 `$0`과 `$1`은 모두 타입 `String` 입니다.

연산자 메서드 (Operator Methods)

사실 위의 클로저 표현식을 더 짧게 작성하는 방법이 있습니다. Swift의 `String` 타입은 보다 큰 연산자(>)의 문자열 별 구현을 `String` 타입의 매개변수 두 개가 있는 메서드로 정의하고, `Bool` 타입의 값을 반환합니다. 이것은 `sorted(by:)` 메서드에 필요한 메서드 타입과 정확하게 일치합니다. 따라서 보다 큰 연산자 자체를 인자로 전달하면, Swift는 문자열에 특화된 구현을 사용하겠다는 의도로 자동으로 추론합니다:

```
reversedNames = names.sorted(by: >)
```

연산자 메서드에 대한 자세한 내용은 연산자 메서드 (Operator Methods)를 참고바랍니다.

후행 클로저 (Trailing Closures)

함수의 마지막 인자로 클로저 표현식을 전달해야 하고 클로저 표현식이 긴 경우 *후행 클로저(trailing closure)*로 작성하는 것이 유용할 수 있습니다. 후행 클로저는 함수의 인자이지만 함수 호출의 소괄호 다음에 작성합니다. 후행 클로저 구문을 사용할 때, 해당 클로저의 인자 레이블을 함수 호출 시에 작성하지 않습니다. 함수 호출은 여러개의 후행 클로저를 포함할 수 있지만; 아래 예시에서는 하나의 후행 클로저만 사용합니다.

```
func someFunctionThatTakesAClosure(closure: () -> Void) {
    // function body goes here
}

// Here's how you call this function without using a trailing closure:

someFunctionThatTakesAClosure(closure: {
    // closure's body goes here
})

// Here's how you call this function with a trailing closure instead:

someFunctionThatTakesAClosure() {
    // trailing closure's body goes here
}
```

위의 클로저 표현식 문법 (Closure Expression Syntax) 섹션에 문자열 정렬 클로저는 후행 클로저로 `sorted(by:)` 메서드의 소괄호 바깥에 작성될 수 있습니다:

```
reversedNames = names.sorted() { $0 > $1 }
```

클로저 표현식이 함수나 메서드의 유일한 인자로 제공되고, 그 표현식을 후행 클로저로 작성하는 경우에는, 함수를 호출할 때 함수나 메서드 이름 뒤에 소괄호 `()`를 작성할 필요가 없습니다:

```
reversedNames = names.sorted { $0 > $1 }
```

후행 클로저는 클로저가 길어서 한 줄로 인라인 작성하기 불가능할 때 유용합니다. 예를 들어 Swift의 `Array` 타입은 단일 인자로 클로저 표현식을 가지는 `map(_:)` 메서드가 있습니다. 이 클로저는 배열의 각 아이템에 대해 한 번 호출

되고, 아이템에 대해 매핑된 대체 값(다른 타입일 수 있음)이 반환됩니다. `map(_:)`에 전달한 클로저에 작성된 코드에 따라 매핑 특성과 반환된 값의 타입을 지정합니다.

제공된 클로저에 각 배열의 요소를 적용한 후에 `map(_:)` 메서드는 기존 배열에 해당 값과 같은 순서로 새로 매핑된 값의 새로운 배열을 반환합니다.

다음은 `Int` 값의 배열을 `String` 값의 배열로 변환하기 위해 후행 클로저와 `map(_:)` 메서드를 어떻게 사용하는지 나타냅니다. 배열 `[16, 58, 510]`은 새로운 배열 `["OneSix", "FiveEight", "FiveOneZero"]`을 생성하는데 사용됩니다:

```
let digitNames = [
    0: "Zero", 1: "One", 2: "Two",   3: "Three", 4: "Four",
    5: "Five", 6: "Six", 7: "Seven", 8: "Eight", 9: "Nine"
]
let numbers = [16, 58, 510]
```

위 코드는 정수와 그 정수에 맞는 영어 표기를 매핑하는 딕셔너리를 생성합니다. 문자열로 변환하기 위한 정수 배열도 정의합니다.

`numbers` 배열을 사용하여 후행 클로저로 `map(_:)` 메서드로 클로저 표현식을 전달하여 `String` 값의 배열을 생성할 수 있습니다:

```
let strings = numbers.map { (number) -> String in
    var number = number
    var output = ""
    repeat {
        output = digitNames[number % 10]! + output
        number /= 10
    } while number > 0
    return output
}
// strings is inferred to be of type [String]
// its value is ["OneSix", "FiveEight", "FiveOneZero"]
```

`map(_:)` 메서드는 배열의 각 요소마다 클로저 표현식을 한 번씩 호출합니다. 매핑할 배열의 값에서 추론할 수 있으므로, 클로저의 입력 매개변수 인 `number` 타입을 지정할 필요가 없습니다.

이 예시에서 변수 `number`는 클로저의 `number` 매개변수 값으로 초기화되기 때문에, 값은 클로저 본문 내에서 수정될 수 있습니다. (함수와 클로저의 매개변수는 기본적으로 상수입니다.) 클로저 표현식은 매핑된 출력 배열에 저장될 타입을 나타내기 위해 `String` 타입도 반환 타입으로 지정합니다.

클로저 표현식은 호출될 때마다 `output`이라는 문자열을 만듭니다. 나머지 연산자(`number % 10`)를 이용하여 `number`의 마지막 숫자를 계산하고, `digitNames` 딕셔너리에서 적절한 문자열로 변환하는데 사용합니다. 클로저는 0보다 큰 정수에 대한 문자열 표현을 생성할 수 있습니다.

> **참고**
>
> `digitNames` 딕셔너리의 서브스크립트 호출 뒤에 느낌표(`!`)가 붙는 이유는, 딕셔너리 서브스크립트는 키가 존재하지 않을 수 있기 때문에 옵셔널 값을 반환하기 때문입니다. 위의 예시에서 `digitNames` 딕셔너리에 `number % 10`은 항상 유효한 서브스크립트 키를 보장하므로 느낌표는 서브스크립트의 옵셔널 반환값에 저장된 `String` 값을 강제로 언래핑 하는데 사용됩니다.

`digitNames` 딕셔너리에서 반환된 문자열이 `output` 앞에 추가되어 숫자의 문자열 버전을 역순으로 조립합니다. (표현식 `number % 10`은 `16`의 경우 `6`, `58`의 경우 `8`, `510`의 경우 `0`을 제공합니다.)

그러면 `number` 변수는 `10`으로 나누어 집니다. 이것은 정수이기 때문에, 소수점 아래는 버림이 되고, `16`은 `1`, `58`은 `5`, `510`은 `51`이 됩니다.

이 과정은 `number`가 `0`이 될 때까지 반복하고, 최종적으로 만들어진 `output` 문자열이 클로저에 의해 반환되고, `map(_:)` 메서드에 의해 결과 배열에 추가됩니다.

위의 예시에서 사용된 후행 클로저 구문은, 클로저의 기능을 해당 함수를 지원하는 바로 뒤에 깔끔하게 캡슐화하여 작성할 수 있게 해줍니다. 이를 통해 클로저 전체를 `map(_:)` 메서드의 소괄호로 감쌀 필요가 없습니다.

함수가 여러 개의 클로저를 인자로 받을 경우, 첫 번째 후행 클로저는 인자 레이블 없이 작성하고, 그 이후의 후행 클로저는 인자 레이블을 명시해야 합니다. 예를 들어 아래의 함수는 사진 갤러리에서 사진을 불러오는 기능을 수행합니다:

```
func loadPicture(from server: Server, completion: (Picture) -> Void, onFailure: () -> Void) {
    if let picture = download("photo.jpg", from: server) {
        completion(picture)
    } else {
        onFailure()
    }
}
```

이 함수를 호출하여 사진을 불러올 때, 두 개의 클로저를 전달합니다. 첫 번째 클로저는 사진 다운로드 완료 후에 사진을 표시하는 완료 핸들러입니다. 두 번째 클로저는 오류를 표시하는 오류 핸들러입니다.

```
loadPicture(from: someServer) { picture in
    someView.currentPicture = picture
} onFailure: {
    print("Couldn't download the next picture.")
}
```

이 예시에서 `loadPicture(from:completion:onFailure:)` 함수는 네트워크 작업을 백그라운드로 전달하고 네트워크 작업이 완료되면 두 완료 핸들러 중 하나를 호출합니다. 이러한 방식으로 함수를 작성하면 두 상황을 모두 처리하는 하나의 클로저를 사용하는 대신 성공적인 다운로드 후 사용자 인터페이스를 업데이트 하는 코드와 네트워크 오류를 처리하는 코드를 명확하게 분리할 수 있습니다.

> **참고**
>
> 완료 핸들러(Completion handlers)는 여러 핸들러가 중첩되어 있으면 읽기 어려울 수 있습니다. 이것을 대체하기 위해선 동시성 (Concurrency)에서 설명되어진 것과 같이 비동기 코드를 사용하면 됩니다.

캡처값 (Capturing Values)

클로저는 정의된 주변 컨텍스트로부터 상수와 변수를 *캡처(capture)*할 수 있습니다. 그러면 클로저는 상수와 변수를 정의한 원래 범위가 더 이상 존재하지 않더라도 본문 내에서 해당 상수와 변수의 값을 참조하고 수정할 수 있습니다.

Swift에서 값을 캡처할 수 있는 가장 간단한 클로저 형태는 다른 함수의 본문 내에 작성하는 중첩 함수입니다. 중첩 함수는 바깥 함수의 어떠한 인자도 캡처할 수 있고 바깥 함수 내에 정의된 상수와 변수를 캡처할 수도 있습니다.

아래는 `incrementer`라는 중첩 함수가 포함된 `makeIncrementer`라는 함수의 예시입니다. 중첩된 `incrementer()` 함수는 자신을 둘러싼 컨텍스트에서 `runningTotal`과 `amount`인 두 값을 캡처합니다. 이 값을 캡처한 후에 `incrementer`는 호출될 때마다 `runningTotal`을 `amount`만큼 증가시키는 `makeIncrementer`에 의해 클로저로 반환됩니다.

```
func makeIncrementer(forIncrement amount: Int) -> () -> Int {
    var runningTotal = 0
    func incrementer() -> Int {
        runningTotal += amount
        return runningTotal
    }
    return incrementer
}
```

`makeIncrementer`의 반환 타입은 `() -> Int`입니다. 이것은 단순한 값이 아닌 함수를 반환한다는 의미입니다. 반환하는 함수에는 매개변수가 없으며, 호출될 때마다 `Int` 값을 반환합니다. 함수가 다른 함수를 반환하는 방법을 알아보려면 반환 타입으로 함수 타입 (Function Types as Return Types)을 참고바랍니다.

`makeIncrementer(forIncrement:)` 함수는 반환될 현재 증가분을 저장하기 위해 `runningTotal`이라는 정수 변수를 정의합니다. 이 변수는 `0`으로 초기화 됩니다.

`makeIncrementer(forIncrement:)` 함수는 `forIncrement`라는 인자 레이블과 `amount`라는 매개변수 이름을 가지는 하나의 `Int` 매개변수를 가집니다. 이 매개변수에 전달된 인자값은 반환된 증가 함수가 호출될 때마다 `runningTotal`을 얼마나 증가시켜야 하는지 지정합니다. `makeIncrementer` 함수는 실제 증가를 수행하는 `incrementer`라는 중첩 함수를 정의합니다. 이 함수는 간단하게 `amount`를 `runningTotal`에 더하고 그 결과를 반환합니다.

단독으로 고려할 때, `incrementer()` 함수는 비정상적으로 보일 수 있습니다:

```
func incrementer() -> Int {
    runningTotal += amount
    return runningTotal
}
```

`incrementer()` 함수는 매개변수가 없음에도 함수 본문 내에 `runningTotal`과 `amount`를 참조하고 있습니다. 둘러싸인 함수에 `runningTotal`과 `amount`에 대한 *참조(reference)*를 캡처하고 함수 내에서 사용합니다. 참조를 캡처하는 것은 `makeIncrementer` 호출이 종료될 때, `runningTotal`과 `amount`가 사라지지 않고 다음에 `incrementer` 함수가 호출될 때 `runningTotal`을 사용할 수 있습니다.

> **참고**
>
> 최적화를 위해, Swift는 값이 클로저에 의해 변경되지 않고, 클로저가 생성된 후 값이 변경되지 않는 경우 해당 값을 복사하여 저장할 수 있습니다.
>
> Swift는 더이상 필요하지 않을때 변수를 처리하는 것과 관련된 모든 메모리 관리도 처리합니다.

다음은 `makeIncrementer` 동작에 대한 예시입니다:

```
let incrementByTen = makeIncrementer(forIncrement: 10)
```

이 예시에서는 호출될 때마다 `runningTotal` 변수에 `10`을 더하는 증가 함수를 참조하도록 `incrementByTen`이라는 상수를 설정합니다. 함수를 여러 번 호출하면, 이 동작이 어떻게 수행되는지 확인할 수 있습니다:

```
incrementByTen()
// returns a value of 10
incrementByTen()
// returns a value of 20
incrementByTen()
// returns a value of 30
```

두 번째 증가 함수를 생성하면, 새로운 별도의 `runningTotal` 변수에 대한 고유한 참조를 저장합니다:

```
let incrementBySeven = makeIncrementer(forIncrement: 7)
incrementBySeven()
// returns a value of 7
```

기존 증가 함수(`incrementByTen`)를 다시 호출하면, 해당 함수가 캡처한 자신의 `runningTotal` 변수만 계속 증가하며, `incrementBySeven`으로 캡처된 변수에는 영향을 주지 않습니다:

```
incrementByTen()
// returns a value of 40
```

> **참고**
>
> 클래스 인스턴스의 프로퍼티에 클로저를 할당하고, 클로저가 인스턴스나 인스턴스의 멤버를 참조하여 해당 인스턴스를 캡처하면 클로저와 인스턴스 사이에 강한 참조 순환이 생성됩니다. Swift는 캡처 리스트를 사용하여 이러한 강한 참조 순환을 끊습니다. 자세한 내용은 클로저의 강한 순환 참조 (Strong Reference Cycles for Closures)를 참고바랍니다.

클로저는 참조 타입 (Closures Are Reference Types)

위 예시에서 `incrementBySeven`과 `incrementByTen`은 상수이지만 이러한 상수가 참조하는 클로저는 캡처한 `runningTotal` 변수를 계속 증가시킬 수 있습니다. 이는 함수와 클로저가 *참조 타입(reference types)*이기 때문입니다.

함수나 클로저를 상수나 변수에 할당할 때마다, 실제로 해당 상수나 변수를 함수나 클로저에 대한 참조로 설정합니다. 위의 예시에서 `incrementByTen`이 참조하는 클로저 자체는 상수이지만, 그 클로저의 내용은 고정된 것이 아닙니다.

이것은 또한 서로 다른 두 상수나 변수에 클로저를 할당한다면 두 상수나 변수는 모두 같은 클로저를 참조한다는 의미입니다.

```
let alsoIncrementByTen = incrementByTen
alsoIncrementByTen()
// returns a value of 50

incrementByTen()
// returns a value of 60
```

위의 예시는 `alsoIncrementByTen` 호출은 `incrementByTen` 호출과 같음을 보여줍니다. 두 상수 모두 같은 클로저를 참조하기 때문에, 둘 다 증가하고 같은 러닝 합계를 반환합니다.

탈출 클로저 (Escaping Closures)

함수의 인자로 클로저를 전달하지만 함수가 반환된 후 호출되는 클로저를 함수를 *탈출(escape)*한다고 말합니다. 클로저를 매개변수로 갖는 함수를 선언할 때, 이 클로저가 탈출할 수 있음을 나타내기 위해 매개변수의 타입 전에 `@escaping`을 작성합니다.

클로저가 탈출할 수 있는 한 가지 방법은 함수 바깥에 정의된 변수에 저장되는 경우입니다. 예를 들어 비동기적 작업을 시작하는 대부분의 함수는 완료 핸들러로 클로저를 인자로 사용합니다. 이 함수는 작업을 시작한 후에 반환되지만 작업이 완료될 때까지 클로저가 호출되지 않습니다 - 클로저는 나중에 호출되려면 탈출해야 합니다. 예를 들어:

```
var completionHandlers: [() -> Void] = []
func someFunctionWithEscapingClosure(completionHandler: @escaping () -> Void) {
    completionHandlers.append(completionHandler)
}
```

`someFunctionWithEscapingClosure(_:)` 함수는 인자로 클로저를 가지고 있고 함수 외부에 선언된 배열에 추가합니다. 함수의 매개변수에 `@escaping`을 표시하지 않으면, 컴파일시 오류가 발생합니다.

클래스 인스턴스를 참조하는 `self`가 탈출 클로저 안에서 사용되는 경우에는 특히 주의가 필요합니다. 탈출 클로저에 `self` 캡처는 강한 참조 순환이 생기기 쉽습니다. 참조 순환에 대한 자세한 내용은 자동 참조 카운팅 (Automatic Reference Counting)을 참고바랍니다.

일반적으로 클로저는 클로저 내부에서 변수를 사용할 때 암시적으로 변수를 캡처하지만, 이 경우에는 명시적으로 작성해야 합니다. `self`를 캡처하려면, 사용할 때 명시적으로 `self`를 작성하거나, 클로저의 캡처 리스트에 `self`를 포함합니다. `self`를 명시적으로 작성하는 것은 의도를 분명하게 표현하고, 참조 순환이 없음을 확인하도록 유도하는 역할도 합니다. 예를 들어 아래 코드에서 `someFunctionWithEscapingClosure(_:)`에 전달된 클로저는 명시적으로 `self`를 참조합니다. 반대로 `someFunctionWithNonescapingClosure(_:)`에 전달된 클로저는 탈출하지 않는 클로저이므로, 암시적으로 `self`를 참조할 수 있습니다.

```
func someFunctionWithNonescapingClosure(closure: () -> Void) {
    closure()
}

class SomeClass {
    var x = 10
    func doSomething() {
        someFunctionWithEscapingClosure { self.x = 100 }
        someFunctionWithNonescapingClosure { x = 200 }
    }
}

let instance = SomeClass()
instance.doSomething()
print(instance.x)
// Prints "200".

completionHandlers.first?()
print(instance.x)
// Prints "100".
```

다음은 클로저의 캡처 리스트에 포함하여 `self`를 캡처하고 암시적으로 `self`를 참조하는 `doSomething()` 입니다:

```
class SomeOtherClass {
    var x = 10
    func doSomething() {
        someFunctionWithEscapingClosure { [self] in x = 100 }
        someFunctionWithNonescapingClosure { x = 200 }
    }
}
```

`self`가 구조체나 열거형 인스턴스이면, 항상 암시적으로 `self`를 참조할 수 있습니다. 그러나 탈출 클로저는 구조체나 열거형 인스턴스이면 `self`에 대한 변경 가능한 참조를 캡처할 수 없습니다. 구조체와 열거형은 구조체와 열거형은 값 타입 (Structures and Enumerations Are Value Types)에서 설명했듯이 공유된 가변성을 허용하지 않습니다.

```
struct SomeStruct {
    var x = 10
    mutating func doSomething() {
        someFunctionWithNonescapingClosure { x = 200 }  // OK
        someFunctionWithEscapingClosure { x = 100 }     // Error
    }
}
```

위의 예시에서 `someFunctionWithEscapingClosure` 함수 호출은 변경 가능한 메서드 내부에 있기 때문에 오류이고 `self`는 변경 가능합니다. 이것은 탈출 클로저는 구조체인 `self`를 변경가능한 참조로 캡처할 수 없다는 규칙을 위반합니다.

자동 클로저 (Autoclosures)

*자동 클로저(autoclosure)*는 함수에 인자로 전달되는 표현식을 감싸기 위해 자동으로 생성되는 클로저입니다. 이 클로저는 인자를 가지지 않으며, 호출될 때, 내부에 감싸진 표현식의 값을 반환합니다. 이러한 구문상의 편의를 통해 명시적 클로저 대신에 일반 표현식을 작성하여 함수의 매개변수 주위의 중괄호를 생략할 수 있습니다.

자동 클로저를 가지는 함수를 호출하는 것은 일반적이지만 이러한 함수를 구현하는 것은 일반적이지 않습니다. 예를 들어 `assert(condition:message:file:line:)` 함수는 `condition`과 `message` 매개변수에 대한 자동 클로저를 가집니다; `condition` 매개변수는 오직 디버그 빌드에서만 평가되고, `message` 매개변수는 `condition`이 `false`에서만 평가됩니다.

클로저가 호출될 때까지 코드 내부 실행이 되지 않기 때문에, 자동 클로저는 평가를 지연시킬 수 있습니다. 평가 지연은 사이드 이펙트가 있거나 계산 비용이 큰 코드를 제어된 시점에 실행하고자 할 때 유용합니다. 아래 코드는 클로저가 어떻게 평가를 지연하는지 보여줍니다.

```
var customersInLine = ["Chris", "Alex", "Ewa", "Barry", "Daniella"]
print(customersInLine.count)
// Prints "5".

let customerProvider = { customersInLine.remove(at: 0) }
print(customersInLine.count)
// Prints "5".

print("Now serving \(customerProvider())!")
// Prints "Now serving Chris!".
print(customersInLine.count)
// Prints "4".
```

클로저 내부의 코드에 의해 `customersInLine` 배열의 첫 번째 요소는 삭제되지만, 클로저가 실제로 호출되기 전까지 삭제되지 않습니다. 클로저가 호출되지 않으면, 클로저 내부의 표현식은 평가되지 않고, 배열의 요소도 삭제되지 않습니다. `customerProvider` 타입은 `String`이 아니고 `() -> String`입니다 - 매개변수가 없고 문자열을 반환하는 함수입니다.

함수의 인자로 클로저를 전달할 때도 동일한 지연 평가 동작을 가질 수 있습니다.

```
// customersInLine is ["Alex", "Ewa", "Barry", "Daniella"]
func serve(customer customerProvider: () -> String) {
    print("Now serving \(customerProvider())!")
}
serve(customer: { customersInLine.remove(at: 0) } )
// Prints "Now serving Alex!".
```

위에서 `serve(customer:)` 함수는 고객의 이름을 반환하는 명시적 클로저를 인자를 가집니다. 아래 `serve(customer:)`의 버전은 같은 동작을 수행하지만, 명시적 클로저를 가지는 대신에 매개변수 타입에 `@autoclosure` 속성을 표기하여 자동 클로저를 가집니다. 이제 이 함수는 마치 `String` 타입 값을 인자로 받는 것처럼 호출할 수 있습니다. `customerProvider` 매개변수의 타입은 `@autoclosure` 속성으로 표시되어 있기 때문에 인자는 자동으로 클로저로 변환됩니다.

```
// customersInLine is ["Ewa", "Barry", "Daniella"]
func serve(customer customerProvider: @autoclosure () -> String) {
    print("Now serving \(customerProvider())!")
}
serve(customer: customersInLine.remove(at: 0))
// Prints "Now serving Ewa!".
```

참고

자동 클로저 남용은 코드 이해를 어렵게 만들 수 있습니다. 평가가 지연된다는 사실이 컨텍스트와 함수 이름만으로도 명확히 드러나야 합니다.

탈출이 허용되는 자동 클로저를 사용하고 싶다면, `@autoclosure`와 `@escaping` 속성을 함께 사용해야 합니다. `@escaping` 속성은 위의 탈출 클로저 (Escaping Closures)에 설명되어 있습니다.

```
// customersInLine is ["Barry", "Daniella"]
var customerProviders: [() -> String] = []
func collectCustomerProviders(_ customerProvider: @autoclosure @escaping () -> String) {
    customerProviders.append(customerProvider)
}
collectCustomerProviders(customersInLine.remove(at: 0))
collectCustomerProviders(customersInLine.remove(at: 0))

print("Collected \(customerProviders.count) closures.")
// Prints "Collected 2 closures.".
for customerProvider in customerProviders {
    print("Now serving \(customerProvider())!")
}
// Prints "Now serving Barry!".
// Prints "Now serving Daniella!".
```

위의 코드에서 `customerProvider` 인자로 전달된 클로저를 호출하는 대신에 `collectCustomerProviders(_:)` 함수는 클로저를 `customerProviders` 배열에 추가합니다. 이 배열은 함

수의 범위 밖에 선언되어 있기 때문에, 배열에 저장된 클로저는 함수가 반환된 후에 실행될 수 있다는 의미입니다. 그 결과 `customerProvider` 인자의 값은 함수의 범위를 벗어날 수 있어야 합니다.

언어 가이드

열거형 (Enumerations)

가능한 값의 목록을 정의하는 커스텀 타입을 모델링합니다.

*열거형(enumeration)*은 관련된 값의 그룹을 위한 공통된 타입을 정의하고 이 값을 코드에서 타입 안전성을 유지하면서 다룰 수 있도록 해줍니다.

C에 익숙하다면, C 열거형이 관련된 이름을 정수값 집합에 매핑한다는 점을 알고 있을 것입니다. Swift에서의 열거형은 훨씬 유연하고, 열거형의 각 케이스에 값을 제공하지 않아도 됩니다. 값(원시값)이 각 열거형 케이스로 제공된다면, 그 값은 문자열, 문자, 정수, 부동소수점 타입일 수 있습니다.

또는 열거형 케이스는 다른 언어에서 유니언이나 variant로 알려진 기능과 유사하게 각 열거형 케이스에 연관값을 지정하여 케이스마다 서로 다른 타입의 값을 저장할 수도 있습니다. 하나의 열거형 내에 관련된 케이스를 정의하면서, 각 케이스에 적절한 타입의 다른 값을 연관시킬 수 있습니다.

Swift의 열거형은 그 자체로 일급 타입입니다. 열거형의 현재값에 대한 추가 정보를 제공하는 연산 프로퍼티(computed property)와 열거형이 나타내는 값과 관련된 기능을 제공하는 인스턴스 메서드와 같이 기존에는 클래스만 지원하던 많은 기능을 채택할 수 있습니다. 열거형은 케이스 초기값을 제공하기 위해 이니셜라이저를 정의할 수도 있습니다; 열거형은 기존 구현을 넘어 기능적으로 확장될 수도 있고; 표준 기능을 제공하기 위해 프로토콜을 채택할 수 있습니다.

이러한 기능의 자세한 내용은 프로퍼티 (Properties), 메서드 (Methods), 초기화 (Initialization), 확장 (Extensions), 프로토콜 (Protocols)을 참고바랍니다.

열거형 문법 (Enumeration Syntax)

열거형은 `enum` 키워드와 중괄호 안에 모든 정의를 위치시켜 나타냅니다:

```
enum SomeEnumeration {
    // enumeration definition goes here
}
```

다음 예시는 나침반의 네 가지 주요 방향을 나타냅니다:

```
enum CompassPoint {
    case north
    case south
    case east
    case west
}
```

열거형에 정의된 값 (`north`, `south`, `east`, `west`)은 *열거형 케이스(enumeration cases)*입니다. `case` 키워드를 사용하여 새로운 열거형 케이스를 선언합니다.

참고

Swift 열거형 케이스는 C와 Objective-C 같은 언어와는 달리, 기본적으로 정수값을 가지지 않습니다. 위 예시 `CompassPoint`에 `north`, `south`, `east`, `west`는 암시적으로 `0`, `1`, `2`, `3`에 해당하지 않습니다. 대신 각 열거형 케이스는 그 자체로 독립적인 값이며, 명시적으로 정의된 `CompassPoint` 타입을 가집니다.

여러 개의 케이스는 콤마로 구분하여 한 줄로 표기할 수 있습니다:

```
enum Planet {
    case mercury, venus, earth, mars, jupiter, saturn, uranus, neptune
}
```

각 열거형 정의는 새로운 타입을 정의합니다. Swift의 다른 타입처럼 열거형의 이름 (`CompassPoint`, `Planet`)은 대문자로 시작합니다. 열거형 타입 이름은 코드가 보다 자연스럽고 명확하게 읽히기 때문에, 복수형보다는 단수형을 사용하는 것이 좋습니다:

```
var directionToHead = CompassPoint.west
```

`directionToHead` 타입은 `CompassPoint`의 가능한 값 중 하나로 초기화될 때, 추론됩니다. 한 번 `directionToHead`가 `CompassPoint`로 선언되면, 이후에는 더 짧게 점 문법을 사용하여 다른 `CompassPoint` 값으로 설정할 수 있습니다:

```
directionToHead = .east
```

`directionToHead`의 타입은 이미 알고 있으므로, 값을 설정할 때 타입을 생략할 수 있습니다. 이러한 방식은 명시적으로 타입이 지정된 열거형 값으로 작업할 때 코드를 쉽게 읽을 수 있습니다.

스위치 구문에서 열거형 값 일치 (Matching Enumeration Values with a Switch Statement)

개별 열거형 값을 `switch` 구문을 사용하여 일치시킬 수 있습니다:

```
directionToHead = .south
switch directionToHead {
case .north:
    print("Lots of planets have a north")
case .south:
    print("Watch out for penguins")
case .east:
    print("Where the sun rises")
case .west:
    print("Where the skies are blue")
}
// Prints "Watch out for penguins".
```

이 코드를 아래와 같이 읽을 수 있습니다:

“`directionToHead` 값을 고려합니다. 그 값이 `.north`인 경우, `"Lots of planets have a north"`를 출력합니다. 그 값이 `.south`인 경우, `"Watch out for penguins"`를 출력합니다.”

…그리고 나머지 케이스도 마찬가지입니다.

제어 흐름 (Control Flow)에서 설명했듯이, `switch` 구문은 열거형 케이스를 고려할 때 완벽해야 합니다. `.west`에 대한 `case`가 생략된다면, `CompassPoint`의 모든 케이스를 고려하지 않았기 때문에 컴파일 되지 않습니다. 이러한 완전성 요구는 열거형 케이스를 실수로 누락하는 것을 방지합니다.

모든 열거형 케이스에 대해 `case`를 제공하는 것이 적절하지 않은 경우, 명시되지 않은 나머지 모든 케이스를 처리할 수 있도록 `default` 케이스를 제공할 수 있습니다:

```
let somePlanet = Planet.earth
switch somePlanet {
case .earth:
    print("Mostly harmless")
default:
    print("Not a safe place for humans")
}
// Prints "Mostly harmless".
```

열거형 케이스 반복 (Iterating over Enumeration Cases)

일부 열거형에서는 해당 열거형의 모든 케이스를 모아놓은 컬렉션이 있으면 유용합니다. 이를 위해 열거형 이름 뒤에 `: CaseIterable`을 작성하여 활성화 합니다. 그러면 Swift는 열거형 타입에 `allCases` 프로퍼티를 통해 모든 케이스의 컬렉션을 제공합니다. 다음은 그 예시입니다:

```
enum Beverage: CaseIterable {
    case coffee, tea, juice
}
let numberOfChoices = Beverage.allCases.count
print("\(numberOfChoices) beverages available")
// Prints "3 beverages available".
```

위의 예시에서 `Beverage` 열거형에 모든 케이스를 포함하는 컬렉션에 접근하기 위해 `Beverage.allCases`를 사용합니다. 일반 컬렉션처럼 `allCases`를 사용할 수 있습니다 - 컬렉션의 요소는 열거형 타입의 인스턴스이기 때문에 이 경우에 각 요소는 `Beverage` 값입니다. 위의 예시는 얼마나 많은 케이스가 존재하는지 계산하고, 아래의 예시는 `for-in` 루프를 사용하여 모든 케이스를 반복합니다.

```
for beverage in Beverage.allCases {
    print(beverage)
}
// coffee
// tea
// juice
```

위 예시에서 사용된 문법은 열거형이 `CaseIterable` 프로토콜을 준수하도록 표시한 것입니다. 프로토콜에 대한 자세한 내용은 프로토콜 (Protocols)을 참조 바랍니다.

연관값 (Associated Values)

이전 섹션의 예시는 열거형 케이스가 그 자체로 정의된(그리고 타입이 지정된) 값임을 보여줍니다. 상수나 변수를 `Planet.earth`로 설정하고, 나중에 값을 확인할 수 있습니다. 그러나 경우에 따라 각 케이스 값에 다른 타입의 데이터를 함께 저장해야 할 때가 있습니다. 이러한 추가적인 정보를 *연관값(associated value)*이라고 하며, 이 값은 해당 케이스를 사용할 때마다 달라질 수 있습니다.

Swift의 열거형은 어떤 타입이든 연관값을 저장할 수 있도록 정의할 수 있으며, 필요하다면 케이스마다 서로 다른 타입의 연관값을 가질 수도 있습니다. 이와 유사한 개념은 다른 프로그래밍 언어에서 식별 집합체(discriminated unions), 태그 집합체(tagged unions), *변형 가능한 집합체(variants)*로 알려져 있습니다.

예를 들어 재고 추적 시스템이 두 가지 타입의 바코드로 제품을 추적해야 된다고 가정해 보겠습니다. 어떤 제품은 숫자 `0`에서 `9`를 사용하는 UPC 형식의 1D 바코드 레이블이 부착되어 있습니다. 각 바코드에는 숫자 시스템과 이어서 다섯 자리의 제조업체 코드와 다섯 자리의 제품 코드가 있습니다. 다음에는 코드가 올바르게 스캔되었는지 확인하기 위해 검사 숫자가 있습니다:



다른 제품은 모든 ISO 8859-1 문자도 사용할 수 있고 2,953개의 문자까지 인코딩할 수 있는 QR 코드 형식의 2D 바코드 레이블이 부착되어 있습니다:



재고 추적 시스템은 UPC 바코드를 네 개의 정수로 된 튜플로 저장하고 QR 코드 바코드를 모든 길이의 문자열로 저장하는 것이 편리합니다.

Swift에서 두 타입의 바코드를 정의하는 열거형은 다음과 같습니다:

```
enum Barcode {
    case upc(Int, Int, Int, Int)
    case qrCode(String)
}
```

이것은 아래와 같이 읽을 수 있습니다:

"(`Int, Int, Int, Int`) 타입의 연관값을 가진 `upc`나 `String` 타입의 연관값을 가진 `qrCode`를 취할 수 있는 `Barcode`라는 열거형 타입을 정의합니다."

이 정의는 실제 `Int`나 `String` 값을 제공하지 않습니다 - 이것은 단지 `Barcode.upc`나 `Barcode.qrCode`와 같을 때, `Barcode` 상수와 변수에 저장할 수 있는 연관값의 타입을 정의할 뿐입니다.

그러면 이러한 타입을 이용하여 새로운 바코드를 생성할 수 있습니다:

```
var productBarcode = Barcode.upc(8, 85909, 51226, 3)
```

이 예시는 `productBarcode`라는 새로운 변수를 생성하고, 연관값으로 `(8, 85909, 51226, 3)`이라는 튜플을 가진 `Barcode.upc` 값을 할당합니다.

같은 제품에 대해 다른 바코드 타입을 할당할 수도 있습니다:

```
productBarcode = .qrCode("ABCDEFGHIJKLMNOP")
```

이 시점에서 기존의 `Barcode.upc`와 그에 해당하는 정수값은 새로운 `Barcode.qrCode`와 해당 문자열로 대체됩니다. `Barcode` 타입의 상수와 변수는 `.upc`나 `.qrCode` (연관값과 함께) 저장할 수 있지만 동시에 둘 다 저장할 수 없습니다.

스위치 구문에서 열거형 값 일치 (Matching Enumeration Values with a Switch Statement)에서의 예시와 유사하게 스위치 구문을 이용하여 바코드 타입을 확인할 수 있습니다. 그러나 이번에는 스위치 구문을 통해 연관값을 추출할 수 있습니다. `switch` 케이스의 본문 내에서 사용하기 위해 상수(`let` 접두사)나 변수(`var` 접두사)로 연관값을 추출합니다:

```
switch productBarcode {
case .upc(let numberSystem, let manufacturer, let product, let check):
    print("UPC: \(numberSystem), \(manufacturer), \(product), \(check).")
case .qrCode(let productCode):
    print("QR code: \(productCode).")
}
// Prints "QR code: ABCDEFGHIJKLMNOP.".
```

열거형 케이스의 연관값 모두 상수로 추출하거나 변수로 추출하려면, 케이스 이름 앞에 `let`이나 `var`을 선언하여 표현할 수 있습니다:

```
switch productBarcode {
case let .upc(numberSystem, manufacturer, product, check):
    print("UPC : \(numberSystem), \(manufacturer), \(product), \(check).")
case let .qrCode(productCode):
    print("QR code: \(productCode).")
}
// Prints "QR code: ABCDEFGHIJKLMNOP.".
```

열거형의 케이스 중 하나만 일치시키는 경우에 - 예를 들어 일치되는 케이스의 연관값을 추출하는 경우 - 스위치 구문의 전체 케이스를 작성하는 대신 `if-case` 구문을 사용할 수 있습니다. 다음은 그 예시입니다:

```
if case .qrCode(let productCode) = productBarcode {
    print("QR code: \(productCode).")
}
```

이 코드의 `productBarcode` 변수는 `.qrCode(let productCode)` 패턴과 일치합니다. 그리고 스위치 케이스에서와 같이 `let`은 상수로 연관값을 추출합니다. `if-case` 구문에 대한 자세한 내용은 패턴 (Patterns)을 참고바랍니다.

원시값 (Raw Values)

연관값 (Associated Values)에서의 바코드 예시는 열거형 케이스에 다른 타입의 연관값을 저장할 수 있음을 보여줍니다. 연관값과는 별개로, 열거형 케이스는 모두 동일한 타입의 기본값 (원시값(raw values))으로 미리 채워질 수도 있습니다.

다음은 명명된 열거형 케이스와 함께 ASCII 값을 저장하는 예시입니다:

```
enum ASCIIControlCharacter: Character {
    case tab = "\t"
    case lineFeed = "\n"
    case carriageReturn = "\r"
}
```

여기서 `ASCIIControlCharacter`라는 열거형의 원시값은 `Character` 타입으로 정의되고, 일반적인 ASCII 제어 문자 중 일부를 설정합니다. `Character` 값은 문자열과 문자 (Strings and Characters)에 설명되어 있습니다.

원시값은 문자열, 문자, 정수 또는 부동소수점 숫자 타입이 가능합니다. 각 원시값은 열거형 선언 내에서 유일한 값이어야 합니다.

열거형에 추가값을 부여하기 위해 원시값과 연관값 모두 사용할 수 있지만 둘의 차이점을 이해하는 것이 중요합니다. 원시값은 위의 세 가지 ASCII 코드와 같이 코드에서 열거형 케이스를 정의할 때 선택합니다. 특정 열거형 케이스의 원시값은 항상 동일합니다. 반면에 연관값은 열거형 케이스 중 하나를 사용하여 새로운 상수나 변수를 생성할 때 선택하고, 특정 열거형 케이스의 연관값은 다른 값을 선택할 수 있습니다.

암시적으로 할당된 원시값 (Implicitly Assigned Raw Values)

정수나 문자열 원시값이 저장된 열거형으로 동작 시, 각 케이스에 명시적으로 원시값을 가질 필요가 없습니다. 원시값을 설정하지 않으면 Swift는 자동으로 값을 할당합니다.

예를 들어 정수를 원시값으로 사용하면, 각 케이스의 암시적 값은 이전 케이스보다 하나씩 증가시킵니다. 첫 번째 케이스에 값이 지정되지 않으면 `0`으로 설정합니다.

아래의 열거형은 태양으로부터 각 행성의 순서를 정수값으로 나타내는 이전 `Planet` 열거형을 확장한 버전입니다:

```
enum Planet: Int {
    case mercury = 1, venus, earth, mars, jupiter, saturn, uranus, neptune
}
```

위의 예시에서 `Planet.mercury`는 명시적으로 원시값 `1`을 가지고, `Planet.venus`는 암시적으로 원시값 `2`를 가집니다.

원시값으로 문자열이 사용될 때, 각 케이스의 이름 자체가 암시적으로 원시값으로 자동 할당됩니다.

아래의 열거형은 각 방향의 이름을 문자열 원시값으로 나타내는 이전 `CompassPoint` 열거형을 확장한 버전입니다:

```
enum CompassPoint: String {
    case north, south, east, west
}
```

위의 예시에서 `CompassPoint.south`는 암시적으로 `"south"`라는 원시값을 가지고, 다른 케이스도 마찬가지로 케이스 이름을 원 값으로 자동 할당받습니다.

`rawValue` 프로퍼티를 사용하여 열거형 케이스의 원시값에 접근할 수 있습니다:

```
let earthsOrder = Planet.earth.rawValue
// earthsOrder is 3

let sunsetDirection = CompassPoint.west.rawValue
// sunsetDirection is "west"
```

원시값으로 초기화 (Initializing from a Raw Value)

원시값 타입으로 열거형을 정의하면, 열거형은 원시값의 타입(`rawValue`라는 매개변수)을 사용하는 이니셜라이저를 제공하고 열거형 케이스나 `nil`을 반환합니다. 이 이니셜라이저를 사용하여 열거형의 새 인스턴스를 생성할 수 있습니다.

이 예시는 7 원시값으로 천왕성을 식별합니다:

```
let possiblePlanet = Planet(rawValue: 7)
// possiblePlanet is of type Planet? and equals Planet.uranus
```

모든 `Int` 값이 행성과 일치하는 것은 아닙니다. 이러한 점 때문에, 원시값 이니셜라이저는 항상 옵셔널 열거형 케이스를 반환합니다. 위의 예시에서 `possiblePlanet`은 `Planet?` 타입이나 "옵셔널 `Planet`" 타입입니다.

> **참고**
>
> 원시값 이니셜라이저는 모든 원시값을 열거형 케이스로 반환할 수 없으므로, 실패 가능 이니셜라이저입니다. 자세한 내용은 실패 가능한 이니셜라이저 (Failable Initializers)를 참고바랍니다.

11의 위치로 행성을 찾는다면, 원시값 이니셜라이저에서 반환된 옵셔널 `Planet` 값은 `nil`입니다:

```
let positionToFind = 11
if let somePlanet = Planet(rawValue: positionToFind) {
    switch somePlanet {
    case .earth:
        print("Mostly harmless")
    default:
        print("Not a safe place for humans")
    }
} else {
    print("There isn't a planet at position \(positionToFind)")
}
// Prints "There isn't a planet at position 11".
```

이 예시는 11의 원시값으로 행성을 찾기위해 옵셔널 바인딩을 사용합니다. `if let somePlanet = Planet(rawValue: 11)` 구문은 옵셔널 `Planet`을 생성하고, 해당 값이 존재한다면 옵셔널 `Planet`의 값을 `somePlanet`에 설정합니다. 이 경우 11의 위치로 행성을 가져올 수 없으므로, `else` 구문이 실행됩니다.

재귀 열거형 (Recursive Enumerations)

*재귀 열거형(recursive enumeration)*은 열거형 케이스에 하나 이상의 연관값으로 열거형의 다른 인스턴스를 가지고 있는 열거형입니다. 열거형 케이스가 재귀적임을 나타내기 위해 케이스 작성 전에 `indirect`를 작성하여, 컴파일러에게 간접 참조(indirection) 계층을 삽입하도록 지시합니다.

예를 들어 다음은 간단한 산술 표현식을 저장하는 열거형입니다:

```
enum ArithmeticExpression {
    case number(Int)
    indirect case addition(ArithmeticExpression, ArithmeticExpression)
    indirect case multiplication(ArithmeticExpression, ArithmeticExpression)
}
```

열거형 시작 전에 `indirect`를 작성하여 연관값을 가진 모든 열거형 케이스에 간접 참조를 적용할 수 있습니다:

```
indirect enum ArithmeticExpression {
    case number(Int)
    case addition(ArithmeticExpression, ArithmeticExpression)
    case multiplication(ArithmeticExpression, ArithmeticExpression)
}
```

이 열거형은 세 가지 종류의 산술 표현식을 저장할 수 있습니다: 단순 숫자, 두 표현식의 덧셈, 두 표현식의 곱셈. `addition`과 `multiplication` 케이스는 산술 표현식인 연관값을 가지고 있습니다 - 이 연관값은 중첩 표현식을 가능하게 해줍니다. 예를 들어 `(5 + 4) * 2` 표현식은 곱셈의 오른쪽은 하나의 숫자를 가지고 있고 왼쪽은 다른 표현식을 가지고 있습니다. 데이터는 중첩되기 때문에, 데이터를 저장하는 열거형도 중첩을 지원해야 합니다 - 이것은 재귀 열거형을 의미입니다. 아래의 코드는 `(5 + 4) * 2`에 대해 생성되는 `ArithmeticExpression` 재귀 열거형을 나타냅니다:

```
let five = ArithmeticExpression.number(5)
let four = ArithmeticExpression.number(4)
let sum = ArithmeticExpression.addition(five, four)
let product = ArithmeticExpression.multiplication(sum, ArithmeticExpression.number(2))
```

재귀 함수는 재귀 구조를 가진 데이터로 작업하는 직관적인 방법입니다. 예를 들어 다음은 산술 표현식을 평가하는 함수입니다:

```
func evaluate(_ expression: ArithmeticExpression) -> Int {
    switch expression {
    case let .number(value):
        return value
    case let .addition(left, right):
        return evaluate(left) + evaluate(right)
    case let .multiplication(left, right):
        return evaluate(left) * evaluate(right)
    }
}

print(evaluate(product))
// Prints "18".
```

이 함수는 단순 숫자인 경우 해당 연관값을 그대로 반환하여 평가합니다. 덧셈이나 곱셈인 경우에는 왼쪽 표현식을 평가하고, 오른쪽 표현식을 평가한 다음에 두 결과를 더하거나 곱하여 평가합니다.

언어 가이드

구조체와 클래스 (Structures and Classes)

데이터를 캡슐화하는 커스텀 타입을 모델링합니다.

*구조체(Structures)*와 *클래스(classes)*는 범용적이고 유용한 구조로, 프로그램 코드를 구성하는 핵심적인 빌딩 블록 역할을 합니다. 상수, 변수, 함수를 정의하는 것과 같은 구문을 사용하여 구조체와 클래스에 프로퍼티와 메서드를 정의하여 기능을 추가할 수 있습니다.

다른 프로그래밍 언어와 달리, Swift는 커스텀 구조체와 클래스에 대해 별도의 인터페이스와 구현 파일을 만들 필요가 없습니다. Swift에서 단일 파일로 구조체나 클래스를 정의하면, 해당 클래스나 구조체에 대한 외부 인터페이스가 자동으로 다른 코드에서 사용할 수 있습니다.

> **참고**
>
> 클래스의 인스턴스는 전통적으로 *객체(object)*라고 불립니다. 그러나 Swift에서는 구조체와 클래스가 다른 언어보다 훨씬 유사한 기능을 제공하기 때문에, 이 챕터의 대부분에서는 구조체와 클래스 인스턴스 모두에 적용되는 공통 기능을 설명합니다. 따라서 이 문서에서는 더 일반적인 용어인 *인스턴스(instance)*를 사용합니다.

구조체와 클래스 비교 (Comparing Structures and Classes)

Swift에서 구조체와 클래스는 공통점이 많습니다. 둘 다 다음과 같은 기능을 지원합니다:

- 값을 저장하기 위한 프로퍼티 정의
- 기능을 제공하기 위한 메서드 정의
- 서브스크립트를 통해 값에 접근할 수 있는 문법 제공
- 초기화 상태를 설정하기 위한 이니셜라이저 정의
- 기본 구현을 넘어서 기능을 확장할 수 있도록 확장 지원
- 특정 기능을 표준화하기 위한 프로토콜 채택 가능

더 자세한 내용은 프로퍼티 (Properties), 메서드 (Methods), 서브스크립트 (Subscripts), 초기화 (Initialization), 확장 (Extensions), 프로토콜 (Protocols)을 참고바랍니다.

클래스는 구조체에 없는 추가적인 기능도 지원합니다:

- 상속을 사용하면 다른 클래스의 특성을 상속할 수 있습니다.
- 타입 캐스팅을 사용하면 런타임에 클래스 인스턴스의 타입을 확인하고 해석할 수 있습니다.
- 디이니셜라이저(Deinitializers)을 사용하면 클래스의 인스턴스가 할당된 리소스를 해제할 수 있습니다.
- 참조 카운팅은 하나의 클래스 인스턴스를 여러 참조가 공유할 수 있습니다.

더 자세한 내용은 상속 (Inheritance), 타입 캐스팅 (Type Casting), 소멸 (Deinitialization), 자동 참조 카운팅 (Automatic Reference Counting)을 참고바랍니다.

클래스가 지원하는 추가 기능은 더 높은 복잡성을 수반합니다. 일반적인 지침으로는 구조체는 예측 가능하고 이해하기 쉬우므로 구조체를 선호하며, 클래스는 필요한 경우에만 사용하는 것이 좋습니다. 실제 개발에서는 대부분의 커스텀 타입이 구조체나 열거형으로 작성됩니다. 더 자세한 비교는 구조체와 클래스 선택 (Choosing Between Structures and Classes)을 참고바랍니다.

> **참고**
>
> 클래스와 액터(actor)는 많은 특성과 동작을 공유합니다. 액터에 대한 자세한 내용은 동시성 (Concurrency)을 참고바랍니다.

정의 문법 (Definition Syntax)

구조체와 클래스는 유사한 문법으로 정의됩니다. 구조체는 `struct` 키워드로 클래스는 `class` 키워드로 시작합니다. 둘 경우 모두 중괄호 안에 전체 정의를 작성합니다:

```
struct SomeStructure {
    // structure definition goes here
}
class SomeClass {
    // class definition goes here
}
```

> **참고**
>
> 새로운 구조체나 클래스를 정의할 때마다, 새로운 Swift 타입을 정의합니다. Swift의 표준 타입 (`String`, `Int`, `Bool` 등)과 일관되도록, 타입 이름은 `UpperCamelCase` (`SomeStructure`, `SomeClass` 등)로 작성합니다. 프로퍼티와 메서드는 타입 이름과 구분을 위해 `lowerCamelCase` (`frameRate`, `incrementCount` 등)으로 작성합니다.

다음은 구조체 정의와 클래스 정의의 예시입니다:

```
struct Resolution {
    var width = 0
    var height = 0
}
class VideoMode {
    var resolution = Resolution()
    var interlaced = false
    var frameRate = 0.0
    var name: String?
}
```

위 예시는 픽셀 기반의 화면 해상도를 설명하는 `Resolution`이라는 새로운 구조체를 정의합니다. 이 구조체는 `width`와 `height`라 불리는 저장 프로퍼티를 가지고 있습니다. 저장 프로퍼티는 구조체나 클래스의 일부로 함께 저장되는 상수나 변수입니다. 이 두 프로퍼티는 정수값 `0`으로 초기값이 설정되므로 `Int` 타입으로 추론됩니다.

위 예시는 비디오 화면에 대한 특정 비디오 모드를 나타내는 `VideoMode`라는 새로운 클래스를 정의합니다. 이 클래스는 네 개의 저장 프로퍼티를 가지고 있습니다. 첫 번째 `resolution`은 새로운 `Resolution` 인스턴스로 초기화되며, 이에 따라 해당 프로퍼티의 타입은 자동으로 `Resolution`으로 추론됩니다. 다른 세 프로퍼티는, `interlaced`는 `false`("비인터레이스 비디오")로 재생 프레임 속도는 `0.0`으로 `name`은 옵셔널 `String` 값으로 초기화 되어 새로운 `VideoMode` 인스턴스를 초기화합니다. `name` 프로퍼티는 옵셔널 타입이므로 기본값은 자동으로 `nil`이며, "`name` 값 없음"입니다.

구조체와 클래스 인스턴스 (Structure and Class Instances)

`Resolution` 구조체 정의와 `VideoMode` 클래스 정의는 `Resolution`이나 `VideoMode`가 어떤 구조를 가지는지만 설명합니다. 특정 해상도나 비디오 모드를 직접 나타내는 것은 아닙니다. 실제 데이터를 사용하려면, 해당 구조체나 클래스의 인스턴스 생성이 필요합니다.

인스턴스 생성 문법은 구조체와 클래스 모두 매우 유사합니다:

```
let someResolution = Resolution()
let someVideoMode = VideoMode()
```

구조체와 클래스 모두 새로운 인스턴스를 생성하기위해 이니셜라이저를 사용합니다. 이니셜라이저의 가장 단순한 형태는 `Resolution()`나 `VideoMode()`와 같이 클래스나 구조체 타입 이름 뒤에 빈 소괄호를 붙여 사용하는 것입니다. 이렇게 하면 모든 프로퍼티가 기본값으로 초기화되는 클래스나 구조체의 새로운 인스턴스를 생성합니다. 클래스와 구조체 초기화에 대한 자세한 내용은 초기화 (Initialization)를 참고바랍니다.

프로퍼티 접근 (Accessing Properties)

*점 표기법(dot syntax)*을 사용하여 인스턴스의 프로퍼티에 접근할 수 있습니다. 점 표기법에서는 인스턴스 이름 뒤에 마침표(`.`)를 붙이고, 그 뒤에 공백없이 프로퍼티 이름을 작성합니다:

```
print("The width of someResolution is \(someResolution.width)")
// Prints "The width of someResolution is 0".
```

이 예시에서 `someResolution.width`는 `someResolution` 인스턴스의 `width` 프로퍼티를 참조하고 기본값 `0`을 반환합니다.

`VideoMode` 인스턴스의 `resolution` 프로퍼티에 `width` 프로퍼티와 같이 프로퍼티 안의 하위 프로퍼티에 접근할 수 있습니다:

```
print("The width of someVideoMode is \(someVideoMode.resolution.width)")
// Prints "The width of someVideoMode is 0".
```

변수 프로퍼티에 새로운 값을 할당하기 위해 점 표기법을 사용할 수 있습니다:

```
someVideoMode.resolution.width = 1280
print("The width of someVideoMode is now \(someVideoMode.resolution.width)")
// Prints "The width of someVideoMode is now 1280".
```

구조체의 멤버와이즈 이니셜라이저 (Memberwise Initializers for Structure Types)

모든 구조체는 새로운 구조체 인스턴스의 멤버 프로퍼티를 초기화 할 때 사용할 수 있는 자동으로 생성되는 *멤버와이즈 이니셜라이저(memberwise intializer)*를 가집니다. 새로운 인스턴스의 프로퍼티 초기값은 이름으로 멤버와이즈 초기화에 전달할 수 있습니다:

```
let vga = Resolution(width: 640, height: 480)
```

구조체와 다르게, 클래스 인스턴스는 멤버와이즈 초기화를 제공하지 않습니다. 초기화에 대한 자세한 설명은 초기화 (Initialization)을 참고바랍니다.

구조체와 열거형은 값 타입 (Structures and Enumerations Are Value Types)

*값 타입(value type)*은 변수나 상수에 할당되거나 함수에 전달될 때 복사되는 타입입니다.

실제로 값 타입에 대해 이전 챕터에서 광범위하게 다뤘습니다. 실제로 Swift의 기본 타입 - 정수, 부동소수점, Boolean, 문자열, 배열, 딕셔너리 - 는 모두 값 타입이며, 내부적으로 구조체로 구현되어 있습니다.

Swift에서 모든 구조체와 열거형은 값 타입입니다. 이것은 생성한 구조체와 열거형 인스턴스와 - 가지고 있는 프로퍼티 중 모든 값 타입은 - 코드에서 전달되거나 할당될 때 항상 복사됩니다.

> **참고**
>
> 배열, 딕셔너리, 문자열과 같은 Swift 표준 라이브러리에 정의된 컬렉션은 복사에 따른 성능 비용을 줄이기 위해 최적화된 방식을 사용합니다. 복사를 즉시 수행하지 않고, 원본 인스턴스와 복사본이 동일한 메모리 영역을 공유합니다. 컬렉션 중 하나가 수정되면, 요소는 수정되기 직전에 복사됩니다. 코드에서 보이는 동작은 항상 바로 복사가 일어나는 것처럼 보입니다.

이전 예시에서의 `Resolution` 구조체를 사용하는 다음 예시를 살펴봅시다:

```
let hd = Resolution(width: 1920, height: 1080)
var cinema = hd
```

이 예시는 `hd`라는 상수를 선언하고 풀 HD 비디오 (1920 픽셀 너비와 1080 픽셀 높이)의 너비와 높이를 초기화하는 `Resolution` 인스턴스를 생성합니다.

그리고 나서 `cinema`라는 변수를 선언하고 `hd`의 현재값을 설정합니다. `Resolution`은 구조체이므로, 기존 인스턴스의 복사본이 만들어지고 이 복사본이 `cinema`에 할당됩니다. `hd`와 `cinema`가 현재 같은 너비와 높이를 가지지만, 내부적으로는 완벽하게 다른 인스턴스 입니다.

다음으로 `cinema`의 `width` 프로퍼티를 디지털 시네마 프로젝션에 사용되는 약간 더 넓은 2K 표준 (2048 픽셀 너비와 1080 픽셀 높이)으로 수정됩니다:

```
cinema.width = 2048
```

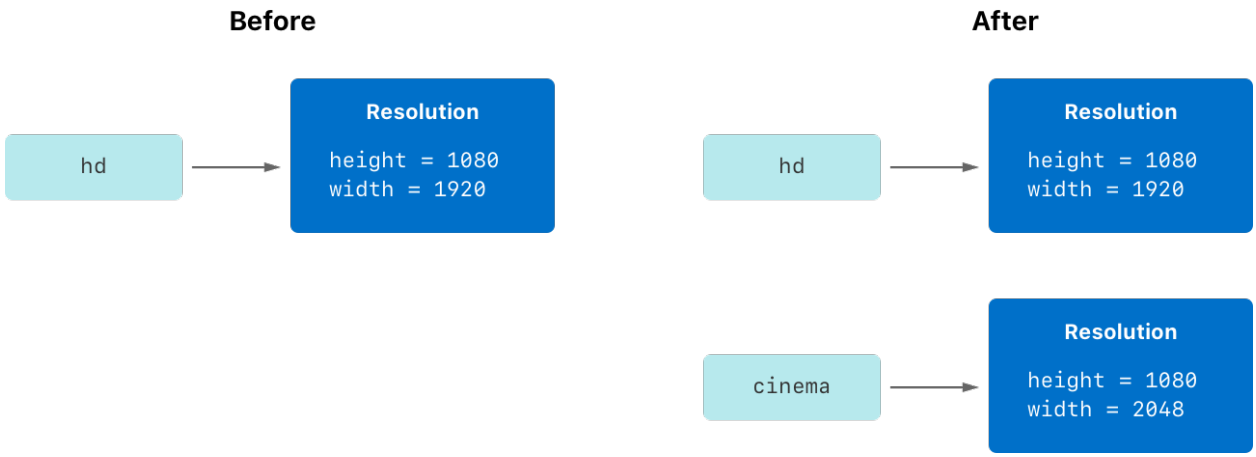
`cinema`의 `width` 프로퍼티를 확인하면 `2048`로 바뀐 것을 확인할 수 있습니다:

```
print("cinema is now \(cinema.width) pixels wide")
// Prints "cinema is now 2048 pixels wide".
```

그러나 기존 `hd` 인스턴스의 `width` 프로퍼티는 `1920`의 기존값을 그대로 가지고 있습니다:

```
print("hd is still \(hd.width) pixels wide")
// Prints "hd is still 1920 pixels wide".
```

`cinema`에 `hd`의 현재값이 주어졌을 때, `hd`에 저장된 값은 새로운 `cinema` 인스턴스에 복사됩니다. 그 결과 두 인스턴스는 동일한 숫자 값을 가지지만, 서로 완전히 독립적인 인스턴스 입니다. 분리된 인스턴스이기 때문에, 아래의 그림과 같이 `cinema`에 너비를 `2048`로 설정해도 `hd`에 저장된 너비에는 영향을 주지 않습니다:



열거형에서도 같은 동작이 이뤄집니다:

```
enum CompassPoint {
    case north, south, east, west
    mutating func turnNorth() {
        self = .north
    }
}
var currentDirection = CompassPoint.west
let rememberedDirection = currentDirection
currentDirection.turnNorth()

print("The current direction is \(currentDirection)")
print("The remembered direction is \(rememberedDirection)")
// Prints "The current direction is north".
// Prints "The remembered direction is west".
```

`rememberedDirection`에 `currentDirection`의 값이 할당될 때, 실질적으로 해당 값의 복사본이 설정됩니다. 이후에 `currentDirection`의 값을 변경해도 `rememberedDirection`의 저장된 원래 값 복사본에는 영향을 주지 않습니다.

클래스는 참조 타입 (Classes Are Reference Types)

값 타입과 다르게, *참조 타입(reference types)*은 변수나 상수에 할당될 때나 함수로 전달될 때 복사되지 않습니다. 복사본 대신에 존재하는 같은 인스턴스에 대한 참조가 사용됩니다.

다음은 위에 정의된 `VideoMode` 클래스를 사용하는 예시입니다:

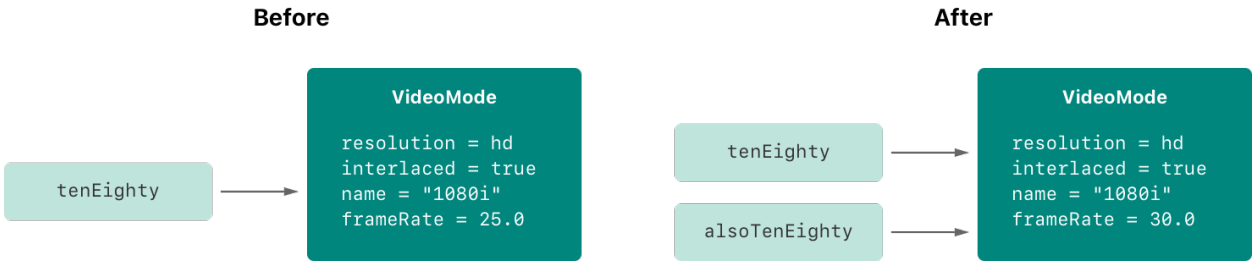
```
let tenEighty = VideoMode()
tenEighty.resolution = hd
tenEighty.interlaced = true
tenEighty.name = "1080i"
tenEighty.frameRate = 25.0
```

이 예시는 `tenEighty`라는 새로운 상수를 선언하고 `VideoMode` 클래스의 새로운 인스턴스를 참조하도록 설정합니다. 비디오 모드는 이전에 `1920` x `1080`의 HD 해상도의 복사본이 할당됩니다. 인터레이스로 설정되고 이름을 "`1080i`"로 설정하고 프레임 속도를 초당 `25.0` 프레임으로 설정합니다.

다음으로 `tenEighty`는 `alsoTenEighty`라는 새로운 상수에 할당되고 `alsoTenEighty`의 프레임 속도를 수정합니다:

```
let alsoTenEighty = tenEighty
alsoTenEighty.frameRate = 30.0
```

클래스는 참조 타입이므로 `tenEighty`와 `alsoTenEighty`는 실질적으로 같은 `VideoMode` 인스턴스를 참조합니다. 실제로는 아래 그림과 같이 하나의 인스턴스에 다른 두 개의 이름을 가지고 있는 것입니다:



`tenEighty`의 `frameRate` 프로퍼티를 확인하면 `VideoMode` 인스턴스에서 `30.0`의 새로운 프레임 속도가 올바르게 설정된 것을 보여줍니다:

```
print("The frameRate property of tenEighty is now \(tenEighty.frameRate)")
// Prints "The frameRate property of tenEighty is now 30.0".
```

이 예시는 참조 타입이 얼마나 추론하기 어려울 수 있는지 보여줍니다. `tenEighty`와 `alsoTenEighty`가 프로그램 내에서 서로 멀리 떨어진 위치에 존재한다면, 비디오 모드가 어디서 어떻게 변경되는지 추적하기 어려울 수 있습니다. `tenEighty`를 사용하는 코드가 있다면, `alsoTenEighty`를 사용하는 코드 역시 고려해야 하고, 그 반대도 마찬가지입니다. 이에 비해 값 타입은 동일한 값과 상호작용하는 모든 코드가 소스 파일에 가까이 있기 때문에 이해하고 유지보수하기 훨씬 쉽습니다.

`tenEighty`와 `alsoTenEighty`는 변수가 아닌 상수로 선언됩니다. 그러나 `tenEighty` 와 `alsoTenEighty` 상수 자체는 실제로 변경되지 않으므로 `tenEighty.frameRate`와 `alsoTenEighty.frameRate`는 여전히 변경 가능합니다. `tenEighty`와 `alsoTenEighty` 자체는 `VideoMode` 인스턴스를 “저장”하지 않습니다 - 대신에 `VideoMode` 인스턴스를 둘 다 참조합니다. 변경되는 것은 `VideoMode` 인스턴스 자체가 아니라, `VideoMode` 인스턴스 내부의 `frameRate` 프로퍼티 입니다.

동일성 연산자 (Identity Operators)

클래스는 참조 타입이기 때문에, 여러 상수나 변수가 내부적으로 동일한 클래스 인스턴스를 참조할 수 있습니다. (구조체와 열거형은 상수나 변수에 할당되거나 함수에 전달될 때 복사되기 때문에 클래스와 같지 않습니다.)

두 개의 상수나 변수가 같은 클래스 인스턴스를 참조하는지 확인하는 것이 유용할 수 있습니다. 이를 위해 Swift는 두 가지 동일성 연산자를 제공합니다:

- 동일 인스턴스(`===`)
- 동일하지 않은 인스턴스(`!==`)

이 연산자를 사용하여 두 상수나 변수가 하나의 동일한 인스턴스를 참조하는지 확인할 수 있습니다:

```
if tenEighty === alsoTenEighty {
    print("tenEighty and alsoTenEighty refer to the same VideoMode instance.")
}
// Prints "tenEighty and alsoTenEighty refer to the same VideoMode instance.".
```

동일 인스턴스(세 개의 등호로 표시나 ===)는 같음(equal to)(두 개의 등호로 표시나 ==)과 같다는 의미는 아닙니다. 동일 인스턴스는 클래스 타입의 두 상수나 변수가 동일한 클래스 인스턴스를 참조한다는 의미입니다. 같음은 두 인스턴스가 값의 관점에서 동일한지, 타입 설계자가 정의한 "동등함"의 기준에 따라 값이 같다고 간주되는지를 의미합니다.

커스텀 구조체와 클래스를 정의할 때, 두 인스턴스가 같은지의 여부를 결정하는 것은 사용자의 몫입니다. ==와 != 연산자의 사용자 정의 방법은 동등 연산자 (Equivalence Operators)에 자세히 설명되어 있습니다.

포인터 (Pointers)

C, C++, 또는 Objective-C에 익숙하다면, 메모리의 주소를 참조하기 위해 *포인터(pointers)*를 사용한다는 것을 알고 있을 것입니다. 어떤 참조 타입의 인스턴스를 가리키기위해 Swift 상수나 변수는 C의 포인터와 유사하지만, 메모리 주소를 직접 가리키는 포인터가 아니며, 포인터 표시를 위해 별표(*)를 작성할 필요가 없습니다. 대신 Swift에서는 이러한 참조를 다른 상수나 변수를 선언하듯이 간단하게 정의할 수 있습니다. 직접 포인터와 상호작용해야 하는 경우를 위해 Swift 표준 라이브러리는 포인터 및 버퍼 타입을 제공합니다 - 자세한 내용은 수동 메모리 관리 (Manual Memory Management)를 참고바랍니다.

언어 가이드

프로퍼티 (Properties)

인스턴스나 타입의 부분인 저장된 값과 계산된 값에 접근합니다.

*프로퍼티(Properties)*는 값을 특정 클래스, 구조체, 열거형에 연결합니다. 저장 프로퍼티(Stored properties)는 인스턴스의 일부로 상수와 변수 값을 저장하는 반면에 연산 프로퍼티(computed properties)는 값을 저장하는 대신에 계산합니다. 연산 프로퍼티(Computed properties)는 클래스, 구조체, 열거형에서 사용할 수 있습니다. 저장 프로퍼티는 클래스와 구조체에서만 제공됩니다.

저장 프로퍼티와 연산 프로퍼티는 보통 특정 타입의 인스턴스와 관련되어 있습니다. 그러나 프로퍼티는 타입 자체에 연결될 수도 있습니다. 이러한 프로퍼티를 타입 프로퍼티라 합니다.

또한 프로퍼티 관찰자(property observers)를 정의하여 프로퍼티의 값이 변경되는 것을 감지하고 커스텀 동작에 응답할 수 있습니다. 프로퍼티 관찰자는 직접 정의한 저장 프로퍼티와 상속받은 프로퍼티에도 추가할 수 있습니다.

프로퍼티 래퍼를 사용하여 여러 프로퍼티의 getter와 setter에서 코드를 재사용 할 수 있습니다.

저장 프로퍼티 (Stored Properties)

가장 단순한 형태의 저장 프로퍼티는 특정 클래스나 구조체 인스턴스의 부분으로 저장되는 상수나 변수입니다. 저장 프로퍼티는 변수 저장 프로퍼티(variable stored properties)(`var` 키워드 사용)나 상수 저장 프로퍼티(constant stored properties)(`let` 키워드 사용)로 쓸 수 있습니다.

기본 프로퍼티 값 (Default Property Values)에서 설명한 대로 저장 프로퍼티에는 정의할 때 기본값을 제공할 수 있습니다. 초기화 중에 저장 프로퍼티에 초기값을 설정하고 수정할 수도 있습니다. 초기화 중 상수 프로퍼티 값 할당 (Assigning Constant Properties During Initialization)에서 설명한 대로 상수 저장 프로퍼티에도 마찬가지 입니다.

아래 예시는 `FixedLengthRange`라는 구조체를 정의합니다. 이 구조체는 범위 길이가 생성된 후에 변경할 수 없는 정수 범위를 나타냅니다:

```
struct FixedLengthRange {
    var firstValue: Int
    let length: Int
}
var rangeOfThreeItems = FixedLengthRange(firstValue: 0, length: 3)
// the range represents integer values 0, 1, and 2
rangeOfThreeItems.firstValue = 6
// the range now represents integer values 6, 7, and 8
```

`FixedLengthRange`의 인스턴스는 `firstValue`라는 변수 저장 프로퍼티가 있으며 `length`라는 상수 저장 프로퍼티가 있습니다. 위의 예시에서 `length`는 새 범위가 생성될 때 초기화되며 상수 프로퍼티이므로 그 이후에는 변경할 수 없습니다.

상수 구조체 인스턴스의 저장 프로퍼티 (Stored Properties of Constant Structure Instances)

구조체 인스턴스를 생성하고 해당 인스턴스를 상수에 할당하면 해당 인스턴스의 프로퍼티가 변수로 선언되어 있어도 인스턴스의 프로퍼티를 수정할 수 없습니다:

```
let rangeOfFourItems = FixedLengthRange(firstValue: 0, length: 4)
// this range represents integer values 0, 1, 2, and 3
rangeOfFourItems.firstValue = 6
// this will report an error, even though firstValue is a variable property
```

`rangeOfFourItems`는 `let` 키워드를 사용하여 상수로 선언되어 있기 때문에, `firstValue`가 변수 프로퍼티로 선언되었더라도 `firstValue` 프로퍼티를 변경할 수 없습니다.

이러한 동작은 구조체가 값 타입이기 때문입니다. 값 타입의 인스턴스가 상수로 선언되면 인스턴스의 모든 프로퍼티에 대해 수정할 수 없습니다.

클래스는 참조 타입이므로 다르게 동작합니다. 참조 타입의 인스턴스를 상수로 할당하더라도, 인스턴스의 변수 프로퍼티는 변경할 수 있습니다.

지연 저장 프로퍼티 (Lazy Stored Properties)

*지연 저장 프로퍼티(lazy stored property)*는 처음 사용될 때까지 초기값이 계산되지 않는 프로퍼티입니다. 지연 저장 프로퍼티는 선언 앞에 `lazy` 수정자를 붙여 나타냅니다.

> **참고**
>
> 인스턴스 초기화가 완료된 후에 초기값을 가져올 수도 있기 때문에 지연 프로퍼티는 `var` 키워드를 사용해 변수로 선언해야 합니다. 상수 프로퍼티는 초기화가 완료되기 전에 값을 가지고 있어야 하므로, 지연 프로퍼티로 선언할 수 없습니다.

지연 프로퍼티는 인스턴스의 초기화가 완료될 때까지 알 수 없는 외부 요인에 따라 초기값이 결정될 경우에 유용합니다. 지연 프로퍼티는 초기값 설정이 복잡하거나 계산 비용이 많이 들 때, 실제로 필요해질 때까지 초기화를 미루고 싶을 때에도 유용합니다.

아래의 예시는 복잡한 클래스에 불필요한 초기화를 피하기 위해 지연 저장 프로퍼티를 사용합니다. 이 예시는 `DataImporter`와 `DataManager`라는 두 개의 클래스를 정의하며, 이 클래스의 전체 내용은 생략되어 있습니다:

```
class DataImporter {
    /*
    DataImporter is a class to import data from an external file.
    The class is assumed to take a nontrivial amount of time to initialize.
    */
    var filename = "data.txt"
    // the DataImporter class would provide data importing functionality here
}

class DataManager {
    lazy var importer = DataImporter()
    var data = [String]()
    // the DataManager class would provide data management functionality here
}

let manager = DataManager()
manager.data.append("Some data")
manager.data.append("Some more data")
// the DataImporter instance for the importer property has not yet been created
```

`DataManager` 클래스는 빈 `String` 배열로 초기화되는 `data`라는 저장 프로퍼티를 가지고 있습니다. 나머지 내용은 보이진 않지만, `DataManager` 클래스의 목적은 `String` 데이터 배열을 관리하고 접근을 제공합니다.

`DataManager` 클래스의 기능 중 일부는 파일에서 데이터를 가져오는 기능입니다. 이 기능은 `DataImporter` 클래스로부터 제공되고, 초기화하는데 시간이 많이 걸린다고 가정합니다. `DataImporter` 인스턴스가 초기화될 때 `DataImporter` 인스턴스는 파일을 열고 메모리로 내용을 읽어야 하기 때문일 수 있습니다.

`DataManager` 인스턴스는 파일로부터 데이터를 가져올 필요 없이 데이터를 관리할 수 있으므로, `DataManager`가 생성될 때 새로운 `DataImporter` 인스턴스를 생성할 필요가 없습니다. 대신에 `DataImporter` 인스턴스를 처음 사용하는 경우에 생성하는 것이 더 적절합니다.

`lazy` 수정자가 붙어 있으므로, `importer` 프로퍼티의 `DataImporter` 인스턴스는 `filename` 프로퍼티를 조회할 때처럼 `importer` 프로퍼티에 처음 접근할 때 생성됩니다:

```
print(manager.importer.filename)
// the DataImporter instance for the importer property has now been created
// Prints "data.txt".
```

참고

`lazy` 수정자가 표시된 프로퍼티는 여러 스레드에서 동시에 접근되고 프로퍼티가 아직 초기화되지 않은 경우 프로퍼티가 한 번만 초기화된다는 보장이 없습니다.

저장 프로퍼티와 인스턴스 변수 (Stored Properties and Instance Variables)

Objective-C에 익숙하다면, 클래스 인스턴스의 일부로 값과 참조를 저장하는 두 가지 방법을 제공한다는 것을 알 수 있습니다. 프로퍼티 외에도, 인스턴스 변수를 사용하여 프로퍼티에 저장되는 값의 백업 저장소로 사용할 수 있습니다.

Swift는 하나의 프로퍼티 선언으로 이러한 개념을 통합합니다. Swift의 프로퍼티는 대응되는 인스턴스 변수를 가지지 않으며, 프로퍼티의 백업 저장소는 직접적으로 접근할 수 없습니다. 이 접근방식은 다양한 상황에서 값에 접근하는 것에 대한 혼동을 피할 수 있으며, 프로퍼티 선언을 하나의 명확한 구문으로 단순화합니다. 이름, 타입, 메모리 관리 특성을 포함하는 - 프로퍼티에 대한 모든 정보는 - 타입 정의의 일부로 단일 위치에 정의됩니다.

연산 프로퍼티 (Computed Properties)

저장 프로퍼티 외에도 클래스, 구조체, 열거형은 값을 실질적으로 저장하지 않는 *연산 프로퍼티(computed properties)*를 정의할 수 있습니다. 대신 다른 프로퍼티와 값을 간접적으로 조회하고 설정하는 getter와 선택적인 setter를 제공합니다.

```
struct Point {
    var x = 0.0, y = 0.0
}
struct Size {
    var width = 0.0, height = 0.0
}
struct Rect {
    var origin = Point()
    var size = Size()
    var center: Point {
        get {
            let centerX = origin.x + (size.width / 2)
            let centerY = origin.y + (size.height / 2)
            return Point(x: centerX, y: centerY)
        }
        set(newCenter) {
```

```
            origin.x = newCenter.x - (size.width / 2)
            origin.y = newCenter.y - (size.height / 2)
        }
    }
}
var square = Rect(origin: Point(x: 0.0, y: 0.0),
                  size: Size(width: 10.0, height: 10.0))
let initialSquareCenter = square.center
// initialSquareCenter is at (5.0, 5.0)
square.center = Point(x: 15.0, y: 15.0)
print("square.origin is now at (\(square.origin.x), \(square.origin.y))")
// Prints "square.origin is now at (10.0, 10.0)".
```

이 예시는 도형을 다루기 위한 세 가지 구조체를 정의합니다:

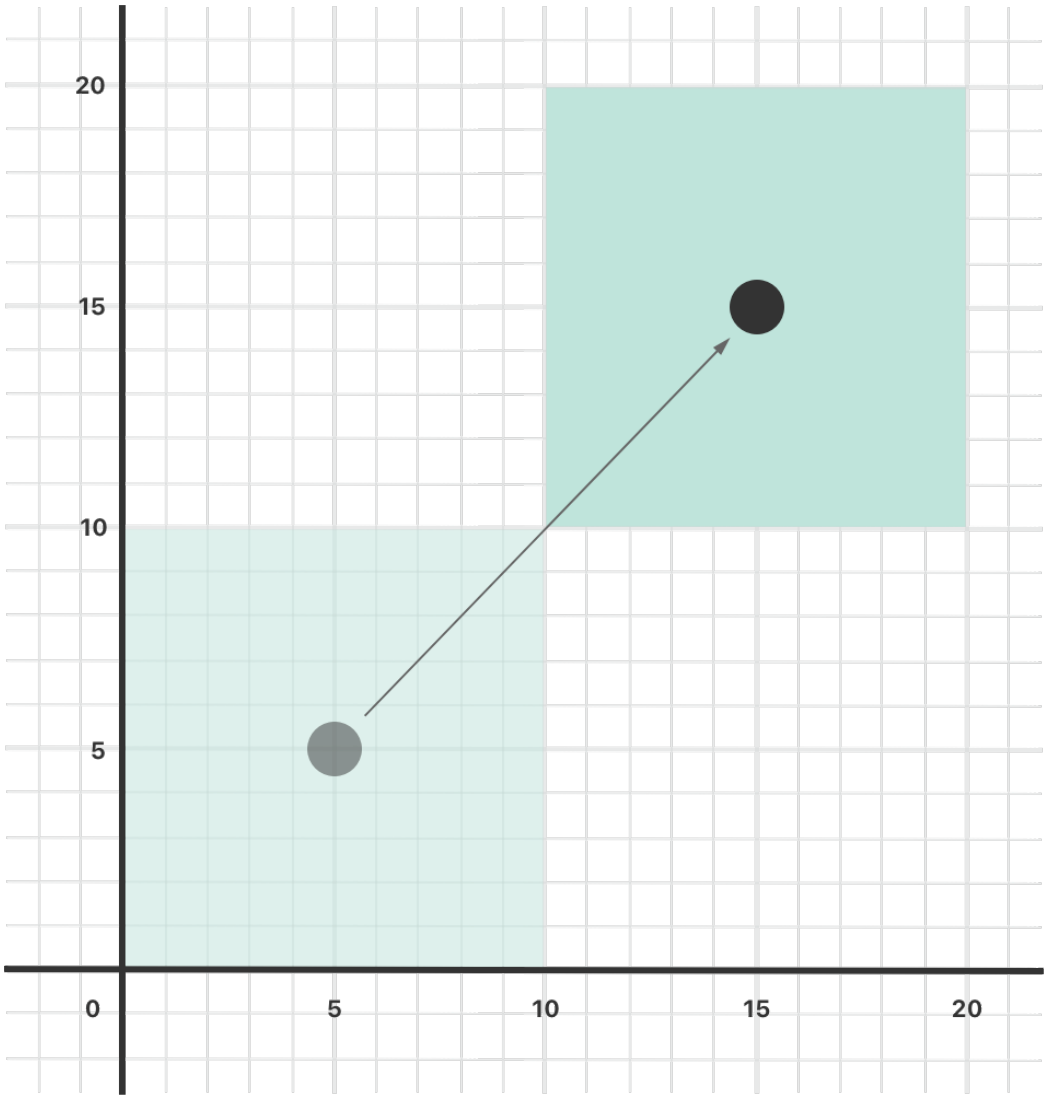
- `Point`는 x, y 좌표의 위치를 캡슐화합니다.
- `Size`는 `width`와 `height`를 캡슐화합니다.
- `Rect`는 원점과 크기로 사각형을 정의합니다.

`Rect` 구조체는 `center`라는 연산 프로퍼티를 제공합니다. `Rect`에 현재 중심 위치는 항상 `origin`과 `size`로 계산될 수 있고, 명시적인 `Point` 값으로 저장할 필요가 없습니다. 대신에 `Rect`는 `center`라는 연산 변수에 대해 커스텀 getter와 setter를 정의하고, 실제 저장 프로퍼티처럼 사각형의 `center`를 동작하도록 합니다.

위의 예시는 `square`라는 새로운 `Rect` 변수를 생성합니다. `square` 변수는 `(0, 0)`의 원점으로 너비와 높이를 `10`으로 초기화 됩니다. 이 사각형은 아래의 다이어그램에서 연한 초록색 사각형으로 표현됩니다.

`square` 변수의 `center` 프로퍼티는 점 표기법(`square.center`)으로 접근되고, `center`에 대한 getter를 호출하여 현재 프로퍼티 값을 가져옵니다. 기존값을 반환하는 대신, getter는 실질적으로 사각형의 중심을 나타내는 새로운 `Point`를 계산하고 반환합니다. 위에서 볼 수 있듯이 getter는 `(5, 5)`의 중심점을 반환합니다.

아래 다이어그램의 진한 초록색 사각형처럼 `center` 프로퍼티는 `(15, 15)`의 새로운 값으로 설정되어 새로운 위치로 사각형이 위와 우측으로 이동합니다. `center` 프로퍼티를 설정하면 `center`의 setter를 호출하고, `origin` 프로퍼티에 저장된 `x`와 `y` 값을 변경하고 사각형을 새로운 위치로 이동합니다.



간략한 Setter 선언 (Shorthand Setter Declaration)

연산 프로퍼티의 setter에서 설정할 새로운 값에 대한 이름을 정의하지 않으면, `newValue`라는 기본 이름이 사용됩니다. 다음은 이러한 간략한 표기법의 이점을 가지는 `Rect` 구조체를 나타냅니다:

```
struct AlternativeRect {
    var origin = Point()
    var size = Size()
    var center: Point {
        get {
            let centerX = origin.x + (size.width / 2)
            let centerY = origin.y + (size.height / 2)
            return Point(x: centerX, y: centerY)
        }
        set {
            origin.x = newValue.x - (size.width / 2)
            origin.y = newValue.y - (size.height / 2)
        }
    }
}
```

간략한 Getter 선언 (Shorthand Getter Declaration)

getter의 전체 본문이 단일 표현식이라면, getter는 암시적으로 해당 표현식을 반환합니다. 다음은 간략한 getter와 setter를 모두 활용한 `Rect` 구조체를 나타냅니다:

```
struct CompactRect {
    var origin = Point()
    var size = Size()
    var center: Point {
        get {
            Point(x: origin.x + (size.width / 2),
                  y: origin.y + (size.height / 2))
        }
        set {
            origin.x = newValue.x - (size.width / 2)
            origin.y = newValue.y - (size.height / 2)
        }
    }
}
```

getter에서 생략한 `return`은 암시적 반환을 가진 함수 (Functions With an Implicit Return)에서 설명된 함수의 `return` 생략 규칙과 동일합니다.

읽기 전용 연산 프로퍼티 (Read-Only Computed Properties)

setter가 없고 getter만 있는 연산 프로퍼티는 *읽기 전용 연산 프로퍼티(read-only computed property)*라고 합니다. 읽기 전용 연산 프로퍼티는 항상 값을 반환하고, 점 표기법으로 접근할 수 있지만 다른 값을 설정할 수 없습니다.

> **참고**
>
> 읽기 전용 연산 프로퍼티를 포함하여 - 연산 프로퍼티는 - 값이 고정되어 있지 않기 때문에 `var` 키워드를 사용해 변수 프로퍼티로 선언되어야 합니다. `let` 키워드는 인스턴스 초기화의 부분으로 한 번 설정되면 값을 변경할 수 없음을 나타내기 위해 오직 상수 프로퍼티에만 사용됩니다.

`get` 키워드와 중괄호를 생략함으로써 읽기 전용 연산 프로퍼티를 간편하게 선언할 수 있습니다:

```
struct Cuboid {
    var width = 0.0, height = 0.0, depth = 0.0
    var volume: Double {
        return width * height * depth
    }
}
let fourByFiveByTwo = Cuboid(width: 4.0, height: 5.0, depth: 2.0)
print("the volume of fourByFiveByTwo is \(fourByFiveByTwo.volume)")
// Prints "the volume of fourByFiveByTwo is 40.0".
```

이 예시는 `width`, `height`, `depth` 프로퍼티를 가지는 3D 직육면체를 나타내는 `Cuboid`라는 새로운 구조체를 정의합니다. 이 구조체는 직육면체의 현재 부피를 계산하고 반환하는 `volume`이라는 읽기 전용 연산 프로퍼티를 가지고 있습니다. 특정 `volume` 값을 위해 어떤 `width`, `height`, `depth` 값을 사용해야 하는지 모호하므로, `volume`을 설정 가능하게 만드는 것은 적절하지 않습니다. 그럼에도 불구하고 현재 계산된 부피를 외부 사용자가 알 수 있도록 읽기 전용 연산 프로퍼티로 제공되는 `Cuboid`는 유용합니다.

프로퍼티 관찰자 (Property Observers)

프로퍼티 관찰자(Property observers)는 프로퍼티의 값이 변경되는지 관찰하고 응답합니다. 프로퍼티 관찰자는 프로퍼티의 현재값이 새로운 값과 같더라도 프로퍼티의 값이 설정될 때 호출됩니다.

다음과 같은 위치에 프로퍼티 관찰자를 추가할 수 있습니다:

- 사용자가 정의한 저장 프로퍼티
- 상속받은 저장 프로퍼티
- 상속받은 연산 프로퍼티

상속된 프로퍼티의 경우, 하위 클래스에서 해당 프로퍼티를 재정의하여 프로퍼티 관찰자를 추가할 수 있습니다. 정의한 연산 프로퍼티의 경우, 관찰자를 생성하는 대신에 프로퍼티의 setter를 이용하여 값 변경을 관찰하고 응답해야 합니다. 재정의 프로퍼티에 대한 자세한 내용은 재정의 (Overriding)를 참고바랍니다.

프로퍼티에는 다음 두 가지 관찰자 중 하나 또는 둘 다 정의할 수 있습니다:

- `willSet`은 값이 저장되기 직전에 호출됩니다.
- `didSet`은 새로운 값이 저장된 직후에 호출됩니다.

`willSet` 관찰자를 구현하면, 상수 매개변수로 새로 설정될 프로퍼티 값이 전달됩니다. `willSet` 구현의 일부로 이 매개변수에 이름을 지정할 수 있습니다. 매개변수 명과 구현 내에 소괄호를 작성하지 않으면, 매개변수는 `newValue`라는 기본 매개변수 명으로 사용됩니다.

마찬가지로 `didSet` 관찰자를 구현하면, 이전 프로퍼티 값을 포함한 상수 매개변수가 전달됩니다. 매개변수 명을 사용하거나 `oldValue`인 기본 매개변수 명을 사용할 수 있습니다. `didSet` 관찰자 내에서 해당 프로퍼티에 새로운 값을 할당하면, 방금 설정된 값을 새로운 값으로 대체합니다.

> **참고**
>
> 상위 클래스의 프로퍼티에 대한 `willSet`과 `didSet` 관찰자는 상위 클래스 초기화가 완료된 후 하위 클래스 이니셜라이저에서 해당 프로퍼티가 설정될 때 호출됩니다. 이것은 클래스가 이니셜라이저 본문에서 프로퍼티를 설정하는 동안에는 호출되지 않습니다.
>
> 초기화 위임에 대한 자세한 내용은 값 타입을 위한 이니셜라이저 위임 (Initializer Delegation for Value Types)과 클래스 타입의 이니셜라이저 위임 (Initializer Delegation for Class Types)을 참고바랍니다.

다음은 `willSet`과 `didSet` 동작에 대한 예시입니다. 아래 예시는 사람이 걷는 동안의 걸음 수를 측정하는 `StepCounter`라는 새로운 클래스를 정의합니다. 이 클래스는 만보계나 다른 걸음 측정기의 입력 데이터와 함께 사용하여 일상생활에서 사람의 운동 루틴을 추적할 수 있습니다.

```
class StepCounter {
    var totalSteps: Int = 0 {
        willSet(newTotalSteps) {
            print("About to set totalSteps to \(newTotalSteps)")
        }
        didSet {
            if totalSteps > oldValue  {
                print("Added \(totalSteps - oldValue) steps")
            }
        }
    }
}
```

```
let stepCounter = StepCounter()
stepCounter.totalSteps = 200
// About to set totalSteps to 200
// Added 200 steps
stepCounter.totalSteps = 360
// About to set totalSteps to 360
// Added 160 steps
stepCounter.totalSteps = 896
// About to set totalSteps to 896
// Added 536 steps
```

`StepCounter` 클래스는 `Int` 타입의 `totalSteps` 프로퍼티를 선언합니다. 이것은 `willSet`과 `didSet` 관찰자를 가진 저장 프로퍼티 입니다.

`totalSteps`의 `willSet`과 `didSet` 관찰자는 새로운 값이 프로퍼티에 할당될 때마다 호출됩니다. 새로운 값이 현재값과 같더라도 항상 호출됩니다.

이 예시의 `willSet` 관찰자는 새로운 값을 위한 `newTotalSteps`의 사용자 매개변수 명을 사용합니다. 이 예시는 단순히 설정될 값을 출력합니다.

`didSet` 관찰자는 `totalSteps` 값이 업데이트된 후에 호출됩니다. 이 관찰자는 새로운 `totalSteps` 값과 이전 값을 비교합니다. 걸음 수가 증가했다면, 걸음 수가 얼마나 증가하였는지 출력합니다. `didSet` 관찰자는 이전 값에 대한 사용자 매개변수 명을 제공하지 않고, 대신에 `oldValue`의 기본 이름을 사용합니다.

> **참고**
>
> 관찰자를 가진 프로퍼티를 in-out 매개변수로 함수에 전달하면, `willSet`과 `didSet` 관찰자는 항상 호출됩니다. 이것은 in-out 매개변수에 대한 복사-입력(copy-in) 및 복사-출력(copy-out) 메모리 모델 때문에 그렇습니다: 함수가 끝날 때 값이 항상 프로퍼티로 다시 작성되기 때문에 관찰자가 호출되는 것입니다. in-out 매개변수에 대한 자세한 내용은 In-Out 매개변수 (In-Out Parameters)를 참고바랍니다.

프로퍼티 래퍼 (Property Wrappers)

프로퍼티 래퍼(property wrapper)는 프로퍼티가 저장되는 방법을 관리하는 코드와 프로퍼티를 정의하는 코드 사이에 분리 계층을 추가합니다. 예를 들어 스레드 안전성 검사를 제공하거나 기본 데이터를 데이터베이스에 저장하는 프로퍼티가 있는 경우 모든 프로퍼티에 해당 코드를 작성해야 합니다. 프로퍼티 래퍼를 사용하면, 이러한 관리 코드를 래퍼를 정의할 때 한 번만 작성하고, 여러 프로퍼티에 재사용할 수 있습니다.

프로퍼티 래퍼를 정의하려면, `wrappedValue` 프로퍼티를 정의한 구조체, 열거형, 클래스를 만들면 됩니다. 아래 코드에서 `TwelveOrLess` 구조체는 래핑된 값이 항상 12 이하의 숫자만 포함됩니다. 더 큰 숫자를 저장하려고 하면 12가 저장됩니다.

```
@propertyWrapper
struct TwelveOrLess {
    private var number = 0
    var wrappedValue: Int {
        get { return number }
        set { number = min(newValue, 12) }
    }
}
```

setter는 새로운 값이 12 이하인지 확인하고, getter는 저장된 값을 반환합니다.

참고

위의 예시에서 `number` 프로퍼티는 `TwelveOrLess`의 구현에서만 `number`가 사용될 수 있도록 `private`로 선언되어 있습니다. 다른곳에서 작성된 코드는 `wrappedValue`를 위한 getter와 setter를 사용하여 값에 접근하고 `number` 자체에는 직접 접근할 수 없습니다. `private`에 대한 정보는 접근 제어 (Access Control)를 참고바랍니다.

프로퍼티에 래퍼를 적용하려면, 래퍼의 이름을 속성처럼 프로퍼티 앞에 작성하면 됩니다. 다음은 사각형의 크기를 정하는 구조체로 `TwelveOrLess` 프로퍼티 래퍼를 사용하여 너비와 높이가 항상 12 이하가 되도록 보장합니다:

```
struct SmallRectangle {
    @TwelveOrLess var height: Int
    @TwelveOrLess var width: Int
}

var rectangle = SmallRectangle()
print(rectangle.height)
// Prints "0".

rectangle.height = 10
print(rectangle.height)
// Prints "10".

rectangle.height = 24
print(rectangle.height)
// Prints "12".
```

`height`와 `width` 프로퍼티는 `TwelveOrLess.number`를 0으로 설정하는 `TwelveOrLess` 정의에서 초기값을 얻습니다. `TwelveOrLess`의 setter는 10을 유효한 값으로 간주하기 때문에, `rectangle.height`에 10을 저장하는 것은 성공적으로 저장합니다. 그러나 24는 `TwelveOrLess`에서 허용하는 값보다 크기 때문에, 24를 저장하려고 하면 `rectangle.height`는 대신 12로 설정되며, 이는 허용된 최대값입니다.

프로퍼티에 래퍼를 적용하면, 컴파일러는 래퍼를 위한 저장소를 제공하는 코드와 래퍼를 통해 프로퍼티에 접근하는 코드를 자동으로 생성합니다. (프로퍼티 래퍼가 감싸진 값을 저장하는 역할을 하기 때문에, 값 자체를 저장하는 코드는 따로 생성되지 않습니다.) 특별한 속성 구문을 사용하지 않고도, 프로퍼티 래퍼의 동작을 직접 코드로 구현할 수도 있습니다. 예를 들어 다음은 이전 코드에서 등장한 `SmallRectangle`을 `@TwelveOrLess` 속성 없이 `TwelveOrLess` 구조체를 명시적으로 사용하여 프로퍼티를 래핑한 버전입니다:

```
struct SmallRectangle {
    private var _height = TwelveOrLess()
    private var _width = TwelveOrLess()
    var height: Int {
        get { return _height.wrappedValue }
        set { _height.wrappedValue = newValue }
    }
    var width: Int {
        get { return _width.wrappedValue }
        set { _width.wrappedValue = newValue }
    }
}
```

`_height`와 `_width` 프로퍼티는 프로퍼티 래퍼인 `TwelveOrLess`의 인스턴스를 저장합니다. `height`와 `width`의 getter와 setter는 각각 `wrappedValue` 프로퍼티에 접근합니다.

래핑된 프로퍼티에 초기값 설정 (Setting Initial Values for Wrapped Properties)

위 예시의 코드는 `TwelveOrLess` 정의 안에서 `number`에 초기값을 부여함으로써, 래핑된 프로퍼티의 초기값을 설정합니다. 이 프로퍼티 래퍼를 사용한 코드는 `TwelveOrLess`로 래핑된 프로퍼티에 다른 초기값을 지정할 수 없습니다 - 예를 들어 `SmallRectangle` 정의에 `height`나 `width` 초기값을 지정할 수 없습니다. 초기값 설정을 지원하거나 다른 커스터마이징을 지원하려면, 프로퍼티 래퍼에 이니셜라이저를 추가해야 합니다. 다음은 래핑된 값과 최대값을 설정하는 초기화를 정의하는 `SmallNumber`라는 확장된 `TwelveOrLess`를 나타냅니다:

```
@propertyWrapper
struct SmallNumber {
    private var maximum: Int
    private var number: Int

    var wrappedValue: Int {
        get { return number }
        set { number = min(newValue, maximum) }
    }

    init() {
        maximum = 12
        number = 0
    }
    init(wrappedValue: Int) {
        maximum = 12
        number = min(wrappedValue, maximum)
    }
    init(wrappedValue: Int, maximum: Int) {
        self.maximum = maximum
        number = min(wrappedValue, maximum)
    }
}
```

`SmallNumber`의 정의는 세 가지 이니셜라이저가 포함되어 있으며 - `init()`, `init(wrappedValue:)`, `init(wrappedValue:maximum:)` 가 포함되어 있으며 - 아래 예시에서는 이 이니셜라이저를 사용해 래핑된 값과 최대값을 설정합니다. 초기화와 이니셜라이저에 대한 자세한 내용은 초기화 (Initialization)를 참고바랍니다.

프로퍼티에 래퍼를 적용할 때 초기값을 지정하지 않으면, Swift는 래퍼를 설정하기 위해 `init()`을 사용합니다. 예를 들어:

```
struct ZeroRectangle {
    @SmallNumber var height: Int
    @SmallNumber var width: Int
}

var zeroRectangle = ZeroRectangle()
print(zeroRectangle.height, zeroRectangle.width)
// Prints "0 0".
```

`height`와 `width`를 래핑하는 `SmallNumber` 인스턴스는 `SmallNumber()`를 호출하여 생성됩니다. 초기화 내부의 코드는 기본값인 0과 12를 사용하여 초기 래핑 값과 초기 최대값을 설정합니다. 이 방식은 이전에 `SmallRectangle`에서 `TwelveOrLess`를 사용했던 예전처럼, 프로퍼티 래퍼가 모든 초기값을 제공합니다. 예시와 다르게 `SmallNumber`이 초기값을 프로퍼티 선언 시 직접 작성할 수 있다는 점에서 다릅니다.

프로퍼티에 초기값을 지정하면, Swift는 래퍼를 설정하기 위해 `init(wrappedValue:)`를 사용합니다. 예를 들어:

```
struct UnitRectangle {
    @SmallNumber var height: Int = 1
    @SmallNumber var width: Int = 1
}

var unitRectangle = UnitRectangle()
print(unitRectangle.height, unitRectangle.width)
// Prints "1 1".
```

프로퍼티에 `= 1`과 같이 초기값을 작성하면, 이는 `init(wrappedValue:)` 초기화 호출로 변환됩니다. `height`와 `width`를 래핑하는 `SmallNumber` 인스턴스는 `SmallNumber(wrappedValue: 1)`을 호출하여 생성됩니다. 이 이니셜라이저는 여기서 지정된 래핑 값을 사용하고, 최대값은 기본값인 12를 사용합니다.

사용자 지정 속성 뒤에 소괄호로 인자를 작성하면, Swift는 해당 인자를 받는 이니셜라이저를 사용하여 래퍼를 설정합니다. 예를 들어 초기값과 최대값을 제공하면, Swift는 `init(wrappedValue:maximum:)` 이니셜라이저를 사용합니다:

```
struct NarrowRectangle {
    @SmallNumber(wrappedValue: 2, maximum: 5) var height: Int
    @SmallNumber(wrappedValue: 3, maximum: 4) var width: Int
}

var narrowRectangle = NarrowRectangle()
print(narrowRectangle.height, narrowRectangle.width)
// Prints "2 3".

narrowRectangle.height = 100
narrowRectangle.width = 100
print(narrowRectangle.height, narrowRectangle.width)
// Prints "5 4".
```

`height`를 래핑하는 `SmallNumber` 인스턴스는 `SmallNumber(wrappedValue: 2, maximum: 5)`를 호출하여 생성되고, `width`를 래핑하는 인스턴스는 `SmallNumber(wrappedValue: 3, maximum: 4)`를 호출하여 생성됩니다.

프로퍼티 래퍼에 인자를 포함하면, 래퍼의 초기 상태를 설정하거나 래퍼가 생성될 때 다른 옵션을 전달할 수 있습니다. 이 구문은 프로퍼티 래퍼를 사용하는 가장 일반적인 방법입니다. 필요한 인자를 속성에 제공하면, 해당 인자가 이니셜라이저에 전달됩니다.

프로퍼티 래퍼 인자를 포함하면, 할당을 사용하여 초기값을 지정할 수도 있습니다. Swift는 할당을 `wrappedValue` 인자처럼 처리하고 저장한 인자를 수용하는 이니셜라이저를 사용합니다. 예를 들어:

```
struct MixedRectangle {
    @SmallNumber var height: Int = 1
    @SmallNumber(maximum: 9) var width: Int = 2
}

var mixedRectangle = MixedRectangle()
print(mixedRectangle.height)
// Prints "1".

mixedRectangle.height = 20
print(mixedRectangle.height)
// Prints "12".
```

`height`를 래핑하는 `SmallNumber` 인스턴스는 `SmallNumber(wrappedValue: 1)`을 호출하여 생성되며, 이때 최대값은 기본값인 12가 사용됩니다. `width`를 래핑하는 인스턴스는 `SmallNumber(wrappedValue: 2, maximum: 9)`를 호출하여 생성됩니다.

프로퍼티 래퍼에서 값 투영 (Projecting a Value From a Property Wrapper)

래핑된 값 외에도 프로퍼티 래퍼는 *투영값(projected value)*을 정의하여 추가적인 기능을 제공할 수 있습니다 - 예를 들어 데이터베이스 접근을 관리하는 프로퍼티 래퍼는 투영값으로 `flushDatabaseConnection()` 메서드를 노출할 수 있습니다. 투영값의 이름은 앞에 달러 표시($)가 붙는 것을 제외하면 래핑된 값과 동일합니다. 코드에서 `$`로 시작하는 프로퍼티를 정의할 수 없기 때문에 투영값은 정의한 프로퍼티와 충돌하지 않습니다.

위의 `SmallNumber` 예시에서 프로퍼티에 너무 큰 수를 설정하려고 하면, 프로퍼티 래퍼가 값을 조정한 뒤 저장합니다. 아래의 코드는 새로운 값을 저장하기 전에 프로퍼티 래퍼가 값을 조정했는지 판단하기 위해 `SmallNumber` 구조체에 `projectedValue` 프로퍼티를 추가합니다.

```
@propertyWrapper
struct SmallNumber {
    private var number: Int
    private(set) var projectedValue: Bool

    var wrappedValue: Int {
        get { return number }
        set {
            if newValue > 12 {
                number = 12
                projectedValue = true
            } else {
                number = newValue
                projectedValue = false
            }
        }
    }

    init() {
        self.number = 0
        self.projectedValue = false
    }
}
struct SomeStructure {
    @SmallNumber var someNumber: Int
}
var someStructure = SomeStructure()

someStructure.someNumber = 4
print(someStructure.$someNumber)
// Prints "false".

someStructure.someNumber = 55
print(someStructure.$someNumber)
// Prints "true".
```

`someStructure.$someNumber`처럼 작성하면 해당 래퍼의 투영값에 접근할 수 있습니다. 4와 같은 작은 숫자를 저장한 후에, `someStructure.$someNumber`의 값은 `false`입니다. 그러나 55와 같은 큰 숫자를 저장하려고 한 경우에는 투영값은 `true`입니다.

프로퍼티 래퍼는 어떤 타입이든 투영값으로 반환할 수 있습니다. 이 예시에서 프로퍼티 래퍼는 숫자가 변경되었는지에 대한 - 정보만 노출합니다 - 그래서 투영값으로 Boolean 값을 노출합니다. 더 많은 정보를 제공해야 하는 래퍼의 경우 다른 타입의 인스턴스를 반환하거나, `self`를 투영값으로 반환하여 래퍼 인스턴스 전체를 노출할 수 있습니다.

타입 내부의 코드에서 투영값에 접근할 때, 프로퍼티 getter나 인스턴스 메서드와 다른 프로퍼티를 접근할 때처럼 프로퍼티 이름 앞에 `self.`을 생략할 수 있습니다. 다음 예시의 코드는 `$height`와 `$width`로 `height`와 `width` 래퍼의 투영값을 참조합니다:

```
enum Size {
    case small, large
}

struct SizedRectangle {
    @SmallNumber var height: Int
    @SmallNumber var width: Int

    mutating func resize(to size: Size) -> Bool {
        switch size {
        case .small:
            height = 10
            width = 20
        case .large:
            height = 100
            width = 100
        }
        return $height || $width
    }
}
```

프로퍼티 래퍼 문법은 getter와 setter가 있는 프로퍼티를 위한 편의 문법(syntactic sugar)이기 때문에 `height`와 `width` 접근하는 방식은 다른 일반적인 프로퍼티에 접근하는 것과 동일하게 동작합니다. 예를 들어 `resize(to:)` 메서드 내의 코드는 `height`와 `width`에 프로퍼티 래퍼를 통해 접근합니다. `resize(to: .large)`를 호출하면, `.large`에 해당하는 스위치 케이스에서 사각형의 높이와 너비를 100으로 설정합니다. 래퍼는 프로퍼티의 값이 12를 초과하지 않도록 막고, 값이 변경되었다는 사실을 기록하기 위해 투영값을 `true`로 설정합니다. `resize(to:)` 메서드의 마지막에 있는 반환 구문은 프로퍼티 래퍼가 `height`나 `width`를 변경했는지 판단하기 위해 `$height`와 `$width`를 체크합니다.

전역 변수와 지역 변수 (Global and Local Variables)

앞에서 설명한 연산 프로퍼티 및 프로퍼티 관찰자 기능은 *전역 변수(global variables)*와 *지역 변수(local variables)*에도 사용할 수 있습니다. 전역 변수는 함수, 메서드, 클로저, 타입 문맥의 외부에 정의된 변수입니다. 지역 변수는 함수, 메서드, 클로저 문맥 안에 정의된 변수입니다.

이전 챕터에서 본 전역과 지역 변수는 모두 *저장 변수(stored variables)*였습니다. 저장 프로퍼티처럼 저장된 변수는 특정 타입의 값을 저장할 수 있으며 그 값을 설정하고 가져오는 기능을 제공합니다.

그러나 전역이나 지역 범위에서도 *연산 변수(computed variables)*를 정의하거나 저장 변수에 관찰자를 정의할 수도 있습니다. 연산 변수는 값을 저장하는 대신 계산하여 제공하며, 연산 프로퍼티와 같은 방법으로 작성됩니다.

> **참고**
>
> 전역 상수와 변수는 항상 지연 방식으로 계산되며, 이는 지연 저장 프로퍼티 (Lazy Stored Properties)에서 설명과 유사합니다. 지연 저장 프로퍼티와 다르게 전역 상수와 변수는 `lazy` 수정자가 필요하지 않습니다.

지역 상수와 변수는 절대 지연 방식으로 계산되지 않습니다.

프로퍼티 래퍼는 지역 저장 변수에는 적용할 수 있지만, 전역 변수나 연산 변수에는 적용할 수 없습니다. 예를 들어 아래 코드에서는 `myNumber`는 프로퍼티 래퍼로 `SmallNumber`를 사용합니다.

```
func someFunction() {
    @SmallNumber var myNumber: Int = 0

    myNumber = 10
    // now myNumber is 10

    myNumber = 24
    // now myNumber is 12
}
```

프로퍼티에 `SmallNumber`를 적용할 때와 마찬가지로, `myNumber` 값을 10으로 설정하는 것은 유효합니다. 프로퍼티 래퍼는 12보다 큰 값을 허용하지 않기 때문에, `myNumber`를 24 대신 12로 설정합니다.

타입 프로퍼티 (Type Properties)

인스턴스 프로퍼티는 특정 타입의 인스턴스에 속하는 프로퍼티입니다. 타입의 새로운 인스턴스를 만들 때마다, 그 인스턴스는 다른 인스턴스와는 별도로 고유한 프로퍼티 값을 설정합니다.

해당 타입의 인스턴스가 아닌, 타입 자체에 속하는 프로퍼티를 정의할 수도 있습니다. 이러한 프로퍼티는 해당 타입의 인스턴스를 몇 개 만들든 오직 하나의 복사본만 존재합니다. 이러한 프로퍼티를 *타입 프로퍼티(type properties)*라고 합니다.

타입 프로퍼티는 모든 인스턴스에서 사용할 수 있는 상수 프로퍼티 (C에서 정적 상수)나 타입의 모든 인스턴스에 전역인 값을 저장하는 변수 프로퍼티 (C에서 static 변수)와 같은 특정 타입의 모든 인스턴스에 공통되는 값을 정의하는 데 유용합니다.

저장 타입 프로퍼티는 변수나 상수로 선언할 수 있습니다. 타입 연산 프로퍼티는 인스턴스 연산 프로퍼티와 같은 방식으로 항상 변수 프로퍼티로 선언됩니다.

> **참고**
>
> 저장 인스턴스 프로퍼티와 다르게 저장 타입 프로퍼티에는 반드시 기본값을 제공해야 합니다. 이는 타입 자체에는 저장 타입 프로퍼티를 초기화할 수 있는 이니셜라이저가 존재하지 않기 때문입니다.
>
> 저장 타입 프로퍼티는 처음 접근될 때 지연 초기화됩니다. 여러 스레드에서 동시에 접근할 때도 한 번만 초기화되는 것이 보장되고, `lazy` 수정자가 필요하지 않습니다.

타입 프로퍼티 문법 (Type Property Syntax)

C와 Objective-C에서 전역 static 변수로 타입에 연결된 static 상수와 변수를 정의합니다. 그러나 Swift에서 타입 프로퍼티를 해당 타입의 정의 내부, 즉 타입의 중괄호 내에 작성하며, 각 타입 프로퍼티는 명시적으로 해당 타입의 범위에 속하도록 선언됩니다.

타입 프로퍼티는 `static` 키워드를 사용하여 정의합니다. 클래스 타입에서 타입 연산 프로퍼티를 정의할 때는, `class` 키워드를 대신 사용하여 하위 클래스에서 해당 프로퍼티를 재정의 할 수 있습니다. 아래의 예시는 저장 타입 프로퍼티와 타입 연산 프로퍼티의 문법을 보여줍니다:

```
struct SomeStructure {
    static var storedTypeProperty = "Some value."
    static var computedTypeProperty: Int {
        return 1
    }
}
enum SomeEnumeration {
    static var storedTypeProperty = "Some value."
    static var computedTypeProperty: Int {
        return 6
    }
}
class SomeClass {
    static var storedTypeProperty = "Some value."
    static var computedTypeProperty: Int {
        return 27
    }
    class var overrideableComputedTypeProperty: Int {
        return 107
    }
}
```

참고

위 예시의 타입 연산 프로퍼티는 읽기 전용 타입 연산 프로퍼티를 위한 것이지만, 인스턴스 연산 프로퍼티와 동일한 문법을 사용하여 읽기 쓰기 타입 연산 프로퍼티를 정의할 수 있습니다.

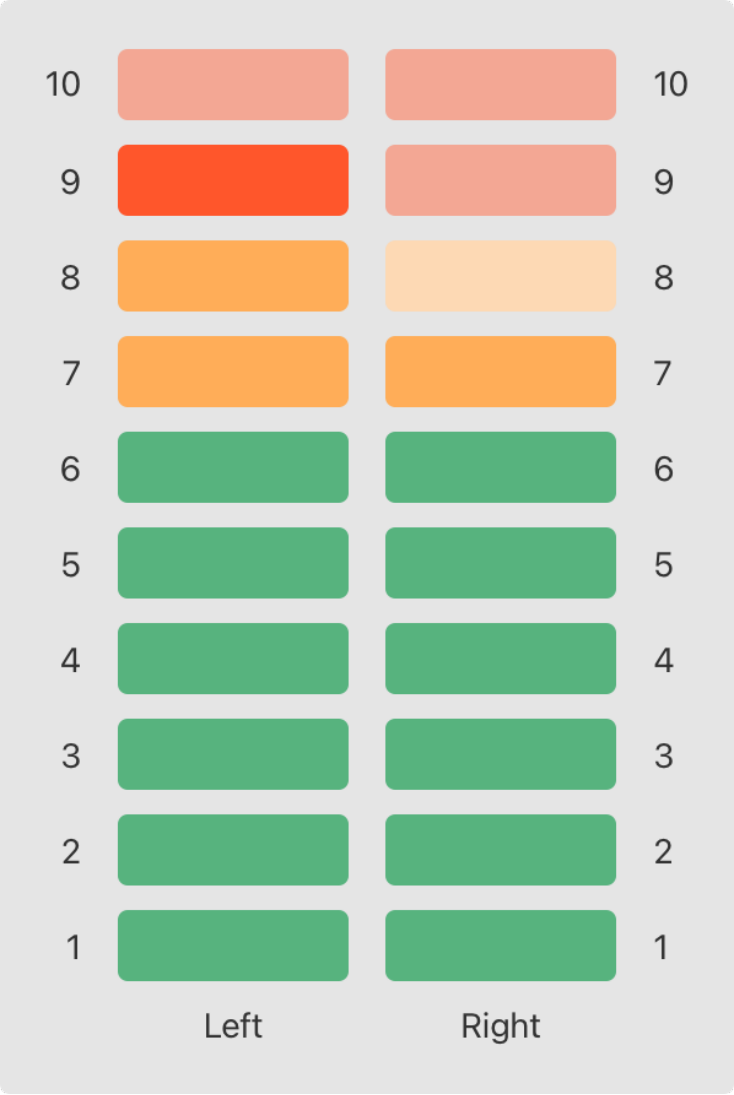
타입 프로퍼티 접근과 설정 (Querying and Setting Type Properties)

타입 프로퍼티는 인스턴스 프로퍼티처럼 점 표기법으로 접근하거나 설정합니다. 그러나 타입 프로퍼티는 해당 타입의 인스턴스가 아닌 타입 자체에 대해 접근하고 설정합니다. 예를 들어:

```
print(SomeStructure.storedTypeProperty)
// Prints "Some value.".
SomeStructure.storedTypeProperty = "Another value."
print(SomeStructure.storedTypeProperty)
// Prints "Another value.".
print(SomeEnumeration.computedTypeProperty)
// Prints "6".
print(SomeClass.computedTypeProperty)
// Prints "27".
```

다음 예시는 여러 오디오 채널에 대한 오디오 레벨 미터를 모델링하는 구조체의 일부분으로 두 개의 저장 타입 프로퍼티를 사용합니다. 각 채널은 `0`부터 `10`까지의 정수인 오디오 레벨을 가지고 있습니다.

아래 그림은 이러한 오디오 채널 중 두 개를 결합하여 스테레오 오디오 레벨 미터를 모델링하는 방법을 보여줍니다. 채널의 오디오 레벨이 `0`이면 해당 채널의 조명은 켜지지 않습니다. 오디오 레벨이 `10`이면 모든 채널의 조명은 켜집니다. 이 그림에서 왼쪽 채널은 현재 `9`의 레벨을 가지고, 오른쪽 채널은 현재 `7`의 레벨을 가집니다:



위에 설명한 오디오 채널은 `AudioChannel` 구조체의 인스턴스로 표현됩니다:

```
struct AudioChannel {
    static let thresholdLevel = 10
    static var maxInputLevelForAllChannels = 0
    var currentLevel: Int = 0 {
        didSet {
            if currentLevel > AudioChannel.thresholdLevel {
                // cap the new audio level to the threshold level
                currentLevel = AudioChannel.thresholdLevel
            }
            if currentLevel > AudioChannel.maxInputLevelForAllChannels {
                // store this as the new overall maximum input level
                AudioChannel.maxInputLevelForAllChannels = currentLevel
            }
        }
    }
}
```

`AudioChannel` 구조체는 해당 기능을 제공하기 위해 두 개의 저장 타입 프로퍼티를 정의합니다. 첫 번째 `thresholdLevel`은 오디오 레벨이 가질 수 있는 최대값을 정의합니다. 모든 `AudioChannel` 인스턴스에 대해 `10`으로 고정된 상수 값입니다. 오디오 신호가 `10`의 값보다 높게 오면, 아래에서 설명된 기준 값으로 변경합니다.

두 번째 타입 프로퍼티는 `maxInputLevelForAllChannels`라는 변수 저장 프로퍼티입니다. 이 프로퍼티는 모든 `AudioChannel` 인스턴스 중에서 최대 입력값을 추적합니다. 초기값은 `0`부터 시작합니다.

`AudioChannel` 구조체는 채널의 현재 오디오 레벨을 `0`에서 `10`으로 표현하는 `currentLevel`이라는 저장 인스턴스 프로퍼티도 정의합니다.

`currentLevel` 프로퍼티는 `currentLevel`이 설정될 때마다 값을 체크하는 `didSet` 프로퍼티 관찰자를 가지고 있습니다. 이 관찰자는 다음 두 가지를 체크합니다:

- `currentLevel`의 새로운 값이 `thresholdLevel`에 허용된 값보다 크면, 프로퍼티 관찰자는 `currentLevel`을 `thresholdLevel`로 강제 제한합니다.
- 제한된 `currentLevel`의 새로운 값이 지금까지 모든 `AudioChannel` 인스턴스가 받은 값 중 가장 크면, 프로퍼티 관찰자는 새로운 `currentLevel` 값을 `maxInputLevelForAllChannels` 타입 프로퍼티에 저장합니다.

> **참고**
>
> 위 두 가지 체크사항 중 첫 번째 항목에서 `didSet` 관찰자는 `currentLevel`을 다른 값으로 설정합니다. 그러나 이것이 관찰자를 다시 호출하진 않습니다.

스트레오 사운드 시스템의 오디오 레벨을 나타내기 위해 `leftChannel`과 `rightChannel`이라 하는 두 개의 새로운 오디오 채널을 생성하기 위해 `AudioChannel` 구조체를 사용할 수 있습니다:

```
var leftChannel = AudioChannel()
var rightChannel = AudioChannel()
```

왼쪽 채널의 `currentLevel`을 `7`로 설정하면, `maxInputLevelForAllChannels` 타입 프로퍼티가 `7`로 업데이트 된 것을 볼 수 있습니다:

```
leftChannel.currentLevel = 7
print(leftChannel.currentLevel)
// Prints "7".
print(AudioChannel.maxInputLevelForAllChannels)
// Prints "7".
```

오른쪽 채널의 `currentLevel`을 `11`로 설정하면, 오른쪽 채널의 `currentLevel` 프로퍼티가 `10`의 최대값으로 변경된 것을 볼 수 있고, `maxInputLevelForAllChannels` 타입 프로퍼티가 `10`으로 업데이트 된 것을 볼 수 있습니다:

```
rightChannel.currentLevel = 11
print(rightChannel.currentLevel)
// Prints "10".
print(AudioChannel.maxInputLevelForAllChannels)
// Prints "10".
```

언어 가이드

메서드 (Methods)

인스턴스나 타입에 속한 함수를 정의하고 호출할 수 있습니다.

*메서드(Methods)*는 특정 타입에 연결된 함수입니다. 클래스, 구조체, 열거형은 해당 타입의 인스턴스 동작을 위한 특정 작업과 기능을 캡슐화하는 인스턴스 메서드를 정의할 수 있습니다. 클래스, 구조체, 열거형은 타입 자체와 연결된 타입 메서드를 정의할 수도 있습니다. 타입 메서드는 Objective-C에서 클래스 메서드와 유사합니다.

Swift에서 구조체와 열거형도 메서드를 정의할 수 있다는 점은 C와 Objective-C와의 가장 큰 차이점입니다. Objective-C에서는 오직 클래스만 메서드를 정의할 수 있습니다. Swift에서는 클래스, 구조체, 열거형 중 어떤 것을 사용하든 메서드를 정의할 수 있는 유연함을 제공합니다.

인스턴스 메서드 (Instance Methods)

*인스턴스 메서드(Instance methods)*는 특정 클래스, 구조체, 열거형의 인스턴스에 속하는 함수입니다. 이 메서드는 인스턴스의 기능을 지원하며, 인스턴스 프로퍼티에 접근하고 수정하는 방법을 제공하거나, 인스턴스의 목적에 관련된 기능을 구현하는데 사용됩니다. 인스턴스 메서드는 함수 (Functions)에서 설명한대로 함수 구문과 완벽하게 동일합니다.

인스턴스 메서드는 해당 타입의 중괄호 내에 작성합니다. 인스턴스 메서드는 해당 타입에 속한 다른 인스턴스 메서드와 프로퍼티에 암시적으로 접근할 수 있습니다. 인스턴스 메서드는 반드시 해당 타입의 특정 인스턴스에 대해 호출할 수 있습니다. 인스턴스 없이 단독으로 호출할 수는 없습니다.

다음은 작업이 발생한 횟수를 계산하는데 사용할 수 있는 간단한 `Counter` 클래스 정의의 예시입니다:

```
class Counter {
    var count = 0
    func increment() {
        count += 1
    }
    func increment(by amount: Int) {
        count += amount
    }
    func reset() {
        count = 0
    }
}
```

`Counter` 클래스는 다음과 같은 세 가지 인스턴스 메서드를 정의합니다:

- `increment()`는 1씩 카운터를 증가시킵니다.
- `increment(by: Int)`는 지정된 정수 크기만큼 카운터를 증가시킵니다.
- `reset()`은 카운터를 0으로 재설정합니다.

`Counter` 클래스는 현재 카운터 값을 추적하는 `count` 프로퍼티 변수도 선언합니다.

프로퍼티와 동일하게 점 표기법으로 인스턴스 메서드를 호출합니다:

```
let counter = Counter()
// the initial counter value is 0
counter.increment()
// the counter's value is now 1
counter.increment(by: 5)
// the counter's value is now 6
counter.reset()
// the counter's value is now 0
```

함수 매개변수는 함수 인자 레이블과 매개변수 이름 (Function Argument Labels and Parameter Names)에서 설명했듯이 함수 본문 내에서 사용하는 이름과 함수 호출할 때 사용하는 인자 레이블을 모두 가질 수 있습니다. 메서드도 타입에 속한 함수이므로, 메서드 매개변수도 동일한 규칙을 따릅니다.

self 프로퍼티 (The self Property)

모든 타입의 인스턴스는 `self` 라는 암시적 프로퍼티를 가지고 있으며, 이는 해당 인스턴스 자체를 나타냅니다. 인스턴스 메서드 내에서 현재 인스턴스를 참조하기 위해 `self` 프로퍼티를 사용합니다.

위 예시에서 `increment()` 메서드는 아래와 같이 작성될 수 있습니다:

```
func increment() {
    self.count += 1
}
```

실제로 코드에서 `self`를 자주 작성할 필요는 없습니다. `self`를 명시적으로 작성하지 않으면, Swift는 메서드 내부에서 알려진 프로퍼티나 메서드 이름을 사용할 때, 현재 인스턴스의 프로퍼티나 메서드를 참조하는 것으로 간주합니다. 이러한 기본 동작은 `Counter`의 세 인스턴스 메서드에서 `self.count`가 아닌 `count`를 사용하는 방식에서 확인할 수 있습니다.

이 규칙의 주요 예외사항은 인스턴스 메서드의 매개변수 이름이 그 인스턴스의 프로퍼티 이름과 동일한 경우입니다. 이러한 경우 매개변수 이름이 더 우선되므로, 프로퍼티를 참조하려면 더 명확한 방식으로 접근해야 합니다. 매개변수 이름과 프로퍼티 이름이 충돌할 경우 `self` 프로퍼티를 사용하여 구분합니다.

다음 예시에서 `x`라는 메서드 매개변수와 같은 이름을 가진 인스턴스 프로퍼티를 구분하기 위해 `self`를 사용합니다:

```
struct Point {
    var x = 0.0, y = 0.0
    func isToTheRightOf(x: Double) -> Bool {
        return self.x > x
    }
}
let somePoint = Point(x: 4.0, y: 5.0)
if somePoint.isToTheRightOf(x: 1.0) {
    print("This point is to the right of the line where x == 1.0")
}
// Prints "This point is to the right of the line where x == 1.0".
```

`self` 접두사가 없으면, Swift는 두 번 모두 `x`가 메서드의 매개변수 `x`를 가리키는 것으로 간주합니다.

인스턴스 메서드 내부에서 값 타입 수정 (Modifying Value Types from Within Instance Methods)

구조체와 열거형은 값 타입입니다. 기본적으로 값 타입의 프로퍼티는 인스턴스 메서드 내부에서 수정할 수 없습니다.

그러나 특정 메서드 내에서 구조체나 열거형의 프로퍼티 수정이 필요하다면 해당 메서드에 대한 변경 동작을 명시적으로 허용할 수 있습니다. 그러면 해당 메서드 내부에서 프로퍼티를 변경할 수 있고 메서드가 끝나면 기존 구조체에 변경 사항이 반영됩니다. 이 메서드 내에서 `self`에 완전히 새로운 인스턴스를 할당할 수도 있으며, 메서드가 종료되면 기존 인스턴스가 이 새 인스턴스로 대체하게 됩니다.

이 동작을 허용하려면 해당 메서드 `func` 키워드 앞에 `mutating` 키워드를 작성하면 됩니다:

```
struct Point {
    var x = 0.0, y = 0.0
    mutating func moveBy(x deltaX: Double, y deltaY: Double) {
        x += deltaX
        y += deltaY
    }
}
var somePoint = Point(x: 1.0, y: 1.0)
somePoint.moveBy(x: 2.0, y: 3.0)
print("The point is now at (\(somePoint.x), \(somePoint.y))")
// Prints "The point is now at (3.0, 4.0)".
```

위의 `Point` 구조체는 mutating 메서드인 `moveBy(x:y:)`를 정의하며, 이 메서드는 `Point` 인스턴스를 일정한 값만큼 이동시킵니다. 새로운 위치를 반환하는 대신에 이 메서드는 실제로 호출된 인스턴스를 직접 수정합니다. 이를 위해 메서드 정의에 `mutating` 키워드를 추가하여 프로퍼티를 수정을 허용합니다.

상수 구조체 인스턴스의 저장 프로퍼티 (Stored Properties of Constant Structure Instances)에서 설명했듯이 프로퍼티가 변수 프로퍼티라도 변경할 수 없기 때문에 구조체 타입의 상수에 대해 mutating 메서드를 호출할 수 없습니다:

```
let fixedPoint = Point(x: 3.0, y: 3.0)
fixedPoint.moveBy(x: 2.0, y: 3.0)
// this will report an error
```

Mutating 메서드 내에서 self 할당 (Assigning to self Within a Mutating Method)

mutating 메서드에서는 암시적인 `self` 프로퍼티에 완전히 새로운 인스턴스를 할당할 수 있습니다. 위에서 본 `Point` 예시는 다음과 같은 방식으로 작성할 수도 있습니다:

```
struct Point {
    var x = 0.0, y = 0.0
    mutating func moveBy(x deltaX: Double, y deltaY: Double) {
        self = Point(x: x + deltaX, y: y + deltaY)
    }
}
```

이 mutating 메서드 `moveBy(x:y:)`는 x와 y 값이 대상 위치로 설정된 새로운 구조체를 생성하고, 이를 `self`에 할당합니다. 이 대체 버전의 메서드를 호출한 최종 결과는 이전 버전의 호출 결과와 동일합니다.

열거형의 mutating 메서드는 동일한 열거형의 다른 케이스를 암시적인 `self`에 설정할 수 있습니다:

```
enum TriStateSwitch {
    case off, low, high
    mutating func next() {
        switch self {
        case .off:
            self = .low
        case .low:
            self = .high
        case .high:
            self = .off
        }
    }
}
var ovenLight = TriStateSwitch.low
ovenLight.next()
// ovenLight is now equal to .high
ovenLight.next()
// ovenLight is now equal to .off
```

이 예시는 세 가지 상태를 가지는 스위치를 나타내는 열거형을 정의합니다. 이 스위치는 `next()` 메서드가 호출될 때마다 전원 상태 (`off`, `low`, `high`)를 순환합니다.

타입 메서드 (Type Methods)

위에 설명한 인스턴스 메서드는 특정 타입의 인스턴스에서 호출되는 메서드입니다. 타입 자체에서 호출되는 메서드도 정의할 수 있습니다. 이런 종류의 메서드를 *타입 메서드(type methods)*라고 합니다. 메서드의 `func` 키워드 앞에 `static` 키워드를 작성하여 타입 메서드를 나타냅니다. 클래스는 대신 `class` 키워드를 사용하여 하위 클래스가 해당 메서드를 재정의 할 수 있습니다.

> **참고**
>
> Objective-C에서는 클래스에서만 타입 수준의 메서드(type-level methods)를 정의할 수 있습니다. Swift에서는 모든 클래스, 구조체, 열거형에 대해서 타입 메서드를 정의할 수 있습니다. 각 타입 메서드는 해당 타입에 명시적으로 속하게 됩니다.

타입 메서드는 인스턴스 메서드처럼 점 표기법을 사용하여 호출합니다. 그러나 해당 타입의 인스턴스가 아닌 타입자체에 대해 호출합니다. `SomeClass`라는 클래스에 타입 메서드를 호출하는 방법은 다음과 같습니다:

```
class SomeClass {
    class func someTypeMethod() {
        // type method implementation goes here
    }
}
SomeClass.someTypeMethod()
```

타입 메서드의 본문 내에서 암시적인 `self` 프로퍼티는 타입의 인스턴스가 아닌 해당 타입 자체를 가리킵니다. 인스턴스 프로퍼티와 인스턴스 메서드 매개변수에서와 같이 `self`를 타입 프로퍼티와 타입 메서드 매개변수를 명확하게 사용할 수 있다는 의미입니다.

일반적으로 타입 메서드 내부에서 타입 이름 없이 사용하는 메서드나 프로퍼티 이름은 모두 타입 수준의 메서드나 프로퍼티를 참조합니다. 타입 메서드 안에서는 다른 타입 메서드를 타입 이름 없이 호출할 수 있습니다. 유사하게 구조체와 열거형에서 정의된 타입 메서드는 타입 프로퍼티에도 마찬가지로 타입 이름 없이 접근할 수 있습니다.

아래 예시는 게임 내에서 플레이어의 레벨이나 스테이지 진행사항을 추적하는 `LevelTracker`라는 구조체를 정의합니다. 이것은 1인용 게임이지만, 단일 디바이스에서 여러 명의 플레이어를 위한 정보를 저장할 수 있습니다.

게임을 처음 시작할 때는 1레벨을 제외한 모든 레벨이 잠겨있습니다. 플레이어가 레벨을 끝낼 때마다 디바이스에 모든 플레이어가 플레이 할 수 있도록 레벨이 잠금 해제됩니다. `LevelTracker` 구조체는 게임의 레벨이 잠금 해제되는 것을 추적하기 위해 타입 프로퍼티와 메서드를 사용합니다. 각 플레이어의 현재 레벨은 인스턴스 프로퍼티를 통해 추적합니다.

```
struct LevelTracker {
    static var highestUnlockedLevel = 1
    var currentLevel = 1

    static func unlock(_ level: Int) {
        if level > highestUnlockedLevel { highestUnlockedLevel = level }
    }

    static func isUnlocked(_ level: Int) -> Bool {
        return level <= highestUnlockedLevel
    }

    @discardableResult
    mutating func advance(to level: Int) -> Bool {
        if LevelTracker.isUnlocked(level) {
            currentLevel = level
            return true
        } else {
            return false
        }
    }
}
```

`LevelTracker` 구조체는 모든 플레이어가 잠금 해제한 가장 높은 레벨을 추적합니다. 이 값은 `highestUnlockedLevel`이라는 타입 프로퍼티에 저장됩니다.

`LevelTracker`는 `highestUnlockedLevel` 프로퍼티와 함께 동작하는 두 개의 타입 함수도 정의합니다. 첫 번째 타입 함수는 `unlock(_:)`이라 하며, 새로운 레벨이 잠금 해제될 때마다 `highestUnlockedLevel`의 값을 갱신합니다. 두 번째 타입 함수는 `isUnlocked(_:)`이라는 편의 타입 함수이며, 특정 레벨이 이미 잠금 해제되었다면 `true`를 반환합니다. (이 타입 메서드는 `LevelTracker.highestUnlockedLevel`로 작성할 필요없이 `highestUnlockedLevel` 타입 프로퍼티를 접근할 수 있습니다.)

타입 프로퍼티와 타입 메서드 외에도 `LevelTracker`는 게임을 통해 각 플레이어의 진행사항을 추적합니다. 플레이어가 현재 플레이 중인 레벨을 추적하는 `currentLevel`이라는 인스턴스 프로퍼티를 사용합니다.

`currentLevel` 프로퍼티를 관리하기 위해 `LevelTracker`는 `advance(to:)`라는 인스턴스 메서드를 정의합니다. `currentLevel`을 갱신 전에 이 메서드는 요청된 새 레벨이 이미 잠금 해제되었는지 판단합니다. `advance(to:)` 메서드는 `currentLevel`을 설정가능한지 아닌지를 나타내기 위해 Boolean 값을 반환합니다. `advance(to:)` 메서드를 호출하여 반환값을 무시하는 경우도 있기 때문에, 이 함수는 `@discardableResult` 속성으로 표시됩니다. 더 자세한 내용은 속성 (Attributes)을 참고바랍니다.

`LevelTracker` 구조체는 아래에 나오는 각 플레이어의 진행 상태를 추적하고 업데이트하는 `Player` 클래스와 함께 사용됩니다:

```
class Player {
    var tracker = LevelTracker()
    let playerName: String
    func complete(level: Int) {
        LevelTracker.unlock(level + 1)
        tracker.advance(to: level + 1)
    }
    init(name: String) {
        playerName = name
    }
}
```

`Player` 클래스는 플레이어의 진행 상태를 추적하기 위해 새로운 `LevelTracker`의 인스턴스를 생성합니다. 플레이어가 특정 레벨을 완료했는지 판단하는 `complete(level:)`이라는 메서드도 제공합니다. 이 메서드는 모든 플레이어에게 다음 레벨을 잠금 해제하고 다음 레벨로 이동하기 위해 플레이어의 진행 상태를 업데이트 합니다. (이전 줄에서 `LevelTracker.unlock(_:)`을 호출하여 해당 레벨을 잠금 해제하기 때문에 `advance(to:)`의 Boolean 반환값은 무시됩니다.)

새로운 플레이어를 위한 `Player` 클래스의 인스턴스를 생성할 수 있고 플레이어가 레벨1을 완료하면 어떤 일이 생기는지 알 수 있습니다:

```
var player = Player(name: "Argyrios")
player.complete(level: 1)
print("highest unlocked level is now \(LevelTracker.highestUnlockedLevel)")
// Prints "highest unlocked level is now 2".
```

게임에서 아직 잠금 해제되지 않은 레벨로 이동을 하려는 두 번째 플레이어를 생성하면 플레이어의 현재 레벨을 실패로 설정합니다:

```
player = Player(name: "Beto")
if player.tracker.advance(to: 6) {
    print("player is now on level 6")
} else {
    print("level 6 has not yet been unlocked")
}
// Prints "level 6 has not yet been unlocked".
```

언어 가이드

서브스크립트 (Subscripts)

컬렉션의 요소에 접근합니다.

클래스, 구조체, 열거형은 *서브스크립트(subscript)*를 정의할 수 있고, 이것은 컬렉션, 리스트, 시퀀스의 구성 요소에 접근할 수 있는 간결한 문법을 제공합니다. 서브스크립트를 사용하면 값을 설정하고 가져오는 별도의 메서드를 만들 필요 없이 인덱스를 통해 간단하게 값을 설정하거나 검색할 수 있습니다. 예를 들어 `Array` 인스턴스의 요소는 `someArray[index]`로 `Dictionary` 인스턴스의 요소는 `someDictionary[key]`로 접근합니다.

하나의 타입에 여러 개의 서브스크립트를 정의할 수 있고, 넘겨주는 인덱스의 타입에 따라 적절한 서브스크립트 오버로드가 선택됩니다. 서브스크립트는 1차원에만 제한되지 않고, 사용자 타입에 맞춰 여러 개의 입력 매개변수로 서브스크립트를 정의할 수 있습니다.

서브스크립트 문법 (Subscript Syntax)

서브스크립트를 사용하면, 인스턴스 이름 뒤에 대괄호 안에 하나 이상의 값을 작성하여 타입의 인스턴스를 조회할 수 있습니다. 서브스크립트 문법은 인스턴스 메서드 문법과 연산 프로퍼티 문법과 유사합니다. 서브스크립트는 `subscript` 키워드로 정의하며, 인스턴스 메서드처럼 하나 이상의 입력 매개변수와 반환 타입을 작성합니다. 인스턴스 메서드와 다르게 서브스크립트는 읽기-쓰기 또는 읽기 전용 형태가 될 수 있습니다. 이러한 동작은 연산 프로퍼티와 같은 방법으로 getter와 setter를 통해 정의합니다:

```
subscript(index: Int) -> Int {
    get {
        // Return an appropriate subscript value here.
    }
    set(newValue) {
        // Perform a suitable setting action here.
    }
}
```

서브스크립트에서 `newValue`의 타입은 해당 서브스크립트의 반환 타입과 동일합니다. 연산 프로퍼티와 마찬가지로, setter의 `(newValue)` 매개변수를 명시하지 않아도 됩니다. 매개변수를 지정하지 않으면, setter에 `newValue`라는 기본 매개변수가 제공됩니다.

읽기 전용 연산 프로퍼티와 마찬가지로 `get` 키워드와 중괄호를 생략하여 읽기 전용 서브스크립트를 간단히 선언할 수 있습니다:

```
subscript(index: Int) -> Int {
    // Return an appropriate subscript value here.
}
```

다음은 정수의 n단 곱셈표를 나타내는 `TimesTable` 구조체를 정의하는 읽기 전용 서브스크립트 구현의 예입니다:

```
struct TimesTable {
    let multiplier: Int
    subscript(index: Int) -> Int {
        return multiplier * index
    }
}
let threeTimesTable = TimesTable(multiplier: 3)
print("six times three is \(threeTimesTable[6])")
// Prints "six times three is 18".
```

이 예시에서 `TimesTable`의 새로운 인스턴스를 생성하여 3단 곱셈표를 나타냅니다. 이는 구조체의 `initializer`에 `multiplier` 값으로 3을 전달함으로써 설정됩니다.

`threeTimesTable[6]`에 대한 호출에서 보여준 것처럼 서브스크립트를 호출하여 `threeTimesTable` 인스턴스를 조회할 수 있습니다. 이것은 3의 6배인 18의 값을 반환하는 3단 곱셈표에서 6번째 값을 요청합니다.

> **참고**
>
> n단 곱셈표는 수학적 규칙을 기반으로 합니다. `threeTimesTable[someIndex]`에 새로운 값을 설정하는 것은 적절하지 않으므로, `TimesTable`의 서브스크립트는 읽기 전용 서브스크립트로 정의됩니다.

서브스크립트 활용 (Subscript Usage)

"서브스크립트"의 정확한 의미는 사용되는 문맥에 따라 다릅니다. 일반적으로 서브스크립트는 컬렉션, 리스트, 시퀀스의 구성 요소에 접근하기 위한 간결한 문법으로 사용됩니다. 특정 클래스나 구조체의 가장 적합한 방식으로 서브스크립트를 구현할 수 있습니다.

예를 들어 Swift의 `Dictionary` 타입은 서브스크립트를 통해 `Dictionary` 인스턴스에 저장된 값을 설정하고 조회할 수 있도록 구현되어 있습니다. 서브스크립트 대괄호 내에 딕셔너리의 키 타입의 키를 제공하고 딕셔너리의 값 타입의 값을 서브스크립트에 할당하여 딕셔너리에 값을 설정할 수 있습니다:

```
var numberOfLegs = ["spider": 8, "ant": 6, "cat": 4]
numberOfLegs["bird"] = 2
```

위의 예시는 `numberOfLegs`라는 변수를 정의하고 세 개의 키-값 쌍을 포함하는 딕셔너리를 초기화합니다. `numberOfLegs` 딕셔너리의 타입은 `[String: Int]`로 유추됩니다. 딕셔너리가 생성된 후에, 이 예시는 서브스크립트 할당을 통해 `"bird"`라는 `String` 키와 2라는 `Int` 값을 딕셔너리에 추가합니다.

`Dictionary` 서브스크립트에 대한 자세한 내용은 딕셔너리 접근과 수정 (Accessing and Modifying a Dictionary)을 참고바랍니다.

> **참고**
>
> Swift의 `Dictionary` 타입은 키-값 서브스크립트를 옵셔널 타입을 사용하는 서브스크립트로 구현합니다. 위의 `numberOfLegs` 딕셔너리에서 키-값 서브스크립트는 타입 `Int?` 또는 "옵셔널 int"의 값을 주고받습니다. `Dictionary` 타입은 모든 키에 값이 있는 것이 아니며, 키에 대해 `nil`을 할당하여 키에 대한 값을 삭제하는 방법을 제공하기 위해 옵셔널 서브스크립트 타입을 사용합니다.

서브스크립트 옵션 (Subscript Options)

서브스크립트는 여러 개의 입력 매개변수를 가질 수 있고, 입력 매개변수는 어떤 타입도 가능합니다. 서브스크립트는 어떤 타입도 반환할 수 있습니다.

함수 처럼, 서브스크립트는 가변 매개변수 (Variadic Parameters)와 매개변수 기본값 (Default Parameter Values)에서 설명했듯이 가변 매개변수를 받거나 기본값을 가질 수 있습니다. 그러나 함수와 다르게, 서브스크립트는 in-out 매개변수를 사용할 수 없습니다.

클래스나 구조체는 필요한 만큼 여러 개의 서브스크립트 구현을 제공할 수 있으며, 서브스크립트가 사용되는 시점에 대괄호 안에 전달된 값의 타입을 기반으로 어떤 서브스크립트가 호출될지가 자동으로 추론됩니다. 이러한 여러 서브스크립트를 정의하는 것을 *서브스크립트 오버로딩(subscript overloading)*이라 합니다.

대부분 서브스크립트는 하나의 매개변수를 가지지만, 타입에 따라 필요한 경우 여러 개의 매개변수를 가진 서브스크립트를 정의할 수도 있습니다. 다음의 예시는 `Double` 값의 2차원 행렬을 나타내는 `Matrix` 구조체를 정의합니다. `Matrix` 구조체의 서브스크립트는 두 개의 정수 매개변수를 가집니다:

```
struct Matrix {
    let rows: Int, columns: Int
    var grid: [Double]
    init(rows: Int, columns: Int) {
        self.rows = rows
        self.columns = columns
        grid = Array(repeating: 0.0, count: rows * columns)
    }
    func indexIsValid(row: Int, column: Int) -> Bool {
        return row >= 0 && row < rows && column >= 0 && column < columns
    }
    subscript(row: Int, column: Int) -> Double {
        get {
            assert(indexIsValid(row: row, column: column), "Index out of range")
            return grid[(row * columns) + column]
        }
        set {
            assert(indexIsValid(row: row, column: column), "Index out of range")
            grid[(row * columns) + column] = newValue
        }
    }
}
```

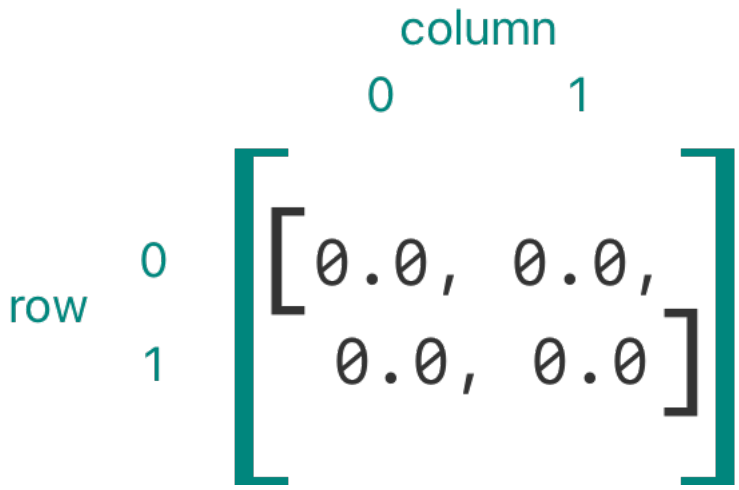
`Matrix`는 `rows`와 `columns`라는 두 개의 매개변수를 가지고 `Double` 타입의 `rows * columns` 값을 저장할 수 있는 큰 배열을 생성하는 초기화를 제공합니다. 행렬의 각 위치는 `0.0`의 초기값이 주어집니다. 이를 위해 배열의 크기와 초기 셀 값 `0.0`을 이니셜라이저에 전달하여, 올바른 크기의 새로운 배열을 생성하고 초기화합니다. 이러한 이니셜라이저는 기본값 배열 생성 (Creating an Array with a Default Value)에 자세히 설명되어 있습니다.

적절한 행과 열의 수를 이니셜라이저에 전달하여 새로운 `Matrix` 인스턴스를 생성할 수 있습니다:

```
var matrix = Matrix(rows: 2, columns: 2)
```

위의 예시는 두 개의 행과 두 개의 열을 가지는 새로운 `Matrix` 인스턴스를 생성합니다. `Matrix` 인스턴스의 `grid` 배열은 행렬을 좌측 상단에서 우측 하단으로 펼친 1차원 배열입니다:

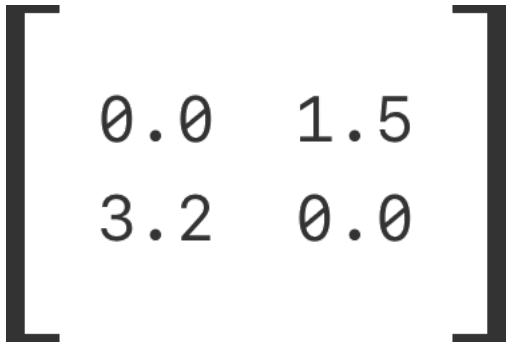




행렬의 값은 콤마로 구분된 서브스크립트에 행과 열 값을 전달하여 설정될 수 있습니다:

```
matrix[0, 1] = 1.5
matrix[1, 0] = 3.2
```

이 두 구문은 서브스크립트의 setter를 호출하여, 행렬의 우측 상단 (`row`가 `0`이고, `column`이 `1`)에 `1.5` 값을 설정하고, 좌측 하단 (`row`가 `1`이고, `column`이 `0`)에 `3.2` 값을 설정합니다:



`Matrix` 서브스크립트의 getter와 setter는 모두 서브스크립트의 `row`와 `column` 값이 유효한지 판단하기 위해 어설션(assertion)을 포함합니다. 이러한 검사를 지원하기 위해, `Matrix`는 요청한 `row`와 `column`이 행렬의 범위 안에 있는지를 판단하기 위해 `indexIsValid(row:column:)`이라는 편의 메서드를 제공합니다:

```
func indexIsValid(row: Int, column: Int) -> Bool {
    return row >= 0 && row < rows && column >= 0 && column < columns
}
```

어셜션은 행렬 범위를 넘어서 서브스크립트를 접근하려고 하면 오류가 발생합니다:

```
let someValue = matrix[2, 2]
// This triggers an assert, because [2, 2] is outside of the matrix bounds.
```

타입 서브스크립트 (Type Subscripts)

위에서 설명했듯이 인스턴스 서브스크립트는 특정 타입의 인스턴스에 대해 호출하는 서브스크립트입니다. 타입 자체에 대해 호출할 수 있는 서브스크립트도 정의할 수 있습니다. 이런 종류의 서브스크립트를 *타입 서브스크립트(type subscript)*라고 합니다. `subscript` 키워드 앞에 `static` 키워드를 작성하여 타입 서브스크립트를 정의합니다. 클래스는 하위 클래스에서 서브스크립트의 구현을 재정의 할 수 있게 `class` 키워드를 대신 사용할 수도 있습니다. 아래 예시는 타입 서브스크립트를 어떻게 정의하고 호출하는지 보여줍니다:

```
enum Planet: Int {
    case mercury = 1, venus, earth, mars, jupiter, saturn, uranus, neptune
    static subscript(n: Int) -> Planet {
        return Planet(rawValue: n)!
    }
}
let mars = Planet[4]
print(mars)
```

언어 가이드

상속 (Inheritance)

하위 클래스를 통해 기능을 추가하거나 재정의합니다.

클래스는 다른 클래스의 메서드, 프로퍼티, 다른 특성을 *상속(inherit)*할 수 있습니다. 클래스가 다른 클래스를 상속받으면, 상속받는 클래스를 *하위 클래스(subclass)*라고 하고, 상속해주는 클래스를 *상위 클래스(superclass)*라고 합니다. 상속은 Swift에서 클래스가 다른 타입과 구분되는 기본적인 특징입니다.

Swift의 클래스는 상위 클래스의 메서드, 프로퍼티, 서브스크립트를 호출하고 접근할 수 있으며, 메서드, 프로퍼티, 서브스크립트 동작을 수정하기 위해 재정의한 버전을 제공할 수 있습니다. Swift는 재정의에 일치하는 상위 클래스 정의가 있는지 확인하고 재정의가 올바른지 확인하는데 도움을 줍니다.

클래스는 프로퍼티의 값이 변경될 때 알려주기 위해 상속받은 프로퍼티에 프로퍼티 관찰자를 추가할 수도 있습니다. 프로퍼티 관찰자는 해당 프로퍼티가 저장 프로퍼티나 연산 프로퍼티로 정의되었는지 상관없이 프로퍼티에 추가할 수 있습니다.

기본 클래스 정의 (Defining a Base Class)

다른 클래스로부터 상속받지 않은 클래스를 *기본 클래스(base class)*라고 합니다.

> **참고**
>
> Swift의 클래스는 범용 기본 클래스로부터 자동으로 상속되지 않습니다. 상위 클래스를 명시하지 않고 정의한 클래스는 자동으로 빌드할 때 기본 클래스가 됩니다.

아래 예시는 `Vehicle`이라는 기본 클래스를 정의합니다. 이 기본 클래스는 `currentSpeed`라는 저장 프로퍼티를 가지며, 기본값은 `0.0`입니다 (프로퍼티 타입을 `Double`로 추론합니다). `currentSpeed` 프로퍼티는 읽기 전용 연산 프로퍼티인 `description`에서 사용되며, 이 프로퍼티는 차량의 문자열 설명을 반환합니다.

`Vehicle` 클래스는 `makeNoise`라는 메서드도 정의합니다. 이 메서드는 기본 `Vehicle` 인스턴스에서 실질적으로 아무런 동작을 하지 않지만, 후에 `Vehicle`의 하위 클래스에서 커스텀될 예정입니다:

```
class Vehicle {
    var currentSpeed = 0.0
    var description: String {
        return "traveling at \(currentSpeed) miles per hour"
    }
    func makeNoise() {
        // do nothing - an arbitrary vehicle doesn't necessarily make a noise
    }
}
```

타입 이름 뒤에 빈 소괄호를 작성한 이니셜라이저으로 `Vehicle`의 새로운 인스턴스를 생성합니다:

```
let someVehicle = Vehicle()
```

새로운 `Vehicle` 인스턴스를 생성하면 차량의 현재 속도의 설명을 출력하기 위해 `description` 프로퍼티에 접근할 수 있습니다:

```
print("Vehicle: \(someVehicle.description)")
// Vehicle: traveling at 0.0 miles per hour
```

`Vehicle` 클래스는 임의의 차량에 대한 공통 특성을 정의하지만, 그 자체로는 많이 사용되지 않습니다. 더 유용하게 사용하기 위해, 더 구체적인 종류의 차량을 설명하는 하위 클래스를 정의해야 합니다.

서브클래싱 (Subclassing)

*서브클래싱(Subclassing)*은 기존 클래스를 기반으로 새로운 클래스를 만드는 작업입니다. 하위 클래스는 기존 클래스의 특성을 상속받으며, 이를 기반으로 동작을 세분화할 수 있습니다. 하위 클래스에 새로운 특성도 추가할 수 있습니다.

하위 클래스가 어떤 상위 클래스를 기반으로 하고 있는지를 나타내기 위해, 하위 클래스 이름 다음에 콜론을 쓰고 그 뒤에 상위 클래스 이름을 작성합니다:

```
class SomeSubclass: SomeSuperclass {
    // subclass definition goes here
}
```

다음의 예시는 `Vehicle`을 상위 클래스로 하는 `Bicycle`이라는 하위 클래스를 정의합니다:

```
class Bicycle: Vehicle {
    var hasBasket = false
}
```

새로운 `Bicycle` 클래스는 `currentSpeed`와 `description` 프로퍼티와 `makeNoise()` 메서드와 같은 `Vehicle`의 모든 특성을 자동으로 갖습니다.

상속한 특성 외에도 `Bicycle` 클래스는 `hasBasket`이라는 새로운 저장 프로퍼티를 정의하며, 기본값은 `false`입니다 (프로퍼티를 `Bool` 타입으로 유추됩니다).

기본적으로 생성한 새로운 `Bicycle` 인스턴스는 바구니를 가지고 있지 않습니다. 인스턴스가 생성된 후에 특정 `Bicycle` 인스턴스의 `hasBasket` 프로퍼티를 `true`로 설정할 수 있습니다:

```
let bicycle = Bicycle()
bicycle.hasBasket = true
```

`Bicycle` 인스턴스에서 상속받은 `currentSpeed` 프로퍼티를 수정할 수 있고, 상속받은 `description` 프로퍼티를 조회할 수도 있습니다:

```
bicycle.currentSpeed = 15.0
print("Bicycle: \(bicycle.description)")
// Bicycle: traveling at 15.0 miles per hour
```

하위 클래스도 서브클래싱할 수 있습니다. 다음 예시는 “tandem”이라는 2인승 자전거를 위한 `Bicycle`의 하위 클래스를 정의합니다:

```
class Tandem: Bicycle {
    var currentNumberOfPassengers = 0
}
```

`Tandem`은 `Bicycle`의 모든 프로퍼티와 메서드를 상속받고, `Bicycle`은 다시 `Vehicle`의 모든 프로퍼티와 메서드를 상속받습니다. `Tandem` 하위 클래스는 기본값 `0`인 `currentNumberOfPassengers`라는 새로운 저장 프로퍼티도 추가합니다.

`Tandem`의 인스턴스를 생성하면, 새로운 프로퍼티와 상속된 프로퍼티를 사용할 수 있고, `Vehicle`로부터 상속된 읽기 전용 `description` 프로퍼티도 조회할 수 있습니다:

```
let tandem = Tandem()
tandem.hasBasket = true
tandem.currentNumberOfPassengers = 2
tandem.currentSpeed = 22.0
print("Tandem: \(tandem.description)")
// Tandem: traveling at 22.0 miles per hour
```

재정의 (Overriding)

하위 클래스는 상위 클래스로부터 상속받은 인스턴스 메서드, 타입 메서드, 인스턴스 프로퍼티, 타입 프로퍼티, 서브스크립트를 자신만의 사용자 구현을 제공할 수 있습니다. 이것을 *재정의(overriding)*라고 합니다.

상속된 특성을 재정의하려면, 재정의할 선언 앞에 `override` 키워드를 추가합니다. 이렇게 하면 재정의를 명확하게 제공하고 실수로 일치하는 정의를 제공하지 않도록 합니다. 실수로 재정의하면 예기치 않은 동작을 야기하고, `override` 키워드 없이 재정의하면 코드가 컴파일될 때 오류가 발생합니다.

`override` 키워드는 컴파일러에게 현재 정의하려는 멤버가 상위 클래스(또는 그 상위 클래스)에서 실제로 존재하는지 확인하도록 합니다. 이 검사를 통해 재정의하는 정의가 옳다고 보장합니다.

상위 클래스 메서드, 프로퍼티, 서브스크립트 접근 (Accessing Superclass Methods, Properties, and Subscripts)

하위 클래스에서 메서드, 프로퍼티, 서브스크립트 재정의를 제공할 때, 재정의의 부분으로 상위 클래스 구현을 사용하는 것이 유용할 때가 있습니다. 예를 들어 기존 구현의 동작을 세분화하거나, 기존 상속된 변수에 수정된 값을 저장할 수 있습니다.

이럴 때는 `super` 접두사를 사용하여 상위 클래스의 메서드, 프로퍼티, 서브스크립트에 접근합니다:

- `someMethod()`라는 재정의된 메서드는 재정의한 메서드 구현 내에서 `super.someMethod()`를 호출하여 상위 클래스 버전의 `someMethod()` 를 호출할 수 있습니다.
- `someProperty`라는 재정의된 프로퍼티는 재정의한 getter나 setter 구현 내에서 `super.someProperty`로 상위 클래스의 `someProperty`를 접근할 수 있습니다.
- `someIndex`를 위한 재정의된 서브스크립트는 재정의한 서브스크립트 구현 내에서 `super[someIndex]`로 상위 클래스의 같은 서브스크립트에 접근할 수 있습니다.

메서드 재정의 (Overriding Methods)

상속받은 인스턴스 메서드나 타입 메서드를 재정의하여, 하위 클래스 내에서 메서드의 맞춤형 또는 대체 구현을 제공합니다.

아래 예시는 `Train`이라는 `Vehicle`의 새로운 하위 클래스를 정의하고, `Vehicle`을 상속한 `Train`에 `makeNoise()` 메서드를 재정의합니다:

```
class Train: Vehicle {
    override func makeNoise() {
        print("Choo Choo")
    }
}
```

`Train`의 새로운 인스턴스를 생성하고 `makeNoise()` 메서드를 호출한다면, `Train` 하위 클래스의 메서드가 호출되는 것을 볼 수 있습니다:

```
let train = Train()
train.makeNoise()
// Prints "Choo Choo".
```

프로퍼티 재정의 (Overriding Properties)

상속받은 인스턴스나 타입 프로퍼티를 재정의하여 하위 클래스에서 상속된 프로퍼티 값이 변경될 때 반응할 수 있는 해당 프로퍼티에 커스텀 getter와 setter를 제공하거나, 프로퍼티 관찰자를 추가할 수 있습니다.

프로퍼티 getter와 setter 재정의 (Overriding Property Getters and Setters)

상속된 프로퍼티가 저장 프로퍼티인지 연산 프로퍼티인지는 상관없이 모든 상속된 프로퍼티를 재정의 하기위해 커스텀 getter와 setter를 제공할 수 있습니다. 상속된 프로퍼티가 저장 프로퍼티인지 연산 프로퍼티인지 하위 클래스에서는 알 수 없습니다 - 상속된 프로퍼티의 특정 이름과 타입만 알고 있습니다. 컴파일러가 동일한 이름과 타입을 가진 상위 클래스 프로퍼티와 일치하는지 확인할 수 있도록 항상 재정의하는 프로퍼티의 이름과 타입을 모두 명시해야 합니다.

읽기 전용 프로퍼티를 재정의하여 읽기-쓰기 프로퍼티로 바꿀 수 있습니다. 그러나 상속된 읽기-쓰기 프로퍼티를 읽기 전용 프로퍼티로 변경하는 것은 허용되지 않습니다.

> **참고**
>
> 프로퍼티 재정의의 부분으로 setter를 제공하면 getter도 제공해야 합니다. 재정의한 getter 내에서 상속된 프로퍼티의 값을 수정하고 싶지 않다면, `someProperty`가 재정의하는 프로퍼티의 이름이라면, getter에서 `super.someProperty`를 반환하면 상속된 값을 간단하게 전달할 수 있습니다.

다음의 예시는 `Vehicle`의 하위 클래스인 `Car`라는 새로운 클래스를 정의합니다. `Car` 클래스는 기본 정수값이 `1`인 `gear`라는 새로운 저장 프로퍼티를 가집니다. `Car` 클래스는 현재 기어를 포함한 설명을 제공하기 위해 `Vehicle`로부터 상속받은 `description` 프로퍼티를 재정의합니다:

```
class Car: Vehicle {
    var gear = 1
    override var description: String {
        return super.description + " in gear \(gear)"
    }
}
```

description 프로퍼티의 재정의는 Vehicle 클래스의 description 프로퍼티를 반환하는 super.description 호출로 시작합니다. 그런 다음 Car 클래스의 description은 현재 기어에 대한 정보를 추가합니다.

Car 클래스의 인스턴스를 생성하고 gear와 currentSpeed 프로퍼티를 설정하면 description 프로퍼티는 Car 클래스에 정의한 설명을 반환하는 것을 볼 수 있습니다:

```
let car = Car()
car.currentSpeed = 25.0
car.gear = 3
print("Car: \(car.description)")
// Car: traveling at 25.0 miles per hour in gear 3
```

프로퍼티 관찰자 재정의 (Overriding Property Observers)

상속된 프로퍼티에 프로퍼티 관찰자를 추가하기 위해 프로퍼티 재정의를 사용할 수 있습니다. 이것은 기존에 구현된 프로퍼티가 어떻든 상관없이 상속된 프로퍼티의 값이 변경될 때 알림을 받을 수 있습니다. 자세한 내용은 프로퍼티 관찰자 (Property Observers)를 참고바랍니다.

> **참고**
>
> 상속된 저장 프로퍼티 상수나 상속된 읽기 전용 연산 프로퍼티에 프로퍼티 관찰자를 추가할 수 없습니다. 이런 프로퍼티에 값을 설정할 수 없으므로 재정의의 부분으로 willSet이나 didSet 구현을 제공하기에 적절하지 않습니다.
>
> 같은 프로퍼티에 재정의한 setter와 재정의한 프로퍼티 관찰자를 동시에 제공할 수 없습니다. 프로퍼티의 값이 변경되는 것을 관찰하기 원하고 이미 프로퍼티에 커스텀 setter를 제공하고 있다면 커스텀 setter 내에서 간단하게 값 변경을 관찰할 수 있습니다.

다음 예시는 Car의 하위 클래스 인 AutomaticCar라는 새로운 클래스를 정의합니다. AutomaticCar 클래스는 현재 속도에 기반하여 적절한 기어를 자동으로 선택하는 자동 기어 박스가 있는 자동차를 나타냅니다:

```
class AutomaticCar: Car {
    override var currentSpeed: Double {
        didSet {
            gear = Int(currentSpeed / 10.0) + 1
        }
    }
}
```

AutomaticCar 인스턴스에 currentSpeed 프로퍼티가 설정될 때마다, 프로퍼티의 didSet 관찰자는 새로운 속도에 적절한 기어를 인스턴스의 gear 프로퍼티에 설정합니다. 특히 프로퍼티 관찰자는 새로운 currentSpeed 값을 10으로 나눈 뒤, 가까운 정수로 내림한 다음 1을 더한 값을 기어로 선택합니다. 35.0의 속도는 기어를 4로 선택합니다:

```
let automatic = AutomaticCar()
automatic.currentSpeed = 35.0
print("AutomaticCar: \(automatic.description)")
// AutomaticCar: traveling at 35.0 miles per hour in gear 4
```

재정의 방지 (Preventing Overrides)

final 표시를 통해 실수로 메서드, 프로퍼티, 서브스크립트를 재정의 하는 것을 방지할 수 있습니다. 재정의를 방지하려면 메서드, 프로퍼티, 서브스크립트의 키워드 앞에 `final` 수정자를 작성합니다 (`final var`, `final func`, `final class func`, `final subscript` 와 같이 작성).

하위 클래스에서 final 메서드, final 프로퍼티, final 서브스크립트를 재정의 하면 컴파일 시 오류가 발생합니다. 확장한 클래스에 추가된 메서드, 프로퍼티, 서브스크립트는 확장의 정의 내에서 final로 표시할 수도 있습니다. 자세한 내용은 확장 (Extensions)을 참고바랍니다.

클래스 정의에 `class` 키워드 전에 `final` 키워드를 표시하여 final로 전체 클래스를 표시할 수 있습니다(`final class`). final 클래스를 하위 클래스하려는 모든 시도는 컴파일 시 오류가 발생합니다.

언어 가이드

초기화 (Initialization)

타입의 저장 프로퍼티에 초기값을 설정하고 초기 설정을 수행합니다.

*초기화(Initialization)*는 클래스, 구조체, 열거형의 인스턴스를 사용하기 위해 준비하는 과정입니다. 이 과정에서는 인스턴스의 각 저장 프로퍼티에 초기값을 설정하고 새로운 인스턴스가 사용되기 전에 다른 설정이나 초기화를 수행하는 것을 포함합니다.

특정 타입의 새로운 인스턴스를 생성하기 위해 특정 메서드를 호출하는 것처럼 *이니셜라이저(initializers)*을 정의하여 초기화를 구현합니다. Objective-C 이니셜라이저과 달리 Swift 이니셜라이저는 값을 반환하지 않습니다. 초기화의 주요 역할은 처음 사용되기 전에 타입의 새로운 인스턴스가 올바르게 초기화되는 것을 보장하는 것입니다.

클래스 타입의 인스턴스는 클래스의 인스턴스가 할당 해제되기 전에 정리를 수행하는 *디이니셜라이저(deinitializer)*도 구현할 수 있습니다. 자세한 내용은 소멸 (Deinitialization)를 참고바랍니다.

저장 프로퍼티에 초기값 설정 (Setting Initial Values for Stored Properties)

클래스와 구조체는 해당 클래스나 구조체의 인스턴스가 생성되기 전까지 모든 저장 프로퍼티에 적절한 초기값을 반드시 설정해야 합니다. 저장 프로퍼티는 불확정한 상태로 남아있을 수 없습니다.

이니셜라이저 내에서 저장 프로퍼티의 초기값을 설정하거나, 프로퍼티의 정의 시 기본값을 할당할 수 있습니다. 이러한 동작에 대해 다음 섹션에 설명하도록 하겠습니다.

> **참고**
>
> 저장 프로퍼티에 기본값을 할당하거나 이니셜라이저 내에 값을 설정할 때, 해당 프로퍼티의 값은 프로퍼티 관찰자 호출 없이 직접 설정됩니다.

이니셜라이저 (Initializers)

*이니셜라이저(Initializers)*는 특정 타입의 새로운 인스턴스를 생성하기 위해 호출됩니다. 가장 간단한 형식의 이니셜라이저는 `init` 키워드를 사용하여 작성하며 매개변수가 없는 인스턴스 메서드와 같습니다:

```
init() {
    // perform some initialization here
}
```

아래의 예시는 화씨 온도를 저장하는 `Fahrenheit`라는 새로운 구조체를 정의합니다. `Fahrenheit` 구조체는 `Double` 타입의 `temperature`라는 하나의 저장 프로퍼티를 가지고 있습니다:

```
struct Fahrenheit {
    var temperature: Double
    init() {
        temperature = 32.0
    }
}
var f = Fahrenheit()
print("The default temperature is \(f.temperature)° Fahrenheit")
// Prints "The default temperature is 32.0° Fahrenheit".
```

이 구조체는 매개변수가 없는 단일 이니셜라이저 `init`을 정의하며, `32.0`(물이 얼어버리는 화씨 온도)의 값이 저장된 온도로 초기화합니다.

기본 프로퍼티 값 (Default Property Values)

위에서 본 것처럼 이니셜라이저 내에서 저장 프로퍼티의 초기값을 설정할 수 있습니다. 또는 프로퍼티 선언 시 *기본 프로퍼티 값(default property value)*을 지정할 수 있습니다. 프로퍼티를 정의할 때 프로퍼티의 초기값을 할당하는 것으로 기본 프로퍼티 값을 지정합니다.

> **참고**
>
> 프로퍼티가 항상 같은 초기값을 갖는다면, 이니셜라이저 내에서 값을 설정하기보다 기본값을 제공하는 것이 좋습니다. 결과는 같지만, 기본값은 프로퍼티의 초기화를 선언에 더 가깝게 연결합니다. 이것은 이니셜라이저를 더 짧고, 명확하게 하고 기본값으로부터 프로퍼티의 타입을 유추할 수 있습니다. 기본값은 이 챕터 후반부에 설명하듯이 기본 이니셜라이저와 이니셜라이저 상속을 더 간단한 형태로 작성할 수 있습니다.

프로퍼티가 선언될 때 `temperature`에 기본값을 제공하여 더 간단한 형태로 위의 `Fahrenheit` 구조체를 작성할 수 있습니다:

```
struct Fahrenheit {
    var temperature = 32.0
}
```

커스텀 이니셜라이저 (Customizing Initialization)

초기화 과정은 입력 매개변수와 선택적 프로퍼티 타입, 또는 상수 프로퍼티를 초기화 중에 할당하는 방식으로 커스텀할 수 있습니다.

초기화 매개변수 (Initialization Parameters)

이니셜라이저 정의 시 *초기화 매개변수(initialization parameters)*를 제공하여 초기화 과정을 커스텀할 수 있습니다. 초기화 매개변수는 함수 및 메서드 매개변수와 동일한 기능과 구문을 가지고 있습니다.

다음 예시는 섭씨 온도를 저장하는 `Celsius`라는 구조체를 정의합니다. `Celsius` 구조체는 `init(fromFahrenheit:)`와 `init(fromKelvin:)`이라는 두 개의 커스텀 이니셜라이저를 구현하며, 각각 다른 온도 단위에서 값을 받아 구조체의 새로운 인스턴스를 초기화합니다:

```
struct Celsius {
    var temperatureInCelsius: Double
    init(fromFahrenheit fahrenheit: Double) {
        temperatureInCelsius = (fahrenheit - 32.0) / 1.8
    }
    init(fromKelvin kelvin: Double) {
        temperatureInCelsius = kelvin - 273.15
    }
}
let boilingPointOfWater = Celsius(fromFahrenheit: 212.0)
// boilingPointOfWater.temperatureInCelsius is 100.0
let freezingPointOfWater = Celsius(fromKelvin: 273.15)
// freezingPointOfWater.temperatureInCelsius is 0.0
```

첫 번째 이니셜라이저는 `fromFahrenheit`라는 인자 레이블과 `fahrenheit`라는 매개변수 이름을 가진 하나의 초기화 매개변수를 가지고 있습니다. 두 번째 이니셜라이저는 `fromKelvin`라는 인자 레이블과 `kelvin`이라는 매개변수 이름을 가진 하나의 초기화 매개변수를 가지고 있습니다. 두 이니셜라이저 모두 전달받은 하나의 값을 섭씨 온도로 변환하고, `temperatureInCelsius`라는 프로퍼티에 값을 저장합니다.

매개변수 이름과 인자 레이블 (Parameter Names and Argument Labels)

함수와 메서드의 매개변수와 마찬가지로, 초기화 매개변수는 이니셜라이저의 본문 내에서 사용하는 매개변수 이름과 이니셜라이저를 호출할 때 사용하는 인자 레이블 모두 가질 수 있습니다.

그러나 이니셜라이저는 함수와 메서드처럼 소괄호 앞에 식별용 함수 이름을 가지지 않습니다. 따라서 이니셜라이저의 매개변수 이름과 타입이 어떤 이니셜라이저를 호출해야 하는지 식별하는데 특히 중요한 역할을 합니다. 이러한 이유 때문에 Swift는 인자 레이블을 제공하지 않으면 이니셜라이저의 모든 매개변수에 대해 자동으로 인자 레이블을 제공합니다.

다음 예시는 `red`, `green`, `blue`라는 세 개의 상수 프로퍼티를 가지는 `Color`라는 구조체를 정의합니다. 이 프로퍼티는 색상에서의 빨강, 초록, 파랑의 비율을 나타내기 위해 `0.0`에서 `1.0`사이의 값을 저장합니다.

`Color`는 빨강, 초록, 파랑 성분에 대해 `Double` 타입의 세 개의 적절한 이름의 매개변수를 가진 이니셜라이저를 제공합니다. `Color`는 단일 `white` 매개변수를 가지는 두 번째 이니셜라이저도 제공하는데, 세 가지 생상 성분 모두에 동일한 값을 지정할 때 사용합니다.

```
struct Color {
    let red, green, blue: Double
    init(red: Double, green: Double, blue: Double) {
        self.red   = red
        self.green = green
        self.blue  = blue
    }
    init(white: Double) {
        red   = white
        green = white
        blue  = white
    }
}
```

두 이니셜라이저 모두 각 이니셜라이저 매개변수에 이름이 지정된 값을 제공하여 새로운 `Color` 인스턴스를 생성할 수 있습니다:

```
let magenta = Color(red: 1.0, green: 0.0, blue: 1.0)
let halfGray = Color(white: 0.5)
```

이러한 이니셜라이저를 호출할 때는 반드시 인자 레이블을 사용해야 합니다. 인자 레이블이 정의되어 있는 경우, 이를 생략하면 컴파일 시 오류가 발생합니다:

```
let veryGreen = Color(0.0, 1.0, 0.0)
// this reports a compile-time error - argument labels are required
```

인자 레이블이 없는 이니셜라이저 매개변수 (Initializer Parameters Without Argument Labels)

이니셜라이저 매개변수에 대해 인자 레이블을 사용하고 싶지 않은 경우, 매개변수 인자 레이블 위치에 언더바(_)를 작성하여 기본 동작을 무시할 수 있습니다.

다음은 초기화 매개변수 (Initialization Parameters)에서의 `Celsius` 예시를 확장한 것으로 이미 섭씨 온도 값인 `Double` 값을 이용해 새로운 `Celsius` 인스턴스를 생성할 수 있는 추가 이니셜라이저를 포함하고 있습니다:

```
struct Celsius {
    var temperatureInCelsius: Double
    init(fromFahrenheit fahrenheit: Double) {
        temperatureInCelsius = (fahrenheit - 32.0) / 1.8
    }
    init(fromKelvin kelvin: Double) {
        temperatureInCelsius = kelvin - 273.15
    }
    init(_ celsius: Double) {
        temperatureInCelsius = celsius
    }
}
let bodyTemperature = Celsius(37.0)
// bodyTemperature.temperatureInCelsius is 37.0
```

이니셜라이저 호출 `Celsius(37.0)`은 인자 레이블 없이도 그 의도가 명확합니다. 따라서 이니셜라이저를 `init(_ celsius: Double)`로 작성하여 이름 없는 `Double` 값을 제공하여 호출할 수 있습니다.

옵셔널 프로퍼티 타입 (Optional Property Types)

커스텀 타입에 저장 프로퍼티가 있는데, 해당 프로퍼티가 논리적으로 “값이 없을 수 있음”이 허용되는 경우 - 초기화 중에 값을 설정할 수 없거나, 나중에 “값 없음” 상태가 될 수 있는 경우 - 그 프로퍼티를 옵셔널 타입으로 선언할 수 있습니다. 옵셔널 타입(optional type)의 프로퍼티는 자동으로 초기화 동안 “아직 값 없음”을 가진다는 의도를 위해 `nil`의 값으로 초기화 됩니다.

다음의 예시는 `response`라는 옵셔널 `String` 프로퍼티를 가지는 `SurveyQuestion`이라는 클래스를 정의합니다:

```
class SurveyQuestion {
    var text: String
    var response: String?
    init(text: String) {
        self.text = text
    }
    func ask() {
        print(text)
    }
```

```
}
let cheeseQuestion = SurveyQuestion(text: "Do you like cheese?")
cheeseQuestion.ask()
// Prints "Do you like cheese?".
cheeseQuestion.response = "Yes, I do like cheese."
```

설문 질문에 대한 응답은 실제로 질문되기 전까지 알 수 없으므로, `response` 프로퍼티는 `String?` 타입, 즉 "옵셔널 `String`" 타입으로 선언됩니다. 새로운 인스턴스 `SurveyQuestion`이 초기화 될 때, "아직 문자열 없음"이라는 의미의 기본값 `nil`이 자동으로 할당됩니다.

초기화 중 상수 프로퍼티 값 할당 (Assigning Constant Properties During Initialization)

상수 프로퍼티는 초기화가 완료되기 전까지만 값을 확정적으로 할당하면 되므로, 초기화 중 언제든지 값을 할당할 수 있습니다. 상수 프로퍼티에 값이 할당되면 더 이상 수정할 수 없습니다.

> **참고**
>
> 클래스 인스턴스의 경우, 상수 프로퍼티는 해당 프로퍼티를 정의한 클래스에서만 초기화 중에 수정할 수 있습니다. 하위 클래스에서는 해당 상수 프로퍼티를 수정할 수 없습니다.

위의 `SurveyQuestion` 예시를 수정하여 `SurveyQuestion`의 인스턴스가 생성되면 질문이 바뀌지 않는다는 것을 나타내기 위해 질문의 `text` 프로퍼티를 변수 프로퍼티 대신 상수 프로퍼티로 정의할 수 있습니다. 이제 `text` 프로퍼티는 상수이지만, 여전히 클래스의 이니셜라이저 내에서는 값을 설정할 수 있습니다:

```
class SurveyQuestion {
    let text: String
    var response: String?
    init(text: String) {
        self.text = text
    }
    func ask() {
        print(text)
    }
}
let beetsQuestion = SurveyQuestion(text: "How about beets?")
beetsQuestion.ask()
// Prints "How about beets?".
beetsQuestion.response = "I also like beets. (But not with cheese.)"
```

기본 이니셜라이저 (Default Initializers)

Swift는 모든 프로퍼티에 기본값을 제공하고 별도의 이니셜라이저를 하나도 제공하지 않는 구조체나 클래스에 대해 *기본 이니셜라이저(default initializer)*를 제공합니다. 기본 이니셜라이저는 모든 프로퍼티가 기본값으로 설정된 새로운 인스턴스를 생성합니다.

이 예시는 쇼핑 목록에 있는 항목의 이름, 수량, 구매 상태를 캡슐화하는 `ShoppingListItem`이라는 클래스를 정의합니다:

```
class ShoppingListItem {
    var name: String?
    var quantity = 1
    var purchased = false
}
var item = ShoppingListItem()
```

`ShoppingListItem` 클래스의 모든 프로퍼티는 기본값을 가지고 있고, 상속받은 상위 클래스가 없는 기본 클래스이므로, `ShoppingListItem`은 자동으로 모든 프로퍼티를 기본값으로 설정한 새로운 인스턴스를 생성하는 기본 이니셜라이저 구현을 제공합니다. (`name` 프로퍼티는 코드에서 값을 작성하지 않았지만 옵셔널 `String` 프로퍼티이므로, 자동으로 `nil`의 기본값이 할당됩니다.) 위의 예시는 `ShoppingListItem` 클래스의 기본 이니셜라이저를 사용하여 `ShoppingListItem()`으로 클래스의 새로운 인스턴스를 생성하고, 이를 `item`이라는 변수에 할당합니다.

구조체 타입의 멤버와이즈 이니셜라이저 (Memberwise Initializers for Structure Types)

구조체 타입은 커스텀 이니셜라이저를 정의하지 않으면 자동으로 *멤버와이즈 이니셜라이저(memberwise initializer)*를 받습니다. 기본 이니셜라이저와 다르게 구조체는 저장 프로퍼티에 기본값이 없어도 멤버와이즈 이니셜라이저를 받습니다.

멤버와이즈 이니셜라이저는 새로운 구조체 인스턴스의 멤버 프로퍼티를 초기화하는 간편한 방법입니다. 새로운 인스턴스의 프로퍼티에 대한 초기값은 이름으로 멤버와이즈 이니셜라이저에 전달할 수 있습니다.

아래의 예시는 `width`와 `height`라는 두 개의 프로퍼티를 가지는 `Size`라는 구조체를 정의합니다. 두 프로퍼티 모두 `0.0`의 기본값이 할당되어 `Double` 타입으로 추론됩니다.

`Size` 구조체는 자동으로 새로운 `Size` 인스턴스를 초기화할 수 있는 `init(width:height:)` 멤버와이즈 이니셜라이저를 받습니다:

```
struct Size {
    var width = 0.0, height = 0.0
}
let twoByTwo = Size(width: 2.0, height: 2.0)
```

멤버와이즈 이니셜라이저를 호출할 때, 기본값을 가지는 모든 프로퍼티는 생략할 수 있습니다. 위의 예시에서 `Size` 구조체는 `height`와 `width` 프로퍼티 둘 다 기본값을 가지고 있습니다. 하나 또는 프로퍼티 둘 다 생략할 수 있고, 이니셜라이저는 생략된 항목에 대해 기본값을 사용합니다. 예를 들어:

```
let zeroByTwo = Size(height: 2.0)
print(zeroByTwo.width, zeroByTwo.height)
// Prints "0.0 2.0".

let zeroByZero = Size()
print(zeroByZero.width, zeroByZero.height)
// Prints "0.0 0.0".
```

값 타입을 위한 이니셜라이저 위임 (Initializer Delegation for Value Types)

이니셜라이저는 인스턴스의 초기화의 부분을 수행하기 위해 다른 이니셜라이저를 호출할 수 있습니다. 이 과정을 *이니셜라이저 위임(initializer delegation)*이라고 하며, 여러 이니셜라이저에 중복되는 코드를 피할 수 있도록 해줍니다.

이니셜라이저 위임이 동작하는 방식과 허용하는 위임 형태에 대한 규칙은 값 타입과 클래스 타입에 따라 다릅니다. 값 타입(구조체와 열거형)은 상속을 지원하지 않기 때문에, 이니셜라이저 위임 과정이 비교적 단순하며, 오직 자신이 제공하는 다른 이니셜라이저에만 위임할 수 있습니다. 그러나 클래스는 상속 (Inheritance)에서 설명했듯이 다른 클래스로부터 상속받을 수 있습니다. 이는 상속받은 모든 저장 프로퍼티에 적절한 값이 할당되도록 추가적인 책임이 있음을 의미합니다. 이러한 책임은 아래 클래스 상속과 초기화 (Class Inheritance and Initialization)에 설명되어 있습니다.

값 타입에서 커스텀 이니셜라이저를 작성할 때 같은 값 타입 내의 다른 이니셜라이저를 참조하기 위해 `self.init`을 사용합니다. 이니셜라이저 내에서만 `self.init`을 호출할 수 있습니다.

값 타입에 대해 커스텀 이니셜라이저를 정의하면 해당 타입에 대해 기본 이니셜라이저 (또는 구조체의 경우 멤버와이즈 이니셜라이저)에 더 이상 접근할 수 없습니다. 이 제약은 복잡한 이니셜라이저에서 제공하는 필수 설정 절차를 자동 이니셜라이저를 사용해 우회해버리는 상황을 방지하기 위한 것입니다.

> **참고**
>
> 기본 이니셜라이저와 멤버와이즈 이니셜라이저를 그대로 유지하면서도 커스텀 이니셜라이저를 추가하고 싶다면, 커스텀 이니셜라이저를 값 타입의 원래 구현이 아닌 확장에서 작성해야 합니다. 자세한 내용은 확장 (Extensions)을 참고바랍니다.

다음의 예시는 기하학적 사각형을 나타내는 `Rect` 구조체를 정의합니다. 이 예시는 `Size`와 `Point`라는 두 개의 보조 구조체가 필요하고, 둘 다 모든 프로퍼티에 대해 `0.0`의 기본값을 제공합니다:

```
struct Size {
    var width = 0.0, height = 0.0
}
struct Point {
    var x = 0.0, y = 0.0
}
```

아래의 `Rect` 구조체는 세 가지 방법 중 하나로 초기화할 수 있습니다 - 기본 0으로 초기화된 `origin`과 `size`를 사용하거나, 특정한 원점과 크기를 제공하거나, 특정한 중앙점과 크기를 제공하는 방법입니다. 이러한 초기화 옵션은 `Rect` 구조체의 정의에 포함된 세 개의 커스텀 이니셜라이저로 구현되어 있습니다:

```
struct Rect {
    var origin = Point()
    var size = Size()
    init() {}
    init(origin: Point, size: Size) {
        self.origin = origin
        self.size = size
    }
    init(center: Point, size: Size) {
        let originX = center.x - (size.width / 2)
        let originY = center.y - (size.height / 2)
        self.init(origin: Point(x: originX, y: originY), size: size)
    }
}
```

첫 번째 `Rect` 이니셜라이저인 `init()`은 구조체가 커스텀 이니셜라이저를 가지지 않았을 경우 자동으로 제공받게 되는 기본 이니셜라이저와 기능적으로 동일합니다. 이 이니셜라이저는 빈 중괄호 `{}`로 표시된 빈 본문을 가집니다. 이 이니셜라이저를 호출하면 `origin`과 `size` 프로퍼티 각각 `Point(x: 0.0, y: 0.0)`와 `Size(width: 0.0, height: 0.0)`이라는 프로퍼티 정의의 기본값으로 초기화 되는 `Rect` 인스턴스를 반환합니다:

```
let basicRect = Rect()
// basicRect's origin is (0.0, 0.0) and its size is (0.0, 0.0)
```

두 번째 `Rect` 이니셜라이저인 `init(origin:size:)`는 구조체가 커스텀 이니셜라이저가 없으면 자동으로 제공받는 멤버와이즈 이니셜라이저와 기능적으로 동일합니다. 이 이니셜라이저는 전달된 `origin`과 `size` 인자값을 해당 저장 프로퍼티에 단순히 할당합니다:

```
let originRect = Rect(origin: Point(x: 2.0, y: 2.0),
                      size: Size(width: 5.0, height: 5.0))
// originRect's origin is (2.0, 2.0) and its size is (5.0, 5.0)
```

세 번째 `Rect` 이니셜라이저인 `init(center:size:)`는 약간 더 복잡합니다. 이 이니셜라이저는 `center`와 `size` 값을 기반으로 적절한 원점을 계산하는 것으로 시작합니다. 그런 다음 `init(origin:size:)` 이니셜라이저를 호출(또는 위임)하여 새로 계산된 원점과 크기로 해당 프로퍼티에 저장합니다:

```
let centerRect = Rect(center: Point(x: 4.0, y: 4.0),
                      size: Size(width: 3.0, height: 3.0))
// centerRect's origin is (2.5, 2.5) and its size is (3.0, 3.0)
```

`init(center:size:)` 이니셜라이저는 `origin`과 `size`의 새로운 값을 해당 프로퍼티에 직접 할당할 수 있습니다. 그러나 `init(center:size:)` 이니셜라이저가 이미 해당 기능을 정확히 제공하는 기존 이니셜라이저를 활용하는 것이 더 편리하고 의도도 더 명확하게 드러낼 수 있습니다.

참고

`init()`과 `init(origin:size:)` 이니셜라이저를 직접 정의하지 않고 이 예시를 작성하는 다른 방법은 확장(Extensions)을 참고바랍니다.

클래스 상속과 초기화 (Class Inheritance and Initialization)

클래스의 저장 프로퍼티는 - 상위 클래스로부터 상속받은 프로퍼티를 포함하여 - 모두 초기화 과정에서 초기값이 반드시 할당되어야 합니다.

Swift는 클래스의 모든 저장 프로퍼티가 초기값을 가질 수 있도록 두 가지 종류의 이니셜라이저를 정의합니다. 이 이니셜라이저들은 지정 이니셜라이저(designated initializer)와 편의 이니셜라이저(convenience initializer)로 알려져 있습니다.

지정 이니셜라이저와 편의 이니셜라이저 (Designated Initializers and Convenience Initializers)

*지정 이니셜라이저(Designated initializers)*는 클래스의 기본 이니셜라이저입니다. 지정 이니셜라이저는 해당 클래스에서 정의한 모든 프로퍼티를 완벽하게 초기화하고, 초기화 과정을 상위 클래스 체인으로 이어가기 위해 적절한 상위 클래스의 이니셜라이저를 호출합니다.

클래스는 일반적으로 지정 이니셜라이저를 매우 적게 가지며, 클래스에 하나만 있는 경우도 흔합니다. 지정 이니셜라이저는 초기화가 이루어지는 “중심 경로(funnel)” 역할을 하며, 이 경로를 통해 초기화가 상위 클래스 체인으로 이어집니다.

모든 클래스는 최소 하나의 지정 이니셜라이저를 가지고 있어야 합니다. 경우에 따라 이 요구사항은 아래의 자동 이니셜라이저 상속 (Automatic Initializer Inheritance)에서 설명했듯이 상위 클래스의 지정 이니셜라이저를 하나 이상 상속받는 것으로 충족됩니다.

*편의 이니셜라이저(Convenience initializers)*는 클래스의 보조 이니셜라이저입니다. 편의 이니셜라이저는 동일한 클래스 내의 지정 이니셜라이저를 호출하되, 일부 매개변수를 기본값으로 설정하여 호출할 수 있습니다. 특정 사용 사례나 입력 타입에 대한 해당 클래스의 인스턴스를 생성하기 위해 편의 이니셜라이저를 정의할 수도 있습니다.

클래스에 편의 이니셜라이저가 반드시 필요한 것은 아니며, 필요하지 않다면 제공하지 않아도 됩니다. 일반적인 초기화 패턴을 간편하게 처리하거나, 클래스 초기화의 의도를 더 명확히 하고자 할 때 편의 이니셜라이저를 만드는 것이 좋습니다.

지정 이니셜라이저와 편의 이니셜라이저의 문법 (Syntax for Designated and Convenience Initializers)

클래스의 지정 이니셜라이저는 값 타입의 간단한 이니셜라이저와 동일한 방법으로 작성됩니다:

```
init(<#parameters#>) {
   <#statements#>
}
```

편의 이니셜라이저는 동일한 형식으로 작성되지만, `init` 키워드 앞에 `convenience` 한정자를 공백으로 구분하여 작성합니다:

```
convenience init(<#parameters#>) {
   <#statements#>
}
```

클래스 타입의 이니셜라이저 위임 (Initializer Delegation for Class Types)

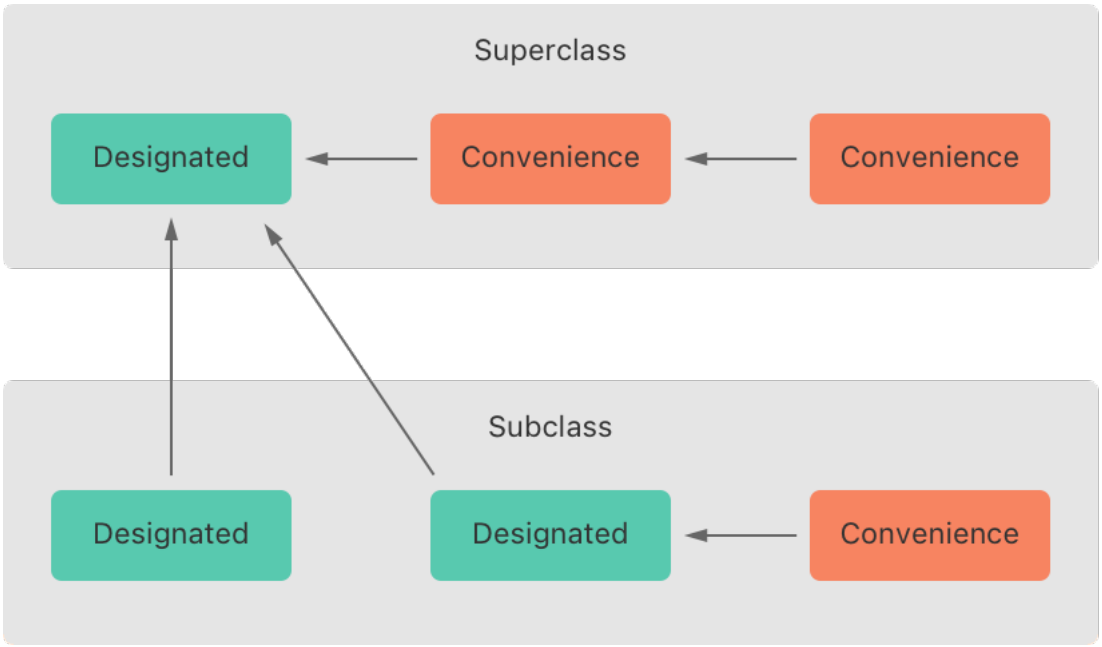
지정 이니셜라이저와 편의 이니셜라이저 사이의 관계를 단순화하기 위해, Swift는 이니셜라이저 사이의 위임 호출에 대해 다음 세 가지 규칙을 적용합니다:

- **규칙 1**: 지정 이니셜라이저는 자신과 가장 가까운 상위 클래스의 지정 이니셜라이저를 호출해야 합니다.
- **규칙 2**: 편의 이니셜라이저는 같은 클래스의 다른 이니셜라이저를 호출해야 합니다.
- **규칙 3**: 편의 이니셜라이저는 결국 지정 이니셜라이저를 호출해야 합니다.

이 규칙을 기억하는 간단한 방법은 아래와 같습니다:

- 지정 이니셜라이저는 항상 위로 위임해야 합니다.
- 편의 이니셜라이저는 항상 옆으로 위임해야 합니다.

이러한 규칙은 아래의 그림에 설명되어 있습니다:

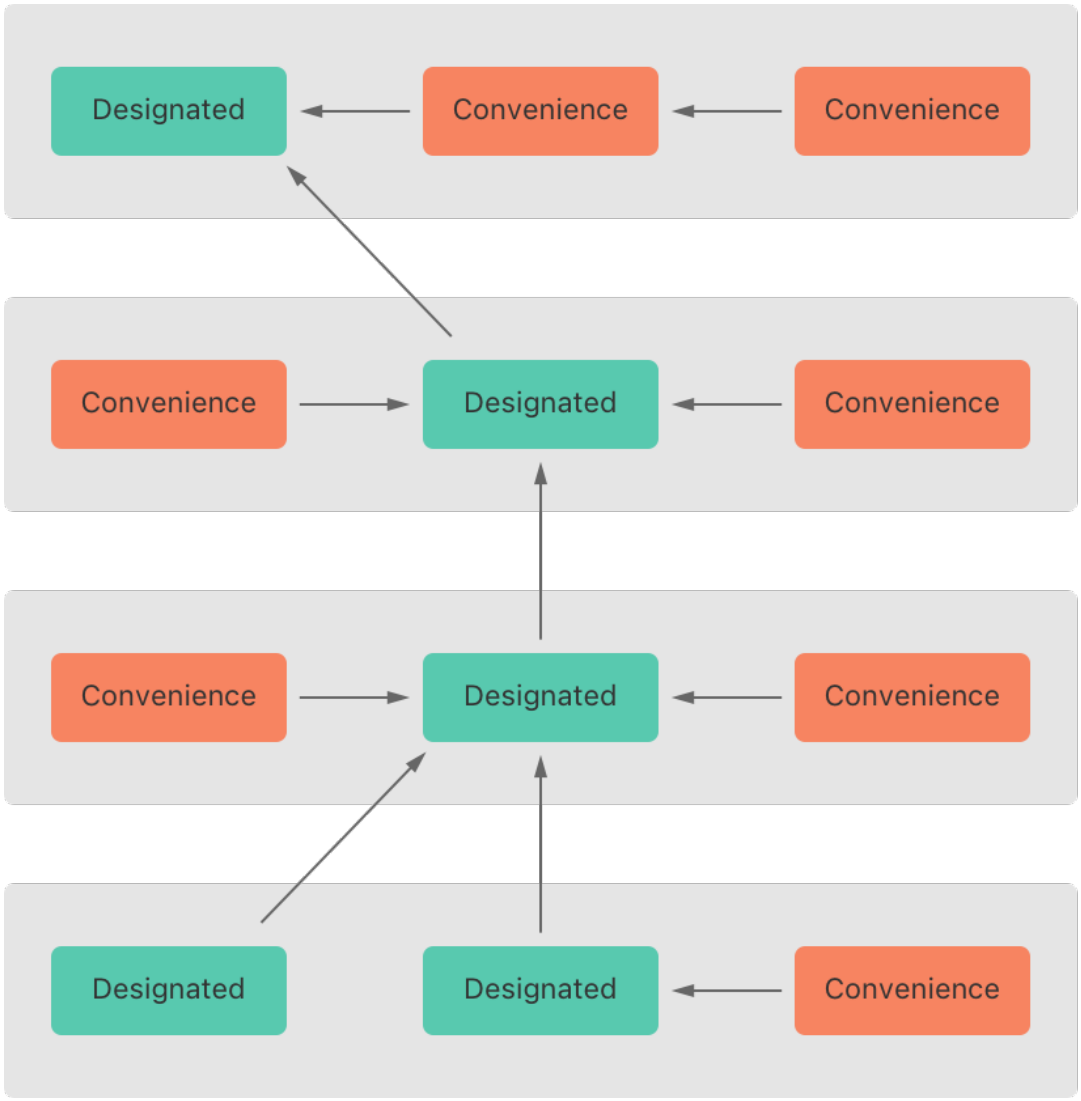


여기에서 상위 클래스는 하나의 지정 이니셜라이저와 두 개의 편의 이니셜라이저를 가지고 있습니다. 하나의 편의 이니셜라이저는 다른 편의 이니셜라이저를 호출하고, 그 이니셜라이저는 다시 지정 이니셜라이저를 호출합니다. 이는 위의 규칙 2와 3을 충족합니다. 상위 클래스는 더 상위의 상위 클래스를 가지고 있지 않으므로 규칙 1은 적용되지 않습니다.

그림에 하위 클래스는 두 개의 지정 이니셜라이저와 하나의 편의 이니셜라이저를 가지고 있습니다. 편의 이니셜라이저는 같은 클래스의 다른 이니셜라이저를 호출해야 하므로, 지정 이니셜라이저 중 하나를 호출해야 합니다. 이는 위의 규칙 2와 3을 충족합니다. 두 개의 지정 이니셜라이저는 모두 상위 클래스의 지정 이니셜라이저를 호출해야 하므로, 규칙 1도 충족됩니다.

> **참고**
>
> 이러한 규칙은 각 클래스의 인스턴스를 생성하는 방법에 영향을 주지 않습니다. 위 다이어그램에 모든 이니셜라이저는 해당 클래스의 완전한 인스턴스를 생성하는 데 사용할 수 있습니다. 규칙은 클래스의 이니셜라이저 구현을 작성할 때만 영향을 줍니다.

아래 그림은 네 개의 클래스에 대한 더 복잡한 클래스 계층도를 보여줍니다. 이 계층도에서 지정 이니셜라이저는 클래스 초기화의 “중심 경로”로 작동하며, 클래스 간의 관계를 단순화하는 방식을 설명합니다:



2단계 초기화 (Two-Phase Initialization)

Swift에서 클래스 초기화는 두 단계로 이루어집니다. 1단계에서는 각 저장 프로퍼티가 해당 프로퍼티를 정의한 클래스에 의해 초기값을 할당받습니다. 모든 저장 프로퍼티의 초기 상태가 결정되면, 2단계가 시작되며, 각 클래스는 새 인스턴스가 사용 가능 상태가 되기 전에 자신의 저장 프로퍼티를 추가로 구성할 수 있는 기회를 가집니다.

2단계 초기화는 클래스 계층 구조 안에서 각 클래스에 완전한 유연성을 부여하면서도, 안전한 초기화를 보장합니다. 2단계 초기화는 프로퍼티 값이 초기화되기 전에 접근하는 것을 방지하고, 다른 이니셜라이저가 예기치 않게 다른 값을 설정하는 것을 방지합니다.

> **참고**
>
> Swift의 2단계 초기화는 Objective-C의 초기화와 유사합니다. 주요 차이점은 Objective-C에서는 1단계 동안 모든 프로퍼티에 0 또는 null 값(`0` 또는 `nil`)을 할당합니다. Swift는 커스텀 초기값을 설정할 수 있고, `0`이나 `nil`이 유효한 초기값이 아닌 타입도 처리할 수 있다는 점에서 더 유연합니다.

Swift 컴파일러는 2단계 초기화가 오류없이 완료되도록 다음 네 가지 안전성 검사를 수행합니다:

- **안전성 검사 1**: 지정 이니셜라이저는 상위 클래스 이니셜라이저를 호출하기 전에 자신이 정의한 모든 프로퍼티를 초기화해야 합니다.

위에서 언급했듯이 객체의 메모리는 모든 저장 프로퍼티의 초기 상태가 경정된 이후에야 완전히 초기화된 것으로 간주합니다. 이 규칙이 충족하려면 지정 이니셜라이저는 위임을을 넘기기 전에 자신의 모든 프로퍼티를 초기화해야 합니다.

- **안전성 검사 2**: 지정 이니셜라이저는 상속받은 프로퍼티에 값을 할당하기 전에 상위 클래스 이니셜라이저를 호출해야 합니다. 그렇지 않으면 지정 이니셜라이저에 할당한 새로운 값이 상위 클래스의 초기화 과정 중에 덮어써질 수 있습니다.
- **안전성 검사 3**: 편의 이니셜라이저는 모든 프로퍼티 (같은 클래스에 정의한 프로퍼티 포함)에 값을 할당하기 전에 다른 이니셜라이저를 호출해야 합니다. 그렇지 않으면 편의 이니셜라이저에 할당한 새로운 값은 자체 클래스의 지정 이니셜라이저에 의해 덮어써질 수 있습니다.
- **안전성 검사 4**: 초기화의 1단계가 완료되기 전까지는 어떤 인스턴스 메서드도 호출할 수 없으며, 인스턴스 프로퍼티를 읽거나 `self`를 값으로 참조할 수 없습니다.

1단계가 끝나기 전까지는 클래스 인스턴스가 완전히 유효하지 않습니다. 1단계가 끝나고 클래스 인스턴스가 유효한 것으로 판단된 후에만 프로퍼티에 접근할 수 있고 메서드를 호출할 수 있습니다.

다음은 위의 네 가지 검사 규칙에 따른 2단계 초기화의 흐름입니다:

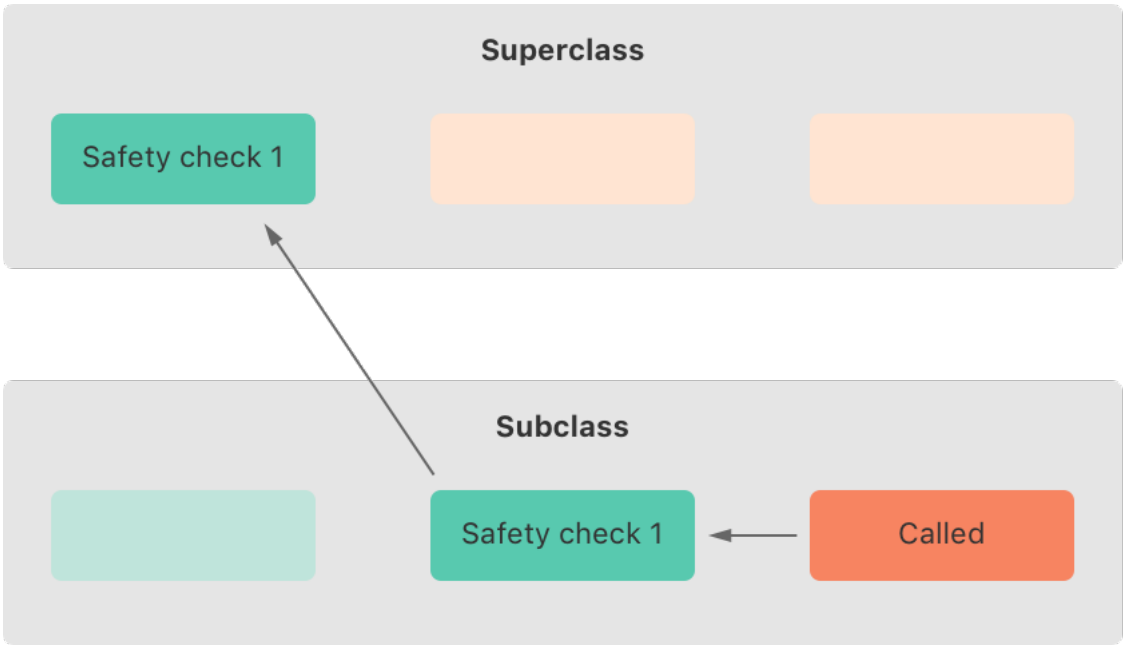
1단계

- 클래스의 지정 또는 편의 이니셜라이저가 호출됩니다.
- 해당 클래스 인스턴스의 메모리가 할당됩니다. 하지만 아직 초기화되지는 않았습니다.
- 지정 이니셜라이저는 해당 클래스가 정의한 저장 프로퍼티가 모두 값이 있는지 확인합니다. 이러한 저장 프로퍼티의 메모리를 초기화합니다.
- 상위 클래스의 지정 이니셜라이저를 호출해 해당 클래스의 프로퍼티를 초기화합니다.
- 이 과정은 상속 체계의 최상위 클래스까지 계속됩니다.
- 최상위 클래스에서 모든 저장 프로퍼티가 초기화되면, 인스턴스의 메모리는 완전히 초기화된 것으로 간주되며 1단계가 완료됩니다.

2단계

- 상속 체계의 최상위 클래스부터 아래 방향으로 내려가면서 각 지정 이니셜라이저는 인스턴스를 추가로 구성할 수 있습니다. 이니셜라이저는 이제 `self`에 접근할 수 있으며 프로퍼티를 수정하거나 메서드를 호출할 수 있습니다.
- 마지막으로 처음 호출된 편의 이니셜라이저가 인스턴스를 구성하거나 `self`를 사용할 수 있습니다.

다음은 하위 클래스와 상위 클래스를 대상으로 한 초기화 호출의 1단계를 보여줍니다:



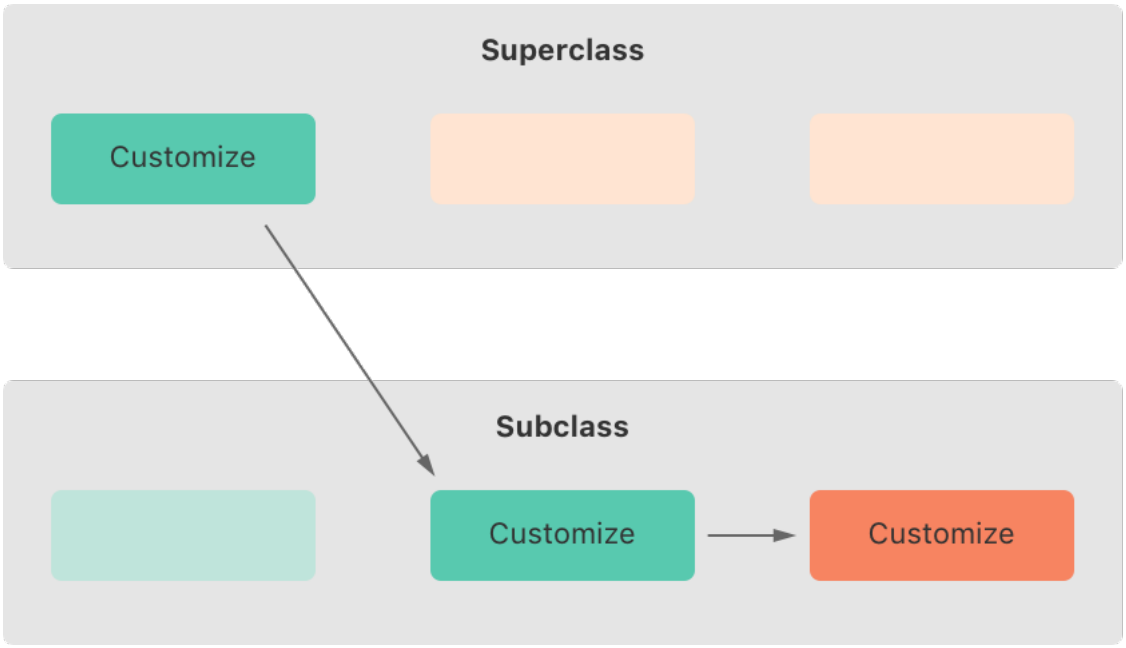
이 예시에서 초기화는 하위 클래스의 편의 이니셜라이저를 호출하며 시작합니다. 이 편의 이니셜라이저는 아직 프로퍼티를 수정할 수 없습니다. 이것은 같은 클래스의 지정 이니셜라이저로 위임합니다.

지정 이니셜라이저는 안전성 검사 1에 따라 하위 클래스의 모든 프로퍼티가 초기화되었는지 확인합니다. 그런 다음 상위 클래스의 지정 이니셜라이저를 호출하여 상위 체계까지 초기화를 계속합니다.

상위 클래스의 지정 이니셜라이저는 상위 클래스의 모든 프로퍼티가 초기화되었는지 확인합니다. 더 이상 초기화할 상위 클래스가 없으므로, 위임은 여기서 끝납니다.

상위 클래스의 모든 프로퍼티가 초기화되면 메모리는 완전히 초기화된 것으로 간주하고 1단계가 완료됩니다.

다음은 같은 초기화 호출에 대한 2단계를 나타냅니다:



상위 클래스의 지정 이니셜라이저는 인스턴스를 추가로 구성할 수 있는 기회를 가집니다 (꼭 하지 않아도 됩니다).

상위 클래스의 지정 이니셜라이저가 완료되면, 하위 클래스의 지정 이니셜라이저도 추가 구성을 수행할 수 있습니다 (역시 선택 사항입니다).

마지막으로 하위 클래스의 지정 이니셜라이저가 완료되면, 처음 호출되었던 편의 이니셜라이저가 최종적으로 인스턴스를 추가로 구성할 수 있습니다.

이니셜라이저 상속과 재정의 (Initializer Inheritance and Overriding)

Objective-C의 하위 클래스와 다르게, Swift 하위 클래스는 기본적으로 상위 클래스의 이니셜라이저를 상속하지 않습니다. Swift는 단순한 이니셜라이저가 더 특화된 하위 클래스에 상속되어, 완전히 초기화되지 않거나 잘못 초기화된 인스턴스를 생성하는 상황을 방지합니다.

> **참고**
>
> 상위 클래스의 이니셜라이저는 특정 조건을 만족할 경우에만 상속되지만, 안전하고 적절한 경우에만 상속됩니다. 자세한 내용은 아래의 자동 이니셜라이저 상속 (Automatic Initializer Inheritance)을 참고바랍니다.

하위 클래스에서 상위 클래스와 동일한 이니셜라이저를 제공하고 싶다면, 하위 클래스 안에 해당 이니셜라이저의 커스텀 구현을 제공해야 합니다.

하위 클래스의 이니셜라이저가 상위 클래스의 지정 이니셜라이저와 시그니처가 일치한다면, 이는 해당 지정 이니셜라이저를 재정의하는 것입니다. 따라서 하위 클래스의 이니셜라이저 정의 앞에 `override` 수정자를 작성해야 합니다. 기본 이니셜라이저 (Default Initializers)에서 설명했듯이 자동으로 제공된 기본 이니셜라이저를 재정의하는 경우에도 마찬가지입니다.

프로퍼티, 메서드, 서브스크립트를 재정의할 때처럼, `override` 수정자가 사용되면 Swift는 해당 상위 클래스에 재정의할 수 있는 지정 이니셜라이저가 실제로 존재하는지 확인하고, 재정의하는 이니셜라이저의 매개변수 역시 올바르게 정의되었는지 검증합니다.

> **참고**
>
> 상위 클래스의 지정 이니셜라이저를 재정의할 때는, 하위 클래스에서 편의 이니셜라이저로 구현하더라도 항상 `override` 수정자를 작성해야 합니다.

반대로 하위 클래스에서 상위 클래스의 편의 이니셜라이저와 시그니처가 일치하는 이니셜라이저를 작성하는 경우, 위의 클래스 타입의 이니셜라이저 위임 (Initializer Delegation for Class Types)에서 설명한 규칙에 따라 해당 상위 클래스의 편의 이니셜라이저를 하위 클래스에서 직접 호출할 수는 없습니다. 따라서 하위 클래스는 상위 클래스의 이니셜라이저를 재정의하는 것이 아닙니다. 결과적으로 상위 클래스의 편의 이니셜라이저와 시그니처가 같은 구현을 제공하더라도 `override` 수정자를 작성하지 않습니다.

아래 예시는 `Vehicle`이라는 기본 클래스를 정의합니다. 이 기본 클래스는 `numberOfWheels`라는 저장 프로퍼티를 선언하며, 이 프로퍼티는 기본 `Int` 값 `0`을 가집니다. `numberOfWheels` 프로퍼티는 `description`이라는 연산 프로퍼티에서 사용되어, 해당 차량의 특성을 설명하는 `String` 타입의 설명을 생성합니다:

```
class Vehicle {
    var numberOfWheels = 0
    var description: String {
        return "\(numberOfWheels) wheel(s)"
    }
}
```

`Vehicle` 클래스는 저장 프로퍼티에 대해 기본값을 제공하고, 별도의 커스텀 이니셜라이저는 제공하지 않습니다. 그 결과 기본 이니셜라이저 (Default Initializers)에서 설명했듯이 자동으로 기본 이니셜라이저를 얻습니다. 기본 이니셜라이저(사용 가능한 경우)는 항상 클래스의 지정 이니셜라이저로 간주되며, 이를 통해 `numberOfWheels` 값이 `0`인 새로운 `Vehicle` 인스턴스를 생성할 수 있습니다:

```
let vehicle = Vehicle()
print("Vehicle: \(vehicle.description)")
// Vehicle: 0 wheel(s)
```

다음 예시는 `Bicycle`이라는 `Vehicle`의 하위 클래스를 정의합니다:

```
class Bicycle: Vehicle {
    override init() {
        super.init()
        numberOfWheels = 2
    }
}
```

`Bicycle` 하위 클래스는 커스텀 지정 이니셜라이저인 `init()`을 정의합니다. 이 지정 이니셜라이저는 `Bicycle`의 상위 클래스의 지정 이니셜라이저와 시그니처가 일치하므로, `Bicycle`에서 이 이니셜라이저를 정의할 때는 `override` 수정자를 표기해야 합니다.

`Bicycle`의 `init()` 이니셜라이저는 먼저 `super.init()`을 호출하여, `Bicycle`의 상위 클래스인 `Vehicle`의 기본 이니셜라이저를 호출합니다. 이 호출을 통해 `Bicycle`이 해당 프로퍼티를 수정하기 전에, `Vehicle`로부터 상속받은 `numberOfWheels` 프로퍼티가 초기화되도록 보장됩니다. `super.init()` 호출 이후에는 기존의 `numberOfWheels` 값을 새로운 값인 `2`로 변경합니다.

`Bicycle` 인스턴스를 생성하면, 상속받은 `description` 연산 프로퍼티를 호출하여 `numberOfWheels` 프로퍼티가 어떻게 업데이트되었는지 확인할 수 있습니다:

```
let bicycle = Bicycle()
print("Bicycle: \(bicycle.description)")
// Bicycle: 2 wheel(s)
```

하위 클래스의 이니셜라이저가 초기화 과정의 2단계에서 별도의 커스터마이징을 수행하지 않고, 상위 클래스가 동기적이며 인자가 없는 지정 이니셜라이저를 가지고 있다면, 모든 하위 클래스의 저장 프로퍼티에 값을 할당한 후에 `super.init()` 호출을 생략할 수 있습니다. 상위 클래스의 이니셜라이저가 비동기라면, 명시적으로 `await super.init()`을 작성해야 합니다.

아래 예시는 `Vehicle`의 다른 하위 클래스인 `Hoverboard`라는 클래스를 정의합니다. `Hoverboard` 클래스의 이니셜라이저에서는 `color` 프로퍼티만 설정합니다. `super.init()`을 명시적으로 호출하는 대신에, 상위 클래스의 이니셜라이저가 암시적으로 호출되어 초기화가 완료됩니다.

```
class Hoverboard: Vehicle {
    var color: String
    init(color: String) {
        self.color = color
        // super.init() implicitly called here
    }
    override var description: String {
        return "\(super.description) in a beautiful \(color)"
    }
}
```

`Hoverboard`의 인스턴스는 `Vehicle` 이니셜라이저에서 제공하는 기본 바퀴 수를 사용합니다.

```
let hoverboard = Hoverboard(color: "silver")
print("Hoverboard: \(hoverboard.description)")
// Hoverboard: 0 wheel(s) in a beautiful silver
```

참고

하위 클래스는 초기화 중에 상속받은 변수 프로퍼티는 수정할 수 있지만, 상수 프로퍼티는 수정할 수 없습니다.

자동 이니셜라이저 상속 (Automatic Initializer Inheritance)

위에서 언급했듯이, 하위 클래스는 기본적으로 상위 클래스의 이니셜라이저를 상속받지 않습니다. 그러나 특정 조건이 충족하면 상위 클래스의 이니셜라이저가 자동으로 상속됩니다. 실제로 이것은 대부분의 경우에 이니셜라이저를 재정의할 필요 없이, 안전한 경우 최소한의 코드로 상위 클래스의 이니셜라이저를 상속받을 수 있습니다.

하위 클래스에서 도입한 모든 새로운 프로퍼티에 기본값을 제공하면, 아래의 두 가지 규칙이 적용됩니다:

- **규칙 1**: 하위 클래스가 자체 지정 이니셜라이저를 정의하지 않으면, 상위 클래스의 모든 지정 이니셜라이저를 자동으로 상속받습니다.
- **규칙 2**: 하위 클래스가 상위 클래스의 모든 지정 이니셜라이저를 구현한 경우 - 규칙 1에 따라 상속받거나, 직접 구현함으로써 - 상위 클래스의 모든 편의 이니셜라이저도 자동으로 상속됩니다.

이러한 규칙은 하위 클래스가 자체적으로 편의 이니셜라이저를 추가하더라도 여전히 적용됩니다.

참고

규칙 2를 만족시키기 위해, 하위 클래스가 상위 클래스의 지정 이니셜라이저를 편의 이니셜라이저 형태로 구현할 수도 있습니다.

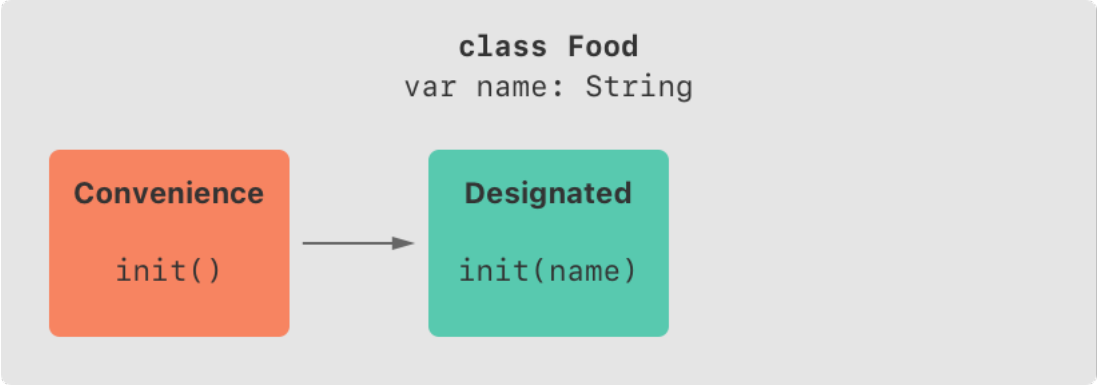
지정 이니셜라이저와 편의 이니셜라이저의 동작 (Designated and Convenience Initializers in Action)

다음 예시는 지정 이니셜라이저, 편의 이니셜라이저, 자동 이니셜라이저 상속이 실제로 어떻게 동작하는지 보여줍니다. 이 예시는 `Food`, `RecipeIngredient`, `ShoppingListItem`이라는 세 개의 클래스로 구성된 계층 구조를 정의하고, 이니셜라이저가 상호작용하는 방식을 보여줍니다.

계층 구조의 기반이 되는 클래스는 `Food`이며, 이는 음식 이름을 캡슐화하기 위한 간단한 클래스입니다. `Food` 클래스는 `name`이라는 `String` 프로퍼티를 도입하고, `Food` 인스턴스를 생성하기 위한 두 개의 이니셜라이저를 제공합니다:

```
class Food {
    var name: String
    init(name: String) {
        self.name = name
    }
    convenience init() {
        self.init(name: "[Unnamed]")
    }
}
```

아래의 그림은 `Food` 클래스의 이니셜라이저 체인을 나타냅니다:



클래스는 기본 멤버와이즈 이니셜라이저를 제공하지 않기 때문에, Food 클래스는 name이라는 하나의 인자를 가지는 지정 이니셜라이저를 제공합니다. 이 이니셜라이저는 특정 이름으로 새로운 Food 인스턴스를 생성할 때 사용할 수 있습니다:

```
let namedMeat = Food(name: "Bacon")
// namedMeat's name is "Bacon"
```

Food 클래스의 `init(name: String)` 이니셜라이저는 새로운 Food 인스턴스에 모든 저장 프로퍼티는 완벽하게 초기화 되므로 지정 이니셜라이저로 제공됩니다. Food 클래스는 상위 클래스가 없으므로, `init(name: String)` 이니셜라이저는 초기화를 완료하기 위해 `super.init()`을 호출할 필요가 없습니다.

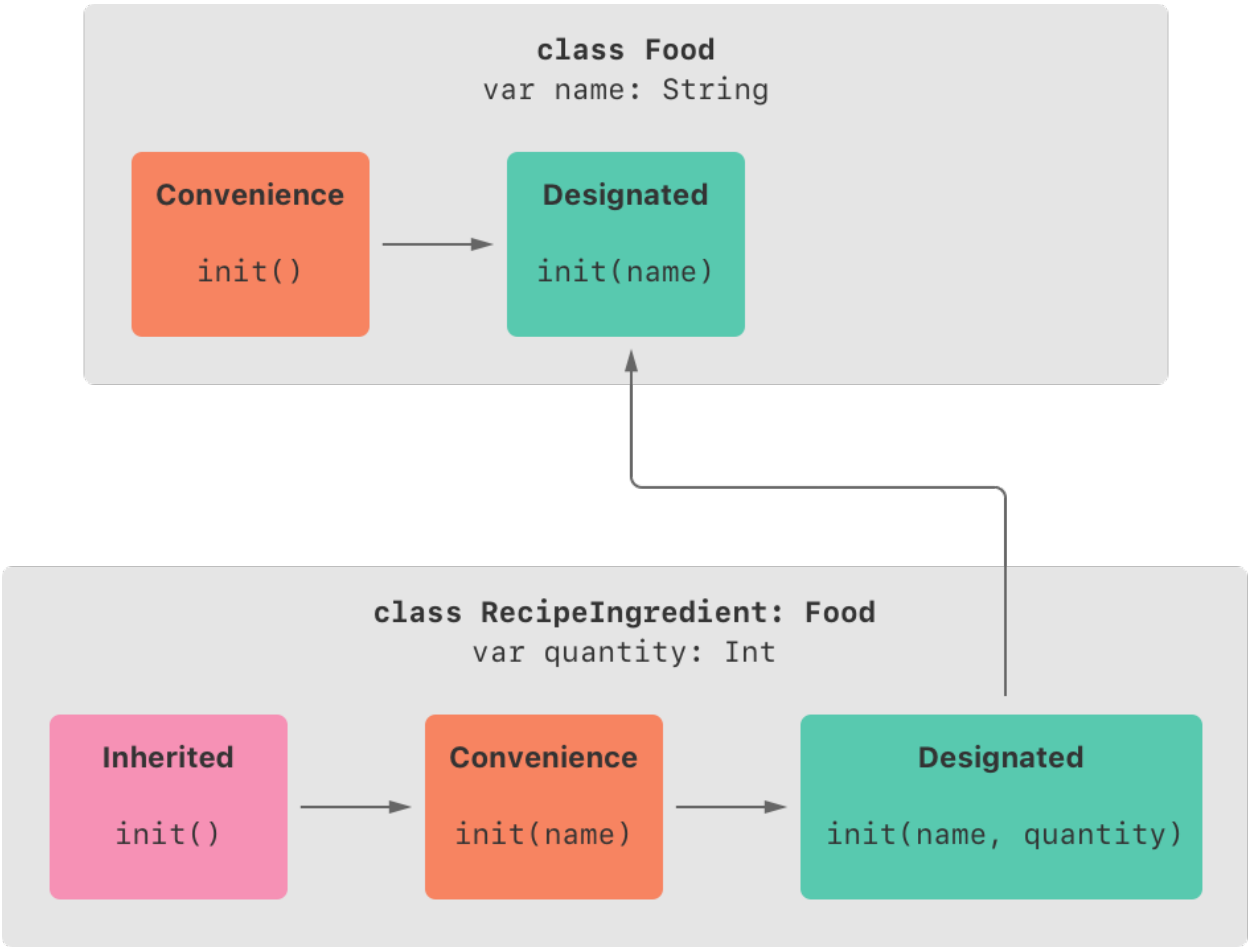
Food 클래스는 인자가 없는 편의 이니셜라이저 `init()`도 제공합니다. `init()` 이니셜라이저는 `[Unnamed]`을 name 값으로 사용하여 새로운 음식 이름을 설정하기 위해 Food 클래스의 `init(name: String)` 이니셜라이저로 위임합니다:

```
let mysteryMeat = Food()
// mysteryMeat's name is "[Unnamed]"
```

계층 구조의 두 번째 클래스는 Food의 하위 클래스인 RecipeIngredient입니다. RecipeIngredient 클래스는 요리 레시피의 재료를 모델링합니다. quantity라는 Int 타입의 프로퍼티를 추가로 도입하고 (이 클래스는 Food로 부터 name 프로퍼티도 상속받음) RecipeIngredient 인스턴스를 생성하기 위한 두 개의 이니셜라이저를 정의합니다:

```
class RecipeIngredient: Food {
    var quantity: Int
    init(name: String, quantity: Int) {
        self.quantity = quantity
        super.init(name: name)
    }
    override convenience init(name: String) {
        self.init(name: name, quantity: 1)
    }
}
```

아래의 그림은 RecipeIngredient 클래스의 이니셜라이저 체인을 보여줍니다:



`RecipeIngredient` 클래스는 하나의 지정 이니셜라이저 `init(name: String, quantity: Int)`를 가지고 있으며, 이를 통해 새로운 `RecipeIngredient` 인스턴스의 모든 프로퍼티를 초기화할 수 있습니다. 이 이니셜라이저는 먼저 전달받은 `quantity` 인자를 `RecipeIngredient` 클래스에서 새롭게 추가한 `quantity` 프로퍼티에 할당합니다. 그런 후에 `Food` 클래스의 `init(name: String)` 이니셜라이저로 위임합니다. 이 과정은 위의 2단계 초기화 (Two-Phase Initialization)에서 설명한 안전성 검사 1에 충족합니다.

`RecipeIngredient`는 편의 이니셜라이저인 `init(name: String)`도 정의하고 있으며, 이것은 이름만으로 `RecipeIngredient` 인스턴스를 생성하기 위해 사용합니다. 이 편의 이니셜라이저는 수량 없이 `RecipeIngredient` 인스턴스를 생성할 때, 수량을 `1`로 설정합니다. 이러한 편의 이니셜라이저의 정의는 여러 개의 단일 수량 `RecipeIngredient` 인스턴스를 더 빠르고 간편하게 생성할 수 있으며, 코드 중복을 피할 수 있습니다. 이 편의 이니셜라이저는 클래스의 지정 이니셜라이저에 `quantity` 값으로 `1`을 전달하여 위임합니다.

`RecipeIngredient`가 `init(name: String)`을 편의 이니셜라이저로 제공하지만, `Food`의 `init(name: String)` 지정 이니셜라이저와 같은 매개변수를 가지고 있습니다. 이 편의 이니셜라이저는 상위 클래스의 지정 이니셜라이저를 재정의하기 때문에, `override` 수정자를 붙여줘야 합니다 (자세한 내용은 이니셜라이저 상속과 재정의 (Initializer Inheritance and Overriding)에 설명되어 있습니다).

`RecipeIngredient`는 편의 이니셜라이저로 `init(name: String)`을 제공하지만, `RecipeIngredient`는 상위 클래스의 지정 이니셜라이저의 모든 구현을 제공했습니다. 따라서 `RecipeIngredient`는 자동으로 모든 상위 클래스의 편의 이니셜라이저를 상속받습니다.

이 예시에서 `RecipeIngredient`의 상위 클래스인 `Food`는 `init()`이라는 단일 편의 이니셜라이저를 가지고 있습니다. 따라서 이 이니셜라이저는 `RecipeIngredient`에 의해 상속됩니다. 상속된 `init()`은 `Food`에서와 동일하게 동작하지만, `Food`의 `init(name: String)`이 아닌 `RecipeIngredient`의 `init(name: String)`으로 위임됩니다.

이 세 가지 이니셜라이저 모두 새로운 `RecipeIngredient` 인스턴스를 생성할 수 있습니다:

```
let oneMysteryItem = RecipeIngredient()
let oneBacon = RecipeIngredient(name: "Bacon")
let sixEggs = RecipeIngredient(name: "Eggs", quantity: 6)
```

계층 구조에서 세 번째이자 마지막 클래스는 `ShoppingListItem`이라는 `RecipeIngredient`의 하위 클래스입니다. `ShoppingListItem` 클래스는 쇼핑 목록에 표시되는 요리 재료를 모델링합니다.

쇼핑 목록에 모든 아이템은 "미구매"로 시작합니다. 이것을 표현하기 위해 `ShoppingListItem`은 기본값이 `false`인 `purchased`라는 Boolean 프로퍼티를 도입합니다. `ShoppingListItem`은 `ShoppingListItem` 인스턴스의 설명을 제공하는 `description`이라는 연산 프로퍼티도 추가합니다:

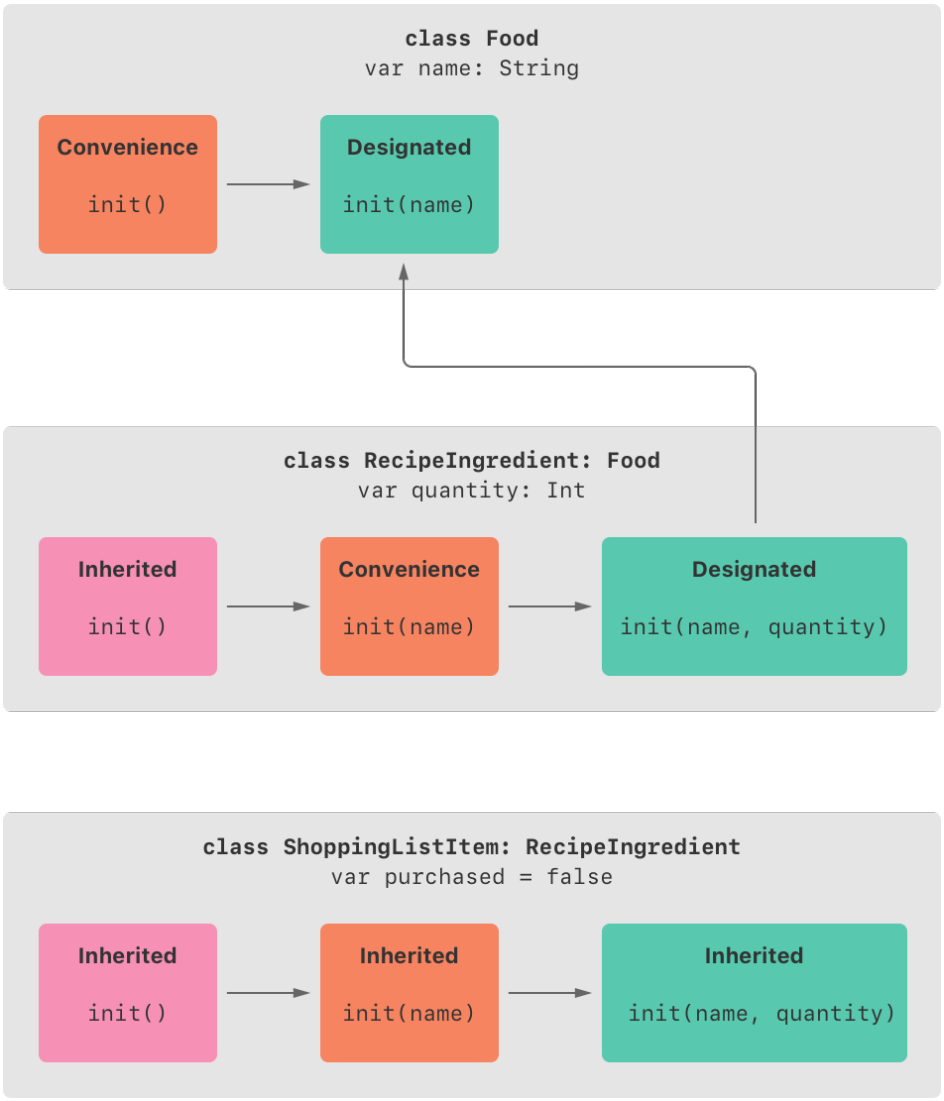
```
class ShoppingListItem: RecipeIngredient {
    var purchased = false
    var description: String {
        var output = "\(quantity) x \(name)"
        output += purchased ? " ✔" : " ✘"
        return output
    }
}
```

참고

쇼핑 목록의 항목들은(여기서 모델링된 방식에 따르면) 항상 미구매로 시작하기 때문에, `ShoppingListItem`은 `purchased`에 대한 초기값을 제공하기 위한 이니셜라이저를 정의하지 않습니다.

`ShoppingListItem`은 자신이 새롭게 도입한 모든 프로퍼티에 대해 기본값을 제공하고, 별도의 이니셜라이저를 정의하지 않으므로, 자동으로 상위 클래스의 모든 지정 이니셜라이저와 편의 이니셜라이저를 상속받습니다.

아래의 그림은 세 클래스에 대한 모든 이니셜라이저 체인을 나타냅니다:



상속받은 세 가지 이니셜라이저를 모두 사용하여 새로운 `ShoppingListItem` 인스턴스를 생성할 수 있습니다:

```
var breakfastList = [
    ShoppingListItem(),
    ShoppingListItem(name: "Bacon"),
    ShoppingListItem(name: "Eggs", quantity: 6),
]
breakfastList[0].name = "Orange juice"
breakfastList[0].purchased = true
for item in breakfastList {
    print(item.description)
}
// 1 x Orange juice ✔
// 1 x Bacon ✘
// 6 x Eggs ✘
```

여기서 `breakfastList`라는 새로운 배열은 세 개의 새로운 `ShoppingListItem` 인스턴스를 포함하는 배열 리터럴로 생성됩니다. 배열의 타입은 `[ShoppingListItem]`으로 추론됩니다. 배열이 생성된 후에 배열의 첫 번째 `ShoppingListItem`의 이름을 `"[Unnamed]"`에서 `"Orange juice"`로 변경하고, 구매된 것으로 표시합니다. 배열의 각 항목에 대한 설명을 출력하면, 기본 상태가 예상대로 설정되었음을 확인할 수 있습니다.

실패 가능한 이니셜라이저 (Failable Initializers)

클래스, 구조체, 열거형을 정의할 때, 이니셜라이저가 실패할 수 있도록 만드는 것이 유용할 수 있습니다. 이 실패는 유효하지 않은 초기화 매개변수 값, 필수 외부 리소스의 부재 또는 초기화를 성공적으로 완료하지 못하게 하는 다른 조건에 의해 발생할 수 있습니다.

실패 가능한 초기화 조건에 대응하기 위해, 클래스, 구조체, 열거형 정의 내에서 하나 이상의 실패 가능한 이니셜라이저를 정의할 수 있습니다. 실패 가능한 이니셜라이저는 `init` 키워드 뒤에 물음표(`init?`)를 표기하여 작성합니다.

> **참고**
>
> 동일한 매개변수 타입과 이름을 가지는 실패 가능한 이니셜라이저와 실패하지 않는 이니셜라이저를 정의할 수 없습니다.

실패 가능한 이니셜라이저는 초기화하려는 타입의 옵셔널 값을 생성합니다. 초기화 중 실패를 유동하고 싶을 때는 실패 가능한 이니셜라이저 내에 `return nil`을 작성하여 나타냅니다.

> **참고**
>
> 엄밀히 말하면 이니셜라이저는 값을 반환하지 않습니다. 오히려 이니셜라이저의 역할은 초기화가 끝날 때까지 `self`가 완전하고 올바르게 초기화되도록 보장하는 것입니다. 초기화 실패를 나타내기 위해 `return nil`을 작성하지만, 초기화 성공을 나타내기 위해 `return` 키워드를 사용하지는 않습니다.

예를 들어 숫자 타입 변환에서 실패 가능한 이니셜라이저가 사용됩니다. 숫자 타입 간 변환 시 값이 정확하게 유지되는지 확인하려면, `init(exactly:)` 이니셜라이저를 사용합니다. 변환으로 인해 값이 유지되지 못할 경우, 이니셜라이저는 실패합니다.

```
let wholeNumber: Double = 12345.0
let pi = 3.14159

if let valueMaintained = Int(exactly: wholeNumber) {
    print("\(wholeNumber) conversion to Int maintains value of \(valueMaintained)")
}
// Prints "12345.0 conversion to Int maintains value of 12345".

let valueChanged = Int(exactly: pi)
// valueChanged is of type Int?, not Int

if valueChanged == nil {
    print("\(pi) conversion to Int does not maintain value")
}
// Prints "3.14159 conversion to Int does not maintain value".
```

아래 예시는 `Animal`이라는 구조체를 정의하고, `species`라는 `String` 상수 프로퍼티를 가지고 있습니다. `Animal` 구조체는 `species`라는 하나의 매개변수를 가진 실패 가능한 이니셜라이저도 정의합니다. 이 이니셜라이저는 전달된 `species` 값이 빈 문자열인지 확인합니다. 빈 문자열이면, 초기화는 실패됩니다. 반대로 `species` 프로퍼티의 값은 설정되고 초기화는 성공합니다:

```
struct Animal {
    let species: String
    init?(species: String) {
        if species.isEmpty { return nil }
        self.species = species
    }
}
```

이 실패 가능한 이니셜라이저를 사용하여 새로운 `Animal` 인스턴스를 초기화하고, 초기화가 성공했는지 확인할 수 있습니다:

```
let someCreature = Animal(species: "Giraffe")
// someCreature is of type Animal?, not Animal

if let giraffe = someCreature {
    print("An animal was initialized with a species of \(giraffe.species)")
}
// Prints "An animal was initialized with a species of Giraffe".
```

실패 가능한 이니셜라이저의 `species` 매개변수에 빈 문자열 값을 전달하면, 이니셜라이저는 초기화가 실패합니다:

```
let anonymousCreature = Animal(species: "")
// anonymousCreature is of type Animal?, not Animal

if anonymousCreature == nil {
    print("The anonymous creature could not be initialized")
}
// Prints "The anonymous creature could not be initialized".
```

참고

빈 문자열 값(`"Giraffe"`가 아닌 `""`)을 확인하는 것은 옵셔널 `String` 값이 없음을 나타내는 `nil`을 확인하는 것과는 다릅니다. 위의 예시에서 빈 문자열(`""`)은 유효한 옵셔널이 아닌 `String`입니다. 그러나 `species` 프로퍼티의 값으로 빈 문자열을 갖는 것은 동물에 대해 적절하지 않습니다. 이러한 제약을 모델링하기 위해 실패 가능한 이니셜라이저는 빈 문자열이 발견되면 초기화에 실패합니다.

열거형의 실패 가능한 이니셜라이저 (Failable Initializers for Enumerations)

하나 이상의 매개변수를 기반으로 적절한 열거형 케이스를 선택하기 위해 실패 가능한 이니셜라이저를 사용할 수 있습니다. 이 이니셜라이저는 제공된 매개변수가 적절한 열거형 케이스와 일치하지 않으면 실패할 수 있습니다.

아래의 예시는 `TemperatureUnit`이라는 열거형을 정의하며, 가능한 세 가지 상태(`kelvin`, `celsius`, `fahrenheit`)를 가집니다. 실패 가능한 이니셜라이저는 온도 단위를 나타내는 `Character` 값에 대한 적절한 열거형 케이스를 찾습니다:

```
enum TemperatureUnit {
    case kelvin, celsius, fahrenheit
    init?(symbol: Character) {
        switch symbol {
        case "K":
            self = .kelvin
        case "C":
```

```
            self = .celsius
        case "F":
            self = .fahrenheit
        default:
            return nil
        }
    }
}
```

이 실패 가능한 이니셜라이저를 사용하여 세 가지 가능한 상태 중 적절한 열거형 케이스를 선택하고, 매개변수가 어떠한 상태도 일치하지 않으면 초기화를 실패하게 만들 수 있습니다:

```
let fahrenheitUnit = TemperatureUnit(symbol: "F")
if fahrenheitUnit != nil {
    print("This is a defined temperature unit, so initialization succeeded.")
}
// Prints "This is a defined temperature unit, so initialization succeeded.".

let unknownUnit = TemperatureUnit(symbol: "X")
if unknownUnit == nil {
    print("This is not a defined temperature unit, so initialization failed.")
}
// Prints "This is not a defined temperature unit, so initialization failed.".
```

원시값을 가진 열거형의 실패 가능한 이니셜라이저 (Failable Initializers for Enumerations with Raw Values)

원시값을 가진 열거형은 적절한 원시값 타입의 `rawValue`라는 매개변수를 받아, 일치하는 값을 찾으면 일치하는 열거형 케이스를 선택하거나, 일치하는 값이 없으면 초기화 실패를 나타내기 위해 자동으로 실패 가능한 이니셜라이저를 갖습니다.

위의 `TemperatureUnit` 예시를 `Character` 타입의 원시값을 사용하도록 다시 작성하면, `init?(rawValue:)` 이니셜라이저를 활용할 수 있습니다:

```
enum TemperatureUnit: Character {
    case kelvin = "K", celsius = "C", fahrenheit = "F"
}

let fahrenheitUnit = TemperatureUnit(rawValue: "F")
if fahrenheitUnit != nil {
    print("This is a defined temperature unit, so initialization succeeded.")
}
// Prints "This is a defined temperature unit, so initialization succeeded.".

let unknownUnit = TemperatureUnit(rawValue: "X")
if unknownUnit == nil {
    print("This is not a defined temperature unit, so initialization failed.")
}
// Prints "This is not a defined temperature unit, so initialization failed.".
```

초기화 실패의 전파 (Propagation of Initialization Failure)

클래스, 구조체, 열거형의 실패 가능한 이니셜라이저는 같은 클래스, 구조체, 열거형 내의 다른 실패 가능한 이니셜라이저로 위임할 수 있습니다. 마찬가지로 하위 클래스의 실패 가능한 이니셜라이저도 상위 클래스의 실패 가능한 이니셜라이저로 위임할 수 있습니다.

이러한 경우에 위임된 이니셜라이저가 초기화를 실패하면, 전체 초기화 과정은 즉시 실패로 처리되며, 이후의 초기화 코드는 실행되지 않습니다.

> **참고**
>
> 실패 가능한 이니셜라이저는 실패하지 않는 이니셜라이저로 위임할 수도 있습니다. 실패하지 않는 기존 초기화 과정에 실패 가능성을 추가해야 할 때 이 방식을 사용합니다.

아래 예시는 `CartItem`이라는 `Product`의 하위 클래스를 정의합니다. `CartItem` 클래스는 온라인 쇼핑 카트에 있는 상품을 모델링합니다. `CartItem`은 `quantity`라는 저장 상수 프로퍼티를 도입하고, 적어도 이 프로퍼티는 1이상의 값을 가지도록 보장합니다:

```
class Product {
    let name: String
    init?(name: String) {
        if name.isEmpty { return nil }
        self.name = name
    }
}

class CartItem: Product {
    let quantity: Int
    init?(name: String, quantity: Int) {
        if quantity < 1 { return nil }
        self.quantity = quantity
        super.init(name: name)
    }
}
```

`CartItem`의 실패 가능한 이니셜라이저는 `quantity` 값이 1이상인지 확인하는 것으로 시작합니다. `quantity`가 유효하지 않으면, 전체 초기화 과정은 즉시 실패하고 더 이상 초기화 코드는 실행되지 않습니다. 마찬가지로 `Product`의 실패 가능한 이니셜라이저는 `name` 값을 검사하고, `name`이 빈 문자열이라면 즉시 초기화 프로세스는 실패합니다.

이름을 가지고 1이상의 양을 가진 `CartItem` 인스턴스를 생성하면, 초기화는 성공합니다:

```
if let twoSocks = CartItem(name: "sock", quantity: 2) {
    print("Item: \(twoSocks.name), quantity: \(twoSocks.quantity)")
}
// Prints "Item: sock, quantity: 2".
```

`quantity` 값이 `0`인 `CartItem` 인스턴스를 생성하려고 하면, `CartItem` 이니셜라이저는 초기화에 실패합니다:

```
if let zeroShirts = CartItem(name: "shirt", quantity: 0) {
    print("Item: \(zeroShirts.name), quantity: \(zeroShirts.quantity)")
} else {
    print("Unable to initialize zero shirts")
}
// Prints "Unable to initialize zero shirts".
```

마찬가지로 빈 name 값을 가진 CartItem 인스턴스를 생성하려고 하면, 상위 클래스인 Product의 이니셜라이저가 초기화에 실패합니다:

```
if let oneUnnamed = CartItem(name: "", quantity: 1) {
    print("Item: \(oneUnnamed.name), quantity: \(oneUnnamed.quantity)")
} else {
    print("Unable to initialize one unnamed product")
}
// Prints "Unable to initialize one unnamed product".
```

실패 가능한 이니셜라이저 재정의 (Overriding a Failable Initializer)

다른 이니셜라이저처럼 상위 클래스의 실패 가능한 이니셜라이저를 하위 클래스에서 재정의할 수 있습니다. 또한 하위 클래스에서 상위 클래스의 실패 가능한 이니셜라이저를 실패하지 않는 이니셜라이저로 재정의할 수도 있습니다. 이를 통해 상위 클래스의 초기화는 실패할 수 있지만, 하위 클래스에서는 실패하지 않는 초기화를 정의할 수 있습니다.

실패 가능한 상위 클래스 이니셜라이저를 실패하지 않는 하위 클래스 이니셜라이저로 재정의하면, 상위 클래스의 이니셜라이저를 호출할 때에는 반드시 강제 언래핑을 통해 호출해야 합니다.

> **참고**
>
> 실패 가능한 이니셜라이저는 실패하지 않는 이니셜라이저로 재정의할 수 있지만, 그 반대는 불가능합니다.

아래의 예시는 Document라는 클래스를 정의합니다. 이 클래스는 비어 있지 않은 문자열 값이나 nil은 가능하지만, 빈 문자열은 불가능한 name 프로퍼티를 가지고 초기화할 수 있는 문서를 모델링합니다:

```
class Document {
    var name: String?
    // this initializer creates a document with a nil name value
    init() {}
    // this initializer creates a document with a nonempty name value
    init?(name: String) {
        if name.isEmpty { return nil }
        self.name = name
    }
}
```

다음 예시는 Document의 하위 클래스인 AutomaticallyNamedDocument를 정의합니다. AutomaticallyNamedDocument 하위 클래스는 Document에서 제공하는 지정 이니셜라이저 둘 다 재정의합니다. 이러한 재정의는 AutomaticallyNamedDocument 인스턴스가 이름 없이 초기화되거나 빈 문자열이 init(name:) 이니셜라이저에 전달되면 초기 name 값을 "[Untitled]"로 갖도록 보장합니다:

```
class AutomaticallyNamedDocument: Document {
    override init() {
        super.init()
        self.name = "[Untitled]"
    }
    override init(name: String) {
        super.init()
        if name.isEmpty {
            self.name = "[Untitled]"
        } else {
            self.name = name
        }
    }
}
```

`AutomaticallyNamedDocument`는 상위 클래스의 실패 가능한 `init?(name:)` 이니셜라이저를 실패하지 않는 `init(name:)` 이니셜라이저로 재정의합니다. `AutomaticallyNamedDocument`는 빈 문자열의 경우를 상위 클래스와 다르게 처리하므로, 이니셜라이저가 실패할 필요가 없으므로 대신 실패하지 않는 이니셜라이저 버전을 제공합니다.

하위 클래스의 실패하지 않는 이니셜라이저 구현에서 상위 클래스의 실패 가능한 이니셜라이저를 호출할 때 강제 언래핑을 사용할 수 있습니다. 예를 들어 아래의 `UntitledDocument` 하위 클래스는 항상 `"[Untitled]"` 이름을 가지고, 초기화 동안에 상위 클래스의 실패 가능한 `init(name:)` 이니셜라이저를 사용합니다.

```
class UntitledDocument: Document {
    override init() {
        super.init(name: "[Untitled]")!
    }
}
```

이러한 경우에 상위 클래스의 `init(name:)` 이니셜라이저가 빈 문자열을 이름으로 받아 호출 된 경우, 강제 언래핑 작업을 통해 런타임 오류가 발생합니다. 그러나 문자열 리터럴로 호출되기 때문에, 이니셜라이저가 실패하지 않는 것을 알 수 있으므로, 이 경우 런타임 오류가 발생하지 않습니다.

init! 실패 가능한 이니셜라이저 (The init! Failable Initializer)

일반적으로 `init` 키워드 뒤에 물음표를 배치하여(`init?`) 적절한 타입의 옵셔널 인스턴스를 생성하는 실패 가능한 이니셜라이저를 정의합니다. 또는 적절한 타입의 암시적 언래핑 옵셔널 인스턴스를 생성하는 실패 가능한 이니셜라이저를 정의할 수 있습니다. 물음표 대신에 `init` 키워드 뒤에 느낌표를 위치시키면 됩니다(`init!`).

`init?`에서 `init!`으로 또는 그 반대로 위임할 수 있으며, `init?`를 `init!`으로 또는 그 반대로 재정의 할 수 있습니다. `init`에서 `init!`으로 위임할 수도 있지만 그렇게 하면 `init!` 이니셜라이저가 초기화에 실패하면 단정문(assertion)이 발생합니다.

필수 이니셜라이저 (Required Initializers)

클래스 이니셜라이저 정의 앞에 `required` 수정자를 작성하여, 클래스를 상속받는 모든 하위 클래스가 해당 이니셜라이저를 구현해야 함을 나타냅니다:

```
class SomeClass {
    required init() {
        // initializer implementation goes here
    }
}
```

이니셜라이저 요구사항이 상속 계층의 더 아래 하위 클래스에도 적용된다는 것을 나타내기 위해 필수 이니셜라이저를 하위 클래스에서 구현할 때에도 반드시 이니셜라이저 정의 앞에 `required` 수정자를 작성해야 합니다. 필수 지정 이니셜라이저를 재정의할 때 `override` 수정자를 작성하지 않습니다:

```
class SomeSubclass: SomeClass {
    required init() {
        // subclass implementation of the required initializer goes here
    }
}
```

참고

상속받은 이니셜라이저로 요구사항을 충족할 수 있으면, 필수 이니셜라이저를 명시적으로 구현을 제공하지 않아도 됩니다.

클로저나 함수로 기본 프로퍼티 값 설정 (Setting a Default Property Value with a Closure or Function)

저장 프로퍼티의 기본값에 커스텀나 초기 설정이 필요한 경우, 해당 프로퍼티에 대한 커스텀된 기본값을 제공하기 위해 클로저나 전역 함수를 사용할 수 있습니다. 프로퍼티가 속한 타입의 새로운 인스턴스가 초기화 될 때마다, 클로저나 함수가 호출되고 그것의 반환값은 프로퍼티의 기본값으로 할당됩니다.

이러한 종류의 클로저나 함수는 일반적으로 프로퍼티와 같은 타입의 임시 값을 생성하고 원하는 초기 상태를 나타내도록 해당 값을 조정한 다음 프로퍼티의 기본값으로 사용되기 위해 임시 값을 반환합니다.

다음은 기본 프로퍼티 값을 제공하기 위해 클로저를 사용하는 방법에 대한 예시입니다:

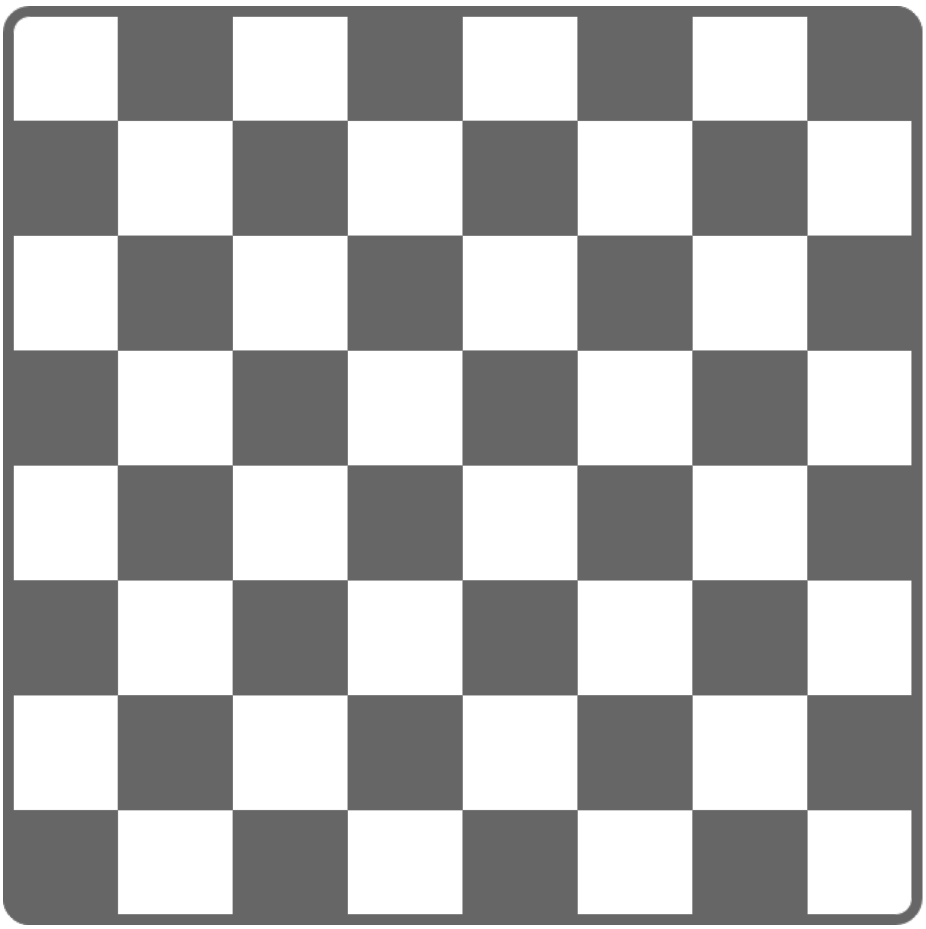
```
class SomeClass {
    let someProperty: SomeType = {
        // create a default value for someProperty inside this closure
        // someValue must be of the same type as SomeType
        return someValue
    }()
}
```

클로저의 중괄호 끝에 빈 소괄호 쌍이 옵니다. 이것은 Swift가 클로저를 즉시 실행하도록 합니다. 만약 소괄호를 생략한다면 클로저의 반환값이 아닌 프로퍼티에 클로저 자체를 할당하려고 합니다.

참고

프로퍼티를 초기화하기 위해 클로저를 사용하면, 클로저가 실행될 때 다른 인스턴스는 아직 초기화 되지 않았다는 것을 기억해야 합니다. 해당 프로퍼티에 기본값이 있더라도 클로저 내에서 다른 프로퍼티 값에 접근할 수 없다는 의미입니다. 또한 암시적인 `self`를 사용할 수 없으며, 인스턴스의 메서드를 호출할 수 없습니다.

아래 예시는 체스 게임을 위한 보드를 모델링하는 `Chessboard`라는 구조체를 정의합니다. 체스는 검은색과 하얀색의 사각형이 번갈아 가며 8 x 8 보드에서 플레이 됩니다.



게임보드를 표현하기 위해 `Chessboard` 구조체는 64개의 `Bool` 값 배열인 `boardColors`라는 하나의 프로퍼티를 가집니다. 배열에 `true` 값은 검은색 사각형을 표시하고 `false` 값은 하얀색 사각형을 표시합니다. 배열의 첫 번째 항목은 게임보드의 좌측 상단을 나타내고, 배열의 마지막 항목은 게임보드의 우측 하단을 나타냅니다.

`boardColors` 배열은 색깔 값을 설정하기 위한 클로저로 초기화됩니다:

```
struct Chessboard {
    let boardColors: [Bool] = {
        var temporaryBoard = [Bool]()
        var isBlack = false
        for i in 1...8 {
            for j in 1...8 {
                temporaryBoard.append(isBlack)
                isBlack = !isBlack
            }
            isBlack = !isBlack
        }
        return temporaryBoard
    }()
    func squareIsBlackAt(row: Int, column: Int) -> Bool {
        return boardColors[(row * 8) + column]
    }
}
```

새로운 `Chessboard` 인스턴스가 생성될 때마다 클로저가 실행되고, `boardColors`의 기본값은 계산되어 반환됩니다. 위의 예시의 클로저는 `temporaryBoard`라는 임시 배열에 보드의 각 사각형에 대한 적절한 색깔을 계산하고 설

정하고 설정이 완료되면 클로저의 반환값으로 임시 배열을 반환합니다. 반환된 배열 값은 `boardColors`에 저장되고 `squareIsBlackAt(row:column:)` 유틸리티 함수로 조회할 수 있습니다:

```
let board = Chessboard()
print(board.squareIsBlackAt(row: 0, column: 1))
// Prints "true".
print(board.squareIsBlackAt(row: 7, column: 7))
// Prints "false".
```

언어 가이드

소멸 (Deinitialization)

정리가 필요한 리소스를 해제합니다.

*디이니셜라이저(deinitializer)*는 클래스 인스턴스가 할당 해제되기 직전에 호출됩니다. 이니셜라이저는 `init` 키워드로 작성하는 것과 유사하게 디이니셜라이저는 `deinit` 키워드로 작성합니다. 디이니셜라이저는 클래스 타입에서만 사용 가능합니다.

소멸 동작 방식 (How Deinitialization Works)

Swift는 인스턴스가 더 이상 필요하지 않을 때, 자원 확보를 위해 인스턴스를 자동으로 할당 해제합니다. Swift는 자동 참조 카운팅 (Automatic Reference Counting)에서 설명했듯이 *자동 참조 카운팅(ARC)*을 통해 인스턴스의 메모리를 관리합니다. 일반적으로 인스턴스가 할당 해제될 때 수동으로 정리 작업을 수행할 필요는 없습니다. 그러나 직접 관리하는 리소스가 있다면, 추가적인 정리작업이 필요한 경우가 있습니다. 예를 들어 파일을 열고 데이터를 작성하는 클래스를 생성했다면, 클래스 인스턴스를 할당 해제하기 전에 파일을 닫아야 합니다.

클래스 디이니셜라이저는 클래스 당 하나의 디이니셜라이저만 가지고 있습니다. 디이니셜라이저는 매개변수가 없고 소괄호 없이 작성합니다:

```
deinit {
    // perform the deinitialization
}
```

디이니셜라이저는 인스턴스가 할당 해제 되기 직전에 자동으로 호출됩니다. 디이니셜라이저를 직접 호출할 수는 없습니다. 상위 클래스 디이니셜라이저는 하위 클래스로 상속되고, 상위 클래스 디이니셜라이저는 하위 클래스 디이니셜라이저 구현이 끝날 때 자동으로 호출됩니다. 상위 클래스 디이니셜라이저는 하위 클래스가 자신의 디이니셜라이저를 제공하지 않더라도 항상 호출됩니다.

인스턴스는 디이니셜라이저가 호출되기 전에는 할당 해제되지 않기 때문에, 디이니셜라이저는 인스턴스의 모든 프로퍼티에 접근할 수 있고 프로퍼티 기반으로 동작을 변경할 수 있습니다 (닫아야 하는 파일의 이름을 조회하는 경우 등).

디이니셜라이저 동작 (Deinitializers in Action)

다음은 디이니셜라이저에 대한 동작의 예시입니다. 이 예시는 간단한 게임을 위해 `Bank`와 `Player`라는 두 개의 새로운 타입을 정의합니다. `Bank` 클래스는 유통 중인 코인이 10,000개를 넘을 수 없는 가상 화폐를 관리합니다. 게임에서는 하나의 `Bank`만 가질 수 있으므로, `Bank`는 현재 상태를 저장하고 관리하는 타입 프로퍼티와 타입 메서드를 가지는 클래스로 구현됩니다:

```
class Bank {
    static var coinsInBank = 10_000
    static func distribute(coins numberOfCoinsRequested: Int) -> Int {
        let numberOfCoinsToVend = min(numberOfCoinsRequested, coinsInBank)
        coinsInBank -= numberOfCoinsToVend
        return numberOfCoinsToVend
```

```
    }
    static func receive(coins: Int) {
        coinsInBank += coins
    }
}
```

Bank는 coinsInBank 프로퍼티를 통해 현재 코인의 갯수를 추적합니다. 또한 코인의 분배와 회수를 처리하기 위해 distribute(coins:)와 receive(coins:)인 두 개의 메서드를 제공합니다.

distribute(coins:) 메서드는 코인을 분배하기 전에 은행에 코인이 충분한지 검사합니다. 코인이 충분하지 않으면, Bank는 요청된 수보다 더 작은 수를 반환합니다 (그리고 코인이 은행에 남아있지 않은 경우 0을 반환합니다). 제공된 실제 코인 수를 나타내는 정수를 반환합니다.

receive(coins:) 메서드는 회수한 코인의 수를 은행의 코인 저장소에 추가합니다.

Player 클래스는 게임에 플레이어를 나타냅니다. 각 플레이어는 언제든지 지갑에 특정 수의 동전을 보관합니다. 이것은 플레이어의 coinsInPurse 프로퍼티에 표시됩니다:

```
class Player {
    var coinsInPurse: Int
    init(coins: Int) {
        coinsInPurse = Bank.distribute(coins: coins)
    }
    func win(coins: Int) {
        coinsInPurse += Bank.distribute(coins: coins)
    }
    deinit {
        Bank.receive(coins: coinsInPurse)
    }
}
```

각 Player 인스턴스는 초기화 하는 동안 은행에서 지정된 수의 코인을 시작 허용량으로 초기화되지만 사용할 수 있는 코인이 충분하지 않은 경우 Player 인스턴스는 해당 수보다 적게 받을 수 있습니다.

Player 클래스는 은행에서 특정 수의 코인을 가져오고 플레이어의 지갑에 더하는 win(coins:) 메서드를 정의합니다. Player 클래스는 Player 인스턴스가 할당 해제되기 전에 호출되는 디이니셜라이저도 구현합니다. 여기서 디이니셜라이저는 단순하게 플레이어의 모든 코인을 은행에 반환합니다:

```
var playerOne: Player? = Player(coins: 100)
print("A new player has joined the game with \(playerOne!.coinsInPurse) coins")
// Prints "A new player has joined the game with 100 coins".
print("There are now \(Bank.coinsInBank) coins left in the bank")
// Prints "There are now 9900 coins left in the bank".
```

새로운 Player 인스턴스는 가능하면 100개의 코인을 요청하여 생성됩니다. Player 인스턴스는 playerOne이라는 옵셔널 Player 변수에 저장됩니다. 플레이어는 언제나 게임을 떠날 수 있기 때문에 옵셔널 변수가 사용됩니다. 이 옵셔널은 현재 게임의 플레이어인지 확인합니다.

playerOne은 옵셔널이기 때문에, 기본 코인의 수를 출력하기 위해 coinsInPurse 프로퍼티에 접근할 때와 win(coins:) 메서드가 호출될 때마다 느낌표(!)를 붙입니다:

```
playerOne!.win(coins: 2_000)
print("PlayerOne won 2000 coins & now has \(playerOne!.coinsInPurse) coins")
// Prints "PlayerOne won 2000 coins & now has 2100 coins".
print("The bank now only has \(Bank.coinsInBank) coins left")
// Prints "The bank now only has 7900 coins left".
```

여기서 플레이어는 2,000 코인을 가지고 있습니다. 플레이어의 지갑은 2,100 코인이 있고, 은행은 7,900 코인이 남아 있습니다.

```
playerOne = nil
print("PlayerOne has left the game")
// Prints "PlayerOne has left the game".
print("The bank now has \(Bank.coinsInBank) coins")
// Prints "The bank now has 10000 coins".
```

플레이어는 이제 게임을 떠납니다. 이것을 나타내기 위해 옵셔널 `playerOne` 변수에 "`Player` 인스턴스 없음"을 나타내는 `nil`을 설정합니다. 이 시점에 `Player` 인스턴스에 대한 `playerOne` 변수의 참조가 끊어집니다. 다른 프로퍼티나 변수도 더 이상 `Player` 인스턴스를 참조하지 않으므로, 메모리 확보를 위해 할당 해제 됩니다. 이것이 일어나기 직전에 자동으로 디이니셜라이저가 호출되고, 코인은 은행에 반환됩니다.

언어 가이드

옵셔널 체이닝 (Optional Chaining)

언래핑 없이 옵셔널 값의 멤버에 접근합니다.

*옵셔널 체이닝(Optional chaining)*은 현재 `nil`일 수 있는 옵셔널인 프로퍼티, 메서드, 서브스크립트를 접근하고 호출하는 과정입니다. 옵셔널에 값이 포함되어 있으면, 프로퍼티, 메서드, 서브스크립트는 호출에 성공합니다; 옵셔널이 `nil`이면, 프로퍼티, 메서드, 서브스크립트 호출은 `nil`을 반환합니다. 여러 번의 접근을 함께 연결될 수 있고, 체인에 어느 부분이라도 `nil`이면 전체 체인은 실패합니다.

> **참고**
>
> Swift의 옵셔널 체이닝은 Objective-C에서 `nil`에게 메세지를 보내는 것과 유사하지만, 모든 타입에 대해 동작하고 성공이나 실패 여부를 확인할 수 있습니다.

강제 언래핑의 대안으로 옵셔널 체이닝 (Optional Chaining as an Alternative to Forced Unwrapping)

프로퍼티, 메서드, 서브스크립트를 호출하려는 `nil`이 아닌 옵셔널 값 뒤에 물음표(`?`)를 배치하여 옵셔널 체이닝을 지정합니다. 이것은 옵셔널 값을 강제 언래핑하기 위해 옵셔널 값 뒤에 느낌표(`!`)를 배치하는 것과 유사합니다. 이것의 가장 큰 차이점은 옵셔널이 `nil`일 때 옵셔널 체이닝은 실패하는 반면에 강제 언래핑은 런타임 오류가 발생합니다.

옵셔널 체이닝은 `nil` 값에 대해 호출될 수 있다는 사실을 반영하기 위해 접근하는 프로퍼티, 메서드, 서브스크립트가 옵셔널 값이 아닌 값을 반환하더라도 항상 옵셔널 값으로 반환합니다. 옵셔널 반환값으로 옵셔널 체이닝 호출이 성공(반환된 옵셔널 체이닝에 값이 포함됨)했는지, 실패 (반환된 옵셔널 값은 `nil`)했는지 확인할 수 있습니다.

특히 옵셔널 체이닝 호출의 결과는 예상되는 반환값과 동일한 타입이지만 옵셔널로 래핑됩니다. 일반적으로 `Int`로 반환하는 프로퍼티는 옵셔널 체이닝을 통해 접근하면 `Int?`를 반환합니다.

다음의 코드 예시는 옵셔널 체이닝이 강제 언래핑과 어떻게 다른지 보여주고 성공 여부를 확인할 수 있도록 합니다.

먼저, `Person`과 `Residence`라는 두 개의 클래스를 정의합니다:

```
class Person {
    var residence: Residence?
}

class Residence {
    var numberOfRooms = 1
}
```

`Residence` 인스턴스는 기본값이 `1`인 `numberOfRooms`라는 `Int` 프로퍼티를 가지고 있습니다. `Person` 인스턴스는 `Residence?` 타입의 옵셔널 `residence` 프로퍼티를 가지고 있습니다.

새로운 `Person` 인스턴스를 생성하면, `residence` 프로퍼티는 옵셔널 규칙에 따라 `nil`로 초기화 됩니다. 아래의 코드에서 `john`은 `residence` 프로퍼티 값이 `nil`입니다:

```
let john = Person()
```

이 사람의 `residence`의 `numberOfRooms` 프로퍼티에 접근하기위해 `residence` 뒤에 느낌표를 배치하여 값을 언래핑 하면, 값이 없기 때문에 런타임 오류가 발생합니다:

```
let roomCount = john.residence!.numberOfRooms
// this triggers a runtime error
```

`john.residence`가 `nil`이 아니고 `roomCount`에 적절한 방의 갯수를 포함하는 `Int`로 설정하면 위의 코드는 정상 동작합니다. 그러나 `residence`가 `nil`이면 이 코드는 항상 런타임 오류가 발생합니다.

옵셔널 체이닝은 `numberOfRooms`의 값에 접근하기 위한 대안으로 제공합니다. 옵셔널 체이닝을 사용하기 위해 느낌표 위치에 물음표를 사용합니다:

```
if let roomCount = john.residence?.numberOfRooms {
    print("John's residence has \(roomCount) room(s).")
} else {
    print("Unable to retrieve the number of rooms.")
}
// Prints "Unable to retrieve the number of rooms.".
```

이것은 Swift에게 옵셔널 `residence` 프로퍼티를 “체이닝”하고 `residence`가 존재하면 `numberOfRooms` 값을 가져오도록 합니다.

`numberOfRooms` 접근하기 위한 시도는 실패할 수 있으므로, 옵셔널 체이닝은 `Int?` 타입이나 “옵셔널 `Int`“의 값을 반환합니다. 위 예시에서 `residence`가 `nil`일 경우, `numberOfRooms` 접근이 불가능한 사실을 반영하기 위해 옵셔널 `Int`는 `nil`입니다. 옵셔널 `Int`는 언래핑 한 정수에 옵셔널 바인딩으로 접근되고 `roomCount` 상수에 옵셔널이 아닌 값을 할당합니다.

`numberOfRooms`가 옵셔널 `Int`가 아니더라도 마찬가지입니다. 옵셔널 체이닝으로 조회된다는 것은 `numberOfRooms` 호출은 항상 `Int`가 아닌 `Int?`를 반환한다는 의미입니다.

더 이상 `nil` 값을 갖지 않기 위해 `john.residence`에 `Residence` 인스턴스를 할당할 수 있습니다:

```
john.residence = Residence()
```

`john.residence`는 `nil`이 아닌 실제 `Residence` 인스턴스를 포함합니다. 이전처럼 옵셔널 체이닝으로 `numberOfRooms`에 접근하면, 기본 `numberOfRooms` 값인 `1`을 포함하는 `Int?`를 반환할 것입니다:

```
if let roomCount = john.residence?.numberOfRooms {
    print("John's residence has \(roomCount) room(s).")
} else {
    print("Unable to retrieve the number of rooms.")
}
// Prints "John's residence has 1 room(s).".
```

옵셔널 체이닝에 대한 모델 클래스 정의 (Defining Model Classes for Optional Chaining)

한 단계 이상 깊은 프로퍼티, 메서드, 서브스크립트를 호출하기 위해 옵셔널 체이닝을 사용할 수 있습니다. 이를 통해 복잡하게 연결된 타입의 모델에서 하위 프로퍼티까지 깊이 접근하거나 해당 하위 프로퍼티의 프로퍼티, 메서드, 서브스크립트에 접근 가능한지 확인할 수 있습니다.

아래의 코드는 다단계 옵셔널 체이닝 예시를 포함해서 이후 여러 예시에서 사용할 네 개의 모델 클래스를 정의합니다. 이 클래스는 관련 프로퍼티, 메서드, 서브스크립트를 가지는 `Room`과 `Address` 클래스를 추가하여 위의 `Person`과 `Residence` 모델을 확장합니다.

`Person` 클래스는 이전과 같은 방식으로 정의됩니다:

```
class Person {
    var residence: Residence?
}
```

`Residence` 클래스는 이전보다 더 복잡합니다. 이제 `Residence` 클래스는 `[Room]` 타입의 빈 배열을 가지고 초기화 되는 `rooms`라는 변수 프로퍼티를 정의합니다:

```
class Residence {
    var rooms: [Room] = []
    var numberOfRooms: Int {
        return rooms.count
    }
    subscript(i: Int) -> Room {
        get {
            return rooms[i]
        }
        set {
            rooms[i] = newValue
        }
    }
    func printNumberOfRooms() {
        print("The number of rooms is \(numberOfRooms)")
    }
    var address: Address?
}
```

이 `Residence` 버전은 `Room` 인스턴스의 배열을 저장하기 때문에 `numberOfRooms` 프로퍼티는 저장 프로퍼티가 아닌 연산 프로퍼티로 구현됩니다. 연산 `numberOfRooms` 프로퍼티는 `rooms` 배열에서 `count` 프로퍼티의 값을 반환합니다.

`rooms` 배열에 접근하는 축약 구문을 위해 `Residence`는 `rooms` 배열에 요청된 인덱스로 방에 접근을 제공하는 읽기-쓰기 서브스크립트를 제공합니다.

이 `Residence`는 주택에 있는 방의 갯수를 출력하는 `printNumberOfRooms`라는 메서드도 제공합니다.

마지막으로 `Residence`는 `Address?` 타입을 가지는 `address`라는 옵셔널 프로퍼티를 정의합니다. 이 프로퍼티에 대한 `Address` 클래스 타입은 아래에 정의되어 있습니다.

`rooms` 배열에 사용된 `Room` 클래스는 `name`이라는 프로퍼티와 적절한 방 이름을 프로퍼티에 설정하는 이니셜라이저를 가진 클래스 입니다:

```
class Room {
    let name: String
    init(name: String) { self.name = name }
}
```

이 모델의 마지막 클래스는 `Address`라는 클래스입니다. 이 클래스는 `String?` 타입의 세 개의 옵셔널 프로퍼티를 가집니다. `buildingName`과 `buildingNumber`인 처음 두 프로퍼티는 주소로 특정 건물을 나타냅니다. 세 번째 프로퍼티 인 `street`는 주소에 대한 도로의 이름으로 사용됩니다:

```
class Address {
    var buildingName: String?
    var buildingNumber: String?
    var street: String?
    func buildingIdentifier() -> String? {
        if let buildingNumber, let street {
            return "\(buildingNumber) \(street)"
        } else if buildingName != nil {
            return buildingName
        } else {
            return nil
        }
    }
}
```

`Address` 클래스는 `String?`의 반환 타입을 가지는 `buildingIdentifier()`라는 메서드도 제공합니다. 이 메서드는 주소의 프로퍼티를 확인하고 값이 있으면 `buildingName`을 반환하거나 둘 다 값이 있으면 `street`과 연결된 `buildingNumber`를 반환하고 값이 없으면 `nil`을 반환합니다.

옵셔널 체이닝을 통한 프로퍼티 접근 (Accessing Properties Through Optional Chaining)

강제 언래핑의 대안으로 옵셔널 체이닝 (Optional Chaining as an Alternative to Forced Unwrapping)에서 설명했듯이 옵셔널 체이닝을 사용하면 옵셔널 값의 프로퍼티에 접근하고, 그 접근이 성공했는지 확인할 수 있습니다.

새로운 `Person` 인스턴스를 생성하기 위해 위에 정의된 클래스를 사용하고, 이전처럼 `numberOfRooms` 프로퍼티에 접근합니다:

```
let john = Person()
if let roomCount = john.residence?.numberOfRooms {
    print("John's residence has \(roomCount) room(s).")
} else {
    print("Unable to retrieve the number of rooms.")
}
// Prints "Unable to retrieve the number of rooms.".
```

`john.residence`는 `nil`이기 때문에 옵셔널 체이닝은 이전과 같이 실패를 호출합니다.

옵셔널 체이닝을 통해 프로퍼티의 값을 설정할 수도 있습니다:

```
let someAddress = Address()
someAddress.buildingNumber = "29"
someAddress.street = "Acacia Road"
john.residence?.address = someAddress
```

이 예시에서 `john.residence`이 `nil`이기 때문에 `john.residence`의 `address` 프로퍼티에 설정하기 위한 시도는 실패 할 것입니다.

이 할당은 = 연산자 오른쪽에 있는 코드가 실행되지 않으므로 옵셔널 체이닝의 일부입니다. 이전 예시에서 상수에 접근하는 것은 어떠한 영향도 없기 때문에 `someAddress`가 평가되지 않는다는 것을 쉽게 파악할 수 없습니다. 아래의 리스트는 동일한 할당을 수행하지만 주소를 생성하기 위해 함수를 사용합니다. 이 함수는 값을 반환하기 전에 “Function was called”를 출력하여 = 연산자 오른쪽에 있는 코드가 실행되었음을 알 수 있습니다.

```
func createAddress() -> Address {
    print("Function was called.")

    let someAddress = Address()
    someAddress.buildingNumber = "29"
    someAddress.street = "Acacia Road"

    return someAddress
}
john.residence?.address = createAddress()
```

아무것도 출력되지 않기 때문에, `createAddress()` 함수가 호출되지 않음을 알 수 있습니다.

옵셔널 체이닝을 통한 메서드 호출 (Calling Methods Through Optional Chaining)

옵셔널 체이닝을 사용하면 옵셔널 값에 대해 메서드를 호출하고 그 호출이 성공했는지 확인할 수 있습니다. 해당 메서드가 반환값을 정의하지 않아도 사용할 수 있습니다.

`Residence` 클래스에 `printNumberOfRooms()` 메서드는 `numberOfRooms`의 현재값을 출력합니다. 메서드는 다음과 같습니다:

```
func printNumberOfRooms() {
    print("The number of rooms is \(numberOfRooms)")
}
```

이 메서드는 반환 타입이 지정되지 않았습니다. 그러나 반환 타입이 없는 함수와 메서드는 반환값 없는 함수 (Functions Without Return Values)에서 설명했듯이 `Void`의 암시적 반환 타입을 가지고 있습니다. 이것은 `()`의 값이나 빈 튜플을 반환한다는 의미입니다.

옵셔널 체이닝을 사용하여 이 메서드를 호출하면, 반환값은 옵셔널 타입이기 때문에 메서드의 반환 타입은 `Void`가 아닌 `Void?`입니다. 메서드가 반환값을 정의하지 않았어도 `printNumberOfRooms()` 메서드 호출이 가능한지 `if` 구문을 사용하여 확인할 수 있습니다. `printNumberOfRooms` 호출의 반환값을 `nil`과 비교하여 메서드 호출이 성공했는지 확인합니다:

```
if john.residence?.printNumberOfRooms() != nil {
    print("It was possible to print the number of rooms.")
} else {
    print("It was not possible to print the number of rooms.")
}
// Prints "It was not possible to print the number of rooms.".
```

옵셔널 체이닝을 통해 프로퍼티를 설정하려는 경우에도 마찬가지입니다. 옵셔널 체이닝을 통한 프로퍼티 접근(Accessing Properties Through Optional Chaining)에서 위의 예시는 `residence` 프로퍼티가 `nil`이지만 `john.residence`에 대해 `address` 값을 설정 하려고 합니다. 옵셔널 체이닝을 통해 프로퍼티를 설정하려는 모든 시도는 `nil`과 비교하여 프로퍼티에 값이 성공적으로 설정되었는지 확인할 수 있는 `Void?` 타입의 값을 반환합니다:

```
if (john.residence?.address = someAddress) != nil {
    print("It was possible to set the address.")
} else {
    print("It was not possible to set the address.")
}
// Prints "It was not possible to set the address.".
```

옵셔널 체이닝을 통한 서브스크립트 접근 (Accessing Subscripts Through Optional Chaining)

옵셔널 값의 서브스크립트에서 값을 조회하고 설정하기 위해 해당 서브스크립트 호출이 성공했는지 확인하기 위해 옵셔널 체이닝을 사용할 수 있습니다.

> **참고**
>
> 옵셔널 체이닝을 통해 옵셔널 값의 서브스크립트에 접근할 때, 물음표는 서브스크립트의 대괄호 전에 위치합니다. 옵셔널 체이닝 물음표는 항상 옵셔널 표현구 부분의 바로 다음에 위치합니다.

아래의 예시는 `Residence` 클래스에 정의된 서브스크립트를 사용하여 `john.residence` 프로퍼티에 `rooms` 배열의 첫 번째 방 이름을 조회합니다. 현재 `john.residence`는 `nil`이므로, 서브스크립트 호출은 실패합니다:

```
if let firstRoomName = john.residence?[0].name {
    print("The first room name is \(firstRoomName).")
} else {
    print("Unable to retrieve the first room name.")
}
// Prints "Unable to retrieve the first room name.".
```

서브스크립트 호출에 옵셔널 체이닝 물음표는 `john.residence`가 옵셔널 체이닝이 시도되는 옵셔널 값이기 때문에 `john.residence`와 대괄호 사이에 위치합니다.

유사하게 옵셔널 체이닝으로 서브스크립트를 사용하여 새로운 값을 설정할 수 있습니다:

```
john.residence?[0] = Room(name: "Bathroom")
```

`residence`가 현재 `nil`이므로 이 서브스크립트 설정도 실패합니다.

rooms 배열에 하나 이상의 Room 인스턴스 가지고 john.residence에 실제 Residence 인스턴스를 생성하고 할당하면 옵셔널 체이닝으로 Residence 서브스크립트를 사용하여 rooms 배열의 항목을 접근할 수 있습니다:

```
let johnsHouse = Residence()
johnsHouse.rooms.append(Room(name: "Living Room"))
johnsHouse.rooms.append(Room(name: "Kitchen"))
john.residence = johnsHouse

if let firstRoomName = john.residence?[0].name {
    print("The first room name is \(firstRoomName).")
} else {
    print("Unable to retrieve the first room name.")
}
// Prints "The first room name is Living Room.".
```

옵셔널 타입의 서브스크립트 접근 (Accessing Subscripts of Optional Type)

서브스크립트가 Swift의 Dictionary 타입의 키 서브스크립트와 같이 옵셔널 타입의 값을 반환하는 경우 옵셔널 반환값을 연결하기 위해 서브스크립트의 닫는 대괄호 다음에 물음표를 추가합니다:

```
var testScores = ["Dave": [86, 82, 84], "Bev": [79, 94, 81]]
testScores["Dave"]?[0] = 91
testScores["Bev"]?[0] += 1
testScores["Brian"]?[0] = 72
// the "Dave" array is now [91, 82, 84] and the "Bev" array is now [80, 94, 81]
```

위의 예시는 String 키를 Int 값 배열에 매핑하는 두 개의 키-값 쌍을 포함하는 testScores라는 딕셔너리를 정의합니다. 이 예시는 "Dave" 배열의 첫 번째 항목에 91을 설정하고; "Bev" 배열의 첫 번째 항목에 1을 더하고; "Brian" 키에 대한 배열의 첫 번째 항목에 값을 설정하기 위해 옵셔널 체이닝을 사용합니다. testScores 딕셔너리는 "Dave"와 "Bev"에 대한 키를 가지고 있으므로, 처음 두 호출은 성공합니다. testScores 딕셔너리는 "Brian"에 대한 키를 가지고 있지 않으므로, 세 번째 호출은 실패합니다.

여러 단계의 체이닝 연결 (Linking Multiple Levels of Chaining)

여러 단계의 옵셔널 체이닝을 연결하여 모델 내에서 프로퍼티, 메서드, 서브스크립트를 더 깊게 접근할 수 있습니다. 그러나 여러 단계의 옵셔널 체이닝은 반환된 값에 옵셔널 수준을 더 추가하지 않습니다.

다른 말로 표현하면:

- 조회하려는 타입이 옵셔널이 아니면, 옵셔널 체이닝 때문에 옵셔널이 됩니다.
- 조회하려는 타입이 이미 옵셔널이면, 체이닝을 해도 더 많은 옵셔널이 되지 않습니다.

따라서:

- 옵셔널 체이닝으로 Int 값을 조회하려고 하면 사용된 체이닝의 수준과 상관없이 항상 Int?가 반환됩니다.
- 유사하게 옵셔널 체이닝으로 Int? 값을 조회하려고 하면 사용된 체이닝의 수준과 상관없이 항상 Int?가 반환됩니다.

아래의 예시는 john의 residence 프로퍼티의 address 프로퍼티의 street 프로퍼티에 접근합니다. 여기에서는 residence와 address 프로퍼티를 통해 연결하기 위해 두 단계 수준의 옵셔널 체이닝이 사용되며 둘 다 옵셔널 타입입니다:

```
if let johnsStreet = john.residence?.address?.street {
    print("John's street name is \(johnsStreet).")
} else {
    print("Unable to retrieve the address.")
}
// Prints "Unable to retrieve the address.".
```

`john.residence`의 값은 현재 유효한 `Residence` 인스턴스를 가지고 있습니다. 그러나 `john.residence.address`의 값은 현재 `nil`입니다. 이것 때문에 `john.residence?.address?.street`을 호출하면 실패합니다.

위의 예시에서는 `street` 프로퍼티의 값을 가져려고 합니다. 이 프로퍼티의 타입은 `String?`입니다. 따라서 `john.residence?.address?.street`의 반환값은 두 단계 옵셔널 체이닝이 프로퍼티의 옵셔널 타입에 적용되었지만 `String?`입니다.

`john.residence.address`에 대한 값으로 `Address` 인스턴스를 설정하고, 주소의 `street` 프로퍼티에 대해 값을 설정하면, 여러 단계의 옵셔널 체이닝을 통해 `street` 프로퍼티의 값에 접근할 수 있습니다:

```
let johnsAddress = Address()
johnsAddress.buildingName = "The Larches"
johnsAddress.street = "Laurel Street"
john.residence?.address = johnsAddress

if let johnsStreet = john.residence?.address?.street {
    print("John's street name is \(johnsStreet).")
} else {
    print("Unable to retrieve the address.")
}
// Prints "John's street name is Laurel Street.".
```

이 예시에서 `john.residence`의 값은 현재 유효한 `Residence` 인스턴스 이므로, `john.residence`의 `address` 프로퍼티에 값을 설정하는 것이 가능합니다.

옵셔널 반환값을 가지는 메서드에 대한 체이닝 (Chaining on Methods with Optional Return Values)

이전 예시는 어떻게 옵셔널 체이닝을 통해 옵셔널 타입의 프로퍼티의 값을 가져오는지 보여줍니다. 옵셔널 체이닝은 옵셔널 타입의 값을 반환하는 메서드를 호출하고 필요하다면 메서드의 반환값을 연결해서도 사용할 수 있습니다.

아래의 예시는 옵셔널 체이닝을 통해 `Address` 클래스의 `buildingIdentifier()` 메서드를 호출합니다. 이 메서드는 `String?` 타입의 값을 반환합니다. 위에서 설명했듯이 옵셔널 체이닝 이후 이 메서드 호출의 반환 타입은 `String?`입니다:

```
if let buildingIdentifier = john.residence?.address?.buildingIdentifier() {
    print("John's building identifier is \(buildingIdentifier).")
}
// Prints "John's building identifier is The Larches.".
```

메서드의 반환값에서 더 옵셔널 체이닝을 수행하려면, 메서드의 소괄호 다음에 옵셔널 체이닝 물음표를 위치시키면 됩니다:

```
if let beginsWithThe =
    john.residence?.address?.buildingIdentifier()?.hasPrefix("The") {
    if beginsWithThe {
        print("John's building identifier begins with \"The\".")
    } else {
        print("John's building identifier does not begin with \"The\".")
    }
}
// Prints "John's building identifier begins with "The".".
```

참고

위의 예시에서 연결하려는 옵셔널 값이 `buildingIdentifier()` 메서드 자체가 아닌 `buildingIdentifier()` 메서드의 반환값이므로 소괄호 다음에 옵셔널 체이닝 물음표를 위치시킵니다.

언어 가이드

오류 처리 (Error Handling)

오류에 대응하고 복구합니다.

*오류 처리(Error handling)*는 프로그램의 오류 조건에 대응하고 복구하는 과정입니다. Swift는 런타임에 복구 가능한 오류를 던지고 포착하고 전파하고 조작하기 위한 최고 수준의 지원을 제공합니다.

일부 작업은 항상 실행을 완료하거나 유용한 출력을 생성한다고 보장되지 않습니다. 옵셔널은 값이 없음을 나타내는 데 사용되지만 작업이 실패할 경우 코드가 그에 따라 응답할 수 있도록 오류의 원인을 이해하는 것이 유용한 경우가 많습니다.

예를 들어 디스크의 파일에서 데이터를 읽고 처리하는 작업을 생각해 봅시다. 지정된 위치에 파일이 존재하지 않거나, 파일에 읽기 권한이 없거나, 적절한 형식으로 인코딩 되지 않는 것을 포함하여 작업이 실패할 수 있는 많은 방법이 있습니다. 이러한 다른 상황을 구분하면 프로그램에서 일부 오류를 해결하고 해결할 수 없는 오류를 전달할 수 있습니다.

> **참고**
>
> Swift에서 오류 처리는 Cocoa와 Objective-C에 `NSError`를 사용하는 오류 처리 패턴과 상호 운용됩니다. 이 클래스에 대한 자세한 내용은 Swift에서 Cocoa 오류 처리 (Handling Cocoa Errors in Swift)를 참고바랍니다.

오류 표현과 던지기 (Representing and Throwing Errors)

Swift에서 오류는 `Error` 프로토콜을 준수하는 타입의 값으로 표현됩니다. 이 빈 프로토콜은 오류를 처리하는 것에 대해 사용될 수 있음을 나타냅니다.

Swift 열거형은 관련된 오류 조건의 그룹을 모델링하는데 특히 적합하며 연관값을 사용하여 오류의 특성의 추가 정보를 전달할 수 있습니다. 예를 들어 다음은 게임 내에서 자동 판매기를 작동하는 오류 조건을 나타내는 방법입니다:

```
enum VendingMachineError: Error {
    case invalidSelection
    case insufficientFunds(coinsNeeded: Int)
    case outOfStock
}
```

오류가 발생하면 예상치 못한 일이 발생하여 정상적인 흐름을 계속할 수 없음을 나타낼 수 있습니다. `throw` 구문을 사용하여 오류를 발생 시킵니다. 예를 들어 다음의 코드는 자동 판매기에 다섯 개의 코인이 더 필요하다고 오류를 발생 시킵니다:

```
throw VendingMachineError.insufficientFunds(coinsNeeded: 5)
```

오류 처리 (Handling Errors)

오류가 발생할 때, 주변 코드가 오류 처리를 담당해야 합니다 - 예를 들어 문제를 수정하거나, 다른 방법을 시도하거나, 사용자에게 오류를 알리는 방법으로 오류를 처리해야 합니다.

Swift에서는 오류를 처리하는 네 가지 방법이 있습니다. 함수에서 해당 함수를 호출하는 코드로 오류를 전파하거나, `do-catch` 구문을 사용하여 오류를 처리하거나, 옵셔널 값으로 오류를 처리하거나, 오류가 발생하지 않을 것이라고 주장할 수 있습니다. 이런 접근은 아래에 설명되어 있습니다.

함수에서 오류가 발생하면, 프로그램의 흐름이 변경되므로, 코드에서 오류가 발생할 수 있는 위치를 신속하게 알 수 있어야 합니다. 코드에서 이러한 위치를 식별하려면 오류가 발생할 수 있는 함수, 메서드, 이니셜라이저를 호출하는 코드 이전에 `try`나 `try?`나 `try!` 키워드를 작성합니다. 이 키워드는 아래에 설명되어 있습니다.

> **참고**
>
> Swift에서 오류 처리는 `try`, `catch`, `throw` 키워드를 사용하는 다른 언어에서 오류 처리와 유사합니다. Objective-C를 포함하여 많은 언어에서의 예외 처리와 달리 Swift에서 오류 처리는 계산 비용이 많이 드는 과정인 호출 스택 해제가 포함되지 않습니다. 따라서 `throw` 구문의 성능 특성은 `return` 구문의 성능 특성과 비슷합니다.

던지는 함수를 이용한 오류 전파 (Propagating Errors Using Throwing Functions)

오류가 발생할 수 있는 함수, 메서드, 이니셜라이저를 나타내기 위해 함수의 선언 중 매개변수 뒤에 `throws` 키워드를 작성합니다. `throws`로 표시된 함수는 던지는 함수라고 합니다. 함수의 반환 타입이 지정되어 있으면, `throws` 키워드는 반환 화살표(`->`) 전에 작성합니다.

```
func canThrowErrors() throws -> String

func cannotThrowErrors() -> String
```

던지는 함수는 내부에서 발생한 오류를 호출된 범위로 전파합니다.

> **참고**
>
> 던지는 함수는 오류를 전파만 할 수 있습니다. 던지기 선언이 되지 않은 함수 내에서 발생된 모든 오류는 함수 내에서 처리되어야 합니다.

아래 예시에서 `VendingMachine` 클래스는 요청된 항목이 불가능 하거나, 품절이거나, 현재 입금액을 초과하는 비용이 있는 경우, 적절한 `VendingMachineError`를 발생하는 `vend(itemNamed:)` 메서드를 가지고 있습니다:

```
struct Item {
    var price: Int
    var count: Int
}

class VendingMachine {
    var inventory = [
        "Candy Bar": Item(price: 12, count: 7),
        "Chips": Item(price: 10, count: 4),
        "Pretzels": Item(price: 7, count: 11)
    ]
    var coinsDeposited = 0

    func vend(itemNamed name: String) throws {
        guard let item = inventory[name] else {
            throw VendingMachineError.invalidSelection
        }

        guard item.count > 0 else {
```

```
            throw VendingMachineError.outOfStock
        }

        guard item.price <= coinsDeposited else {
            throw VendingMachineError.insufficientFunds(coinsNeeded: item.price - coinsDeposit
  ed)
        }

        coinsDeposited -= item.price

        var newItem = item
        newItem.count -= 1
        inventory[name] = newItem

        print("Dispensing \(name)")
    }
}
```

`vend(itemNamed:)` 메서드의 구현은 스낵 구매 요구사항 중 하나라도 충족하지 않으면 `guard` 구문을 사용하여 메서드를 일찍 종료시키고 적절한 오류를 발생합니다. `throw` 구문은 프로그램 제어를 즉시 이동하므로, 항목은 요구사항이 모두 만족해야만 판매됩니다.

`vend(itemNamed:)` 메서드는 발생하는 오류를 전파하기 때문에, 이 메서드를 호출하는 코드는 `do-catch` 구문, `try?`나 `try!`를 사용하여 오류를 처리하거나 계속 전파해야 합니다. 예를 들어 아래 예시에서 `buyFavoriteSnack(person:vendingMachine:)`은 던지는 함수이며, `vend(itemNamed:)` 메서드에서 발생한 오류는 `buyFavoriteSnack(person:vendingMachine:)` 함수가 호출된 지점까지 전파될 것입니다.

```
let favoriteSnacks = [
    "Alice": "Chips",
    "Bob": "Licorice",
    "Eve": "Pretzels",
]
func buyFavoriteSnack(person: String, vendingMachine: VendingMachine) throws {
    let snackName = favoriteSnacks[person] ?? "Candy Bar"
    try vendingMachine.vend(itemNamed: snackName)
}
```

이 예시에서 `buyFavoriteSnack(person:vendingMachine:)` 함수는 주어진 사람의 좋아하는 스낵을 검색하고 `vend(itemNamed:)` 메서드를 호출하여 그 제품을 구입합니다. `vend(itemNamed:)` 메서드는 오류를 발생할 수 있으므로, `try` 키워드를 앞에 두고 호출합니다.

던지는 이니셜라이저는 던지는 함수와 같은 방법으로 오류를 전파할 수 있습니다. 예를 들어 아래의 리스트에서 `PurchasedSnack` 구조체의 이니셜라이저는 초기화 과정에서 던지는 함수를 호출하고 발생하는 모든 오류를 호출자에게 전파하여 처리합니다.

```
struct PurchasedSnack {
    let name: String
    init(name: String, vendingMachine: VendingMachine) throws {
        try vendingMachine.vend(itemNamed: name)
        self.name = name
    }
}
```

Do-Catch 사용하여 오류 처리 (Handling Errors Using Do-Catch)

do-catch 구문은 코드의 블록을 실행하여 오류를 처리합니다. 오류가 do 절에서 발생되면, catch 절과 비교하여 오류를 처리할 수 있는 항목을 결정합니다.

다음은 do-catch 구문의 일반적인 형태입니다:

```
do {
    try <#expression#>
    <#statements#>
} catch <#pattern 1#> {
    <#statements#>
} catch <#pattern 2#> where <#condition#> {
    <#statements#>
} catch <#pattern 3#>, <#pattern 4#> where <#condition#> {
    <#statements#>
} catch {
    <#statements#>
}
```

처리할 수 있는 오류가 무엇인지 나타내기 위해 catch 뒤에 패턴을 작성합니다. catch 절이 패턴을 가지고 있지 않다면, 이 절은 모든 오류와 일치하고 error라는 이름을 가진 지역 상수로 오류를 바인딩합니다. 패턴 일치에 대한 자세한 내용은 패턴 (Patterns)을 참고바랍니다.

예를 들어 다음 코드는 VendingMachineError 열거형의 세 가지 케이스에 대해 일치합니다.

```
var vendingMachine = VendingMachine()
vendingMachine.coinsDeposited = 8
do {
    try buyFavoriteSnack(person: "Alice", vendingMachine: vendingMachine)
    print("Success! Yum.")
} catch VendingMachineError.invalidSelection {
    print("Invalid Selection.")
} catch VendingMachineError.outOfStock {
    print("Out of Stock.")
} catch VendingMachineError.insufficientFunds(let coinsNeeded) {
    print("Insufficient funds. Please insert an additional \(coinsNeeded) coins.")
} catch {
    print("Unexpected error: \(error).")
}
// Prints "Insufficient funds. Please insert an additional 2 coins.".
```

위의 예시에서 buyFavoriteSnack(person:vendingMachine:) 함수는 오류를 발생할 수 있으므로 try 표현식으로 호출됩니다. 오류가 발생하면, 실행이 즉시 catch 절로 전송되어 전파가 계속 될 것인지 여부를 결정합니다. 패턴이 일치하지 않으면, 오류는 마지막 catch 절에 의해 포착되고 지역 error 상수에 바인딩됩니다. 오류가 발생하지 않으면, do 구문의 나머지 구문이 실행됩니다.

catch 절은 do 절에서 발생할 수 있는 모든 오류를 처리할 필요는 없습니다. catch 절이 오류를 처리하지 않으면, 오류는 주변에 전파합니다. 그러나 전파된 오류는 주변 범위에서 처리되어야 합니다. 던지지 않는 함수에서는 do-catch 구문에서 오류를 처리해야 합니다. 던지는 함수에서는 do-catch 구문이나 호출자가 오류를 처리해야 합니다. 오류가 처리되지 않고 범위의 최상위로 전파되면 타임 오류를 발생합니다.

예를 들어 위의 예시는 VendingMachineError가 아닌 모든 오류가 호출 함수에서 포착되도록 작성할 수 있습니다:

```
func nourish(with item: String) throws {
    do {
        try vendingMachine.vend(itemNamed: item)
    } catch is VendingMachineError {
        print("Couldn't buy that from the vending machine.")
    }
}

do {
    try nourish(with: "Beet-Flavored Chips")
} catch {
    print("Unexpected non-vending-machine-related error: \(error)")
}
// Prints "Couldn't buy that from the vending machine.".
```

`nourish(with:)` 함수에서 `vend(itemNamed:)`가 `VendingMachineError` 열거형에 케이스 중 하나의 오류를 발생하면, `nourish(with:)`는 메세지를 출력하여 오류를 처리합니다. 그렇지 않으면 `nourish(with:)`는 호출 부분으로 오류를 전파합니다. 이 오류는 일반적인 `catch` 절에 의해 포착됩니다.

연관된 오류를 포착하기 위한 다른 방법은 콤마로 구분하여 `catch` 다음에 리스트 형식으로 작성하는 것입니다. 예를 들어:

```
func eat(item: String) throws {
    do {
        try vendingMachine.vend(itemNamed: item)
    } catch VendingMachineError.invalidSelection, VendingMachineError.insufficientFunds, Vendi
  ngMachineError.outOfStock {
        print("Invalid selection, out of stock, or not enough money.")
    }
}
```

`eat(item:)` 함수는 포착할 자동 판매기 오류를 나열하며, 오류 텍스트는 해당 리스트의 항목에 해당합니다. 이 세 가지 오류 중 어떤 오류가 발생하면 이 `catch` 절은 메세지를 출력하여 처리합니다. 나중에 추가될 수 있는 오류를 포함하여 다른 오류는 주변 범위로 전파됩니다.

오류를 옵셔널 값으로 변환 (Converting Errors to Optional Values)

오류를 옵셔널 값으로 변환하여 처리하기 위해 `try?`를 사용합니다. `try?` 표현식을 실행하는 동안 오류가 발생하면 이 표현식의 값은 `nil`입니다. 예를 들어 다음 코드에서 `x`와 `y`는 같은 값을 가지고 동작합니다:

```
func someThrowingFunction() throws -> Int {
    // ...
}

let x = try? someThrowingFunction()

let y: Int?
do {
    y = try someThrowingFunction()
} catch {
    y = nil
}
```

`someThrowingFunction()`이 오류를 발생하면, `x`와 `y`의 값은 `nil`입니다. 그렇지 않으면 `x`와 `y`의 값은 반환된 함수 값입니다. `x`와 `y`는 `someThrowingFunction()`이 반환하는 타입의 옵셔널 입니다. 여기서 함수는 정수를 반환하므로, `x`와 `y`는 옵셔널 정수입니다.

`try?`를 사용하면 모든 오류를 같은 방식으로 처리하려는 경우 간결한 오류 처리 코드를 작성할 수 있습니다. 예를 들어 다음 코드는 여러 접근방식을 사용하여 데이터를 가져오거나 모든 접근방식이 실패하면 `nil`을 반환합니다.

```
func fetchData() -> Data? {
    if let data = try? fetchDataFromDisk() { return data }
    if let data = try? fetchDataFromServer() { return data }
    return nil
}
```

오류 전파 비활성화 (Disabling Error Propagation)

가끔 던지는 함수나 메서드가 실제로 런타임 오류를 발생하지 않는다는 사실을 알고 있습니다. 이러한 경우 표현식 전에 오류 전파를 비활성화 하기 위해 `try!` 를 작성할 수 있고 오류를 발생하지 않는다고 호출을 래핑할 수 있습니다. 오류가 발생한다면 런타임 오류가 발생합니다.

예를 들어 다음의 코드는 주어진 경로의 이미지를 로드하거나 이미지를 로드할 수 없을 때는 오류를 발생하는 `loadImage(atPath:)` 함수를 사용합니다. 이러한 경우 이미지는 이미지는 애플리케이션과 함께 제공되고 런타임에 오류가 발생하지 않으므로 오류 전파를 비활성화 하는 것이 적절합니다.

```
let photo = try! loadImage(atPath: "./Resources/John Appleseed.jpg")
```

오류 타입 지정 (Specifying the Error Type)

위에 모든 예시에서 사용하는 오류 처리의 방법에서 오류는 `Error` 프로토콜을 준수하는 타입의 값입니다. 이러한 접근방식은 코드가 실행되는 동안 발생하는 모든 오류와 다른 곳에서 전파된 오류에 대해 미리 알 수 없습니다. 이것은 오류가 시간이 지남에 따라 변할 수 있음을 나타냅니다. 의존성을 사용하는 라이브러리를 포함하여 새로운 라이브러리는 새로운 오류를 던질 수 있고, 개발이나 테스트 중에 보여지지 않는 실패 모드를 나타낼 수 있습니다. 위의 예시에서 오류를 처리하는 코드는 특정 `catch` 구문이 없는 오류 처리를 위해 기본 케이스를 포함합니다.

대부분의 Swift 코드는 던지는 오류에 대해 타입을 지정하지 않습니다. 그러나, 다음과 같은 특별한 경우에 특정 타입의 오류로 제한할 수 있습니다:

- 동적 메모리 할당을 지원하지 않는 임베디드 시스템에서 코드를 실행하는 경우입니다. `any Error`나 다른 박싱 프로토콜 타입의 인스턴스는 오류를 저장하기 위해 런타임 때 메모리에 할당해야 합니다. 반대로, 특정 타입의 오류를 발생하려면 Swift는 오류에 대한 힙 할당을 피할 수 있습니다.
- 오류가 라이브러리 처럼 일부 코드를 상세히 구현하고, 인터페이스의 부분이 아닌 경우입니다. 오류는 라이브러리에서만 발생하고, 다른 의존성이나 라이브러리의 클라이언트에서 발생하지 않으므로, 가능한 모든 실패 목록을 만들 수 있습니다. 그리고 오류는 라이브러리의 세부 구현 내용이므로, 라이브러리 내에서 처리됩니다.
- 코드에서 클로저 인자를 가지고 클로저로 부터 오류를 전파하는 함수와 같이 제네릭 매개변수에 의해 전파되는 오류의 경우입니다. 특정 오류 타입을 전파하는 것과 `rethrows`를 사용하는 것에 대한 비교는 오류를 다시 던지는 함수와 메서드 (Rethrowing Functions and Methods)를 참고바랍니다.

예를 들어, 평점을 요약하고 다음의 오류 타입을 사용한다고 생각해 봅시다:

```
enum StatisticsError: Error {
    case noRatings
    case invalidRating(Int)
}
```

함수는 오류로 `StatisticsError` 값만 던지는 것을 지정하기 위해, 함수를 선언할 때 `throws` 대신에 `throws(StatisticsError)`를 작성합니다. 이 구문은 선언에서 `throws` 다음에 오류 타입을 작성하기 때문에 *타입이 지정된 던지기(typed throws)*라고 부릅니다. 예를 들어, 아래의 함수는 오류로 `StatisticsError` 값을 던집니다.

```
func summarize(_ ratings: [Int]) throws(StatisticsError) {
    guard !ratings.isEmpty else { throw .noRatings }

    var counts = [1: 0, 2: 0, 3: 0]
    for rating in ratings {
        guard rating > 0 && rating <= 3 else { throw .invalidRating(rating) }
        counts[rating]! += 1
    }

    print("*", counts[1]!, "-- **", counts[2]!, "-- ***", counts[3]!)
}
```

위 코드에서, `summarize(_:)` 함수는 1에서 3을 나타내는 평점의 목록을 요약합니다. 이 함수는 입력이 올바르지 않으면 `StatisticsError`의 인스턴스를 던집니다. 위 코드에서 함수의 오류 타입은 이미 정의되었으므로, 오류를 던질 때 타입을 생략합니다. 이와 같이 함수에서 오류가 발생하면 `throw StatisticsError.noRatings`로 작성하는 대신, `throw .noRatings`로 축약형으로 작성할 수 있습니다.

함수의 시작 부분에 오류 타입을 지정해서 작성하면, Swift는 다른 오류 타입을 던지는지 확인합니다. 예를 들어, 이 챕터의 위 예시에서 `summarize(_:)` 함수에 `VendingMachineError`를 사용하면, 코드는 컴파일 때 오류가 발생합니다.

일반적인 던지는 함수내에서 타입이 지정된 던지기를 사용하는 함수를 호출할 수 있습니다:

```
func someThrowingFunction() throws {
    let ratings = [1, 2, 3, 2, 2, 1]
    try summarize(ratings)
}
```

위 코드에서 `someThrowingFunction()`에서 오류 타입을 지정하지 않아, `any Error`를 던집니다. `throws(any Error)`로 명시적으로 오류 타입을 작성할 수도 있습니다; 아래 코드는 위 코드와 동일합니다:

```
func someThrowingFunction() throws(any Error) {
    let ratings = [1, 2, 3, 2, 2, 1]
    try summarize(ratings)
}
```

이 코드에서, `someThrowingFunction()`은 `summarize(_:)`에서 던지는 모든 오류를 전파합니다. `summarize(_:)`에서 발생하는 오류는 `StatisticsError` 값이며, `someThrowingFunction()`에 유효한 오류입니다.

`Never`의 반환 타입으로 반환하지 않는 함수를 작성할 수 있는 것처럼, 절대 오류를 던지지 않는 `throws(Never)`로 함수를 작성할 수 있습니다:

```
func nonThrowingFunction() throws(Never) {
  // ...
}
```

이 함수는 던지기 위해 `Never` 타입의 값을 생성할 수 없으므로, 오류를 던질 수 없습니다.

함수의 오류 타입을 지정하는 것 외에도, `do-catch` 구문에 오류 타입을 지정하여 작성할 수도 있습니다. 예를 들어:

```
let ratings = []
do throws(StatisticsError) {
    try summarize(ratings)
} catch {
    switch error {
    case .noRatings:
        print("No ratings available")
    case .invalidRating(let rating):
        print("Invalid rating: \(rating)")
    }
}
// Prints "No ratings available".
```

이 코드에서, `do throws(StatisticsError)`를 작성하는 것은 `do-catch` 구문에서 오류로 `StatisticsError` 값을 발생합니다. 다른 `do-catch` 구문과 같이, `catch` 구문은 가능한 모든 오류를 처리하거나 주변에 처리되지 않은 오류에 대해 전파합니다. 이 코드는 `switch` 구문을 사용하여 모든 오류를 처리합니다. 패턴을 가지지 않는 다른 `catch` 구문과 같이 이 구문은 모든 오류와 일치시키고 `error`라는 이름의 지역 상수에 바인딩합니다. `do-catch` 구문은 `StatisticsError` 값을 던지므로, `error`는 `StatisticsError` 타입의 값입니다.

위의 `catch` 구문은 가능한 오류를 일치시키고 처리하기 위해 `switch` 구문을 사용합니다. 오류 처리 코드없이 `StatisticsError`에 새로운 케이스를 추가하면, Swift는 `switch` 구문이 완전하기 않기 때문에 오류가 발생합니다. 자체 오류를 모두 처리하는 라이브러리의 경우에, 이러한 접근방식을 사용하여 새로운 오류를 처리하기 위한 새로운 코드를 가져오도록 할 수 있습니다.

함수나 `do` 블록에서 하나의 타입으로 오류를 던지면, Swift는 타입이 지정된 던지기를 사용하는 코드라고 추론합니다. 이 짧은 구문을 사용하면, 위 예시에서 `do-catch`를 아래와 같이 작성할 수 있습니다:

```
let ratings = []
do {
    try summarize(ratings)
} catch {
    switch error {
    case .noRatings:
        print("No ratings available")
    case .invalidRating(let rating):
        print("Invalid rating: \(rating)")
    }
}
// Prints "No ratings available".
```

위 `do-catch` 블록에서 오류 타입을 지정하지 않지만, Swift는 `StatisticsError`를 던진다고 추론합니다. Swift가 타입이 지정된 던지기로 추론하는 것을 방지하기 위해 `throws(any Error)`를 명시적으로 작성할 수 있습니다.

정리 작업 지정 (Specifying Cleanup Actions)

코드의 현재 블록이 종료되기 직전에 어떠한 작업을 수행하려면 `defer` 구문을 사용합니다. 이 구문을 사용하여 오류가 발생하여 종료되거나 `return`이나 `break`와 같은 구문에 의해 종료되는 방식에 상관없이 필요한 정리를 수행할 수 있습니다. 예를 들어 `defer` 구문을 사용하여 파일 설명자가 닫히고 수동으로 할당된 메모리가 해제되도록 할 수 있습니다.

`defer` 구문은 현재 범위가 종료될 때까지 실행을 연기합니다. 이 구문은 `defer` 키워드와 나중에 실행될 구문으로 구성되어 있습니다. 지연된 구문은 `break`이나 `return` 구문과 같이 구문의 밖으로 제어를 이동하거나 오류를 발생시키는 코드를 포함할 수 없습니다. 지연된 동작은 소스 코드에 작성된 순서와 반대로 실행됩니다. 즉 첫 번째 `defer` 구문의 코드는 마지막에 실행되고, 두 번째 `defer` 구문의 코드는 마지막에서 두 번째로 실행되는 식입니다. 소스 코드의 마지막 `defer` 구문은 가장 먼저 실행됩니다.

```
func processFile(filename: String) throws {
    if exists(filename) {
        let file = open(filename)
        defer {
            close(file)
        }
        while let line = try file.readline() {
            // Work with the file.
        }
        // close(file) is called here, at the end of the scope.
    }
}
```

위의 예시는 `open(_:)` 함수에 `close(_:)`에 대한 호출이 있는지 확인하기 위해 `defer` 구문을 사용합니다.

오류 처리 코드가 포함되어 있지 않아도 `defer` 구문을 사용할 수 있습니다. 더 자세한 내용은 연기된 동작 (Deferred Actions)을 참고바랍니다.

언어 가이드

동시성 (Concurrency)

비동기 동작을 수행합니다.

Swift는 구조화된 방식으로 비동기(asynchronous)와 병렬(parallel) 코드 작성을 지원합니다. *비동기 코드(Asynchronous code)*는 일시적으로 중단되었다가 다시 실행할 수 있지만 한 번에 프로그램의 한 부분만 실행됩니다. 프로그램에서 코드를 일시 중단하고 다시 실행하면 UI 업데이트와 같은 짧은 작업을 계속 진행하면서 네트워크를 통해 데이터를 가져오거나 파일을 분석하는 것과 같은 긴 실행 작업을 계속할 수 있습니다. *병렬 코드(Parallel code)*는 동시에 코드의 여러 부분이 실행됨을 의미합니다 -- 예를 들어 4코어 프로세서의 컴퓨터는 각 코어가 하나의 작업을 수행하므로 코드의 네 부분을 동시에 실행할 수 있습니다. 병렬과 비동기 코드를 사용하는 프로그램은 한 번에 여러 작업을 수행하고, 외부 시스템을 기다리는 작업은 일시 중단됩니다. 이 챕터의 나머지 부분에서는 동시성이라는 용어를 비동기 코드와 병렬 코드의 일반적인 조합을 가리키는 의미로 사용합니다.

병렬이나 비동기 코드의 추가적인 스케줄링 유연성에는 복잡성이 증가합니다. 동시성 코드를 작성할 때 어떤 코드가 동시에 실행될지 미리 알 수 없으며, 코드 실행 순서도 항상 알 수 없을 수 있습니다. 동시성 코드에서 흔히 발생하는 문제는 여러 코드가 일부 공유 가능한 가변 상태에 접근하려고 할 때 발생하고 - 이것을 *데이터 경쟁(data race)*이라고 합니다. 동시성을 지원하는 언어 수준을 사용하면, Swift는 데이터 경쟁을 감지하고 방지하며 대부분의 데이터 경쟁은 컴파일 시에 오류를 발생시킵니다. 일부 데이터 경쟁은 코드를 실행될 때까지 감지되지 않을 수 있습니다; 이러한 데이터 경쟁은 코드 실행을 종료시킵니다. 이 챕터에서 설명한대로 액터와 격리를 사용하면 데이터 경쟁으로부터 보호할 수 있습니다.

> **참고**
>
> 이전에 동시성 코드를 작성한 적이 있다면, 스레드 동작에 익숙할 것입니다. Swift의 동시성 모델은 스레드 위에 구축되어 있지만, 직접적으로 스레드를 다루지는 않습니다. Swift의 비동기 함수는 실행 중인 스레드를 포기할 수 있습니다. 그러면 첫 번째 함수가 차단되어 있는 동안 해당 스레드에서 다른 비동기 함수가 실행될 수 있습니다. 비동기 함수의 실행이 재개되면, Swift는 해당 함수가 어떤 스레드에서 실행될지 어떠한 보장도 하지 않습니다.

Swift의 언어 지원을 사용하지 않고, 동시성 코드를 작성할 수 있지만 해당 코드는 읽기가 더 어려운 경우가 많습니다. 예를 들어 다음 코드는 사진 이름 목록을 다운로드하고, 해당 목록의 첫 번째 사진을 다운로드하고, 해당 사진을 사용자에게 보여줍니다:

```
listPhotos(inGallery: "Summer Vacation") { photoNames in
    let sortedNames = photoNames.sorted()
    let name = sortedNames[0]
    downloadPhoto(named: name) { photo in
        show(photo)
    }
}
```

이 간단한 경우에도 완료 핸들러가 연속해서 작성되어야 하므로, 결국 중첩 클로저를 작성하게 됩니다. 이 스타일에서 더 많이 중첩된 복잡한 코드는 빠르게 파악하기 어려울 수 있습니다.

비동기 함수 정의와 호출 (Defining and Calling Asynchronous Functions)

*비동기 함수(asynchronous function)*나 *비동기 메서드(asynchronous method)*는 실행 도중에 일시적으로 중단될 수 있는 특수한 함수나 메서드 입니다. 이것은 완료될 때까지 실행되거나 오류가 발생하거나 반환되지 않는 일반적인 동기 함수(synchronous function)나 메서드(synchronous method)와 대조됩니다. 비동기 함수나 메서드도 이 세 가지 중 하나를 수행하지만 무언가를 기다리고 있을 때 중간에 일시 중지될 수도 있습니다. 비동기 함수나 메서드의 본문 내에서 실행을 일시 중지할 수 있는 부분을 표시합니다.

함수나 메서드가 비동기임을 나타내려면 던지는 함수(throwing function)를 나타내기 위해 `throws` 사용하는 것과 유사하게 매개변수 뒤의 선언에 `async` 키워드를 작성합니다. 함수나 메서드가 값을 반환한다면, 반환 화살표(`->`) 앞에 `async`를 작성합니다. 예를 들어 갤러리에 사진의 이름을 가져오는 방법은 아래와 같습니다:

```
func listPhotos(inGallery name: String) async -> [String] {
    let result = // ... some asynchronous networking code ...
    return result
}
```

비동기와 던지는 함수나 메서드는 `throws` 앞에 `async`를 작성합니다.

비동기 메서드를 호출할 때, 해당 메서드가 반환할 때까지 실행이 일시 중단됩니다. 중단될 가능성이 있는 지점을 표시하기 위해 호출 앞에 `await`를 작성합니다. 이것은 오류가 있는 경우 프로그램의 흐름이 변경 가능함을 나타내기 위해 던지는 함수를 호출할 때 `try`를 작성하는 것과 같습니다. 비동기 메서드 내에서 실행 흐름은 다른 비동기 메서드를 호출할 때만 일시 중단될 수 있습니다 -- 중단은 암시적이거나 선점적이지 않습니다 -- 이것은 가능한 모든 중단 지점이 `await`로 표시된다는 의미입니다. 코드에서 중단 가능한 모든 지점을 표시하면 동시성 코드를 읽기 쉽고 이해하기 쉽게 만들어 줍니다.

> **참고**
>
> **번역자 보충** `async let`이나 비동기 `defer`의 작업을 범위 종료 시 기다릴 때는 암시적인 중단 지점이 생길 수 있습니다. Swift 6.4의 비동기 정리 작업은 비동기 defer (Asynchronous Defer)를 참고바랍니다.

예를 들어 아래 코드는 갤러리에 모든 사진의 이름을 가져온 다음에 첫 번째 사진을 보여줍니다:

```
let photoNames = await listPhotos(inGallery: "Summer Vacation")
let sortedNames = photoNames.sorted()
let name = sortedNames[0]
let photo = await downloadPhoto(named: name)
show(photo)
```

`listPhotos(inGallery:)`와 `downloadPhoto(named:)` 함수 모두 네트워크 요청을 필요로 하기 때문에, 완료하는데 비교적 오랜 시간이 걸릴 수 있습니다. 반환 화살표 전에 `async`를 작성하여 둘 다 비동기로 만들면 이 코드는 사진이 준비될 때까지 기다리는 동안 앱의 나머지 코드가 계속 실행될 수 있습니다.

위 예시의 동시성을 이해하기 위한 실행 순서는 다음과 같습니다:

1. 코드는 첫 번째 줄에서 실행을 시작하고, 첫 번째 `await`까지 실행됩니다. `listPhotos(inGallery:)` 함수를 호출하고 해당 함수가 반환할 때까지 실행을 일시 중단합니다.
2. 이 코드의 실행이 일시 중단되는 동안 같은 프로그램의 다른 동시 코드가 실행됩니다. 예를 들어 오랜 시간 실행되는 백그라운드 태스크가 새로운 사진의 목록을 업데이트 할 수 있습니다. 이 코드는 `await`로 표시된 다음 중단 지점까지 실행되거나 코드의 마지막 부분까지 실행됩니다.

3 `listPhotos(inGallery:)`가 반환한 후에, 이 코드는 해당 지점에서 계속 실행됩니다. 반환된 값을 `photoNames`에 할당합니다.

4 `sortedNames`와 `name`을 정의하는 라인은 일반적인 동기 코드입니다. 이 라인은 `await`로 표시되지 않았으므로, 가능한 중단 지점이 없습니다.

5 다음 `await`는 `downloadPhoto(named:)` 함수에 대한 호출을 표시합니다. 이 코드는 해당 함수가 반환할 때까지 실행을 다시 일시 중단하여 다른 동시 코드에 실행할 기회를 제공합니다.

6 `downloadPhoto(named:)`가 반환한 후에 반환값은 `photo`에 할당되고 `show(_:)`를 호출할 때 인자로 전달합니다.

`await`로 표시된 코드의 중단이 가능한 지점은 비동기 함수나 메서드가 반환하기를 기다리는 동안 현재 코드 부분의 실행을 일시적으로 중단할 수 있음을 나타냅니다. Swift가 현재 스레드에서 코드의 실행을 일시 중단하고 대신 해당 스레드에서 다른 코드를 실행하기 때문에 이것을 *스레드 양보(yielding the thread)*라고도 부릅니다. `await`가 있는 코드는 실행을 일시 중단할 수 있어야 하므로, 프로그램의 특정 위치에서만 비동기 함수나 메서드를 호출할 수 있습니다:

- 비동기 함수, 메서드, 프로퍼티 본문의 코드
- `@main`으로 표시된 구조체, 클래스, 열거형의 정적(static) `main()` 메서드의 코드
- 아래의 비구조 동시성 (Unstructured Concurrency)에 보이는 것처럼 구조화되지 않은 하위 태스크의 코드

`Task.sleep(for:tolerance:clock:)` 메서드는 동시성 동작이 어떻게 동작하는지 알기 위해 간단한 코드를 작성할 때 유용합니다. 이 메서드는 주어진 시간만큼 현재 태스크를 중단합니다. 다음은 네트워크 동작을 지연시키기 위해 `sleep(for:tolerance:clock:)`을 사용하는 `listPhotos(inGallery:)` 함수 입니다.

```
func listPhotos(inGallery name: String) async throws -> [String] {
    try await Task.sleep(for: .seconds(2))
    return ["IMG001", "IMG99", "IMG0404"]
}
```

위 코드에서 `listPhotos(inGallery:)`은 `Task.sleep(for:tolerance:clock:)` 호출이 오류를 발생할 수 있으므로 비동기와 던지기를 모두 작성하였습니다. 이 `listPhotos(inGallery:)`를 호출하려면, `try`와 `await` 모두 작성해야 합니다:

```
let photos = try await listPhotos(inGallery: "A Rainy Weekend")
```

비동기 함수는 던지는 함수와 유사한 점이 있습니다: 비동기나 던지는 함수를 정의할 때, `async`나 `throws`를 표시하고, 이 함수를 호출할 때 `await`나 `try`를 작성합니다. 비동기 함수는 던지는 함수가 다른 던지는 함수를 호출할 수 있듯이 다른 비동기 함수를 호출할 수 있습니다.

그러나, 매우 중요한 다른 점이 있습니다. 오류를 처리하기 위해 `do-catch` 블록에 코드를 래핑하거나, 오류를 다른 곳에서 처리하기 위해 오류를 저장하는 `Result`를 사용할 수 있습니다. 이러한 접근방식은 던지지 않는 코드에서 던지는 함수를 호출할 수 있습니다. 예를 들어:

```
func availableRainyWeekendPhotos() -> Result<[String], Error> {
    return Result {
        try listDownloadedPhotos(inGallery: "A Rainy Weekend")
    }
}
```

반면에, 동기 코드에서 호출하고 결과를 기다리기 위해 비동기 코드를 래핑하는 방법은 없습니다. Swift 표준 라이브러리는 의도적으로 안전하지 않은 기능을 생략합니다 - 이를 구현하려고 하면 미묘한 경합, 스레드 이슈, 데드락과 같은 문제가 발생할 수 있습니다. 기존 프로젝트에 비동기 코드를 추가할 때, 상위 계층에서 시작하는 것이 좋습니다. 구체적으로 비동기를 사용할 코드의 최상위 계층부터 변환한 다음에, 호출하는 함수와 메서드를 하나씩 변환하면서 프로젝트의 아키텍처를 계층별로 작업합니다. 동기 코드는 비동기 코드를 호출할 수 있는 방법이 없으므로, 하위 계층(bottom-up)부터 접근하는 방식은 사용할 수 없습니다.

비동기 시퀀스 (Asynchronous Sequences)

이전 섹션의 `listPhotos(inGallery:)` 함수는 비동기적으로 배열의 모든 요소가 준비된 후에 전체 배열을 한 번에 반환합니다. 또 다른 접근방식은 *비동기 시퀀스(asynchronous sequence)*를 사용하여 한 번에 컬렉션의 한 요소를 기다리는 것입니다. 비동기 시퀀스에 대한 조회 동작은 다음과 같습니다:

```
import Foundation

let handle = FileHandle.standardInput
for try await line in handle.bytes.lines {
    print(line)
}
```

일반적인 `for-in` 루프 대신에 위의 예시는 `for` 다음에 `await`를 작성합니다. 비동기 함수나 메서드를 호출할 때와 마찬가지로 `await` 작성은 가능한 중단 지점을 나타냅니다. `for-await-in` 루프는 다음 요소를 사용할 수 있을 때까지 기다리고 각 반복이 시작될 때 잠재적으로 실행을 일시 중단합니다.

`Sequence` 프로토콜에 준수성을 추가하여 `for-in` 루프에서 자체 타입을 사용할 수 있는 것과 같은 방식으로 `AsyncSequence` 프로토콜에 준수성을 추가하여 `for-await-in` 루프에서 자체 타입을 사용할 수 있습니다.

병렬로 비동기 함수 호출 (Calling Asynchronous Functions in Parallel)

`await`를 사용하여 비동기 함수를 호출하면 한 번에 코드의 한 부분만 실행됩니다. 비동기 코드가 실행되는 동안 호출자는 코드의 다음 라인을 실행하기 위해 이동하기 전에 해당 코드가 완료될 때까지 기다립니다. 예를 들어 갤러리에서 처음 세 장의 사진을 가져오려면 다음과 같이 `downloadPhoto(named:)` 함수에 대한 세 번의 호출을 기다릴 수 있습니다:

```
let firstPhoto = await downloadPhoto(named: photoNames[0])
let secondPhoto = await downloadPhoto(named: photoNames[1])
let thirdPhoto = await downloadPhoto(named: photoNames[2])

let photos = [firstPhoto, secondPhoto, thirdPhoto]
show(photos)
```

이 방식에는 중요한 단점이 있습니다: 다운로드가 비동기이고 다운로드가 진행되는 동안 다른 작업을 수행할 수 있지만 `downloadPhoto(named:)`에 대한 호출은 한 번에 하나만 실행됩니다. 각 사진은 다음 사진이 다운로드를 시작하기 전에 완료됩니다. 그러나 이런 작업을 기다릴 필요가 없습니다 - 각 사진은 개별적으로 또는 동시에 다운로드 할 수 있습니다.

비동기 함수를 호출하고 주변의 코드를 병렬로 실행하려면 상수를 정의할 때 `let` 앞에 `async`를 작성하고 상수를 사용할 때마다 `await`를 작성합니다.

```
async let firstPhoto = downloadPhoto(named: photoNames[0])
async let secondPhoto = downloadPhoto(named: photoNames[1])
async let thirdPhoto = downloadPhoto(named: photoNames[2])

let photos = await [firstPhoto, secondPhoto, thirdPhoto]
show(photos)
```

이 예시에서 세 번의 `downloadPhoto(named:)` 호출은 이전 호출이 완료되길 기다리지 않고 시작됩니다. 사용할 수 있는 시스템 자원이 충분하다면 동시에 실행할 수 있습니다. 코드가 함수의 결과를 기다리기 위해 일시 중단되지 않기 때문에 이러한 함수 호출 중 어느 것도 `await`로 표시하지 않습니다. 대신 `photos`가 정의된 라인까지 실행이 계속됩니다 - 이 시점에서 프로그램은 이 비동기 호출의 결과를 필요로 하므로 세 장의 사진이 모두 다운로드 될 때까지 실행을 일시 중단하기 위해 `await`를 작성합니다.

다음은 이 두 접근방식의 차이점에 대해 생각할 수 있는 방법입니다:

- 다음 줄의 코드가 해당 함수의 결과에 따라 달라지면 `await`를 사용하여 비동기 함수를 호출합니다. 이것은 순차적으로 실행되는 작업을 생성합니다.
- 코드의 뒤쪽에 이르러서야 결과가 필요할 때 `async-let`을 사용하여 비동기 함수를 호출합니다. 이렇게 하면 병렬로 수행할 수 있는 작업이 생성됩니다.
- `await`와 `async-let`은 모두 일시 중단되는 동안 다른 코드를 실행할 수 있도록 합니다.
- 두 경우 모두 비동기 함수가 반환될 때까지 필요한 경우 실행이 일시 중단됨을 나타내기 위해 가능한 일시 중단 지점을 `await`로 표시합니다.

동일한 코드에서 이 두 가지 접근방식을 혼합할 수도 있습니다.

Task와 Task Group (Tasks and Task Groups)

*태스크(task)*는 프로그램의 일부로 비동기적으로 실행할 수 있는 작업 단위입니다. 모든 비동기 코드는 어떠한 태스크의 일부로 실행됩니다. 태스크는 한 번에 하나의 작업만 수행하지만, 여러 태스크를 생성하면, Swift는 동시에 수행하기 위해 태스크를 스케쥴링 할 수 있습니다.

이전 섹션에서 설명한 `async-let` 구문은 암시적으로 하위 태스크를 생성합니다 - 이 문법은 프로그램에서 무슨 태스크를 수행해야 될지 이미 알고 있을 때 잘 동작합니다. 우선 순위와 취소를 더 잘 제어할 수 있고 동적으로 태스크를 생성할 수 있는 태스크 그룹(task group) (`TaskGroup`의 인스턴스)과 해당 그룹에 하위 태스크를 추가할 수도 있습니다.

태스크는 계층 구조로 정렬됩니다. 주어진 태스크 그룹의 태스크는 동일한 상위 태스크를 가지고 각 태스크에는 하위 태스크가 있을 수도 있습니다. 태스크와 태스크 그룹 간의 명시적 관계 때문에 이 접근방식을 *구조 동시성(structured concurrency)*이라고 합니다. 명시적 부모(parent)-자식(child) 관계는 태스크 간에 여러 이점이 있습니다:

- 부모 태스크에서, 하위 태스크의 완료를 기다리는 일을 빠뜨릴 수 없습니다.
- 하위 태스크에서 더 높은 우선 순위로 설정되면, 상위 태스크의 우선 순위는 자동으로 높아집니다.
- 상위 태스크가 취소되면, 각 하위 태스크는 자동으로 취소됩니다.
- 태스크-로컬 값(Task-local value)은 하위 태스크에 효율적이고 자동으로 전파됩니다.

다음은 여러 사진을 다운로드하는 코드를 나타냅니다:

```
await withTaskGroup(of: Data.self) { group in
    let photoNames = await listPhotos(inGallery: "Summer Vacation")
    for name in photoNames {
        group.addTask {
            return await downloadPhoto(named: name)
        }
    }

    for await photo in group {
        show(photo)
    }
}
```

위 코드는 새로운 태스크 그룹을 생성하고, 갤러리에 사진을 다운로드하는 하위 태스크를 생성합니다. Swift는 조건이 되는만큼 비동기적으로 여러 태스크를 수행합니다. 하위 태스크에서 사진 다운로드가 끝나자마자 해당 사진은 보여집니다. 하위 태스크 완료에 대한 순서를 보장하지 않으므로, 갤러리에 사진은 무작위로 보여질 수 있습니다.

> **참고**
>
> 사진 다운로드 코드에서 오류가 발생할 수 있다면, `withThrowingTaskGroup(of:returning:body:)`을 호출해야 합니다.

위 코드에서 각 사진은 다운로드되고 보여지므로, 태스크 그룹은 결과를 반환하지 않습니다. 결과를 반환하는 태스크 그룹의 경우, `withTaskGroup(of:returning:body:)`에 전달하는 클로저 내에 결과를 누적하는 코드를 추가합니다.

```
let photos = await withTaskGroup(of: Data.self) { group in
    let photoNames = await listPhotos(inGallery: "Summer Vacation")
    for name in photoNames {
        group.addTask {
            return await downloadPhoto(named: name)
        }
    }

    var results: [Data] = []
    for await photo in group {
        results.append(photo)
    }

    return results
}
```

이전 예시와 같이, 이 예시는 각 사진을 다운로드하는 하위 태스크를 생성합니다. 이전 예시와 다르게, `for-await-in` 루프는 다음 하위 태스크가 완료될 때까지 기다리고, 해당 태스크의 결과를 결과 배열에 추가한 다음에, 모든 하위 태스크가 완료될 때까지 기다립니다. 마지막으로, 태스크 그룹은 다운로드한 사진의 배열을 전체 결과로 반환합니다.

태스크 취소 (Task Cancellation)

> **참고**
>
> **번역자 보충** 필수 정리 작업을 둘러싸는 태스크의 취소로부터 보호하는 API는 정리 작업의 취소 차단 (Cancellation Shields for Cleanup)을 참고바랍니다.

Swift 동시성은 협동 취소 모델(cooperative cancellation model)을 사용합니다. 각 태스크는 실행 중에 적절할 때 취소여부를 확인하고, 적절하게 취소에 응답합니다. 태스크의 종류에 따라 취소에 대한 응답은 다음 중 하나에 해당합니다:

- `CancellationError`와 같은 오류 발생
- `nil`이나 빈 컬렉션 반환
- 부분적으로 완료된 태스크 반환

사진이 크거나 네트워크가 느려서 사진을 다운로드하는데 오랜 시간이 걸릴 수 있습니다. 모든 태스크가 완료되기를 기다리지 않고 태스크를 멈추려면, 태스크가 취소되었는지 확인하고 취소된 경우에 실행을 중지합니다. 태스크가 취소되었는지 확인하는 방법은 두 가지가 있습니다: `Task.checkCancellation()` 타입 메서드를 호출하거나, `Task.isCancelled` 타입 프로퍼티를 읽어서 확인할 수 있습니다. `checkCancellation()`을 호출하는 것은 태스크가 취소되었으면 오류가 발생합니다; 던지는 태스크는 태스크 외부로 오류를 전파하여 모든 태스크를 중지시킬 수 있습니다. 이것은 구현과 이해가 간단하다는 이점이 있습니다. 더 유연한 방법으로는, 네트워크 연결을 닫고 임시 파일을 지우는 것과 같은 태스크 중지의 부분으로 정리 작업을 수행할 수 있는 `isCancelled` 프로퍼티를 사용합니다.

```
let photos = await withTaskGroup { group in
    let photoNames = await listPhotos(inGallery: "Summer Vacation")
    for name in photoNames {
        let added = group.addTaskUnlessCancelled {
            Task.isCancelled ? nil : await downloadPhoto(named: name)
        }
        guard added else { break }
    }

    var results: [Data] = []
    for await photo in group {
        if let photo { results.append(photo) }
    }
    return results
}
```

위의 코드는 이전 버전과 몇 가지 다른 점이 있습니다:

- 취소 후에 새로운 작업이 시작되지 않도록 각 태스크는 `TaskGroup.addTaskUnlessCancelled(priority:operation:)` 메서드를 사용해서 추가됩니다.
- `addTaskUnlessCancelled(priority:operation:)`을 호출할 때마다, 코드는 하위 태스크가 새로 추가되었는지 확인합니다. 그룹이 취소되면 `added`의 값은 `false`이고 - 코드는 추가 사진 다운로드를 중지합니다.
- 각 태스크는 사진을 다운로드 하기 전에 취소 여부를 검사합니다. 태스크가 취소 되었으면, `nil`을 반환합니다.
- 결국, 태스크 그룹은 결과를 수집할 때 `nil` 값이면 건너뜁니다. `nil`을 반환해서 취소를 처리한다는 것은 태스크 그룹은 부분적인 결과를 반환할 수 있다는 의미입니다 - 취소했을 때 이미 다운로드된 사진은 사진을 파기하는 대신에 다운로드된 사진을 반환합니다.

참고

태스크가 외부에서 취소되었는지 확인하려면, 타입 프로퍼티 대신에 `Task.isCancelled` 인스턴스 프로퍼티를 사용합니다.

즉시 취소에 대한 알림이 필요한 경우에 `withTaskCancellationHandler(operation:onCancel:isolation:)` 함수를 사용합니다. 예를 들어:

```
let task = Task {
    await withTaskCancellationHandler {
        // ...
    } onCancel: {
        print("Canceled!")
    }
}

// ... some time later...
task.cancel()  // Prints "Canceled!".
```

취소 처리를 사용할 때, 태스크 취소는 여전히 협조적입니다: 태스크가 완료될 때까지 수행하거나 취소를 확인하고 조기 중지합니다. 취소 처리가 시작될 때 태스크는 여전히 수행 중이므로, 경쟁 조건(race condition)이 생성될 수 있는 태스크와 취소 처리간의 상태 공유를 피해야 합니다.

비구조 동시성 (Unstructured Concurrency)

이전 섹션에서 설명한 동시성에 대한 구조화된 접근방식 외에도 Swift는 비구조 동시성(unstructured concurrency)을 지원합니다. 태스크 그룹의 일부인 태스크와 달리 *비구조 태스크(unstructured task)*에는 상위 태스크가 없습니다. 프로그램이 필요로 하는 방식으로 비구조 태스크를 관리할 수 있는 유연성이 있지만 올바른 동작을 위한 책임도 있습니다.

비구조 태스크를 생성하여 주변 코드와 유사하게 실행하려면, `Task.init(name:priority:operation:)` 이니셜라이저를 호출해야 합니다. 새로운 태스크는 기본적으로 현재 태스크와 동일한 액터 격리(actor isolation), 우선순위(priority), 태스크-로컬 상태(task-local state)를 가지고 실행됩니다. 주변 코드로부터 더 독립적인 비구조 태스크, 특히 *분리 태스크(detached task)*라고 알려진 것을 생성하려면 `Task.detached(name:priority:operation:)` 정적 메서드를 호출해야 합니다. 새로운 태스크는 기본적으로 어떠한 액터 격리없이 실행되고 현재 태스크의 우선순위나 태스크-로컬 상태를 상속받지 않습니다. 이 두 작업 모두 예를 들어 결과를 기다리거나 취소하는 것과 같이 상호작용이 가능한 태스크를 반환합니다.

```
let newPhoto = // ... some photo data ...
let handle = Task {
    return await add(newPhoto, toGalleryNamed: "Spring Adventures")
}
let result = await handle.value
```

분리 태스크(detached tasks) 관리에 대한 자세한 내용은 `Task`를 참고바랍니다.

격리 (Isolation)

이전 섹션에서는 동시성 작업을 분할하는 접근방식에 대해 논의했습니다. 이러한 작업은 앱의 UI와 같은 공유 데이터를 변경하는 것이 포함될 수 있습니다. 코드의 다른 부분이 동시에 동일한 데이터를 수정할 수 있다면, 이것은 데이터 경쟁(data race)를 초래할 위험이 있습니다. Swift는 코드를 데이터 경쟁으로부터 보호합니다: 데이터를 읽거나 수정할 때마다, Swift는 다른 코드가 동시에 해당 데이터를 수정하지 않도록 보장합니다. 이런 보장을 *데이터 격리(data isolation)*라고 합니다. 데이터를 격리하는 세 가지 주요 방법이 있습니다:

1. 불변 데이터(immutable data)는 항상 격리됩니다. 상수는 수정할 수 없으므로, 상수를 읽을 때 다른 코드에서 상수를 수정할 위험이 없습니다.

2 현재 태스크만 참조하는 데이터는 항상 격리됩니다. 지역 변수는 안전하게 읽고 쓸 수 있으며, 이것은 태스크 외부의 코드가 해당 메모리에 대한 참조를 가지고 있지 않아 다른 코드가 이 데이터를 수정할 수 없기 때문입니다. 또한 클로저에서 변수를 캡처하면, Swift는 해당 클로저가 동시에 사용되지 않도록 보장합니다.

3 액터에 의해 보호되는 데이터는 해당 데이터에 접근하는 코드가 액터에 격리된 경우 격리됩니다. 현재 함수가 액터에 격리되어 있다면, 해당 액터에 의해 보호되는 데이터를 안전하게 읽고 쓸 수 있으며, 이것은 동일한 액터에 격리된 다른 모든 코드가 실행되기 전에 자신의 차례를 기다려야 하기 때문입니다.

메인 액터 (The Main Actor)

액터는 코드가 해당 데이터에 번갈아 접근하도록 강제함으로써 가변 데이터에 대한 접근을 보호하는 객체입니다. 많은 프로그램에서 가장 중요한 액터는 *메인 액터(main actor)*입니다. 앱에서 메인 액터는 UI를 표시하는데 사용되는 모든 데이터를 보호합니다. 메인 액터는 UI 렌더링, UI 이벤트 처리, UI를 조회하거나 업데이트해야 하는 코드 실행을 번갈아 수행합니다.

코드에서 동시성을 사용하기 전에는 모든 것이 메인 액터에서 실행됩니다. 긴 실행이나 리소스 집약적인 코드를 식별하면 안전하고 올바른 방식으로 이 작업을 메인 액터에서 분리할 수 있습니다.

> **참고**
>
> 메인 액터는 메인 스레드와 밀접한 관련이 있지만 동일한 것은 아닙니다. 메인 액터는 비공개 가변 상태를 가지고, 메인 스레드는 해당 상태에 대한 접근을 직렬화합니다. 메인 액터에서 코드를 실행하면, Swift는 해당 코드를 메인 스레드에서 실행합니다. 이러한 연결 때문에 두 용어가 상호 교환적으로 사용되는 것을 볼 수 있습니다. 코드는 메인 액터와 상호작용하며; 메인 스레드는 더 낮은 수준의 구현 세부사항입니다.

메인 액터에서 작업을 실행하는 몇 가지 방법이 있습니다. 함수가 항상 메인 액터에서 실행되도록 하려면, `@MainActor` 속성으로 표시해야 합니다:

```
@MainActor
func show(_: Data) {
    // ... UI code to display the photo ...
}
```

위 코드에서 `show(_:)` 함수의 `@MainActor` 속성은 이 함수가 메인 액터에서만 실행되도록 요구합니다. 메인 액터에서 실행 중인 다른 코드 내에서 `show(_:)`를 동기 함수처럼 호출할 수 있습니다. 그러나 메인 액터에서 실행되지 않는 코드에서 `show(_:)`를 호출하려면 `await`를 포함하고 비동기 함수로 호출해야 합니다. 이것은 메인 액터로 전환하는 것이 잠재 중단 지점(potential suspension point)을 도입하기 때문입니다. 예를 들어:

```
func downloadAndShowPhoto(named name: String) async {
    let photo = await downloadPhoto(named: name)
    await show(photo)
}
```

위 코드에서 `downloadPhoto(named:)`와 `show(_:)` 함수는 모두 호출 시 중단될 수 있습니다. 이 코드는 또한 일반적인 패턴을 보여줍니다: 백그라운드에서 긴 실행과 CPU 집약적 작업을 수행한 다음에 UI를 업데이트하기 위해 메인 액터로 전환하는 것입니다. `downloadAndShowPhoto(named:)` 함수는 메인 액터에 있지 않으므로, `downloadPhoto(named:)`의 작업도 메인 액터에서 실행되지 않습니다. `show(_:)` 함수는 `@MainActor` 속성으로 표시되어 있으므로, UI 업데이트하는 `show(_:)`의 작업만 메인 액터에서 실행됩니다.

클로저가 메인 액터에서 실행되도록 하려면, 클로저 시작 부분에서 캡처 목록과 `in` 앞에 `@MainActor`를 작성합니다.

```
let photo = await downloadPhoto(named: "Trees at Sunrise")
Task { @MainActor in
    show(photo)
}
```

위 코드는 이전 코드 목록의 `downloadAndShowPhoto(named:)`와 유사하지만 이 예시의 코드는 UI 업데이트를 기다리지 않습니다. 또한 구조체, 클래스, 열거형에 `@MainActor`를 작성하여 모든 메서드와 프로퍼티에 대한 모든 접근이 메인 액터에서 실행되도록 할 수 있습니다:

```
@MainActor
struct PhotoGallery {
    var photoNames: [String]
    func drawUI() { /* ... other UI code ... */ }
}
```

위 코드의 `PhotoGallery` 구조체는 `photoNames` 프로퍼티의 이름을 사용하여 어떤 사진을 표시할지 결정하고 화면에 사진을 그립니다. `photoNames`가 UI에 영향을 미치므로, 이것을 변경하는 코드는 접근을 직렬화하기 위해 메인 액터에서 실행되어야 합니다.

프레임워크를 기반으로 구축하는 경우, 해당 프레임워크의 프로토콜과 기본 클래스는 일반적으로 이미 `@MainActor`로 표시되어 있으므로 이 경우에 자신의 타입에 `@MainActor`를 작성할 필요는 없습니다. 다음은 간단한 예시입니다:

```
@MainActor
protocol View { /* ... */ }

// Implicitly @MainActor
struct PhotoGalleryView: View { /* ... */ }
```

위 코드에서 SwiftUI와 같은 프레임워크는 `View` 프로토콜을 정의합니다. 프로토콜 선언에 `@MainActor`를 작성하여 이 프로토콜을 준수하는 `PhotoGalleryView`와 같은 타입도 암시적으로 `@MainActor`로 표시됩니다. `View`가 기본 클래스이고 `PhotoGalleryView`가 하위 클래스인 경우에도 동일한 동작을 볼 수 있습니다 - 하위 클래스는 암시적으로 `@MainActor`로 표시됩니다.

위 예시에서 `PhotoGallery`는 전체 구조체를 메인 액터에서 보호합니다. 더 세밀한 제어를 위해 메인 스레드에서 접근하거나 실행되어야 하는 프로퍼티나 메서드에만 `@MainActor`를 작성할 수 있습니다:

```
struct PhotoGallery {
    @MainActor var photoNames: [String]
    var hasCachedPhotos = false

    @MainActor func drawUI() { /* ... UI code ... */ }
    func cachePhotos() { /* ... networking code ... */ }
}
```

위 `PhotoGallery` 버전에서 `drawUI()` 메서드는 화면에 갤러리의 사진을 그리므로 메인 액터에 격리되어야 합니다. `photoNames` 프로퍼티는 UI를 직접 생성하지는 않지만 `drawUI()` 함수가 UI를 그리는데 사용하는 상태를 저장하므로 이 프로퍼티 또한 메인 액터에서만 접근되어야 합니다. 반면에 `hasCachedPhotos` 프로퍼티에 대한 변경은

UI와 상호작용하지 않고 `cachePhotos()` 메서드는 메인 액터에서 실행할 필요가 있는 상태에 접근하지 않습니다. 따라서 이것은 `@MainActor`로 표시되지 않습니다.

이전 예시와 마찬가지로 프레임워크를 사용하여 UI를 구축하는 경우에 해당 프레임워크의 프로퍼티 래퍼는 아마도 UI 상태 프로퍼티를 `@MainActor`로 표시할 것입니다. 프로퍼티 래퍼를 정의할 때 `wrappedValue` 프로퍼티가 `@MainActor`로 표시되면, 해당 프로퍼티 래퍼를 적용하는 모든 프로퍼티도 암시적으로 `@MainActor`로 표시됩니다.

액터 (Actors)

Swift는 메인 액터를 제공합니다 - 또한 자신만의 액터를 정의할 수도 있습니다. 액터(Actors)는 동시성 코드 간에 정보를 안전하게 공유할 수 있게 해줍니다.

클래스와 마찬가지로 액터(actors)는 참조 타입이므로 클래스는 참조 타입 (Classes Are Reference Types)에서 값 타입과 참조 타입의 비교는 클래스 뿐만 아니라 액터에도 적용됩니다. 클래스와 다르게 액터는 한 번에 하나의 태스크만 변경 가능한 상태에 접근할 수 있도록 허용하므로 여러 태스크의 코드가 액터의 동일한 인스턴스와 상호작용 하는 것은 안전합니다. 예를 들어 다음은 온도를 기록하는 액터 입니다:

```
actor TemperatureLogger {
    let label: String
    var measurements: [Int]
    private(set) var max: Int

    init(label: String, measurement: Int) {
        self.label = label
        self.measurements = [measurement]
        self.max = measurement
    }
}
```

`actor` 키워드를 사용하여 액터를 도입하고 중괄호로 정의합니다. `TemperatureLogger` 액터는 액터 외부의 다른 코드가 접근할 수 있는 프로퍼티가 있으며 액터 내부의 코드만 최대값을 업데이트 할 수 있게 `max` 프로퍼티를 제한합니다.

구조체와 클래스와 같은 이니셜라이저로 액터의 인스턴스를 생성합니다. 액터의 프로퍼티나 메서드에 접근할 때 일시 중단 지점을 나타내기 위해 `await`를 사용합니다. 예를 들어:

```
let logger = TemperatureLogger(label: "Outdoors", measurement: 25)
print(await logger.max)
// Prints "25".
```

이 예시에서 `logger.max`에 접근하는 것은 일시 중단이 발생할 수 있는 지점입니다. 액터는 한 번에 하나의 태스크만 변경 가능한 상태에 접근할 수 있도록 허용하므로 다른 태스크의 코드가 이미 로거와 상호작용하고 있는 경우 이 코드는 프로퍼티 접근을 기다리는 동안 일시 중단됩니다.

대조적으로 액터의 코드는 액터의 프로퍼티에 접근할 때 `await`를 작성하지 않습니다. 예를 들어 새로운 온도로 `TemperatureLogger`를 업데이트 하는 메서드입니다:

```
extension TemperatureLogger {
    func update(with measurement: Int) {
        measurements.append(measurement)
        if measurement > max {
            max = measurement
        }
    }
}
```

`update(with:)` 메서드는 액터에서 이미 실행 중이므로 `max`와 같은 프로퍼티에 대한 접근을 `await`로 표시하지 않습니다. 이 메서드는 액터가 변경 가능한 상태와 상호작용하기 위해 한 번에 하나의 태스크만 허용하는 이유 중 하나를 보여줍니다: 액터의 상태에 대한 일부 업데이트는 일시적으로 불변성을 깨뜨립니다. `TemperatureLogger` 액터는 온도 목록과 최대 온도를 추적하고 새로운 측정값을 기록할 때 최대 온도를 업데이트합니다. 업데이트 도중에 새로운 측정값을 추가한 후 `max`를 업데이트하지 않았다면 온도 로거는 일시적으로 일관성 없는 상태가 됩니다. 다른 태스크가 동일한 인스턴스에 접근하지 못하도록 방지하면 다음 문제를 예방할 수 있습니다:

1. 코드는 `update(with:)` 메서드를 호출합니다. 먼저 `measurements` 배열을 업데이트 합니다.
2. 코드에서 `max`를 업데이트 하기 전에 다른 코드에서 최대값과 온도 배열을 읽습니다.
3. 코드는 `max`를 변경하여 업데이트를 완료합니다.

이러한 경우 다른 곳에서 실행 중인 코드는 데이터가 일시적으로 잘못된 상태에 있었기 때문에 `update(with:)` 호출 중간에 액터에 대한 접근이 잘못된 정보를 읽습니다. Swift 액터는 한 번에 하나의 태스크만 해당 상태에 접근을 허용하고 해당 코드는 `await`로 표시된 일시 중단 지점 위치에서만 중단될 수 있기 때문에 이 문제를 방지할 수 있습니다. `update(with:)`는 일시 중단 지점을 포함하지 않으므로 다른 코드는 업데이트 중간에 데이터에 접근할 수 없습니다.

액터 외부의 코드에서 구조체나 클래스의 프로퍼티에 접근하는 것과 같이 프로퍼티에 직접적으로 접근하면 컴파일 때 오류가 발생합니다. 예를 들어:

```
print(logger.max)  // Error
```

`await` 작성 없이 `logger.max`에 접근하는 것은 액터의 프로퍼티가 해당 액터의 격리된 로컬 상태이기 때문에 실패합니다. 이 프로퍼티에 접근하는 코드는 액터의 부분으로 수행되어야 하고, 비동기 작업인 `await`를 작성해야 합니다. Swift는 액터에서 수행하는 코드만 액터의 로컬 상태에 접근할 수 있도록 보장합니다. 이 보장을 *액터 격리(actor isolation)*라고 합니다.

Swift 동시성 모델의 다음의 요소들은 공유된 변경 가능한 상태를 보다 쉽게 이해하고 안전하게 다룰 수 있도록 함께 작동합니다:

- 일시 중단 가능 지점 사이의 코드는 다른 동시 코드의 중단없이 순차적으로 수행됩니다. 그러나 여러 개의 동시성 코드는 동시에 실행될 수 있으므로, 다른 코드가 동시에 실행되고 있을 수 있습니다.
- 액터의 로컬 상태와 상호작용하는 코드는 해당 액터에서만 수행됩니다.
- 액터는 한 번에 하나의 코드만 수행합니다.

이러한 보장 때문에, `await`가 없는 액터 내부에 있는 코드는 프로그램의 다른 위치에서 일시적으로 유효하지 않은 상태를 관찰하지 않고 업데이트를 수행할 수 있습니다. 예를 들어, 아래의 코드는 측정된 온도를 화씨에서 섭씨로 변환합니다:

```
extension TemperatureLogger {
    func convertFahrenheitToCelsius() {
        for i in measurements.indices {
            measurements[i] = (measurements[i] - 32) * 5 / 9
        }
    }
}
```

위 코드는 측정된 온도를 한 번에 하나씩 변환합니다. map 작업이 진행되는 동안, 일부 온도는 화씨로 다른 온도는 섭씨로 표시됩니다. 그러나 이 코드는 `await`를 포함하지 않으므로, 메서드에서 일시 중단 지점이 없습니다. 이 메서드가 수정하는 상태는 액터에 속하며, 해당 코드가 액터에서 수행될 때를 제외하고는 코드를 읽거나 수정하지 못하도록 합니다. 이것은 온도 변환이 진행되는 동안 다른 코드가 부분적으로 변환된 온도를 읽을 수 없다는 것을 의미합니다.

임시 중단 지점을 생략해서 임시적으로 유효하지 않은 상태를 보호하는 액터의 코드를 작성하는 것 외에도 해당 코드를 동기 메서드로 이동할 수 있습니다. 위에 `convertFahrenheitToCelsius()` 메서드는 동기 메서드이므로, 절대 임시 중단 지점이 포함되지 않음을 보장합니다. 이 함수는 데이터 모델을 일시적으로 불일치하게 만드는 코드를 캡슐화하고, 작업이 완료되어 데이터 일관성을 복원하기 전까지 다른 코드를 수행할 수 없음을 읽기 쉽게 만들어 줍니다. 이 기간동안 Swift가 이 코드에서 프로그램의 다른 부분의 코드를 실행하도록 전환하지 않는 것이 중요합니다. 향후 이 함수에 비동기 코드를 추가하여, 임시 중단 지점을 도입하면 버그가 발생하는 대신에 컴파일 오류가 발생합니다.

전역 액터 (Global Actors)

메인 액터는 `MainActor` 타입의 전역 싱글턴 인스턴스입니다. 액터는 일반적으로 여러 인스턴스를 가질 수 있으며, 각 인스턴스는 독립적인 격리를 제공합니다. 이것이 액터의 모든 격리된 데이터를 해당 액터의 인스턴스 프로퍼티로 선언하는 이유입니다. 그러나 `MainActor`는 싱글턴이므로 - 이 타입의 단일 인스턴스만 존재 - 타입만으로 액터를 식별하기에 충분하기 때문에 속성으로만 메인 액터 격리를 표시할 수 있습니다. 이 접근방식은 가장 적합한 방식으로 코드를 구성할 수 있는 더 많은 유연성을 제공합니다.

`@globalActor` 속성을 사용하여 자신만의 싱글턴 전역 액터를 정의할 수 있으며, 이것은 globalActor에서 설명합니다.

Sendable 타입 (Sendable Types)

태스크(Task)와 액터(actor)는 프로그램을 동시에 안전하게 실행할 수 있는 조각으로 나눌 수 있습니다. 태스크나 액터의 인스턴스 내에서 변수와 프로퍼티와 같은 변경 가능한 상태를 포함하는 프로그램의 일부분을 *동시성 도메인(concurrency domain)*이라고 부릅니다. 어떤 데이터는 데이터가 변경 가능한 상태를 포함하지만 동시 접근에 대해 보호되지 않으므로 동시성 도메인 간에 공유될 수 없습니다.

한 동시성 도메인에서 다른 동시성 도메인으로 공유될 수 있는 타입을 Sendable 타입(sendable type)이라고 합니다. 예를 들어 액터 메서드를 호출할 때 인자로 전달하거나 태스크의 결과로 반환될 수 있습니다. 이 챕터의 앞부분에 있는 예시들은 동시성 도메인 간에 전달되는 데이터는 항상 안전한 간단한 값 타입을 사용하기 때문에 Sendable 타입에 대해 논의하지 않았습니다. 반대로 일부 타입은 동시성 도메인 간에 전달이 안전하지 않습니다. 예를 들어 변경 가능한 프로퍼티를 포함하고 해당 프로퍼티에 순차적으로 접근하지 않는 클래스는 서로 다른 태스크 간에 클래스의 인스턴스를 전달할 때 예상할 수 없고 잘못된 결과를 생성할 수 있습니다.

`Sendable` 프로토콜을 채택하여 Sendable 타입으로 표시합니다. 이 프로토콜은 어떠한 코드 요구사항을 가지지 않지만 Swift가 적용하는 의미론적 요구사항이 있습니다. 일반적으로 타입을 Sendable한 것으로 나타내기 위해 세 가지 방법이 있습니다:

- 타입은 값 타입이고 변경 가능한 상태가 다른 Sendable 데이터로 구성되어 있는 경우입니다 - 예를 들어 Sendable 저장 프로퍼티가 있는 구조체나 Sendable 연관값이 있는 열거형이 있습니다.
- 타입은 변경 가능한 상태가 없으며 불변 상태는 다른 Sendable 데이터로 구성되어 있는 경우입니다 - 예를 들어 읽기 전용 프로퍼티만 있는 구조체나 클래스가 있습니다.
- 타입이 `@MainActor`로 표시된 클래스나 특정 스레드나 큐에서 프로퍼티에 순차적으로 접근하는 클래스와 같이 변경 가능한 상태의 안전성을 보장하는 코드를 가지고 있는 경우입니다.

의미론적 요구사항의 자세한 목록은 Sendable 프로토콜을 참고바랍니다.

Sendable 프로퍼티만 가지는 구조체와 Sendable 연관값만 가지는 열거형과 같이 어떠한 타입은 항상 Sendable합니다. 예를 들어:

```
struct TemperatureReading: Sendable {
    var measurement: Int
}

extension TemperatureLogger {
    func addReading(from reading: TemperatureReading) {
        measurements.append(reading.measurement)
    }
}

let logger = TemperatureLogger(label: "Tea kettle", measurement: 85)
let reading = TemperatureReading(measurement: 45)
await logger.addReading(from: reading)
```

`TemperatureReading`은 Sendable 프로퍼티만 가지는 구조체이며 `public`이나 `@usableFromInline`으로 표시되지 않은 구조체이므로 암시적으로 Sendable합니다. 다음은 `Sendable` 프로토콜 준수가 암시되는 구조체입니다:

```
struct TemperatureReading {
    var measurement: Int
}
```

명시적으로 타입을 Sendable하지 않는 것으로 나타내려면 `Sendable`에 대한 사용 불가능한 준수를 작성해야 합니다:

```
struct FileDescriptor {
    let rawValue: Int
}

@available(*, unavailable)
extension FileDescriptor: Sendable {}
```

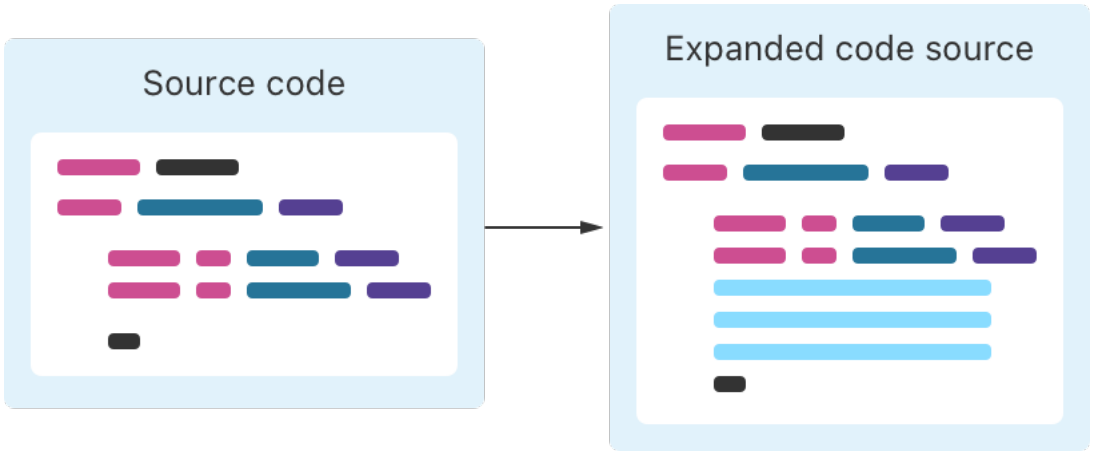
프로토콜 암시적 준수 (Implicit Conformance to a Protocol)에서 논의한대로 프로토콜에 대한 암시적 준수를 억제하는데 사용 불가능한 준수를 사용할 수도 있습니다.

언어 가이드

매크로 (Macros)

컴파일 때 코드를 생성하기 위해 매크로를 사용합니다.

매크로(Macro)는 소스 코드를 컴파일 할 때 변환하므로 반복적인 코드를 직접 작성하지 않아도 됩니다. 컴파일하는 동안 Swift는 코드를 빌드하기 전에 코드에 모든 매크로를 확장합니다.



매크로 확장은 항상 추가 작업입니다: 매크로는 새로운 코드를 추가하지만, 기존의 코드를 절대 삭제하거나 수정하지 않습니다.

> **참고**
>
> **번역자 주** 위의 영문 원본 설명에는 본문 매크로의 예외가 반영되어 있지 않습니다. attached의 `body` 역할은 함수나 접근자의 본문을 생성하거나 수정하거나 교체할 수 있습니다.

매크로 입력과 매크로 확장의 출력은 구문적으로 Swift 코드가 유효한지 확인합니다. 마찬가지로 매크로에 전달하는 값과 매크로에 의해 생성된 코드의 값이 올바른 타입인지 확인합니다. 추가적으로 매크로를 확장할 때 매크로의 구현에서 오류가 발생하면, 컴파일 오류가 발생합니다. 이러한 보장은 매크로를 사용하는 코드에 대해 더 쉽게 추론할 수 있으며, 매크로를 잘못 사용하거나 매크로 구현에 버그가 있는 경우 이러한 문제를 쉽게 알 수 있습니다.

> **참고**
>
> **번역자 주** 본문 매크로의 원래 함수 본문은 문법적으로 올바른 코드여야 하지만, 타입 검사는 매크로가 확장된 후에 수행됩니다. 모든 매크로 입력이 확장 전에 타입 검사를 받는다고 일반화하면 안 됩니다.

Swift는 두 종류의 매크로를 가지고 있습니다:

- *독립 매크로(Freestanding macro)*는 선언에 첨부되지 않고 자체적으로 나타납니다.
- *첨부 매크로(Attached macro)*는 매크로가 첨부된 선언을 수정합니다.

첨부 매크로와 독립 매크로는 약간 다르게 호출하지만, 모두 매크로 확장에 대해 동일한 모델을 따르고, 동일한 접근 방식을 사용하여 구현합니다. 다음 섹션에서 해당 두 종류 매크로에 대해 더 자세히 설명합니다.

독립 매크로 (Freestanding Macros)

독립 매크로(freestanding macro)를 호출하기 위해 이름 앞에 숫자 기호(#)를 작성하고 이름 뒤 소괄호 안에 매크로의 인자를 작성합니다. 예를 들어:

```
func myFunction() {
    print("Currently running \(#function)")
    #warning("Something's wrong")
}
```

첫 번째 줄에서 `#function`은 Swift 표준 라이브러리에 `function()`을 호출합니다. 이 코드를 컴파일 할 때, Swift는 `#function`을 현재 함수의 이름으로 대체하는 매크로의 구현을 호출합니다. 이 코드를 실행하고 `myFunction()`을 호출할 때, “Currently running myFunction()“을 출력합니다. 두 번째 줄에서 `#warning`은 Swift 표준 라이브러리에서 `warning(_:)` 매크로를 호출하여 커스텀 컴파일 경고를 생성합니다.

독립 매크로는 `#function`과 같이 값을 생성하거나 `#warning`과 같이 컴파일 때 동작을 수행할 수 있습니다.

첨부 매크로 (Attached Macros)

첨부 매크로(attached macro)를 호출하려면 매크로 이름 앞에 기호(@) 를 작성하고, 매크로 이름 뒤 소괄호에 인자를 작성합니다.

첨부 매크로는 첨부된 선언을 수정합니다. 새로운 메서드를 정의하거나 프로토콜의 준수성을 추가하는 것과 같이 해당 선언에 코드를 추가합니다.

예를 들어 매크로를 사용하지 않는 다음의 코드를 살펴봅시다:

```
struct SundaeToppings: OptionSet {
    let rawValue: Int
    static let nuts = SundaeToppings(rawValue: 1 << 0)
    static let cherry = SundaeToppings(rawValue: 1 << 1)
    static let fudge = SundaeToppings(rawValue: 1 << 2)
}
```

이 코드에서 `SundaeToppings` 옵션 셋의 각 옵션은 반복적이고 수동적인 이니셜라이저 호출을 포함합니다. 이러한 코드는 줄 끝에 잘못된 숫자를 입력하는 것과 같은 새로운 옵션을 추가할 때 실수하기 쉽습니다.

다음의 코드는 매크로를 사용한 버전입니다:

```
@OptionSet<Int>
struct SundaeToppings {
    private enum Options: Int {
        case nuts
        case cherry
        case fudge
    }
}
```

이 버전의 `SundaeToppings`는 `@OptionSet` 매크로를 호출합니다. 이 매크로는 private 열거형의 케이스 목록을 읽고, 각 옵션에 대한 상수 목록을 생성하고 `OptionSet` 프로토콜의 준수성을 추가합니다.

비교를 위해 `@OptionSet` 매크로의 확장 버전은 다음과 같습니다. 이 코드를 작성하지 않고 매크로의 확장을 보기 위해 Swift에 요청한 경우에만 볼 수 있습니다.

```
struct SundaeToppings {
    private enum Options: Int {
        case nuts
        case cherry
        case fudge
    }

    typealias RawValue = Int
    var rawValue: RawValue
    init() { self.rawValue = 0 }
    init(rawValue: RawValue) { self.rawValue = rawValue }
    static let nuts: Self = Self(rawValue: 1 << Options.nuts.rawValue)
    static let cherry: Self = Self(rawValue: 1 << Options.cherry.rawValue)
    static let fudge: Self = Self(rawValue: 1 << Options.fudge.rawValue)
}
extension SundaeToppings: OptionSet { }
```

private 열거형 뒤에 모든 코드는 `@OptionSet` 매크로에서 가져옵니다. static 변수를 생성하기 위해 매크로를 사용하는 `SundaeToppings`의 버전은 이전의 수동으로 된 코드보다 더 읽기 쉽고 유지보수 하기도 더 쉽습니다.

매크로 선언 (Macro Declarations)

대부분 Swift 코드에서 함수나 타입과 같은 기호를 구현할 때, 별도의 선언이 없습니다. 그러나, 매크로의 경우 선언과 구현은 분리되어 있습니다. 매크로의 선언은 매크로의 이름, 매크로가 가지는 매개변수, 어디서 사용될 수 있는지, 어떤 코드를 생성하는지가 포함됩니다. 매크로의 구현은 Swift 코드를 생성하여 매크로를 확장하는 코드를 포함합니다.

매크로는 `macro` 키워드로 선언합니다. 예를 들어 이전 예시에서 사용된 `@OptionSet` 매크로에 대한 선언의 부분입니다:

```
public macro OptionSet<RawType>() =
        #externalMacro(module: "SwiftMacros", type: "OptionSetMacro")
```

첫 번째 줄은 매크로의 이름과 매크로의 인자를 지정합니다 - 이름은 `OptionSet`이고 인자는 가지고 있지 않습니다. 두 번째 줄은 Swift 표준 라이브러리의 `externalMacro(module:type:)` 매크로를 사용하여 Swift 매크로의 구현 위치를 알려줍니다. 이 경우에 `SwiftMacros` 모듈은 `@OptionSet` 매크로를 구현하는 `OptionSetMacro`를 포함합니다.

`OptionSet`은 첨부 매크로이므로, 매크로의 이름은 구조체와 클래스 이름처럼 대문자 카멜 케이스로 사용합니다. 독립 매크로는 변수와 함수 이름처럼 소문자 카멜 케이스로 이름을 가집니다.

> **참고**
>
> 매크로는 항상 `public`으로 선언됩니다. 매크로를 선언하는 코드는 매크로를 사용하는 코드의 모듈과 다르므로, public이 아닌 매크로는 적용할 수 없습니다.

매크로 선언은 매크로의 역할을 정의합니다 - 매크로가 호출될 수 있는 코드의 위치와 매크로가 생성할 수 있는 코드의 종류를 나타냅니다. 모든 매크로는 하나 이상의 역할을 가지고 있고, 매크로 선언의 앞에 속성으로 작성합니다. 다음은 `@OptionSet`에 대한 역할의 속성을 포함한 선언의 부분을 나타냅니다:

```
@attached(member)
@attached(extension, conformances: OptionSet)
public macro OptionSet<RawType>() =
        #externalMacro(module: "SwiftMacros", type: "OptionSetMacro")
```

이 선언에서 `@attached` 속성은 매크로 역할마다 한 번씩, 총 두 번 나타납니다. 첫 번째는 `@attached(member)`를 사용하고 매크로가 적용된 타입에 새로운 멤버를 추가한다고 나타냅니다. `@OptionSet` 매크로는 `OptionSet` 프로토콜에 의해 요구되는 `init(rawValue:)` 이니셜라이저와 멤버를 추가합니다. 두 번째는 `@attached(extension, conformances: OptionSet)`를 사용하고 `@OptionSet`이 `OptionSet` 프로토콜의 준수성을 추가한다고 나타냅니다. `@OptionSet` 매크로는 매크로를 적용한 타입을 확장하여 `OptionSet` 프로토콜의 준수성을 추가합니다.

독립 매크로에 대해 매크로의 역할을 지정하기 위해 `@freestanding` 속성을 작성합니다:

```
@freestanding(expression)
public macro line<T: ExpressibleByIntegerLiteral>() -> T =
        /* ... location of the macro implementation... */
```

위의 `#line` 매크로는 `expression` 역할을 가집니다. 매크로 표현식은 값을 생성하거나, 컴파일 때 경고를 생성하듯이 어떠한 동작을 수행합니다.

매크로의 역할 외에도 매크로의 선언은 매크로가 생성하는 기호의 이름에 대한 정보를 제공합니다. 매크로 선언이 이름을 제공하면, 해당 이름을 사용하는 선언만 생성하므로, 생성된 코드는 이해하고 디버그 하는데 도움을 줍니다. 다음은 `@OptionSet` 선언의 전체입니다:

```
@attached(member, names: named(RawValue), named(rawValue),
        named(`init`), arbitrary)
@attached(extension, conformances: OptionSet)
public macro OptionSet<RawType>() =
        #externalMacro(module: "SwiftMacros", type: "OptionSetMacro")
```

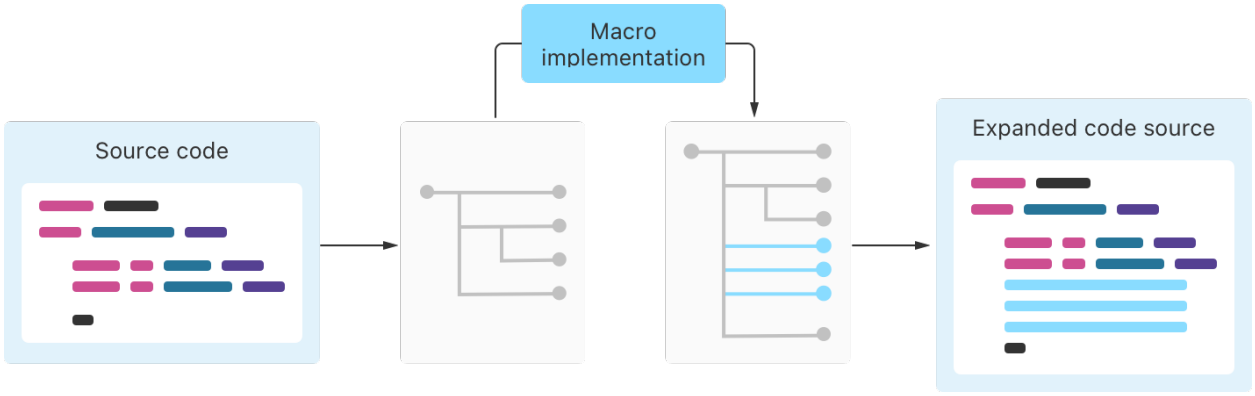
위의 선언에서 `@attached(member)` 매크로는 `@OptionSet` 매크로가 생성하는 각 기호에 대해 `names:` 레이블 뒤에 인자로 포함합니다. 매크로는 `RawValue`, `rawValue`, `init`이라는 이름의 기호에 대한 선언을 추가합니다 - 이러한 이름은 미리 알고 있기 때문에, 명시적으로 매크로 선언에 나열합니다.

매크로 선언은 매크로를 사용할 때까지 알 수 없는 이름의 선언을 생성하기위해 이름 뒤에 `arbitrary`도 포함합니다. 예를 들어 `@OptionSet` 매크로가 위의 `SundaeToppings`에 적용하면, 열거형 케이스 인 `nuts`, `cherry`, `fudge`에 해당하는 타입 프로퍼티를 생성합니다.

매크로 역할의 종류와 더 자세한 내용은 속성 (Attributes)의 attached와 freestanding을 참고바랍니다.

매크로 확장 (Macro Expansion)

매크로를 사용하는 Swift 코드를 빌드할 때, 컴파일러는 매크로의 구현을 확장하기 위해 호출합니다.



특히, Swift는 아래와 같은 방식으로 매크로를 확장합니다:

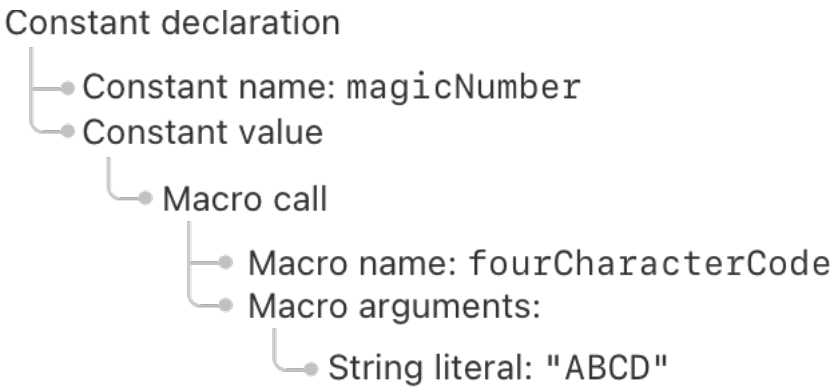
1. 컴파일러는 코드를 읽고, 구문의 메모리 표현을 생성합니다.
2. 컴파일러는 메모리 표현의 일부분을 매크로 구현에 전송하여 매크로를 확장합니다.
3. 컴파일러는 확장된 형태로 매크로 호출을 대체합니다.
4. 컴파일러는 확장된 소스 코드를 사용하여 완료될 때까지 계속 진행합니다.

특정 단계를 진행하려면, 다음을 참고 바랍니다:

```
let magicNumber = #fourCharacterCode("ABCD")
```

`#fourCharacterCode` 매크로는 네 글자의 문자열을 가지고 문자열의 ASCII 값에 해당하는 부호 없는 32비트 정수를 반환합니다. 일부 파일 형식은 이러한 정수가 크기가 작으면서 디버거에서도 읽기 쉽기 때문에, 데이터를 식별하기 위해 사용합니다. 아래 매크로 구현 (Implementing a Macro) 섹션에서는 이러한 매크로를 어떻게 구현하는지 보여줍니다.

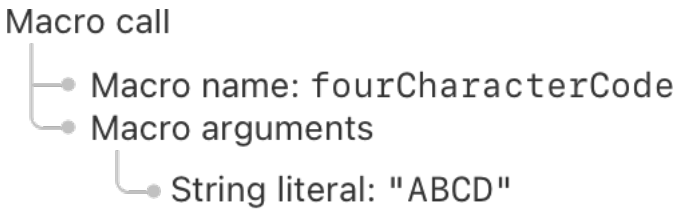
위 코드에서 매크로를 확장하기 위해, 컴파일러는 Swift 파일을 읽고 *추상 구문 트리(abstract syntax tree)*이나 AST라고 알려진 해당 코드의 메모리 표현을 생성합니다. AST는 해당 구조와 상호작용하는 코드를 더 쉽게 작성하기 위해 코드의 구조를 명시적으로 만듭니다 - 이것은 컴파일러나 매크로 구현과 비슷합니다. 다음은 일부 상세정보를 단순화한 위 코드에 대한 AST 표현입니다.



위 다이어그램은 이 코드의 구조가 메모리에서 어떻게 표현되는지 보여줍니다. AST의 각 요소는 소스 코드의 부분에 해당합니다. “상수 선언(Constant declaration)” AST 요소는 상수 선언의 두 부분을 표현하는 두 개의 하위 요소를 가지고 있습니다: 두 부분은 상수의 이름과 상수의 값입니다. “매크로 호출(Macro call)” 요소는 매크로의 이름과 매크로에 전달되는 인자의 목록을 표현하는 하위 요소를 가지고 있습니다.

이 AST의 구성 부분으로, 컴파일러는 소스 코드가 Swift에 유효한지 확인합니다. 예를 들어 `#fourCharacterCode`는 문자열이어야 한다는 하나의 인자를 가집니다. 정수 인자를 전달하거나 문자열 리터럴 끝에 쌍따옴표(")를 빼먹으면, 오류가 발생합니다.

컴파일러는 매크로를 호출하는 위치를 코드에서 찾고, 해당 매크로를 구현한 외부 바이너리를 로드합니다. 각 매크로 호출에 대해 컴파일러는 AST의 부분을 매크로의 구현에 전달합니다. 다음은 AST의 해당 부분을 표현합니다:



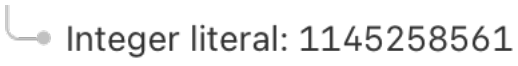
`#fourCharacterCode` 매크로의 구현은 매크로를 확장할 때 입력으로 이 부분의 AST를 읽습니다. 매크로의 구현은 입력으로 받은 AST에서만 동작합니다. 이 의미는 매크로는 앞, 뒤에 오는 코드에 관계없이 항상 같은 방식으로 확장합니다. 이 제한은 Swift가 변경되지 않은 매크로 확장을 피할 수 있으므로, 매크로 확장을 더 쉽게 이해하도록 돕고, 코드 빌드 속도에 도움을 줍니다.

Swift는 매크로를 구현한 코드를 제한하여 매크로 작성자가 실수로 다른 입력을 읽는 것을 방지합니다:

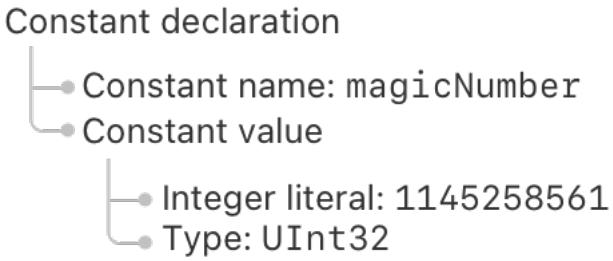
- 매크로 구현에 전달된 AST는 매크로를 표현하는 AST 요소만 포함하고 앞이나 뒤에 오는 코드를 포함하지 않습니다.
- 매크로 구현은 파일 시스템이나 네트워크 접근을 방지하는 샌드박스 환경(sandboxed environment)에서 실행됩니다.

이러한 안전장치 외에도, 매크로의 작성자는 매크로의 입력 외에 항목을 읽거나 수정하지 않을 책임이 있습니다. 예를 들어 매크로의 확장은 현재 시간에 의존하지 않아야 합니다.

`#fourCharacterCode`의 구현은 확장된 코드를 포함하는 새로운 AST를 생성합니다. 다음은 코드가 컴파일러에 무엇을 반환하는지 나타냅니다:



컴파일러가 이 확장을 받으면, 매크로 호출을 포함하는 AST 요소를 매크로의 확장이 포함된 요소로 대체합니다. 매크로 확장 후에, 컴파일러는 프로그램이 여전히 Swift에 유효하고 모든 타입이 올바른지 다시 확인합니다. 그러면 컴파일 될 수 있는 최종 AST 가 생성됩니다.



이 AST는 다음과 같은 Swift 코드에 해당합니다:

```
let magicNumber = 1145258561 as UInt32
```

이 예시에서 입력 소스 코드는 하나의 매크로만 가지지만, 실제 프로그램에서는 동일한 매크로의 여러 인스턴스와 다른 매크로에 대한 여러 호출이 있을 수 있습니다. 컴파일러는 한 번에 하나씩 매크로를 확장합니다.

하나의 매크로가 다른 매크로 안에서 나타나면, 외부 매크로가 먼저 확장됩니다 - 이렇게 되면 확장되기 전에 외부 매크로는 내부 매크로를 수정할 수 있습니다.

매크로 구현 (Implementing a Macro)

매크로를 구현하기 위해, 두 개의 구성요소를 만듭니다: 매크로 확장을 수행하는 타입과 API로 노출하도록 매크로를 선언한 라이브러리 입니다. 매크로 구현은 매크로의 클라이언트 빌드의 부분으로 수행되기 때문에, 매크로와 해당 클라이언트를 함께 개발하는 경우에도 이러한 부분은 매크로를 사용하는 코드와 별개로 빌드됩니다.

Swift Package Manager를 사용하여 새로운 매크로를 생성하기 위해, `swift package init --type macro`를 수행합니다 - 이것은 매크로 구현과 선언에 대한 템플릿을 포함하여 몇 개의 파일을 생성합니다.

기존 프로젝트에 매크로를 추가하기 위해, 다음과 같이 `Package.swift` 파일의 시작 부분을 수정합니다:

- `swift-tools-version`의 Swift tools 버전을 5.9이상으로 지정합니다.
- `CompilerPluginSupport` 모듈을 가져옵니다.
- `platforms` 목록에 최소 배포 타겟으로 macOS 10.15를 포함합니다.

아래의 코드는 `Package.swift` 파일의 예시를 보여줍니다.

```
// swift-tools-version: 5.9

import PackageDescription
import CompilerPluginSupport

let package = Package(
    name: "MyPackage",
    platforms: [ .iOS(.v17), .macOS(.v13)],
    // ...
)
```

다음으로 기존 `Package.swift` 파일에 매크로 구현에 대한 타겟과 매크로 라이브러리에 대한 타겟을 추가합니다. 예를 들어, 해당 프로젝트와 일치하는 이름으로 변경하여 다음과 같이 추가할 수 있습니다:

```
targets: [
    // Macro implementation that performs the source transformations.
    .macro(
        name: "MyProjectMacros",
        dependencies: [
            .product(name: "SwiftSyntaxMacros", package: "swift-syntax"),
            .product(name: "SwiftCompilerPlugin", package: "swift-syntax")
        ]
    ),

    // Library that exposes a macro as part of its API.
    .target(name: "MyProject", dependencies: ["MyProjectMacros"]),
]
```

위 코드는 두 개의 타겟을 정의합니다: `MyProjectMacros`는 매크로의 구현을 포함하고, `MyProject`는 해당 매크로를 사용 가능하도록 만듭니다.

매크로의 구현은 AST를 사용하여 구조화된 방식으로 Swift 코드와 상호작용하기 위해 SwiftSyntax 모듈을 사용합니다. Swift Package Manager로 새로운 매크로 패키지를 생성하면, 생성된 `Package.swift` 파일은 자동으로

SwiftSyntax에 대한 의존성을 추가합니다. 기존 프로젝트에 매크로를 추가하면, `Package.swift` 파일에 SwiftSyntax에 대한 의존성을 추가합니다:

```
dependencies: [
    .package(url: "https://github.com/swiftlang/swift-syntax", from: "509.0.0")
],
```

매크로의 역할에 따라, 매크로 구현이 준수해야하는 SwiftSyntax에 적절한 프로토콜이 있습니다. 예를 들어 이전 섹션에 `#fourCharacterCode`를 생각해 봅시다. 다음은 해당 매크로를 구현하는 구조체입니다:

```
import SwiftSyntax
import SwiftSyntaxMacros

public struct FourCharacterCode: ExpressionMacro {
    public static func expansion(
        of node: some FreestandingMacroExpansionSyntax,
        in context: some MacroExpansionContext
    ) throws -> ExprSyntax {
        guard let argument = node.argumentList.first?.expression,
              let segments = argument.as(StringLiteralExprSyntax.self)?.segments,
              segments.count == 1,
              case .stringSegment(let literalSegment)? = segments.first
        else {
            throw CustomError.message("Need a static string")
        }

        let string = literalSegment.content.text
        guard let result = fourCharacterCode(for: string) else {
            throw CustomError.message("Invalid four-character code")
        }

        return "\(raw: result) as UInt32"
    }
}

private func fourCharacterCode(for characters: String) -> UInt32? {
    guard characters.count == 4 else { return nil }

    var result: UInt32 = 0
    for character in characters {
        result = result << 8
        guard let asciiValue = character.asciiValue else { return nil }
        result += UInt32(asciiValue)
    }
    return result
}
enum CustomError: Error { case message(String) }
```

이 매크로를 기존에 Swift Package Manager 프로젝트에 추가하는 경우에, 매크로 타겟에 대한 시작 지점으로 역할하고 타겟을 정의하는 매크로 목록의 타입을 추가합니다:

```
import SwiftCompilerPlugin

@main
struct MyProjectMacros: CompilerPlugin {
    var providingMacros: [Macro.Type] = [FourCharacterCode.self]
}
```

`#fourCharacterCode` 매크로는 표현식을 생성하는 독립 매크로이므로, 구현한 `FourCharacterCode` 타입은 `ExpressionMacro` 프로토콜을 준수합니다. `ExpressionMacro` 프로토콜은 AST를 확장하는 `expansion(of:in:)` 메서드인 하나의 요구사항을 가집니다. 매크로 역할과 해당 SwiftSyntax 프로토콜의 목록은 속성 (Attributes)의 attached와 freestanding을 참고바랍니다.

`#fourCharacterCode` 매크로를 확장하기 위해, Swift는 이 매크로를 사용하는 코드에 대한 AST를 매크로 구현을 포함하는 라이브러리로 보냅니다. 라이브러리에서 Swift는 `FourCharacterCode.expansion(of:in:)`을 호출하고, AST와 컨텍스트를 인자로 메서드에 전달합니다. `expansion(of:in:)`의 구현은 `#fourCharacterCode`에 인자로 전달된 문자열을 찾고, 부호 없는 32-bit 정수 리터럴 값으로 계산합니다.

위 예시에서 첫 번째 `guard` 블록은 AST에서 문자열 리터럴을 추출하고, `literalSegment`에 해당 AST 요소를 할당합니다. 두 번째 `guard` 블록에서 private `fourCharacterCode(for:)` 함수를 호출합니다. 해당 블록들은 매크로가 올바르게 사용되지 않으면 오류가 발생합니다 - 오류 메세지는 잘못된 호출 부분에서 컴파일 오류가 나타납니다. 예를 들어 `#fourCharacterCode("AB" + "CD")`로 매크로를 호출하면 컴파일러는 “Need a static string.“이라는 오류가 발생합니다.

`expansion(of:in:)` 메서드는 AST에서 표현식을 표현하는 SwiftSyntax의 타입인 `ExprSyntax`의 인스턴스를 반환합니다. 이 타입은 `StringLiteralConvertible` 프로토콜을 준수하기 때문에, 매크로 구현은 결과를 생성하기 위해 가벼운 구문(lightweight syntax)으로 문자열 리터럴을 사용합니다. 매크로 구현에서 반환하는 모든 SwiftSyntax 타입은 `StringLiteralConvertible`을 준수하므로, 모든 종류의 매크로를 구현할 때 이 접근방식을 사용할 수 있습니다.

매크로 개발과 디버깅 (Developing and Debugging Macros)

매크로는 테스트를 사용하는 개발에 적합합니다: 외부상태에 의존하지 않고 외부상태에 변경없이 하나의 AST를 다른 AST로 변환합니다. 추가로 테스트에 입력 설정을 단순화하는 문자열 리터럴에서 구문 노드를 생성할 수 있습니다. 예상되는 값과 비교하기 위해 문자열을 가져오는 AST의 `description` 프로퍼티를 읽을 수도 있습니다. 예를 들어 다음은 이전 섹션의 `#fourCharacterCode` 매크로의 테스트입니다:

```
let source: SourceFileSyntax =
    """
    let abcd = #fourCharacterCode("ABCD")
    """

let file = BasicMacroExpansionContext.KnownSourceFile(
    moduleName: "MyModule",
    fullFilePath: "test.swift"
)

let context = BasicMacroExpansionContext(sourceFiles: [source: file])

let transformedSF = source.expand(
    macros:["fourCharacterCode": FourCharacterCode.self],
    in: context
)
```

```
let expectedDescription =
    """
    let abcd = 1145258561 as UInt32
    """

precondition(transformedSF.description == expectedDescription)
```

위 예시는 사전 조건을 사용하여 매크로를 테스트 하지만, 테스트 프레임워크를 사용할 수도 있습니다.

언어 가이드

타입 캐스팅 (Type Casting)

값의 런타임 타입을 확인하고, 더 구체적인 타입의 정보를 제공합니다.

*타입 캐스팅(Type casting)*은 인스턴스의 타입을 확인하거나 해당 인스턴스를 자신의 클래스 계층 구조의 다른 상위 클래스나 하위 클래스로 취급하는 방법입니다.

Swift에서 타입 캐스팅은 `is`와 `as` 연산자로 구현합니다. 이 두 연산자는 값의 타입을 확인하거나 값을 다른 타입으로 변환하는 간단하고 효과적인 방법을 제공합니다.

프로토콜 준수 여부 검사 (Checking for Protocol Conformance)에서 설명한대로 타입 캐스팅을 사용하여 타입이 프로토콜을 준수하는지 확인할 수도 있습니다.

타입 캐스팅을 위한 클래스 계층 정의 (Defining a Class Hierarchy for Type Casting)

클래스와 하위 클래스의 계층도와 함께 타입 캐스팅을 사용하여 특정 클래스 인스턴스의 타입을 확인하고 같은 계층도 내에서 다른 클래스로 인스턴스를 변환할 수 있습니다. 아래의 세 코드는 타입 캐스팅의 예시에서 사용하기 위해 클래스의 계층도와 해당 클래스의 인스턴스를 포함하는 배열을 정의합니다.

첫 번째 코드는 `MediaItem`이라는 새로운 기본 클래스를 정의합니다. 이 클래스는 디지털 미디어 라이브러리에 나타나는 모든 종류의 항목에 대한 기본 기능을 제공합니다. 특히 `String` 타입의 `name` 프로퍼티와 `init(name:)` 이니셜라이저를 선언합니다. (영화와 노래를 포함하여 모든 미디어 항목은 이름을 가지고 있다고 가정합니다.)

```
class MediaItem {
    var name: String
    init(name: String) {
        self.name = name
    }
}
```

다음 코드는 `MediaItem`의 두 하위 클래스를 정의합니다. 첫 번째 하위 클래스 `Movie`는 영화나 필름에 대한 추가 정보를 캡슐화합니다. 기본 `MediaItem` 클래스의 상위에 `director` 프로퍼티와 해당 이니셜라이저를 추가합니다. 두 번째 하위 클래스 `Song`은 기본 클래스의 상위에 `artist` 프로퍼티와 이니셜라이저를 추가합니다:

```
class Movie: MediaItem {
    var director: String
    init(name: String, director: String) {
        self.director = director
        super.init(name: name)
    }
}

class Song: MediaItem {
    var artist: String
    init(name: String, artist: String) {
        self.artist = artist
        super.init(name: name)
    }
}
```

마지막 코드는 두 `Movie` 인스턴스와 세 개의 `Song` 인스턴스를 포함하는 `library`라는 배열 상수를 생성합니다. `library` 배열의 타입은 배열 리터럴의 내용으로 초기화하여 추론됩니다. Swift의 타입 검사기는 `Movie`와 `Song`이 `MediaItem`의 상위 클래스를 공통으로 가지고 있으므로, `library` 배열에 대해 `[MediaItem]` 타입으로 추론할 수 있습니다:

```
let library = [
    Movie(name: "Casablanca", director: "Michael Curtiz"),
    Song(name: "Blue Suede Shoes", artist: "Elvis Presley"),
    Movie(name: "Citizen Kane", director: "Orson Welles"),
    Song(name: "The One And Only", artist: "Chesney Hawkes"),
    Song(name: "Never Gonna Give You Up", artist: "Rick Astley")
]
// the type of "library" is inferred to be [MediaItem]
```

`library`에 저장된 항목은 여전히 `Movie`와 `Song` 인스턴스 입니다. 그러나 이 배열의 항목을 반복하면, 항목은 `Movie`나 `Song`이 아닌 `MediaItem` 타입으로 받습니다. 기본 타입으로 작업을 하려면, 아래에 설명된대로 타입을 확인하거나 다른 타입으로 다운 캐스트(downcast) 해야합니다.

타입 검사 (Checking Type)

인스턴스가 특정 하위 클래스 타입인지 확인하기 위해 타입 검사 연산자(type check operator)(`is`)를 사용합니다. 이 타입 검사 연산자는 인스턴스가 하위 클래스 타입이면 `true` 아니면 `false`를 반환합니다.

아래의 예시는 `library` 배열의 `Movie`와 `Song` 인스턴스의 개수를 나타내는 두 변수 `movieCount`와 `songCount`를 정의합니다:

```
var movieCount = 0
var songCount = 0

for item in library {
    if item is Movie {
        movieCount += 1
    } else if item is Song {
        songCount += 1
    }
}

print("Media library contains \(movieCount) movies and \(songCount) songs")
// Prints "Media library contains 2 movies and 3 songs".
```

이 예시는 `library` 배열의 모든 항목을 반복합니다. 반복문 실행 시에 `for-in` 루프는 배열에서 다음 `MediaItem`을 `item` 상수에 설정합니다.

`item is Movie`는 현재 `MediaItem`이 `Movie` 인스턴스이면 `true`를 반환하고 아니면 `false`를 반환합니다. 유사하게 `item is Song`은 항목이 `Song` 인스턴스인지 확인합니다. `for-in` 루프의 마지막에 `movieCount`와 `songCount`의 값은 `MediaItem` 인스턴스에서 각 타입을 포함하는 개수를 나타냅니다.

다운 캐스팅 (Downcasting)

특정 클래스 타입의 상수나 변수는 하위 클래스의 인스턴스를 참조할 수 있습니다. 이것이 필요하다고 생각하는 경우, 타입 캐스트 연산자(type cast operator)(`as?` 또는 `as!`)를 사용하여 하위 클래스 타입으로 *다운 캐스트(downcast)*할 수 있습니다.

다운 캐스팅은 실패할 수 있으므로, 타입 캐스트 연산자는 두 가지 다른 형태로 제공됩니다. 조건부 형식 `as?`은 다운 캐스트를 하려고 할 때 타입의 옵셔널 값을 반환합니다. 강제 형식 `as!`은 다운 캐스트를 시도하고 단일 복합 동작으로 강제 언래핑합니다.

다운 캐스트가 성공할지 확신이 없을 때, 조건부 형식의 타입 캐스트 연산자(`as?`)를 사용합니다. 이 연산자의 형식은 항상 옵셔널 값을 반환하고, 다운 캐스트가 불가능하면 `nil`을 반환합니다. 이것은 다운 캐스트의 성공여부를 확인하는 용도로도 사용 가능합니다.

다운 캐스트가 항상 성공할 것이라는 확신이 있을 때만, 강제 형식의 타입 캐스트 연산자(`as!`)를 사용해야 합니다. 이 연산자의 형식은 유효하지 않은 클래스 타입으로 다운 캐스트를 시도하면 런타임 오류가 발생합니다.

아래의 예시는 `library`의 각 `MediaItem`을 반복하고 각 항목에 대한 적절한 설명을 출력합니다. 이렇게 하려면 `MediaItem`이 아닌, `Movie`나 `Song`으로 각 항목에 접근해야 합니다. 이렇게 해야 설명에 사용할 `Movie`나 `Song`의 `director`나 `artist` 프로퍼티에 접근할 수 있도록 하기위해 필요합니다.

이 예시는 배열의 각 항목은 `Movie`나 `Song`일 수 있습니다. 각 항목에 사용할 실제 클래스를 미리 알지 못하므로, 루프를 통해 매번 다운 캐스트를 확인하기 위해 타입 캐스트 연산자(`as?`)의 조건부 형식을 사용하는 것이 적절합니다:

```
for item in library {
    if let movie = item as? Movie {
        print("Movie: \(movie.name), dir. \(movie.director)")
    } else if let song = item as? Song {
        print("Song: \(song.name), by \(song.artist)")
    }
}
```

```
// Movie: Casablanca, dir. Michael Curtiz
// Song: Blue Suede Shoes, by Elvis Presley
// Movie: Citizen Kane, dir. Orson Welles
// Song: The One And Only, by Chesney Hawkes
// Song: Never Gonna Give You Up, by Rick Astley
```

이 예시는 `Movie`로 현재 `item`을 다운 캐스트 하는 것으로 시작합니다. `item`은 `MediaItem` 인스턴스 이므로, `Movie`일 수도 있고; 마찬가지로 `Song`일 수 있습니다. 또는 기본 `MediaItem`일 수도 있습니다. 이 불확실성 때문에, 타입 캐스트 연산자의 `as?` 형식은 하위 클래스 타입으로 다운 캐스트를 시도할 때 옵셔널 값으로 반환합니다. `item as? Movie`의 결과는 `Movie?` 타입이거나 "옵셔널 `Movie`"입니다.

라이브러리 배열에 `Song` 인스턴스로 적용할 때 `Movie`로 다운 캐스팅은 실패합니다. 이것을 대응하기 위해 위의 예시는 옵셔널 `Movie`가 실제 값이 포함되어 있는지 확인하기 위해 옵셔널 바인딩을 사용합니다. (다운 캐스트가 성공되었는지 확인합니다.) 이 옵셔널 바인딩은 "`if let movie = item as? Movie`"로 작성되고, 다음과 같이 읽을 수 있습니다:

"`Movie`로 `item`을 접근하려고 합니다. 성공하면 반환된 옵셔널 `Movie`를 `movie`라는 새로운 임시 상수에 설정합니다."

다운 캐스팅이 성공하면, `movie`의 프로퍼티는 `director`의 이름을 포함하여 `Movie` 인스턴스에 대해 설명을 출력하는데 사용합니다. 유사한 원칙을 사용하여 `Song` 인스턴스를 확인하고, 라이브러리에서 `Song`을 찾을 때마다 `artist` 이름을 포함하여 적절한 설명을 출력하는데 사용합니다.

> **참고**
>
> 캐스팅은 실제로 인스턴스를 수정하거나 값을 변경하지 않습니다. 기본 인스턴스는 동일하게 유지됩니다; 캐스트된 타입의 인스턴스로 처리하고 접근합니다.

Any와 AnyObject에 대한 타입 캐스팅 (Type Casting for Any and AnyObject)

Swift는 비특정 타입 작업을 위해 두 가지 특별한 타입을 제공합니다:

- `Any`는 함수 타입을 포함하여 모든 타입의 인스턴스를 나타낼 수 있습니다.
- `AnyObject`는 모든 클래스 타입의 인스턴스를 나타낼 수 있습니다.

제공하는 동작과 기능이 명시적으로 필요한 경우에만 `Any`와 `AnyObject`를 사용합니다. 코드에서 작업할 것으로 예상되는 타입에 대해 구체적으로 지정하는 것이 항상 좋습니다.

다음은 함수 타입과 비 클래스 타입을 포함하여 다른 타입에 혼합으로 작업하기 위해 `Any`를 사용하는 예시입니다. 이 예시는 `Any` 타입의 값을 저장할 수 있는 `things`라는 배열을 생성합니다:

```
var things: [Any] = []

things.append(0)
things.append(0.0)
things.append(42)
things.append(3.14159)
things.append("hello")
things.append((3.0, 5.0))
things.append(Movie(name: "Ghostbusters", director: "Ivan Reitman"))
things.append({ (name: String) -> String in "Hello, \(name)" })
```

`things` 배열은 두 `Int` 값, 두 `Double` 값, 하나의 `String` 값, 하나의 `(Double, Double)` 타입의 튜플, "Ghostbusters" 영화, `String` 값과 다른 `String` 값을 반환하는 클로저 표현식을 포함합니다.

`Any`나 `AnyObject` 타입으로만 알려진 상수나 변수의 특정 타입을 알아보려면 `switch` 구문의 케이스로 `is`나 `as` 패턴을 사용할 수 있습니다. 아래의 예시는 `things` 배열에 항목을 반복하고 `switch` 구문으로 각 항목의 타입을 조회합니다. 몇몇의 `switch` 구문의 케이스는 일치된 값을 지정된 타입의 상수에 바인딩하여 해당 값을 출력할 수 있도록 합니다:

```
for thing in things {
    switch thing {
    case 0 as Int:
        print("zero as an Int")
    case 0 as Double:
        print("zero as a Double")
    case let someInt as Int:
        print("an integer value of \(someInt)")
    case let someDouble as Double where someDouble > 0:
        print("a positive double value of \(someDouble)")
    case is Double:
        print("some other double value that I don't want to print")
    case let someString as String:
        print("a string value of \"\(someString)\"")
    case let (x, y) as (Double, Double):
        print("an (x, y) point at \(x), \(y)")
    case let movie as Movie:
        print("a movie called \(movie.name), dir. \(movie.director)")
    case let stringConverter as (String) -> String:
        print(stringConverter("Michael"))
    default:
        print("something else")
    }
}

// zero as an Int
// zero as a Double
// an integer value of 42
// a positive double value of 3.14159
// a string value of "hello"
// an (x, y) point at 3.0, 5.0
// a movie called Ghostbusters, dir. Ivan Reitman
// Hello, Michael
```

참고

`Any` 타입은 옵셔널 타입을 포함하여 모든 타입의 값을 나타냅니다. Swift는 `Any` 타입의 값이 기대되는 곳에 옵셔널 값을 사용하면 경고를 줍니다. `Any` 값으로 옵셔널 값을 사용하는 것이 필요하다면 아래와 같이 `as` 연산자를 사용하여 명시적으로 옵셔널을 `Any`로 변환할 수 있습니다.

```
let optionalNumber: Int? = 3
things.append(optionalNumber)        // Warning
things.append(optionalNumber as Any) // No warning
```

언어 가이드

중첩 타입 (Nested Types)

타입의 범위 내에 다른 타입을 정의합니다.

열거형은 특정 클래스나 구조체의 기능을 지원하기 위해 생성합니다. 유사하게 더 복잡한 타입의 컨텍스트 내에서 사용하기 위해 유틸리티 구조체와 일반적으로 특정 타입과 함께 사용되는 프로토콜을 정의하는 것이 편리할 수 있습니다. 이를 위해 Swift는 *중첩 타입(nested types)*을 정의할 수 있으며, 지원하는 타입의 정의 내에서 열거형, 구조체, 프로토콜과 같은 타입을 중첩할 수 있습니다.

다른 타입 내에서 타입을 중첩하려면 지원하는 타입의 외부 중괄호 내에 정의를 작성해야 합니다. 타입은 필요에 따라 원하는 만큼 여러 단계로 중첩할 수 있습니다.

중첩 타입의 동작 (Nested Types in Action)

아래의 예시는 블랙잭 게임에서 사용되는 카드 게임을 모델링하는 `BlackjackCard`라는 구조체를 정의합니다. `BlackjackCard` 구조체는 `Suit`와 `Rank`라는 두 중첩된 열거형을 포함합니다.

블랙잭에서 에이스 카드의 값은 1이나 11입니다. 이러한 특징은 `Rank` 열거형 내에 중첩된 `Values`라는 구조체로 나타냅니다:

```
struct BlackjackCard {

    // nested Suit enumeration
    enum Suit: Character {
        case spades = "♠", hearts = "♡", diamonds = "♢", clubs = "♣"
    }

    // nested Rank enumeration
    enum Rank: Int {
        case two = 2, three, four, five, six, seven, eight, nine, ten
        case jack, queen, king, ace
        struct Values {
            let first: Int, second: Int?
        }
        var values: Values {
            switch self {
            case .ace:
                return Values(first: 1, second: 11)
            case .jack, .queen, .king:
                return Values(first: 10, second: nil)
            default:
                return Values(first: self.rawValue, second: nil)
            }
        }
    }

    // BlackjackCard properties and methods
    let rank: Rank, suit: Suit
    var description: String {
        var output = "suit is \(suit.rawValue),"
        output += " value is \(rank.values.first)"
```

```
        if let second = rank.values.second {
            output += " or \(second)"
        }
        return output
    }
}
```

`Suit` 열거형은 기호를 나타내기 위해 `Character` 타입을 원시값으로 네 가지 일반적인 카드 모양을 나타냅니다.

`Rank` 열거형은 카드 값을 나타내기 위해 `Int` 타입을 원시값으로 가능한 13가지 카드 순위를 나타냅니다. (이 `Int` 타입의 원시값은 Jack, Queen, King, Ace 카드에는 사용되지 않습니다.)

위에서 말했듯이 `Rank` 열거형은 `Values`라는 자신의 중첩된 구조체를 더 정의합니다. 이 구조체는 대부분의 카드는 하나의 값을 가지지만 에이스 카드는 두 값을 가지는 사실을 캡슐화합니다. `Values` 구조체는 다음과 같이 두 프로퍼티를 정의합니다:

- `Int` 타입의 `first`
- `Int?`나 "옵셔널 `Int`" 타입의 `second`

`Rank`는 `Values` 구조체의 인스턴스를 반환하는 `values`라는 연산 프로퍼티도 정의합니다. 이 연산 프로퍼티는 카드의 순위를 고려하고 그 순위를 기반으로 적절한 값으로 새로운 `Values` 인스턴스를 초기화합니다. `jack`, `queen`, `king`, `ace`에 대한 특별한 값을 사용합니다. 숫자 카드에 대해서는 순위의 `Int` 타입의 원시값을 사용합니다.

`BlackjackCard` 구조체는 `rank`와 `suit`의 두 프로퍼티를 가지고 있습니다. 이름과 카드의 값 설명을 만들기 위해 `rank`와 `suit`에 저장된 값을 사용하는 `description`이라는 연산 프로퍼티도 정의합니다. `description` 프로퍼티는 화면에 표시하기 위해 두 번째 값이 있는지 확인하고 있으면, 두 번째 값에 대해 상세 설명을 추가합니다.

`BlackjackCard`는 커스텀 이니셜라이저가 없는 구조체이기 때문에, 구조체 타입의 멤버와이즈 이니셜라이저 (Memberwise Initializers for Structure Types)에서 설명했듯이 암시적 멤버별 이니셜라이저를 가지고 있습니다. `theAceOfSpades`라는 새로운 상수를 초기화하기 위해 이니셜라이저를 사용할 수 있습니다:

```
let theAceOfSpades = BlackjackCard(rank: .ace, suit: .spades)
print("theAceOfSpades: \(theAceOfSpades.description)")
// Prints "theAceOfSpades: suit is ♠, value is 1 or 11".
```

`Rank`와 `Suit`는 `BlackjackCard` 내에 중첩되어 있지만, 타입은 컨텍스트로부터 유추될 수 있기 때문에 이 인스턴스의 초기화는 케이스 이름(`.ace`와 `.spades`)만으로 열거형 케이스를 참조할 수 있습니다. 위의 예시에서 `description` 프로퍼티는 스페이드의 에이스는 `1`이나 `11`의 값을 가지고 있다고 올바르게 보여줍니다.

중첩 타입 참조 (Referring to Nested Types)

중첩 타입을 그 정의된 범위 밖에서 사용하려면 그 타입이 중첩된 외부 타입의 이름을 접두사로 붙여야 합니다:

```
let heartsSymbol = BlackjackCard.Suit.hearts.rawValue
// heartsSymbol is "♡"
```

위의 예시에서 `Suit`, `Rank`, `Values`의 이름은 정의된 컨텍스트에 따라 자연스럽게 규정되기 때문에 이름을 의도적으로 짧게 유지할 수 있습니다.

언어 가이드

확장 (Extensions)

기존 타입에 기능을 추가합니다.

*확장(Extensions)*은 기존의 클래스, 구조체, 열거형, 프로토콜 타입에 새로운 기능을 추가합니다. 이것은 기존 소스 코드에 접근 권한이 없는 타입을 확장하는 기능이 포함됩니다 (*소급 모델링(retroactive modeling)*이라고 함). 확장은 Objective-C의 카테고리와 유사합니다. (Objective-C 카테고리와 다르게 Swift 확장은 이름이 없습니다.)

Swift에서 확장은 다음을 수행할 수 있습니다:

- 인스턴스 연산 프로퍼티와 타입 연산 프로퍼티 추가
- 인스턴스 메서드와 타입 메서드 정의
- 새로운 이니셜라이저 제공
- 서브스크립트 정의
- 새로운 중첩 타입 정의와 사용
- 기존 타입이 프로토콜을 준수하도록 함

Swift에서 프로토콜을 확장하여 프로토콜 요구사항 구현을 제공하거나 준수하는 타입의 기능을 추가할 수도 있습니다. 더 자세한 내용은 프로토콜 확장 (Protocol Extensions)을 참고바랍니다.

> **참고**
>
> 확장은 타입에 새로운 기능을 추가할 수 있지만, 기존 기능을 재정의할 수는 없습니다.

확장 문법 (Extension Syntax)

`extension` 키워드를 사용하여 확장을 선언합니다:

```
extension SomeType {
    // new functionality to add to SomeType goes here
}
```

확장은 하나 이상의 프로토콜을 채택하여 기존 타입을 확장할 수 있습니다. 프로토콜 준수를 추가할 때, 클래스나 구조체를 작성하는 것과 동일한 방법으로 프로토콜 이름을 작성합니다:

```
extension SomeType: SomeProtocol, AnotherProtocol {
    // implementation of protocol requirements goes here
}
```

이러한 방법으로 프로토콜 준수를 추가하는 것에 대한 자세한 설명은 확장으로 프로토콜 준수 추가 (Adding Protocol Conformance with an Extension)를 참고바랍니다.

확장은 제네릭 타입 확장 (Extending a Generic Type)에 설명된대로 기존 제네릭 타입을 확장하기 위해 사용할 수 있습니다. 제네릭 Where 절이 있는 확장 (Extensions with a Generic Where Clause)에 설명된대로 제네릭 타입을 확장하여 조건부로 기능을 추가할 수도 있습니다.

> **참고**
>
> 기존 타입에 새로운 기능을 추가하기 위해 확장을 정의한다면, 새로운 기능은 확장이 정의되기 전에 생성된 기존 인스턴스에서도 사용 가능합니다.

연산 프로퍼티 (Computed Properties)

확장은 기존 타입에 인스턴스 연산 프로퍼티와 타입 연산 프로퍼티를 추가할 수 있습니다. 이 예시는 거리 단위 작업에 필요한 Swift `Double` 타입의 다섯 개의 인스턴스 연산 프로퍼티를 추가합니다:

```
extension Double {
    var km: Double { return self * 1_000.0 }
    var m: Double { return self }
    var cm: Double { return self / 100.0 }
    var mm: Double { return self / 1_000.0 }
    var ft: Double { return self / 3.28084 }
}
let oneInch = 25.4.mm
print("One inch is \(oneInch) meters")
// Prints "One inch is 0.0254 meters".
let threeFeet = 3.ft
print("Three feet is \(threeFeet) meters")
// Prints "Three feet is 0.914399970739201 meters".
```

이 연산 프로퍼티는 특정 길이의 단위를 `Double` 값으로 표현합니다. 연산 프로퍼티로 구현 되었지만, 이러한 프로퍼티의 이름은 거리 변환을 수행하기 위해 해당 리터럴 값을 사용하는 방법으로 부동소수점 값에 점 문법을 사용할 수 있습니다.

이 예시에서 `1.0`의 `Double` 값은 "1 미터"로 표시됩니다. 표현식 `1.m`은 `1.0`의 `Double` 값을 계산한 것으로 간주하므로 `m` 연산 프로퍼티가 `self`를 반환하는 이유입니다.

다른 단위는 미터로 계산된 값으로 표현되기 위해 변환을 요구합니다. 1 킬로미터는 1,000 미터와 같으므로 `km` 연산 프로퍼티는 미터로 표현된 숫자로 변환하기 위해 `1_000.00`의 값을 곱합니다. 유사하게 3.28084 피트는 1 미터이므로 `ft` 연산 프로퍼티는 피트를 미터로 변환하기 위해 `Double` 값을 `3.28084`로 나눕니다.

이 프로퍼티는 읽기 전용 연산 프로퍼티이므로, 간결함을 위해 `get` 키워드 없이 표현합니다. 반환값은 `Double` 타입의 값이고, `Double`이 허용되는 수학적 계산 내에서는 사용될 수 있습니다:

```
let aMarathon = 42.km + 195.m
print("A marathon is \(aMarathon) meters long")
// Prints "A marathon is 42195.0 meters long".
```

> **참고**
>
> 확장은 새로운 연산 프로퍼티를 추가할 수 있지만 저장 프로퍼티나 기존 프로퍼티에 프로퍼티 관찰자를 추가할 수 없습니다.

이니셜라이저 (Initializers)

확장은 기존 타입에 새로운 이니셜라이저를 추가할 수 있습니다. 이것은 이니셜라이저 매개변수로 고유한 커스텀 타입을 받아들이기 위해 다른 타입을 확장하거나 타입의 기존 구현의 부분으로 포함되지 않는 추가 초기화 옵션을 제공할 수 있습니다.

확장은 클래스에 새로운 편의 이니셜라이저를 추가할 수 있지만 새로운 지정 이니셜라이저나 디이니셜라이저는 클래스에 추가할 수 없습니다. 지정 이니셜라이저와 디이니셜라이저는 항상 기존 클래스 구현에 의해 제공되어야 합니다.

모든 저장 프로퍼티에 기본값을 제공하고 모든 커스텀 이니셜라이저를 정의하지 않은 값 타입으로 이니셜라이저를 추가하기 위해 확장을 사용하면 확장의 이니셜라이저 내에서 값 타입에 대해 기본 이니셜라이저와 멤버별 이니셜라이저를 호출할 수 있습니다. 값 타입을 위한 이니셜라이저 위임 (Initializer Delegation for Value Types)에서 설명한대로 값 타입의 기존 구현의 부분으로 이니셜라이저를 작성한 경우에는 해당되지 않습니다.

다른 모듈에서 선언된 구조체에 이니셜라이저를 추가하기 위해 확장을 사용한다면 새로운 이니셜라이저는 정의한 모듈에서 이니셜라이저를 호출할 때까지 `self`를 접근할 수 없습니다.

아래의 예시는 사각형을 나타내기 위해 커스텀 `Rect` 구조체를 정의합니다. 이 예시는 모든 프로퍼티에 `0.0`의 기본값을 제공하는 `Size`와 `Point`라는 두 구조체도 정의합니다:

```
struct Size {
    var width = 0.0, height = 0.0
}
struct Point {
    var x = 0.0, y = 0.0
}
struct Rect {
    var origin = Point()
    var size = Size()
}
```

`Rect` 구조체는 모든 프로퍼티에 대해 기본값을 제공하므로 기본 이니셜라이저 (Default Initializers)에서 설명한대로 자동으로 기본 이니셜라이저와 멤버별 이니셜라이저를 받습니다. 이 이니셜라이저는 새로운 `Rect` 인스턴스를 생성하기 위해 사용할 수 있습니다:

```
let defaultRect = Rect()
let memberwiseRect = Rect(origin: Point(x: 2.0, y: 2.0),
   size: Size(width: 5.0, height: 5.0))
```

특정 중심점과 크기를 가지는 이니셜라이저를 제공하기 위해 `Rect` 구조체를 확장할 수 있습니다:

```
extension Rect {
    init(center: Point, size: Size) {
        let originX = center.x - (size.width / 2)
        let originY = center.y - (size.height / 2)
        self.init(origin: Point(x: originX, y: originY), size: size)
    }
}
```

이 새로운 이니셜라이저는 제공된 `center` 점과 `size` 값을 기반으로 적절한 원점을 계산하는 것으로 시작합니다. 그러면 이 이니셜라이저는 적절한 프로퍼티에 새로운 원점과 크기 값을 저장하는 구조체의 자동 멤버별 이니셜라이저 `init(origin:size:)`를 호출합니다:

```
let centerRect = Rect(center: Point(x: 4.0, y: 4.0),
                      size: Size(width: 3.0, height: 3.0))
// centerRect's origin is (2.5, 2.5) and its size is (3.0, 3.0)
```

참고

확장으로 새로운 이니셜라이저를 제공하면, 이니셜라이저가 완료되면 각 인스턴스가 완전히 초기화 되었는지 확인해야 합니다.

메서드 (Methods)

확장은 기존 타입에 새로운 인스턴스 메서드와 타입 메서드를 추가할 수 있습니다. 아래 예시는 `Int` 타입으로 `repetitions`라는 새로운 인스턴스 메서드를 추가합니다:

```
extension Int {
    func repetitions(task: () -> Void) {
        for _ in 0..<self {
            task()
        }
    }
}
```

`repetitions(task:)` 메서드는 매개변수가 없고 반환값이 없는 함수를 나타내는 `() -> Void` 타입의 인자를 가집니다.

이 확장을 정의한 후에, 여러 번 작업을 수행하기 위해 모든 정수로 `repetitions(task:)` 메서드를 호출할 수 있습니다:

```
3.repetitions {
    print("Hello!")
}
// Hello!
// Hello!
// Hello!
```

인스턴스 메서드 변경 (Mutating Instance Methods)

확장에 추가된 인스턴스 메서드는 인스턴스 자체로 수정 또는 변경할 수 있습니다. `self`나 프로퍼티를 수정하는 구조체와 열거형 메서드는 기존 구현의 mutating 메서드와 같이 `mutating`으로 인스턴스 메서드를 표시해야 합니다.

아래 예시는 원래 값을 제곱하는 Swift의 `Int` 타입의 `square`라는 새로운 mutating 메서드를 추가합니다:

```
extension Int {
    mutating func square() {
        self = self * self
    }
}
var someInt = 3
someInt.square()
// someInt is now 9
```

서브스크립트 (Subscripts)

확장은 기존 타입에 새로운 서브스크립트를 추가할 수 있습니다. 이 예시는 Swift의 `Int` 타입에 정수 서브스크립트를 추가합니다. 이 서브스크립트 `[n]`은 숫자의 오른쪽 부터 `n`에 위치하는 자리의 숫자를 반환합니다:

- `123456789[0]`은 `9`를 반환
- `123456789[1]`은 `8`을 반환

등등:

```
extension Int {
    subscript(digitIndex: Int) -> Int {
        var decimalBase = 1
        for _ in 0..<digitIndex {
            decimalBase *= 10
        }
        return (self / decimalBase) % 10
    }
}
746381295[0]
// returns 5
746381295[1]
// returns 9
746381295[2]
// returns 2
746381295[8]
// returns 7
```

`Int` 값이 요청된 인덱스에 대해 충분한 자리를 가지고 있지 않다면, 숫자 왼쪽에 0이 채워진 것처럼 서브스크립트 구현은 `0`을 반환합니다:

```
746381295[9]
// returns 0, as if you had requested:
0746381295[9]
```

중첩 타입 (Nested Types)

확장은 기존 클래스, 구조체, 열거형에 새로운 중첩 타입을 추가할 수 있습니다:

```
extension Int {
    enum Kind {
        case negative, zero, positive
    }
    var kind: Kind {
        switch self {
        case 0:
            return .zero
        case let x where x > 0:
            return .positive
        default:
            return .negative
        }
    }
}
```

이 예시는 `Int`에 새로운 중첩된 열거형을 추가합니다. `Kind`라는 열거형은 특정 정수 표현에 대한 숫자의 종류를 나타냅니다. 명확하게 숫자가 음수, 0, 양수 인지 표현합니다.

이 예시는 정수에 대해 적절한 `Kind` 열거형 케이스를 반환하는 `kind`라는 새로운 인스턴스 연산 프로퍼티를 `Int`에 추가합니다.

이 중첩 열거형은 모든 `Int` 값에서 사용할 수 있습니다:

```
func printIntegerKinds(_ numbers: [Int]) {
    for number in numbers {
        switch number.kind {
        case .negative:
            print("- ", terminator: "")
        case .zero:
            print("0 ", terminator: "")
        case .positive:
            print("+ ", terminator: "")
        }
    }
    print("")
}
printIntegerKinds([3, 19, -27, 0, -6, 0, 7])
// Prints "+ + - 0 - 0 + ".
```

`printIntegerKinds(_:)` 함수는 `Int` 값의 배열을 입력 받고 반복합니다. 배열의 각 정수에 대해 함수는 해당 정수의 `kind` 연산 프로퍼티를 고려하고 적절한 설명을 출력합니다.

참고

`number.kind`는 `Int.Kind` 타입으로 이미 알고 있습니다. 이러한 점 때문에 모든 `Int.Kind` 케이스 값은 `switch` 구문 내에서 `Int.Kind.negative`가 아닌 `.negative`로 짧게 작성할 수 있습니다.

언어 가이드

프로토콜 (Protocols)

타입이 구현해야 하는 요구사항을 정의합니다.

*프로토콜(protocol)*은 특정 작업이나 기능에 적합한 메서드, 프로퍼티, 그리고 다른 요구사항의 청사진을 정의합니다. 프로토콜은 클래스, 구조체, 열거형이 채택하여 이 요구사항의 구현을 제공할 수 있습니다. 프로토콜의 요구사항을 만족하는 모든 타입은 프로토콜을 *준수(conform)*한다고 합니다.

프로토콜을 준수하는 타입이 구현해야 하는 요구사항을 명시하는 것 외에도, 프로토콜을 확장하여 이 요구사항 중 일부를 구현하거나 준수하는 타입이 활용할 수 있는 추가 기능을 구현할 수도 있습니다.

프로토콜 문법 (Protocol Syntax)

프로토콜은 클래스, 구조체, 열거형과 유사한 방법으로 정의합니다:

```
protocol SomeProtocol {
    // protocol definition goes here
}
```

커스텀 타입이 특정 프로토콜을 채택한다고 나타내려면 타입 이름 뒤에 콜론과 함께 프로토콜 이름을 나열합니다. 여러 프로토콜을 채택하려면 콤마로 구분하여 나열할 수 있습니다:

```
struct SomeStructure: FirstProtocol, AnotherProtocol {
    // structure definition goes here
}
```

클래스에 상위 클래스가 존재한다면, 상위 클래스 다음에 콤마로 구분하여 채택하는 모든 프로토콜을 나열합니다:

```
class SomeClass: SomeSuperclass, FirstProtocol, AnotherProtocol {
    // class definition goes here
}
```

참고

프로토콜은 타입이기 때문에 Swift에서 다른 타입의 이름처럼 (`Int`, `String`, `Double` 등) 대문자로 시작합니다 (`FullyNamed`, `RandomNumberGenerator` 등).

프로퍼티 요구사항 (Property Requirements)

프로토콜은 이를 준수하는 타입이 특정 이름과 타입을 가진 인스턴스 프로퍼티나 타입 프로퍼티를 제공하도록 요구할 수 있습니다. 프로토콜은 프로퍼티가 저장 프로퍼티나 연산 프로퍼티인지 지정하지 않습니다 - 필요한 프로퍼티 이름과 타입만 지정합니다. 프로토콜은 각 프로퍼티가 읽기만 가능한지 읽기와 쓰기가 모두 가능한지도 지정합니다.

프로토콜이 프로퍼티의 읽기와 쓰기를 모두 요구하는 경우, 상수 저장 프로퍼티나 읽기 전용 계산 프로퍼티는 해당 요구사항을 충족할 수 없습니다. 프로토콜이 프로퍼티의 읽기만 요구하는 경우, 모든 종류의 프로퍼티로 요구사항을 충족할 수 있고 필요한 경우 쓰기가 가능하게 구현해도 무방합니다.

프로퍼티 요구사항은 항상 `var` 키워드로 변수 프로퍼티로 선언됩니다. 읽기와 쓰기 가능한 프로퍼티는 타입 선언 뒤에 `{ get set }`으로 작성하여 나타내고, 읽기만 가능한 프로퍼티는 `{ get }`으로 작성하여 나타냅니다.

```
protocol SomeProtocol {
    var mustBeSettable: Int { get set }
    var doesNotNeedToBeSettable: Int { get }
}
```

프로토콜에 타입 프로퍼티 요구사항을 정의하려면 `static` 키워드를 붙여야 합니다. 클래스에 타입 프로퍼티 요구사항을 구현할 때는 `class`나 `static` 키워드를 사용할 수 있지만, 프로토콜에서는 `static` 키워드만 사용해야 합니다:

```
protocol AnotherProtocol {
    static var someTypeProperty: Int { get set }
}
```

다음은 단일 인스턴스 프로퍼티 요구사항을 가지는 프로토콜 예시입니다:

```
protocol FullyNamed {
    var fullName: String { get }
}
```

`FullyNamed` 프로토콜은 이를 준수하는 타입이 완벽한 이름을 제공하도록 요구합니다. 이 프로토콜은 준수 타입의 성격에 대해 다른 것은 지정하지 않으며 - 해당 타입이 전체 이름을 제공해야 된다고만 요구합니다. 이것은 `FullyNamed` 타입은 `String` 타입의 `fullName`이라는 읽기만 가능한 인스턴스 프로퍼티를 가져야 합니다.

다음은 `FullyNamed` 프로토콜을 채택하고 준수하는 구조체의 예시입니다:

```
struct Person: FullyNamed {
    var fullName: String
}
let john = Person(fullName: "John Appleseed")
// john.fullName is "John Appleseed"
```

이 예시는 특정 이름을 가진 사람을 나타내는 `Person`이라는 구조체를 정의합니다. 정의의 첫 번째 줄에 `FullyNamed` 프로토콜을 채택합니다.

`Person`의 각 인스턴스는 `String` 타입의 `fullName`이라는 단일 저장 프로퍼티를 가집니다. 이것은 `FullyNamed` 프로토콜의 단일 요구사항과 일치하고 `Person`은 프로토콜을 올바르게 준수하고 있다는 의미입니다. (Swift는 프로토콜 요구사항이 충족되지 않으면 컴파일 시 오류가 발생합니다.)

다음은 `FullyNamed` 프로토콜을 채택하고 준수하는 더 복잡한 클래스입니다:

```
class Starship: FullyNamed {
    var prefix: String?
    var name: String
    init(name: String, prefix: String? = nil) {
        self.name = name
        self.prefix = prefix
    }
    var fullName: String {
        return (prefix != nil ? prefix! + " " : "") + name
    }
}
var ncc1701 = Starship(name: "Enterprise", prefix: "USS")
// ncc1701.fullName is "USS Enterprise"
```

이 클래스는 스타십에 대해 읽기 전용 연산 프로퍼티로 `fullName` 프로퍼티 요구사항을 구현합니다. 각 `Starship` 클래스 인스턴스는 필수 `name`과 옵셔널 `prefix`를 저장합니다. `fullName` 프로퍼티는 `prefix` 값이 존재하면 사용하여 `name`의 앞에 추가해 스타십의 전체 이름을 생성합니다.

메서드 요구사항 (Method Requirements)

프로토콜은 이를 준수하는 타입이 특정 인스턴스 메서드와 타입 메서드를 구현하도록 요구할 수 있습니다. 이 메서드는 일반 인스턴스 메서드와 타입 메서드를 정의하는 방식과 동일하게 프로토콜의 정의에 작성되지만 중괄호나 메서드 본문 없이 작성합니다. 가변 매개변수(variadic parameter)도 일반 메서드와 같은 규칙에 따라 사용할 수 있습니다. 그러나 프로토콜의 정의 내에서 메서드 매개변수에 대해 기본값은 지정할 수 없습니다.

타입 프로퍼티 요구사항과 마찬가지로 프로토콜에 타입 메서드 요구사항을 정의할 때 `static` 키워드를 항상 표기합니다. 클래스에 타입 메서드 요구사항을 구현할 때는 `class`나 `static` 키워드를 사용할 수 있지만, 프로토콜에서는 `static` 키워드만 사용해야 합니다:

```
protocol SomeProtocol {
    static func someTypeMethod()
}
```

다음 예시는 단일 인스턴스 메서드 요구사항을 가지는 프로토콜을 정의합니다:

```
protocol RandomNumberGenerator {
    func random() -> Double
}
```

`RandomNumberGenerator` 프로토콜은 `random` 이라는 인스턴스 메서드 요구사항을 가지며 호출할 때마다 `Double` 값을 반환합니다. 프로토콜에 지정하지 않았지만, 이 값은 `0.0`부터 `1.0`미만의 숫자라고 가정합니다.

`RandomNumberGenerator` 프로토콜은 각 난수가 생성되는 방법에 대해 어떠한 것도 가정하지 않습니다 - 이것은 새로운 난수를 생성하는 표준 방법을 제공할 것을 요구합니다.

다음은 `RandomNumberGenerator` 프로토콜을 채택하고 준수하는 클래스의 구현입니다. 이 클래스는 *선형 합동 생성기(linear congruential generator)*로 알려진 의사 난수(pseudorandom number) 생성 알고리즘을 구현합니다:

```
class LinearCongruentialGenerator: RandomNumberGenerator {
    var lastRandom = 42.0
    let m = 139968.0
    let a = 3877.0
    let c = 29573.0
    func random() -> Double {
        lastRandom = ((lastRandom * a + c)
            .truncatingRemainder(dividingBy:m))
        return lastRandom / m
    }
}
let generator = LinearCongruentialGenerator()
print("Here's a random number: \(generator.random())")
// Prints "Here's a random number: 0.3746499199817101".
print("And another one: \(generator.random())")
// Prints "And another one: 0.729023776863283".
```

Mutating 메서드 요구사항 (Mutating Method Requirements)

메서드가 속한 인스턴스를 수정이나 변경해야 하는 경우가 있습니다. 값 타입(구조체와 열거형)의 인스턴스 메서드의 경우 메서드의 `func` 키워드 앞에 `mutating` 키워드를 작성하여 메서드가 속한 인스턴스와 인스턴스의 모든 프로퍼티를 수정할 수 있음을 나타냅니다. 이 프로세스는 인스턴스 메서드 내부에서 값 타입 수정 (Modifying Value Types from Within Instance Methods)에 설명되어 있습니다.

프로토콜 인스턴스 메서드 요구사항이 프로토콜을 채택하는 모든 타입의 인스턴스도 변경할 수 있도록 정의하는 경우 프로토콜의 정의에서 해당 메서드에 `mutating` 키워드를 붙여야 합니다. 이를 통해 구조체와 열거형도 프로토콜을 채택하고 메서드 요구사항을 충족할 수 있습니다.

> **참고**
>
> `mutating`으로 프로토콜 인스턴스 메서드 요구사항을 표시하면, 클래스에 대한 해당 메서드의 구현을 작성할 때 `mutating` 키워드를 작성할 필요가 없습니다. `mutating` 키워드는 구조체와 열거형에서만 사용합니다.

아래의 예시는 `toggle`이라는 단일 인스턴스 메서드 요구사항을 정의하는 `Togglable`이라는 프로토콜을 정의합니다. 이름에서 알 수 있듯이, `toggle()` 메서드는 해당 타입의 프로퍼티를 수정하여 프로토콜을 준수하는 타입의 상태를 전환하거나 반전하기 위한 것입니다.

`toggle()` 메서드는 `Togglable` 프로토콜 정의에서 `mutating` 키워드로 표시되고, 이 메서드가 호출될 때 준수하는 인스턴스의 상태를 변경한다고 나타냅니다:

```
protocol Togglable {
    mutating func toggle()
}
```

구조체나 열거형에 `Togglable` 프로토콜을 구현하면, 해당 구조체나 열거형은 `mutating`으로 표시된 `toggle()` 메서드의 구현을 제공하여 프로토콜을 준수할 수 있습니다.

아래의 예시는 `OnOffSwitch`라는 열거형을 정의합니다. 이 열거형은 열거형 케이스 인 `on`과 `off`를 나타내기 위해 두 가지 상태를 가집니다. 이 열거형의 `toggle` 구현은 `Togglable` 프로토콜의 요구사항에 맞게 `mutating`으로 표시됩니다:

```
enum OnOffSwitch: Togglable {
    case off, on
    mutating func toggle() {
        switch self {
        case .off:
            self = .on
        case .on:
            self = .off
        }
    }
}
var lightSwitch = OnOffSwitch.off
lightSwitch.toggle()
// lightSwitch is now equal to .on
```

이니셜라이저 요구사항 (Initializer Requirements)

프로토콜은 이를 준수하는 타입이 특정 이니셜라이저를 구현하도록 요구할 수 있습니다. 이 이니셜라이저는 일반 이니셜라이저를 정의하는 방식과 동일하게 프로토콜의 정의에 작성되지만 중괄호나 이니셜라이저 본문 없이 작성합니다.

```
protocol SomeProtocol {
    init(someParameter: Int)
}
```

프로토콜 이니셜라이저 요구사항의 클래스 구현 (Class Implementations of Protocol Initializer Requirements)

프로토콜의 이니셜라이저 요구사항은 준수하는 클래스에서 지정 이니셜라이저나 편의 이니셜라이저로 구현할 수 있습니다. 이 경우 모두 `required` 수정자를 이니셜라이저 구현에 표시해야 합니다:

```
class SomeClass: SomeProtocol {
    required init(someParameter: Int) {
        // initializer implementation goes here
    }
}
```

`required` 수정자를 사용하면 프로토콜을 준수하는 클래스의 모든 하위 클래스가 이니셜라이저 요구사항을 명시적으로 구현하거나 상속받도록 보장합니다.

더 자세한 정보는 필수 이니셜라이저 (Required Initializers)을 참고바랍니다.

> **참고**
>
> `final` 수정자로 표시한 클래스는 하위 클래스가 될 수 없으므로 프로토콜 이니셜라이저 구현에 `required` 수정자를 표시할 필요가 없습니다. `final` 수정자에 대한 자세한 내용은 재정의 방지 (Preventing Overrides)를 참고바랍니다.

하위 클래스가 상위 클래스의 지정 이니셜라이저를 재정의 하고 프로토콜의 동일한 이니셜라이저 요구사항도 구현해야 하면, `required`와 `override` 수정자 둘 다 이니셜라이저 구현에 표시해야 합니다:

```
protocol SomeProtocol {
    init()
}

class SomeSuperClass {
    init() {
        // initializer implementation goes here
    }
}

class SomeSubClass: SomeSuperClass, SomeProtocol {
    // "required" from SomeProtocol conformance; "override" from SomeSuperClass
    required override init() {
        // initializer implementation goes here
    }
}
```

실패 가능한 이니셜라이저 요구사항 (Failable Initializer Requirements)

프로토콜은 실패 가능한 이니셜라이저 (Failable Initializers)에 정의한대로 준수하는 타입에 대해 실패 가능한 이니셜라이저 요구사항을 정의할 수 있습니다.

실패 가능한 이니셜라이저 요구사항은 준수하는 타입에서 실패 가능한 이니셜라이저나 실패 불가능한 이니셜라이저로 충족될 수 있습니다. 실패 불가능한 이니셜라이저 요구사항은 실패 불가능한 이니셜라이저나 암시적 언래핑 된 실패 가능한 이니셜라이저로 충족될 수 있습니다.

의미론적 요구사항만 있는 프로토콜 (Protocols that Have Only Semantic Requirements)

위의 모든 프로토콜 예시는 일부 메서드나 프로퍼티를 요구하지만 프로토콜 선언에 요구사항을 포함하지 않아도 됩니다. 프로토콜을 사용해 의미론적(semantic) 요구사항을 기술할 수도 있습니다 - 이것은 해당 타입의 값이 어떻게 동작하고 어떤 연산을 지원하는지에 대한 요구사항을 의미합니다.

Swift 표준 라이브러리는 여러 프로토콜을 정의하며 이것은 어떠한 메서드나 프로퍼티를 요구하지 않습니다:

- `Sendable`: Sendable 타입 (Sendable Types)에서 설명한대로 동시성 도메인 간 공유될 수 있는 값을 위한 프로토콜입니다.
- `Copyable`: Borrowing 매개변수와 Consuming 매개변수 (Borrowing and Consuming Parameters)에서 설명한대로 함수에 전달될 때 Swift가 복사할 수 있는 값을 위한 프로토콜입니다.
- `BitwiseCopyable`: 비트 단위로 복사될 수 있는 값을 위한 프로토콜입니다.

이러한 프로토콜의 요구사항에 대한 자세한 내용은 이 문서의 개요를 참고바랍니다.

다른 프로토콜을 채택할 때와 동일한 문법을 사용하여 이러한 프로토콜을 채택합니다. 유일한 차이점은 프로토콜의 요구사항을 구현하는 메서드나 프로퍼티 선언이 포함되어 있지 않습니다. 예를 들어:

```
struct MyStruct: Copyable {
    var counter = 12
}

extension MyStruct: BitwiseCopyable { }
```

위 코드는 새로운 구조체를 정의합니다. `Copyable`은 의미론적 요구사항만 가지므로 구조체 선언에 프로토콜을 채택하기 위한 코드가 없습니다. 유사하게 `BitwiseCopyable`은 의미론적 요구사항만 가지므로 프로토콜을 채택하는 확장은 빈 본문을 가집니다.

보통 이러한 프로토콜의 준수를 작성할 필요가 없습니다 - 대신 프로토콜 암시적 준수 (Implicit Conformance to a Protocol)에서 설명한대로 Swift는 암시적으로 준수를 추가합니다.

프로토콜을 타입으로 사용 (Protocols as Types)

프로토콜은 실제로 어떤 기능도 구현하지 않습니다. 그럼에도 불구하고 코드에서 프로토콜을 타입으로 사용할 수 있습니다.

프로토콜을 타입으로 사용하는 가장 일반적인 방법은 제네릭 제약조건(generic constraint)으로 프로토콜을 사용하는 것입니다. 제네릭 제약조건이 있는 코드는 프로토콜을 준수하는 어떠한 타입에서 동작할 수 있고 구체적인 타입은 API를 사용하는 코드에 의해 선택됩니다. 예를 들어 제네릭 타입의 인자를 받는 함수를 호출할 때, 호출자는 타입을 선택합니다.

불투명 타입(opaque type)을 사용하는 코드는 프로토콜을 준수하는 일부 타입에서 동작합니다. 실제 타입은 컴파일 시간에 알 수 있으며, API 구현에서 해당 타입을 선택하지만, 해당 타입의 식별자는 API의 클라이언트로 부터 숨겨집니다. 불투명 타입을 사용하면 API 의 자세한 구현이 추상 계층을 통해 노출되는 것을 방지할 수 있습니다 - 예를 들어, 함수의 반환 타입을 숨기고 값이 특정 프로토콜을 준수한다는 것만 보장할 수 있습니다.

박싱 프로토콜 타입(boxed protocol type)을 사용하는 코드는 런타임 때 선택된 프로토콜을 준수하는 모든 타입에서 동작할 수 있습니다. 이런 런타임 유연성을 지원하기 위해, Swift는 필요할 때 성능 비용을 발생하는 *박스(box)* 라는 간접 계층을 추가합니다. 이러한 유연성 때문에 Swift는 컴파일 시에 실제 타입을 알 수 없으므로, 프로토콜에 의해 요구되는 멤버만 접근할 수 있습니다. 실제 타입의 다른 API에 접근하려면 런타임에서 캐스팅이 필요합니다.

프로토콜을 제네릭 제약조건으로 사용하는 것에 대한 자세한 내용은 제네릭 (Generics)을 참고바랍니다. 불투명 타입과 박싱 프로토콜 타입에 대한 자세한 내용은 불투명 타입과 박싱 프로토콜 타입 (Opaque and Boxed Protocol Types)을 참고바랍니다.

위임 (Delegation)

*위임(Delegation)*은 클래스나 구조체가 일부 책임을 다른 타입의 인스턴스에 넘길 수 있도록 하는 디자인 패턴입니다. 이 디자인 패턴은 위임할 책임을 캡슐화한 프로토콜을 정의해 구현하며, 이를 준수하는 타입(위임자)은 위임된 기능을 반드시 구현해야 합니다. 위임은 특정 동작에 응답하거나, 해당 소스의 구체적인 타입을 알 필요 없이 외부 소스에서 데이터를 가져올 때 사용할 수 있습니다.

아래 예시는 주사위 게임과 게임의 진행 사항을 관찰하는 위임에 대한 중첩된 프로토콜을 정의합니다:

```
class DiceGame {
    let sides: Int
    let generator = LinearCongruentialGenerator()
    weak var delegate: Delegate?

    init(sides: Int) {
        self.sides = sides
    }

    func roll() -> Int {
        return Int(generator.random() * Double(sides)) + 1
```

```
    }

    func play(rounds: Int) {
        delegate?.gameDidStart(self)
        for round in 1...rounds {
            let player1 = roll()
            let player2 = roll()
            if player1 == player2 {
                delegate?.game(self, didEndRound: round, winner: nil)
            } else if player1 > player2 {
                delegate?.game(self, didEndRound: round, winner: 1)
            } else {
                delegate?.game(self, didEndRound: round, winner: 2)
            }
        }
        delegate?.gameDidEnd(self)
    }

    protocol Delegate: AnyObject {
        func gameDidStart(_ game: DiceGame)
        func game(_ game: DiceGame, didEndRound round: Int, winner: Int?)
        func gameDidEnd(_ game: DiceGame)
    }
}
```

`DiceGame` 클래스는 각 플레이어가 주사위를 굴리고, 주사위를 굴려 가장 높은 숫자를 얻는 플레이어가 승리하는 게임을 구현합니다. 이전 챕터의 예시에서 사용한 선형 합동 생성기를 사용하여 주사위 굴림에 대한 난수를 생성합니다.

`DiceGame.Delegate` 프로토콜은 주사위 게임의 진행 사항을 추적하기 위해 채택할 수 있습니다. `DiceGame.Delegate` 프로토콜은 주사위 게임의 내에서 사용되기 때문에, `DiceGame` 클래스 내에 중첩되어 있습니다. 프로토콜은 외부 선언이 제네릭하지 않으면, 구조체와 클래스와 같은 타입 선언 내에 중첩될 수 있습니다. 중첩 타입에 대한 자세한 내용은 중첩 타입 (Nested Types)을 참고바랍니다.

강한 순환 참조를 방지하기 위해, 위임자(delegate)는 약한 참조로 선언됩니다. 약한 참조에 대한 자세한 내용은 클래스 인스턴스 간의 강한 순환 참조 (Strong Reference Cycles Between Class Instances)를 참고바랍니다. 프로토콜을 클래스 전용 프로토콜로 표시하면 `DiceGame` 클래스는 위임자가 약한 참조로 사용되어야 한다고 선언할 수 있습니다. 클래스 전용 프로토콜 (Class-Only Protocols)에서 설명했듯이 클래스 전용 프로토콜은 `AnyObject`의 상속받음으로 표시합니다.

`DiceGame.Delegate`는 게임의 진행 사항을 추적하기 위해 세 개의 메서드를 제공합니다. 이 세 개의 메서드는 위의 `play(rounds:)` 메서드에서 게임 로직으로 사용됩니다. `DiceGame` 클래스는 새로운 게임이 시작하거나, 새로운 차례가 시작하거나, 게임이 끝날 때 해당 위임 메서드를 호출합니다.

`delegate` 프로퍼티는 옵셔널 `DiceGame.Delegate`이므로, `play(rounds:)` 메서드는 옵셔널 체이닝 (Optional Chaining)에서 설명한 대로 위임에 대한 메서드를 호출할 때마다 옵셔널 체이닝을 사용합니다. `delegate` 프로퍼티가 `nil`이면 이 위임자 호출은 무시됩니다. `delegate` 프로퍼티가 `nil`이 아니면 이 위임자 메서드가 호출되고, 매개변수로 `DiceGame` 인스턴스를 전달합니다.

다음 예시는 `DiceGame.Delegate` 프로토콜을 채택하는 `DiceGameTracker`라는 클래스는 나타냅니다:

```
class DiceGameTracker: DiceGame.Delegate {
    var playerScore1 = 0
    var playerScore2 = 0
    func gameDidStart(_ game: DiceGame) {
        print("Started a new game")
        playerScore1 = 0
        playerScore2 = 0
    }
    func game(_ game: DiceGame, didEndRound round: Int, winner: Int?) {
        switch winner {
            case 1:
                playerScore1 += 1
                print("Player 1 won round \(round)")
            case 2: playerScore2 += 1
                print("Player 2 won round \(round)")
            default:
                print("The round was a draw")
        }
    }
    func gameDidEnd(_ game: DiceGame) {
        if playerScore1 == playerScore2 {
            print("The game ended in a draw.")
        } else if playerScore1 > playerScore2 {
            print("Player 1 won!")
        } else {
            print("Player 2 won!")
        }
    }
}
```

`DiceGameTracker` 클래스는 `DiceGame.Delegate` 프로토콜에 의해 요구되는 세 가지 메서드를 모두 구현합니다. 이 메서드는 새로운 게임이 시작되면 플레이어의 점수를 모두 0으로 초기화하고, 각 라운드가 끝날 때마다 점수를 업데이트하고, 게임이 끝나면 승자를 발표합니다.

`DiceGame`과 `DiceGameTracker`의 실제 동작은 다음과 같습니다:

```
let tracker = DiceGameTracker()
let game = DiceGame(sides: 6)
game.delegate = tracker
game.play(rounds: 3)
// Started a new game
// Player 2 won round 1
// Player 2 won round 2
// Player 1 won round 3
// Player 2 won!
```

확장으로 프로토콜 준수 추가 (Adding Protocol Conformance with an Extension)

기존 타입의 소스 코드에 접근할 수 없더라도 새로운 프로토콜을 채택하고 준수하도록 확장할 수 있습니다. 확장은 기존 타입에 새로운 프로퍼티, 메서드, 서브스크립트를 추가할 수 있으므로, 프로토콜이 요구하는 모든 요구사항을 추가할 수 있습니다. 자세한 내용은 확장 (Extensions)을 참고바랍니다.

참고

확장을 통해 타입에 프로토콜 준수를 추가하면, 기존 인스턴스도 자동으로 해당 프로토콜을 채택하고 준수합니다.

예를 들어 `TextRepresentable`이라는 프로토콜은 텍스트로 표현할 수 있는 모든 타입이 구현할 수 있습니다. 이것은 자신에 대한 설명이거나 현재 상태의 텍스트 버전일 수 있습니다:

```
protocol TextRepresentable {
    var textualDescription: String { get }
}
```

위에서 `Dice` 클래스는 확장을 통해 `TextRepresentable`을 채택하고 준수할 수 있습니다:

```
extension Dice: TextRepresentable {
    var textualDescription: String {
        return "A \(sides)-sided dice"
    }
}
```

이 확장은 `Dice`가 원래 구현에서 프로토콜을 채택한 것과 동일하게 새로운 프로토콜을 채택합니다. 프로토콜 이름은 콜론으로 구분된 타입 이름 뒤에 제공되고, 프로토콜의 모든 요구사항은 확장의 중괄호 내에 구현합니다.

이제 모든 `Dice` 인스턴스는 `TextRepresentable`로 취급될 수 있습니다:

```
let d12 = Dice(sides: 12, generator: LinearCongruentialGenerator())
print(d12.textualDescription)
// Prints "A 12-sided dice".
```

마찬가지로 `SnakesAndLadders` 게임 클래스도 확장을 통해 `TextRepresentable` 프로토콜을 채택하고 준수할 수 있습니다:

```
extension SnakesAndLadders: TextRepresentable {
    var textualDescription: String {
        return "A game of Snakes and Ladders with \(finalSquare) squares"
    }
}
print(game.textualDescription)
// Prints "A game of Snakes and Ladders with 25 squares".
```

조건부 프로토콜 준수 (Conditionally Conforming to a Protocol)

제네릭 타입은 타입의 제네릭 매개변수가 프로토콜을 준수하는 경우와 같이 특정 조건에서만 프로토콜의 요구사항을 충족시킬 수 있습니다. 확장에서 제약조건을 명시하여 제네릭 타입이 조건부로 프로토콜을 준수하도록 만들 수 있습니다. 제네릭 `where` 절을 작성하여 채택할 프로토콜 이름 뒤에 제약조건을 명시합니다. 제네릭 `where` 절에 대한 자세한 내용은 제네릭 Where 절 (Generic Where Clauses)을 참고바랍니다.

다음 확장은 `Array` 인스턴스가 저장하는 항목이 `TextRepresentable`을 준수할 때에만 `TextRepresentable` 프로토콜을 준수하도록 합니다.

```
extension Array: TextRepresentable where Element: TextRepresentable {
    var textualDescription: String {
        let itemsAsText = self.map { $0.textualDescription }
        return "[" + itemsAsText.joined(separator: ", ") + "]"
    }
}
let myDice = [d6, d12]
print(myDice.textualDescription)
// Prints "[A 6-sided dice, A 12-sided dice]".
```

확장으로 프로토콜 채택 선언 (Declaring Protocol Adoption with an Extension)

타입이 이미 프로토콜의 모든 요구사항을 준수하지만, 해당 프로토콜을 채택한다고 아직 명시하지 않은 경우, 빈 확장을 사용하여 프로토콜을 채택하도록 만들 수 있습니다:

```
struct Hamster {
    var name: String
    var textualDescription: String {
        return "A hamster named \(name)"
    }
}
extension Hamster: TextRepresentable {}
```

`Hamster`의 인스턴스는 `TextRepresentable`이 요구되는 타입 어디에서나 사용할 수 있습니다:

```
let simonTheHamster = Hamster(name: "Simon")
let somethingTextRepresentable: TextRepresentable = simonTheHamster
print(somethingTextRepresentable.textualDescription)
// Prints "A hamster named Simon".
```

참고

타입은 요구사항이 충족된다고 해서 프로토콜을 자동으로 채택하지 않습니다. 항상 프로토콜 채택을 명시적으로 선언해야 합니다.

자동 생성 구현을 사용하여 프로토콜 채택 (Adopting a Protocol Using a Synthesized Implementation)

Swift는 많은 경우에 `Equatable`, `Hashable`, `Comparable`에 대해 프로토콜 준수성을 자동으로 제공할 수 있습니다. 이렇게 합성 구현을 사용하면 프로토콜 요구사항 구현을 위해 반복적인 상용구 코드를 작성할 필요가 없습니다.

Swift는 다음과 같은 커스텀 타입에 대해 `Equatable`의 합성 구현을 제공합니다:

- `Equatable` 프로토콜을 준수하는 저장 프로퍼티만 있는 구조체
- `Equatable` 프로토콜을 준수하는 연관 타입만 있는 열거형
- 연관 타입이 없는 열거형

==의 합성 구현을 받기 위해선, == 연산자를 직접 구현하지 않고 원래 선언을 포함한 파일에서 `Equatable`을 채택한다고 선언합니다. `Equatable` 프로토콜은 `!=`의 기본 구현도 제공합니다.

아래의 예시는 `Vector2D` 구조체와 유사한 3차원의 벡터 `(x, y, z)`에 대한 `Vector3D` 구조체를 정의합니다. `x`, `y`, `z` 프로퍼티는 모두 `Equatable` 타입이므로, `Vector3D`는 등가 연산자의 합성 구현을 받습니다.

```
struct Vector3D: Equatable {
    var x = 0.0, y = 0.0, z = 0.0
}

let twoThreeFour = Vector3D(x: 2.0, y: 3.0, z: 4.0)
let anotherTwoThreeFour = Vector3D(x: 2.0, y: 3.0, z: 4.0)
if twoThreeFour == anotherTwoThreeFour {
    print("These two vectors are also equivalent.")
}
// Prints "These two vectors are also equivalent.".
```

Swift는 다음과 같은 커스텀 타입에 대해 `Hashable`의 합성 구현을 제공합니다:

- `Hashable` 프로토콜을 준수하는 저장 프로퍼티만 가지는 구조체
- `Hashable` 프로토콜을 준수하는 연관 타입만 가지는 열거형
- 연관 타입이 없는 열거형

`hash(into:)`의 합성 구현을 받기 위해선, `hash(into:)` 메서드를 직접 구현하지 않고 원래 선언을 포함한 파일에서 `Hashable`을 채택한다고 선언합니다.

Swift는 원시값이 없는 열거형에 대해 `Comparable`의 합성 구현을 제공합니다. 열거형이 연관 타입을 가지고 있다면, 모두 `Comparable` 프로토콜을 준수해야 합니다. <의 합성 구현을 받기 위해선, < 연산자를 직접 구현하지 않고 원래 열거형 선언을 포함한 파일에서 `Comparable`을 채택한다고 선언합니다. <=, >, >=의 `Comparable` 프로토콜의 기본 구현도 자동으로 제공합니다.

아래의 예시는 초보자, 중급자, 전문가의 케이스를 가진 `SkillLevel` 열거형을 정의합니다. 전문가는 가진 별의 숫자에 따라 추가적으로 순위가 매겨집니다.

```
enum SkillLevel: Comparable {
    case beginner
    case intermediate
    case expert(stars: Int)
}
var levels = [SkillLevel.intermediate, SkillLevel.beginner,
              SkillLevel.expert(stars: 5), SkillLevel.expert(stars: 3)]
for level in levels.sorted() {
    print(level)
}
// Prints "beginner".
// Prints "intermediate".
// Prints "expert(stars: 3)".
// Prints "expert(stars: 5)".
```

프로토콜 암시적 준수 (Implicit Conformance to a Protocol)

일부 프로토콜은 너무 흔해서 새로운 타입을 선언할 때 거의 항상 작성하게 됩니다. 다음의 프로토콜의 경우 프로토콜의 요구사항을 구현하는 타입을 정의할 때 Swift가 자동으로 준수를 추론하므로 직접 작성할 필요가 없습니다:

- `Copyable`

- `Sendable`
- `BitwiseCopyable`

명시적으로 준수를 작성할 수 있지만 코드의 동작을 변경하지 않습니다. 암시적 준수를 억제하려면 준수 목록에서 프로토콜 이름 앞에 물결표(~)를 작성합니다:

```
struct FileDescriptor: ~Sendable {
    let rawValue: Int
}
```

위 코드는 POSIX 파일 디스크립터를 감싸는 래퍼의 일부를 보여줍니다. `FileDescriptor` 구조체는 `Sendable` 프로토콜의 요구사항을 모두 충족하여 일반적으로 Sendable합니다. 그러나 `~Sendable` 작성하는 것은 암시적 준수를 억제합니다. 파일 디스크립터는 열린 파일을 식별하고 상호작용하기 위해 정수를 사용하고 정수값은 Sendable 함에도 불구하고 이를 Sendable하지 않게 만드는 것은 특정 버그를 피하는데 도움을 줄 수 있습니다.

암시적 준수를 억제하는 다른 방법은 비가용성으로 표시된 확장을 사용하는 것입니다:

```
@available(*, unavailable)
extension FileDescriptor: Sendable { }
```

이전 예시처럼 코드의 한 곳에 `~Sendable`을 작성하면, 프로그램의 다른 곳에 있는 코드가 `FileDescriptor` 타입에 `Sendable` 준수를 추가하여 확장할 수 있습니다. 반면에 이 예시의 비가용성 확장은 `Sendable`의 암시적 준수를 억제할 뿐만 아니라 코드의 다른 곳에 있는 어떤 확장도 해당 타입에 `Sendable` 준수를 추가하는 것을 방지합니다.

> **참고**
>
> 위에 언급한 프로토콜 외에도 분산 액터(distributed actors)는 `Codable` 프로토콜을 암시적으로 준수합니다.

프로토콜 타입의 컬렉션 (Collections of Protocol Types)

프로토콜은 프로토콜을 타입으로 사용 (Protocols as Types)에서 언급했듯이 배열이나 딕셔너리와 같은 컬렉션의 타입으로 사용할 수 있습니다. 이 예시는 `TextRepresentable`에 대한 배열을 생성합니다:

```
let things: [TextRepresentable] = [game, d12, simonTheHamster]
```

이제 배열에 항목을 반복할 수 있고, 각 항목의 설명을 출력할 수 있습니다:

```
for thing in things {
    print(thing.textualDescription)
}
// A game of Snakes and Ladders with 25 squares
// A 12-sided dice
// A hamster named Simon
```

`thing` 상수는 `TextRepresentable` 타입입니다. 실제 인스턴스가 `Dice`, `DiceGame`, `Hamster` 중 하나 이지만 이 타입은 아닙니다. 그럼에도 불구하고 `TextRepresentable` 타입이므로, `TextRepresentable`은

`textualDescription` 프로퍼티를 가지고 있다는 것을 알고 있으므로 루프를 통해 `thing.textualDescription`에 안전하게 접근할 수 있습니다.

프로토콜 상속 (Protocol Inheritance)

프로토콜은 하나 이상의 다른 프로토콜을 상속할 수 있고, 상속받은 요구사항에 새로운 요구사항을 추가할 수 있습니다. 프로토콜 상속의 문법은 클래스 상속 문법과 유사하지만, 여러 프로토콜을 콤마로 구분하여 나열할 수 있습니다:

```
protocol InheritingProtocol: SomeProtocol, AnotherProtocol {
    // protocol definition goes here
}
```

다음은 위의 `TextRepresentable` 프로토콜을 상속하는 프로토콜의 예입니다:

```
protocol PrettyTextRepresentable: TextRepresentable {
    var prettyTextualDescription: String { get }
}
```

이 예시는 `TextRepresentable`을 상속하는 `PrettyTextRepresentable`이라는 새로운 프로토콜을 정의합니다. `PrettyTextRepresentable`을 채택하는 타입은 `TextRepresentable`의 모든 요구사항을 만족해야 하며, 추가로 `PrettyTextRepresentable`의 모든 요구사항도 만족해야 합니다. 이 예시에서 `PrettyTextRepresentable`은 `String`을 반환하는 `prettyTextualDescription`이라는 읽기 전용 프로퍼티 요구사항을 추가합니다.

`SnakesAndLadders` 클래스는 확장을 통해 `PrettyTextRepresentable`을 채택하고 준수할 수 있습니다:

```
extension SnakesAndLadders: PrettyTextRepresentable {
    var prettyTextualDescription: String {
        var output = textualDescription + ":\n"
        for index in 1...finalSquare {
            switch board[index] {
            case let ladder where ladder > 0:
                output += "▲ "
            case let snake where snake < 0:
                output += "▼ "
            default:
                output += "○ "
            }
        }
        return output
    }
}
```

이 확장은 `PrettyTextRepresentable` 프로토콜을 채택하고 `SnakesAndLadders` 타입에 대해 `prettyTextualDescription` 프로퍼티의 구현을 제공한다고 나타냅니다. `PrettyTextRepresentable`을 채택한 타입은 `TextRepresentable`도 채택해야 하므로, `prettyTextualDescription`의 구현은 출력 문자열을 위해 `TextRepresentable` 프로토콜에서 `textualDescription` 프로퍼티를 접근하는 것으로 시작합니다. 콜론과 줄 바꿈을 추가하고 정리된 텍스트 표현을 시작으로 사용합니다. 그런 다음 보드 사각형의 배열을 통해 반복하고, 각 사각형의 내용을 나타내기 위해 기호를 추가합니다:

- 사각형의 값이 `0`보다 크면, 사다리의 밑부분이 되고 ▲로 표시합니다.

- 사각형의 값이 `0`보다 작으면, 뱀의 머리이고 ▼으로 표시합니다.
- 사각형의 값이 `0`이고 "자유" 정사각형이면, ○으로 표시합니다.

`prettyTextualDescription` 프로퍼티는 이제 모든 `SnakesAndLadders` 인스턴스의 텍스트 설명을 출력하기 위해 사용할 수 있습니다:

```
print(game.prettyTextualDescription)
// A game of Snakes and Ladders with 25 squares:
// ○ ○ ▲ ○ ○ ▲ ○ ○ ▲ ▲ ○ ○ ○ ▼ ○ ○ ○ ○ ▼ ○ ○ ▼ ○ ▼ ○
```

클래스 전용 프로토콜 (Class-Only Protocols)

프로토콜 상속 목록에 `AnyObject` 프로토콜을 추가하여 해당 프로토콜을 클래스 타입(구조체나 열거형이 아닌)에서만 채택할 수 있게 제한할 수 있습니다.

```
protocol SomeClassOnlyProtocol: AnyObject, SomeInheritedProtocol {
    // class-only protocol definition goes here
}
```

위의 예시에서 `SomeClassOnlyProtocol`은 클래스 타입에서만 채택할 수 있습니다.
`SomeClassOnlyProtocol`을 구조체나 열거형에 채택하면 컴파일 오류가 발생합니다.

> **참고**
>
> 프로토콜 요구사항이 참조 타입의 특성을 필요로 하거나 기대하는 경우 클래스 전용 프로토콜을 사용합니다. 참조 타입과 값 타입에 대한 자세한 내용은 구조체와 열거형은 값 타입 (Structures and Enumerations Are Value Types)과 클래스는 참조 타입 (Classes Are Reference Types)을 참고바랍니다.

프로토콜의 상속 목록에 특정 클래스를 지정하여 프로토콜을 준수하는 모든 타입이 해당 클래스의 하위 클래스여야 한다고 요구할 수도 있습니다.

```
class Renderer {
    func render(_ text: String) {
        print(text)
    }
}

protocol Widget: Renderer {
    var title: String { get }
}

extension Widget {
    func display() {
        render(title)
    }
}

class ButtonWidget: Renderer, Widget {
    var title: String
    init(title: String) {
        self.title = title
    }
}
```

```
let button = ButtonWidget(title: "OK")
button.display()
// Prints "OK".
```

위 예시에서 `Widget`은 `Renderer`의 하위 클래스만 채택할 수 있습니다. 프로토콜이 모든 준수 타입을 `Renderer`의 하위 클래스로 보장하므로, `Widget` 프로토콜의 확장에서 `render(_:)`를 직접 호출할 수 있습니다. 상위 클래스 요구사항은 `AnyObject`를 사용하는 것보다 더 엄격합니다: 모든 클래스 타입을 허용하는 대신 특정 클래스 계층으로 준수를 제한합니다. 프로토콜의 구현이 특정 기본 클래스의 구체적인 동작에 의존하는 경우에 사용합니다.

프로토콜 합성 (Protocol Composition)

동시에 여러 프로토콜을 준수하는 타입을 요구하는 것이 유용할 수 있습니다. *프로토콜 합성(protocol composition)*을 사용하여 여러 프로토콜을 단일 요구사항으로 결합할 수 있습니다. 프로토콜 합성은 모든 프로토콜의 요구사항을 가진 임시 로컬 프로토콜을 정의한 것처럼 동작합니다. 프로토콜 합성은 새로운 프로토콜 타입을 정의하지 않습니다.

프로토콜 합성은 `SomeProtocol & AnotherProtocol` 형태입니다. 앰퍼샌드(&)로 구분하여 많은 프로토콜을 나열할 수 있습니다. 프로토콜 목록 외에도 프로토콜 합성은 클래스 타입 하나를 포함시켜 필수 상위 클래스를 지정할 수도 있습니다.

다음은 `Named`와 `Aged`라는 두 프로토콜을 함수 매개변수에 단일 요구사항으로 결합한 예입니다:

```
protocol Named {
    var name: String { get }
}
protocol Aged {
    var age: Int { get }
}
struct Person: Named, Aged {
    var name: String
    var age: Int
}
func wishHappyBirthday(to celebrator: Named & Aged) {
    print("Happy birthday, \(celebrator.name), you're \(celebrator.age)!")
}
let birthdayPerson = Person(name: "Malcolm", age: 21)
wishHappyBirthday(to: birthdayPerson)
// Prints "Happy birthday, Malcolm, you're 21!".
```

이 예시에서 `Named` 프로토콜은 `name`이라는 읽기 전용 `String` 프로퍼티인 단일 요구사항을 가지고 있습니다. `Aged` 프로토콜은 `age`라는 읽기 전용 `Int` 프로퍼티인 단일 요구사항을 가지고 있습니다. 두 프로토콜 모두 `Person` 구조체에서 채택합니다.

예시는 `wishHappyBirthday(to:)` 함수도 정의합니다. `celebrator` 매개변수의 타입은 "`Named`와 `Aged` 프로토콜 모두 준수하는 타입"이라는 의미인 `Named & Aged`입니다. 요구된 프로토콜 모두 준수하는 한 함수에 전달하는 특정 타입은 중요하지 않습니다.

그런 다음 `birthdayPerson`이라는 새로운 `Person` 인스턴스를 생성하고, `wishHappyBirthday(to:)` 함수에 새로운 인스턴스를 전달합니다. `Person`은 프로토콜을 모두 준수하기 때문에 이 호출은 유효하고 `wishHappyBirthday(to:)` 함수는 생일 메세지를 출력할 수 있습니다.

다음은 `Location` 클래스와 이전 예시에서의 `Named` 프로토콜을 결합한 예입니다:

```
class Location {
    var latitude: Double
    var longitude: Double
    init(latitude: Double, longitude: Double) {
        self.latitude = latitude
        self.longitude = longitude
    }
}
class City: Location, Named {
    var name: String
    init(name: String, latitude: Double, longitude: Double) {
        self.name = name
        super.init(latitude: latitude, longitude: longitude)
    }
}
func beginConcert(in location: Location & Named) {
    print("Hello, \(location.name)!")
}

let seattle = City(name: "Seattle", latitude: 47.6, longitude: -122.3)
beginConcert(in: seattle)
// Prints "Hello, Seattle!".
```

`beginConcert(in:)` 함수는 "`Location`의 하위 클래스이면서 `Named` 프로토콜을 준수하는 모든 타입"이라는 의미의 `Location & Named` 타입의 매개변수를 가집니다. 이 경우 `City`는 이 요구사항에 충족합니다.

`Person`은 `Location`의 하위 클래스가 아니므로, `beginConcert(in:)` 함수로 `birthdayPerson` 전달은 유효하지 않습니다. 마찬가지로 `Named` 프로토콜을 준수하지 않고 `Location`의 하위 클래스를 만들어 타입의 인스턴스로 `beginConcert(in:)`을 호출하는 것도 유효하지 않습니다.

프로토콜 준수 여부 검사 (Checking for Protocol Conformance)

프로토콜 준수 여부를 확인하거나 특정 프로토콜로 캐스팅 하기 위해 타입 캐스팅 (Type Casting)에서 설명한 `is`와 `as` 연산자를 사용할 수 있습니다. 프로토콜을 확인하고 캐스팅하는 것은 타입을 확인하고 캐스팅하는 것과 정확하게 같은 문법을 따릅니다:

- `is` 연산자는 인스턴스가 프로토콜을 준수한다면 `true`를 반환하고, 그렇지 않으면 `false`를 반환합니다.
- 다운캐스트 연산자의 `as?` 버전은 프로토콜의 타입의 옵셔널 값을 반환하고 인스턴스가 프로토콜을 준수하지 않으면 `nil`을 반환합니다.
- 다운캐스트 연산자의 `as!` 버전은 프로토콜 타입으로 강제로 다운캐스트하고 다운캐스트가 성공하지 못하면 런타임 오류가 발생합니다.

아래 예시는 `area`라는 읽기 전용 `Double` 프로퍼티의 단일 프로퍼티 요구사항을 가지는 `HasArea`라는 프로토콜을 정의합니다:

```
protocol HasArea {
    var area: Double { get }
}
```

다음은 `HasArea` 프로토콜을 준수하는 `Circle`과 `Country`인 두 개의 클래스 입니다:

```
class Circle: HasArea {
    let pi = 3.1415927
    var radius: Double
    var area: Double { return pi * radius * radius }
    init(radius: Double) { self.radius = radius }
}
class Country: HasArea {
    var area: Double
    init(area: Double) { self.area = area }
}
```

`Circle` 클래스는 저장 프로퍼티 `radius`를 기반으로 연산 프로퍼티 `area`를 구현합니다. `Country` 클래스는 저장 프로퍼티로 직접 `area` 요구사항을 구현합니다. 두 클래스 모두 `HasArea` 프로토콜을 준수합니다.

다음은 `HasArea` 프로토콜을 준수하지 않는 `Animal`이라는 클래스 입니다:

```
class Animal {
    var legs: Int
    init(legs: Int) { self.legs = legs }
}
```

`Circle`, `Country`, `Animal` 클래스는 공통 상위 클래스가 없습니다. 그럼에도 불구하고 모두 클래스이므로 이 세 가지 타입의 인스턴스는 타입 `AnyObject`의 값을 저장하는 배열에 저장할 수 있습니다:

```
let objects: [AnyObject] = [
    Circle(radius: 2.0),
    Country(area: 243_610),
    Animal(legs: 4)
]
```

`objects` 배열은 2의 반지름을 가진 `Circle` 인스턴스, 제곱 킬로미터 단위로 영국의 면적으로 초기화 된 `Country` 인스턴스, 네 개의 다리를 가진 `Animal` 인스턴스를 포함하는 배열 리터럴로 초기화 됩니다.

`objects` 배열은 이제 반복될 수 있고, 배열의 각 객체는 `HasArea` 프로토콜을 준수하는지 확인할 수 있습니다:

```
for object in objects {
    if let objectWithArea = object as? HasArea {
        print("Area is \(objectWithArea.area)")
    } else {
        print("Something that doesn't have an area")
    }
}
// Area is 12.5663708
// Area is 243610.0
// Something that doesn't have an area
```

배열의 객체가 `HasArea` 프로토콜을 준수하면, `as?` 연산자가 반환하는 옵셔널 값은 `objectWithArea`라는 상수에 옵셔널 바인딩으로 언래핑합니다. `objectWithArea` 상수는 `HasArea` 타입으로 알고 있으므로, `area` 프로퍼티는 접근 가능하고 안전하게 출력할 수 있습니다.

기본 객체는 캐스팅 과정에 의해 변경되지 않습니다. 그것은 계속해서 `Circle`, `Country`, `Animal`입니다. 그러나 `objectWithArea` 상수에 저장될 때, `HasArea` 타입으로만 알고 있으므로, `area` 프로퍼티만 접근 가능합니다.

옵셔널 프로토콜 요구사항 (Optional Protocol Requirements)

프로토콜에 *옵셔널 요구사항(optional requirements)*을 정의할 수 있습니다. 이 요구사항은 프로토콜을 준수하는 타입이 구현할 필요가 없습니다. 옵셔널 요구사항은 프로토콜의 정의에 `optional` 수정자를 앞에 붙입니다. 옵셔널 요구사항은 Objective-C와 상호 운용되는 코드를 작성할 수 있습니다. 프로토콜과 옵셔널 요구사항 모두 `@objc` 속성으로 표시되어야 합니다. `@objc` 프로토콜은 구조체나 열거형에서 채택할 수 없고 클래스에만 채택할 수 있습니다.

옵셔널 요구사항의 메서드나 프로퍼티를 사용할 때, 그 타입은 자동으로 옵셔널이 됩니다. 예를 들어 `(Int) -> String` 타입의 메서드는 `((Int) -> String)?`이 됩니다. 전체 함수 타입은 메서드의 반환값이 아니라 옵셔널로 래핑됩니다.

옵셔널 프로토콜 요구사항은 프로토콜을 준수하는 타입에 요구사항이 구현되어 있지 않을 수 있으므로, 옵셔널 체이닝으로 호출할 수 있습니다. 호출할 때 `someOptionalMethod?(someArgument)`와 같이 메서드의 이름 뒤에 물음표를 작성하여 옵셔널 메서드를 호출합니다. 옵셔널 체이닝에 대한 자세한 내용은 옵셔널 체이닝 (Optional Chaining)에서 확인할 수 있습니다.

다음 예시는 증가하는 값을 제공하기 위해 외부 데이터 소스를 사용하는 `Counter`라는 정수 카운팅 클래스를 정의합니다. 이 데이터 소스는 두 개의 옵셔널 요구사항을 가진 `CounterDataSource` 프로토콜로 정의됩니다:

```
@objc protocol CounterDataSource {
    @objc optional func increment(forCount count: Int) -> Int
    @objc optional var fixedIncrement: Int { get }
}
```

`CounterDataSource` 프로토콜은 `increment(forCount:)`라는 옵셔널 메서드 요구사항과 `fixedIncrement`라는 옵셔널 프로퍼티 요구사항을 정의합니다. 이 요구사항은 `Counter` 인스턴스에 대해 적절한 증가 값을 제공하기 위한 두 가지 방법을 정의합니다.

> **참고**
>
> 엄밀히 말하면 프로토콜 요구사항을 구현하지 않고도 `CounterDataSource`를 준수하는 커스텀 클래스를 작성할 수 있습니다. 둘 다 옵셔널 입니다. 기술적으로는 가능하지만 좋은 데이터 소스로는 적합합지 않습니다.

아래 정의된 `Counter` 클래스는 `CounterDataSource?` 타입의 옵셔널 `dataSource` 프로퍼티를 가지고 있습니다:

```
class Counter {
    var count = 0
    var dataSource: CounterDataSource?
    func increment() {
        if let amount = dataSource?.increment?(forCount: count) {
            count += amount
        } else if let amount = dataSource?.fixedIncrement {
            count += amount
        }
    }
}
```

`Counter` 클래스는 `count`라는 변수 프로퍼티에 현재값을 저장합니다. `Counter` 클래스는 메서드가 호출될 때마다 `count` 프로퍼티를 증가시키는 `increment`라는 메서드도 정의합니다.

`increment()` 메서드는 먼저 데이터 소스의 `increment(forCount:)` 메서드를 통해 증가 값을 조회하려고 합니다. `increment()` 메서드는 `increment(forCount:)` 호출에 대해 옵셔널 체이닝을 사용하고 메서드의 단일 인자로 현재 `count` 값을 전달합니다.

여기서 두 단계 옵셔널 체이닝을 사용합니다. 먼저 `dataSource`는 `nil`일 수 있으므로, `dataSource`가 `nil`이 아닌 경우에만 `increment(forCount:)` 호출해야 된다는 것을 나타내기 위해 `dataSource` 이름 뒤에 물음표를 붙입니다. 두 번째로 옵셔널 요구사항 이므로, `dataSource`가 존재하더라도 `increment(forCount:)`가 구현되어 있다고 보장하지 않습니다. 여기서 `increment(forCount:)`가 구현되어 있지 않다는 가능성을 옵셔널 체이닝으로 처리합니다. `increment(forCount:)`가 존재할 경우에만 `increment(forCount:)` 호출이 이뤄지고 존재하지 않으면 `nil`입니다. 이것이 `increment(forCount:)` 이름 뒤에 물음표가 붙는 이유입니다.

`increment(forCount:)` 호출은 위의 2가지 이유로 실패할 수 있으므로, 이 호출은 항상 옵셔널 `Int` 값을 반환합니다. `increment(forCount:)`가 `CounterDataSource`의 정의에서 옵셔널이 아닌 `Int` 값으로 반환하더라도 마찬가지입니다. 두 개의 옵셔널 체이닝 연산자가 차례로 있지만 결과는 여전히 단일 옵셔널로 래핑됩니다. 여러 개 옵셔널 체이닝 연산자 사용에 대한 자세한 내용은 여러 단계의 체이닝 연결 (Linking Multiple Levels of Chaining)을 참고바랍니다.

`increment(forCount:)` 호출 후에 반환한 옵셔널 `Int`는 옵셔널 바인딩을 사용하여 `amount`라는 상수에 언래핑됩니다. 옵셔널 `Int`에 값이 포함되어 있다면 -- 이것은 위임자와 메서드가 모두 존재하고 메서드가 값을 반환 한 경우 -- 언래핑된 `amount`는 `count` 저장 프로퍼티에 추가하고 증가를 완료합니다.

`dataSource`가 `nil`이거나 데이터 소스가 `increment(forCount:)`를 구현하지 않아 `increment(forCount:)` 메서드로 부터 값을 조회할 수 없는 경우에는 `increment()` 메서드는 데이터 소스의 `fixedIncrement` 프로퍼티를 대신 조회하려고 합니다. `fixedIncrement` 프로퍼티도 옵셔널 요구사항이므로, 그 값은 `fixedIncrement`가 `CounterDataSource` 프로토콜 정의에 옵셔널이 아닌 `Int` 프로퍼티로 정의되었어도 옵셔널 `Int` 값입니다.

다음은 데이터 소스를 매번 조회할 때마다 3의 상수 값을 반환하는 간단한 `CounterDataSource` 구현 입니다. 옵셔널 `fixedIncrement` 프로퍼티 요구사항을 구현하여 이것을 수행합니다:

```
class ThreeSource: NSObject, CounterDataSource {
    let fixedIncrement = 3
}
```

새로운 `Counter` 인스턴스의 데이터 소스로 `ThreeSource` 인스턴스를 사용할 수 있습니다:

```
var counter = Counter()
counter.dataSource = ThreeSource()
for _ in 1...4 {
    counter.increment()
    print(counter.count)
}
// 3
// 6
// 9
// 12
```

위의 코드는 새로운 `Counter` 인스턴스를 생성하고 이 데이터 소스를 새로운 `ThreeSource` 인스턴스로 설정하고 카운터의 `increment()` 메서드를 네 번 호출합니다. 예상대로 카운터의 `count` 프로퍼티는 `increment()`가 호출될 때마다 3씩 증가합니다.

다음은 `Counter` 인스턴스를 현재 `count` 값에서 0으로 증가나 감소시키는 `TowardsZeroSource`라는 더 복잡한 데이터 소스입니다:

```
class TowardsZeroSource: NSObject, CounterDataSource {
    func increment(forCount count: Int) -> Int {
        if count == 0 {
            return 0
        } else if count < 0 {
            return 1
        } else {
            return -1
        }
    }
}
```

`TowardsZeroSource` 클래스는 `CounterDataSource` 프로토콜의 옵셔널 `increment(forCount:)` 메서드를 구현하고 카운트 방향을 정하기 위해 `count` 인자값을 사용합니다. `count`가 0이면 이 메서드는 더이상 카운트 작업을 진행하지 않음을 나타내기 위해 `0`을 반환합니다.

`-4`부터 0까지 카운트 하기 위해 기존 `Counter` 인스턴스에 `TowardsZeroSource`의 인스턴스를 사용할 수 있습니다. 카운터가 0에 도달하면 더이상 카운팅이 동작하지 않습니다:

```
counter.count = -4
counter.dataSource = TowardsZeroSource()
for _ in 1...5 {
    counter.increment()
    print(counter.count)
}
// -3
// -2
// -1
// 0
// 0
```

프로토콜 확장 (Protocol Extensions)

프로토콜은 확장을 통해 메서드, 이니셜라이저, 서브스크립트, 연산 프로퍼티 구현을 준수 타입에 제공할 수 있습니다. 이를 통해 각 타입의 개별 구현이나 전역 함수가 아닌 프로토콜 자체에 동작을 정의할 수 있습니다.

예를 들어 `RandomNumberGenerator` 프로토콜은 확장을 통해 임의의 `Bool` 값을 반환하기 위해 필수 `random()` 메서드의 결과를 사용하는 `randomBool()` 메서드를 제공할 수 있습니다:

```
extension RandomNumberGenerator {
    func randomBool() -> Bool {
        return random() > 0.5
    }
}
```

프로토콜 확장을 생성함으로써 모든 준수 타입은 추가 수정없이 이 메서드를 사용할 수 있습니다.

```
let generator = LinearCongruentialGenerator()
print("Here's a random number: \(generator.random())")
// Prints "Here's a random number: 0.3746499199817101".
print("And here's a random Boolean: \(generator.randomBool())")
// Prints "And here's a random Boolean: true".
```

프로토콜 확장은 준수 타입에 구현을 추가할 수 있지만 다른 프로토콜을 확장하거나 다른 프로토콜을 상속할 수 없습니다. 프로토콜 상속은 항상 프로토콜 선언 자체에서만 지정할 수 있습니다.

기본 구현 제공 (Providing Default Implementations)

프로토콜의 메서드나 연산 프로퍼티 요구사항에 기본 구현을 제공하기 위해 프로토콜 확장을 사용할 수 있습니다. 준수 타입이 필수 메서드나 프로퍼티의 자체 구현을 제공하면, 그 구현은 확장에서 제공하는 기본 구현 대신 사용됩니다.

> **참고**
>
> 확장으로 제공된 기본 구현을 가진 프로토콜 요구사항은 옵셔널 프로토콜 요구사항과 다릅니다. 준수 타입이 자체 구현을 제공하지 않아도 되지만, 기본 구현을 가지므로 옵셔널 체이닝 없이 호출할 수 있습니다.

예를 들어 `TextRepresentable` 프로토콜을 상속하는 `PrettyTextRepresentable` 프로토콜은 `textualDescription` 프로퍼티 접근의 결과를 반환하기 위해 필요한 `prettyTextualDescription` 프로퍼티의 기본 구현을 제공할 수 있습니다:

```
extension PrettyTextRepresentable  {
    var prettyTextualDescription: String {
        return textualDescription
    }
}
```

프로토콜 확장에 제약조건 추가 (Adding Constraints to Protocol Extensions)

프로토콜 확장을 정의할 때, 확장의 메서드와 프로퍼티를 사용할 수 있기 전에 준수 타입이 만족해야 하는 제약조건을 지정할 수 있습니다. 제네릭 `where` 절을 작성하여 확장하는 프로토콜의 이름 뒤에 제약조건을 작성합니다. 제네릭 `where` 절의 자세한 내용은 제네릭 Where 절 (Generic Where Clauses)을 참고바랍니다.

예를 들어 `Equatable` 프로토콜을 준수하는 항목의 모든 컬렉션에 적용하는 `Collection` 프로토콜의 확장을 정의할 수 있습니다. 컬렉션의 요소를 Swift 표준 라이브러리의 `Equatable` 프로토콜로 제한하면 두 요소 간의 같음과 다름에 대한 확인을 위해 ==와 != 연산자를 사용할 수 있습니다.

```
extension Collection where Element: Equatable {
    func allEqual() -> Bool {
        for element in self {
            if element != self.first {
                return false
            }
        }
        return true
    }
}
```

`allEqual()` 메서드는 컬렉션에 모든 요소가 같을 때만 `true`를 반환합니다.

모든 요소가 같고 하나만 다른 정수의 두 배열을 생각해 봅시다:

```
let equalNumbers = [100, 100, 100, 100, 100]
let differentNumbers = [100, 100, 200, 100, 200]
```

배열은 `Collection`을 준수하고 정수는 `Equatable`을 준수하므로, `equalNumbers`와 `differentNumbers`는 `allEqual()` 메서드를 사용할 수 있습니다:

```
print(equalNumbers.allEqual())
// Prints "true".
print(differentNumbers.allEqual())
// Prints "false".
```

참고

준수 타입이 동일한 메서드나 프로퍼티에 대한 구현을 제공하는 여러 제약조건의 확장에 대한 요구사항을 충족한다면 Swift는 가장 특화된 제약조건의 구현을 사용합니다.

언어 가이드

제네릭 (Generics)

여러 타입에 대해 동작하는 코드를 작성하고 해당 타입의 요구사항을 지정합니다.

*제네릭 코드(Generic code)*는 정의한 요구사항에 따라 모든 타입에서 동작할 수 있는 유연하고 재사용 가능한 함수와 타입을 작성할 수 있습니다. 중복을 피하고 명확하고 추상적인 방식으로 의도를 표현하는 코드를 작성할 수 있습니다.

제네릭은 Swift의 강력한 기능 중 하나이고, Swift 표준 라이브러리의 많은 부분이 제네릭 코드로 작성되어 있습니다. 사실 모르고 있더라도 Language Guide 전체에서 제네릭을 사용합니다. 예를 들어 Swift의 `Array`와 `Dictionary` 타입은 둘 다 제네릭 컬렉션 입니다. `Int` 값을 가진 배열이나 `String` 값을 가진 배열 또는 실제로 Swift에서 생성할 수 있는 다른 모든 타입에 대한 배열을 생성할 수 있습니다. 마찬가지로 모든 지정된 타입의 값을 저장하기 위한 딕셔너리를 생성할 수 있고 해당 타입에 대한 제한은 없습니다.

제네릭이 해결하는 문제 (The Problem that Generics Solve)

다음은 두 `Int` 값을 바꾸는 `swapTwoInts(_:_:)`라는 제네릭이 아닌 함수를 나타냅니다:

```
func swapTwoInts(_ a: inout Int, _ b: inout Int) {
    let temporaryA = a
    a = b
    b = temporaryA
}
```

이 함수는 In-Out 매개변수 (In-Out Parameters)에서 설명한대로 a와 b의 값을 바꾸기 위해 in-out 매개변수를 사용하여 만듭니다.

`swapTwoInts(_:_:)` 함수는 b의 값을 a로 그리고 a의 값을 b로 바꿉니다. 두 개의 `Int` 변수의 값을 바꾸기 위해 이 함수를 호출할 수 있습니다:

```
var someInt = 3
var anotherInt = 107
swapTwoInts(&someInt, &anotherInt)
print("someInt is now \(someInt), and anotherInt is now \(anotherInt)")
// Prints "someInt is now 107, and anotherInt is now 3".
```

`swapTwoInts(_:_:)` 함수는 유용하지만 `Int` 값만 사용이 가능합니다. 두 개의 `String` 값이나 두 개의 `Double` 값을 바꾸길 원하면 아래의 `swapTwoStrings(_:_:)`와 `swapTwoDoubles(_:_:)` 함수와 같이 더 많은 함수를 작성해야 합니다:

```
func swapTwoStrings(_ a: inout String, _ b: inout String) {
    let temporaryA = a
    a = b
    b = temporaryA
}

func swapTwoDoubles(_ a: inout Double, _ b: inout Double) {
```

```
    let temporaryA = a
    a = b
    b = temporaryA
}
```

`swapTwoInts(_:_:)`, `swapTwoStrings(_:_:)`, `swapTwoDoubles(_:_:)` 함수의 본문이 동일하다는 것을 알 수 있습니다. 차이점은 받아들이는 값의 타입만 다릅니다 (`Int`, `String`, `Double`).

모든 타입의 두 값을 바꾸는 단일 함수로 작성하면 더 유용하고 유연합니다. 제네릭 코드는 이러한 함수를 작성할 수 있습니다. (이 함수의 제네릭 버전은 아래에 정의되어 있습니다.)

> **참고**
>
> 이 세 개의 함수는 `a`와 `b`의 타입이 모두 같아야 합니다. `a`와 `b`가 같은 타입이 아니면 바꾸는 것은 불가능합니다. Swift는 타입 안정 언어이고 `String` 타입의 변수와 `Double` 타입의 변수가 서로 값을 바꾸도록 허락하지 않습니다. 이러한 시도는 컴파일 오류가 발생합니다.

제네릭 함수 (Generic Functions)

*제네릭 함수(Generic functions)*는 모든 타입과 함께 동작할 수 있습니다. 다음은 `swapTwoValues(_:_:)`라는 위의 `swapTwoInts(_:_:)` 함수의 제네릭 버전입니다:

```
func swapTwoValues<T>(_ a: inout T, _ b: inout T) {
    let temporaryA = a
    a = b
    b = temporaryA
}
```

`swapTwoValues(_:_:)` 함수의 본문은 `swapTwoInts(_:_:)` 함수의 본문과 동일합니다. 그러나 `swapTwoValues(_:_:)`의 첫 번째 줄은 `swapTwoInts(_:_:)`와 약간 다릅니다. 다음은 첫 번째 줄이 어떻게 다른지 보여줍니다:

```
func swapTwoInts(_ a: inout Int, _ b: inout Int)
func swapTwoValues<T>(_ a: inout T, _ b: inout T)
```

함수의 제네릭 버전은 `Int`, `String`, `Double`과 같은 실제 타입 이름 대신에 이 경우 `T`라는 임의의 타입 이름을 사용합니다. 이 임의의 타입 이름 `T`는 어떤 타입인지 나타내지 않지만 `T`가 무슨 타입이든 `a`와 `b`는 모두 같은 타입이어야 한다고 나타냅니다. `T`의 실제 타입은 `swapTwoValues(_:_:)` 함수가 호출될 때마다 결정됩니다.

제네릭 함수와 제네릭이 아닌 함수 사이의 다른 차이점은 제네릭 함수의 이름(`swapTwoValues(_:_:)`)에 바로 임의의 타입 이름(`T`)이 꺾쇠 괄호 내(`<T>`)에 위치한다는 것입니다. 이 괄호는 `T`는 `swapTwoValues(_:_:)` 함수 정의 내에서 임의의 타입 이름이라고 Swift에게 말합니다. `T`는 임의의 타입이므로, Swift는 `T`라는 실제 타입을 찾지 않습니다.

`swapTwoValues(_:_:)` 함수는 이제 `swapTwoInts`와 동일한 방식으로 호출할 수 있지만 두 값이 서로 동일한 타입이면 모든 타입의 두 값을 전달할 수 있다는 점이 다릅니다. `swapTwoValues(_:_:)`가 호출될 때마다 `T`로 사용한 타입은 함수에 전달된 값의 타입으로 추론합니다.

아래의 두 예시에서 `T`는 각각 `Int`와 `String`으로 추론됩니다:

```
var someInt = 3
var anotherInt = 107
swapTwoValues(&someInt, &anotherInt)
// someInt is now 107, and anotherInt is now 3

var someString = "hello"
var anotherString = "world"
swapTwoValues(&someString, &anotherString)
// someString is now "world", and anotherString is now "hello"
```

참고

위에 정의된 `swapTwoValues(_:_:)` 함수는 Swift 표준 라이브러리에 포함된 `swap`이라는 제네릭 함수에 의해 영감을 받았습니다. 자체 코드에서 `swapTwoValues(_:_:)` 함수의 동작이 필요하다면, 직접 구현한 함수보다 Swift에 존재하는 `swap(_:_:)` 함수를 사용할 수 있습니다.

타입 매개변수 (Type Parameters)

위의 `swapTwoValues(_:_:)` 예시에서 임의의 타입 `T`는 *타입 매개변수(type parameter)*의 예입니다. 타입 매개변수는 임의의 타입을 지정하고 이름을 지정하며, 함수 이름 바로 뒤에 꺾쇠 괄호(예: `<T>`) 사이에 작성합니다.

타입 매개변수를 지정하면 함수의 매개변수 타입 (`swapTwoValues(_:_:)` 함수의 `a`와 `b` 매개변수), 함수의 반환 타입, 함수의 본문 내에 타입 어노테이션 등에 사용할 수 있습니다. 각각의 경우 타입 매개변수는 함수가 호출될 때 실제 타입으로 대체됩니다. (위의 `swapTwoValues(_:_:)` 예시에서 `T`는 첫 번째 함수가 호출될 때 `Int`로 대체되고, 두 번째 호출될 때 `String`으로 대체됩니다.)

여러 타입 매개변수가 필요하다면 꺾쇠 괄호 안에 타입 매개변수 이름을 콤마로 구분해 나열할 수 있습니다.

타입 매개변수 이름 (Naming Type Parameters)

대부분의 경우 타입 매개변수는 타입 매개변수와 제네릭 타입 간의 관계나 함수 간의 관계를 나타내기 위해 `Dictionary<Key, Value>`에서 `Key`와 `Value` 그리고 `Array<Element>`에서 `Element`와 같이 설명이 포함된 이름을 사용합니다. 그러나 의미있는 관계가 없을 때는 위의 `swapTwoValues(_:_:)` 함수에서 `T`와 같이 `T`, `U`, `V`와 같은 단일 문자를 사용하여 이름을 지정하는 것이 일반적입니다.

타입 매개변수에 `T`와 `MyTypeParameter`와 같이 대문자 카멜 케이스 이름을 사용하여 값이 아닌 타입을 위한 플레이스 홀더라고 나타냅니다.

참고

타입 매개변수에 이름이 필요하지 않고 제네릭 타입 제약조건이 단순하다면, 불투명 매개변수 타입 (Opaque Parameter Types)에서 설명한대로 경량 문법을 사용할 수 있습니다.

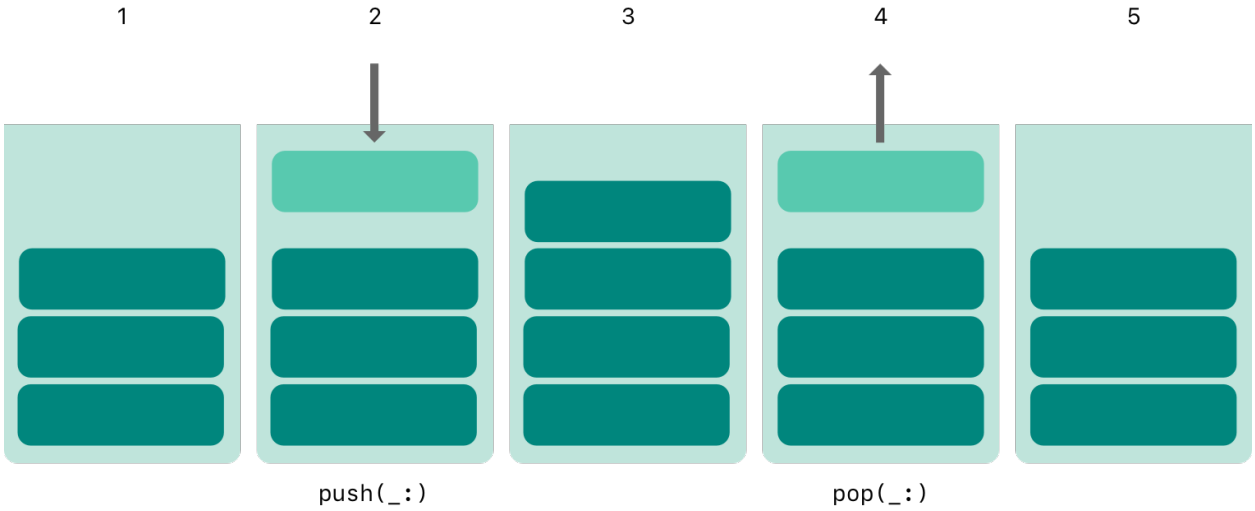
제네릭 타입 (Generic Types)

제네릭 함수 외에도 Swift에서는 *제네릭 타입(generic types)*을 정의할 수 있습니다. 제네릭 타입은 `Array`나 `Dictionary`처럼 모든 타입에서 동작할 수 있는 커스텀 클래스, 구조체, 열거형입니다.

이번 섹션은 `Stack`이라는 제네릭 컬렉션 타입을 어떻게 작성하는지 보여줍니다. 스택은 배열과 유사하게 순서가 있는 값의 집합이지만, Swift의 `Array` 타입보다 더 제한된 연산만 허용합니다. 배열은 모든 위치에 값을 삽입하고 제거할 수 있습니다. 그러나 스택은 값을 컬렉션의 끝에 추가하는 것만 허락합니다 (새로운 값을 *푸시(push)*한다고 표현합니다). 마찬가지로 스택은 컬렉션의 끝부분의 값만 제거할 수 있습니다 (값을 *팝(pop)*한다고 표현합니다).

> **참고**
>
> 스택의 개념은 네비게이션 계층도에서 뷰 컨트롤러를 모델링 하는 `UINavigationController` 클래스에서 사용됩니다. `UINavigationController` 클래스에서 네비게이션 스택에 뷰 컨트롤러를 추가(푸시)하기 위해 `pushViewController(_:animated:)` 메서드를 호출하고 네비게이션 스택에 뷰 컨트롤러를 삭제(팝)하기 위해 `popViewControllerAnimated(_:)` 메서드를 호출합니다. 스택은 컬렉션 관리에서 “후입 선출(LIFO)” 접근방식이 필요할 때 유용한 컬렉션 모델입니다.

아래의 그림은 스택에 대한 푸시와 팝에 대한 동작을 보여줍니다:



1 스택에 현재 세 개의 값이 있습니다.

2 네 번째 값은 스택의 상단에 푸시됩니다.

3 스택은 이제 네 개의 값을 가지며 최근 값이 가장 상단에 위치합니다.

4 스택의 최상단 항목을 팝합니다.

5 값을 팝한 후에 스택은 다시 세 개의 값만 가집니다.

다음은 스택의 제네릭이 아닌 버전을 어떻게 작성하는지 나타내며, 이 경우 `Int` 값의 스택에 대한 것을 보여줍니다:

```
struct IntStack {
    var items: [Int] = []
    mutating func push(_ item: Int) {
        items.append(item)
    }
    mutating func pop() -> Int {
        return items.removeLast()
    }
}
```

이 구조체는 스택에 값을 저장하기 위해 `items`라는 `Array` 프로퍼티를 사용합니다. `Stack`은 스택에 값을 푸시하고 팝하기 위해 `push`와 `pop`인 두 개의 메서드를 제공합니다. 구조체의 `items` 배열을 수정하거나 변경이 필요하므로 이 메서드는 `mutating`으로 표시됩니다.

그러나 위에 `IntStack` 타입은 `Int` 값만 사용할 수 있습니다. 모든 타입의 값으로 스택을 관리할 수 있는 제네릭 `Stack` 구조체를 정의하는 것이 더 유용합니다.

다음은 같은 코드의 제네릭 버전입니다:

```
struct Stack<Element> {
    var items: [Element] = []
    mutating func push(_ item: Element) {
        items.append(item)
    }
    mutating func pop() -> Element {
        return items.removeLast()
    }
}
```

`Stack`의 제네릭 버전은 기본적으로 제네릭이 아닌 버전과 동일하지만, `Int`의 실제 타입 대신에 `Element`라는 타입 매개변수를 사용합니다. 이 타입 매개변수는 구조체의 이름 바로 다음에 꺾쇠 괄호 내에(`<Element>`) 작성합니다.

`Element`는 나중에 제공할 타입에 대한 임의의 이름을 정의합니다. 이 타입은 구조체의 정의 내 어디서나 `Element`로 참조할 수 있습니다. 이 경우에 `Element`는 아래의 세 군데에서 사용됩니다:

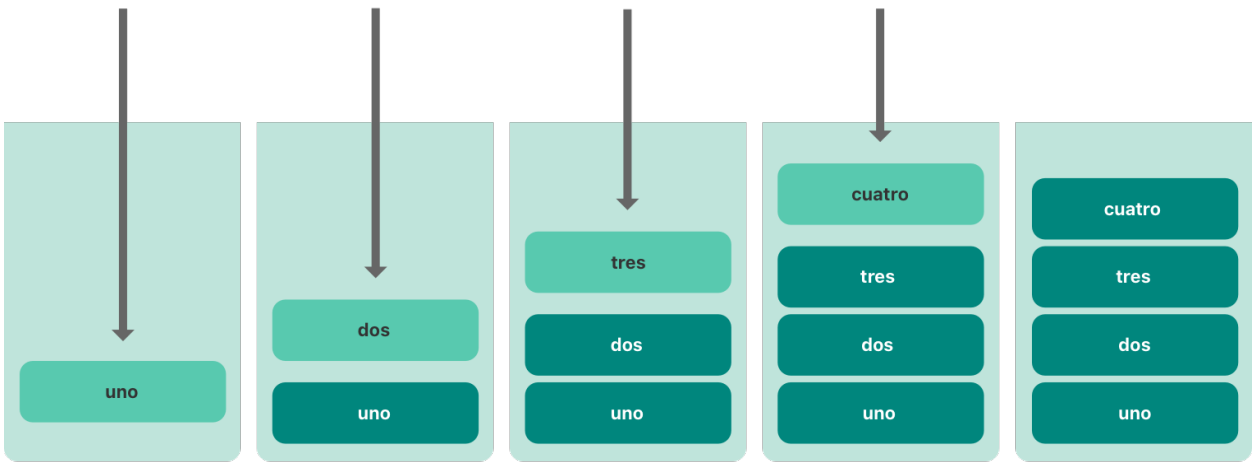
- `items`라는 프로퍼티를 생성할 때, `Element` 타입의 빈 배열로 초기화
- `item`이라는 단일 매개변수를 가지는 `push(_:)` 메서드를 지정할 때 `Element` 타입의 매개변수
- `pop()` 메서드에 의해 반환된 값이 `Element` 타입의 값을 반환

이제 제네릭 타입이므로 `Stack`은 `Array`나 `Dictionary`처럼 Swift에서 모든 유효한 타입의 스택을 생성하기위해 사용할 수 있습니다.

꺾쇠 괄호 내에 스택에 저장될 타입을 작성하여 새로운 `Stack` 인스턴스를 생성합니다. 예를 들어 문자열의 새로운 스택을 생성하려면, `Stack<String>()`이라 작성합니다:

```
var stackOfStrings = Stack<String>()
stackOfStrings.push("uno")
stackOfStrings.push("dos")
stackOfStrings.push("tres")
stackOfStrings.push("cuatro")
// the stack now contains 4 strings
```

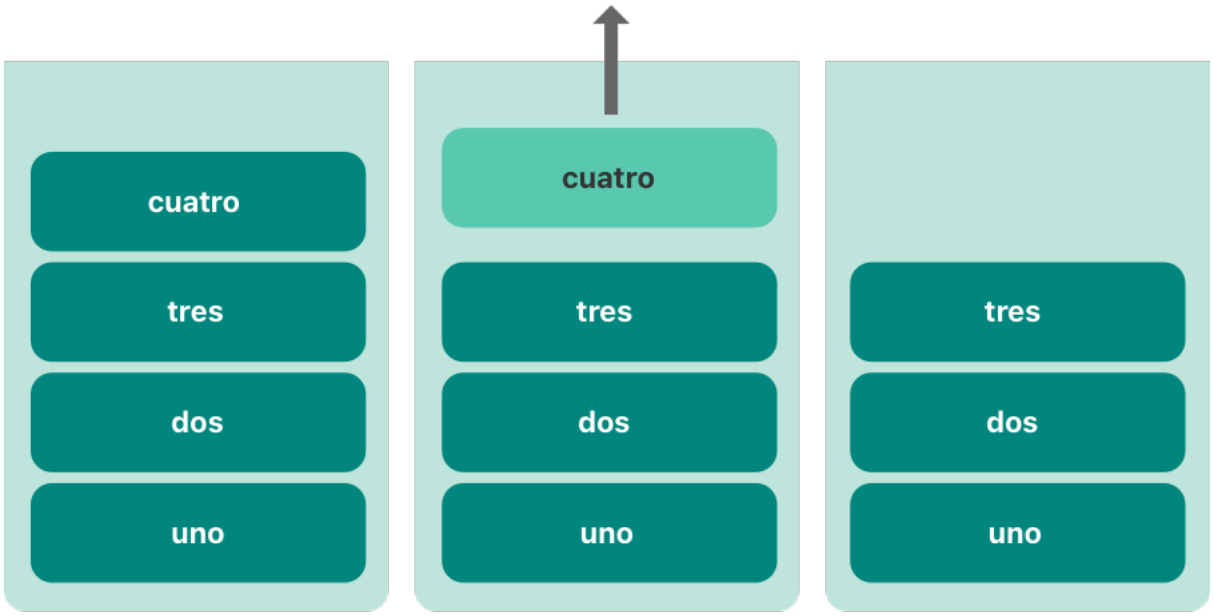
다음은 `stackOfStrings`가 스택에 네 개의 값을 푸쉬한 후를 나타냅니다:



스택에 값을 팝하여 제거하고 반환되는 값은 "cuatro"입니다:

```
 let fromTheTop = stackOfStrings.pop()
// fromTheTop is equal to "cuatro", and the stack now contains 3 strings
```

다음은 스택이 값을 팝한 후를 보여줍니다:



제네릭 타입 확장 (Extending a Generic Type)

제네릭 타입을 확장할 때, 확장의 정의에 타입 매개변수 목록을 제공하지 않습니다. 대신에 기존 타입 정의의 타입 매개변수 목록이 확장의 본문 내에서 사용 가능하고 기존 타입 매개변수 이름을 그대로 사용할 수 있습니다.

다음의 예시는 스택에 팝 없이 스택의 가장 상단의 항목을 반환하는 `topItem`이라는 읽기 전용 연산 프로퍼티를 추가하기 위해 제네릭 `Stack` 타입을 확장합니다:

```
extension Stack {
    var topItem: Element? {
        return items.isEmpty ? nil : items[items.count - 1]
    }
}
```

`topItem` 프로퍼티는 `Element` 타입의 옵셔널 값을 반환합니다. 스택이 비어있다면 `topItem`은 `nil`을 반환하고; 스택이 비어 있지 않다면 `topItem`은 `items` 배열의 마지막 항목을 반환합니다.

이 확장은 타입 매개변수 목록을 정의하지 않습니다. 대신에 `Element`라는 `Stack` 타입의 타입 매개변수 이름을 `topItem` 연산 프로퍼티의 옵셔널 타입으로 확장 내에서 사용합니다.

`topItem` 연산 프로퍼티는 최상단 항목의 삭제없이 접근하고 조회하기 위해 모든 `Stack` 인스턴스에서 사용할 수 있습니다.

```
if let topItem = stackOfStrings.topItem {
    print("The top item on the stack is \(topItem).")
}
// Prints "The top item on the stack is tres.".
```

제네릭 타입의 확장은 아래 제네릭 Where 절이 있는 확장 (Extensions with a Generic Where Clause)에서 설명했듯이 확장된 타입의 인스턴스는 새로운 기능을 얻기 위해 충족해야 할 요구사항도 포함할 수 있습니다.

타입 제약 (Type Constraints)

`swapTwoValues(_:_:)` 함수와 `Stack` 타입은 모든 타입에서 동작 가능합니다. 그러나 가끔 제네릭 함수와 제네릭 타입으로 사용할 수 있는 타입에 *타입 제약(type constraints)*을 강제로 포함해야 유용할 수 있습니다. 타입 제약은 타입 매개변수가 특정 클래스를 상속받거나 특정 프로토콜 또는 프로토콜 합성을 준수해야 함을 명시합니다.

예를 들어 Swift의 `Dictionary` 타입은 딕셔너리에 대해 키로 사용할 수 있는 타입에 제한을 둡니다. 딕셔너리 (Dictionaries)에서 설명했듯이, 딕셔너리 키의 타입은 hashable이어야 합니다. 즉, 고유하게 표현할 수 있는 방법을 제공해야 합니다. `Dictionary`는 키가 이미 포함되어 있는지 확인할 수 있도록 키가 hashable이어야 합니다. 이 요구사항이 없으면 `Dictionary`는 특정 키에 대해 값을 삽입이나 대체해야 하는지 알 수 없으며 이미 딕셔너리에 있는 주어진 키에 대한 값을 찾을 수 없습니다.

이 요구사항은 `Dictionary`의 키 타입에서 타입 제약으로 강제되고, 키 타입이 Swift 표준 라이브러리에 정의된 특별한 프로토콜인 `Hashable` 프로토콜을 준수해야 함을 명시합니다. Swift의 기본 타입(`String`, `Int`, `Double`, `Bool` 등)은 기본적으로 hashable입니다. `Hashable` 프로토콜을 준수하는 자체 커스텀 타입을 만드는 것에 대한 자세한 내용은 Hashable 프로토콜 준수 (Conforming to the Hashable Protocol)를 참고바랍니다.

커스텀 제네릭 타입을 생성할 때 고유 타입 제약을 정의할 수 있고, 이러한 제약은 제네릭 프로그래밍에 많은 기능을 제공합니다. `Hashable`같은 추상 개념은 구체적인 타입이 아닌 개념적 특성 측면에서 타입을 특성화 합니다.

타입 제약 문법 (Type Constraint Syntax)

타입 제약은 타입 매개변수 이름 뒤에 콜론과 함께 클래스나 프로토콜 제약을 작성하여 타입 매개변수 목록에 포함합니다. 제네릭 함수에서 타입 제약에 대한 기본 문법은 아래와 같습니다 (제네릭 타입에도 동일하게 적용됩니다):

```
func someFunction<T: SomeClass, U: SomeProtocol>(someT: T, someU: U) {
    // function body goes here
}
```

위의 가상 함수는 두 개의 타입 매개변수를 가집니다. 첫 번째 타입 매개변수인 `T`는 `T`가 `SomeClass`의 하위 클래스여야 함을 나타냅니다. 두 번째 타입 매개변수 `U`는 `U`가 `SomeProtocol` 프로토콜을 준수해야하는 타입 제약이 있습니다.

타입 제약 동작 (Type Constraints in Action)

다음은 `findIndex(ofString:in:)`이라는 제네릭이 아닌 함수이며, 이것은 찾을 `String` 값과 찾을 `String` 값의 배열을 인자로 받습니다. `findIndex(ofString:in:)` 함수는 배열에서 일치하는 첫 번째 문자열의 인덱스를 옵셔널 `Int`로 반환하거나, 찾지 못하면 `nil`을 반환합니다:

```
func findIndex(ofString valueToFind: String, in array: [String]) -> Int? {
    for (index, value) in array.enumerated() {
        if value == valueToFind {
            return index
        }
    }
    return nil
}
```

`findIndex(ofString:in:)` 함수는 문자열 배열에서 문자열 값을 찾기 위해 사용할 수 있습니다:

```
let strings = ["cat", "dog", "llama", "parakeet", "terrapin"]
if let foundIndex = findIndex(ofString: "llama", in: strings) {
    print("The index of llama is \(foundIndex)")
}
// Prints "The index of llama is 2".
```

그러나 배열에서 값의 인덱스를 찾는 원리는 문자열뿐 아니라 어떤 타입에도 적용할 수 있습니다. 문자열 대신 `T` 타입을 사용하여 동일한 기능을 제네릭 함수로 작성할 수 있습니다.

다음은 `findIndex(of:in:)`이라는 `findIndex(ofString:in:)`의 제네릭 버전을 나타냅니다. 이 함수는 배열의 옵셔널 값이 아닌 옵셔널 인덱스를 반환하므로 반환 타입이 여전히 `Int?`입니다. 하지만 이 함수는 아래 설명된 이유로 컴파일되지 않습니다:

```
func findIndex<T>(of valueToFind: T, in array:[T]) -> Int? {
    for (index, value) in array.enumerated() {
        if value == valueToFind {
            return index
        }
    }
    return nil
}
```

이 함수는 위와 같이 작성하면 컴파일되지 않습니다. 이 문제는 "`if value == valueToFind`"에서 발생합니다. Swift의 모든 타입이 동등 연산자(==)로 비교할 수 있는 것은 아닙니다. 예를 들어 복잡한 데이터 모델을 표현하기 위해 클래스나 구조체를 생성한다면 해당 클래스나 구조체에 대해 "같음"의 의미는 Swift가 자동으로 추론할 수 없습니다. 이로 인해 이 코드가 모든 타입의 `T`에 대해 작동한다고 보장할 수 없으므로 컴파일 오류가 발생합니다.

그러나 Swift 표준 라이브러리는 `Equatable`이라는 프로토콜을 정의하며, 이 프로토콜을 준수하는 타입은 두 개의 값을 비교하기 위한 동등 연산자(==)와 비동등 연산자(!=)를 구현해야 합니다. Swift의 표준 타입은 `Equatable` 프로토콜을 자동으로 지원합니다.

`Equatable`인 모든 타입은 동등 연산자를 지원하기 때문에 `findIndex(of:in:)` 함수에서 안전하게 사용할 수 있습니다. 이 사실을 표현하기 위해 함수를 정의할 때 타입 매개변수의 정의에 `Equatable` 타입 제약을 작성합니다:

```
func findIndex<T: Equatable>(of valueToFind: T, in array:[T]) -> Int? {
    for (index, value) in array.enumerated() {
        if value == valueToFind {
            return index
        }
    }
    return nil
}
```

`findIndex(of:in:)`의 단일 타입 매개변수는 "`T` 타입은 `Equatable` 프로토콜을 준수합니다"라는 의미로 `T: Equatable`로 작성됩니다.

이제 `findIndex(of:in:)` 함수는 정상적으로 컴파일되고 `Double`이나 `String`과 같이 `Equatable`을 준수하는 타입에 사용할 수 있습니다:

```
let doubleIndex = findIndex(of: 9.3, in: [3.14159, 0.1, 0.25])
// doubleIndex is an optional Int with no value, because 9.3 isn't in the array
let stringIndex = findIndex(of: "Andrea", in: ["Mike", "Malcolm", "Andrea"])
// stringIndex is an optional Int containing a value of 2
```

연관 타입 (Associated Types)

프로토콜을 정의할 때, 프로토콜의 정의의 부분으로 하나 이상의 연관 타입(associated types)을 선언하는게 더 유용한 경우가 있습니다. *연관 타입(associated type)*은 프로토콜의 부분으로 사용되는 타입의 임의의 이름을 제공합니다. 연관 타입에 사용할 실제 타입은 프로토콜을 채택할 때까지 정해지지 않습니다. 연관 타입은 `associatedtype` 키워드로 지정합니다.

연관 타입의 동작 (Associated Types in Action)

다음은 `Item`이라는 연관 타입을 선언하는 `Container`라는 프로토콜의 예입니다:

```
protocol Container {
    associatedtype Item
    mutating func append(_ item: Item)
    var count: Int { get }
    subscript(i: Int) -> Item { get }
}
```

`Container` 프로토콜은 모든 컨테이너가 제공해야 하는 세 가지 필수 요건을 정의합니다:

- `append(_:)` 메서드를 사용하여 컨테이너에 새로운 항목을 추가할 수 있어야 합니다.
- `Int` 값을 반환하는 `count` 프로퍼티를 통해 컨테이너 항목의 카운트에 접근할 수 있어야 합니다.
- `Int` 인덱스 값을 사용하는 서브스크립트로 컨테이너의 각 항목을 조회할 수 있어야 합니다.

이 프로토콜은 컨테이너의 항목이 어떻게 저장되는지, 어떤 타입이어야 하는지 지정하지 않습니다. 이 프로토콜은 이 세 가지 기능만 제공하면 `Container`로 간주할 수 있습니다. 준수하는 타입은 이 세 가지 요구사항을 만족하는 한, 추가적인 기능을 제공할 수 있습니다.

`Container` 프로토콜을 준수하는 모든 타입은 저장하는 값의 타입을 지정할 수 있어야 합니다. 특히 올바른 타입의 항목만 컨테이너에 추가되야 하며, 서브스크립트에 의해 반환된 항목의 타입도 명확해야 합니다.

이러한 요구사항을 정의하기 위해 `Container` 프로토콜은 컨테이너에 대한 타입이 무엇인지 알지 못해도 컨테이너가 보유할 항목의 타입을 참조할 방법이 필요합니다. `Container` 프로토콜은 `append(_:)` 메서드에 전달된 모든 값이 컨테이너의 요소 타입과 같은 타입을 가져야 하며, 컨테이너의 서브스크립트에 의해 반환된 값은 컨테이너의 요소 타입과 같은 타입으로 지정해야 합니다.

이를 달성하기 위해 `Container` 프로토콜은 `associatedtype Item`으로 작성한 `Item`이라는 연관 타입을 선언합니다. 이 프로토콜은 `Item`이 무엇인지 정의하지 않고 - 해당 정보는 모든 준수하는 타입이 제공할 수 있도록 남겨둡니다. 그럼에도 불구하고 `Item` 별칭은 `Container`의 항목 타입을 참조하고 `append(_:)` 메서드와 서브스크립트에서 사용하는 타입을 정의하고 모든 `Container`의 동작이 예상되도록 제공합니다.

다음은 위의 제네릭 타입 (Generic Types)에서 제네릭이 아닌 `IntStack` 타입을 `Container` 프로토콜에 맞게 수정한 버전입니다:

```
struct IntStack: Container {
    // original IntStack implementation
    var items: [Int] = []
    mutating func push(_ item: Int) {
        items.append(item)
    }
    mutating func pop() -> Int {
        return items.removeLast()
    }
    // conformance to the Container protocol
    typealias Item = Int
    mutating func append(_ item: Int) {
        self.push(item)
    }
    var count: Int {
        return items.count
    }
    subscript(i: Int) -> Int {
        return items[i]
    }
}
```

`IntStack` 타입은 `Container` 프로토콜의 세 가지 요구사항을 모두 구현하고, 이 요구사항을 만족하기 위해 `IntStack` 타입의 기존 기능을 래핑합니다.

또한 `IntStack`은 `Container`의 구현에 대해 `Item`이 `Int` 타입임을 지정합니다. `typealias Item = Int`의 정의는 `Container` 프로토콜의 구현에 대해 `Item`의 추상적 타입에서 `Int`의 구체적인 타입으로 변환합니다.

Swift의 타입 추론 덕분에 실제로 `IntStack`의 정의에 `Item`의 구체적인 타입인 `Int`를 선언할 필요가 없습니다. `IntStack`은 `Container` 프로토콜의 모든 요구사항을 준수하기 때문에, Swift는 `append(_:)` 메서드의 `item` 매개변수의 타입과 서브스크립트의 반환 타입에서 사용하는 `Item`을 추론할 수 있습니다. 실제로 위의 코드에서 `typealias Item = Int` 줄을 지워도 `Item`에 대해 무슨 타입을 사용하는지 명확하기 때문에 동작에는 이상이 없습니다.

`Container` 프로토콜을 준수하는 제네릭 `Stack` 타입도 만들 수도 있습니다:

```
struct Stack<Element>: Container {
    // original Stack<Element> implementation
    var items: [Element] = []
    mutating func push(_ item: Element) {
        items.append(item)
    }
    mutating func pop() -> Element {
        return items.removeLast()
    }
    // conformance to the Container protocol
    mutating func append(_ item: Element) {
        self.push(item)
    }
    var count: Int {
        return items.count
    }
    subscript(i: Int) -> Element {
        return items[i]
    }
}
```

이번에는 타입 매개변수 `Element`는 `append(_:)` 메서드의 `item` 매개변수와 서브스크립트의 반환 타입으로 사용합니다. 따라서 Swift는 `Element`가 특정 컨테이너에 대해 `Item`으로 사용하는 적절한 타입이라고 추론할 수 있습니다.

기존 타입을 확장하여 연관 타입 지정 (Extending an Existing Type to Specify an Associated Type)

확장으로 프로토콜 준수 추가 (Adding Protocol Conformance with an Extension)에서 설명한대로 프로토콜의 준수성을 추가하기 위해 기존 타입을 확장할 수 있습니다. 여기에는 연관 타입이 있는 프로토콜을 포함합니다.

Swift의 `Array` 타입은 이미 `append(_:)` 메서드, `count` 프로퍼티, `Int` 인덱스로 요소를 조회하는 서브스크립트를 제공합니다. 이 세 가지 기능은 `Container` 프로토콜의 요구사항과 일치합니다. 이것은 `Array`가 `Container` 프로토콜을 채택하도록 선언하는 것으로 간단하게 `Container` 프로토콜을 준수하도록 `Array`를 확장할 수 있다는 의미입니다. 확장으로 프로토콜 채택 선언 (Declaring Protocol Adoption with an Extension)에서 설명했듯이 빈 확장을 사용하여 이 작업을 수행할 수 있습니다:

```
extension Array: Container {}
```

배열의 기존 `append(_:)` 메서드와 서브스크립트는 위의 제네릭 `Stack` 타입과 같이 `Item`에 대해 사용할 적절한 타입을 추론할 수 있습니다. 확장 정의 후에 모든 `Array`는 `Container`로 사용할 수 있습니다.

연관 타입에 제약 추가 (Adding Constraints to an Associated Type)

타입 제약으로 연관 타입을 추가하여 프로토콜을 준수하는 타입이 해당 제약을 만족하도록 할 수 있습니다. 예를 들어 다음의 코드는 컨테이너의 항목이 `Equatable`을 준수해야 하는 것을 요구하는 `Container`의 버전을 정의합니다.

```
protocol Container {
    associatedtype Item: Equatable
    mutating func append(_ item: Item)
    var count: Int { get }
    subscript(i: Int) -> Item { get }
}
```

이 `Container` 버전을 준수하려면 컨테이너의 `Item` 타입은 `Equatable` 프로토콜을 준수해야 합니다.

연관 타입의 제약조건에서 프로토콜 사용 (Using a Protocol in Its Associated Type's Constraints)

프로토콜은 자신의 요구사항으로 자신을 사용할 수도 있습니다. 예를 들어 다음은 `suffix(_:)` 메서드의 요구사항을 추가한 `Container` 프로토콜입니다. `suffix(_:)` 메서드는 컨테이너 끝에서 지정된 숫자 만큼의 `Suffix` 타입의 인스턴스에 저장한 요소를 반환합니다.

```
protocol SuffixableContainer: Container {
    associatedtype Suffix: SuffixableContainer where Suffix.Item == Item
    func suffix(_ size: Int) -> Suffix
}
```

이 프로토콜에서 `Suffix`는 위의 `Container` 예시에서 `Item` 타입과 같은 연관 타입입니다. `Suffix`는 두 개의 제약을 가지고 있습니다: `SuffixableContainer` 프로토콜을 준수해야 하며 (현재 정의되어 있는 프로토콜입니다), `Item` 타입은 컨테이너의 `Item` 타입과 동일해야 합니다. `Item`에 대한 제약은 아래의 제네릭 Where 절이 있는 연관 타입 (Associated Types with a Generic Where Clause)에서 설명할 제네릭 `where` 절입니다.

다음은 `SuffixableContainer` 프로토콜의 준수성을 추가하는 위 제네릭 타입 (Generic Types)의 `Stack` 타입의 확장입니다:

```
extension Stack: SuffixableContainer {
    func suffix(_ size: Int) -> Stack {
        var result = Stack()
        for index in (count-size)..<count {
            result.append(self[index])
        }
        return result
    }
    // Inferred that Suffix is Stack.
}
var stackOfInts = Stack<Int>()
stackOfInts.append(10)
stackOfInts.append(20)
stackOfInts.append(30)
let suffix = stackOfInts.suffix(2)
// suffix contains 20 and 30
```

위의 예시에서 `Stack`에 대한 `Suffix` 연관 타입도 `Stack`이므로 `Stack`에서 접미사 연산자는 다른 `Stack`을 반환합니다. 또한 `SuffixableContainer`를 준수하는 타입은 자체와 다른 `Suffix` 타입을 가질 수 있습니다 - 접미사 연산자는 다른 타입을 반환할 수 있다는 의미입니다. 예를 들어 다음은 `IntStack` 대신 `Stack<Int>`를 접미사 타입으로 사용하여 `SuffixableContainer` 준수성을 추가하는 제네릭이 아닌 `IntStack` 타입에 대한 확장입니다:

```
extension IntStack: SuffixableContainer {
    func suffix(_ size: Int) -> Stack<Int> {
        var result = Stack<Int>()
        for index in (count-size)..<count {
            result.append(self[index])
        }
        return result
    }
    // Inferred that Suffix is Stack<Int>.
}
```

제네릭 Where 절 (Generic Where Clauses)

타입 제약 (Type Constraints)에서 설명한 타입 제약을 사용하면, 제네릭 함수, 서브스크립트, 타입에 연결된 타입 매개변수에 대한 요구사항을 정의할 수 있습니다.

연관 타입에 대한 요구사항을 정의하는 것도 유용할 수 있습니다. *제네릭 where 절(generic where clause)*을 정의하여 이것을 수행할 수 있습니다. 제네릭 `where` 절을 사용하면 연관 타입이 특정 프로토콜을 준수해야 하거나 특정 타입 매개변수와 연관 타입이 동일해야 한다고 요구할 수 있습니다. 제네릭 `where` 절은 `where` 키워드로 시작하고, 이어서 연관 타입이나 타입과 연관 타입 사이의 동등 관계에 대한 제약이 따라옵니다. 타입이나 함수의 본문의 여는 중괄호 바로 전에 제네릭 `where` 절을 작성합니다.

아래의 예시는 두 `Container` 인스턴스가 같은 순서로 같은 항목을 가지고 있는지 확인하는 `allItemsMatch`라는 제네릭 함수를 정의합니다. 이 함수는 모든 항목이 일치하면 `true`의 Boolean 값을 반환하고 그렇지 않으면 `false`의 값을 반환합니다.

확인할 두 컨테이너는 같은 타입의 컨테이너 일 필요는 없지만(같을 수도 있음), 같은 타입의 항목을 가지고 있어야 합니다. 이 요구사항은 타입 제약과 제네릭 `where` 절의 조합으로 표현됩니다:

```
func allItemsMatch<C1: Container, C2: Container>
    (_ someContainer: C1, _ anotherContainer: C2) -> Bool
    where C1.Item == C2.Item, C1.Item: Equatable {

        // Check that both containers contain the same number of items.
        if someContainer.count != anotherContainer.count {
            return false
        }

        // Check each pair of items to see if they're equivalent.
        for i in 0..<someContainer.count {
            if someContainer[i] != anotherContainer[i] {
                return false
            }
        }

        // All items match, so return true.
        return true
}
```

이 함수는 `someContainer`와 `anotherContainer`라는 두 개의 인자를 가집니다. `someContainer` 인자는 `C1` 타입이고, `anotherContainer` 인자는 `C2` 타입입니다. `C1`과 `C2` 모두 함수가 호출될 때 결정되는 두 가지 컨테이너 타입에 대한 타입 매개변수입니다.

함수의 두 타입 매개변수에 대한 요구사항은 다음과 같습니다:

- `C1`은 `Container` 프로토콜을 준수해야 합니다(`C1: Container`).
- `C2`는 `Container` 프로토콜을 준수해야 합니다(`C2: Container`).
- `C1`에 대한 `Item`은 `C2`에 대한 `Item`과 동일해야 합니다 (`C1.Item == C2.Item`).
- `C1`에 대한 `Item`은 `Equatable` 프로토콜을 준수해야 합니다 (`C1.Item: Equatable`).

첫 번째와 두 번째 요구사항은 함수의 타입 매개변수 목록에 정의되고, 세 번째와 네 번째 요구사항은 함수의 제네릭 `where` 절에 정의됩니다.

이 요구사항은 아래를 의미합니다:

- `someContainer`는 `C1` 타입의 컨테이너 입니다.
- `anotherContainer`는 `C2` 타입의 컨테이너 입니다.
- `someContainer`와 `anotherContainer`는 같은 타입의 항목을 포함합니다.
- `someContainer`의 항목은 서로 다름을 확인하기 위해 비동등 연산자(`!=`)를 사용하여 체크할 수 있습니다.

세 번째와 네 번째 요구사항은 `anotherContainer`의 항목은 `someContainer`의 항목과 정확히 동일한 타입이므로 `!=` 연산자로 확인할 수 있음을 의미합니다.

이 요구사항은 서로 다른 컨테이너 타입이더라도 두 컨테이너를 비교하기 위해 `allItemsMatch(_:_:)` 함수를 사용할 수 있습니다.

`allItemsMatch(_:_:)` 함수는 두 컨테이너 모두 항목을 같은 수만큼 가지고 있는지 확인하는 것으로 시작합니다. 항목의 수가 다르다면 일치하지 않으므로 함수는 `false`를 반환합니다.

이 검사를 수행한 후에 함수는 `for-in` 루프와 반열림 범위 연산자(`..<`)로 `someContainer`의 모든 항목을 반복합니다. 각 항목에 대해 함수는 `someContainer`의 항목이 `anotherContainer`의 항목과 동일한지 확인합니다. 두 항목이 동일하지 않으면 두 개의 컨테이너는 일치하지 않고 함수는 `false`를 반환합니다.

불일치하는 항목을 찾지 못하고 루프가 끝나면 두 개의 컨테이너는 일치하고 함수는 `true`를 반환합니다.

다음은 `allItemsMatch(_:_:)` 함수가 어떻게 동작하는지 보여줍니다:

```
var stackOfStrings = Stack<String>()
stackOfStrings.push("uno")
stackOfStrings.push("dos")
stackOfStrings.push("tres")

var arrayOfStrings = ["uno", "dos", "tres"]

if allItemsMatch(stackOfStrings, arrayOfStrings) {
    print("All items match.")
} else {
    print("Not all items match.")
}
// Prints "All items match.".
```

위의 예시는 `String` 값을 저장하는 `Stack` 인스턴스를 생성하고, 스택에 세 개의 문자열을 푸시합니다. 이 예시는 스택과 동일한 세 개의 문자열을 포함하는 `Array` 인스턴스도 생성합니다. 스택과 배열은 다른 타입이지만 둘 다 `Container` 프로토콜을 준수하고, 둘 다 같은 타입의 값을 포함합니다. 따라서 이 둘을 컨테이너 인자로 `allItemsMatch(_:_:)` 함수를 호출합니다. 위의 예시에서 `allItemsMatch(_:_:)` 함수는 두 컨테이너의 모든 항목이 정확히 일치한다고 알려줍니다.

제네릭 Where 절이 있는 확장 (Extensions with a Generic Where Clause)

확장에 제네릭 `where` 절을 사용할 수도 있습니다. 아래의 예시는 이전 예시의 제네릭 `Stack` 구조체에 `isTop(_:)` 메서드를 추가하기 위해 확장합니다.

```
extension Stack where Element: Equatable {
    func isTop(_ item: Element) -> Bool {
        guard let topItem = items.last else {
            return false
        }
        return topItem == item
    }
}
```

새로운 `isTop(_:)` 메서드는 먼저 스택이 비어있는지 확인하고 그 다음에 주어진 항목과 스택의 가장 상단의 항목을 비교합니다. 제네릭 `where` 절이 없이 시도한다면 문제가 발생합니다: `isTop(_:)`의 구현은 `==` 연산자를 사용하지만 `Stack`의 정의에서 항목은 `Equatable`을 준수하지 않으므로 `==` 연산자를 사용한 결과는 컴파일 오류가 발생합니다. 제네릭 `where` 절을 사용하여 확장에 새로운 요구사항을 추가할 수 있으므로, 확장은 스택의 항목이 `Equatable`을 준수할 때만 `isTop(_:)` 메서드를 추가합니다.

다음은 `isTop(_:)` 메서드가 어떻게 동작하는지 보여줍니다:

```
if stackOfStrings.isTop("tres") {
    print("Top element is tres.")
} else {
    print("Top element is something else.")
}
// Prints "Top element is tres.".
```

`Equatable`을 준수하지 않은 항목을 가진 스택에서 `isTop(_:)` 메서드를 호출하면 컴파일 오류가 발생합니다.

```
struct NotEquatable { }
var notEquatableStack = Stack<NotEquatable>()
let notEquatableValue = NotEquatable()
notEquatableStack.push(notEquatableValue)
notEquatableStack.isTop(notEquatableValue)  // Error
```

프로토콜 확장에도 제네릭 `where` 절을 사용할 수 있습니다. 아래의 예시는 이전 예시의 `Container` 프로토콜에 `startsWith(_:)` 메서드를 추가하기 위해 확장합니다.

```
extension Container where Item: Equatable {
    func startsWith(_ item: Item) -> Bool {
        return count >= 1 && self[0] == item
    }
}
```

`startsWith(_:)` 메서드는 먼저 컨테이너가 하나 이상의 항목을 가지고 있는지 확인하고 그런 다음 컨테이너의 첫 번째 항목이 주어진 항목과 일치하는지 확인합니다. 새로운 `startsWith(_:)` 메서드는 컨테이너의 항목이 `Equatable`을 준수하면 위에서 사용된 스택과 배열을 포함하여 `Container` 프로토콜을 준수하는 모든 타입에서 사용할 수 있습니다.

```
if [9, 9, 9].startsWith(42) {
    print("Starts with 42.")
} else {
    print("Starts with something else.")
}
// Prints "Starts with something else.".
```

위의 예시에서 제네릭 `where` 절은 `Item`이 프로토콜을 준수해야 한다고 요구하지만, `Item`이 특정 타입을 요구하도록 제네릭 `where` 절을 작성할 수도 있습니다. 예를 들어:

```
extension Container where Item == Double {
    func average() -> Double {
        var sum = 0.0
        for index in 0..<count {
            sum += self[index]
        }
        return sum / Double(count)
    }
}
print([1260.0, 1200.0, 98.6, 37.0].average())
// Prints "648.9".
```

이 예시는 `Item` 타입이 `Double`인 컨테이너에 `average()` 메서드를 추가합니다. 컨테이너의 항목을 더하고 컨테이너의 수로 나누어 평균을 계산합니다. 부동소수점 나누기가 가능하기 위해 카운트를 `Int`에서 `Double`로 명시적으로 변환합니다.

다른 곳에서 작성하는 제네릭 `where` 절과 마찬가지로 확장의 제네릭 `where` 절에 여러 요구사항을 포함할 수 있습니다. 콤마로 각 요구사항을 구분합니다.

컨텍스트 Where 절 (Contextual Where Clauses)

제네릭 타입의 컨텍스트에서 이미 작업중인 경우 별도의 제네릭 타입 제약조건이 없는 선언에 제네릭 `where` 절을 작성할 수 있습니다. 예를 들어 제네릭 타입의 서브스크립트나 제네릭 타입 확장의 메서드에 제네릭 `where` 절을 작성할 수 있습니다. `Container` 구조체는 제네릭이고, 아래의 예시에서 `where` 절은 컨테이너에서 이러한 새로운 메서드를 사용할 수 있도록 충족해야 하는 타입 제약조건을 지정합니다.

```
extension Container {
    func average() -> Double where Item == Int {
        var sum = 0.0
        for index in 0..<count {
            sum += Double(self[index])
        }
        return sum / Double(count)
    }
    func endsWith(_ item: Item) -> Bool where Item: Equatable {
        return count >= 1 && self[count-1] == item
    }
}
let numbers = [1260, 1200, 98, 37]
print(numbers.average())
// Prints "648.75".
print(numbers.endsWith(37))
// Prints "true".
```

이 예시는 항목이 정수이면 `Container`에 `average()` 메서드를 추가하고, 항목이 `Equatable`을 준수하면 `endsWith(_:)` 메서드를 추가합니다. 두 함수 모두 `Container`의 기존 선언에 제네릭 `Item` 타입 매개변수에 대해 타입 제약조건을 추가한 제네릭 `where` 절을 포함합니다.

컨텍스트 `where` 절 사용없이 코드를 작성하고 싶다면, 두 개의 확장을 작성하고 각 확장에 제네릭 `where` 절을 추가하면 됩니다. 위의 예시와 아래의 예시는 같은 동작을 가집니다.

```
extension Container where Item == Int {
    func average() -> Double {
        var sum = 0.0
        for index in 0..<count {
            sum += Double(self[index])
        }
        return sum / Double(count)
    }
}
extension Container where Item: Equatable {
    func endsWith(_ item: Item) -> Bool {
        return count >= 1 && self[count-1] == item
    }
}
```

컨텍스트 `where` 절을 사용하는 이 예시의 버전에서 각 메서드의 제네릭 `where` 절은 해당 메서드를 사용할 수 있도록 충족해야 할 요구사항을 명시하기 때문에 `average()`와 `endsWith(_:)`의 구현은 모두 같은 확장에 있습니다. 이러한 요구사항을 확장의 제네릭 `where` 절로 이동하면 동일한 상황에서 메서드를 사용할 수 있지만 요구사항 당 하나의 확장이 필요합니다.

제네릭 Where 절이 있는 연관 타입 (Associated Types with a Generic Where Clause)

연관 타입에 제네릭 `where` 절을 포함할 수 있습니다. 예를 들어 Swift 표준 라이브러리에서 `Sequence` 프로토콜이 사용하는 것과 같이 이터레이터(iterator)를 포함하는 `Container`의 버전을 만들고 싶다고 가정해 봅시다. 작성 방법은 아래와 같습니다:

```
protocol Container {
    associatedtype Item
    mutating func append(_ item: Item)
    var count: Int { get }
    subscript(i: Int) -> Item { get }

    associatedtype Iterator: IteratorProtocol where Iterator.Element == Item
    func makeIterator() -> Iterator
}
```

`Iterator`의 제네릭 `where` 절은 이터레이터(iterator)의 타입과 상관없이 컨테이너의 항목과 동일한 항목 타입의 요소를 탐색해야 합니다. `makeIterator()` 함수는 컨테이너의 이터레이터(iterator)에 접근을 제공합니다.

프로토콜이 다른 프로토콜을 상속할 때 프로토콜 선언에 제네릭 `where` 절을 포함하여 상속받은 연관 타입에 제약조건을 추가합니다. 예를 들어 다음의 코드는 `Item`이 `Comparable`을 준수하도록 요구하는 `ComparableContainer` 프로토콜을 선언합니다:

```
protocol ComparableContainer: Container where Item: Comparable { }
```

제네릭 서브스크립트 (Generic Subscripts)

서브스크립트도 제네릭으로 만들 수 있고, 제네릭 `where` 절을 포함할 수 있습니다. `subscript` 다음 꺾쇠 괄호 안에 임의의 타입 이름을 작성하고, 서브스크립트의 본문의 열린 중괄호 전에 제네릭 `where` 절을 작성합니다. 예를 들어:

```
extension Container {
    subscript<Indices: Sequence>(indices: Indices) -> [Item]
        where Indices.Iterator.Element == Int {
            var result: [Item] = []
            for index in indices {
                result.append(self[index])
            }
            return result
    }
}
```

`Container` 프로토콜의 확장은 시퀀스의 인덱스를 가지고 각 주어진 인덱스의 항목을 포함한 배열을 반환하는 서브스크립트를 추가합니다. 이 제네릭 서브스크립트는 다음과 같은 제약이 있습니다:

- 꺾쇠 괄호에 제네릭 매개변수 `Indices`는 표준 라이브러리의 `Sequence` 프로토콜을 준수하는 타입이어야 합니다.
- 서브스크립트는 `Indices` 타입의 인스턴스 인 `indices`라는 단일 매개변수를 가집니다.
- 제네릭 `where` 절은 시퀀스에 대한 이터레이터(iterator)가 `Int` 타입의 요소여야 합니다. 이렇게 하면 시퀀스의 인덱스는 컨테이너에 사용되는 인덱스와 동일한 타입입니다.

종합하면 이 제약조건은 `indices` 파리미터에 전달된 값은 정수 시퀀스입니다.

암시적 제약조건 (Implicit Constraints)

명시적으로 제약조건을 작성하는 것 외에도 제네릭 코드의 많은 곳에서 `Copyable`와 같은 일반적인 프로토콜 준수를 암시적으로 요구합니다.

이렇게 작성할 필요없는 제네릭 제약조건은 *암시적 제약조건(implicit constraints)*라고 합니다. 예를 들어 다음 함수 선언 모두 `MyType`이 복사 가능해야 함을 요구합니다:

```
function someFunction<MyType> { ... }
function someFunction<MyType: Copyable> { ... }
```

위 코드에서 첫 번째 선언은 암시적 제약조건을 가지고 있고 두 번째 버전은 명시적으로 준수를 나열합니다. 대부분의 코드에서 타입은 이러한 일반적인 프로토콜을 암시적으로 준수합니다. 자세한 내용은 프로토콜 암시적 준수 (Implicit Conformance to a Protocol)을 참고바랍니다.

Swift의 대부분 타입은 이러한 프로토콜을 준수하므로, 거의 모든 곳에 이것을 작성하는 것은 반복적입니다. 대신 예외적인 경우에만 표시함으로써 일반적인 제약조건이 생략된 곳을 강조할 수 있습니다. 암시적 제약조건을 억제하려면 프로토콜 이름 앞에 물결표(~)를 작성합니다. `~Copyable`은 "복사 가능할 수도 있음"으로 읽을 수 있습니다 - 이 억제된 제약조건은 이 위치에서 복사 가능한 타입과 복사 불가능한 타입 모두를 허용합니다. `~Copyable`은 타입이 복사 불가능할 것을 요구하는 것이 아닙니다. 예를 들어:

```
func f<MyType>(x: inout MyType) {
    let x1 = x  // The value of x1 is a copy of x's value.
    let x2 = x  // The value of x2 is a copy of x's value.
}

func g<AnotherType: ~Copyable>(y: inout AnotherType) {
    let y1 = y  // The assignment consumes y's value.
    let y2 = y  // Error: Value consumed more than once.
}
```

위 코드에서 함수 `f()`는 `MyType`이 복사 가능할 것을 암시적으로 요구합니다. 함수 본문 내에서 `x`의 값은 할당으로 `x1`과 `x2`에 복사됩니다. 반면에 `g()`는 `AnotherType`에 암시적 제약조건을 억제하여 복사 가능한 값과 복사 불가능한 값 모두 전달할 수 있게 합니다. 함수 본문 내에서 `y`의 값을 복사할 수 없으며 이것은 `AnotherType`이 복사 불가능할 수 있기 때문입니다. 할당은 `y`의 값을 소비하며 값을 두 번이상 소비하면 오류가 발생합니다. `y`와 같이 복사 불가능한 값은 in-out, borrowing, consuming 매개변수로 전달되어야 합니다 - 더 많은 정보는 Borrowing 매개변수와 Consuming 매개변수 (Borrowing and Consuming Parameters)를 참고바랍니다.

주어진 프로토콜에 암시적 제약조건이 언제 제네릭 코드에 포함되는지에 대한 자세한 내용은 해당 프로토콜의 참조를 참고바랍니다.

언어 가이드

불투명 타입과 박싱 프로토콜 타입 (Opaque and Boxed Protocol Types)

값의 타입에 대한 자세한 구현을 숨깁니다.

Swift는 값의 타입에 대한 정보를 숨기는 두 가지 방법을 제공합니다: 불투명 타입(opaque type)과 박싱 프로토콜 타입(boxed protocol type)입니다. 반환값의 실제 타입이 비공개로 유지될 수 있으므로 모듈과 모듈을 호출하는 코드 사이의 경계에서 타입 정보를 숨기는 것이 유용합니다.

불투명 타입을 반환하는 함수나 메서드는 반환하는 값의 타입 정보를 숨깁니다. 함수의 반환 타입으로 구체적인 타입을 제공하는 대신에 반환값이 어떤 프로토콜을 준수하는지 작성합니다. 불투명 타입은 타입 식별을 유지합니다 -- 컴파일러는 타입 정보에 접근할 수 있지만 모듈의 클라이언트는 그럴 수 없습니다.

박싱 프로토콜 타입은 주어진 프로토콜을 준수하는 타입의 인스턴스를 저장할 수 있습니다. 박싱 프로토콜 타입은 타입 식별을 유지하지 않습니다 -- 값의 구체적인 타입은 런타임까지 알 수 없으며, 저장되는 값에 따라 타입도 변경될 수 있습니다.

불투명 타입이 해결하는 문제 (The Problem that Opaque Types Solve)

예를 들어 ASCII 도형을 그리는 모듈을 작성한다고 가정합시다. ASCII 도형의 기본 특성은 도형을 문자열로 표현하는 `draw()` 함수이며, 이 함수는 `Shape` 프로토콜의 요구사항으로 정의할 수 있습니다:

```
protocol Shape {
    func draw() -> String
}

struct Triangle: Shape {
    var size: Int
    func draw() -> String {
        var result: [String] = []
        for length in 1...size {
            result.append(String(repeating: "*", count: length))
        }
        return result.joined(separator: "\n")
    }
}
let smallTriangle = Triangle(size: 3)
print(smallTriangle.draw())
// *
// **
// ***
```

아래 코드와 같이 제네릭을 사용하여 모양을 수직으로 뒤집는 것과 같은 작업을 구현할 수 있습니다. 그러나 이 접근 방식에는 한계가 있습니다: 뒤집힌 결과의 타입은 이를 생성하는데 사용된 구체적인 제네릭 타입을 노출합니다.

```
struct FlippedShape<T: Shape>: Shape {
    var shape: T
    func draw() -> String {
        let lines = shape.draw().split(separator: "\n")
        return lines.reversed().joined(separator: "\n")
    }
}
let flippedTriangle = FlippedShape(shape: smallTriangle)
print(flippedTriangle.draw())
// ***
// **
// *
```

아래의 코드와 같이 두 개의 모양을 수직으로 결합하는 `JoinedShape<T: Shape, U: Shape>` 구조체를 정의하면 뒤집힌 삼각형을 다른 삼각형과 결합하는 `JoinedShape<Triangle, FlippedShape<Triangle>>`과 같은 타입을 생성합니다.

```
struct JoinedShape<T: Shape, U: Shape>: Shape {
    var top: T
    var bottom: U
    func draw() -> String {
        return top.draw() + "\n" + bottom.draw()
    }
}
let joinedTriangles = JoinedShape(top: smallTriangle, bottom: flippedTriangle)
print(joinedTriangles.draw())
// *
// **
// ***
// ***
// **
// *
```

도형 생성에 대한 자세한 내용을 노출하면 ASCII 도형 모듈의 공개 인터페이스에 포함되지 않은 타입이 반환 타입을 명시해야 하므로 유출될 수 있습니다. 모듈 내에 코드는 다양한 방법으로 같은 도형을 구축할 수 있으며, 도형을 사용하는 모듈 외부에서의 다른 코드는 변환 목록에 대한 세부 구현 정보를 고려할 필요가 없습니다. `JoinedShape`와 `FlippedShape` 같은 래퍼 타입은 모듈 사용자에게 중요하지 않으므로, 노출되지 않아야 합니다. 모듈의 공개 인터페이스는 도형을 결합하고 뒤집는 것과 같은 작업으로 구성되며, 이러한 작업은 다른 `Shape` 값을 반환합니다.

불투명 타입 반환 (Returning an Opaque Type)

불투명 타입은 제네릭 타입과 반대라고 생각할 수 있습니다. 제네릭 타입을 사용하면 함수를 호출하는 코드에서 해당 함수의 매개변수에 대한 타입을 선택하고 함수 구현에서 추상화된 방식으로 값을 반환할 수 있습니다. 예를 들어 다음 코드의 함수는 호출자에 따라 달라지는 타입을 반환합니다:

```
func max<T>(_ x: T, _ y: T) -> T where T: Comparable { ... }
```

`max(_:_:)` 호출하는 코드는 `x`와 `y`에 대한 값을 선택하고, 이 값의 타입에 따라 `T`의 구체적인 타입을 결정합니다. 코드 호출은 `Comparable` 프로토콜을 준수하는 모든 타입을 사용할 수 있습니다. 함수 내부의 코드는 일반적인 방식으로 작성되므로 호출자가 제공하는 타입이 무엇이든 처리할 수 있습니다. `max(_:_:)`의 구현은 모든 `Comparable` 타입이 공유하는 기능만 사용합니다.

불투명 반환 타입을 사용하는 함수의 경우 이러한 역할이 반전됩니다. 불투명 타입을 사용하면 함수 구현에서 함수를 호출하는 코드에서 추상화된 방식으로 반환되는 값의 타입을 선택할 수 있습니다. 예를 들어 다음의 예시에서 함수는 도형의 실제 타입을 노출하지 않고 사다리꼴을 반환합니다.

```
struct Square: Shape {
    var size: Int
    func draw() -> String {
        let line = String(repeating: "*", count: size)
        let result = Array<String>(repeating: line, count: size)
        return result.joined(separator: "\n")
    }
}

func makeTrapezoid() -> some Shape {
    let top = Triangle(size: 2)
    let middle = Square(size: 2)
    let bottom = FlippedShape(shape: top)
    let trapezoid = JoinedShape(
        top: top,
        bottom: JoinedShape(top: middle, bottom: bottom)
    )
    return trapezoid
}
let trapezoid = makeTrapezoid()
print(trapezoid.draw())
// *
// **
// **
// **
// **
// *
```

이 예시에서 `makeTrapezoid()` 함수는 `some Shape`로 반환 타입을 선언하고; 그 결과로 함수는 구체적인 타입을 지정하지 않고 `Shape` 프로토콜을 준수하는 특정 타입의 값을 반환합니다. 이러한 방식으로 `makeTrapezoid()`를 작성하면 공용 인터페이스로 만들어지는 도형을 타입으로 지정하지 않고도 반환하는 값이 도형인 공용 인터페이스의 본질적인 측면을 표현할 수 있습니다. 이 구현은 두 개의 삼각형과 하나의 사각형을 사용하지만, 이 함수는 반환 타입을 변경하지 않고 다양한 방법으로 다시 작성할 수 있습니다.

이 예시는 불투명 반환 타입이 제네릭 타입과 반대 역할임을 보여줍니다. `makeTrapezoid()` 안에 코드는 필요한 모든 타입을 반환할 수 있지만 반환 타입이 `Shape` 프로토콜을 준수해야 합니다. 함수를 호출하는 코드는 `makeTrapezoid()`에 의해 반환된 모든 `Shape` 값이 동작할 수 있도록 제네릭 함수의 구현과 같이 일반적인 방식으로 작성되어야 합니다.

불투명 반환 타입은 제네릭과 결합할 수도 있습니다. 다음 코드의 함수는 모두 `Shape` 프로토콜을 준수하는 어떤 타입의 값을 반환합니다.

```
func flip<T: Shape>(_ shape: T) -> some Shape {
    return FlippedShape(shape: shape)
}
func join<T: Shape, U: Shape>(_ top: T, _ bottom: U) -> some Shape {
    JoinedShape(top: top, bottom: bottom)
}

let opaqueJoinedTriangles = join(smallTriangle, flip(smallTriangle))
print(opaqueJoinedTriangles.draw())
```

```
// *
// **
// ***
// ***
// **
// *
```

이 예시의 `opaqueJoinedTriangles` 값은 불투명 타입이 해결하는 문제 (The Problem that Opaque Types Solve) 섹션의 제네릭 예시에서 `joinedTriangles`와 동일합니다. 그러나 이 예시의 값과 다르게 `flip(_:)`과 `join(_:_:)`은 반환하는 제네릭 도형 연산자인 실제 타입을 래핑하여 불투명 반환 타입으로 해당 타입이 표시되지 않도록 합니다. 두 함수는 의존하는 타입이 제네릭이고 함수의 타입 매개변수가 `FlippedShape`와 `JoinedShape`에 필요한 타입 정보를 전달하기 때문에 모두 제네릭입니다.

불투명 반환 타입을 가진 함수가 여러 위치에서 반환하는 경우 가능한 모든 반환값은 동일한 타입을 가져야 합니다. 제네릭 함수의 경우 해당 반환 타입은 함수의 제네릭 타입 매개변수로 사용할 수 있지만 여전히 단일 타입이어야 합니다. 예를 들어 다음은 사각형에 대한 특수 케이스를 포함하는 유효하지 않은 버전의 도형 뒤집기 함수입니다:

```
func invalidFlip<T: Shape>(_ shape: T) -> some Shape {
    if shape is Square {
        return shape // Error: Return types don't match.
    }
    return FlippedShape(shape: shape) // Error: Return types don't match.
}
```

이 함수를 `Square`로 호출하면 `Square`를 반환하고, 그렇지 않으면 `FlippedShape`를 반환합니다. 이것은 단일 타입의 값을 반환해야 한다는 요구사항을 위반하고 `invalidFlip(_:)` 코드는 유효하지 않습니다. `invalidFlip(_:)`을 고치기 위한 한 가지 방법은 사각형의 특수한 경우를 `FlippedShape`의 구현 내부로 옮겨서 함수가 항상 `FlippedShape` 값을 반환하도록 합니다:

```
struct FlippedShape<T: Shape>: Shape {
    var shape: T
    func draw() -> String {
        if shape is Square {
            return shape.draw()
        }
        let lines = shape.draw().split(separator: "\n")
        return lines.reversed().joined(separator: "\n")
    }
}
```

항상 단일 타입을 반환해야 한다고 해서 불투명 반환 타입에 제네릭 사용을 막지는 않습니다. 다음은 함수의 타입 매개변수가 반환값의 구체적인 타입에 포함되는 예입니다:

```
func `repeat`<T: Shape>(shape: T, count: Int) -> some Collection {
    return Array<T>(repeating: shape, count: count)
}
```

이 경우에 반환값의 구체적인 타입은 `T`에 따라 달라집니다: 전달되는 도형이 무엇이든 `repeat(shape:count:)`는 해당 도형의 배열을 생성하고 반환합니다. 그럼에도 불구하고 반환값의 구체적인 타입은 `[T]`를 가지므로 불투명 반환 타입을 가지는 함수는 단일 타입의 값만 가져야 한다는 요구사항을 충족합니다.

박싱 프로토콜 타입 (Boxed Protocol Types)

박싱 프로토콜 타입(Boxed protocol type)은 "프로토콜을 준수하는 T 타입이 존재합니다."라는 문구와 같이 *존재적 타입(existential type)*이라고도 합니다. 박싱 프로토콜을 만들려면 프로토콜의 이름 앞에 any를 붙입니다. 예를 들면 다음과 같습니다:

```
struct VerticalShapes: Shape {
    var shapes: [any Shape]
    func draw() -> String {
        return shapes.map { $0.draw() }.joined(separator: "\n\n")
    }
}

let largeTriangle = Triangle(size: 5)
let largeSquare = Square(size: 5)
let vertical = VerticalShapes(shapes: [largeTriangle, largeSquare])
print(vertical.draw())
```

위의 예시에서 `VerticalShapes`는 `[any Shape]`로 `shapes`의 타입을 선언합니다 - 박싱된 Shape 요소의 배열입니다. 배열에 각 요소는 다른 타입이 올 수 있고, 각 타입은 Shape 프로토콜을 준수해야 합니다. 이런 런타임 유연성을 제공하기위해 Swift는 필요할 때 간접 계층을 추가합니다 - 이런 간접 계층을 *박스(box)*라고 부르며, 성능 비용(performance cost)이 발생할 수 있습니다.

`VerticalShapes` 타입에는 Shape 프로토콜이 요구하는 메서드, 프로퍼티, 서브스크립트를 사용할 수 있습니다. 예를 들어, `VerticalShapes`의 `draw()` 메서드는 배열의 각 요소에서 `draw()` 메서드를 호출합니다. Shape는 `draw()` 메서드를 요구하므로 이 메서드는 사용 가능합니다. 반대로 Shape에 의해 요구되지 않은 삼각형(triangle)의 `size` 프로퍼티 또는 다른 프로퍼티나 메서드를 접근하려고 하면 오류가 발생합니다.

`shapes`를 사용할 수 있는 세 가지 타입이 있습니다:

- `struct VerticalShapes<S: Shape>`와 `var shapes: [S]`을 작성하여 제네릭을 사용하면, 배열의 모든 요소가 동일한 도형 타입이 되고, 그 타입의 식별자는 외부 코드에 노출됩니다.
- `var shapes: [some Shape]`을 작성하여 불투명 타입을 사용하면, 배열의 모든 요소가 동일한 도형 타입이 되고 그 타입의 식별자는 숨겨집니다.
- `var shapes: [any Shape]`을 작성하여 박싱 프로토콜 타입을 사용하면, 배열에 서로 다른 타입의 요소를 저장할 수 있고, 각 타입의 식별자는 숨겨집니다.

이 경우 박싱 프로토콜 타입은 `VerticalShapes`의 호출자가 다른 종류의 도형을 혼합할 수 있는 유일한 접근방식입니다.

박싱 값(boxed value)의 타입을 알고 있는 경우에 as 변환을 사용할 수 있습니다. 예를 들어:

```
if let downcastTriangle = vertical.shapes[0] as? Triangle {
    print(downcastTriangle.size)
}
// Prints "5".
```

더 자세한 내용은 다운 캐스팅 (Downcasting)을 참고바랍니다.

불투명 타입과 박싱 프로토콜 타입의 차이점 (Differences Between Opaque Types and Boxed Protocol Types)

불투명 타입을 반환하는 것은 함수의 반환 타입으로 박싱 프로토콜 타입을 사용하는 것과 매우 유사하지만, 이 두 종류의 반환 타입은 타입 정체성을 유지하는지 여부에서 차이가 있습니다. 불투명 타입은 하나의 특정 타입을 참조하지만, 함수를 호출하는 쪽에서는 그 타입을 알 수 없습니다; 박싱 프로토콜 타입은 프로토콜을 준수하는 모든 타입을 참조할 수 있습니다. 일반적으로 박싱 프로토콜 타입은 저장하는 값의 타입에 대해 더 많은 유연성을 제공하고 불투명 타입을 사용하면 이러한 타입에 대해 더 강력한 보장을 할 수 있습니다.

예를 들어 다음은 불투명 반환 타입 대신에 반환 타입으로 박싱 프로토콜 타입을 사용하는 `flip(_:)`의 버전입니다:

```
func protoFlip<T: Shape>(_ shape: T) -> Shape {
    return FlippedShape(shape: shape)
}
```

`protoFlip(_:)`의 버전은 `flip(_:)`과 동일한 본문을 가지고, 항상 동일한 타입의 값을 반환합니다. `flip(_:)`과 다르게 `protoFlip(_:)`이 반환하는 값은 항상 동일한 타입을 가질 필요가 없으며 - `Shape` 프로토콜을 준수하기만 하면 됩니다. 다르게 말하면 `protoFlip(_:)`은 `flip(_:)`이 만든 것보다 더 느슨한 API 계약을 가집니다. 이것은 여러 타입의 값을 반환하는 유연성을 보유합니다:

```
func protoFlip<T: Shape>(_ shape: T) -> Shape {
    if shape is Square {
        return shape
    }

    return FlippedShape(shape: shape)
}
```

개정된 코드의 버전은 전달되는 도형에 따라 `Square` 인스턴스나 `FlippedShape` 인스턴스를 반환합니다. 이 함수에 의해 반환된 두 개의 뒤집힌 모양은 완전히 다른 타입을 가질 수 있습니다. 이 함수의 다른 유효한 버전은 같은 도형을 여러 번 뒤집을 때도 다른 타입의 값을 반환할 수 있습니다. `protoFlip(_:)`의 구체적이지 않은 반환 타입은 타입 정보에 의존하는 많은 작업이 반환된 값에서 사용할 수 없음을 의미합니다. 예를 들어 이 함수에 의해 반환된 결과를 비교하는 == 연산자를 사용할 수 없습니다.

```
let protoFlippedTriangle = protoFlip(smallTriangle)
let sameThing = protoFlip(smallTriangle)
protoFlippedTriangle == sameThing  // Error
```

예시의 마지막 라인에서 몇 가지 이유로 오류가 발생합니다. 즉각적인 문제는 `Shape`는 프로토콜 요구사항으로 == 연산자를 포함하지 않습니다. 이것을 추가하려고 하면 다음 문제는 == 연산자는 좌변과 우변 인자의 타입을 알아야 합니다. 이러한 종류의 연산자는 일반적으로 프로토콜을 채택하는 구체적인 타입과 일치하는 `Self` 타입의 인자를 사용하지만 프로토콜에 `Self` 요구사항을 추가하면 프로토콜을 타입으로 사용할 때 발생하는 타입 삭제가 허용되지 않습니다.

함수의 반환 타입으로 박싱 프로토콜 타입을 사용하면 프로토콜을 준수하는 모든 타입을 유연하게 반환할 수 있습니다. 그러나 이러한 유연성의 대가는 반환된 값에 대해 일부 작업을 수행할 수 없습니다. 이 예시는 박싱 프로토콜 타입을 사용하여 보존되지 않는 구체적인 타입 정보에 따라 == 연산자를 사용할 수 없는 것을 보여줍니다.

박싱 프로토콜 타입을 사용한다고 도형 변환의 중첩이 항상 불가능한 것은 아닙니다. 삼각형을 뒤집은 결과는 `Shape` 타입의 값이고 `protoFlip(_:)` 함수는 `Shape` 프로토콜을 준수하는 타입을 인자로 가집니다. 박싱 프로토

콜 타입 자체는 Shape를 준수하지 않지만, Swift는 지원하는 호출 위치에서 existential을 암시적으로 열어 구체적인 타입을 제네릭 매개변수에 바인딩합니다. 따라서 현재 Swift에서는 여러 변환을 적용하는 `protoFlip(protoFlip(smallTriangle))` 같은 코드가 유효한 호출입니다.

참고

번역자 주 이 문단은 영문 원본에 남아 있는 설명을 현재 Swift의 동작에 맞춰 정정했습니다. 박싱 프로토콜의 자체 준수와 제네릭 호출의 암시적 열기는 서로 다른 개념입니다. 자세한 내용은 박싱 프로토콜과 제네릭 호출 (Boxed Protocols and Generic Calls)을 참고바랍니다.

반대로 불투명 타입은 기본 타입의 정체성을 보존합니다. Swift는 연관 타입을 추론할 수 있으므로, 박싱 프로토콜 타입을 반환값으로 사용할 수 없는 위치에 불투명 반환값을 사용할 수 있습니다. 예를 들어 다음은 제네릭 (Generics)의 `Container` 프로토콜의 버전입니다:

```
protocol Container {
    associatedtype Item
    var count: Int { get }
    subscript(i: Int) -> Item { get }
}
extension Array: Container { }
```

연관 타입이 있는 프로토콜도 함수의 반환 타입으로 `any Container`를 사용할 수 있습니다. 또한 제네릭 타입이 필요한지 추론하기 위해 함수 본문의 외부에 충분한 정보가 없으므로 제네릭 반환 타입의 제약조건을 사용할 수 없습니다.

참고

번역자 주 연관 타입이 있다는 이유만으로 박싱 프로토콜 반환이 금지되는 것은 아닙니다. 아래의 첫 번째 오류는 Swift 6 언어 모드에서 명시적인 any가 필요하기 때문입니다. `any Container`와 `some Container`가 보존하는 타입 정보의 차이는 박싱 프로토콜과 제네릭 호출 (Boxed Protocols and Generic Calls)을 참고바랍니다.

```
// Error: Use 'any Container' for a boxed protocol return type.
func makeProtocolContainer<T>(item: T) -> Container {
    return [item]
}

// Error: Not enough information to infer C.
func makeProtocolContainer<T, C: Container>(item: T) -> C {
    return [item]
}
```

반환 타입으로 불투명 타입 `some Container`을 사용하면 함수가 컨테이너를 반환한다고 API를 표현할 수 있고, 컨테이너의 타입을 명시하지 않을 수 있습니다:

```
func makeOpaqueContainer<T>(item: T) -> some Container {
    return [item]
}
let opaqueContainer = makeOpaqueContainer(item: 12)
let twelve = opaqueContainer[0]
print(type(of: twelve))
// Prints "Int".
```

`twelve`의 타입은 `Int`로 추론되며 이것은 불투명 타입이 타입 추론이 동작한다는 것을 보여줍니다. `makeOpaqueContainer(item:)`의 구현에서 불투명 컨테이너의 타입은 `[T]`입니다. 이 경우에 `T`는 `Int`이므로 반환값은 정수의 배열이고 연관 타입 `Item`은 `Int`로 추론됩니다. `Container`의 서브스크립트는 `twelve`의 타입도 `Int`로 추론된다는 의미의 `Item`을 반환합니다.

불투명 매개변수 타입 (Opaque Parameter Types)

불투명 타입을 반환하기 위해 `some`을 작성하는 것 외에도 함수, 서브스크립트, 이니셜라이저의 매개변수 타입에 `some`을 작성할 수도 있습니다. 그러나 매개변수 타입으로 `some`을 작성하는 것은 불투명 타입이 아닌 제네릭을 위한 축약형일 뿐입니다. 예를 들어 아래의 두 함수 모두 동일합니다:

```
func drawTwiceGeneric<SomeShape: Shape>(_ shape: SomeShape) -> String {
    let drawn = shape.draw()
    return drawn + "\n" + drawn
}

func drawTwiceSome(_ shape: some Shape) -> String {
    let drawn = shape.draw()
    return drawn + "\n" + drawn
}
```

`drawTwiceGeneric(_:)` 함수는 `SomeShape`라는 제네릭 타입 매개변수를 선언하고 `SomeShape`는 `Shape` 프로토콜을 준수하도록 요구하는 제약조건을 가집니다. `drawTwiceSome(_:)` 함수는 인자에 `some Shape` 타입을 사용합니다. 이것은 해당 함수에 `Shape` 프로토콜을 준수하는 타입을 요구하는 제약조건과 함께 새롭고 이름이 없는 제네릭 타입 매개변수를 생성합니다. 제네릭 타입은 이름을 가지지 않으므로, 함수의 다른 곳에서는 타입을 참조할 수 없습니다.

두 개 이상의 매개변수 타입 앞에 `some`을 작성하면, 각 제네릭 타입은 독립적입니다. 예를 들어:

```
func combine(shape s1: some Shape, with s2: some Shape) -> String {
    return s1.draw() + "\n" + s2.draw()
}

combine(smallTriangle, trapezoid)
```

`combine(shape:with:)` 함수에서 첫 번째와 두 번째 매개변수 타입은 모두 `Shape` 프로토콜을 준수해야 하지만 둘 다 동일한 타입이어야 한다는 제약조건은 없습니다. `combine(shape:with:)`를 호출할 때, 두 개의 다른 모양을 전달할 수 있습니다 - 이 경우에는 하나는 삼각형이고 다른 하나는 사다리꼴입니다.

제네릭 (Generics)에서 설명한대로 명명 제네릭 타입 매개변수를 위한 문법과 다르게 이 경량 문법은 제네릭 `where` 절이나 동일 타입(==) 제약조건을 포함할 수 없습니다. 또한 매우 복잡한 제약조건에 경량 문법을 사용하면 읽기 어려울 수 있습니다.

언어 가이드

자동 참조 카운팅 (Automatic Reference Counting)

객체의 생명주기와 관계를 모델링합니다.

Swift는 앱의 메모리 사용량을 추적하고 관리하기 위해 자동 참조 카운팅(Automatic Reference Counting)(ARC)을 사용합니다. 대부분의 경우에 Swift에서 메모리 관리는 "자동으로 동작"을 의미하고 메모리 관리에 대해 생각할 필요가 없습니다. ARC는 인스턴스가 더 이상 필요치 않을 때 자동으로 해당 인스턴스가 사용하던 메모리를 해제합니다.

그러나 몇몇의 경우에 ARC는 메모리를 관리하기 위해 코드 간의 관계에 대한 추가 정보를 요구합니다. 여기에서는 이러한 상황을 설명하고 앱의 메모리를 관리하기 위해 ARC를 어떻게 사용하는지 보여줍니다. Swift에서 ARC를 사용하는 것은 Objective-C에서 ARC 사용에 대한 ARC 전환 릴리즈 노트 (Transitioning to ARC Release Notes)에서 설명한 접근방식과 매우 유사합니다.

참조 카운팅은 클래스의 인스턴스에만 적용됩니다. 구조체와 열거형은 참조 타입이 아니고 값 타입이고 참조로 저장되거나 전달되지 않습니다.

ARC의 동작 방식 (How ARC Works)

클래스의 새로운 인스턴스가 생성될 때마다 ARC는 인스턴스에 대한 정보를 저장하기 위해 메모리 공간을 할당합니다. 이 메모리는 인스턴스의 타입 정보와 해당 인스턴스와 연관된 저장 프로퍼티의 값이 포함됩니다.

또한 인스턴스가 더 이상 필요하지 않을 때, ARC는 메모리가 다른 용도로 사용할 수 있도록 인스턴스가 사용한 메모리를 해제합니다. 이렇게 하면 클래스 인스턴스가 더 이상 필요하지 않을 때, 메모리를 차지하지 않도록 보장합니다.

그러나 ARC가 아직 사용 중인 인스턴스를 해제하면, 더 이상 해당 인스턴스의 프로퍼티에 접근하거나 해당 인스턴스의 메서드를 호출할 수 없습니다. 실제로 인스턴스에 접근하려고 하면 앱은 크래시가 발생합니다.

인스턴스가 여전히 필요한 동안 사라지지 않도록 ARC는 각 클래스 인스턴스를 참조하는 프로퍼티, 상수, 변수가 얼마나 많은지 추적합니다. ARC는 인스턴스의 참조가 하나라도 존재하는 한 인스턴스를 해제하지 않습니다.

이것을 가능하게 하려면, 프로퍼티, 상수, 변수에 클래스 인스턴스를 할당할 때마다 해당 프로퍼티, 상수, 변수는 인스턴스에 *강한 참조(strong reference)*를 만듭니다. 이 참조를 "강한" 참조라고 하며 해당 인스턴스를 단단히 붙잡고 있어서 강한 참조가 남아있는 한 해제되지 않도록 합니다.

ARC 동작 (ARC in Action)

다음은 어떻게 Automatic Reference Counting이 동작하는지 보여주는 예시입니다. 이 예시는 `Person` 이라는 간단한 클래스로 시작하고, `name`이라는 저장 상수 프로퍼티를 정의합니다:

```
class Person {
    let name: String
    init(name: String) {
        self.name = name
        print("\(name) is being initialized")
    }
```

```
    deinit {
        print("\(name) is being deinitialized")
    }
}
```

Person 클래스는 인스턴스의 name 프로퍼티를 설정하고 메세지를 출력하는 이니셜라이저를 가지고 있습니다. Person 클래스는 클래스의 인스턴스가 해제될 때 메시지를 출력하는 디이니셜라이저도 가지고 있습니다.

다음 코드에서는 Person? 타입의 세 개의 변수를 정의하고, 이것은 다음 코드에서 새로운 Person 인스턴스에 여러 참조를 설정하기 위해 사용됩니다. 이 변수는 Person이 아닌 Person?인 옵셔널 타입 이므로, nil로 자동으로 초기화되고 현재는 Person 인스턴스를 참조하지 않습니다.

```
var reference1: Person?
var reference2: Person?
var reference3: Person?
```

이제 새로운 Person 인스턴스를 생성하고 이 세 개의 변수 중 하나에 할당합니다:

```
reference1 = Person(name: "John Appleseed")
// Prints "John Appleseed is being initialized".
```

Person 클래스의 이니셜라이저를 호출하면 "John Appleseed is being initialized" 메세지가 출력됩니다. 이것은 초기화가 발생했음을 의미합니다.

새로운 Person 인스턴스는 reference1 변수에 할당되기 때문에 이제 reference1에서 새로운 Person 인스턴스에 대한 강한 참조가 생깁니다. 하나의 강한 참조가 있기 때문에 ARC는 이 Person을 메모리에 유지하고 해제하지 않습니다.

동일한 Person 인스턴스를 두 변수에 할당하면, 해당 인스턴스에 대한 강한 참조가 두 개 더 설정됩니다:

```
reference2 = reference1
reference3 = reference1
```

이제 이 단일 Person 인스턴스에 세 개의 강한 참조가 있습니다.

기존의 참조를 포함하여 두 개의 변수에 nil을 할당하여 강한 참조를 중단하면 하나의 강한 참조만 남고 Person 인스턴스는 해제되지 않습니다:

```
reference1 = nil
reference2 = nil
```

ARC는 마지막 강한 참조를 중단할 때까지 Person 인스턴스를 해제하지 않고, 마지막 강한 참조가 중단되면 Person 인스턴스를 더 이상 사용하지 않는 것이 명백해집니다:

```
reference3 = nil
// Prints "John Appleseed is being deinitialized".
```

클래스 인스턴스 간의 강한 순환 참조 (Strong Reference Cycles Between Class Instances)

위의 예시에서 ARC는 새로 생성한 `Person` 인스턴스의 참조 개수를 추적하고, 더 이상 필요하지 않을 때 `Person` 인스턴스를 해제할 수 있습니다.

그러나 클래스 인스턴스가 절대 강한 참조가 0이 될 수 없는 코드를 작성할 수 있습니다. 이는 두 클래스 인스턴스가 서로에 대한 강한 참조를 유지하면 각 인스턴스가 서로를 계속 살아 있게 만듭니다. 이것을 *강한 순환 참조(strong reference cycle)*라고 합니다.

강한 순환 참조를 해결하려면 클래스 간의 관계를 강한 참조 대신 약한 참조(weak)나 미소유 참조(unowned)로 정의해야 합니다. 이 프로세스는 클래스 인스턴스 간의 강한 순환 참조 해결 (Resolving Strong Reference Cycles Between Class Instances)에 설명되어 있습니다. 그러나 강한 순환 참조를 어떻게 해결하는지 배우기 전에 어떻게 순환이 발생하는지 이해하는 것이 좋습니다.

다음은 실수로 강한 순환 참조를 생성하는 예시입니다. 이 예시는 `Person`과 `Apartment`라는 두 개의 클래스를 정의하고, 아파트와 거주자를 모델링합니다:

```
class Person {
    let name: String
    init(name: String) { self.name = name }
    var apartment: Apartment?
    deinit { print("\(name) is being deinitialized") }
}

class Apartment {
    let unit: String
    init(unit: String) { self.unit = unit }
    var tenant: Person?
    deinit { print("Apartment \(unit) is being deinitialized") }
}
```

모든 `Person` 인스턴스는 `String` 타입의 `name` 프로퍼티와 초기값이 `nil`인 옵셔널 `apartment` 프로퍼티를 가지고 있습니다. `apartment` 프로퍼티는 사람이 항상 아파트를 가지고 있지 않기 때문에 옵셔널입니다.

유사하게 모든 `Apartment` 인스턴스는 `String` 타입의 `unit` 프로퍼티와 초기값이 `nil`인 옵셔널 `tenant` 프로퍼티를 가지고 있습니다. `tenant` 프로퍼티는 아파트가 항상 세입자를 보유하는 것이 아니므로 옵셔널입니다.

이 클래스 모두 디이니셜라이저를 정의하고, 클래스의 인스턴스가 해제될 때 메세지를 출력합니다. 이것을 통해 `Person`과 `Apartment`의 인스턴스가 해제되는지 확인할 수 있습니다.

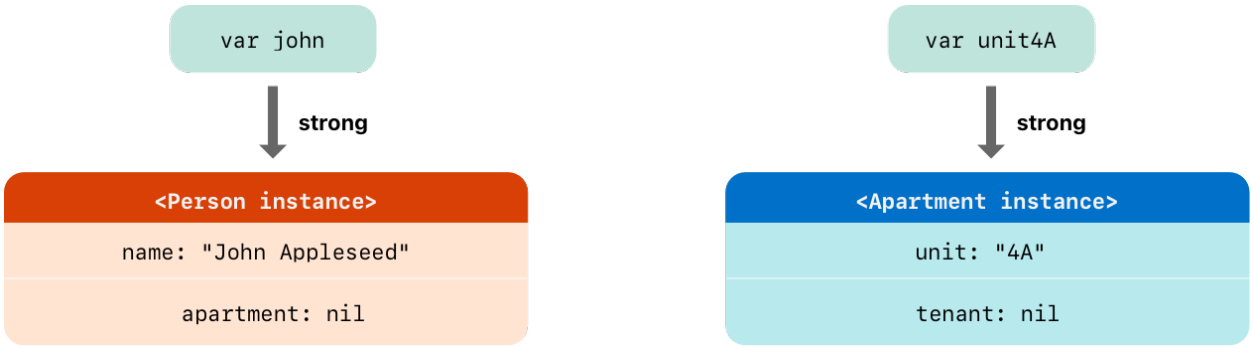
다음 코드는 `Apartment`와 `Person` 인스턴스를 설정할 `john`과 `unit4A`라는 옵셔널 타입의 두 개의 변수를 정의합니다. 이 변수 모두 옵셔널이므로 초기값으로 `nil`을 가집니다:

```
var john: Person?
var unit4A: Apartment?
```

이제 `Person` 인스턴스와 `Apartment` 인스턴스를 생성할 수 있고, 이 인스턴스를 `john`과 `unit4A` 변수에 할당할 수 있습니다:

```
john = Person(name: "John Appleseed")
unit4A = Apartment(unit: "4A")
```

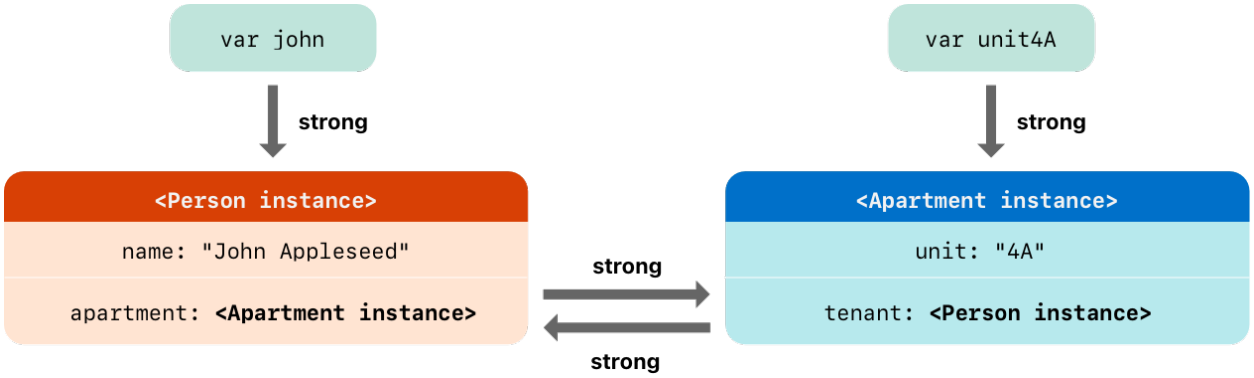
다음은 강한 참조가 이 두 인스턴스를 생성하고 할당한 후에 어떻게 되는지 보여줍니다. `john` 변수는 이제 `Person` 인스턴스에 대해 강한 참조를 가지고 있고, `unit4A` 변수는 `Apartment` 인스턴스에 대한 강한 참조를 가지고 있습니다:



이제 사람은 아파트를 가지고 아파트는 소유자를 가지도록 두 인스턴스를 함께 연결할 수 있습니다. 느낌표(`!`)는 `john`과 `unit4A` 옵셔널 변수에 저장된 인스턴스를 언래핑하고 접근하기 위해 사용되므로 해당 인스턴스의 프로퍼티를 설정할 수 있습니다:

```
john!.apartment = unit4A
unit4A!.tenant = john
```

다음은 두 인스턴스를 함께 연결한 후에 강한 참조가 어떻게 되는지 보여줍니다:

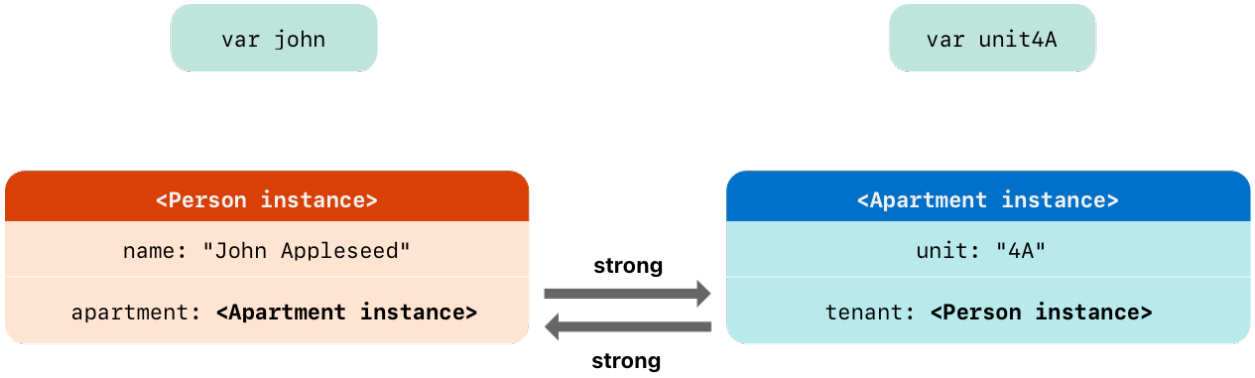


불행히도 이 두 인스턴스 연결은 서로 간의 강한 순환 참조를 생성합니다. `Person` 인스턴스는 이제 `Apartment` 인스턴스에 대한 강한 참조를 가지고, `Apartment` 인스턴스는 `Person` 인스턴스에 대한 강한 참조를 가집니다. 따라서 `john`과 `unit4A` 변수의 강한 참조를 중단해도 참조 카운트는 0으로 떨어지지 않고 인스턴스는 ARC에 의해 해제되지 않습니다:

```
john = nil
unit4A = nil
```

두 변수를 `nil`로 설정해도 디이니셜라이저는 호출되지 않습니다. 강한 순환 참조는 `Person`과 `Apartment` 인스턴스가 해제되는 것을 방지하여 앱에서 메모리 누수를 유발합니다.

다음은 `john`과 `unit4A` 변수를 `nil`로 설정한 후에 강한 참조를 나타냅니다:



Person과 Apartment 인스턴스 간의 강한 참조는 남아있고 중단할 수 없습니다.

클래스 인스턴스 간의 강한 순환 참조 해결 (Resolving Strong Reference Cycles Between Class Instances)

Swift는 클래스 타입의 프로퍼티에서 강한 순환 참조를 해결하기 위해 두 가지 방법을 제공합니다: 약한 참조(weak references)와 미소유 참조(unowned references)입니다.

약한 참조와 미소유 참조를 사용하면 순환 참조의 한 인스턴스가 다른 인스턴스를 강하게 참조하지 않고 참조할 수 있습니다. 이렇게 하면 인스턴스는 강한 순환 참조를 만들지 않고도 서로를 참조할 수 있습니다.

다른 인스턴스의 생명주기가 더 짧은 경우 - 즉, 다른 인스턴스가 먼저 해제될 수 있을 때 약한 참조를 사용합니다. 위의 Apartment 예시에서 아파트는 어느 시점에 세입자가 없을 수 있으므로 이러한 경우 약한 참조는 순환 참조를 끊는 적절한 방법입니다. 반대로 다른 인스턴스의 생명주기가 동일하거나 더 긴 경우 미소유 참조를 사용합니다.

약한 참조 (Weak References)

*약한 참조(weak reference)*는 참조하는 인스턴스를 강하게 유지하지 않는 참조이므로 ARC가 참조된 인스턴스를 해제하는 것을 중지하지 않습니다. 이러한 동작은 참조가 강한 순환 참조가 되는 것을 방지합니다. 약한 참조는 프로퍼티나 변수 선언 전에 weak 키워드를 위치시켜 나타냅니다.

약한 참조는 참조하는 인스턴스를 강하게 유지하지 않기 때문에 약한 참조가 참조하는 동안 해당 인스턴스가 해제될 수 있습니다. 따라서 ARC는 참조하는 인스턴스가 해제되면 nil로 약한 참조를 자동으로 설정합니다. 그리고 약한 참조는 런타임에 값을 nil로 변경하는 것을 허락해야 하므로 항상 옵셔널 타입의 상수가 아닌 변수로 선언됩니다.

다른 옵셔널 값과 같이 약한 참조의 값 존재를 확인할 수 있고 더 이상 존재하지 않는 유효하지 않은 인스턴스에 대해 참조하는 일은 없습니다.

> **참고**
>
> 프로퍼티 관찰자는 ARC가 약한 참조를 nil로 설정할 때 호출되지 않습니다.

아래의 예시는 위의 Person과 Apartment 예시와 동일하지만, 한 가지 중요한 차이점이 있습니다. 이번에는 Apartment 타입의 tenant 프로퍼티가 약한 참조로 선언됩니다:

```
class Person {
    let name: String
    init(name: String) { self.name = name }
    var apartment: Apartment?
    deinit { print("\(name) is being deinitialized") }
}
```

```
class Apartment {
    let unit: String
    init(unit: String) { self.unit = unit }
    weak var tenant: Person?
    deinit { print("Apartment \(unit) is being deinitialized") }
}
```

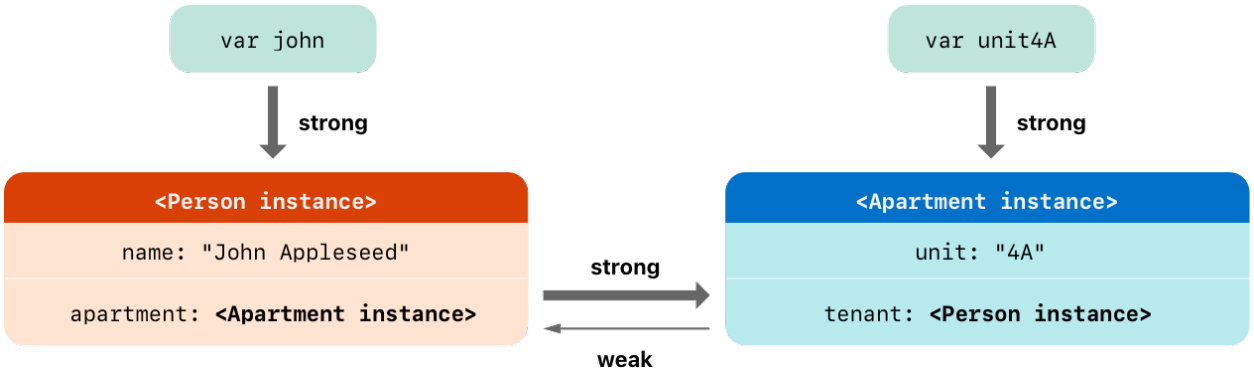
두 변수(`john`과 `unit4A`)에서의 강한 참조와 두 인스턴스 간의 연결은 이전과 같이 생성됩니다:

```
var john: Person?
var unit4A: Apartment?

john = Person(name: "John Appleseed")
unit4A = Apartment(unit: "4A")

john!.apartment = unit4A
unit4A!.tenant = john
```

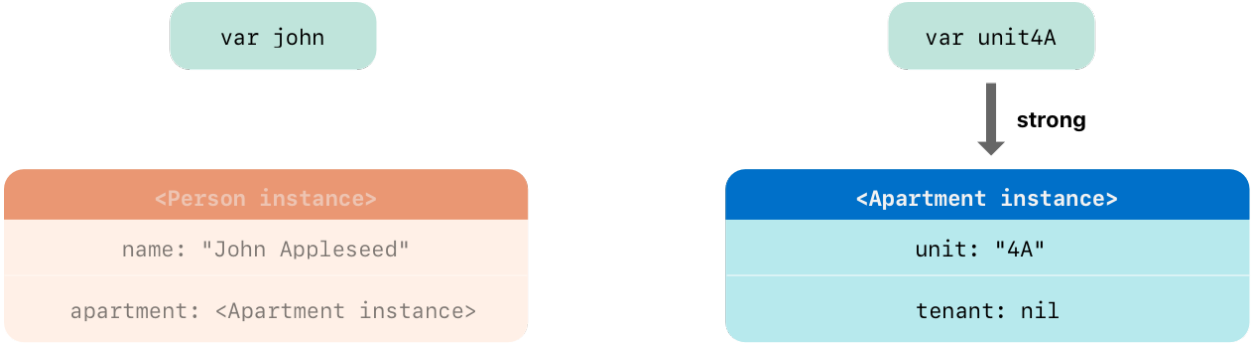
다음은 두 인스턴스를 연결되면 어떻게 참조되는지 보여줍니다:



`Person` 인스턴스는 `Apartment` 인스턴스에 대해 여전히 강한 참조를 가지고 있지만 `Apartment` 인스턴스는 이제 `Person` 인스턴스에 대해 약한 참조를 가지고 있습니다. 이것은 `john` 변수에 `nil`을 설정하여 강한 참조를 끊으면 `Person` 인스턴스에 대해 더 이상 강한 참조가 아님을 의미합니다:

```
john = nil
// Prints "John Appleseed is being deinitialized".
```

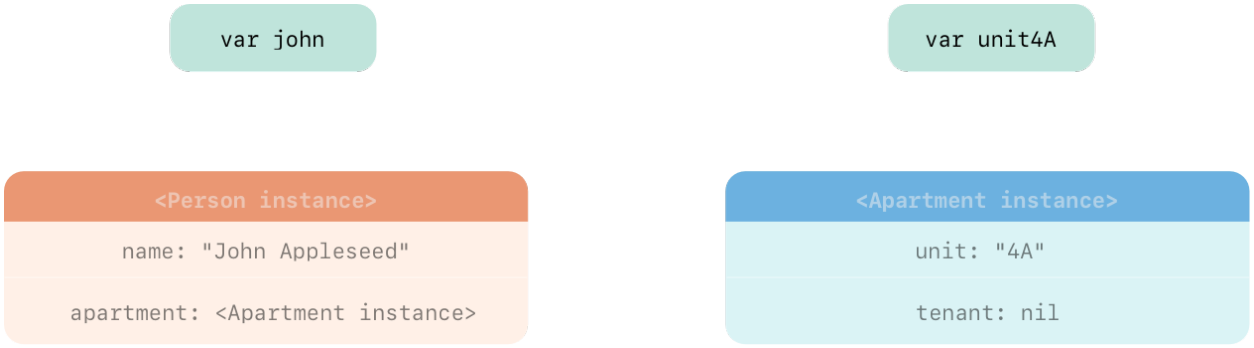
더 이상 `Person` 인스턴스에 대해 강한 참조를 가지지 않기 때문에, 해제되고 `tenant` 프로퍼티는 `nil`로 설정됩니다:



`Apartment` 인스턴스에 대한 유일한 강한 참조는 `unit4A` 변수에서 가져온 것입니다. 해당 강한 참조를 끊으면, `Apartment` 인스턴스에 대한 강한 참조는 더 이상 없습니다:

```
unit4A = nil
// Prints "Apartment 4A is being deinitialized".
```

더 이상 `Apartment` 인스턴스에 대한 강한 참조를 가지지 않기 때문에, 이것 또한 해제됩니다:



참고

가비지 컬렉션을 사용하는 시스템에서는 메모리 압력이 가비지 컬렉션을 트리거 할 때만 강한 참조가 없는 객체가 할당 해제되기 때문에 간단한 캐싱 메커니즘을 구현하는데 약한 포인터가 사용되는 경우가 있습니다. 그러나 ARC를 사용하면 마지막 강한 참조가 제거되자마자 값이 해제되어 약한 참조는 이러한 용도에 적합하지 않습니다.

미소유 참조 (Unowned References)

약한 참조와 마찬가지로 *미소유 참조(unowned reference)*도 참조하는 인스턴스를 강하게 유지하지 않습니다. 그러나 약한 참조와 다르게 미소유 참조는 다른 인스턴스의 생명주기가 같거나 더 긴 경우에 사용합니다. 미소유 참조는 프로퍼티나 변수 선언 전에 `unowned` 키워드를 위치시켜 나타냅니다.

약한 참조와 달리 미소유 참조는 항상 값을 가지고 있는 것으로 예상합니다. 결과적으로 미소유로 만들어진 값은 옵셔널이 아니며, ARC는 미소유 참조의 값을 `nil`로 설정하지 않습니다.

중요

미소유 참조는 참조 대상 인스턴스가 해제되지 않았다는 것을 항상 확신하는 경우에만 사용합니다.

인스턴스가 해제된 후에 미소유 참조의 값에 접근하려고 하면 런타임 오류가 발생합니다.

다음 예시는 은행 고객과 고객의 신용카드를 모델링하는 `Customer`와 `CreditCard`인 두 개의 클래스를 정의합니다. 이 두 클래스는 프로퍼티로 서로의 인스턴스를 각각 저장하고, 강한 순환 참조가 발생할 가능성이 있습니다.

`Customer`와 `CreditCard` 간의 관계는 위의 약한 참조 예시에서 본 `Apartment`와 `Person` 간의 관계와 약간 다릅니다. 이 데이터 모델에서 고객은 신용카드를 가지고 있거나 가지고 있지 않을 수 있지만 신용카드는 항상 고객과 연관되어 있습니다. `CreditCard` 인스턴스는 참조하는 `Customer`보다 오래 지속되지 않습니다. 이것을 표현하기 위해 `Customer` 클래스는 옵셔널 `card` 프로퍼티를 가지지만 `CreditCard` 클래스는 미소유(옵셔널 아님) `customer` 프로퍼티를 가집니다.

또한 새로운 `CreditCard` 인스턴스는 커스텀 `CreditCard` 이니셜라이저에 `number` 값과 `customer` 인스턴스를 전달해야만 생성할 수 있습니다. 이렇게 하면 `CreditCard` 인스턴스가 생성될 때 `CreditCard` 인스턴스에 항상 연관된 `customer` 인스턴스를 가지고 있습니다.

신용카드는 항상 고객이 있으므로 강한 순환 참조를 피하기 위해 `customer` 프로퍼티에 미소유 참조로 정의합니다:

```
class Customer {
    let name: String
    var card: CreditCard?
    init(name: String) {
        self.name = name
    }
    deinit { print("\(name) is being deinitialized") }
}

class CreditCard {
    let number: UInt64
    unowned let customer: Customer
    init(number: UInt64, customer: Customer) {
        self.number = number
        self.customer = customer
    }
    deinit { print("Card #\(number) is being deinitialized") }
}
```

참고

`CreditCard` 클래스의 `number` 프로퍼티는 `Int`가 아닌 `UInt64` 타입으로 정의되어 `number` 프로퍼티의 용량이 32비트와 64비트 시스템 모두에 16자리 카드번호를 저장할 수 있을만큼 충분히 크도록 합니다.

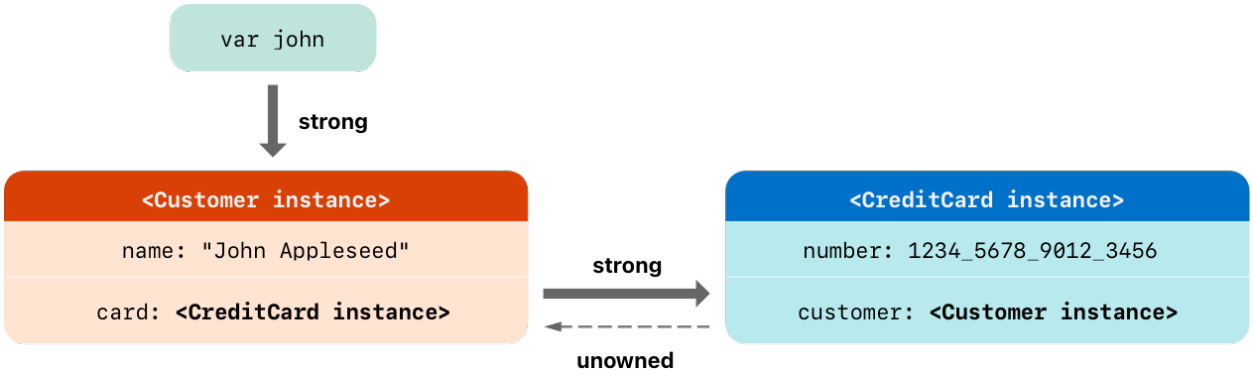
다음 코드는 `john`이라는 옵셔널 `Customer` 변수를 정의하고, 구체적인 고객에 대한 참조를 저장하는데 사용합니다. 이 변수는 옵셔널이므로 `nil`의 초기값을 가집니다:

```
var john: Customer?
```

이제 `Customer` 인스턴스를 생성할 수 있고 해당 고객의 `card` 프로퍼티로 새로운 `CreditCard` 인스턴스를 초기화하고 할당할 수 있습니다:

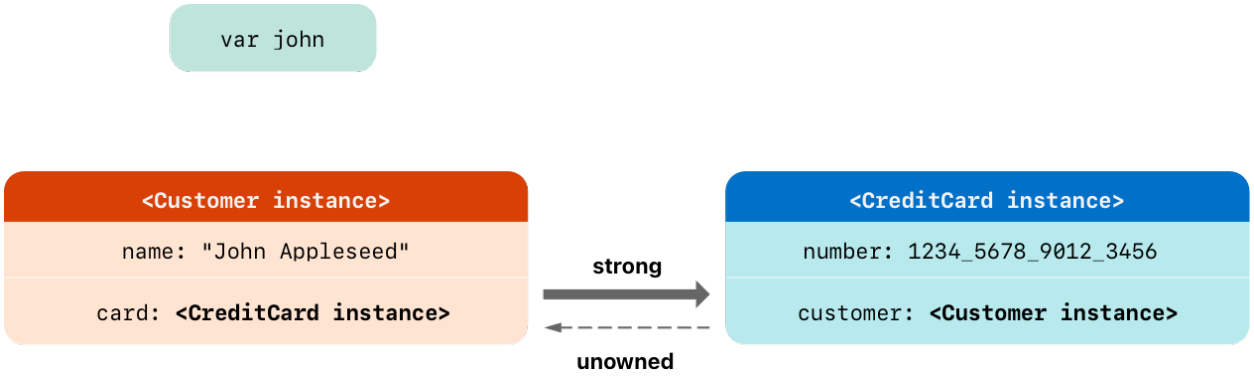
```
john = Customer(name: "John Appleseed")
john!.card = CreditCard(number: 1234_5678_9012_3456, customer: john!)
```

다음은 두 인스턴스가 연결되어 어떻게 참조하는지 보여줍니다:



`Customer` 인스턴스는 이제 `CreditCard` 인스턴스에 대한 강한 참조를 가지고 있고 `CreditCard` 인스턴스는 `Customer` 인스턴스에 대해 미소유 참조를 가지고 있습니다.

미소유 customer 참조 때문에, john 변수의 강한 참조를 끊으면 Customer 인스턴스에 대해 더 이상 강한 참조를 가지지 않습니다:



더 이상 Customer 인스턴스에 대해 강한 참조가 아니므로 해제됩니다. 그 다음에 더 이상 CreditCard 인스턴스에 대해 강한 참조가 아니므로 이것도 해제됩니다:

```
john = nil
// Prints "John Appleseed is being deinitialized".
// Prints "Card #1234567890123456 is being deinitialized".
```

위의 마지막 코드는 john 변수를 nil로 설정한 후에 Customer 인스턴스와 CreditCard 인스턴스 모두 “deinitialized”를 출력하는 디이니셜라이저를 보여줍니다.

> **참고**
>
> 위의 예시는 안전한 미소유 참조를 어떻게 사용하는지 보여줍니다. Swift는 런타임 안전 검사가 필요하지 않을 때 사용하는 안전하지 않은 미소유 참조(unsafe unowned references)를 제공합니다 - 예를 들어 성능상의 이유가 한 예입니다. 모든 안전하지 않은 작업과 마찬가지로 해당 코드의 안전성은 개발자가 직접 책임져야 합니다.
>
> 안전하지 않은 미소유 참조는 unowned(unsafe)로 표시합니다. 참조하는 인스턴스가 해제된 후에 안전하지 않은 미소유 참조에 접근하려고 하면 프로그램은 인스턴스가 사용한 메모리 위치에 접근하고, 이것은 안전하지 않은 작업입니다.

미소유 옵셔널 참조 (Unowned Optional References)

클래스에 대한 옵셔널 참조도 미소유로 표기할 수 있습니다. ARC 소유권 모델 측면에서 미소유 옵셔널 참조(unowned optional reference)와 약한 참조는 모두 같은 컨텍스트에서 사용할 수 있습니다. 차이점은 미소유 옵셔널 참조를 사용할 때, 유효한 객체를 참조하거나 nil로 설정해야 합니다.

다음은 학교의 학과에서 제공하는 강좌를 모델링하는 예시입니다:

```
class Department {
    var name: String
    var courses: [Course]
    init(name: String) {
        self.name = name
        self.courses = []
    }
}

class Course {
    var name: String
    unowned var department: Department
```

```
    unowned var nextCourse: Course?
    init(name: String, in department: Department) {
        self.name = name
        self.department = department
        self.nextCourse = nil
    }
}
```

`Department`는 학과에서 제공하는 각 강좌에 강한 참조를 유지합니다. ARC 소유권 모델에서 학과는 강좌를 소유하고 있습니다. `Course`는 두 개의 미소유 참조를 가지고 있으며, 하나는 학과에 대한 것이고, 다른 하나는 학생이 수강해야 될 강좌입니다; `Course`는 이 두 객체를 소유하지 않습니다. 모든 강좌는 학과에 속하므로 `department` 프로퍼티는 옵셔널이 아닙니다. 그러나 일부 강좌는 후속 과정이 없기 때문에 `nextCourse` 프로퍼티는 옵셔널 입니다.

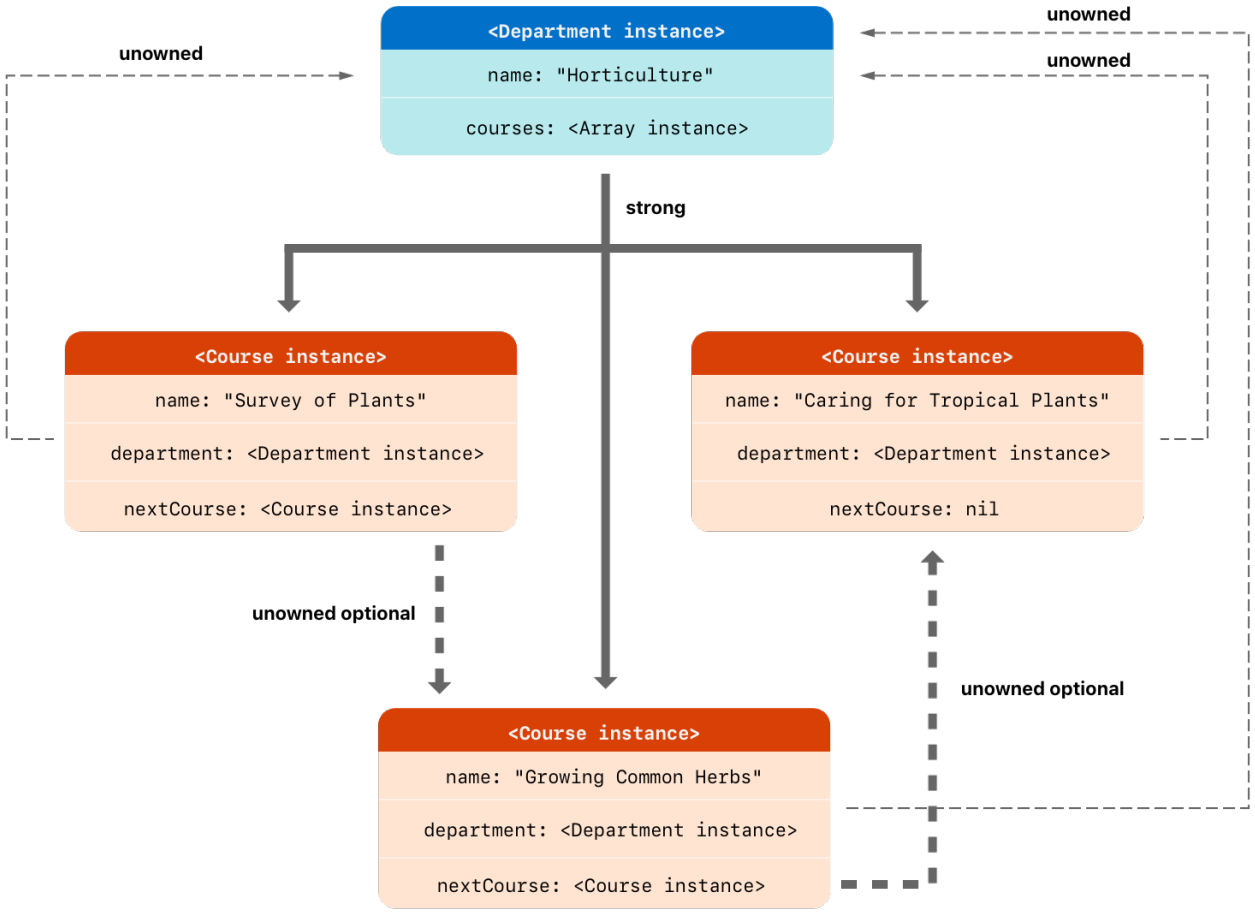
다음은 이러한 클래스를 사용하는 예시입니다:

```
let department = Department(name: "Horticulture")

let intro = Course(name: "Survey of Plants", in: department)
let intermediate = Course(name: "Growing Common Herbs", in: department)
let advanced = Course(name: "Caring for Tropical Plants", in: department)

intro.nextCourse = intermediate
intermediate.nextCourse = advanced
department.courses = [intro, intermediate, advanced]
```

위의 코드는 학과와 그 학과의 세 개의 강좌를 생성합니다. 입문과 중급 과목 모두 `nextCourse` 프로퍼티에 저장된 다음 과목을 제안하며 이는 학생이 과목을 완료한 후 수강해야 하는 과목에 대한 미소유 옵셔널 참조를 유지합니다.



미소유 옵셔널 참조는 참조하는 클래스 인스턴스를 강하게 유지하지 않으므로, ARC가 해당 인스턴스를 해제하는 것을 제한하지 않습니다. ARC에서 미소유 참조와 동일하게 동작하지만, 미소유 옵셔널 참조는 `nil`이 될 수 있다는 점이 다릅니다.

미소유 참조와 마찬가지로 `nextCourse`가 항상 해제되지 않은 강좌를 참조하도록 직접 관리해야 합니다. 예를 들어 `department.courses`에서 강좌를 삭제할 때 다른 강좌가 해당 강좌를 참조하고 있다면 그 참조도 삭제해야 합니다.

> **참고**
>
> 옵셔널 값의 타입은 `Optional`이며, Swift 표준 라이브러리의 열거형입니다. 그러나 옵셔널은 값 타입에 `unowned`를 표시할 수 없는 규칙에 대해 예외입니다.
>
> 클래스를 감싸는 옵셔널은 참조 카운팅을 사용하지 않으므로, 옵셔널에 대해 강한 참조를 유지할 필요가 없습니다.

미소유 참조와 암시적 언래핑 옵셔널 프로퍼티 (Unowned References and Implicitly Unwrapped Optional Properties)

위의 약한 참조와 미소유 참조에 대한 예시는 강한 순환 참조를 중단해야 하는 대표적인 두 가지 상황을 다룹니다.

`Person`과 `Apartment` 예시는 두 프로퍼티가 `nil`이 될 수 있는 상황에서 강한 순환 참조가 발생할 수 있는 상황을 보여줍니다. 이 상황은 약한 참조로 해결하는 것이 가장 좋습니다.

`Customer`와 `CreditCard` 예시는 하나의 프로퍼티는 `nil`이 허용되고 다른 프로퍼티는 `nil`일 수 없는 상황에서 강한 순환 참조가 발생할 수 있는 상황을 보여줍니다. 이 상황은 미소유 참조로 해결하는 것이 가장 좋습니다.

그러나 세 번째 상황도 존재하며, 이 상황은 두 프로퍼티 모두 항상 값이 있어야 하고, 초기화가 완료된 후 `nil`이 되어서는 안되는 경우입니다. 이 상황에서는 한 클래스에는 미소유 프로퍼티를 다른 클래스에는 암시적 언래핑 옵셔널 프로퍼티를 사용하는 것이 유용합니다.

이렇게 하면 두 프로퍼티는 초기화가 완료된 후 바로 접근이 가능하고(옵셔널 언래핑 없이), 순환 참조도 피할 수 있습니다. 여기서는 이러한 관계를 어떻게 설정하는지 보여줍니다.

아래의 예시는 `Country`와 `City`인 두 개의 클래스를 정의하고, 각 클래스는 서로의 인스턴스를 프로퍼티로 저장합니다. 이 데이터 모델에서 모든 국가는 항상 수도를 가지고 있고, 모든 도시는 항상 국가에 속해 있어야 합니다. 이것을 표현하기 위해 `Country` 클래스는 `capitalCity` 프로퍼티를 가지고 있고, `City` 클래스는 `country` 프로퍼티를 가지고 있습니다:

```
class Country {
    let name: String
    var capitalCity: City!
    init(name: String, capitalName: String) {
        self.name = name
        self.capitalCity = City(name: capitalName, country: self)
    }
}

class City {
    let name: String
    unowned let country: Country
    init(name: String, country: Country) {
        self.name = name
        self.country = country
    }
}
```

두 클래스 간의 상호 의존성을 설정하기 위해 `City`의 이니셜라이저는 `Country` 인스턴스를 가지고 있고, `country` 프로퍼티에 저장합니다.

`City`의 이니셜라이저는 `Country`의 이니셜라이저 내에서 호출됩니다. 그러나 `Country`의 이니셜라이저는 2단계 초기화 (Two-Phase Initialization)에서 설명했듯이 새로운 `Country` 인스턴스가 완전히 초기화될 때까지 `City` 이니셜라이저에 `self`를 전달할 수 없습니다.

이 요구사항을 처리하려면 `Country`의 `capitalCity` 프로퍼티를 암시적 언래핑 옵셔널 프로퍼티로 선언하며, 이것은 타입 뒤에 느낌표(`City!`)로 표시합니다. `capitalCity` 프로퍼티는 다른 옵셔널과 같이 `nil`의 기본값을 가지지만 암시적 언래핑 옵셔널 (Implicitly Unwrapped Optionals)에서 설명했듯이 언래핑할 필요없이 값에 접근할 수 있다는 의미입니다.

`capitalCity`는 기본적으로 `nil`을 가지므로 새로운 `Country` 인스턴스는 `Country` 인스턴스가 이니셜라이저 내에서 `name` 프로퍼티를 설정해야 완전히 초기화 된 것으로 간주됩니다. 이것은 `Country` 이니셜라이저는 `name` 프로퍼티가 설정되면 암시적 `self` 프로퍼티를 참조하고 전달할 수 있다는 의미입니다. 따라서 `Country` 이니셜라이저는 `capitalCity` 프로퍼티를 설정할 때 `City` 이니셜라이저의 매개변수로 `self`를 전달할 수 있습니다.

이것으로 인해 `Country` 와 `City` 인스턴스를 강한 순환 참조 없이 단일 구문으로 생성할 수 있고, `capitalCity` 프로퍼티에 느낌표를 사용할 필요없이 직접 접근할 수 있습니다:

```
var country = Country(name: "Canada", capitalName: "Ottawa")
print("\(country.name)'s capital city is called \(country.capitalCity.name)")
// Prints "Canada's capital city is called Ottawa".
```

위의 예시에서 암시적 언래핑 옵셔널의 사용은 모든 2단계 클래스 이니셜라이저 요구사항을 충족한다는 의미입니다. `capitalCity` 프로퍼티는 초기화가 완료되면 옵셔널이 아닌 값처럼 사용되고 접근할 수 있지만 강한 순환 참조도 피할 수 있습니다.

클로저의 강한 순환 참조 (Strong Reference Cycles for Closures)

위에서 두 클래스 인스턴스 프로퍼티가 서로 강한 참조를 유지할 때 강한 순환 참조가 어떻게 생성될 수 있는지 보았습니다. 또한 강한 순환 참조를 끊기 위해 약한 참조와 미소유 참조를 어떻게 사용해야 하는지 보았습니다.

강한 순환 참조는 클래스 인스턴스의 프로퍼티에 클로저를 할당하고, 해당 클로저의 본문에서 인스턴스를 캡처하는 경우에도 발생할 수 있습니다. 이 캡처는 클로저의 본문에서 `self.someProperty`와 같은 인스턴스의 프로퍼티에 접근하거나 클로저에서 `self.someMethod()`와 같은 인스턴스의 메서드를 호출하기 때문에 발생할 수 있습니다. 두 경우 모두 클로저가 `self`를 "캡처"하여 강한 순환 참조를 생성합니다.

이 강한 순환 참조는 클래스와 같이 클로저는 *참조 타입(reference types)*이기 때문에 발생합니다. 프로퍼티에 클로저를 할당하면 해당 클로저의 참조를 할당하는 것입니다. 본질적으로 두 강한 참조는 서로 유지되므로 위에서와 같은 문제가 발생합니다. 그러나 두 클래스 인스턴스가 아니라 이번에는 클래스 인스턴스와 클로저가 서로를 참조합니다.

Swift는 이런 문제를 해결하기 위해 *클로저 캡처 리스트(closure capture list)*을 제공합니다. 그러나 클로저 캡처 리스트로 강한 순환 참조를 끊는 방법에 대해 배우기 전에 이러한 순환이 어떻게 발생하는지 이해하는 것이 더 유용합니다.

아래의 예시는 `self`를 참조하는 클로저를 사용할 때 강한 순환 참조가 어떻게 생성될 수 있는지 보여줍니다. 이 예시는 HTML 문서의 개별 요소에 대한 간단한 모델을 제공하는 `HTMLElement`라는 클래스를 정의합니다:

```
class HTMLElement {

    let name: String
    let text: String?

    lazy var asHTML: () -> String = {
        if let text = self.text {
            return "<\(self.name)>\(text)</\(self.name)>"
        } else {
            return "<\(self.name) />"
        }
    }

    init(name: String, text: String? = nil) {
        self.name = name
        self.text = text
    }

    deinit {
        print("\(name) is being deinitialized")
    }

}
```

`HTMLElement` 클래스는 요소의 이름을 나타내는 `name` 프로퍼티를 정의하며, 이 프로퍼티는 제목 요소를 나타내는 `"h1"`, 단락 요소를 나타내는 `"p"`, 개행 요소를 나타내는 `"br"` 등을 나타냅니다. `HTMLElement`는 옵셔널 `text` 프로퍼티도 정의하며, 이 프로퍼티는 HTML 요소 내의 렌더링될 텍스트를 나타내는 문자열을 설정할 수 있습니다.

이 간단한 두 프로퍼티 외에도 `HTMLElement` 클래스는 `asHTML`이라는 지연 프로퍼티(lazy property)를 정의합니다. 이 프로퍼티는 `name`과 `text`를 HTML 문자열로 결합하는 클로저를 참조합니다. `asHTML` 프로퍼티는 `() -> String` 타입 또는 "매개변수를 가지지 않고, `String` 값을 반환하는 함수" 타입입니다.

기본적으로 `asHTML` 프로퍼티는 HTML 태그 문자열을 반환하는 클로저가 할당됩니다. 이 태그는 옵셔널 `text` 값이 있으면 포함하고, `text`가 않으면 텍스트 없이 태그만 포함합니다. 단락 요소에 대해 클로저는 `text` 프로퍼티가 `"some text"`나 `nil`인지에 따라 `"<p>some text</p>"`나 `"<p />"`을 반환합니다.

asHTML 프로퍼티는 인스턴스 메서드와 비슷하게 이름이 지어지고 사용됩니다. 그러나 asHTML은 인스턴스 메서드가 아니라 클로저 프로퍼티이기 때문에, 특정 HTML 요소에 대해 HTML 렌더링을 변경하기 원하면 커스텀 클로저로 asHTML 프로퍼티의 값을 변경할 수 있습니다.

예를 들어 asHTML 프로퍼티는 빈 HTML 태그를 반환하지 않도록 text 프로퍼티가 nil이면 일부 텍스트를 기본값으로 하는 클로저로 설정할 수 있습니다:

```
let heading = HTMLElement(name: "h1")
let defaultText = "some default text"
heading.asHTML = {
    return "<\(heading.name)>\(heading.text ?? defaultText)</\(heading.name)>"
}
print(heading.asHTML())
// Prints "<h1>some default text</h1>".
```

참고

asHTML 프로퍼티는 지연 프로퍼티로 선언되어 있으며, 이것은 요소가 실제로 HTML 출력 타겟에 대한 문자열 값으로 렌더링되어야 하는 경우에만 필요하기 때문입니다. asHTML이 지연 프로퍼티라는 사실은 초기화가 완료되고 self가 존재할 때까지 접근할 수 없으므로 기본 클로저 내에서 self를 참조할 수 있다는 의미입니다.

HTMLElement 클래스는 단일 이니셜라이저를 제공하고, 이것은 새로운 요소를 초기화하기 위해 name 인자와 필요하면 text 인자를 가집니다. 이 클래스는 디이니셜라이저도 정의하고, 이것은 HTMLElement 인스턴스가 해제될 때 메세지를 출력합니다.

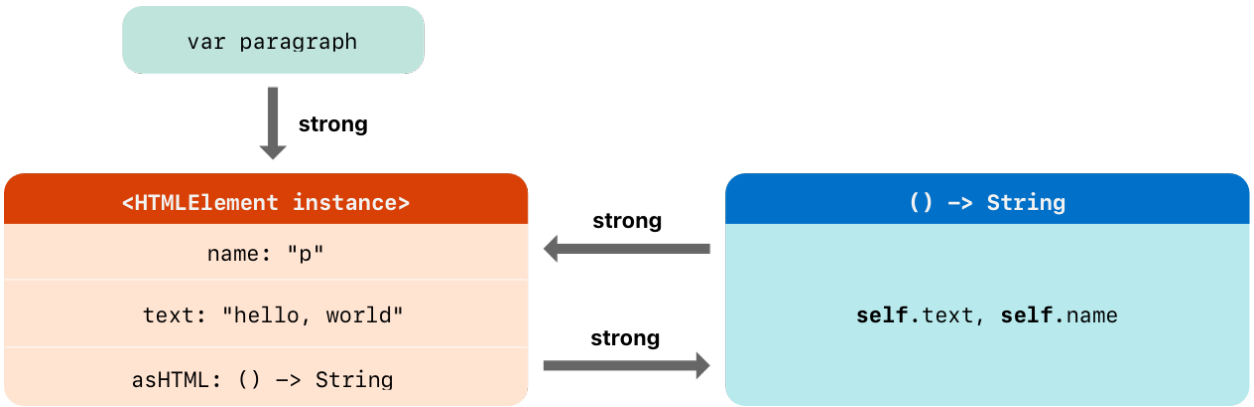
다음은 HTMLElement 클래스가 새로운 인스턴스를 생성하고 출력하는 방법을 보여줍니다:

```
var paragraph: HTMLElement? = HTMLElement(name: "p", text: "hello, world")
print(paragraph!.asHTML())
// Prints "<p>hello, world</p>".
```

참고

위의 paragraph 변수는 옵셔널 HTMLElement로 정의되어 있고, 이것은 강한 순환 참조의 존재를 보여주기 위해 아래에서 nil로 설정할 수 있기 때문입니다.

안타깝게도 위에서 작성한 HTMLElement 클래스는 HTMLElement 인스턴스와 기본 asHTML 값에 사용된 클로저 간에 강한 순환 참조를 생성합니다. 다음은 어떻게 순환이 발생했는지 보여줍니다:



인스턴스의 asHTML 프로퍼티는 클로저에 대해 강한 참조를 유지합니다. 그러나 클로저는 본문 내에서 self를 참조하기 때문에 (self.name과 self.text를 참조하는 방법 처럼), 클로저는 self를 캡처하고, 이것은

`HTMLElement` 인스턴스에 다시 강한 참조를 유지한다는 의미입니다. 이로 인해 둘 사이에 강한 순환 참조가 생성됩니다. (클로저에서 값을 캡처하는 것에 대한 자세한 내용은 캡처값 (Capturing Values)을 참고 바랍니다.)

> **참고**
>
> 클로저가 여러 번 `self`를 참조하지만, `HTMLElement` 인스턴스에 대한 강한 참조는 한 번만 캡처됩니다.

`paragraph` 변수를 `nil`로 설정하고 `HTMLElement` 인스턴스에 대한 강한 참조를 끊으면 강한 순환 참조 때문에 `HTMLElement` 인스턴스와 해당 클로저 모두 해제되지 않습니다:

```
paragraph = nil
```

`HTMLElement` 디이니셜라이저에 메세지는 출력되지 않으며, `HTMLElement` 인스턴스는 해제되지 않음을 보여줍니다.

클로저의 강한 순환 참조 해결 (Resolving Strong Reference Cycles for Closures)

클로저와 클래스 인스턴스 간의 강한 순환 참조는 클로저 정의에 *캡처 리스트(capture list)*를 정의하여 해결합니다. 캡처 리스트는 클로저의 본문 내에서 하나 이상의 참조 타입을 캡처할 때 사용할 규칙을 정의합니다. 두 클래스 간의 강한 순환 참조와 마찬가지로 캡처된 각 참조를 강한 참조가 아닌 약한 참조나 미소유 참조로 선언합니다. 약한 참조나 미소유 참조의 적절한 선택은 코드의 다른 부분 간의 관계에 따라 다릅니다.

> **참고**
>
> Swift는 클로저 내에서 `self`의 멤버를 참조할 때마다 `someProperty`나 `someMethod()`가 아닌 `self.someProperty`나 `self.someMethod()`로 작성해야 합니다. 이것은 실수로 `self`를 캡처할 수 있다는 것을 기억하는데 도움이 됩니다.

캡처 리스트 정의 (Defining a Capture List)

캡처 리스트에서 각 항목은 `weak`나 `unowned` 키워드와 클래스 인스턴스(예: `self`) 또는 어떤 값으로 초기화된 변수(예: `delegate = self.delegate`)의 쌍으로 정의합니다. 여러 쌍은 대괄호 내에 콤마로 구분하여 작성합니다.

클로저에 매개변수 목록과 반환 타입이 있다면, 캡처 리스트를 클로저의 매개변수 목록과 반환 타입 앞에 위치시킵니다:

```
lazy var someClosure = {
    [unowned self, weak delegate = self.delegate]
    (index: Int, stringToProcess: String) -> String in
    // closure body goes here
}
```

클로저가 매개변수 목록이나 반환 타입을 지정하지 않고 문맥에서 추론될 수 있다면, 캡처 리스트를 클로저의 가장 처음에 작성하고 이어서 `in` 키워드를 작성합니다:

```
lazy var someClosure = {
    [unowned self, weak delegate = self.delegate] in
    // closure body goes here
}
```

약한 참조와 미소유 참조 (Weak and Unowned References)

클로저와 캡처한 인스턴스가 항상 서로를 참조하고 항상 같은 시간에 해제될 때 클로저의 캡처를 미소유 참조로 정의합니다.

반대로 캡처된 참조가 향후에 `nil`이 될 수 있다면 약한 참조로 정의합니다. 약한 참조는 항상 옵셔널 타입이고, 참조하는 인스턴스가 해제되면 자동으로 `nil`이 됩니다. 이를 사용하여 클로저의 본문 내에서 해당 참조가 존재하는지 확인할 수 있습니다.

> **참고**
>
> 캡처된 참조가 `nil`이 되지 않으면, 약한 참조보다 미소유 참조로 항상 캡처해야 합니다.

미소유 참조는 위의 클로저의 강한 순환 참조 (Strong Reference Cycles for Closures)에서 `HTMLElement` 예시의 강한 순환 참조를 해결하는데 적절한 캡처 방법입니다. 다음은 순환을 피하기 위해 `HTMLElement` 클래스를 어떻게 작성하는지 보여줍니다:

```
class HTMLElement {

    let name: String
    let text: String?

    lazy var asHTML: () -> String = {
        [unowned self] in
        if let text = self.text {
            return "<\(self.name)>\(text)</\(self.name)>"
        } else {
            return "<\(self.name) />"
        }
    }

    init(name: String, text: String? = nil) {
        self.name = name
        self.text = text
    }

    deinit {
        print("\(name) is being deinitialized")
    }

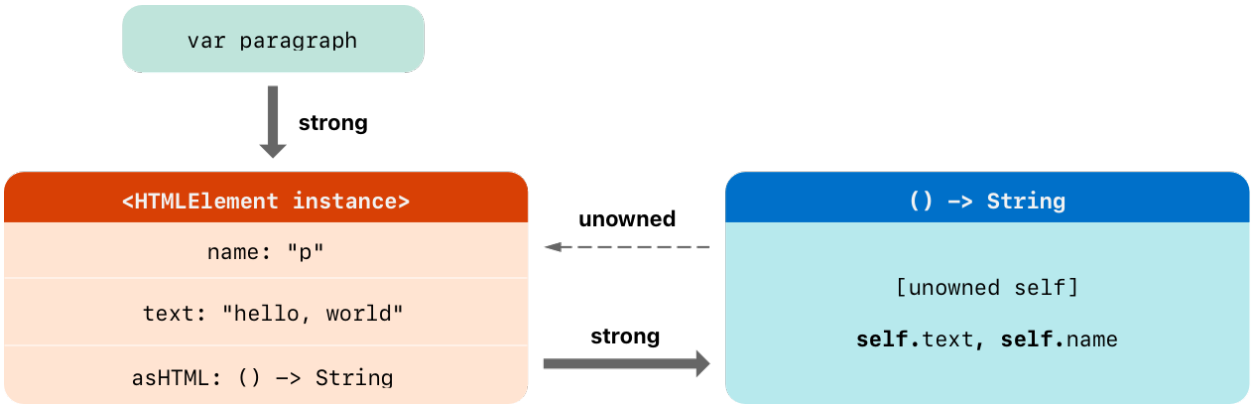
}
```

`HTMLElement`의 이전 구현과 동일하지만, `asHTML` 클로저 내에 캡처 리스트를 추가합니다. 이러한 경우에 캡처 리스트는 `[unowned self]`이고, 이것은 "강한 참조가 아닌 미소유 참조로 `self`를 캡처합니다"라는 의미입니다.

이전과 같이 `HTMLElement` 인스턴스를 생성하고 출력할 수 있습니다:

```
var paragraph: HTMLElement? = HTMLElement(name: "p", text: "hello, world")
print(paragraph!.asHTML())
// Prints "<p>hello, world</p>".
```

다음은 캡처 리스트가 참조에서 어떻게 보이는지 나타냅니다:



이번에는 클로저가 `self`를 미소유 참조로 캡처하므로, `HTMLElement` 인스턴스를 강하게 유지하지 않습니다. `paragraph` 변수의 강한 참조를 `nil`로 설정하면, `HTMLElement` 인스턴스는 해제되고, 디이니셜라이저 메세지를 출력합니다.

```
paragraph = nil
// Prints "p is being deinitialized".
```

캡처 리스트에 대한 더 자세한 내용은 캡처 목록 (Capture Lists)을 참고바랍니다.

언어 가이드

메모리 안전성 (Memory Safety)

메모리 접근할 때 충돌을 피하도록 코드를 구조화합니다.

기본적으로 Swift는 코드에서 발생할 안전하지 않은 동작을 방지합니다. 예를 들어 Swift는 변수가 사용되기 전에 초기화 되고, 메모리가 해제된 후에는 접근되지 않으며, 배열 인덱스는 범위를 벗어난 오류가 있는지 확인합니다.

Swift는 또한 동일한 메모리 영역에 대해 다중 접근이 충돌하지 않도록 보장하고, 이것은 메모리의 위치를 수정하는 코드가 해당 메모리에 대한 접근 권한을 갖도록 보장합니다. Swift는 메모리를 자동으로 관리하기 때문에, 대부분의 경우 메모리 접근에 대해 생각할 필요가 없습니다. 그러나 어디서 충돌이 발생할 수 있는지 이해하는 것이 중요하며, 메모리 접근이 충돌하는 코드를 작성하지 않도록 해야 합니다. 코드에 충돌이 있다면, 컴파일 때나 런타임 때 오류가 발생합니다.

메모리 접근 충돌 이해 (Understanding Conflicting Access to Memory)

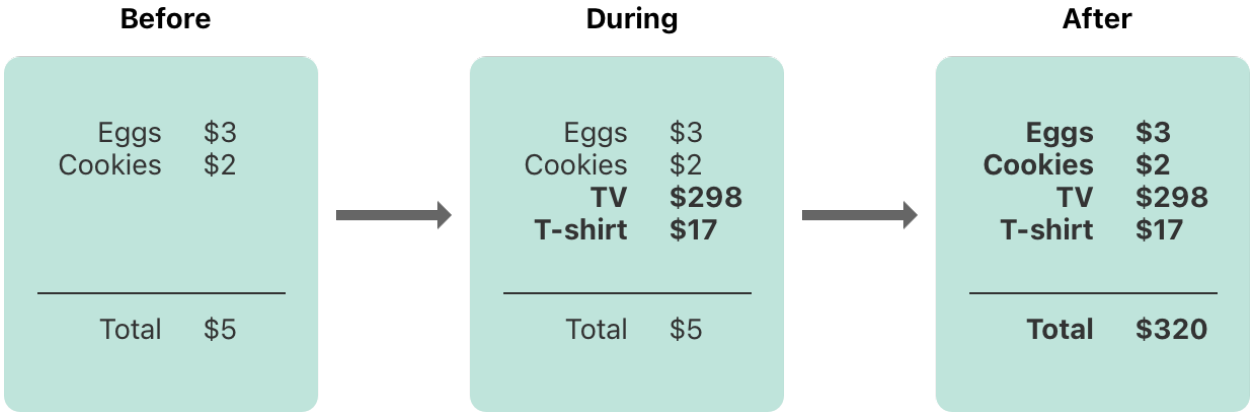
메모리에 접근하는 것은 변수의 값을 설정하거나 함수에 인자를 전달하는 것과 같은 동작을 할 때 코드에서 발생합니다. 예를 들어 다음 코드는 읽기 접근과 쓰기 접근 모두 포함합니다:

```
// A write access to the memory where one is stored.
var one = 1

// A read access from the memory where one is stored.
print("We're number \(one)!")
```

메모리 접근 충돌은 코드의 다른 부분이 같은 시간에 같은 메모리 위치에 접근할 때 발생할 수 있습니다. 같은 시간에 메모리 위치에 다중 접근은 예측할 수 없거나 일관성 없는 동작이 발생할 수 있습니다. Swift는 코드의 여러 라인에 걸쳐 값을 수정할 수 있으므로, 값이 수정되는 중간에 값에 접근할 수 있습니다.

비슷한 문제로 종이에 쓰여진 예산을 어떻게 업데이트 하는지 생각해 볼 수 있습니다. 예산을 업데이트 하는 것은 2단계로 이루어집니다: 먼저 항목의 이름과 가격을 추가한 다음에 현재 목록에 있는 항목을 반영하기 위해 총 금액을 변경합니다. 업데이트 전후에 아래 그림과 같이 예산에서 정보를 읽고 올바른 답을 얻을 수 있습니다.



예산에 항목을 추가하는 동안에는 총 금액이 새로 추가된 항목에 대해 업데이트되지 않았기 때문에 임시적으로 유효하지 않은 상태입니다. 항목을 추가하는 동안 총 금액을 읽으면 잘못된 정보를 얻습니다.

이 예시는 또한 메모리 접근 충돌을 수정할 때 발생할 수 있는 문제를 보여줍니다: 때로는 충돌을 수정하는 방법이 여러 가지일 수 있으며, 각 방법이 서로 다른 결과를 낼 수 있고, 어떤 결과가 올바른지 항상 명확하지 않습니다. 이 예시에서 기존 총 금액이 필요한지, 업데이트된 총 금액이 필요한지에 따라 $5이나 $320이 모두 올바른 답이 될 수 있습니다. 충돌 접근을 수정하기 전에 수행할 작업의 의도를 파악해야 합니다.

> **참고**
>
> 동시성이나 다중 스레드 코드를 작성해본 적이 있다면, 메모리 접근 충돌이 익숙한 문제일 수 있습니다. 그러나 여기서 설명한 접근 충돌은 단일 스레드에서 발생할 수 있으며 동시성이나 다중 스레드 코드와는 관련이 없습니다.
>
> 단일 스레드 내에서 메모리 접근 충돌이 발생하면 Swift는 컴파일 시나 런타임 시 오류가 발생합니다. 다중 스레드 코드의 경우 Thread Sanitizer를 사용하여 스레드 간에 충돌하는 접근을 감지할 수 있습니다.

메모리 접근의 특징 (Characteristics of Memory Access)

메모리 접근이 충돌을 일으킬 수 있는지 판단할 때 고려해야 할 특성은 세 가지입니다: 접근이 읽기 접근인지 쓰기 접근인지, 접근의 지속시간, 접근하는 메모리의 위치입니다. 구체적으로 아래의 조건을 모두 만족하는 두 접근이 있다면 충돌이 발생합니다:

- 두 접근이 모두 읽기 접근인 경우가 아니고, 두 접근이 모두 원자적 접근인 경우도 아닙니다.
- 메모리의 같은 위치에 접근합니다.
- 접근하는 시간이 겹칩니다.

읽기 접근과 쓰기 접근의 차이는 일반적으로 명확합니다: 쓰기 접근은 메모리의 위치의 값을 변경하지만, 읽기 접근은 변경하지 않습니다. 메모리의 위치는 예를 들어 변수, 상수, 프로퍼티와 같이 실제로 접근하는 항목을 나타냅니다. 메모리 접근의 시간은 즉시(instantaneous) 발생하거나 장기적(long-term)일 수 있습니다.

접근이 atomic이라는 것은 `Atomic` 또는 `AtomicLazyReference`의 atomic 연산을 호출하는 경우이거나, C atomic 연산자만 사용하는 경우를 의미합니다; 그렇지 않으면 nonatomic입니다. C의 atomic 함수 목록은 `stdatomic(3)` 메뉴얼 페이지에서 확인 가능합니다.

접근이 즉시인 경우는 접근이 시작되고 종료되기 전에 다른 코드를 실행할 수 없는 경우입니다. 일반적으로 두 개의 즉시 접근은 동시에 발생할 수 없습니다. 대부분 메모리 접근은 즉시 접근입니다. 예를 들어 아래 코드 목록의 모든 읽기 접근과 쓰기 접근은 즉시 접근입니다:

```
func oneMore(than number: Int) -> Int {
    return number + 1
}

var myNumber = 1
myNumber = oneMore(than: myNumber)
print(myNumber)
// Prints "2".
```

그러나 메모리에 접근하는 여러 방법 중에는 다른 코드 실행에 걸쳐있는 장기(long-term) 접근도 있습니다. 즉각 접근(instantaneous access)과 장기 접근(long-term access)의 차이점은 장기 접근이 시작되고 종료되기 전에 다른 코드가 실행될 수 있다는 것입니다. 이것을 *겹침(overlap)*이라 합니다. 장기 접근은 다른 장기 접근이나 즉시 접근과 겹칠 수 있습니다.

겹치는 접근은 함수와 메서드의 in-out 매개변수를 사용하거나 구조체의 mutating 메서드를 사용하는 코드에서 주로 나타납니다. Swift 코드에서 장기 접근이 사용되는 구체적인 경우는 아래에서 설명합니다.

In-Out 매개변수에 대한 접근 충돌 (Conflicting Access to In-Out Parameters)

함수는 모든 in-out 매개변수에 대해 장기 쓰기 접근을 가지고 있습니다. in-out 매개변수에 대한 쓰기 접근은 모든 일반 매개변수가 평가된 후에 시작되고, 해당 함수 호출이 끝날 때까지 지속됩니다. In-out 매개변수가 여러 개인 경우 쓰기 접근은 매개변수가 나타나는 순서와 동일한 순서로 시작됩니다.

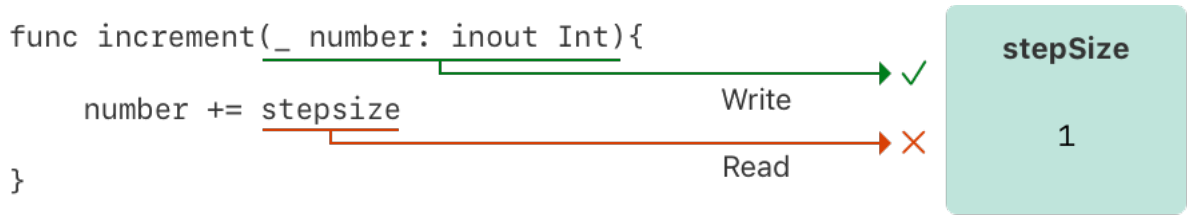
이 장기 쓰기 접근의 결과 중 하나는 범위 규칙과 접근 제어가 허용하더라도 기존 변수에 대한 모든 접근은 충돌을 생성하므로 in-out 파리미터로 전달된 원래 변수에 접근할 수 없습니다. 예를 들어:

```
var stepSize = 1

func increment(_ number: inout Int) {
    number += stepSize
}

increment(&stepSize)
// Error: Conflicting accesses to stepSize.
```

위의 코드에서 `stepSize`는 전역 변수이고, 일반적으로 `increment(_:)` 내에서 접근할 수 있습니다. 그러나 `stepSize`에 읽기 접근은 `number`에 쓰기 접근과 겹칩니다. 아래 그림에서 보았듯이 `number`와 `stepSize` 모두 같은 메모리의 위치를 참조합니다. 읽기와 쓰기 접근이 같은 메모리를 참조하면서 충돌이 발생합니다.



이 충돌을 해결하는 한 가지 방법은 `stepSize`의 복사본을 명확하게 지정하는 것입니다:

```
// Make an explicit copy.
var copyOfStepSize = stepSize
increment(&copyOfStepSize)

// Update the original.
stepSize = copyOfStepSize
// stepSize is now 2
```

`increment(_:)` 호출 전에 `stepSize`의 복사본을 만들면, `copyOfStepSize`는 현재 `stepSize` 값을 기준으로 증가됩니다. 읽기 접근은 쓰기 접근이 시작되기 전에 끝나므로 충돌이 일어나지 않습니다.

in-out 매개변수에 대한 장기 쓰기 접근의 또 다른 결과는 단일 변수를 같은 함수의 여러 in-out 매개변수에 전달하면 충돌이 발생한다는 것입니다. 예를 들어:

```
func balance(_ x: inout Int, _ y: inout Int) {
    let sum = x + y
    x = sum / 2
    y = sum - x
}
var playerOneScore = 42
var playerTwoScore = 30
balance(&playerOneScore, &playerTwoScore)  // OK
balance(&playerOneScore, &playerOneScore)
// Error: Conflicting accesses to playerOneScore.
```

위의 `balance(_:_:)` 함수는 총 값을 균등하게 나누기 위해 두 매개변수를 수정합니다. `playerOneScore`와 `playerTwoScore`를 전달하면 두 쓰기 접근은 동시에 실행되지만 메모리의 다른 위치를 접근하므로 충돌이 발생하지 않습니다. 반대로 두 매개변수 모두에 대해 값으로 `playerOneScore`를 전달하면 동시에 메모리의 같은 위치를 두 쓰기 접근이 수행되므로 충돌이 일어납니다.

참고

연산자는 함수이므로 in-out 매개변수에 장기 접근을 할 수 있습니다. 예를 들어 `balance(_:_:)`가 <^>라는 연산자 함수라면 `playerOneScore <^> playerOneScore`를 작성하면 `balance(&playerOneScore, &playerOneScore)`와 동일한 충돌이 발생합니다.

메서드에서 self에 대한 접근 충돌 (Conflicting Access to self in Methods)

구조체의 mutating 메서드는 메서드가 호출되는 동안 `self`에 대한 쓰기 접근을 가집니다. 예를 들어 각 플레이어는 데미지를 입으면 줄어드는 체력량과 특수 능력을 사용하면 줄어드는 에너지량을 가지는 게임이 있다고 생각해 봅시다.

```
struct Player {
    var name: String
    var health: Int
    var energy: Int

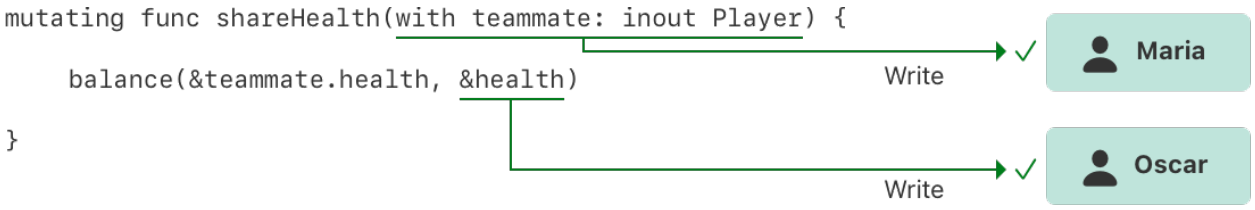
    static let maxHealth = 10
    mutating func restoreHealth() {
        health = Player.maxHealth
    }
}
```

위의 `restoreHealth()` 메서드에서 `self`에 쓰기 접근은 메서드의 처음에서 시작하고 메서드가 반환될 때까지 유지됩니다. 이 경우에 `restoreHealth()` 내부에 `Player` 인스턴스의 프로퍼티에 중복 접근하는 다른 코드는 없습니다. 아래의 `shareHealth(with:)` 메서드는 in-out 매개변수로 다른 `Player` 인스턴스를 가지고 있으며 중복 접근에 대한 가능성을 만듭니다.

```
extension Player {
    mutating func shareHealth(with teammate: inout Player) {
        balance(&teammate.health, &health)
    }
}

var oscar = Player(name: "Oscar", health: 10, energy: 10)
var maria = Player(name: "Maria", health: 5, energy: 10)
oscar.shareHealth(with: &maria)  // OK
```

위의 예시에서 Oscar 플레이어의 체력을 Maria 플레이어와 함께 공유하기 위해 `shareHealth(with:)` 메서드 호출은 중복을 야기시키지 않습니다. `oscar`는 mutating 메서드에서 `self`의 값이기 때문에 해당 메서드 호출 동안 `oscar`에 대한 쓰기 접근이 발생하고, `maria`는 in-out 매개변수로 전달되기 때문에 `maria`에 대한 쓰기 접근이 발생합니다. 아래 그림과 같이 이 접근은 메모리의 다른 위치를 접근합니다. 두 쓰기 접근이 동시에 겹치지만 충돌은 일어나지 않습니다.

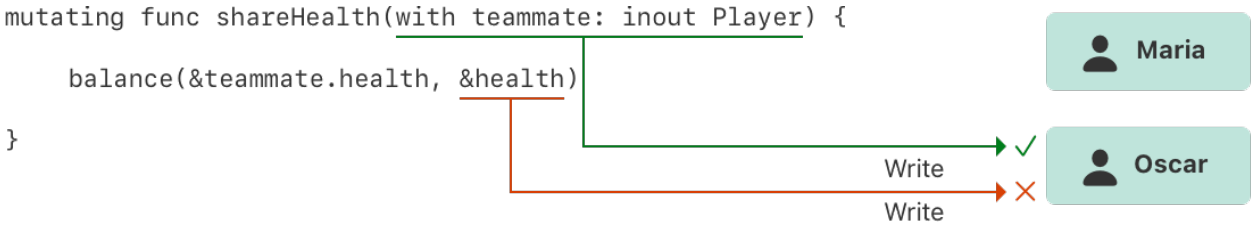


✅ `oscar.shareHealth(with: &maria)`

그러나 `shareHealth(with:)`에 `oscar`를 전달하면 충돌이 발생합니다:

```
oscar.shareHealth(with: &oscar)
// Error: Conflicting accesses to oscar.
```

mutating 메서드는 `self`에 대한 쓰기 접근이 필요하고 동시에 in-out 매개변수는 `teammate`에 쓰기 접근이 필요합니다. 메서드 내에서 `self`와 `teammate` 모두는 아래 그림과 같이 메모리의 같은 위치를 참조합니다. 두 쓰기 접근이 같은 메모리를 참조하므로 충돌이 발생합니다.



❌ `oscar.shareHealth(with: &oscar)`

프로퍼티에 대한 접근 충돌 (Conflicting Access to Properties)

구조체, 튜플, 열거형과 같은 타입은 구조체의 프로퍼티나 튜플의 요소와 같이 개별 구성 값으로 이루어져 있습니다. 이것은 값 타입이기 때문에 값의 일부분을 변경하면 전체 값이 변경되고, 이것은 프로퍼티 중 하나에 읽기 접근이나 쓰기 접근은 전체 값의 읽기 접근이나 쓰기 접근을 요구합니다. 예를 들어 튜플의 요소에 쓰기 접근이 겹치면 충돌이 발생합니다:

```
var playerInformation = (health: 10, energy: 20)
balance(&playerInformation.health, &playerInformation.energy)
// Error: Conflicting access to properties of playerInformation.
```

위의 예시에서 `balance(_:_:)`를 호출할 때 튜플의 요소를 전달하면 `playerInformation`에 중복 쓰기 접근이 므로 충돌이 발생합니다. `playerInformation.health`와 `playerInformation.energy`는 `balance(_:_:)` 함수를 호출할 때 쓰기 접근이 필요하다는 의미의 in-out 매개변수로 전달합니다. 이 경우 튜플 요소의 쓰기 접근은 튜플 전체의 쓰기 접근을 요구합니다. 이것은 `playerInformation`에 두 쓰기 접근이 겹치므로 충돌이 발생합니다.

아래의 코드는 전역 변수에 저장된 구조체의 프로퍼티에 쓰기 접근이 중복될 때 동일한 오류가 발생하는 것을 보여줍니다.

```
var holly = Player(name: "Holly", health: 10, energy: 10)
balance(&holly.health, &holly.energy)  // Error
```

실제로는 대부분의 구조체 프로퍼티에 접근하는 것은 겹쳐도 안전합니다. 예를 들어 위의 예시에서 변수 `holly`는 전역 변수가 아닌 지역 변수로 변경하면 컴파일러는 구조체의 저장 프로퍼티에 중복 접근이 안전하다고 판단합니다:

```
func someFunction() {
    var oscar = Player(name: "Oscar", health: 10, energy: 10)
    balance(&oscar.health, &oscar.energy)  // OK
}
```

위의 예시에서 Oscar의 체력과 에너지는 `balance(_:_:)`에 두 개의 in-out 매개변수로 전달합니다. 컴파일러는 두 개의 저장 프로퍼티가 어떤식으로도 상호작용 하지 않으므로 메모리 안전성이 유지된다고 판단할 수 있습니다.

구조체 프로퍼티의 중복 접근에 대한 제한은 메모리 안전성을 위해 항상 필요한 것은 아닙니다. 메모리 안전성을 보장하는 것을 목표로 하지만, 배타적 접근(exclusive access)은 메모리 안전성보다 더 엄격한 요구사항입니다 - 이것은 어떤 코드는 메모리 안전성은 유지하지만, 배타적 접근을 위반할 수 있다는 의미입니다. Swift는 컴파일러가 메모리의 배타적이지 않은 접근(nonexclusive access)이 안전하다고 증명할 수 있으면 해당 코드를 허용합니다. 특히 아래의 조건이 적용되면 구조체의 프로퍼티에 중복 접근은 안전하다고 증명할 수 있습니다:

- 인스턴스의 저장 프로퍼티만 접근하고, 연산 프로퍼티나 클래스 프로퍼티에는 접근하지 않습니다.
- 구조체는 전역 변수가 아닌 지역 변수의 값입니다.
- 구조체는 클로저에 의해 캡쳐되지 않거나 nonescaping 클로저에 의해서만 캡처됩니다.

컴파일러가 접근이 안전하다는 것을 증명할 수 없으면 접근을 허용하지 않습니다.

언어 가이드

접근 제어 (Access Control)

선언, 파일, 모듈로 코드의 가시성을 관리합니다.

*접근 제어(Access control)*는 다른 소스 파일과 모듈의 코드 접근을 제한합니다. 이 기능은 코드의 세부 구현을 숨기고, 해당 코드에 접근하고 사용할 수 있는 기본 인터페이스를 지정할 수 있습니다.

구체적인 접근 수준을 개별 타입 (클래스, 구조체, 열거형)뿐만 아니라, 해당 타입에 속하는 프로퍼티, 메서드, 이니셜라이저, 서브스크립트에도 지정할 수 있습니다. 프로토콜은 특정 컨텍스트로 제한할 수 있으며, 전역 상수, 변수, 함수도 마찬가지입니다.

다양한 수준의 접근 제어를 제공하는 것 외에도, Swift는 일반적인 상황에 대한 기본 접근 수준을 제공하여 명시적으로 접근 제어 수준을 지정할 필요성을 줄여줍니다. 실제로 단일 앱을 작성한다면, 접근 제어 수준을 지정할 필요가 없습니다.

> **참고**
>
> 접근 제어를 적용할 수 있는 코드의 다양한 요소 (프로퍼티, 타입, 함수 등)는 간결성을 위해 아래에서 “엔티티”라고 합니다.

모듈, 소스 파일, 패키지 (Modules, Source Files, and Packages)

Swift의 접근 제어 모델은 모듈, 소스 파일, 패키지의 개념을 기초로 합니다.

*모듈(module)*은 코드 배포의 단일 단위입니다 - 단일 단위로 빌드되고 Swift의 `import` 키워드로 다른 모듈에서 가져올 수 있는 프레임워크(framework)나 애플리케이션(application)이 모듈에 해당합니다.

Xcode에서 각 빌드 타겟(앱 번들이나 프레임워크)은 Swift에서 개별 모듈로 취급합니다. 앱의 코드를 별도의 프레임워크로 분리하여 - 여러 애플리케이션에서 재사용하도록 구성했다면 - 해당 프레임워크 내의 모든 정의는 앱이나 다른 프레임워크에서 사용할 때 별도의 모듈로 간주됩니다.

*소스 파일(source file)*은 모듈 내 하나의 Swift 소스 코드 파일입니다 (앱이나 프레임워크 내에 단일 파일). 일반적으로 타입마다 별도의 소스 파일에 정의하지만, 하나의 소스 파일에 여러 타입, 함수 등을 정의할 수도 있습니다.

*패키지(package)*는 하나의 단위로 개발되는 모듈의 집합입니다. 패키지를 구성하는 모듈을 Swift 소스 코드에서 정의하지 않고, 사용 중인 빌드 시스템 구성에서 정의합니다. 예를 들어, Swift Package Manager를 사용한다면, `Package.swift` 파일에 PackageDescription 모듈의 API를 사용하여 패키지를 정의하고, Xcode를 사용하면, Package Access Identifier 빌드 설정에서 패키지 이름을 지정합니다.

접근 수준 (Access Levels)

Swift는 코드 내에서 엔티티에 대해 여섯 가지 *접근 수준(access levels)*을 제공합니다. 이 접근 수준은 엔티티가 정의된 소스파일과 관련되며 그 소스 파일이 속한 모듈과 그 모듈이 속한 패키지를 기준으로 결정됩니다.

- open 접근과 public 접근은 정의된 모듈의 모든 소스 파일에서 해당 엔티티를 사용할 수 있으며, 정의한 모듈을 임포트(import)하고 다른 모듈의 소스 파일에서도 엔티티를 사용할 수 있습니다. open 접근이나 public 접근은

일반적으로 프레임워크를 공개 인터페이스로 지정할 때 사용합니다. open 접근과 public 접근의 차이점은 아래에 설명되어 있습니다.

- package 접근은 정의된 패키지 내의 모든 소스 파일에서 해당 엔티티를 사용할 수 있도록 허용하지만, 해당 패키지 외부의 소스 파일에서는 사용할 수 없습니다. package 접근은 여러 모듈로 구성된 앱이나 프레임워크에서 주로 사용합니다.
- internal 접근은 정의된 모듈의 모든 소스 파일에서 해당 엔티티를 사용할 수 있지만, 해당 모듈 외부의 소스 파일에서는 사용할 수 없습니다. internal 접근은 일반적으로 앱이나 프레임워크의 내부 구조체를 정의할 때 사용합니다.
- file-private 접근은 정의된 소스 파일 내에서만 엔티티의 사용을 허용합니다. file-private 접근은 특정 기능의 세부 구현이 파일 전체에서 사용되지만 외부에 노출되지 않도록 하려는 경우에 사용합니다.
- private 접근은 해당 엔티티를 둘러싸는 선언 내부와 같은 파일에 있는 해당 선언의 확장에서만 사용을 허용합니다. private 접근은 특정 기능의 세부 구현이 단일 선언 내에서만 사용되고 외부에 노출되지 않도록 하려는 경우에 사용합니다.

open 접근은 가장 높은(가장 적은 제약) 접근 수준이며 private 접근은 가장 낮은(가장 높은 제약) 접근 수준입니다.

open 접근은 클래스와 클래스 멤버에만 적용되고 아래의 하위 클래스 (Subclassing)에서 설명했듯이 외부 모듈에서 하위 클래스와 재정의를 허용한다는 점에서 public 접근과 다릅니다. 명시적으로 클래스를 open으로 표기하면 다른 모듈이 해당 클래스를 상위 클래스로 사용할 수 있음을 고려하고 그에 따라 클래스 코드를 설계했음을 나타냅니다.

접근 수준의 기본 원칙 (Guiding Principle of Access Levels)

Swift에서 접근 수준은 전반적인 기본 원칙을 따릅니다: 엔티티는 더 낮은(더 제한적) 접근 수준을 가진 다른 엔티티를 기반으로 정의될 수 없습니다.

예를 들어:

- public 변수는 internal, file-private, private 타입으로 정의될 수 없습니다. 해당 타입은 public 변수가 사용되는 모든 곳에서 접근 가능하지 않을 수 있기 때문입니다.
- 함수는 매개변수 타입과 반환 타입보다 더 높은 접근 수준을 가질 수 없습니다. 해당 타입이 호출하는 코드에서 접근 가능하지 않을 수 있기 때문입니다.

이 기본 원칙이 언어의 여러 측면에 어떤 영향을 미치는지는 아래에 자세히 설명되어 있습니다.

기본 접근 수준 (Default Access Levels)

코드의 모든 엔티티 (이 챕터의 후반부에서 설명하는 몇 가지 예외를 제외하고)는 명시적으로 접근 수준을 지정하지 않으면 기본적으로 internal 접근 수준을 가집니다. 이로 인해 많은 경우에 코드에서 명시적으로 접근 수준을 지정할 필요가 없습니다.

단일 타겟 앱에 대한 접근 수준 (Access Levels for Single-Target Apps)

간단한 단일 타겟 앱을 작성할 경우, 앱의 코드는 일반적으로 앱 내에 자체적으로 포함되며 앱의 모듈 외부에서 사용할 수 있도록 만들 필요가 없습니다. internal 접근 수준은 이 요구사항과 일치합니다. 따라서 커스텀 접근 수준을 지정할 필요가 없습니다. 그러나 앱 모듈 내의 다른 코드에서 세부 구현을 숨기기 위해 file private이나 private로 코드의 어떤 부분을 표기할 수 있습니다.

프레임워크에 대한 접근 수준 (Access Levels for Frameworks)

프레임워크를 개발할 때는 프레임워크를 가져오는 앱과 같은 다른 모듈에서 보고 접근할 수 있게 하기 위해 해당 프레임워크에 대한 공용 인터페이스를 open이나 public으로 표시합니다. 이 공용 인터페이스는 프레임워크의 애플리케이션 프로그래밍 인터페이스 또는 API입니다.

> **참고**
>
> 프레임워크의 내부 세부 구현은 여전히 internal 접근 수준을 사용할 수 있으며, 프레임워크의 내부 코드를 숨기기 위해 private이나 file private로 표기할 수 있습니다. 프레임워크 API의 부분이 되길 원한다면 엔티티를 open이나 public으로 표시해야 합니다.

유닛 테스트 타겟에 대한 접근 수준 (Access Levels for Unit Test Targets)

유닛 테스트 타겟이 있는 앱을 작성하는 경우에는 앱의 코드는 테스트 모듈에서 사용할 수 있어야 합니다. 기본적으로 open이나 public으로 표기된 엔티티만 다른 모듈에서 접근할 수 있습니다. 그러나 모듈에 대한 임포트(import) 선언을 `@testable` 속성으로 표시하고 테스트를 활성화하여 해당 모듈을 컴파일하면 유닛 테스트 타겟은 모든 internal 엔티티에 접근할 수 있습니다.

접근 제어 문법 (Access Control Syntax)

엔티티의 선언 앞에 `public`이나 `private`처럼 접근 수준 (Access Levels)에 나열된 수정자 중 하나를 위치시켜 엔티티에 대한 접근 수준을 정의합니다. 예를 들어:

```
public class SomePublicClass {}
internal struct SomeInternalStruct() {}
private func somePrivateFunction() {}
```

위 코드는 `SomePublicClass`는 public으로 `SomeInternalStruct`는 internal로 `somePrivateFunction()`은 private로 선언했습니다. 명시적으로 접근 수준을 작성하지 않으면, 기본 접근 수준 (Default Access Levels)에서 설명한대로 기본 접근 수준 수정자는 `internal`입니다. 예를 들어 아래 코드의 `SomeInternalStruct`는 암시적으로 internal입니다:

```
struct SomeInternalStruct() {}
```

커스텀 타입 (Custom Types)

커스텀 타입에 대해 명시적으로 접근 수준을 지정하고 싶다면 타입을 정의할 때 지정합니다. 그러면 새로운 타입은 접근 수준이 허용하는 범위내에서 사용할 수 있습니다. 예를 들어 file-private 클래스를 정의하면 해당 클래스는 file-private 클래스가 정의된 소스 파일 내에서만 프로퍼티 타입, 함수 매개변수, 반환 타입으로 사용될 수 있습니다.

타입의 접근 제어 수준은 해당 타입의 멤버 (프로퍼티, 메서드, 이니셜라이저, 서브스크립트)의 기본 접근 수준에도 영향을 줍니다. 타입의 접근 수준을 private이나 file private로 정의하면, 멤버의 기본 접근 수준 또한 private이나 file private가 됩니다. 타입의 접근 수준을 internal이나 public (또는 명시적으로 접근 수준을 지정하지 않아 internal의 기본 접근 수준)으로 정의하면 타입의 멤버의 기본 접근 수준은 internal이 됩니다.

중요

Public 타입은 기본적으로 public 멤버가 아닌 internal 멤버를 가집니다. 타입 멤버를 public으로 하려면 명시적으로 표시해야 합니다. 이 요구사항은 타입에 대한 공용 API는 공개되도록 하고 실수로 타입의 내부 작업을 공개 API로 표시되지 않도록 합니다.

```
public class SomePublicClass {                   // explicitly public class
    public var somePublicProperty = 0            // explicitly public class member
    var someInternalProperty = 0                 // implicitly internal class member
    fileprivate func someFilePrivateMethod() {}  // explicitly file-private class member
    private func somePrivateMethod() {}          // explicitly private class member
}

class SomeInternalClass {                        // implicitly internal class
    var someInternalProperty = 0                 // implicitly internal class member
    fileprivate func someFilePrivateMethod() {}  // explicitly file-private class member
    private func somePrivateMethod() {}          // explicitly private class member
}

fileprivate class SomeFilePrivateClass {         // explicitly file-private class
    func someFilePrivateMethod() {}              // implicitly file-private class member
    private func somePrivateMethod() {}          // explicitly private class member
}

private class SomePrivateClass {                 // explicitly private class
    func somePrivateMethod() {}                  // implicitly private class member
}
```

튜플 타입 (Tuple Types)

튜플 타입에 대한 접근 수준은 튜플을 구성하는 타입 중 가장 제한적인 접근 수준으로 결정됩니다. 예를 들어 하나는 internal 접근이고 하나는 private 접근인 두 개의 다른 타입의 튜플을 구성하는 경우 해당 복합 튜플 타입에 대한 접근 수준은 private입니다.

참고

튜플 타입은 클래스, 구조체, 열거형, 함수와 같이 독립적으로 정의되지 않습니다. 튜플 타입의 접근 수준은 튜플 타입을 구성하는 타입에 따라 자동으로 결정되고 명시적으로 지정할 수 없습니다.

함수 타입 (Function Types)

함수 타입에 대한 접근 수준은 함수의 매개변수 타입과 반환 타입 중 가장 제한적인 접근 수준으로 결정됩니다. 함수의 접근 수준이 컨텍스트 기본 접근 수준과 일치하지 않는다면 함수 선언 시 접근 수준을 명시적으로 지정해야 합니다.

아래의 예시는 `someFunction()`이라는 전역 함수를 정의하고, 함수에 접근 수준을 지정하지 않습니다. 이 함수는 "internal"의 기본 접근 수준을 가진다고 생각되지만, 여기서는 그렇지 않습니다. 실제로 `someFunction()`은 아래와 같이 작성되면 컴파일되지 않습니다:

```
func someFunction() -> (SomeInternalClass, SomePrivateClass) {
    // function implementation goes here
}
```

함수의 반환 타입은 위에 커스텀 타입 (Custom Types)에서 정의한 두 개의 커스텀 클래스로 구성된 튜플 타입입니다. 이 클래스 중 하나는 internal로 다른 하나는 private로 정의됩니다. 따라서 튜플 타입의 전체 접근 수준은 private입니다 (튜플의 구성 타입의 최소 접근 수준).

함수의 반환 타입은 private이므로, 함수 선언이 유효하려면 `private` 수정자와 함께 함수의 전체 접근 수준을 표시해야 합니다:

```
private func someFunction() -> (SomeInternalClass, SomePrivateClass) {
    // function implementation goes here
}
```

`someFunction()` 함수가 public이나 internal인 경우에는 함수의 반환 타입에 접근할 수 없기 때문에 함수 정의에 `public`이나 `internal` 수정자로 표시하거나 기본 접근 제어 수준을 사용하는 것은 유효하지 않습니다.

열거형 타입 (Enumeration Types)

열거형의 개별 케이스는 열거형과 같은 접근 수준을 자동으로 받습니다. 각 케이스에 대해 다른 접근 수준을 지정할 수 없습니다.

아래의 예시에서 `CompassPoint` 열거형은 public의 명시적 접근 수준을 갖습니다. 따라서 `north`, `south`, `east`, `west` 열거형 케이스도 public의 접근 수준을 가집니다:

```
public enum CompassPoint {
    case north
    case south
    case east
    case west
}
```

원시값과 연관값 (Raw Values and Associated Values)

열거형 정의의 모든 원시값이나 연관값에 사용되는 타입은 열거형과 같거나 더 높은 접근 수준을 가져야 합니다. 예를 들어 internal 접근 수준을 가진 열거형에 private 타입을 원시값으로 사용할 수 없습니다.

중첩 타입 (Nested Types)

중첩 타입의 접근 수준은 해당 타입이 포함된 타입이 public일 경우를 제외하고 타입과 동일합니다. public 타입 내에 정의된 중첩 타입은 자동으로 internal 접근 수준을 가집니다. public 타입 내에 중첩 타입을 공개하려면, 명시적으로 public 접근 수준으로 중첩 타입을 선언해야 합니다.

하위 클래스 (Subclassing)

현재 접근 컨텍스트에서 접근 가능한 클래스이며, 동일한 모듈에 정의된 클래스는 하위 클래스로 지정할 수 있습니다. 다른 모듈에 정의된 open 클래스도 하위 클래스로 지정할 수 있습니다. 하위 클래스는 상위 클래스의 접근 수준보다 높은 접근 수준을 가질 수 없습니다 - 예를 들어 internal 상위 클래스의 public 하위 클래스를 선언할 수 없습니다.

또한 같은 모듈에 정의된 클래스의 경우 해당 접근 컨텍스트에 표시되는 모든 클래스 멤버 (메서드, 프로퍼티, 이니셜라이저, 서브스크립트)를 재정의할 수 있습니다. 다른 모듈에 정의된 클래스의 경우 모든 open 클래스 멤버를 재정의할 수 있습니다.

재정의는 상속된 클래스 멤버의 접근 수준보다 더 높은 접근 수준으로 설정할 수 있습니다. 아래의 예시에서 클래스 `A`는 public 클래스이고, `someMethod()`라는 file-private 메서드를 가집니다. 클래스 `B`는 `A`를 상속하고 "internal"의 접근 수준을 가집니다. 그럼에도 불구하고 클래스 `B`는 `someMethod()`를 기존 구현보다 높은 접근 수준인 "internal"의 접근 수준을 가진 `someMethod()`의 재정의를 제공합니다:

```
public class A {
    fileprivate func someMethod() {}
}

internal class B: A {
    override internal func someMethod() {}
}
```

상위 클래스의 멤버에 대한 호출이 허용된 접근 수준 컨텍스트 내에서 발생하는 한 하위 클래스 멤버보다 더 낮은 접근 권한을 가진 상위 클래스 멤버를 호출하는 것도 유효합니다 (상위 클래스의 fileprivate 멤버 호출의 경우 같은 소스 파일 내 또는 상위 클래스의 internal 멤버 호출의 경우 같은 모듈 내):

```
public class A {
    fileprivate func someMethod() {}
}

internal class B: A {
    override internal func someMethod() {
        super.someMethod()
    }
}
```

상위 클래스 `A`와 하위 클래스 `B`는 같은 소스 파일에 정의되었으므로, `B`의 `someMethod()`에서 `super.someMethod()`를 호출하는 것은 유효합니다.

상수, 변수, 프로퍼티, 서브스크립트 (Constants, Variables, Properties, and Subscripts)

상수, 변수, 프로퍼티는 타입보다 더 높은 접근 수준을 가질 수 없습니다. 예를 들어 private 타입에 public 프로퍼티를 작성하는 것은 유효하지 않습니다. 유사하게 서브스크립트는 인덱스 타입이나 반환 타입보다 더 높은 접근 수준을 가질 수 없습니다.

상수, 변수, 프로퍼티, 서브스크립트가 private 타입에 사용되는 경우, 상수, 변수, 프로퍼티, 서브스크립트도 `private`로 표시되어야 합니다:

```
private var privateInstance = SomePrivateClass()
```

Getter와 Setter (Getters and Setters)

상수, 변수, 프로퍼티, 서브스크립트에 대한 getter와 setter는 자동으로 속해있는 상수, 변수, 프로퍼티, 서브스크립트와 같은 접근 수준을 받습니다.

해당 변수, 프로퍼티, 서브스크립트의 읽기-쓰기 범위를 제한하기 위해 getter보다 더 낮은 접근 수준으로 setter를 제공할 수 있습니다. setter의 접근 수준을 낮추려면 `var`나 `subscript` 선언 앞에 `fileprivate(set)`, `private(set)`, `internal(set)`, `package(set)`을 작성합니다.

참고

이 규칙은 저장 프로퍼티 뿐만 아니라 연산 프로퍼티에도 적용됩니다. 저장 프로퍼티에 대해 명시적으로 getter와 setter를 작성하지 않아도, Swift는 저장 프로퍼티의 저장소에 대한 접근을 제공하기 위해 getter와 setter를 자동으로 생성합니다. 연산 프로퍼티의 명시적 setter와 같은 방식으로 자동 생성된 setter의 접근 수준을 변경하기 위해 `fileprivate(set)`, `private(set)`, `internal(set)`, `package(set)`을 사용합니다.

아래 예시는 `TrackedString` 이라는 구조체를 정의하고, 문자열 프로퍼티가 몇 번 수정되는지 추적합니다:

```
struct TrackedString {
    private(set) var numberOfEdits = 0
    var value: String = "" {
        didSet {
            numberOfEdits += 1
        }
    }
}
```

`TrackedString` 구조체는 `value`라는 문자열 저장 프로퍼티를 정의하고, 빈 문자열인 `""`의 초기값을 가집니다. 이 구조체는 `numberOfEdits`라는 정수 저장 프로퍼티도 정의하고, `value`가 몇 번이나 수정되는지 추적하는데 사용합니다. 이 추적은 `value` 프로퍼티의 `didSet` 프로퍼티 관찰자로 구현되고, `value` 프로퍼티에 새로운 값이 설정될 때마다 `numberOfEdits`가 증가합니다.

`TrackedString` 구조체와 `value` 프로퍼티는 명시적으로 접근 수준 수정자를 제공하지 않으므로, internal의 기본 접근 수준을 받습니다. 그러나 `numberOfEdits`에 대한 접근 수준은 `private(set)` 수정자로 표기되고, 이것은 프로퍼티의 getter는 여전히 internal의 기본 접근 수준을 가지지만 프로퍼티는 `TrackedString` 구조체의 코드 내에서만 설정 가능함을 나타냅니다. 이렇게 하면 `TrackedString`이 내부적으로 `numberOfEdits` 프로퍼티를 수정할 수 있지만 구조체의 정의 외부에서 사용될 때는 읽기 전용 프로퍼티로 표시할 수 있습니다.

`TrackedString` 인스턴스를 생성하고 문자열 값을 몇 번 수정하면 `numberOfEdits` 프로퍼티 값이 수정된 수와 일치하게 업데이트 되는 것을 확인할 수 있습니다:

```
var stringToEdit = TrackedString()
stringToEdit.value = "This string will be tracked."
stringToEdit.value += " This edit will increment numberOfEdits."
stringToEdit.value += " So will this one."
print("The number of edits is \(stringToEdit.numberOfEdits)")
// Prints "The number of edits is 3".
```

다른 소스 파일에서 `numberOfEdits` 프로퍼티의 현재값은 조회할 수 있지만 다른 소스 파일에서 프로퍼티를 수정할 수 없습니다. 이 제한은 `TrackedString` 편집 추적 기능의 세부 구현을 보호하는 동시에 해당 기능 측면에서 편리한 접근을 제공합니다.

필요한 경우 getter와 setter 모두에 명시적으로 접근 수준을 할당할 수 있습니다. 아래 예시는 `TrackedString` 구조체의 한 버전이고, 해당 구조체는 public의 명시적 접근 수준으로 정의되어 있습니다. 따라서 구조체의 멤버(`numberOfEdits` 프로퍼티 포함)는 기본적으로 internal 접근 수준을 가집니다. `numberOfEdits` 프로퍼티 getter를 public으로 setter를 private로 만들 수 있으며, 이것은 `public`과 `private(set)` 접근 수준 수정자를 함께 사용하면 됩니다:

```
public struct TrackedString {
    public private(set) var numberOfEdits = 0
    public var value: String = "" {
        didSet {
            numberOfEdits += 1
        }
    }
    public init() {}
}
```

이니셜라이저 (Initializers)

커스텀 이니셜라이저는 초기화하는 타입보다 더 낮거나 같은 접근 수준으로 할당할 수 있습니다. 유일한 예외상황은 필수 이니셜라이저입니다 (필수 이니셜라이저 (Required Initializers)에서 설명됨). 필수 이니셜라이저는 자신이 속한 클래스와 동일한 접근 수준을 가져야 합니다.

함수나 메서드 매개변수와 마찬가지로 이니셜라이저의 매개변수의 타입은 이니셜라이저의 자체 접근 수준보다 더 낮을 수 없습니다.

기본 이니셜라이저 (Default Initializers)

기본 이니셜라이저 (Default Initializers)에서 설명했듯이, Swift는 모든 프로퍼티에 기본값을 제공하고 커스텀 이니셜라이저를 제공하지 않는 구조체나 기본 클래스에 대해 인자없는 *기본 이니셜라이저(default initializer)*를 자동으로 제공합니다.

기본 이니셜라이저는 타입이 `public`으로 정의되지 않는 한 초기화하는 타입과 같은 접근 수준을 가집니다. `public`으로 정의된 타입의 경우, 기본 이니셜라이저는 internal로 간주합니다. 다른 모듈에서 사용할 때 인자가 없는 이니셜라이저를 public 타입으로 초기화하려면 명시적으로 타입의 정의에서 public으로 인자가 없는 이니셜라이저를 제공해야 합니다.

구조체의 기본 멤버 이니셜라이저 (Default Memberwise Initializers for Structure Types)

구조체의 모든 저장 프로퍼티가 private라면, 구조체의 기본 멤버 이니셜라이저는 private라고 간주합니다. 마찬가지로 구조체의 모든 저장 프로퍼티가 fileprivate라면, 이니셜라이저는 fileprivate입니다. 그렇지 않으면 이니셜라이저는 internal의 접근 수준을 가집니다.

위의 기본 이니셜라이저와 마찬가지로 다른 모듈에서 사용될 때 멤버 이니셜라이저로 초기화 하는 public 구조체는 타입의 정의에 public 멤버 이니셜라이저를 자체적으로 제공해야 합니다.

프로토콜 (Protocols)

프로토콜 타입에 명시적으로 접근 수준을 할당하려면, 프로토콜을 정의하는 지점에서 지정하면 됩니다. 이를 통해 해당 접근 컨텍스트 내에서만 채택할 수 있는 프로토콜을 생성할 수 있습니다.

프로토콜 정의 내의 각 요구사항의 접근 수준은 자동으로 프로토콜과 같은 접근 수준으로 설정됩니다. 프로토콜 요구사항에 대해 별도로 다른 접근 수준을 설정할 수 없습니다. 이렇게 하면 모든 프로토콜의 요구사항은 프로토콜을 채택하는 모든 타입에서 볼 수 있습니다.

참고

Public 프로토콜을 정의한다면, 프로토콜의 요구사항을 구현할 때 public 접근 수준을 요구합니다. 이것은 public 타입의 멤버는 기본적으로 internal의 접근 수준을 가지는 다른 타입과 다릅니다.

프로토콜 상속 (Protocol Inheritance)

기존 프로토콜을 상속하는 새로운 프로토콜을 정의하면, 새로운 프로토콜은 상속된 프로토콜의 접근 수준 이하여야 합니다. 예를 들어 internal 프로토콜을 상속해서 public 프로토콜을 작성할 수 없습니다.

프로토콜 준수 (Protocol Conformance)

타입은 타입 자체보다 접근 수준이 더 낮은 프로토콜을 준수할 수 있습니다. 예를 들어 다른 모듈에서 사용할 수 있는 public 타입이 프로토콜을 정의한 모듈 내에서만 사용할 수 있는 internal 프로토콜을 준수할 수 있습니다.

타입이 특정 프로토콜을 준수하는 컨텍스트의 접근 수준은 타입의 접근 수준과 프로토콜의 접근 수준 중 더 낮은 수준으로 결정됩니다. 예를 들어 타입이 public이지만 준수하는 프로토콜이 internal인 경우, 해당 타입의 프로토콜에 대한 준수는 internal입니다.

프로토콜을 준수하고 그 요구사항을 구현하기 위해 타입을 작성하거나 확장할 때, 요구사항 구현의 접근 수준은 프로토콜 준수 수준 이상이어야 합니다. 예를 들어 public 타입이 internal 프로토콜을 준수하는 경우, 각 프로토콜 요구사항 구현은 적어도 internal이어야 합니다.

참고

Objective-C처럼 Swift에서 프로토콜 준수성은 전역입니다 -- 같은 프로그램 내에서 하나의 타입이 두 가지 다른 방식으로 같은 프로토콜을 준수하는 것은 불가능 합니다.

확장 (Extensions)

클래스, 구조체, 열거형은 사용할 수 있는 모든 접근 컨텍스트에서 확장할 수 있습니다. 확장에 추가된 모든 타입 멤버는 확장된 기존 타입에 선언된 타입 멤버와 같은 기본 접근 수준을 가집니다. public이나 internal 타입을 확장하면, 추가한 모든 새로운 타입 멤버는 internal의 기본 접근 수준을 가집니다. fileprivate 타입을 확장하면, 추가한 모든 새로운 타입 멤버는 fileprivate의 기본 접근 수준을 가집니다. private 타입을 확장하면, 추가한 모든 새로운 타입 멤버는 private의 기본 접근 수준을 가집니다.

단, 확장에 명시적으로 접근 수준 수정자를 붙이면 (예: `private`) 확장 내에서 정의된 모든 멤버는 새로운 기본 접근 수준으로 설정됩니다. 이 새로운 기본 접근 수준은 확장 내에서 개별 타입 멤버에 의해 재정의될 수 있습니다.

확장을 사용하여 프로토콜 준수성을 추가하는 경우 확장에 명시적 접근 수준 수정자를 제공할 수 없습니다. 대신 프로토콜의 자체 접근 수준은 확장 내 각 구현의 기본 접근 수준으로 적용됩니다.

확장에서 Private 멤버 (Private Members in Extensions)

확장하는 클래스, 구조체, 열거형과 같은 파일에 있는 확장은 확장 안에 코드가 기존 타입 선언의 부분으로 작성된 것처럼 동작합니다. 결과적으로 다음을 수행할 수 있습니다:

- 기존 선언에서 private 멤버를 선언하고, 같은 파일의 확장에서 해당 멤버에 접근 가능합니다.
- 한 확장에서 private 멤버를 선언하고, 같은 파일의 다른 확장에서 해당 멤버에 접근 가능합니다.
- 확장에서 private 멤버를 선언하고, 같은 파일의 기존 선언에서 해당 멤버에 접근 가능합니다.

이 동작은 타입에 private 엔티티가 있는지 여부에 관계없이 확장을 같은 방법으로 사용하여 코드를 구성할 수 있다는 의미입니다. 예를 들어 다음과 같은 간단한 프로토콜이 있습니다:

```
protocol SomeProtocol {
    func doSomething()
}
```

아래와 같이 프로토콜 준수성을 추가하기 위해 확장을 사용할 수 있습니다:

```
struct SomeStruct {
    private var privateVariable = 12
}

extension SomeStruct: SomeProtocol {
    func doSomething() {
        print(privateVariable)
    }
}
```

제네릭 (Generics)

제네릭 타입이나 제네릭 함수의 접근 수준은 제네릭 타입이나 제네릭 함수 자체의 접근 수준과 제네릭 타입 매개변수의 타입 제약의 접근 수준 중 더 낮은 접근 수준으로 결정됩니다.

타입 별칭 (Type Aliases)

정의한 모든 타입 별칭은 접근 제어 측면에서 고유한 타입으로 처리됩니다. 타입 별칭은 원래 타입의 접근 수준 이하의 접근 수준을 가질 수 있습니다. 예를 들어 private 타입 별칭은 private, fileprivate, internal, public, open 타입의 별칭이 가능하지만, public 타입 별칭은 internal, fileprivate, private 타입의 별칭이 불가능합니다.

참고

이 규칙은 프로토콜 준수를 충족하는데 사용되는 관련된 타입의 타입 별칭에도 적용됩니다.

언어 가이드

고급 연산자 (Advanced Operators)

커스텀 연산자를 정의하고, 비트 연산을 수행하고, 빌더 구문을 사용합니다.

기본 연산자 (Basic Operators)에서 설명한 연산자 외에도, Swift는 더 복잡한 값을 조작하는 여러 고급 연산자를 제공합니다. 여기에는 C와 Objective-C에서 익숙한 비트 연산자와 비트 이동 연산자가 포함됩니다.

C의 산술 연산자와 달리, Swift의 산술 연산자는 기본적으로 오버플로우(overflow)가 발생하지 않습니다. 오버플로우 동작이 감지되면 오류로 보고됩니다. 오버플로우 동작을 원한다면, 오버플로우 덧셈 연산자(&+)와 같이 오버플로우를 허용하는 Swift의 별도 연산자를 사용합니다. 이런 오버플로우 연산자는 앰퍼샌드(&)로 시작합니다.

구조체, 클래스, 열거형을 정의할 때, 해당 타입에 대한 표준 Swift 연산자를 직접 구현하여 제공하는 게 유용할 수 있습니다. Swift는 이러한 연산자의 맞춤형 구현을 쉽게 제공하고 각 타입에 대한 동작이 정확히 무엇인지 결정할 수 있습니다.

기본 제공 연산자로 제한하지 않습니다. Swift는 커스텀 우선순위와 결합 방향성을 정의해 커스텀 중위(infix) 연산자, 접두사(prefix) 연산자, 접미사(postfix) 연산자, 할당(assignment) 연산자를 자유롭게 정의할 수 있습니다. 이러한 연산자는 기본 제공 연산자처럼 사용할 수 있고, 기존 타입을 확장하여 정의한 커스텀 연산자를 지원할 수도 있습니다.

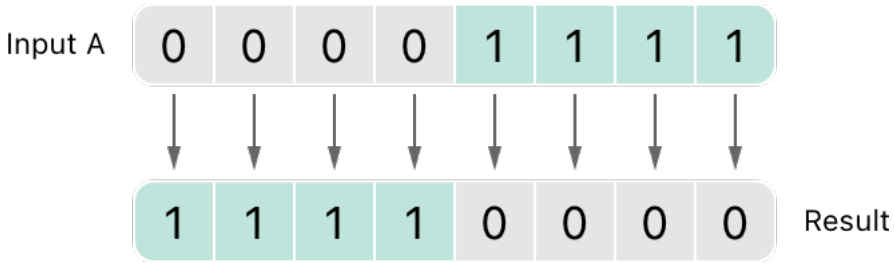
비트 연산자 (Bitwise Operators)

*비트 연산자(Bitwise operators)*는 데이터 구조 내에서 개별 원시 데이터 비트를 조작할 수 있게 해줍니다. 그래픽 프로그래밍과 디바이스 드라이버 생성과 같은 low-level 프로그래밍에서 자주 사용합니다. 비트 연산자는 커스텀 프로토콜의 통신을 위한 데이터 인코딩과 디코딩처럼 외부 소스의 원시 데이터로 작업할 때도 유용할 수 있습니다.

Swift는 아래에서 설명한대로, C에서의 모든 비트 연산자를 지원합니다.

비트 NOT 연산자 (Bitwise NOT Operator)

비트 NOT 연산자(~)는 숫자의 모든 비트를 반전시킵니다:



비트 NOT 연산자는 접두사 연산자이고, 공백없이 동작할 값 바로 앞에 위치합니다:

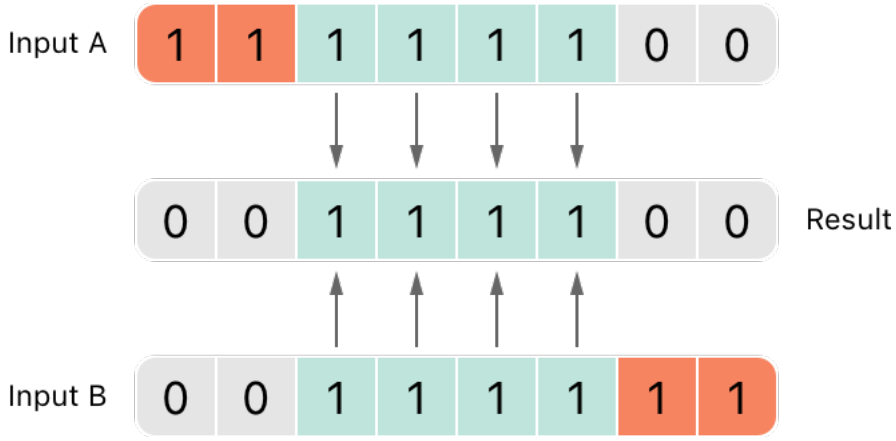
```
let initialBits: UInt8 = 0b00001111
let invertedBits = ~initialBits  // equals 11110000
```

`UInt8` 정수는 8비트를 가지고 있으며, `0`과 `255` 사이의 모든 값을 저장할 수 있습니다. 이 예시는 `UInt8` 정수를 이진값 `00001111`로 초기화하고, 이것은 처음 4비트는 `0`으로 설정하고 다음 4비트는 `1`로 설정합니다. 이것은 십진수 `15`와 같습니다.

비트 NOT 연산자는 `invertedBits`라는 새로운 상수를 생성하기 위해 사용되고, 이 상수는 `initialBits`와 같지만 모든 비트를 반전합니다. 0은 1이 되고 1은 0이 됩니다. `invertedBits`의 값은 `11110000`이고, 이것은 부호 없는 십진수 `240`과 같습니다.

비트 AND 연산자 (Bitwise AND Operator)

비트 AND 연산자(&)는 두 숫자의 비트를 결합합니다. 입력한 숫자 모두 비트가 1일때만 비트에 1을 설정하는 새로운 숫자를 반환합니다:

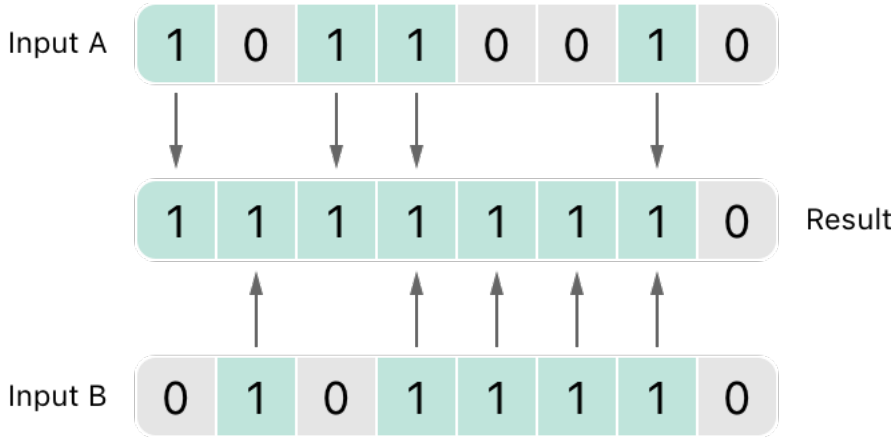


아래의 예시에서 `firstSixBits`와 `lastSixBits`의 값 모두 가운데 4비트에 1을 가지고 있습니다. 비트 AND 연산자는 `00111100`을 만들고, 부호가 없는 십진수 `60`과 같습니다:

```
let firstSixBits: UInt8 = 0b11111100
let lastSixBits: UInt8  = 0b00111111
let middleFourBits = firstSixBits & lastSixBits  // equals 00111100
```

비트 OR 연산자 (Bitwise OR Operator)

비트 OR 연산자(|)는 두 숫자의 비트를 비교합니다. 이 연산자는 입력한 숫자 중 하나라도 비트가 1이면 비트에 1을 설정하는 새로운 숫자를 반환합니다:

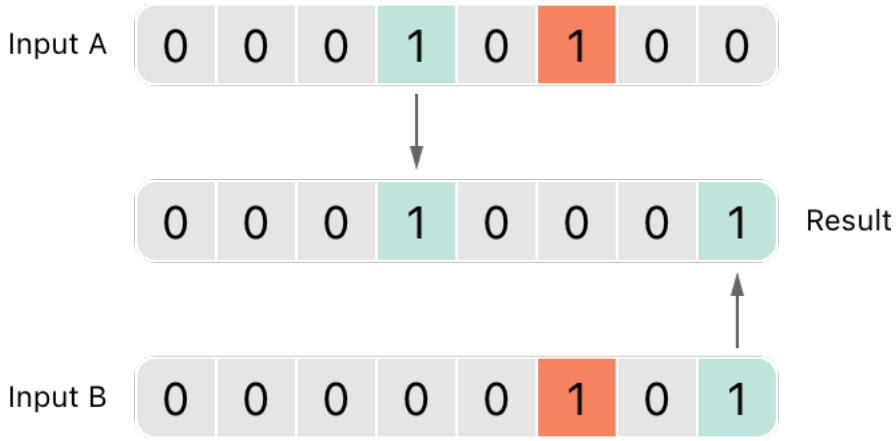


아래 예시에서 `someBits`와 `moreBits`의 값은 다른 비트에 1을 설정합니다. 비트 OR 연산자는 `11111110` 을 만들고, 부호가 없는 십진수 `254`와 같습니다:

```
let someBits: UInt8 = 0b10110010
let moreBits: UInt8 = 0b01011110
let combinedbits = someBits | moreBits  // equals 11111110
```

비트 XOR 연산자 (Bitwise XOR Operator)

비트 XOR 연산자나 "배타적인 OR 연산자"(^)는 두 숫자의 비트를 비교합니다. 이 연산자는 입력한 비트가 같으면 `0`을 설정하고 다르면 1을 설정하는 새로운 숫자를 반환합니다:



아래 예시에서 `firstBits`와 `otherBits`의 값은 각각 다른 위치에 1로 설정된 비트를 가집니다. 비트 XOR 연산자는 이러한 두 비트를 1로 설정합니다. `firstBits`와 `otherBits`의 나머지 비트는 일치하므로 출력값으로 `0`을 설정합니다:

```
let firstBits: UInt8 = 0b00010100
let otherBits: UInt8 = 0b00000101
let outputBits = firstBits ^ otherBits  // equals 00010001
```

비트 왼쪽과 오른쪽 이동 연산자 (Bitwise Left and Right Shift Operators)

비트 왼쪽 이동 연산자(<<)와 비트 오른쪽 이동 연산자(>>)는 아래의 정의된 규칙에 따라 특정 숫자만큼 왼쪽이나 오른쪽으로 숫자의 모든 비트를 이동합니다.

비트 왼쪽과 오른쪽 이동은 정수를 2배로 곱하거나 나누는 효과가 있습니다. 한 비트만큼 왼쪽으로 정수의 비트를 이동하면 값이 2배가 되고, 반대로 한 비트만큼 오른쪽으로 이동하면 기존값의 반이 됩니다.

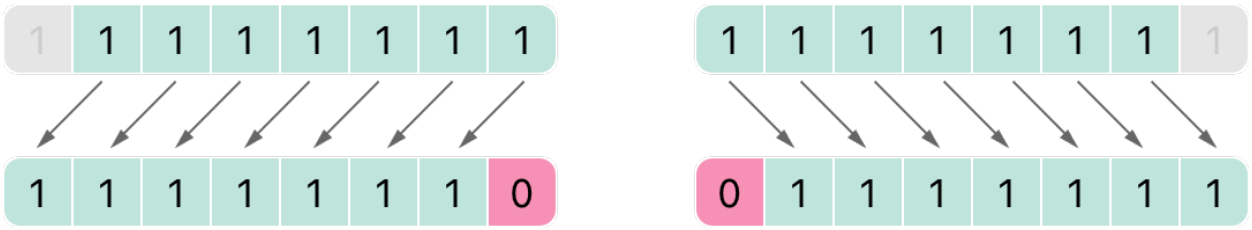
부호 없는 정수의 이동 동작 (Shifting Behavior for Unsigned Integers)

부호 없는 정수의 비트 이동 동작은 다음과 같습니다:

1. 기존 비트는 요청된 숫자만큼 왼쪽이나 오른쪽으로 이동됩니다.
2. 정수의 저장소 범위를 넘어 이동된 모든 비트는 삭제됩니다.
3. 비트가 왼쪽이나 오른쪽으로 이동한 후 뒤에 남겨진 공백에 0을 삽입합니다.

이 접근방식을 *논리 시프트(logical shift)*라고 합니다.

아래 그림은 `11111111 << 1` (11111111을 왼쪽으로 1자리 이동), `11111111 >> 1` (11111111을 오른쪽으로 1자리 이동)의 결과를 보여줍니다. 초록 숫자는 이동되고, 회색 숫자는 삭제되고, 핑크 0이 삽입됩니다:



다음은 Swift 코드에서 비트 이동이 어떻게 이루어지는지 나타냅니다:

```
let shiftBits: UInt8 = 4   // 00000100 in binary
shiftBits << 1             // 00001000
shiftBits << 2             // 00010000
shiftBits << 5             // 10000000
shiftBits << 6             // 00000000
shiftBits >> 2             // 00000001
```

다른 데이터 타입의 값을 인코딩이나 디코딩하기 위해 비트 이동을 사용할 수 있습니다:

```
let pink: UInt32 = 0xCC6699
let redComponent = (pink & 0xFF0000) >> 16    // redComponent is 0xCC, or 204
let greenComponent = (pink & 0x00FF00) >> 8   // greenComponent is 0x66, or 102
let blueComponent = pink & 0x0000FF           // blueComponent is 0x99, or 153
```

이 예시는 분홍색의 Cascading Style Sheets 색상값을 저장하기 위해 `pink`라는 `UInt32` 상수를 사용합니다. CSS 색상값 `#CC6699`는 Swift의 16진수 표현에서 `0xCC6699`로 작성됩니다. 이 색상은 비트 AND 연산자(`&`)와 비트 오른쪽 이동 연산자(`>>`)에 의해 빨강색(`CC`), 녹색(`66`), 파랑색(`99`)으로 분리됩니다.

빨간색 요소는 `0xCC6699`와 `0xFF0000`의 비트 AND 연산자를 수행하여 얻습니다. `0xFF0000`의 0은 `0xCC6699`의 두 번째와 세 번째 바이트를 효과적으로 "마스킹"하여 `6699`를 무시하고 결과로 `0xCC0000`을 남깁니다.

그런 다음 이 숫자는 오른쪽으로 16자리 이동합니다(`>> 16`). 16진수의 각 문자 쌍은 8비트를 사용하므로 오른쪽으로 16자리 이동하면 `0xCC0000`을 `0x0000CC`로 변환합니다. 이것은 `0xCC`와 같으며, 십진수 `204`와 동일합니다.

유사하게 녹색 요소는 `0xCC6699`와 `0x00FF00`의 비트 AND 연산자를 수행하여 `0x006600`의 출력값을 얻습니다. 그런 다음 이 출력값을 오른쪽으로 8자리 이동하여 `0x66`의 값을 얻고 이것은 십진수 `102`와 같습니다.

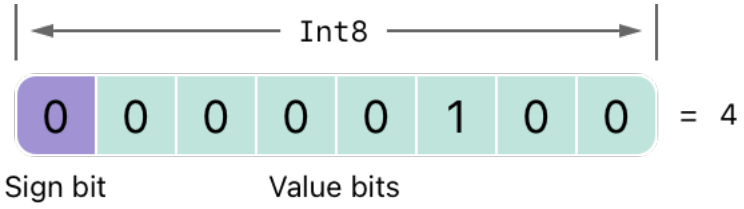
마지막으로 파란색 요소는 `0xCC6699`와 `0x0000FF`의 비트 AND 연산자를 수행하여 `0x000099`의 출력값을 얻습니다. `0x000099`는 이미 `0x99`와 같고 십진수 `153`의 값이므로 오른쪽으로 이동시킬 필요가 없습니다.

부호 있는 정수의 이동 동작 (Shifting Behavior for Signed Integers)

부호 있는 정수의 이동 동작은 부호 없는 정수보다 복잡하며, 이것은 부호 있는 정수가 이진수로 표현되는 방식 때문입니다. (아래 예시는 8비트 부호 있는 정수를 기반으로 설명하지만 모든 크기의 부호 있는 정수에 적용됩니다).

부호 있는 정수는 첫 번째 비트(부호 비트(sign bit))를 사용해 정수가 양수인지 음수인지 나타냅니다. `0`의 부호 비트는 양수를 의미하고 `1`의 부호 비트는 음수를 의미합니다.

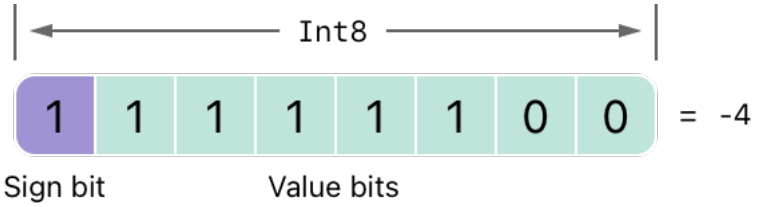
나머지 비트(값 비트(value bits))는 실제 정수값을 저장합니다. 양수는 부호 없는 정수와 같은 방법으로 `0`부터 차례대로 저장됩니다. 다음은 `Int8` 타입이 숫자 `4`를 어떻게 나타내는지 보여줍니다:



부호 비트가 `0`("양수")이고, 7개 비트에 이진수 `4`가 표현되어 있습니다.

그러나 음수는 다르게 저장됩니다. 절대값을 2의 n 제곱에서 빼서 저장되고, 여기서 n은 값 비트의 개수입니다. 8비트 숫자는 7개의 값 비트를 가지므로, 2의 7 제곱이나 128을 의미합니다.

다음은 `Int8` 타입이 숫자 –4를 어떻게 나타내는지 보여줍니다:

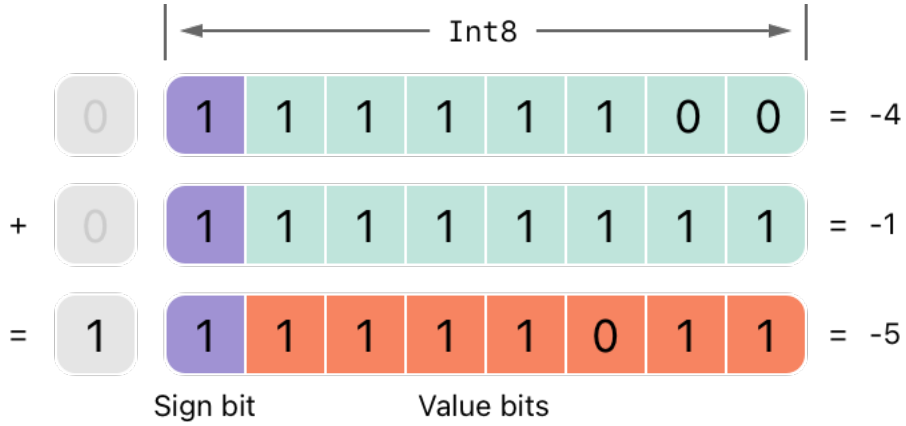


이번에는 부호 비트가 1("음수")이고, 7개 비트에 124(128 – 4)의 이진수를 가집니다:

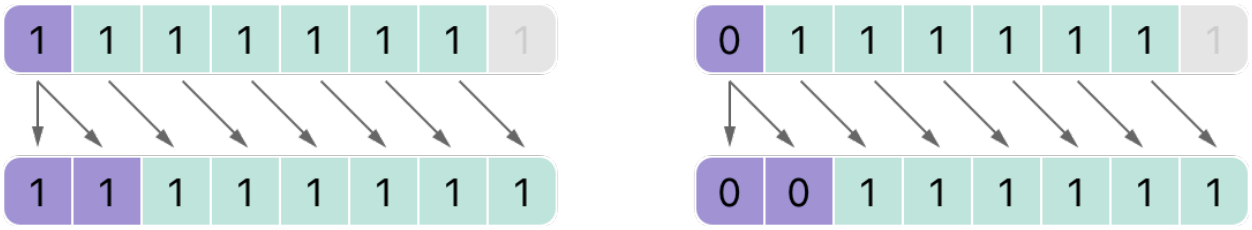


이런 음수 표현을 2의 보수(two's complement) 표현이라고 합니다. 처음에는 이런 표현이 특별한 방법으로 보일 수 있지만, 여러 가지 장점이 있습니다.

첫 번째로 –4에 –1을 더할 수 있고, 이것은 8비트 전체 (부호 비트 포함)를 표준 이진 덧셈을 수행하고, 연산 결과가 8비트를 초과하면 초과된 비트는 버릴 수 있습니다:



두 번째로 2의 보수 표현을 사용하면 음수의 비트도 양수처럼 왼쪽과 오른쪽으로 이동할 수 있고, 왼쪽으로 이동할 때마다 두 배로 늘리거나 오른쪽으로 이동할 때마다 반으로 줄입니다. 이를 위해 부호 있는 정수를 오른쪽으로 이동할 때는 추가 규칙이 있습니다: 부호 있는 정수를 오른쪽으로 이동할 때, 부호 없는 정수와 같은 규칙이 적용되지만 왼쪽에 비어있는 모든 비트에는 0이 아닌 *부호 비트(sign bit)*를 채웁니다.



이러한 작업은 부호 있는 정수가 오른쪽으로 이동한 후에 같은 부호를 갖도록 하고 이것을 *산술 이동(arithmetic shift)*이라고 합니다.

양수와 음수에 저장되는 특별한 방식 때문에, 이것을 오른쪽으로 이동하면 0에 가까워 집니다. 이동하는 동안 부호 비트를 동일하게 유지한다는 것은 값이 0에 가까워 질 때 음의 정수가 음수로 유지됨을 의미합니다.

오버플로우 연산자 (Overflow Operators)

정수 상수나 정수 변수에 가질 수 없는 숫자 값을 삽입하려고 하면 기본적으로 Swift는 잘못된 값을 생성하지 않고 오류가 발생합니다. 이러한 동작은 숫자가 너무 크거나 너무 작은 값으로 작업할 때 추가 안전성을 제공합니다.

예를 들어 `Int16` 정수 타입은 `-32768`과 `32767` 사이의 모든 부호 있는 정수를 가질 수 있습니다. `Int16` 상수나 변수에 범위를 벗어나는 숫자를 설정하려고 하면 오류가 발생합니다:

```
var potentialOverflow = Int16.max
// potentialOverflow equals 32767, which is the maximum value an Int16 can hold
potentialOverflow += 1
// this causes an error
```

값이 너무 크거나 너무 작을 때 오류 처리를 제공하면 경계값 조건을 코딩할 때 더 많은 유연성을 얻을 수 있습니다.

그러나 오버플로우가 발생해도 값을 잘라내면서 계속 진행하고 싶을 때는 오류가 발생하는 대신 오버플로우 동작을 선택할 수 있습니다. Swift는 정수 계산을 위한 오버플로우 동작을 선택하는 세 가지 산술 *오버플로우 연산자(overflow operators)*를 제공합니다. 이러한 연산자는 모두 앰퍼샌드(&)로 시작합니다:

- 오버플로우 덧셈(&+)
- 오버플로우 뺄셈(&-)
- 오버플로우 곱셈(&*)

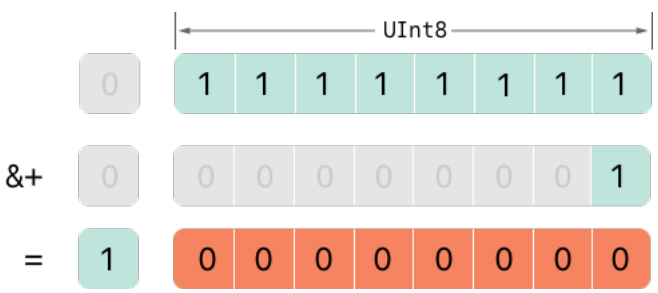
값 오버플로우 (Value Overflow)

숫자는 양의 방향과 음의 방향으로 오버플로우 될 수 있습니다.

다음은 부호 없는 정수가 오버플로우 덧셈 연산자(&+)를 사용하여 양의 방향으로 오버플로우를 허용하면 무슨 일이 발생하는지를 나타내는 예시입니다:

```
var unsignedOverflow = UInt8.max
// unsignedOverflow equals 255, which is the maximum value a UInt8 can hold
unsignedOverflow = unsignedOverflow &+ 1
// unsignedOverflow is now equal to 0
```

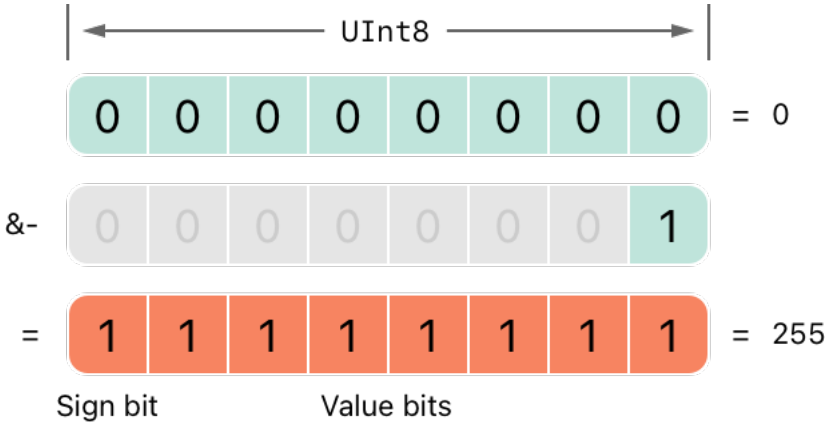
변수 `unsignedOverflow`는 `UInt8`의 최대값(`255`이나 이진수로 `11111111`)으로 초기화됩니다. 그런 다음 오버플로우 덧셈 연산자(&+)를 사용하여 `1`만큼 증가시킵니다. 이것은 `UInt8`이 가질 수 있는 크기보다 큰 바이너리 표현이 적용되어 아래 다이어그램과 같이 경계를 넘어 오버플로우 됩니다. 오버플로우 덧셈 후 `UInt8`의 범위 내에 남아 있는 값은 `00000000` 또는 0입니다.



부호 없는 정수가 음의 방향으로 오버플로우될 때 비슷한 일이 발생합니다. 다음은 오버플로우 뺄셈 연산자(&-)를 사용하는 예시입니다:

```
var unsignedOverflow = UInt8.min
// unsignedOverflow equals 0, which is the minimum value a UInt8 can hold
unsignedOverflow = unsignedOverflow &- 1
// unsignedOverflow is now equal to 255
```

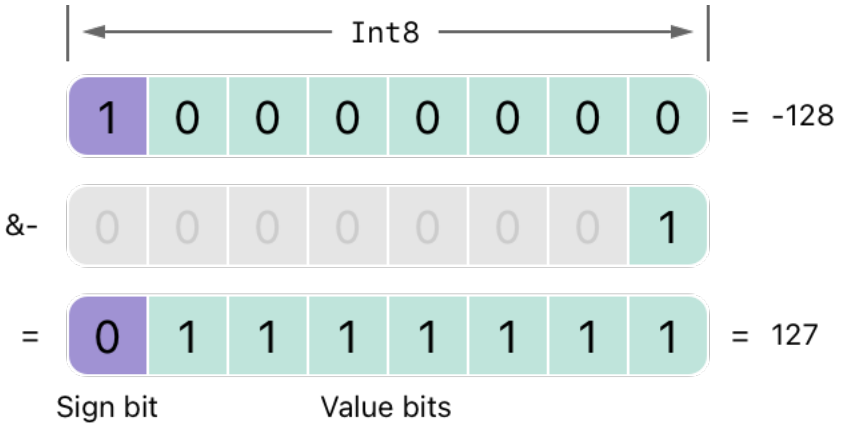
`UInt8`의 최소값은 0이나 이진수로 `00000000`을 가질 수 있습니다. 오버플로우 뺄셈 연산자(&-)를 이용하여 `00000000`에서 1을 뺀다면 숫자는 오버플로우 되고 `11111111`이나 십진수 255입니다.



오버플로우는 부호 있는 정수에서도 발생합니다. 부호 있는 정수에 대한 모든 덧셈과 뺄셈은 비트 방식으로 수행되며 부호 비트는 비트 왼쪽과 오른쪽 이동 연산자 (Bitwise Left and Right Shift Operators)에서 설명한대로 덧셈이나 뺄셈 계산에 포함됩니다.

```
var signedOverflow = Int8.min
// signedOverflow equals -128, which is the minimum value an Int8 can hold
signedOverflow = signedOverflow &- 1
// signedOverflow is now equal to 127
```

`Int8`의 최소값은 `-128`이나 이진수 `10000000`을 가질 수 있습니다. 오버플로우 연산자를 사용하여 이진수에 1을 빼면 이진수 `01111111`이 되고, 이것은 부호 비트를 토글하고 `Int8`이 가질 수 있는 최대 양수값 127이 됩니다.



부호 있는 정수와 부호 없는 정수에 대해 양의 방향의 오버플로우는 최대 유효 정수값에서 최소값으로 돌아가고 음의 방향의 오버플로우는 최소값에서 최대값으로 순환합니다.

우선순위와 결합방향 (Precedence and Associativity)

연산자 *우선순위(precedence)*는 어떤 연산자에 다른 연산자 보다 높은 우선순위를 부여합니다; 이러한 연산자는 먼저 적용됩니다.

연산자 *결합방향(associativity)*은 우선순위가 같은 연산자를 어떻게 결합하는지 정의합니다 - 왼쪽에서부터 결합하는지 오른쪽에서부터 결합하는지 정의합니다. “연산자가 왼쪽의 표현식과 결합합니다” 또는 “연산자가 오른쪽의 표현식과 결합합니다”라는 의미로 생각해야 합니다.

복합 표현식이 계산되는 순서를 이해하기 위해서 각 연산자의 우선순위와 결합방향을 고려하는 것이 중요합니다. 예를 들어 연산자 우선순위는 다음 표현식이 왜 17인지를 설명합니다.

```
2 + 3 % 4 * 5
// this equals 17
```

왼쪽에서 오른쪽으로 읽는다면 표현식은 아래와 같이 계산될 것으로 예상할 수 있습니다:

- 2 더하기 3은 5
- 5 나머지 4는 1
- 1 곱하기 5는 5

그러나 실제 정답은 5가 아닌 17입니다. 우선순위가 높은 연산자는 우선순위가 낮은 연산자보다 먼저 계산됩니다. Swift에서는 C와 같이 나머지 연산자(%)와 곱셈 연산자(*)는 덧셈 연산자(+)보다 우선순위가 높습니다. 결과적으로 나머지 연산자와 곱셈 연산자 모두 덧셈을 고려하기 전에 계산됩니다.

그러나 나머지와 곱셈은 서로 동일한 우선순위를 가집니다. 정확한 순서로 계산을 진행하려면 서로의 결합방향도 고려해야 합니다. 나머지와 곱셈은 모두 연산자 왼쪽의 표현식과 결합됩니다. 연산자 왼쪽에서 시작하여 표현식에 암시적으로 괄호가 있는 것으로 생각해야 합니다:

```
2 + ((3 % 4) * 5)
```

`(3 % 4)`은 3이므로 다음과 같습니다:

```
2 + (3 * 5)
```

`(3 * 5)`는 15이므로 다음과 같습니다:

```
2 + 15
```

이 계산의 결과는 17입니다.

연산자 우선순위 그룹과 결합방향 설정의 전체 목록을 포함하여 Swift 표준 라이브러리에 의해 제공되는 연산자에 대한 자세한 내용은 연산자 선언 (Operator Declarations)을 참고바랍니다.

> **참고**
>
> Swift의 연산자 우선순위와 결합방향 규칙은 C와 Objective-C보다 더 간단하고 예측 가능합니다. 그러나 이것은 C 기반 언어와 정확하게 일치하지 않음을 의미합니다. 기존 코드를 Swift로 이식할 때 연산자 상호작용이 의도한 대로 작동하는지 계속 확인해야 합니다.

연산자 메서드 (Operator Methods)

클래스와 구조체는 기존 연산자의 자체 구현을 제공할 수 있습니다. 이것을 기존 연산자 *오버로딩(overloading)*이라 합니다.

아래의 예시는 커스텀 구조체에 산술 덧셈 연산자(+)를 어떻게 구현하는지 보여줍니다. 이 산술 덧셈 연산자는 두 대상에서 동작하기 때문에 이항 연산자(binary operator)이고, 두 대상 사이에 위치하기 때문에 중위 연산자(infix operator)입니다.

이 예시는 2차원 위치 벡터 `(x, y)`에 대한 `Vector2D` 구조체를 정의한 다음 `Vector2D` 구조체의 인스턴스를 함께 더하기 위해 *연산자 메서드(operator method)*를 구현합니다:

```
struct Vector2D {
    var x = 0.0, y = 0.0
}

extension Vector2D {
    static func + (left: Vector2D, right: Vector2D) -> Vector2D {
        return Vector2D(x: left.x + right.x, y: left.y + right.y)
    }
}
```

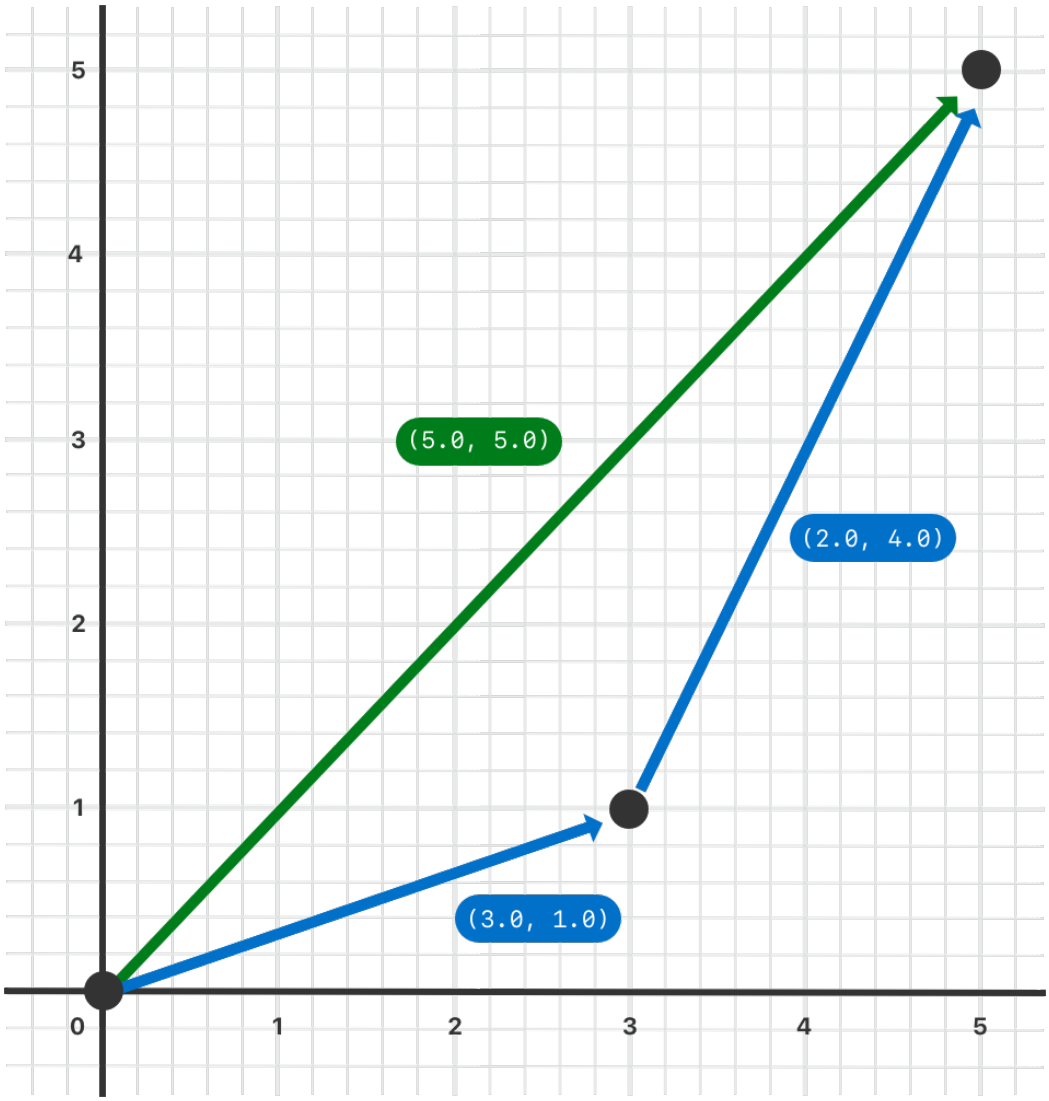
이 연산자 메서드는 `Vector2D`에서 타입 메서드로 정의되고, 오버로드 할 연산자(+)와 일치하는 메서드 이름을 가집니다. 덧셈은 벡터의 필수 동작의 일부가 아니므로, 이 타입 메서드는 `Vector2D`의 메인 구조체 선언이 아닌 `Vector2D`의 확장에 정의합니다. 산술 덧셈 연산자는 이항 연산자이므로, 이 연산자 메서드는 `Vector2D` 타입의 두 입력 매개변수와 `Vector2D` 타입의 하나의 반환하는 출력값을 가집니다.

이 구현에서 입력 매개변수는 `left`와 `right`의 이름을 가지고 이것은 + 연산자의 왼쪽과 오른쪽인 `Vector2D` 인스턴스를 나타냅니다. 이 메서드는 새로운 `Vector2D` 인스턴스를 반환하고, 이것은 두 `Vector2D` 인스턴스의 `x`와 `y` 프로퍼티를 더한 `x`와 `y`를 가진 인스턴스입니다.

이 타입 메서드는 기존 `Vector2D` 인스턴스 사이의 중위 연산자로 사용할 수 있습니다:

```
let vector = Vector2D(x: 3.0, y: 1.0)
let anotherVector = Vector2D(x: 2.0, y: 4.0)
let combinedVector = vector + anotherVector
// combinedVector is a Vector2D instance with values of (5.0, 5.0)
```

이 예시는 벡터 `(3.0, 1.0)`과 `(2.0, 4.0)`을 더하고 아래 그림과 같이 벡터 `(5.0, 5.0)`을 만듭니다.



접두 연산자와 접미 연산자 (Prefix and Postfix Operators)

위에 보여진 예시는 이진 중위 연산자의 커스텀 구현을 보여줍니다. 클래스와 구조체는 표준 단항(unary) 연산자의 구현도 제공할 수 있습니다. 단항 연산자는 단일 대상에서 동작합니다. 대상의 앞(`-a`)에 오면 접두(prefix) 연산자이며 대상의 뒤(`b!`)에 오면 접미(postfix) 연산자 입니다.

접두 단항 연산자나 접미 단항 연산자를 구현하려면 `prefix`나 `postfix` 수정자를 `func` 키워드 앞에 작성하여 연산자 메서드를 선언합니다:

```
extension Vector2D {
    static prefix func - (vector: Vector2D) -> Vector2D {
        return Vector2D(x: -vector.x, y: -vector.y)
    }
}
```

위의 예시는 `Vector2D` 인스턴스에 대한 단항 뺄셈 연산자(`-a`)를 구현합니다. 이 단항 뺄셈 연산자는 접두 연산자이므로 이 메서드는 `prefix` 수정자를 지정해야 합니다.

간단한 숫자값에 대해 단항 뺄셈 연산자는 양수를 음수로 음수를 양수로 변환합니다. `Vector2D` 인스턴스의 `x`와 `y` 프로퍼티에도 동일하게 적용할 수 있습니다:

```
let positive = Vector2D(x: 3.0, y: 4.0)
let negative = -positive
// negative is a Vector2D instance with values of (-3.0, -4.0)
let alsoPositive = -negative
// alsoPositive is a Vector2D instance with values of (3.0, 4.0)
```

복합 할당 연산자 (Compound Assignment Operators)

*복합 할당 연산자(Compound assignment operators)*는 할당 연산자(=)와 다른 연산을 결합합니다. 예를 들어 덧셈 할당 연산자(+=)는 단일 연산으로 덧셈과 할당을 결합합니다. 복합 할당 연산자의 왼쪽 입력 매개변수 타입을 `inout`으로 표시해야 하며, 이것은 매개변수의 값은 연산자 메서드 내에서 직접적으로 수정되기 때문입니다.

아래 예시는 `Vector2D` 인스턴스에 대해 덧셈 할당 연산자 메서드를 구현합니다:

```
extension Vector2D {
    static func += (left: inout Vector2D, right: Vector2D) {
        left = left + right
    }
}
```

덧셈 연산자는 이전에 정의했으므로 여기서 덧셈 과정을 재구현할 필요가 없습니다. 대신에 덧셈 할당 연산자 메서드는 기존에 덧셈 연산자 메서드를 활용하여 왼쪽의 값을 오른쪽 값으로 더하고 왼쪽 값에 설정하여 사용합니다:

```
var original = Vector2D(x: 1.0, y: 2.0)
let vectorToAdd = Vector2D(x: 3.0, y: 4.0)
original += vectorToAdd
// original now has values of (4.0, 6.0)
```

참고

기본 할당 연산자(=)는 오버로드가 불가능합니다. 복합 할당 연산자만 오버로드할 수 있습니다. 마찬가지로 삼항 조건 연산자 (`a ? b : c`)도 오버로드할 수 없습니다.

동등 연산자 (Equivalence Operators)

기본적으로 커스텀 클래스와 구조체는 *동등 연산자(equivalence operators)*인 같음을 나타내는 연산자(==)와 같지 않음을 나타내는 연산자(!=)의 구현을 가지지 않습니다. 일반적으로 == 연산자를 구현하고, != 연산자는 == 연산자의 결과를 부정하는 Swift 표준 라이브러리의 기본 구현을 사용합니다. == 연산자를 구현하는 두 가지 방법이 있습니다: 직접 구현하거나 여러 타입의 경우에 Swift에 구현을 자동 생성하도록 요청할 수 있습니다. 두 경우 모두 Swift 표준 라이브러리의 `Equatable` 프로토콜을 준수해야 합니다.

다른 중위 연산자를 구현하는 것과 같은 방법으로 == 연산자의 구현을 제공합니다:

```
extension Vector2D: Equatable {
    static func == (left: Vector2D, right: Vector2D) -> Bool {
        return (left.x == right.x) && (left.y == right.y)
    }
}
```

위의 예시는 두 `Vector2D` 인스턴스가 같은 값을 가지고 있는지 확인하는 == 연산자를 구현합니다. `Vector2D`의 맥락에서 "동일"의 의미는 "두 인스턴스 모두 같은 x 값과 y 값을 가짐"을 의미하고 이것이 연산자 구현에 사용되는 로직입니다.

이제 두 `Vector2D` 인스턴스가 같은지 확인하기 위해 연산자를 사용할 수 있습니다:

```
let twoThree = Vector2D(x: 2.0, y: 3.0)
let anotherTwoThree = Vector2D(x: 2.0, y: 3.0)
if twoThree == anotherTwoThree {
    print("These two vectors are equivalent.")
}
// Prints "These two vectors are equivalent.".
```

단순한 경우에 자동 생성 구현을 사용하여 프로토콜 채택 (Adopting a Protocol Using a Synthesized Implementation)에서 설명한대로 Swift에 의해 동등 연산자의 자동 생성 구현을 제공받을 수 있습니다.

커스텀 연산자 (Custom Operators)

Swift에서 제공하는 표준 연산자 외에 *커스텀 연산자(custom operators)*를 선언하고 구현할 수 있습니다. 커스텀 연산자를 정의하는데 사용할 수 있는 연산자 문자 목록은 연산자 (Operators)를 참고바랍니다.

새로운 연산자는 `operator` 키워드를 사용하여 전역으로 선언되고, `prefix`, `infix`, `postfix` 수정자를 사용하여 표기할 수 있습니다:

```
prefix operator +++
```

위의 예시는 +++라는 새로운 접두 연산자를 정의합니다. 이 연산자는 Swift에서 기존의 정의되어 있지 않기 때문에, `Vector2D` 인스턴스 작업에 고유한 커스텀 의미를 부여할 수 있습니다. 이 예시에서 +++는 새로운 "두 배로 만들기" 연산자로 취급합니다. 이것은 이전에 정의한 덧셈 할당 연산자로 벡터 자신을 더해서 `Vector2D` 인스턴스의 x와 y 값을 두 배로 만듭니다. +++ 연산자를 구현하기 위해 `Vector2D` 인스턴스에 +++라는 타입 메서드를 다음과 같이 추가합니다:

```
extension Vector2D {
    static prefix func +++ (vector: inout Vector2D) -> Vector2D {
        vector += vector
        return vector
    }
}

var toBeDoubled = Vector2D(x: 1.0, y: 4.0)
let afterDoubling = +++toBeDoubled
// toBeDoubled now has values of (2.0, 8.0)
// afterDoubling also has values of (2.0, 8.0)
```

커스텀 중위 연산자의 우선순위 (Precedence for Custom Infix Operators)

커스텀 중위 연산자는 각 우선순위 그룹에 속해 있습니다. 우선순위 그룹은 해당 연산자가 다른 중위 연산자와 비교해 어느 우선순위와 연산자의 결합방향을 지정합니다. 이러한 특성이 중위 연산자와 다른 중위 연산자의 상호작용에 미치는 영향에 대한 설명은 우선순위와 결합방향 (Precedence and Associativity)을 참고바랍니다.

우선순위 그룹에 명시적으로 속하지 않은 커스텀 중위 연산자는 삼항 조건 연산자의 우선순위 바로 위인 기본 우선순위 그룹이 제공됩니다.

다음의 예시는 `+-`라는 새로운 커스텀 중위 연산자를 `AdditionPrecedence` 우선순위 그룹에 속하도록 정의합니다:

```
infix operator +-: AdditionPrecedence
extension Vector2D {
    static func +- (left: Vector2D, right: Vector2D) -> Vector2D {
        return Vector2D(x: left.x + right.x, y: left.y - right.y)
    }
}
let firstVector = Vector2D(x: 1.0, y: 2.0)
let secondVector = Vector2D(x: 3.0, y: 4.0)
let plusMinusVector = firstVector +- secondVector
// plusMinusVector is a Vector2D instance with values of (4.0, -2.0)
```

이 연산자는 두 벡터의 `x` 값을 더하고 첫 번째 벡터에서 두 번째 벡터의 `y` 값을 뺍니다. 본질적으로 "덧셈" 연산자이기 때문에 `+`나 `-`와 같은 덧셈 중위 연산자와 동일한 우선순위 그룹이 지정되었습니다. Swift 표준 라이브러리에서 제공하는 연산자의 전체 우선순위 그룹과 결합방향 설정 목록은 연산자 선언 (Operator Declarations)을 참고바랍니다. 우선순위 그룹에 대한 자세한 내용과 자체 정의한 연산자와 우선순위 그룹에 대한 문법을 보려면 연산자 선언 (Operator Declaration)을 참고바랍니다.

> **참고**
>
> 접두 연산자나 접미 연산자를 정의할 때 우선순위를 지정하지 않습니다. 그러나 같은 피연산자에 접두 연산자와 접미 연산자를 모두 적용하면, 접미 연산자가 먼저 적용됩니다.

결과 빌더 (Result Builders)

*결과 빌더(result builder)*는 리스트(list)나 트리(tree)와 같은 중첩된 데이터를 자연스러운 선언적인 방식으로 생성하기 위한 문법을 추가하는 타입입니다. 결과 빌더를 사용하는 코드는 `if`와 `for`와 같은 Swift 구문을 포함할 수 있어 조건부나 반복적인 데이터를 처리하기에 적합합니다.

아래 코드는 별과 텍스트를 사용하여 한 줄로 그리기 위해 몇 가지 타입을 정의합니다.

```
protocol Drawable {
    func draw() -> String
}
struct Line: Drawable {
    var elements: [Drawable]
    func draw() -> String {
        return elements.map { $0.draw() }.joined(separator: "")
    }
}
struct Text: Drawable {
    var content: String
    init(_ content: String) { self.content = content }
    func draw() -> String { return content }
}
struct Space: Drawable {
    func draw() -> String { return " " }
}
```

```
struct Stars: Drawable {
    var length: Int
    func draw() -> String { return String(repeating: "*", count: length) }
}
struct AllCaps: Drawable {
    var content: Drawable
    func draw() -> String { return content.draw().uppercased() }
}
```

`Drawable` 프로토콜은 선이나 모양을 그릴 수 있는 항목에 대한 요구사항을 정의합니다: 이 타입은 반드시 `draw()` 메서드를 구현해야 합니다. `Line` 구조체는 단일 선 그리는 것을 나타내며 대부분 그리는 것에 대해 최상위 컨테이너 역할을 합니다. `Line`을 그리기 위해 구조체는 각 선의 요소에서 `draw()`를 호출한 다음 결과 문자열을 하나의 문자열로 연결합니다. `Text` 구조체는 문자열을 그리기의 일부로 만듭니다. `AllCaps` 구조체는 다른 그리기를 감싸고 수정하여 모든 텍스트를 대문자로 변환합니다.

이니셜라이저를 호출하여 이러한 타입으로 그리기가 가능합니다:

```
let name: String? = "Ravi Patel"
let manualDrawing = Line(elements: [
    Stars(length: 3),
    Text("Hello"),
    Space(),
    AllCaps(content: Text((name ?? "World") + "!")),
    Stars(length: 2),
    ])
print(manualDrawing.draw())
// Prints "***Hello RAVI PATEL!**".
```

이 코드는 동작하지만 약간 어색합니다. `AllCaps` 뒤에 중첩된 괄호가 많아 읽기 어렵습니다. `name`이 `nil`일 때 “World”를 사용하는 대체 논리는 `??` 연산자를 사용하여 인라인으로 수행해야 하는데, 더 복잡해지면 읽기 어렵습니다. 그리기를 구성하기 위해 스위치나 `for` 루프를 포함할 수 없습니다. 결과 빌더를 사용하면 일반적인 Swift 코드처럼 보이도록 다시 작성할 수 있습니다.

결과 빌더를 정의하려면, 타입 선언에 `@resultBuilder` 속성을 작성해야 합니다. 예를 들어 이 코드는 `DrawingBuilder` 라는 결과 빌더를 정의하고, 이것은 선언적 문법을 사용하여 그리기를 설명할 수 있습니다:

```
@resultBuilder
struct DrawingBuilder {
    static func buildBlock(_ components: Drawable...) -> Drawable {
        return Line(elements: components)
    }
    static func buildEither(first: Drawable) -> Drawable {
        return first
    }
    static func buildEither(second: Drawable) -> Drawable {
        return second
    }
}
```

`DrawingBuilder` 구조체는 세 개의 메서드를 정의하고, 이것은 결과 빌더 문법의 구현 일부입니다. `buildBlock(_:)` 메서드는 코드 본문에 선을 작성하기 위한 지원을 추가합니다. 해당 본문의 요소를 `Line`으로 결합합니다. `buildEither(first:)`와 `buildEither(second:)` 메서드는 `if-else`에 대한 지원을 추가합니다.

`@DrawingBuilder`를 함수의 매개변수로 적용하여 함수에 전달된 클로저를 결과 빌더가 해당 클로저에서 생성하는 값으로 바꿀 수 있습니다. 예를 들어:

```
func draw(@DrawingBuilder content: () -> Drawable) -> Drawable {
    return content()
}
func caps(@DrawingBuilder content: () -> Drawable) -> Drawable {
    return AllCaps(content: content())
}

func makeGreeting(for name: String? = nil) -> Drawable {
    let greeting = draw {
        Stars(length: 3)
        Text("Hello")
        Space()
        caps {
            if let name = name {
                Text(name + "!")
            } else {
                Text("World!")
            }
        }
        Stars(length: 2)
    }
    return greeting
}
let genericGreeting = makeGreeting()
print(genericGreeting.draw())
// Prints "***Hello WORLD!**".

let personalGreeting = makeGreeting(for: "Ravi Patel")
print(personalGreeting.draw())
// Prints "***Hello RAVI PATEL!**".
```

`makeGreeting(for:)` 함수는 `name` 매개변수를 가지고 와서 개인화 인사말을 그리는데 사용됩니다. `draw(_:)`와 `caps(_:)` 함수는 모두 `@DrawingBuilder` 속성으로 표시된 인자로 단일 클로저를 가집니다. 이러한 함수를 호출할 때, `DrawingBuilder`가 정의하는 특별한 구문을 사용합니다. Swift는 그리기의 선언적 설명을 `DrawingBuilder`의 메서드에 대한 호출로 변환하여 함수 인자로 전달되는 값을 구성합니다. 예를 들어 Swift는 이 예시에서 `caps(_:)` 호출을 다음과 같은 코드로 변환합니다:

```
let capsDrawing = caps {
    let partialDrawing: Drawable
    if let name = name {
        let text = Text(name + "!")
        partialDrawing = DrawingBuilder.buildEither(first: text)
    } else {
        let text = Text("World!")
        partialDrawing = DrawingBuilder.buildEither(second: text)
    }
    return partialDrawing
}
```

Swift는 `if-else` 블록을 `buildEither(first:)`와 `buildEither(second:)` 메서드에 대한 호출로 변환합니다. 코드에서 이러한 메서드를 호출하지 않지만 변환 결과를 표시하면 `DrawingBuilder` 구문을 사용할 때 Swift가 코드를 어떻게 변환하는지 쉽게 확인할 수 있습니다.

특별한 그리기 구문에서 `for` 루프 지원을 추가하기 위해 `buildArray(_:)` 메서드를 추가합니다.

```
extension DrawingBuilder {
    static func buildArray(_ components: [Drawable]) -> Drawable {
        return Line(elements: components)
    }
}
let manyStars = draw {
    Text("Stars:")
    for length in 1...3 {
        Space()
        Stars(length: length)
    }
}
```

위의 코드에서 `for` 루프는 그리기의 배열을 생성하고, `buildArray(_:)` 메서드는 해당 배열을 `Line`으로 변환합니다.

Swift가 빌더 구문을 빌더 타입의 메서드 호출로 변환하는 방법에 대한 전체 목록은 resultBuilder를 참고바랍니다.

언어 참조

언어 참조에 대해 (About the Language Reference)

형식 문법의 표기법을 읽어봅니다.

책의 이 부분은 Swift 프로그래밍 언어의 형식 문법을 설명합니다. 여기서 설명한 문법은 언어를 더 자세히 이해하는데 도움을 주기 위한 것이며, 파서나 컴파일러를 직접 구현하려는 것은 아닙니다.

Swift의 코드의 거의 모든 곳에 나타나는 많은 공통 타입, 함수, 연산자가 실제로 Swift 표준 라이브러리에 정의되어 있으므로 Swift 언어는 상대적으로 작습니다. 이러한 타입, 함수, 연산자는 Swift 언어 자체의 부분은 아니지만 책의 부분에서 의견 및 코드 예시에서 광범위하게 사용됩니다.

문법을 읽는 방법 (How to Read the Grammar)

Swift 프로그래밍 언어의 형식 문법을 설명하는데 사용되는 표기법은 몇 가지 규칙을 따릅니다:

- 화살표(→)는 문법 규칙을 표시하는데 사용되며 “구성될 수 있음”으로 읽을 수 있습니다.
- 문법 카테고리는 기울임꼴 텍스트로 표시되고 문법 생성 규칙의 양쪽에 나타납니다.
- 리터럴 단어와 구두점은 굵게 `boldface constant width` 텍스트로 표시되고 문법 제작 규칙의 오른쪽에만 나타납니다.
- 대체 문법 규칙은 세로 막대로(|)로 구분합니다. 대체 문법이 너무 길어서 쉽게 읽을 수 없을 경우 새 줄에 여러 문법 제작 규칙으로 나뉩니다.
- 경우에 따라 일반 글꼴 텍스트가 문법 생성 규칙의 오른쪽을 설명하는데 사용합니다.
- 옵셔널 구문 카테고리와 리터럴은 후행 물음표인 `*?*`로 표시합니다.

예를 들어, getter-setter 블록의 문법은 다음과 같이 정의됩니다:

> **Grammar of a getter-setter block**
>
> getter-setter-block → { getter-clause setter-clause? } | { setter-clause getter-clause }

이 정의는 getter-setter 블록이 중괄호로 감싸인 getter 절 다음에 옵셔널 setter 절이 올 수 있고 또는 중괄호로 감싸인 setter 절 다음에 getter 절이 올 수도 있음을 나타냅니다. 위의 문법 생성은 다음 두 가지 생성과 동일하며 대안이 명시적으로 설명되어 있습니다:

> **Grammar of a getter-setter block**
>
> getter-setter-block → { getter-clause setter-clause? } getter-setter-block → { setter-clause getter-clause }

언어 참조

어휘 구조 (Lexical Structure)

문법의 최하위 수준의 요소를 사용합니다.

Swift의 *어휘 구조(lexical structure)*는 언어의 유효한 토큰을 형성하는 문자 시퀀스를 설명합니다. 이러한 유효한 토큰은 언어의 최하위 구성 요소를 형성하며 이후 챕터에서 나머지 언어를 설명하는데 사용됩니다. 토큰은 식별자(identifier), 키워드(keyword), 구두점(punctuation), 리터럴(literal), 연산자(operator)로 구성됩니다.

대부분의 경우 토큰은 아래에 지정된 문법의 제약조건 내에서 입력 텍스트에서 가능한 가장 긴 부분 문자열을 고려하여 Swift 소스 파일의 문자에서 생성됩니다.* 이 동작을 최장 일치(longest match) 또는 *최대 뭉크(maximal munch)*라고 합니다.

공백과 주석 (Whitespace and Comments)

공백에는 두가지 용도가 있습니다: 소스 파일에서 토큰을 분리하고 접두 연산자, 접미 연산자, 중위 연산자 (연산자 (Operators) 참조)를 구분 하는데 사용하지만 그렇지 않으면 무시합니다. 다음 문자는 공백으로 간주합니다: 공백(U+0020), 줄바꿈(U+000A), 캐리지 리턴(U+000D), 수평탭(U+0009), 수직(U+000B), 폼피드(U+000C), null(U+0000).

주석은 컴파일러에 의해 공백으로 처리됩니다. 한 줄 주석은 `//`로 시작하고 줄바꿈(U+000A)이나 캐리지 리턴(U+000D)까지 계속됩니다. 여러 줄 주석은 `/*`로 시작하고 `*/`으로 끝납니다. 여러 줄 주석을 중첩할 수 있지만, 주석 마커는 균형을 이루어야 합니다.

주석은 마크업 서식 참조 (Markup Formatting Reference)에 설명한대로 추가 서식 및 마크업을 포함할 수 있습니다.

Grammar of whitespace

whitespace → whitespace-item whitespace? whitespace-item → line-break whitespace-item → inline-space whitespace-item → comment whitespace-item → multiline-comment whitespace-item → U+0000, U+000B, or U+000C

line-break → U+000A line-break → U+000D line-break → U+000D followed by U+000A

inline-spaces → inline-space inline-spaces? inline-space → U+0009 or U+0020

comment → // comment-text line-break multiline-comment → /* multiline-comment-text */

comment-text → comment-text-item comment-text? comment-text-item → Any Unicode scalar value except U+000A or U+000D

multiline-comment-text → multiline-comment-text-item multiline-comment-text?
multiline-comment-text-item → multiline-comment multiline-comment-text-item → comment-text-item
multiline-comment-text-item → Any Unicode scalar value except /* or */

식별자 (Identifiers)

*식별자(Identifiers)*는 A부터 Z까지 대문자 또는 소문자, 언더바(_), 기본 다국어 평면(Basic Multilingual Plane)에 조합하지 않은 영숫자 유니코드 문자(noncombining alphanumeric Unicode character), 개인 사용 영역(Private Use Area)에 없는 기본 다국어 평면 외부의 문자로 시작합니다. 첫 번째 문자 다음에 숫자와 유니코드 조합 문자(combining Unicode characters)도 허용합니다.

언더바로 시작하는 식별자, 첫 번째 인자 레이블이 언더바로 시작하는 서브스크립트, 첫 번째 인자 레이블이 언더바로 시작하는 이니셜라이저는 선언에 `public` 접근-수준 수정자가 있더라도 `internal` 식별자로 처리합니다. 이 규칙을 통해 프레임워크 작성자는 일부 제한사항에 따라 선언이 공개되어야 하는 경우에도 클라이언트와 상호작용하거나 의존해서는 안되는 API의 부분을 표시할 수 있습니다. 또한 두 개의 언더바로 시작하는 식별자는 Swift 컴파일러와 표준 라이브러리 용으로 사용됩니다.

예약어를 식별자로 사용하거나 공백과 같은 특수 문자를 포함하려면, 식별자의 앞뒤에 백틱( )을 넣습니다. 예를 들어 `class`, `100`, `model & test`는 유효한 식별자가 아니지만, class , 100 , model & test 는 유효합니다. 백틱은 식별자의 일부로 간주되지 않습니다; x 와 x는 같은 의미입니다.

원시 식별자(raw identifier)는 다음을 제외한 모든 유니코드 스칼라 값을 포함할 수 있습니다:

- 백틱 ( )
- 역슬래시 (\)
- Null 문자 (U+0000)
- 가로 탭 (U+0009)
- 줄바꿈 (U+000A)
- 세로 탭 (U+000B)
- 캐리지 리턴 (U+000D)
- 다음 줄(Next line) (U+0085)
- 줄바꿈 없는 공백(Nonbreaking space) (U+00A0)
- 오검 공백 표시(Ogham space mark) (U+1680)
- En 쿼드(En quad) (U+2000)
- Em 쿼드(Em quad) (U+2001)
- En 공백(En space) (U+2002)
- Em 공백(Em space) (U+2003)
- Em의 3분의 1 공백(Three-per-em space) (U+2004)
- Em의 4분의 1 공백(Four-per-em space) (U+2005)
- Em의 6분의 1 공백(Six-per-em space) (U+2006)
- 숫자 너비 공백(Figure space) (U+2007)
- 구두점 너비 공백(Punctuation space) (U+2008)
- 얇은 공백(Thin space) (U+2009)

- 극히 얇은 공백(Hair space) (U+200A)
- 줄 구분자(Line separator) (U+2028)
- 문단 구분자(Paragraph separator) (U+2029)
- 좁은 줄바꿈 없는 공백(Narrow nonbreaking space) (U+202F)
- 중간 너비 수학 공백(Medium mathematical space) (U+205F)
- 표의문자 공백(Ideographic space) (U+3000)

원시 식별자는 공백을 포함할 수 있지만, 공백만으로 구성될 수는 없습니다. 원시 식별자에는 왼쪽에서 오른쪽 표시(U+200E)나 오른쪽에서 왼쪽 표시(U+200F)를 포함할 수 있으며 공백으로 간주됩니다. 원시 식별자는 연산자(Operators)에 설명된 연산자 문자를 포함할 수 있지만, 연산자 문자만으로 구성될 수는 없습니다.

참고

Swift 6.2 이전에는 식별자 앞뒤에 백틱을 작성하는 방식을 *이스케이프 식별자(escaped identifier)*라고 불렀으며, 키워드를 식별자로 사용하는 경우로 제한되었습니다.

명시적으로 매개변수 이름이 없는 클로저 내의 매개변수는 암시적으로 `$0`, `$1`, `$2` 등으로 지정됩니다. 이 이름은 클로저의 범위 내에서 유효한 식별자입니다.

프로퍼티 래퍼(property wrapper)의 프로젝션(projection)을 위해 컴파일러는 달러 기호($)로 시작하는 식별자를 자동 생성합니다. 코드는 이 식별자와 상호작용 할 수 있지만, 이 접두사로 식별자를 선언할 수 없습니다. 더 자세한 내용은 속성 (Attributes) 챕터의 propertyWrapper를 참고바랍니다.

Grammar of an identifier

identifier → identifier-head identifier-characters? identifier → raw-identifier identifier → implicit-parameter-name identifier → property-wrapper-projection identifier-list → identifier | identifier **,** identifier-list

identifier-head → Upper- or lowercase letter A through Z identifier-head → **_** identifier-head → U+00A8, U+00AA, U+00AD, U+00AF, U+00B2-U+00B5, or U+00B7-U+00BA identifier-head → U+00BC-U+00BE, U+00C0-U+00D6, U+00D8-U+00F6, or U+00F8-U+00FF identifier-head → U+0100-U+02FF, U+0370-U+167F, U+1681-U+180D, or U+180F-U+1DBF identifier-head → U+1E00-U+1FFF identifier-head → U+200B-U+200D, U+202A-U+202E, U+203F-U+2040, U+2054, or U+2060-U+206F identifier-head → U+2070-U+20CF, U+2100-U+218F, U+2460-U+24FF, or U+2776-U+2793 identifier-head → U+2C00-U+2DFF or U+2E80-U+2FFF identifier-head → U+3004-U+3007, U+3021-U+302F, U+3031-U+303F, or U+3040-U+D7FF identifier-head → U+F900-U+FD3D, U+FD40-U+FDCF, U+FDF0-U+FE1F, or U+FE30-U+FE44 identifier-head → U+FE47-U+FFFD identifier-head → U+10000-U+1FFFD, U+20000-U+2FFFD, U+30000-U+3FFFD, or U+40000-U+4FFFD identifier-head → U+50000-U+5FFFD, U+60000-U+6FFFD, U+70000-U+7FFFD, or U+80000-U+8FFFD identifier-head → U+90000-U+9FFFD, U+A0000-U+AFFFD, U+B0000-U+BFFFD, or U+C0000-U+CFFFD identifier-head → U+D0000-U+DFFFD or U+E0000-U+EFFFD

identifier-character → decimal-digit identifier-character → U+0300-U+036F, U+1DC0-U+1DFF, U+20D0-U+20FF, or U+FE20-U+FE2F identifier-character → identifier-head identifier-characters → identifier-character identifier-characters?

implicit-parameter-name → **$** decimal-digits property-wrapper-projection → **$** identifier-characters

raw-identifier → ` raw-identifier-characters ` raw-identifier-character → Any Unicode scalar value except `, \, U+0000, U+000A, or U+000D raw-identifier-characters → raw-identifier-character raw-identifier-characters?

참고

번역자 주 영문 원본의 형식 문법은 금지 문자를 일부만 나열합니다. 실제 제한은 위의 금지 문자 목록과 이름 전체에 대한 조건을 함께 적용해야 합니다. 원시 식별자의 형식 문법 (Formal Grammar for Raw Identifiers)를 참고바랍니다.

키워드와 구두점 (Keywords and Punctuation)

다음의 키워드는 예약되어 있으므로 식별자 (Identifiers)에서 설명한대로 백틱을 사용하지 않으면 식별자로 사용할 수 없습니다. `inout`, `var`, `let` 이외의 키워드는 백틱을 사용하지 않고도 함수 선언이나 함수 호출에서 매개변수 이름으로 사용할 수 있습니다. 멤버가 키워드와 이름이 같은 경우 해당 멤버에 대한 참조는 멤버 참조와 키워드 사용 사이에 모호성이 있는 경우를 제외하고는 백틱을 사용할 필요가 없습니다 -- 예를 들어 `self`, `Type`, `Protocol`은 명시적 멤버 표현식에서 특별한 의미가 있으므로 해당 컨텍스트에서 백틱을 사용해야 합니다.

- 선언에 사용되는 키워드: `associatedtype`, `borrowing`, `class`, `consuming`, `deinit`, `enum`, `extension`, `fileprivate`, `func`, `import`, `init`, `inout`, `internal`, `let`, `nonisolated`, `open`, `operator`, `precedencegroup`, `private`, `protocol`, `public`, `rethrows`, `static`, `struct`, `subscript`, `typealias`, `var`.
- 구문에 사용되는 키워드: `break`, `case`, `catch`, `continue`, `default`, `defer`, `do`, `else`, `fallthrough`, `for`, `guard`, `if`, `in`, `repeat`, `return`, `switch`, `throw`, `where`, `while`.
- 표현식과 타입에 사용되는 키워드: `Any`, `as`, `await`, `catch`, `false`, `is`, `nil`, `rethrows`, `self`, `Self`, `super`, `throw`, `throws`, `true`, `try`.
- 패턴에 사용되는 키워드: `_`.
- 숫자 기호(#)로 시작하는 키워드: `#available`, `#colorLiteral`, `#else`, `#elseif`, `#endif`, `#fileLiteral`, `#if`, `#imageLiteral`, `#keyPath`, `#selector`, `#sourceLocation`, `#unavailable`.

참고

Swift 5.9 이전에는 다음의 키워드도 예약어입니다: `#column`, `#dsohandle`, `#error`, `#fileID`, `#filePath`, `#file`, `#function`, `#line`, `#warning`. 이것은 현재 Swift 표준 라이브러리에 매크로로 구현되어 있습니다: `column`, `dsohandle`, `error(_:)`, `fileID`, `filePath`, `file`, `function`, `line`, `warning(_:)`.

- 특정 컨텍스트에서 예약된 키워드: `associativity`, `async`, `convenience`, `didSet`, `dynamic`, `final`, `get`, `indirect`, `infix`, `lazy`, `left`, `mutating`, `none`, `nonmutating`, `optional`, `override`, `package`, `postfix`, `precedence`, `prefix`, `Protocol`, `required`, `right`, `set`, `some`, `Type`, `unowned`, `weak`, `willSet`. 컨텍스트 외부에서는 식별자로 사용할 수 있습니다.

다음 토큰은 구두점으로 예약되어 있으므로 커스텀 연산자로 사용할 수 없습니다: `(`, `)`, `{`, `}`, `[`, `]`, `.`, `,`, `:`, `;`, `=`, `@`, `#`, `&`(접두 연산자), `->`, `` ` ``, `?`, `!`(접미 연산자).

리터럴 (Literals)

*리터럴(literal)*은 숫자나 문자열과 같은 타입 값의 소스 코드 표현입니다.

다음은 리터럴의 예시입니다:

```
42               // Integer literal
3.14159          // Floating-point literal
"Hello, world!"  // String literal
/Hello, .*/      // Regular expression literal
true             // Boolean literal
```

리터럴 자체에는 타입이 없습니다. 대신에 리터럴은 무한 정밀도를 가진 구문으로 분석되고 Swift의 타입 추론은 리터럴에 대한 타입을 추론합니다. 예를 들어 `let x: Int8 = 42` 선언에서, Swift는 명시적 타입 설명(`: Int8`)을 사용하여 정수 리터럴 `42`가 `Int8`의 타입이라고 추론합니다. 타입 정보를 명시하지 않은 경우, Swift는 Swift 표준 라이브러리에 정의되고 아래 표에서 나열된 기본 리터럴 타입을 사용합니다. 타입 주석을 사용할 경우, 해당 타입은 리터럴 값에서 초기화할 수 있어야 합니다. 즉, 타입은 아래 표에 나열된 Swift 표준 라이브러리 프로토콜을 준수해야 합니다.

리터럴	기본타입	프로토콜
정수	`Int`	`ExpressibleByIntegerLiteral`
부동소수점	`Double`	`ExpressibleByFloatLiteral`
문자열	`String`	단일 유니코드 스칼라만 포함하는 문자열 리터럴의 경우에는 `ExpressibleByStringLiteral`, `ExpressibleByUnicodeScalarLiteral` 단일 확장 그래프 클러스터(single extended grapheme cluster)만 포함하는 문자열 리터럴의 경우에는 `ExpressibleByExtendedGraphemeClusterLiteral`
정규 표현식	`Regex`	None
Boolean	`Bool`	`ExpressibleByBooleanLiteral`

예를 들어 `let str = "Hello, world"` 선언에서 문자열 리터럴 `"Hello, world"`의 추론된 타입은 기본적으로 `String`입니다. 또한 `Int8`은 `ExpressibleByIntegerLiteral` 프로토콜을 준수하므로 `let x: Int8 = 42` 선언에서 정수 리터럴 `42`에 대해 해당 타입을 사용할 수 있습니다.

Grammar of a literal

literal → numeric-literal | string-literal | regular-expression-literal | boolean-literal | nil-literal

numeric-literal → signed-integer-literal | signed-floating-point-literal boolean-literal → `true` | `false`
nil-literal → `nil`

정수 리터럴 (Integer Literals)

*정수 리터럴(Integer literals)*은 정밀도가 지정되지 않은 정수값을 나타냅니다. 기본적으로 정수 리터럴은 10진수로 표현됩니다; 접두사를 사용하여 다른 진법을 지정할 수 있습니다. 2진수 리터럴은 `0b`로 시작하고, 8진수 리터럴은 `0o`, 16진수 리터럴은 `0x`로 시작합니다.

10진수 리터럴은 `0`부터 `9`까지의 숫자를 포함합니다. 2진수 리터럴은 `0`과 `1`을 포함하고, 8진수 리터럴은 `0`부터 `7`을 포함하고, 16진수 리터럴은 `0`부터 `9` 뿐만 아니라 `A`부터 `F`까지의 대문자나 소문자를 포함합니다.

음의 정수 리터럴은 `-42`와 같이 정수 리터럴 앞에 마이너스 기호(-)로 표현합니다.

가독성을 위해 숫자 사이에 언더바(_)를 허용하지만 무시되므로 리터럴 값에 영향을 주지 않습니다. 정수 리터럴은 `0`으로 시작할 수 있지만 무시되므로 리터럴 값에 영향을 주지 않습니다.

달리 지정하지 않는 한 정수 리터럴의 기본 유추 타입은 Swift 표준 라이브러리 타입 `Int`입니다. Swift 표준 라이브러리는 정수 (Integers)에서 설명했듯이 다양한 크기의 부호 있는 정수와 부호 없는 정수의 타입을 정의합니다.

Grammar of an integer literal

signed-integer-literal → -? integer-literal integer-literal → binary-literal integer-literal → octal-literal integer-literal → decimal-literal integer-literal → hexadecimal-literal

binary-literal → `0b` binary-digit binary-literal-characters? binary-digit → Digit 0 or 1 binary-literal-character → binary-digit | _ binary-literal-characters → binary-literal-character binary-literal-characters?

octal-literal → `0o` octal-digit octal-literal-characters? octal-digit → Digit 0 through 7 octal-literal-character → octal-digit | _ octal-literal-characters → octal-literal-character octal-literal-characters?

decimal-literal → decimal-digit decimal-literal-characters? decimal-digit → Digit 0 through 9 decimal-digits → decimal-digit decimal-digits? decimal-literal-character → decimal-digit | _ decimal-literal-characters → decimal-literal-character decimal-literal-characters?

hexadecimal-literal → `0x` hexadecimal-digit hexadecimal-literal-characters? hexadecimal-digit → Digit 0 through 9, a through f, or A through F hexadecimal-literal-character → hexadecimal-digit | _ hexadecimal-literal-characters → hexadecimal-literal-character hexadecimal-literal-characters?

부동소수점 리터럴 (Floating-Point Literals)

*부동소수점 리터럴(Floating-point literals)*은 정밀도가 지정되지 않은 부동소수점 값을 나타냅니다.

기본적으로 부동소수점 리터럴은 접두사 없이 10진수로 표현되지만 `0x` 접두사를 붙여 16진수로 표현할 수도 있습니다.

10진 부동소수점 리터럴은 10진 숫자열 뒤에 소수부(decimal fraction), 10진 지수(decimal exponent) 중 하나가 오거나 둘 모두가 오는 형태입니다. 소수부는 소수점(`.`)과 소수 자릿수로 구성합니다. 지수는 대문자나 소문자 `e` 접두사 다음에 `e` 앞의 값에 10의 거듭 제곱을 곱한 것을 나타내는 10진수로 구성합니다. 예를 들어 `1.25e2`는 1.25 x 10^{2}을 나타내고, `125.0`입니다. 유사하게 `1.25e-2`는 1.25 x 10^{-2}를 나타내고, `0.0125`입니다.

16진수 부동소수점 리터럴은 `0x` 접두사 다음으로 선택적인 소수부(hexadecimal fraction) 뒤에 지수(hexadecimal exponent)로 구성합니다. 소수부는 소수점과 16진 숫자열로 구성합니다. 지수는 대문자나 소문자 `p` 접두사 다음에 `p` 앞의 값에 2의 거듭 제곱을 곱한 것을 나타내는 10진수로 구성합니다. 예를 들어 `0xFp2`는 15 x 2^{2}를 나타내고 `60`입니다. 유사하게 `0xFp-2`는 15 x 2^{-2}를 나타내고 `3.75`입니다.

음수 부동소수점 리터럴은 `-42.5`와 같이 부동소수점 리터럴 앞에 마이너스 기호(-)를 추가하여 표현합니다.

가독성을 위해 숫자 사이에 언더바(_)를 허용하지만 무시되므로 리터럴 값에 영향을 주지 않습니다. 부동소수점 리터럴은 `0`으로 시작할 수 있지만 마찬가지로 무시되므로 리터럴의 기준이나 값에 영향을 주지 않습니다.

달리 지정하지 않는 한 부동소수점 리터럴의 기본 추론 타입은 Swift 표준 라이브러리 타입 `Double`이며, 이것은 64-비트 부동소수점 숫자를 나타냅니다. Swift 표준 라이브러리는 `Float` 타입도 정의하며, 이것은 32-비트 부동소수점 숫자를 나타냅니다.

Grammar of a floating-point literal

signed-floating-point-literal → –? floating-point-literal floating-point-literal → decimal-literal decimal-fraction? decimal-exponent? floating-point-literal → hexadecimal-literal hexadecimal-fraction? hexadecimal-exponent

decimal-fraction → . decimal-literal decimal-exponent → floating-point-e sign? decimal-literal

hexadecimal-fraction → . hexadecimal-digit hexadecimal-literal-characters? hexadecimal-exponent → floating-point-p sign? decimal-literal

floating-point-e → e | E floating-point-p → p | P sign → + | –

문자열 리터럴 (String Literals)

문자열 리터럴은 따옴표로 묶인 일련의 문자입니다. 한 줄 문자열 리터럴은 쌍따옴표로 묶이며 형식은 다음과 같습니다:

```
"<#characters#>"
```

문자열 리터럴은 이스케이프 처리되지 않은(unescaped) 쌍따옴표("), 이스케이프 처리되지 않은 역슬래시(\), 캐리지 리턴이나 줄바꿈을 포함할 수 없습니다.

여러 줄 문자열 리터럴은 세 개의 쌍따옴표로 묶이며 형식은 다음과 같습니다:

```
"""
<#characters#>
"""
```

한 줄 문자열 리터럴과 다르게 여러 줄 문자열 리터럴은 이스케이프 처리되지 않은 쌍따옴표("), 캐리지 리턴, 줄바꿈을 포함할 수 있습니다. 이스케이프 처리되지 않은 쌍따옴표 세 개를 나란히 포함할 수 없습니다.

여러 줄 문자열 리터럴을 시작하는 `"""` 뒤의 줄바꿈은 문자열의 부분이 아닙니다. 리터럴 종료를 나타내는 `"""` 전의 줄바꿈도 문자열의 부분이 아닙니다. 줄바꿈으로 시작하거나 끝나는 여러 줄 문자열 리터럴을 만드려면 첫 번째나 마지막 줄에 빈 줄을 작성해야 합니다.

여러 줄 문자열 리터럴은 공백과 탭의 조합을 사용하여 들여쓰기 할 수 있습니다: 이 들여쓰기는 문자열에 포함되지 않습니다. 리터럴을 종료하는 `"""`이 들여쓰기를 결정합니다: 리터럴에서 공백이 아닌 모든 줄은 닫는 `"""` 앞에 나타나는 동일한 들여쓰기로 시작해야 합니다; 탭과 공백 사이에는 변환이 없습니다. 들여쓰기 후에 추가 공백과 탭을 포함할 수 있습니다; 이러한 공백과 탭은 문자열에 나타납니다.

여러 줄 문자열 리터럴의 줄바꿈은 줄바꿈 문자를 사용하는 것이 일반적입니다. 소스 파일에 캐리지 리턴과 줄바꿈이 혼합되어 있더라도 문자열의 줄바꿈은 모두 동일합니다.

여러 줄 문자열 리터럴에서 줄 끝에 역슬래시(\)를 작성하면 문자열에서 줄바꿈이 생략됩니다. 역슬래시와 줄바꿈 사이의 공백도 생략합니다. 이 구문을 사용하여 결과 문자열의 값을 변경하지 않고 소스 코드에 여러 줄 문자열 리터럴을 래핑할 수 있습니다.

다음 이스케이프 시퀀스를 사용하여 한 줄과 여러 줄 형식의 문자열 리터럴에 특수 문자를 포함할 수 있습니다:

- Null 문자(`\0`)
- 역슬래시(`\\`)
- 가로 탭(`\t`)
- 줄바꿈(`\n`)
- 캐리지 리턴(`\r`)
- 쌍따옴표(`\"`)
- 홑따옴표(`\'`)
- 유니코드 스칼라(`\u{n}`), n은 한 자리부터 여덟 자리까지의 숫자로 구성된 16진수 입니다.

표현식의 값은 문자열 리터럴에 삽입할 수 있으며, 역슬래시(\) 뒤 소괄호 안에 표현식을 위치시켜 작성합니다. 삽입된 표현식은 문자열 리터럴을 포함할 수 있지만, 이스케이프 처리되지 않은 역슬래시, 캐리지 리턴, 줄바꿈을 포함할 수 없습니다.

예를 들어 아래의 문자열 리터럴은 동일한 값을 가집니다:

```
"1 2 3"
"1 2 \("3")"
"1 2 \(3)"
"1 2 \(1 + 2)"
let x = 3; "1 2 \(x)"
```

확장 구분자(extended delimiters)로 구분된 문자열은 따옴표로 묶인 일련의 문자와 하나 이상의 숫자 기호(#)의 집합입니다. 확장 구분자로 구분된 문자열의 형식은 다음과 같습니다:

```
#"<#characters#>"#

#"""
<#characters#>
"""#
```

확장 구분자로 구분된 문자열에서 특수문자는 일반 문자로 결과 문자열에 나타납니다. 확장 구분자를 사용하여 문자열 보간 생성, 이스케이프 시퀀스 시작, 문자열 종료와 같은 특수 효과를 가지는 문자로 문자열을 만들 수 있습니다.

다음의 예시는 동일한 문자열 값을 생성하는 문자열 리터럴과 확장 구분자로 구분된 문자열을 보여줍니다:

```
let string = #"\(x) \ " \u{2603}"#
let escaped = "\\(x) \\ \" \\u{2603}"
print(string)
// Prints "\(x) \ " \u{2603}".
print(string == escaped)
// Prints "true".
```

확장 구분자로 구분된 문자열에 둘 이상의 숫자 기호를 사용하는 경우 숫자 기호 사이에 공백이 있으면 안됩니다:

```
print(###"Line 1\###nLine 2"###) // OK
print(# # #"Line 1\# # #nLine 2"# # #) // Error
```

확장 구분자를 사용하여 생성한 여러 줄 문자열 리터럴은 일반적인 여러 줄 문자열 리터럴과 동일한 들여쓰기 요구사항을 가집니다.

문자열 리터럴의 기본 추론 타입은 `String`입니다. `String` 타입에 대한 자세한 설명은 문자열과 문자 (Strings and Characters)와 `String`에서 확인 가능합니다.

+ 연산자로 연결된 문자열 리터럴은 컴파일 시 연결됩니다. 예를 들어 아래 예시의 `textA`와 `textB`의 값은 동일하며 런타임 연결이 수행되지 않습니다.

```
let textA = "Hello " + "world"
let textB = "Hello world"
```

Grammar of a string literal

string-literal → static-string-literal | interpolated-string-literal

string-literal-opening-delimiter → extended-string-literal-delimiter? **"** string-literal-closing-delimiter → **"** extended-string-literal-delimiter?

static-string-literal → string-literal-opening-delimiter quoted-text? string-literal-closing-delimiter
static-string-literal → multiline-string-literal-opening-delimiter multiline-quoted-text? multiline-string-literal-closing-delimiter

multiline-string-literal-opening-delimiter → extended-string-literal-delimiter? **"""**
multiline-string-literal-closing-delimiter → **"""** extended-string-literal-delimiter?
extended-string-literal-delimiter → **#** extended-string-literal-delimiter?

quoted-text → quoted-text-item quoted-text? quoted-text-item → escaped-character quoted-text-item → Any Unicode scalar value except **"**, ****, U+000A, or U+000D

multiline-quoted-text → multiline-quoted-text-item multiline-quoted-text? multiline-quoted-text-item → escaped-character multiline-quoted-text-item → Any Unicode scalar value except ****
multiline-quoted-text-item → escaped-newline

interpolated-string-literal → string-literal-opening-delimiter interpolated-text? string-literal-closing-delimiter
interpolated-string-literal → multiline-string-literal-opening-delimiter multiline-interpolated-text? multiline-string-literal-closing-delimiter

interpolated-text → interpolated-text-item interpolated-text? interpolated-text-item → **\(** expression **)** | quoted-text-item

multiline-interpolated-text → multiline-interpolated-text-item multiline-interpolated-text?
multiline-interpolated-text-item → **\(** expression **)** | multiline-quoted-text-item

escape-sequence → **** extended-string-literal-delimiter escaped-character → escape-sequence **0** | escape-sequence **** | escape-sequence **t** | escape-sequence **n** | escape-sequence **r** | escape-sequence **"** | escape-sequence **'** escaped-character → escape-sequence **u** **{** unicode-scalar-digits **}** unicode-scalar-digits → Between one and eight hexadecimal digits

escaped-newline → escape-sequence inline-spaces? line-break

정규 표현식 리터럴 (Regular Expression Literals)

정규 표현식 리터럴(regular expression literal)은 아래와 같이 슬래시(`/`)로 감싸있는 문자의 시퀀스입니다:

```
/<#regular expression#>/
```

정규 표현식 리터럴은 탭이나 스페이스로 시작되지 않아야 하며, 이스케이프 처리되지 않은 슬래시(unescaped slash)(/), 캐리지 리턴(carriage return), 줄바꿈을 포함할 수 없습니다.

정규 표현식 리터럴 내부의 역슬래시는 문자열 리터럴과 같이 이스케이프 문자가 아닌 정규 표현식의 부분으로 인식합니다. 다음의 특수 문자는 문자 그대로 해석되거나 다음의 일반 문자는 특별한 방법으로 해석됨을 나타냅니다. 예를 들어 `/\(/`은 단일 왼쪽 괄호이며, `/\d/`은 단일 숫자입니다.

확장 구분자를 사용하는 정규 표현식 리터럴은 슬래시(/)로 감싸있는 문자의 시퀀스와 하나 이상의 숫자 기호(#) 쌍으로 되어있습니다. 확장 구분자로 구분된 정규 표현식 리터럴은 다음의 형식을 가집니다:

```
#/<#regular expression#>/#

#/
<#regular expression#>
/#
```

확장 구분자를 사용하는 정규 표현식 리터럴은 이스케이프 처리되지 않은 스페이스나 탭으로 시작할 수 있고, 이스케이프 처리되지 않은 슬래시(/)를 포함할 수 있고, 여러 줄로 작성할 수 있습니다. 여러 줄 정규 표현식 리터럴은 시작 구분자는 라인의 끝에 있어야 하고, 끝나는 구분자는 해당 라인에 있어야 합니다. 여러 줄 정규 표현식 리터럴에서 확장된 정규 표현식은 기본적으로 가능합니다 -- 특히, 공백은 무시되고 주석은 허용됩니다.

확장된 구분자에 의해 구분된 정규 표현식 리터럴 형식에 하나 이상의 숫자 기호를 사용한다면 숫자 기호 사이에 공백이 있을 수 없습니다:

```
let regex1 = ##/abc/##       // OK
let regex2 = # #/abc/# #     // Error
```

빈 정규 표현식 리터럴을 만드려면 확장된 구분자 구문을 사용해야 합니다.

Grammar of a regular expression literal

regular-expression-literal → regular-expression-literal-opening-delimiter regular-expression regular-expression-literal-closing-delimiter regular-expression → Any regular expression

regular-expression-literal-opening-delimiter → extended-regular-expression-literal-delimiter? /
regular-expression-literal-closing-delimiter → / extended-regular-expression-literal-delimiter?

extended-regular-expression-literal-delimiter → # extended-regular-expression-literal-delimiter?

연산자 (Operators)

Swift 표준 라이브러리는 사용할 수 있는 여러 가지 연산자를 정의하며 기본 연산자 (Basic Operators)와 고급 연산자 (Advanced Operators)에 설명되어 있습니다. 이 섹션에서는 커스텀 연산자(Custom operators)를 정의하는데 사용할 수 있는 문자를 설명합니다.

커스텀 연산자는 ASCII 문자 /, =, –, +, !, *, %, <, >, &, |, ^, ?, ~ 중 하나로 시작하거나 아래 문법에 정의된 유니코드 문자 중 하나로 시작할 수 있습니다 (수학 연산자(Mathematical Operators), 기타 기호(Miscellaneous

Symbols), 딩뱃(Dingbats) 유니코드 블록의 문자를 포함합니다). 첫 번째 문자 뒤에 유니코드 문자를 결합하는 것도 가능합니다.

커스텀 연산자도 정의할 수 있는데 점(`.`)으로 시작합니다. 이러한 연산자는 여러 개의 점을 포함할 수 있습니다. 예를 들어 `.+.`은 단일 연산자로 취급됩니다. 연산자가 점으로 시작하지 않으면, 다른 곳에 점을 포함할 수 없습니다. 예를 들어 `+.+`은 + 연산자와 `.+` 연산자로 처리됩니다.

물음표(?)를 포함하는 커스텀 연산자를 정의할 수 있지만 단일 물음표 문자로만 이루어진 연산자는 정의할 수 없습니다. 또한 연산자에 느낌표(`!`)를 포함할 수 있지만 접미 연산자(postfix operators)는 물음표나 느낌표로 시작할 수 없습니다.

참고

토큰 =, –>, //, /*, */, ., 접두 연산자 <, &, ?, 중위 연산자 ?, 접미 연산자 >, !, ?는 예약되어 있습니다. 이러한 토큰은 오버로드할 수 없으며, 커스텀 연산자로 사용할 수 없습니다.

연산자 주변의 공백은 연산자가 접두 연산자(prefix operator), 접미 연산자(postfix operator), 이항 연산자(binary operator)로 사용되는지 결정하기 위해 사용됩니다. 이 동작은 다음 규칙에 정리되어 있습니다:

- 연산자 주변 양쪽에 모두 공백이 있거나 없는 경우, 이항 연산자로 처리됩니다. 예를 들어 `a+++b`와 `a +++ b`의 연산자 +++는 이항 연산자로 처리됩니다.
- 연산자가 왼쪽에만 공백이 있다면, 접두 단항 연산자(prefix unary operator)로 처리됩니다. 에를 들어 `a +++b`에서 +++ 연산자는 접두 단항 연산자로 처리됩니다.
- 연산자가 오른쪽에만 공백이 있다면, 접미 단항 연산자(postfix unary operator)로 처리됩니다. 예를 들어 `a+++ b`에서 +++ 연산자는 접미 단항 연산자로 처리됩니다.
- 연산자 왼쪽에 공백이 없지만 바로 이어서 점(`.`)이 오면, 접미 단항 연산자로 처리됩니다. 예를 들어 `a+++.b`에서 +++ 연산자는 접미 단항 연산자로 처리됩니다 (`a +++ .b`가 아닌 `a+++ .b`).

이러한 규칙의 목적에 따라 연산자 앞의 문자 (, [, {, 연산자 뒤의 문자),], }, 문자 ,, ;, :도 공백으로 간주합니다.

미리 정의된 연산자 !나 ?에 왼편에 공백이 없다면, 오른편에 공백 여부와 상관없이 접미 연산자로 처리됩니다. 옵셔널 체이닝 연산자(optional-chaining operator)로 ?를 사용하려면, 왼편에 공백을 가지면 안됩니다. 삼항 조건 연산자(ternary conditional operator)(`? :`)로 사용하려면, 반드시 양쪽에 공백이 있어야 합니다.

중위 연산자(infix operator)의 인자 중 하나가 정규 표현식 리터럴이라면, 연산자는 양쪽에 공백을 가져야 합니다.

특정 구문에서 < 또는 >로 시작하는 연산자는 둘 이상의 토큰으로 분리될 수 있습니다. 나머지는 동일한 방식으로 처리되고 다시 분리될 수 있습니다. 그 결과로 `Dictionary<String, Array<Int>>`와 같은 구문에서 닫는 > 문자 사이를 명확하게 하기 위해 공백을 추가할 필요가 없습니다. 예를 들어 닫는 > 문자는 비트 시프트 >> 연산자로 잘못 해석될 수 있는 단일 토큰으로 처리되지 않습니다.

새로운 커스텀 연산자를 정의하는 방법을 알아보려면, 커스텀 연산자 (Custom Operators)와 연산자 선언 (Operator Declaration)을 참고바랍니다. 기존 연산자를 오버로드 하는 방법을 알아보려면 연산자 메서드 (Operator Methods)를 참고바랍니다.

Grammar of operators

operator → operator-head operator-characters? operator → dot-operator-head dot-operator-characters

operator-head → / | = | – | + | ! | * | % | < | > | & | | | ^ | ~ | ? operator-head → U+00A1-U+00A7
operator-head → U+00A9 or U+00AB operator-head → U+00AC or U+00AE operator-head →

U+00B0-U+00B1 operator-head → U+00B6, U+00BB, U+00BF, U+00D7, or U+00F7 operator-head → U+2016-U+2017 operator-head → U+2020-U+2027 operator-head → U+2030-U+203E operator-head → U+2041-U+2053 operator-head → U+2055-U+205E operator-head → U+2190-U+23FF operator-head → U+2500-U+2775 operator-head → U+2794-U+2BFF operator-head → U+2E00-U+2E7F operator-head → U+3001-U+3003 operator-head → U+3008-U+3020 operator-head → U+3030

operator-character → operator-head operator-character → U+0300-U+036F operator-character → U+1DC0-U+1DFF operator-character → U+20D0-U+20FF operator-character → U+FE00-U+FE0F operator-character → U+FE20-U+FE2F operator-character → U+E0100-U+E01EF operator-characters → operator-character operator-characters?

dot-operator-head → **.** dot-operator-character → **.** | operator-character dot-operator-characters → dot-operator-character dot-operator-characters?

infix-operator → operator prefix-operator → operator postfix-operator → operator

언어 참조

타입 (Types)

내장된 명명 타입(named type)과 복합 타입(compound type)을 사용합니다.

Swift에는 두 가지 종류의 타입이 있습니다: 명명 타입과 복합 타입입니다. *명명 타입(named type)*은 정의될 때 특정 이름을 부여할 수 있는 타입입니다. 명명 타입은 클래스, 구조체, 열거형, 프로토콜을 포함합니다. 예를 들어 `MyClass`라는 커스텀 클래스의 인스턴스는 `MyClass` 타입을 가집니다. 커스텀 명명 타입 외에도 Swift 표준 라이브러리는 배열, 딕셔너리, 옵셔널 값을 나타내는 타입을 포함하여 자주 사용하는 명명 타입이 정의되어 있습니다.

다른 언어에서 일반적으로 기본 타입이나 원시 타입으로 간주되는 데이터 타입(Data types) -- 숫자, 문자, 문자열과 같은 타입도 -- Swift 표준 라이브러리에서는 구조체를 사용하여 정의되고 구현된 명명 타입입니다. 명명 타입이므로, 필요에 따라 프로그램에 맞게 동작을 확장 (Extensions)과 확장 선언 (Extension Declaration)에서 설명한대로 확장 선언을 사용하여 확장할 수 있습니다.

*복합 타입(compound type)*은 Swift 언어 자체에 정의된 이름이 없는 타입입니다. 복합 타입은 두 가지 종류의 타입이 있습니다: 함수 타입과 튜플 타입입니다. 복합 타입은 명명 타입과 다른 복합 타입을 포함할 수 있습니다. 예를 들어 튜플 타입 `(Int, (Int, Int))`는 두 요소를 포함합니다: 첫 번째는 명명 타입인 `Int`, 두 번째는 다른 복합 타입인 `(Int, Int)`입니다.

명명 타입이나 복합 타입을 묶을 소괄호를 넣을 수 있습니다. 그러나 타입을 묶은 소괄호는 아무런 영향을 주지 않습니다. 예를 들어 `(Int)`는 `Int`와 같습니다.

이 챕터에서는 Swift 언어 자체에 정의된 타입과 Swift의 타입 추론에 대해 설명합니다.

Grammar of a type

type → function-type type → array-type type → dictionary-type type → type-identifier type → tuple-type type → optional-type type → implicitly-unwrapped-optional-type type → protocol-composition-type type → opaque-type type → boxed-protocol-type type → metatype-type type → any-type type → self-type type → (type)

타입 주석 (Type Annotation)

*타입 주석(type annotation)*은 변수 타입이나 표현식 타입을 명시적으로 지정합니다. 타입 주석은 아래 예시에서 보여주듯이 콜론(`:`)으로 시작하고 타입으로 끝납니다:

```
let someTuple: (Double, Double) = (3.14159, 2.71828)
func someFunction(a: Int) { /* ... */ }
```

첫 번째 예시에서 표현식 `someTuple`은 튜플 타입 `(Double, Double)`을 갖도록 지정합니다. 두 번째 예시는 함수 `someFunction`의 매개변수 `a`는 타입 `Int`를 갖도록 지정합니다.

타입 주석은 타입 앞에 타입 속성을 포함할 수 있습니다.

Grammar of a type annotation

type-annotation → : attributes? type

타입 식별자 (Type Identifier)

*타입 식별자(type identifier)*는 명명 타입이나 명명 타입 또는 복합 타입의 타입 별칭(type alias)을 참조합니다.

대부분의 경우 타입 식별자는 같은 이름의 명명 타입을 직접 참조합니다. 예를 들어 `Int`는 명명 타입 `Int`를 직접 참조하는 타입 식별자이고, 타입 식별자 `Dictionary<String, Int>`는 명명 타입 `Dictionary<String, Int>`를 직접 참조합니다.

타입 식별자가 동일한 이름의 타입을 참조하지 않는 두 가지 경우가 있습니다. 첫 번째 경우는 타입 식별자가 명명 타입이나 복합 타입의 타입 별칭을 참조하는 경우입니다. 예를 들어 아래 예시에서 타입 주석에서 `Point`는 튜플 타입 `(Int, Int)`를 나타냅니다.

```
typealias Point = (Int, Int)
let origin: Point = (0, 0)
```

두 번째 경우는 타입 식별자가 다른 모듈에서 선언되거나 다른 타입 내에 중첩된 명명 타입을 참조하기 위해 점(`.`) 구문을 사용하는 경우입니다. 예를 들어 다음 코드의 타입 식별자는 `ExampleModule` 모듈에 선언된 명명 타입 `MyType`을 참조합니다.

```
var someValue: ExampleModule.MyType
```

Grammar of a type identifier

type-identifier → type-name generic-argument-clause? | type-name generic-argument-clause? `.` type-identifier
type-name → identifier

튜플 타입 (Tuple Type)

*튜플 타입(tuple type)*은 소괄호 안에 콤마로 구분된 타입의 목록입니다.

튜플 타입을 함수의 반환 타입으로 사용하여 함수가 여러 값을 포함하는 단일 튜플을 반환하도록 할 수 있습니다. 튜플 타입의 요소에 이름을 지정하고 해당 이름을 사용하여 개별 요소의 값을 참조할 수도 있습니다. 요소 이름은 바로 뒤에 콜론(`:`)이 오는 식별자로 지정합니다. 이러한 기능을 보여주는 예시는 여러 개의 반환값이 있는 함수 (Functions with Multiple Return Values)를 참고바랍니다.

튜플 타입의 요소가 이름을 가지는 경우 이름은 타입의 부분입니다.

```
var someTuple = (top: 10, bottom: 12)  // someTuple is of type (top: Int, bottom: Int)
someTuple = (top: 4, bottom: 42) // OK: Names match.
someTuple = (9, 99)              // OK: Names are inferred.
someTuple = (left: 5, right: 5)  // Error: Names don't match.
```

모든 튜플 타입은 두 개 이상의 타입을 포함하고, 단, `Void`는 예외이며, 빈 튜플 타입 인 `()`에 대한 타입 별칭입니다.

Grammar of a tuple type

tuple-type → ( ) | (tuple-type-element , tuple-type-element-list) tuple-type-element-list → tuple-type-element | tuple-type-element , tuple-type-element-list tuple-type-element → element-name type-annotation | type element-name → identifier

함수 타입 (Function Type)

*함수 타입(function type)*은 함수, 메서드, 클로저의 타입을 나타내고 화살표(->)로 매개변수와 반환 타입을 구분하여 구성합니다:

```
(<#parameter type#>) -> <#return type#>
```

*매개변수 타입(parameter type)*은 콤마로 구분된 타입의 목록입니다. *반환 타입(return type)*은 튜플 타입일 수 있으므로, 함수 타입은 여러 값을 반환하는 함수와 메서드를 지원합니다.

함수 타입이 `() -> T`일 경우 (T는 모든 타입) 해당 매개변수는 `autoclosure` 속성을 적용하여 호출 시점에 암시적으로 클로저를 생성할 수 있습니다. 이것은 함수를 호출할 때 명시적으로 클로저를 작성할 필요없이 표현식의 평가를 연기하는 구문상 편리함을 제공합니다. 자동 클로저 함수 타입 매개변수의 예시는 자동 클로저 (Autoclosures)를 참고바랍니다.

함수 타입의 매개변수 타입은 가변 매개변수(variadic parameters)를 가질 수 있습니다. 문법적으로 가변 매개변수는 `Int...`와 같이 기본 타입 이름 뒤에 점 3개(`...`)로 구성합니다. 가변 매개변수는 기본 타입 이름의 요소를 포함하는 배열로 처리합니다. 예를 들어 가변 매개변수 `Int...`는 `[Int]`로 처리합니다. 가변 매개변수를 사용하는 예시는 가변 매개변수 (Variadic Parameters)를 참고바랍니다.

in-out 매개변수(in-out parameter)를 지정하려면 `inout` 키워드를 매개변수 타입 앞에 붙여야 합니다. 가변 매개변수나 반환 타입에는 `inout` 키워드를 사용할 수 없습니다. In-out 매개변수는 In-Out 매개변수 (In-Out Parameters)에 설명되어 있습니다.

함수 타입이 하나의 매개변수만 가지고 있고 매개변수 타입이 튜플 타입인 경우 함수 타입을 작성할 때 튜플 타입을 괄호로 묶어야 합니다. 예를 들어 `((Int, Int)) -> Void`는 튜플 타입 `(Int, Int)`의 단일 매개변수를 가지고 값을 반환하지 않는 함수 타입입니다. 반대로 괄호가 없으면 `(Int, Int) -> Void`는 두 개의 `Int` 매개변수를 가지고 값을 반환하지 않는 함수 타입입니다. 마찬가지로 `Void`는 `()`의 타입 별칭이므로 함수 타입 `(Void) -> Void`는 `(()) -> ()`와 같습니다 -- 이것은 빈 튜플인 단일 인자를 가지는 함수와 같습니다. 이 타입은 `() -> ()`와 같지 않습니다 -- 이것은 인자를 가지지 않는 함수입니다.

함수와 메서드의 인자 이름은 해당 함수 타입의 일부가 아닙니다. 예를 들어:

```
func someFunction(left: Int, right: Int) {}
func anotherFunction(left: Int, right: Int) {}
func functionWithDifferentLabels(top: Int, bottom: Int) {}

var f = someFunction // The type of f is (Int, Int) -> Void, not (left: Int, right: Int) -> Vo
   id.
f = anotherFunction              // OK
f = functionWithDifferentLabels  // OK

func functionWithDifferentArgumentTypes(left: Int, right: String) {}
f = functionWithDifferentArgumentTypes     // Error

func functionWithDifferentNumberOfArguments(left: Int, right: Int, top: Int) {}
f = functionWithDifferentNumberOfArguments // Error
```

인자 레이블은 함수 타입의 일부분이 아니므로, 함수 타입을 작성할 때 생략합니다.

```
var operation: (lhs: Int, rhs: Int) -> Int     // Error
var operation: (_ lhs: Int, _ rhs: Int) -> Int // OK
var operation: (Int, Int) -> Int               // OK
```

함수 타입에 하나 이상의 화살표(->)를 포함하는 경우, 함수 타입은 오른쪽에서 왼쪽으로 그룹화됩니다. 예를 들어 함수 타입 `(Int) -> (Int) -> Int`는 `(Int) -> ((Int) -> Int)`로 해석됩니다 -- 즉, 이 함수는 `Int`를 받아 `Int`를 받고 반환하는 또 다른 함수를 반환합니다.

오류를 던지거나 다시 던질 수 있는 함수 타입은 `throws` 키워드가 포함되어야 합니다. 오류의 타입을 지정하기 위해서 `throws` 뒤 괄호안에 타입을 포함할 수 있습니다. 오류 타입은 `Error` 프로토콜을 준수해야 합니다. 특정 타입 없이 `throws`를 작성하는 것은 `throws(any Error)`로 작성한 것과 같습니다. `throws`를 생략하는 것은 `throws(Never)`로 작성하는 것과 같습니다. 함수가 던지는 오류 타입은 제네릭 타입, 박싱 프로토콜 타입, 불투명 타입을 포함하여 `Error`를 준수하는 모든 타입일 수 있습니다.

함수가 던지는 오류의 타입은 함수 타입의 일부이고, 오류 타입 간의 하위 타입 관계는 해당 함수 타입 간에도 하위 타입 관계가 성립한다는 의미입니다. 예를 들어, 임의로 정의한 `MyError` 타입을 선언하면, 상위 타입에서 하위 타입으로의 함수 타입 간의 관계는 다음과 같습니다:

1. 모든 오류를 던지는 함수는 `throws(any Error)`로 표시합니다.
2. 특정 오류를 던지는 함수는 `throws(MyError)`로 표시합니다.
3. 던지지 않는 함수는 `throws(Never)`로 표시합니다.

이 하위 타입 관계의 결과는 다음과 같습니다:

- 오류를 던지지 않는 함수는 오류를 던지는 함수가 필요한 곳에서 사용할 수 있습니다.
- 구체적인 오류 타입을 던지는 함수는 일반적인 던지는 함수가 필요한 곳에 사용할 수 있습니다.
- 더 구체적인 오류 타입을 던지는 함수는 더 일반적인 오류 타입을 던지는 함수가 필요한 곳에 사용할 수 있습니다.

함수 타입에서 오류 타입으로 연관 타입(associated type)이나 제네릭 타입 매개변수(generic type parameter)를 사용하면, 연관 타입이나 제네릭 타입 매개변수는 암시적으로 `Error` 프로토콜을 준수해야 합니다.

던지는 함수와 다시 던지는 함수는 오류를 던지는 함수와 메서드 (Throwing Functions and Methods)와 오류를 다시 던지는 함수와 메서드 (Rethrowing Functions and Methods)에 설명되어 있습니다.

비동기 함수의 함수 타입은 `async` 키워드로 표시되어야 합니다. `async` 키워드는 함수의 타입의 부분이며, 동기 함수는 비동기 함수의 하위 타입(subtypes)입니다. 결과적으로 비동기 함수와 같은 위치에서 동기 함수를 사용할 수 있습니다. 비동기 함수에 대한 더 자세한 설명은 비동기 함수와 메서드 (Asynchronous Functions and Methods)를 참고바랍니다.

비탈출 클로저의 제약 (Restrictions for Nonescaping Closures)

비탈출 함수(nonescaping function) 타입의 매개변수는 `Any` 타입의 프로퍼티, 변수, 상수에 저장할 수 없습니다. 그렇게 하면 해당 값이 탈출될 수 있기 때문입니다.

비탈출 함수 타입의 매개변수는 다른 비탈출 함수 매개변수 인자로 전달할 수도 없습니다. 이 제약은 Swift가 런타임이 아닌 컴파일 시 메모리 충돌 접근에 대한 검사를 더 많이 수행하는데 도움이 됩니다. 예를 들어:

```
let external: (() -> Void) -> Void = { _ in () }
func takesTwoFunctions(first: (() -> Void) -> Void, second: (() -> Void) -> Void) {
    first { first {} }       // Error
    second { second {}  }    // Error

    first { second {} }      // Error
    second { first {} }      // Error

    first { external {} }    // OK
    external { first {} }    // OK
}
```

위의 코드에서 `takesTwoFunctions(first:second:)`의 두 매개변수는 모두 함수입니다. 두 매개변수 모두 `@escaping`으로 표시되지 않으므로 결과적으로 둘 다 비탈출입니다.

위의 예시에서 "Error"로 표시된 네 개의 함수 호출은 컴파일러 오류를 일으킵니다. `first`와 `second` 매개변수는 비탈출 함수이므로, 다른 비탈출 함수 매개변수 인자로 전달할 수 없습니다. 반대로 "OK"로 표시된 두 개의 함수 호출은 컴파일러 오류를 발생시키지 않습니다. 이 함수 호출은 `external`이 `takesTwoFunctions(first:second:)`의 매개변수가 아니므로 제약에 위배되지 않습니다.

제약을 피해야 하는 경우 매개변수 중 하나를 탈출로 표시하거나, `withoutActuallyEscaping(_:do:)` 함수를 이용하여 탈출 함수로 비탈출 함수 매개변수 중 하나를 임시로 변경해야 합니다. 메모리 충돌 접근을 피하는 것에 대한 자세한 내용은 메모리 안전성 (Memory Safety)을 참고바랍니다.

Grammar of a function type

function-type → attributes? function-type-argument-clause `async`? throws-clause? `->` type

function-type-argument-clause → `(` `)` function-type-argument-clause → `(` function-type-argument-list `...`? `)`

function-type-argument-list → function-type-argument | function-type-argument `,` function-type-argument-list function-type-argument → attributes? parameter-modifier? type | argument-label type-annotation argument-label → identifier

throws-clause → `throws` | `throws` `(` type `)`

배열 타입 (Array Type)

Swift 언어는 Swift 표준 라이브러리의 `Array<Element>` 타입에 대해 다음과 같은 문법을 제공합니다:

```
[<#type#>]
```

다른 표현으로 다음의 두 선언도 동일합니다:

```
let someArray: Array<String> = ["Alex", "Brian", "Dave"]
let someArray: [String] = ["Alex", "Brian", "Dave"]
```

두 경우 모두 상수 `someArray`는 문자열 배열로 선언됩니다. 배열의 요소는 대괄호에 유효한 인덱스 값을 지정하여 서브스크립트를 통해 접근할 수 있습니다: `someArray[0]`는 `"Alex"`인 인덱스 0번째 요소를 나타냅니다.

대괄호 쌍을 중첩하여 다차원 배열을 만들 수 있습니다. 여기서 요소의 기본 타입 이름은 가장 안쪽 대괄호 쌍에 포함됩니다. 예를 들어 대괄호 세 쌍을 이용하여 정수의 3차원 배열을 생성할 수 있습니다:

```
var array3D: [[[Int]]] = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]
```

다차원 배열의 요소에 접근할 때, 가장 왼쪽에 있는 서브스크립트 인덱스는 배열의 가장 바깥 쪽에 있는 요소를 참조합니다. 오른쪽에 있는 다음 서브스크립트 인덱스는 한 차원 더 들어가는 중첩된 배열의 요소를 참조합니다. 이것은 위의 예시에서 `array3D[0]`은 `[[1, 2], [3, 4]]`를 참조하고 `array3D[0][1]`은 `[3, 4]`, `array3D[0][1][1]`은 값 4를 참조합니다.

Swift 표준 라이브러리 `Array` 타입에 대한 자세한 설명은 배열 (Arrays)을 참고바랍니다.

Grammar of an array type

array-type → [type]

딕셔너리 타입 (Dictionary Type)

Swift 언어는 Swift 표준 라이브러리 `Dictionary<Key, Value>` 타입에 대해 아래와 같은 문법을 제공합니다:

```
[<#key type#>: <#value type#>]
```

다른 표현으로 다음의 두 선언도 동일합니다:

```
let someDictionary: [String: Int] = ["Alex": 31, "Paul": 39]
let someDictionary: Dictionary<String, Int> = ["Alex": 31, "Paul": 39]
```

두 경우 모두 상수 `someDictionary`는 키로 문자열과 값으로 정수를 가지는 딕셔너리가 선언됩니다.

딕셔너리의 값은 대괄호에 해당 키를 지정하여 접근할 수 있습니다: `someDictionary["Alex"]`는 키 `"Alex"`와 연관값을 참조합니다. 서브스크립트는 딕셔너리의 값 타입의 옵셔널 값을 반환합니다. 지정한 키가 딕셔너리에 포함되지 않은 경우 서브스크립트는 `nil`을 반환합니다.

딕셔너리의 키 타입은 Swift 표준 라이브러리 `Hashable` 프로토콜을 준수해야 합니다.

Swift 표준 라이브러리 `Dictionary` 타입의 자세한 설명은 딕셔너리 (Dictionaries)를 참고바랍니다.

Grammar of a dictionary type

dictionary-type → [type **:** type]

옵셔널 타입 (Optional Type)

Swift 언어는 접미사 `?`를 Swift 표준 라이브러리에 정의된 명명 타입 `Optional<Wrapped>`의 문법적 편의로 정의합니다. 다른 표현으로 다음의 두 선언은 동일합니다:

```
var optionalInteger: Int?
var optionalInteger: Optional<Int>
```

두 경우 모두 변수 `optionalInteger`는 옵셔널 정수의 타입을 가지도록 선언됩니다. 타입과 `?`사이에는 공백이 없어야 합니다.

타입 `Optional<Wrapped>`는 열거형으로 정의되어 있고, 값이 존재하거나 존재하지 않을 수 있는 것을 나타내는데 `none`과 `some(Wrapped)`의 두 가지 케이스를 가집니다. 모든 타입은 명시적으로 선언되거나 옵셔널 타입으로 암시적으로 변환될 수 있습니다. 선언할 때 초기값을 제공하지 않으면 옵셔널 변수나 프로퍼티는 자동으로 `nil`을 설정합니다.

옵셔널 타입의 인스턴스는 값을 포함할 경우, 아래에서 보듯이 접시마 연산자 `!`를 사용하여 값에 접근할 수 있습니다:

```
optionalInteger = 42
optionalInteger! // 42
```

`nil`의 값을 가지는 옵셔널을 언래핑 하기위해 `!` 연산자를 사용하면 런타임 오류가 발생합니다.

옵셔널 체이닝과 옵셔널 바인딩을 사용하여 옵셔널 표현식에 대해 조건적으로 작업을 수행할 수도 있습니다. 값이 `nil`인 경우에 작업은 더이상 수행되지 않으므로 런타임 오류가 발생하지 않습니다.

더 자세한 정보와 옵셔널 타입 사용에 대한 예시를 보려면 옵셔널 (Optionals)을 참고바랍니다.

> **Grammar of an optional type**
>
> optional-type → type ?

암시적 언래핑 옵셔널 타입 (Implicitly Unwrapped Optional Type)

Swift 언어는 접미사 `!`를 Swift 표준 라이브러리에 정의된 명명 타입 `Optional<Wrapped>`의 문법적 편의로 정의하고, 옵셔널 타입에 접근할 때 자동으로 언래핑된 동작을 추가합니다. `nil`의 값을 가지는 옵셔널에 암시적 언래핑을 사용하려고 하면 런타임 오류가 발생합니다. 암시적 언래핑 동작을 제외하고 다음의 두 선언은 동일합니다:

```
var implicitlyUnwrappedString: String!
var explicitlyUnwrappedString: Optional<String>
```

타입과 `!` 사이에 공백이 포함되지 않습니다.

암시적 언래핑은 해당 타입을 포함하는 선언의 의미를 변경하기 때문에 튜플 타입이나 제너릭 타입 - 딕셔너리나 배열의 요소 타입 - 내에 중첩된 옵셔널 타입은 암시적 언래핑으로 표시할 수 없습니다. 예를 들어:

```
let tupleOfImplicitlyUnwrappedElements: (Int!, Int!)  // Error
let implicitlyUnwrappedTuple: (Int, Int)!             // OK

let arrayOfImplicitlyUnwrappedElements: [Int!]        // Error
let implicitlyUnwrappedArray: [Int]!                  // OK
```

암시적 언래핑 옵셔널은 옵셔널 값과 동일한 `Optional<Wrapped>` 타입을 가지므로 코드에서 옵셔널을 사용할 수 있는 모든 곳에서 사용할 수 있습니다. 예를 들어 옵셔널의 변수, 상수, 프로퍼티에 암시적 언래핑 옵셔널의 값을 할당할 수 있고 그 반대도 가능합니다.

옵셔널과 마찬가지로 암시적 언래핑 옵셔널 변수나 프로퍼티를 선언할 때 초기값을 제공하지 않으면 자동으로 `nil`을 기본값으로 설정합니다.

암시적 언래핑 옵셔널 표현식에 조건적으로 동작을 수행하려면 옵셔널 체이닝을 사용합니다. 값이 `nil`이라면 더 이상 작업이 수행되지 않으며 런타임 오류가 발생하지 않습니다.

암시적 언래핑 옵셔널 타입에 대한 자세한 정보는 암시적 언래핑 옵셔널 (Implicitly Unwrapped Optionals)을 참고 바랍니다.

Grammar of an implicitly unwrapped optional type

implicitly-unwrapped-optional-type → type `!`

프로토콜 합성 타입 (Protocol Composition Type)

*프로토콜 합성 타입(protocol composition type)*은 지정된 프로토콜의 목록에서 각 프로토콜을 준수해야 하는 타입을 정의하거나, 특정 클래스를 상속하면서 동시에 지정된 프로토콜의 목록애서 각 프로토콜을 준수해야 하는 타입을 정의할 수도 있습니다. 프로토콜 합성 타입은 타입 주석, 제네릭 매개변수 절, 제네릭 `where` 절을 지정할 때만 사용할 수 있습니다.

프로토콜 합성 타입은 다음의 형식을 가집니다:

```
<#Protocol 1#> & <#Protocol 2#>
```

프로토콜 합성 타입을 사용하면 타입이 준수하려는 각 프로토콜을 상속하는 새롭운 프로토콜을 명시적으로 정의하지 않아도 타입이 여러 프로토콜의 요구사항을 준수하는 값을 지정할 수 있습니다. 예를 들어 `ProtocolA`, `ProtocolB`, `ProtocolC`를 상속하는 새로운 프로토콜을 선언하는 대신 프로토콜 합성 타입 `ProtocolA & ProtocolB & ProtocolC`를 사용할 수 있습니다. 마찬가지로 `SuperClass`의 하위 클래스와 `ProtocolA`를 준수하는 새로운 프로토콜을 선언하는 대신에 `SuperClass & ProtocolA`를 사용할 수 있습니다.

프로토콜 합성 목록의 각 항목은 다음 중 하나입니다; 이 목록은 최대 하나의 클래스를 포함할 수 있습니다:

- 클래스의 이름
- 프로토콜의 이름
- 기본 타입이 프로토콜 합성 타입, 프로토콜, 클래스인 타입 별칭

프로토콜 합성 타입이 타입 별칭을 포함할 때, 동일한 프로토콜이 정의에 중복해서 나타날 수 있으며 - 중복은 무시됩니다. 예를 들어 아래 코드에서 `PQR`의 정의는 `P & Q & R`과 동일합니다.

```
typealias PQ = P & Q
typealias PQR = PQ & Q & R
```

Grammar of a protocol composition type

protocol-composition-type → type-identifier & protocol-composition-continuation
protocol-composition-continuation → type-identifier | protocol-composition-type

불투명 타입 (Opaque Type)

참고

번역자 보충 Swift 6.4의 `some P?`와 `any P?` 구문 및 프로토콜 합성의 괄호 조건은 some과 any의 옵셔널 구문 (Optional some and any Syntax)을 참고바랍니다.

*불투명 타입(opaque type)*은 구체적인 타입 지정없이 프로토콜이나 프로토콜 합성을 준수하는 타입을 정의합니다.

불투명 타입은 함수나 서브스크립트의 반환 타입, 함수, 서브스크립트, 이니셜라이저 매개변수의 타입, 또는 프로퍼티의 타입으로 나타납니다. 불투명 타입은 배열의 요소 타입이나 옵셔널의 래핑된 타입과 같은 튜플 타입 또는 제네릭 타입의 부분으로 나타날 수 없습니다.

불투명 타입은 다음의 형식을 가집니다:

```
some <#constraint#>
```

위 형식의 *제약조건(constraint)*은 클래스 타입, 프로토콜 타입, 프로토콜 합성 타입, `Any`입니다. 어떤 값이 불투명 타입의 인스턴스로 사용되려면 그 값은 지정된 프로토콜이나 프로토콜 합성을 준수하거나 나열된 클래스를 상속해야 합니다. 불투명 값과 상호작용하는 코드는 *제약조건(constraint)*에 의해 정의된 인터페이스 범위 내에서만 값으로 사용할 수 있습니다.

컴파일 시간에 불투명 타입을 가진 값은 구체적인 타입을 가지며, Swift는 이 내부 타입을 사용해 최적화를 수행할 수 있습니다. 그러나 불투명 타입은 하나의 경계를 형성하며 이 경계를 통해 내부 타입에 대한 정보가 외부로 노출될 수 없습니다.

프로토콜 선언은 불투명 타입을 포함할 수 없습니다. 클래스는 final이 아닌 메서드(nonfinal method)의 반환 타입으로 불투명 타입을 사용할 수 없습니다.

불투명 타입을 반환 타입으로 사용하는 함수는 항상 하나의 동일한 내부 타입을 반환해야 합니다. 반환 타입은 함수의 제네릭 타입 매개변수도 포함할 수 있습니다. 예를 들어 함수 `someFunction<T>()`는 타입 `T`나 `Dictionary<String, T>`의 값을 반환할 수 있습니다.

매개변수에 불투명 타입을 작성하는 것은 제네릭 타입에 대한 편의 문법(syntactic sugar)이며 이것은 제네릭 타입 매개변수의 이름을 지정하지 않는 방식입니다. 암시적 제네릭 타입 매개변수는 불투명 타입에 명시된 프로토콜을 준수하도록 요구하는 제약조건을 가집니다. 여러 불투명 타입을 작성한 경우에는 각각 자체 제네릭 타입 매개변수를 생성합니다. 예를 들어 다음의 선언은 동일합니다:

```
func someFunction(x: some MyProtocol, y: some MyProtocol) { }
func someFunction<T1: MyProtocol, T2: MyProtocol>(x: T1, y: T2) { }
```

두 번째 선언에서 제네릭 타입 매개변수 `T1`과 `T2`는 이름을 가지므로 코드의 다른 곳에서 이 타입을 참조할 수 있습니다. 반면에 첫 번째 선언의 제네릭 타입 매개변수는 이름이 없으므로 다른 코드에서 참조할 수 없습니다.

가변 매개변수(variadic parameter)의 타입에는 불투명 타입을 사용할 수 없습니다.

반환되는 함수 타입의 매개변수나 함수 타입인 매개변수 타입의 매개변수로 불투명 타입을 사용할 수 없습니다. 이러한 위치에서는 함수를 호출하는 쪽에서 알 수 없는 타입의 값을 구성해야 하기 때문입니다.

```
protocol MyProtocol { }
func badFunction() -> (some MyProtocol) -> Void { }  // Error
func anotherBadFunction(callback: (some MyProtocol) -> Void) { }  // Error
```

Grammar of an opaque type

opaque-type → `some` type

박싱 프로토콜 타입 (Boxed Protocol Type)

박싱 프로토콜 타입(Boxed Protocol Type)은 프로토콜이나 프로토콜 합성을 준수하는 타입을 정의하고 프로그램 실행 중에도 준수하는 타입이 변경될 수 있습니다.

박싱 프로토콜 타입은 다음과 같은 형식을 가집니다:

```
any <#constraint#>
```

위 형식의 *제약조건(constraint)*은 프로토콜 타입, 프로토콜 합성 타입, 프로토콜 타입의 메타타입, 프로토콜 합성 타입의 메타타입입니다.

런타임에 박싱 프로토콜 타입의 인스턴스는 제약조건에 충족하는 모든 타입의 값을 포함할 수 있습니다. 이런 동작은 컴파일 시간에 구체적인 타입이 고정되는 불투명 타입(opaque type) 동작과 대조됩니다. 박싱 프로토콜 타입으로 사용되는 추가 간접 수준을 *박싱(boxing)*이라 합니다. 박싱(Boxing)은 일반적으로 별도의 메모리 공간을 할당하고 해당 값에 접근할 때 추가적인 간접 수준을 필요로 하므로 런타임 성능에 영향을 줄 수 있습니다.

`any`를 `Any`나 `AnyObject` 타입에 적용해도 이미 박싱 프로토콜 타입이므로 아무런 효과가 없습니다.

Grammar of a boxed protocol type

boxed-protocol-type → `any` type

메타타입 타입 (Metatype Type)

*메타타입 타입(metatype type)*은 클래스 타입, 구조체 타입, 열거형 타입, 프로토콜 타입을 포함하여 모든 타입의 타입 자체를 나타냅니다.

클래스, 구조체, 열거형 타입의 메타타입은 해당 타입의 이름 다음에 `.Type`을 붙입니다. 런타임 시 해당 프로토콜을 준수하는 구체적인 타입이 아닌 - 프로토콜 타입의 메타타입은 프로토콜의 이름 다음에 `.Protocol`을 붙입니다. 예를 들어 클래스 타입 `SomeClass`의 메타타입은 `SomeClass.Type`이고 프로토콜 `SomeProtocol`의 메타타입은 `SomeProtocol.Protocol`입니다.

접미사 `self` 표현식을 사용하여 타입을 값으로 접근할 수 있습니다. 예를 들어 `SomeClass.self`는 `SomeClass`의 인스턴스가 아닌 `SomeClass` 자체를 반환합니다. 그리고 `SomeProtocol.self`는 런타임 시 `SomeProtocol`을 준수하는 타입의 인스턴스가 아닌 `SomeProtocol` 자체를 반환합니다. 다음 예시와 같이 타입의 인스턴스와 함께 `type(of:)` 함수를 호출하여 해당 인스턴스의 런타임 시 타입을 값으로 접근할 수 있습니다:

```
class SomeBaseClass {
    class func printClassName() {
        print("SomeBaseClass")
    }
}
class SomeSubClass: SomeBaseClass {
    override class func printClassName() {
        print("SomeSubClass")
    }
}
let someInstance: SomeBaseClass = SomeSubClass()
// The compile-time type of someInstance is SomeBaseClass,
// and the runtime type of someInstance is SomeSubClass
type(of: someInstance).printClassName()
// Prints "SomeSubClass".
```

자세한 내용은 Swift 표준 라이브러리의 `type(of:)`을 참고바랍니다.

메타타입 값을 사용하여 해당 타입의 인스턴스를 생성하려면 이니셜라이저 표현식을 사용할 수 있습니다. 클래스 인스턴스의 경우 호출되는 이니셜라이저는 `required` 키워드로 표시하거나 `final` 키워드로 전체 클래스를 표시해야 합니다.

```
class AnotherSubClass: SomeBaseClass {
    let string: String
    required init(string: String) {
        self.string = string
    }
    override class func printClassName() {
        print("AnotherSubClass")
    }
}
let metatype: AnotherSubClass.Type = AnotherSubClass.self
let anotherInstance = metatype.init(string: "some string")
```

Grammar of a metatype type

metatype-type → type `.` `Type` | type `.` `Protocol`

Any 타입 (Any Type)

Any 타입은 다른 모든 타입의 값을 포함할 수 있습니다. Any는 다음 타입의 인스턴스에 대해 구체적인 타입으로 사용할 수 있습니다:

- 클래스, 구조체, 열거형
- `Int.self`와 같은 메타타입
- 다양한 타입으로 구성된 튜플
- 클로저 또는 함수 타입

```
let mixed: [Any] = ["one", 2, true, (4, 5.3), { () -> Int in return 6 }]
```

인스턴스에 대해 구체적인 타입으로 `Any`를 사용할 때, 해당 프로퍼티나 메서드에 접근하려면 먼저 알려진 타입으로 인스턴스를 캐스팅해야 합니다. `Any`의 구체적인 타입인 인스턴스는 본래 동적 타입을 유지하고 `as`, `as?`, `as!`와 같은 타입 캐스팅 연산자(type-cast operators) 중 하나를 사용하여 타입을 캐스팅할 수 있습니다. 예를 들어 `as?`을 사용하여 배열의 첫 번째 객체를 `String`으로 다운캐스트합니다:

```
if let first = mixed.first as? String {
    print("The first item, '\(first)', is a string.")
}
// Prints "The first item, 'one', is a string.".
```

캐스팅에 대한 자세한 내용은 타입 캐스팅 (Type Casting)을 참고바랍니다.

`AnyObject` 프로토콜은 `Any` 타입과 유사합니다. 모든 클래스는 암시적으로 `AnyObject`를 준수합니다. 언어에 의해 정의되는 `Any`와 달리 `AnyObject`는 Swift 표준 라이브러리에 의해 정의됩니다. 더 자세한 내용은 클래스 전용 프로토콜 (Class-Only Protocols)과 `AnyObject`를 참고바랍니다.

Grammar of an Any type

any-type → `Any`

Self 타입 (Self Type)

`Self` 타입은 특정 타입이 아니라 현재 타입을 반복해서 쓰거나 그 이름을 알 필요 없이 편리하게 참조할 수 있습니다.

프로토콜 선언이나 프로토콜 멤버 선언에서 `Self` 타입은 프로토콜을 채택하는 최종 타입을 나타냅니다.

구조체, 클래스, 열거형 선언에서 `Self` 타입은 선언에 의해 도입된 타입을 참조합니다. 타입의 멤버 선언 내에서 `Self` 타입은 해당 타입을 참조합니다. 클래스 선언의 멤버에서 `Self` 타입은 다음의 경우에만 사용할 수 있습니다:

- 메서드의 반환 타입
- 읽기 전용 서브스크립트의 반환 타입
- 읽기 전용 연산 프로퍼티의 타입
- 메서드의 본문

예를 들어 아래의 코드는 반환 타입이 `Self`인 인스턴스 메서드 `f`를 보여줍니다.

```
class Superclass {
    func f() -> Self { return self }
}
let x = Superclass()
print(type(of: x.f()))
// Prints "Superclass".

class Subclass: Superclass { }
let y = Subclass()
print(type(of: y.f()))
// Prints "Subclass".

let z: Superclass = Subclass()
print(type(of: z.f()))
// Prints "Subclass".
```

위의 예시의 마지막 부분은 `Self`가 변수 `z` 자체의 컴파일 시 타입(compile-time type)인 `Superclass`가 아닌 `z` 값의 런타임 타입(runtime type)인 `Subclass`를 참조하는 것을 보여줍니다.

중첩된 타입 선언 내에서 `Self` 타입은 가장 안쪽 타입 선언에 의해 도입된 타입을 참조합니다.

`Self` 타입은 Swift 표준 라이브러리에서 `type(of:)` 함수와 동일한 타입을 참조합니다. 현재 타입의 멤버를 접근하기 위해 `Self.someStaticMember`라고 작성하는 것은 `type(of:self).someStaticMember`로 작성하는 것과 동일합니다.

Grammar of a Self type

self-type → `Self`

타입 상속 절 (Type Inheritance Clause)

*타입 상속 절(type inheritance clause)*은 명명 타입이 상속하는 클래스와 명명 타입이 준수하는 프로토콜을 지정하기 위해 사용됩니다. 타입 상속 절은 콜론(`:`)으로 시작하고, 그 뒤에 타입 식별자의 목록이 옵니다.

클래스 타입은 하나의 상위 클래스를 상속할 수 있고 여러 개의 프로토콜을 준수할 수 있습니다. 클래스를 정의할 때, 상위 클래스의 이름은 타입 식별자의 목록에서 첫 번째로 나타나야 하고, 다음으로 준수하는 여러 개의 프로토콜을 작성합니다. 클래스가 다른 클래스를 상속하지 않으면 해당 목록은 프로토콜로 시작할 수 있습니다. 클래스 상속에 대한 자세한 설명과 예시는 상속 (Inheritance)을 참고바랍니다.

클래스 타입 외 명명 타입은 프로토콜 목록만 상속하거나 준수할 수 있습니다. 프로토콜 타입은 여러 다른 프로토콜을 상속할 수 있습니다. 프로토콜 타입이 다른 프로토콜을 상속할 때, 그 상위 프로토콜의 요구사항이 통합되고 해당 프로토콜을 상속하는 모든 타입은 그 요구사항을 모두 준수해야 합니다. 이 목록의 프로토콜 이름 앞에 물결표(~)를 작성하면 프로토콜 암시적 준수 (Implicit Conformance to a Protocol)에서 설명된 것처럼 해당 프로토콜에 대한 암시적 준수가 억제됩니다.

열거형 정의의 타입 상속 절은 프로토콜 목록이거나 케이스의 원시값(raw values)을 지정하는 열거형인 경우 해당 원시값의 타입을 지정하는 단일 명명 타입일 수 있습니다. 타입 상속 절을 사용하여 원시값의 타입을 지정하는 열거형 정의의 예시는 원시값 (Raw Values)을 참고바랍니다.

Grammar of a type inheritance clause

type-inheritance-clause → `:` type-inheritance-list type-inheritance-list → attributes? ~? type-identifier | attributes? ~? type-identifier `,` type-inheritance-list

타입 추론 (Type Inference)

Swift는 광범위하게 *타입 추론(type inference)*을 사용하므로 코드에서 많은 변수와 표현식의 타입 전체나 타입 일부를 생략할 수 있습니다. 예를 들어 `var x: Int = 0`으로 작성하는 대신에 `var x = 0`으로 타입을 완벽하게 생략하고 작성할 수 있습니다 -- 컴파일러는 `x`를 타입 `Int`의 값으로 추론합니다. 유사하게 컨텍스트에서 전체 타입이 추론될 수 있을 때 타입을 생략할 수 있습니다. 예를 들어 `let dict: Dictionary = ["A": 1]`을 작성하면, 컴파일러는 `dict`이 타입 `Dictionary<String, Int>`이라고 추론합니다.

위의 두 예시에서 타입 정보는 표현식 트리의 잎에서 루트까지 전달됩니다. 즉, `var x: Int = 0`에서 `x`의 타입은 먼저 `0`의 타입을 확인한 다음에 이 타입 정보를 루트(변수 `x`)까지 전달하여 추론합니다.

Swift에서 타입 정보는 루트에서 잎까지 반대로 흐를 수도 있습니다. 예를 들어 다음 예시에서 상수 `eFloat`의 명시적 타입 주석(`: Float`)은 숫자 리터럴 `2.71828`이 `Double`이 아닌 `Float` 타입으로 추론합니다.

```
let e = 2.71828 // The type of e is inferred to be Double.
let eFloat: Float = 2.71828 // The type of eFloat is Float.
```

Swift의 타입 추론은 단일 표현식이나 구문 수준에서 동작합니다. 이것은 표현식에서 생략된 타입이나 타입의 일부를 추론하는데 필요한 모든 정보는 표현식이나 하위 표현식 중 하나를 타입 검사(type-checking)하여 접근할 수 있습니다.

언어 참조

표현식 (Expressions)

값에 접근, 수정, 할당합니다.

> **참고**
>
> **번역자 보충** 모듈의 선언을 명확하게 선택하는 `ModuleName::Name` 구문은 모듈 선택자 (Module Selectors)를 참고바랍니다.

Swift에는 네 종류의 표현식이 있습니다: 접두 표현식(prefix expressions), 중위 표현식(infix expressions), 기본 표현식(primary expressions), 접미 표현식(postfix expressions)입니다. 표현식을 평가하면 값을 반환하거나 부작용(side effect)를 일으키거나 둘 다 발생합니다.

접두 표현식과 중위 표현식은 더 작은 표현식에 연산자를 적용할 수 있습니다. 기본 표현식은 개념적으로 가장 간단한 표현식이며 값에 접근하는 방법을 제공합니다. 접미 표현식은 접두 표현식과 중위 표현식과 같이 함수 호출이나 멤버 접근과 같은 접미사를 사용하여 더 복잡한 표현식을 작성할 수 있습니다. 표현식의 각 종류는 아래 섹션에서 자세하게 설명합니다.

> **Grammar of an expression**
>
> expression → try-operator? await-operator? prefix-expression infix-expressions?

접두 표현식 (Prefix Expressions)

*접두 표현식(Prefix expressions)*은 옵셔널 접두 연산자(prefix operator)와 표현식을 결합합니다. 접두 연산자는 하나의 인자를 받으며 그 인자는 연산자 뒤에 오는 표현식입니다.

이 연산자의 동작에 대한 자세한 설명은 기본 연산자 (Basic Operators)와 고급 연산자 (Advanced Operators)를 참고바랍니다.

Swift 표준 라이브러리에 의해 제공되는 연산자의 자세한 설명은 연산자 선언 (Operator Declarations)을 참고바랍니다.

> **Grammar of a prefix expression**
>
> prefix-expression → prefix-operator? postfix-expression prefix-expression → in-out-expression

In-Out 표현식 (In-Out Expression)

*in-out 표현식(in-out expression)*은 in-out 인자로 함수 호출 표현식에 전달되는 변수를 표시합니다.

```
&<#expression#>
```

in-out 매개변수에 대한 설명과 예시는 In-Out 매개변수 (In-Out Parameters)를 참고바랍니다.

in-out 표현식은 포인터 타입으로 암시적 변환 (Implicit Conversion to a Pointer Type)에서 설명한대로 포인터가 필요한 컨텍스트에서 포인터가 아닌 인자(non-pointer argument)를 제공할 때도 사용합니다.

Grammar of an in-out expression

in-out-expression → & primary-expression

Try 연산자 (Try Operator)

*try 표현식(try expression)*은 `try` 연산자 다음에 오류를 던질 수 있는 표현식으로 구성되어 있습니다. 형식은 다음과 같습니다:

```
try <#expression#>
```

`try` 표현식의 값은 위 형식의 *표현식(expression)*과 같습니다.

*옵셔널 try 표현식(optional-try expression)*은 `try?` 연산자 다음에 오류를 던질 수 있는 표현식으로 구성되어 있습니다. 형식은 다음과 같습니다:

```
try? <#expression#>
```

위 형식의 *표현식(expression)*에서 오류를 던지지 않는다면, 옵셔널 try 표현식의 값은 위 형식의 *표현식(expression)*의 값을 포함하는 옵셔널입니다. 그렇지 않으면 옵셔널 try 표현식의 값은 `nil`입니다.

*강제 try 표현식(forced-try expression)*은 `try!` 연산자 다음에 오류를 던질 수 있는 표현식으로 구성되어 있습니다. 형식은 다음과 같습니다:

```
try! <#expression#>
```

강제 try 표현식의 값은 위 형식의 *표현식(expression)*의 값입니다. 위 형식의 *표현식(expression)*이 오류를 던지면 런타임 오류가 발생합니다.

중위 연산자의 왼쪽 피연산자에 `try`, `try?`, `try!`가 표시되어 있으면, 해당 연산자는 중위 표현식 전체에 적용됩니다. 즉, 괄호를 사용하여 연산자의 적용 범위를 지정할 수 있습니다.

```
// Writing 'try' applies to both function calls.
sum = try someThrowingFunction() + anotherThrowingFunction()

// Writing 'try' applies to both function calls.
sum = try (someThrowingFunction() + anotherThrowingFunction())

// Error: Writing 'try' applies to both function calls.
sum = (try someThrowingFunction()) + anotherThrowingFunction()
```

`try` 표현식은 중위 연산자의 오른쪽 피연산자에 나타날 수 없지만, 중위 연산자가 할당 연산자이거나 `try` 표현식이 괄호로 묶여있는 경우에는 예외입니다.

표현식에 `try`와 `await` 연산자 모두 포함될 경우, `try` 연산자를 먼저 작성합니다.

더 자세한 정보와 `try`, `try?`, `try!` 사용법에 대한 예시는 오류 처리 (Error Handling)를 참고바랍니다.

Grammar of a try expression

try-operator → `try` | `try ?` | `try !`

Await 연산자 (Await Operator)

*await 표현식(await expression)*은 `await` 연산자 다음에 비동기 동작의 결과를 사용하는 표현식으로 구성되어 있습니다. 형식은 다음과 같습니다:

```
await <#expression#>
```

`await` 표현식의 값은 위 형식의 *표현식(expression)*의 값입니다.

`await`로 표시된 표현식은 *잠재적 중단 지점(potential suspension point)*이라 합니다. 비동기 함수의 실행은 `await`로 표시된 각 표현식에서 일시 중단될 수 있습니다. 반대로 비동기 코드의 실행은 이런 지점이 아닌 다른 시점에서는 중단되지 않습니다. 이는 잠재적 중단 지점 사이의 코드가 일시적으로 중단을 깨야하는 상태를 안전하게 업데이트 할 수 있고 다음 잠재적 중단 지점 이전에 해당 작업을 완료하기만 하면 됩니다.

`await` 표현식은 `async(priority:operation:)` 함수에 전달된 후행 클로저와 같이 비동기 컨텍스트 내에서만 사용할 수 있습니다. `defer` 구문의 본문이나 동기 함수 타입의 자동 클로저(autoclosure) 내에서는 사용할 수 없습니다.

중위 연산자(infix operator)의 왼쪽 피연산자에 `await` 연산자로 표시하면 해당 연산자는 전체 중위 표현식에 적용됩니다. 즉, 괄호를 사용하여 연산자의 적용 범위를 명시할 수 있습니다.

```
// Writing 'await' applies to both function calls.
sum = await someAsyncFunction() + anotherAsyncFunction()

// Writing 'await' applies to both function calls.
sum = await (someAsyncFunction() + anotherAsyncFunction())

// Error: Writing 'await' applies to both function calls.
sum = (await someAsyncFunction()) + anotherAsyncFunction()
```

`await` 표현식은 중위 연산자의 우항에 나타날 수 없지만, 중위 연산자가 할당 연산자이거나 `await` 표현식이 괄호로 묶인 경우는 예외입니다.

표현식에 `await`와 `try` 연산자가 모두 포함되면, `try` 연산자를 먼저 작성합니다.

Grammar of an await expression

await-operator → `await`

중위 표현식 (Infix Expressions)

*중위 표현식(Infix expressions)*은 중위 이항 연산자(infix binary operator)의 왼쪽과 오른쪽 인자로 사용되는 표현식을 결합합니다. 형식은 다음과 같습니다:

```
<#left-hand argument#> <#operator#> <#right-hand argument#>
```

이 연산자의 동작에 대한 자세한 설명은 기본 연산자 (Basic Operators)와 고급 연산자 (Advanced Operators)를 참고바랍니다.

Swift 표준 라이브러리에 의해 제공되는 연산자에 대한 자세한 내용은 연산자 선언 (Operator Declarations)을 참고바랍니다.

> **참고**
>
> 구문 분석 시, 중위 연산자로 구성된 표현식은 평평한 목록으로 표현됩니다. 이 목록은 연산자 우선순위를 적용하여 트리로 변환합니다. 예를 들어 표현식 `2 + 3 * 5`는 처음에는 다섯 개의 항목 2, +, 3, *, 5의 평평한 목록으로 이해합니다. 이 과정은 표현식을 (2 + (3 * 5)) 트리로 변환합니다.

Grammar of an infix expression

infix-expression → infix-operator prefix-expression infix-expression → assignment-operator try-operator? await-operator? prefix-expression infix-expression → conditional-operator try-operator? await-operator? prefix-expression infix-expression → type-casting-operator infix-expressions → infix-expression infix-expressions?

할당 연산자 (Assignment Operator)

*할당 연산자(assignment operator)*는 주어진 표현식에 대해 새로운 값을 설정합니다. 다음의 형식을 가집니다:

```
<#expression#> = <#value#>
```

위 형식의 *표현식(expression)*의 값은 위 형식의 *값(value)*을 평가하여 얻어진 값으로 설정합니다. 위 형식의 *표현식(expreesion)*이 튜플이면, 위 형식의 *값(value)*은 튜플이어야 하고 동일한 수의 요소를 가져야 합니다. (중첩된 튜플도 가능합니다.) 위 형식의 *값(value)*에서 위 형식의 *표현식(expression)*에 할당됩니다. 예를 들어:

```
(a, _, (b, c)) = ("test", 9.45, (12, 3))
// a is "test", b is 12, c is 3, and 9.45 is ignored
```

할당 연산자는 값을 반환하지 않습니다.

Grammar of an assignment operator

assignment-operator → =

삼항 조건 연산자 (Ternary Conditional Operator)

*삼항 조건 연산자(ternary conditional operator)*는 조건의 값을 기반으로 주어진 두 개의 값 중 하나로 나타냅니다. 다음의 형식을 가집니다:

```
<#condition#> ? <#expression used if true#> : <#expression used if false#>
```

위 형식의 *조건(condition)*이 `true`이면, 조건부 연산자는 첫 번째 표현식을 평가하고 해당 값을 반환합니다. 그렇지 않으면 두 번째 표현식을 평가하고 해당 값을 반환합니다. 사용하지 않은 표현식은 평가되지 않습니다.

삼항 조건 연산자 사용에 대한 예시는 삼항 조건 연산자 (Ternary Conditional Operator)를 참고바랍니다.

> **Grammar of a conditional operator**
>
> conditional-operator → ? expression **:**

타입 캐스팅 연산자 (Type-Casting Operators)

타입 캐스팅 연산자(type-casting operators)는 네 가지가 있습니다: `is` 연산자, `as` 연산자, `as?` 연산자, `as!` 연산자입니다.

다음의 형식을 가지고 있습니다:

```
<#expression#> is <#type#>
<#expression#> as <#type#>
<#expression#> as? <#type#>
<#expression#> as! <#type#>
```

`is` 연산자는 런타임 시에 위 형식의 *표현식(expression)*이 지정된 위 형식의 *타입(type)*으로 캐스팅될 수 있는지 확인합니다. 위 형식의 *표현식(expression)*이 지정된 위 형식의 *타입(type)*으로 캐스팅될 수 있으면 `true`를 반환하고; 그렇지 않으면 `false`를 반환합니다.

`as` 연산자는 업캐스팅(upcasting)이나 브리징(bridging)과 같이 컴파일 시 변환이 항상 성공하는 경우에 캐스팅을 수행합니다. 업캐스팅(Upcasting)을 사용하면 중간 변수를 사용하지 않고 표현식을 해당 타입의 상위 타입 인스턴스로 사용할 수 있습니다. 다음 예시의 방식은 둘 다 동일합니다:

```
func f(_ any: Any) { print("Function for Any") }
func f(_ int: Int) { print("Function for Int") }
let x = 10
f(x)
// Prints "Function for Int".

let y: Any = x
f(y)
// Prints "Function for Any".

f(x as Any)
// Prints "Function for Any".
```

브리징(Bridging)을 사용하면 `String`과 같은 Swift 표준 라이브러리 타입을 `NSString`과 같은 해당 Foundation 타입으로 사용할 수 있게 해주고 새로운 인스턴스를 생성할 필요가 없습니다. 브리징에 대한 자세한 설명은 Foundation 타입 동작 (Working with Foundation Types)을 참고바랍니다.

`as?` 연산자는 위 형식의 *표현식(expression)*을 지정한 위 형식의 *타입(type)*으로 조건부로 캐스팅을 수행합니다. `as?` 연산자는 지정한 위 형식의 *타입(type)*의 옵셔널로 반환합니다. 런타임 시 캐스팅이 성공하면, 위 형식의 *표현식(expression)*의 값은 옵셔널로 래핑하여 반환됩니다; 그렇지 않으면 `nil`이 반환됩니다. 지정된 위 형식의 *타입(type)*으로 캐스팅이 항상 실패하거나 항상 성공이 보장되면 컴파일 시 오류가 발생합니다.

`as!` 연산자는 위 형식의 *표현식(expression)*을 지정한 위 형식의 *타입(type)*으로 강제 캐스팅을 수행합니다. `as!` 연산자는 옵셔널 타입이 아닌 지정한 위 형식의 *타입(type)*의 값을 반환합니다. 캐스팅이 실패하면 런타임 오류가 발생합니다. `x as! T`의 동작은 `(x as? T)!`의 동작과 동일합니다.

타입 캐스팅에 대한 자세한 내용과 타입 캐스팅 연산자 사용에 대한 예시는 타입 캐스팅 (Type Casting)을 참고바랍니다.

Grammar of a type-casting operator

type-casting-operator → `is` type type-casting-operator → `as` type type-casting-operator → `as ?` type
type-casting-operator → `as !` type

기본 표현식 (Primary Expressions)

*기본 표현식(Primary expressions)*은 표현식의 가장 기본입니다. 표현식 자체로 사용될 수 있으며, 접두 표현식, 중위 표현식, 접미 표현식을 만들기 위해 다른 토큰과 결합될 수 있습니다.

Grammar of a primary expression

primary-expression → identifier generic-argument-clause? primary-expression → literal-expression
primary-expression → self-expression primary-expression → superclass-expression primary-expression →
conditional-expression primary-expression → closure-expression primary-expression →
parenthesized-expression primary-expression → tuple-expression primary-expression →
implicit-member-expression primary-expression → wildcard-expression primary-expression →
macro-expansion-expression primary-expression → key-path-expression primary-expression →
selector-expression primary-expression → key-path-string-expression

리터럴 표현식 (Literal Expression)

*리터럴 표현식(literal expression)*은 일반 리터럴(문자열이나 숫자 등), 배열이나 딕셔너리 리터럴, 플레이그라운드 리터럴로 구성됩니다:

참고

Swift 5.9 이전에는 다음의 특수 리터럴이 인식되었습니다: `#column`, `#dsohandle`, `#fileID`, `#filePath`, `#file`, `#function`, `#line`. 이것은 현재 Swift 표준 라이브러리에 매크로(macro)로 구현됩니다: `column()`, `dsohandle()`, `fileID()`, `filePath()`, `file()`, `function()`, `line()`.

*배열 리터럴(array literal)*은 순서가 있는 값의 컬렉션입니다. 형식은 다음과 같습니다:

```
[<#value 1#>, <#value 2#>, <#...#>]
```

배열의 마지막 표현식 뒤에 콤마를 추가할 수 있습니다. 배열 리터럴의 값은 `[T]` 타입이고, 여기서 T는 표현식 내의 타입을 나타냅니다. 여러 타입의 표현식이 있는 경우 T는 가장 가까운 공통 상위 타입(supertype)입니다. 빈 배열 리터럴은 빈 대괄호 쌍을 사용하여 작성하고 지정된 타입의 빈 배열을 생성할 수 있습니다.

```
var emptyArray: [Double] = []
```

*딕셔너리 리터럴(dictionary literal)*은 순서가 없는 키-값 쌍의 컬렉션입니다. 형식은 다음과 같습니다:

```
[<#key 1#>: <#value 1#>, <#key 2#>: <#value 2#>, <#...#>]
```

딕셔너리에 마지막 표현식 뒤에 콤마를 추가할 수 있습니다. 딕셔너리 리터럴의 값은 `[Key: Value]` 타입을 가지고, 여기서 `Key`는 키 표현식의 타입이고 `Value`는 값 표현식의 타입입니다. 여러 타입의 표현식이 있는 경우 `Key`와 `Value`는 해당 값에 대해 가장 가까운 공통 상위 타입입니다. 빈 딕셔너리 리터럴은 빈 배열 리터럴과 구분하기 위해 대괄호 쌍 내에 콜론(`[:]`)을 작성합니다. 빈 딕셔너리 리터럴을 지정한 키와 값 타입으로 빈 딕셔너리 리터럴을 생성할 수 있습니다.

```
var emptyDictionary: [String: Double] = [:]
```

*플레이그라운드 리터럴(playground literal)*은 Xcode에서 색상, 파일, 이미지를 대화형으로 나타낼 수 있는 표현식입니다. Xcode 외부의 일반 텍스트에서 플레이그라운드 리터럴은 특수 리터럴 구문을 사용하여 표현합니다.

Xcode에서 플레이그라운드 리터럴 사용에 대한 정보는 Xcode Help의 색상, 파일, 이미지 리터럴 추가하기 (Add a color, file, or image literal)를 참고바랍니다.

Grammar of a literal expression

literal-expression → literal literal-expression → array-literal | dictionary-literal | playground-literal

array-literal → `[` array-literal-items? `,`? `]` array-literal-items → array-literal-item | array-literal-item `,` array-literal-items array-literal-item → expression

dictionary-literal → `[` dictionary-literal-items `,`? `]` | `[` `:` `]` dictionary-literal-items → dictionary-literal-item | dictionary-literal-item `,` dictionary-literal-items dictionary-literal-item → expression `:` expression

playground-literal → `#colorLiteral` `(` `red` `:` expression `,` `green` `:` expression `,` `blue` `:` expression `,` `alpha` `:` expression `)` playground-literal → `#fileLiteral` `(` `resourceName` `:` expression `)`
playground-literal → `#imageLiteral` `(` `resourceName` `:` expression `)`

Self 표현식 (Self Expression)

`self` 표현식은 현재 타입 또는 해당 타입의 인스턴스를 명시적으로 참조하는 표현입니다. 형식은 다음과 같습니다:

```
self
self.<#member name#>
self[<#subscript index#>]
self(<#initializer arguments#>)
self.init(<#initializer arguments#>)
```

이니셜라이저, 서브스크립트, 인스턴스 메서드에서 `self`는 해당 코드가 속한 타입의 인스턴스를 참조합니다. 타입 메서드에서 `self`는 해당 코드가 속한 타입 자체를 참조합니다.

`self` 표현식은 멤버에 접근할 때 범위를 지정하는데 사용되고 함수 매개변수와 같이 범위에 같은 이름의 다른 변수가 있을 때 혼돈을 방지하기 위해 사용됩니다. 예를 들어:

```
class SomeClass {
    var greeting: String
    init(greeting: String) {
        self.greeting = greeting
    }
}
```

값 타입의 mutating 메서드에서 해당 값 타입의 새 인스턴스를 `self`에 할당할 수 있습니다. 예를 들어:

```
struct Point {
    var x = 0.0, y = 0.0
    mutating func moveBy(x deltaX: Double, y deltaY: Double) {
        self = Point(x: x + deltaX, y: y + deltaY)
    }
}
```

Grammar of a self expression

self-expression → `self` | self-method-expression | self-subscript-expression | self-initializer-expression

self-method-expression → `self .` identifier self-subscript-expression → `self [` function-call-argument-list `]`
 self-initializer-expression → `self . init`

상위 클래스 표현식 (Superclass Expression)

*상위 클래스 표현식(superclass expression)*을 사용하면 클래스의 상위 클래스와 상호작용할 수 있습니다. 다음 형식 중 하나를 사용합니다:

```
super.<#member name#>
super[<#subscript index#>]
super.init(<#initializer arguments#>)
```

첫 번째 형식은 상위 클래스의 멤버에 접근하기 위해 사용합니다. 두 번째 형식은 상위 클래스의 서브스크립트 구현에 접근하기 위해 사용합니다. 세 번째 형식은 상위 클래스의 이니셜라이저에 접근하기 위해 사용합니다.

하위 클래스는 멤버, 서브스크립트, 이니셜라이저 구현 내에서 상위 클래스 표현식을 사용하여 상위 클래스의 구현을 활용할 수 있습니다.

Grammar of a superclass expression

superclass-expression → superclass-method-expression | superclass-subscript-expression | superclass-initializer-expression

superclass-method-expression → `super .` identifier superclass-subscript-expression → `super [` function-call-argument-list `]` superclass-initializer-expression → `super . init`

조건 표현식 (Conditional Expression)

*조건 표현식(conditional expression)*은 조건의 값을 기반으로 여러 주어진 값 중 하나로 평가됩니다. 다음의 형식을 가집니다:

```
if <#condition 1#> {
   <#expression used if condition 1 is true#>
} else if <#condition 2#> {
   <#expression used if condition 2 is true#>
} else {
   <#expression used if both conditions are false#>
}

switch <#expression#> {
case <#pattern 1#>:
    <#expression 1#>
case <#pattern 2#> where <#condition#>:
    <#expression 2#>
default:
    <#expression 3#>
}
```

조건 표현식은 `if` 구문이나 `switch` 구문과 같은 동작과 구문을 가지지만, 아래에서 설명하는 다른점을 가집니다.

조건 표현식은 다음 컨텍스트에서만 나타납니다:

- 변수에 할당된 값.
- 변수나 상수 선언에서 초기값.
- `throw` 표현식에서 발생시키는 오류.
- 함수, 클로저, 프로퍼티 getter의 반환된 값.
- 조건 표현식의 분기 안에서의 값.

조건 표현식의 분기는 조건에 상관없이 값을 생성하므로 완벽합니다. 이것은 각 `if` 분기는 적절한 `else` 분기가 필요합니다.

각 분기는 하나의 표현식만 포함되어야 하고, 이 표현식은 해당 조건이 참일 때 그 분기의 결과 값으로 사용하거나 `throw` 구문이나 반환하지 않는 함수 호출로 구성될 수 있습니다.

각 분기는 같은 타입의 값을 반환해야 합니다. 각 분기의 타입 검사는 독립적이기 때문에, 분기가 다른 종류의 리터럴을 포함하거나 분기의 값이 `nil`과 같을 때 값의 타입을 명시해야 합니다. 이 정보를 제공해야 할 때, 할당되는 결과 변수에 타입 명시를 추가하거나 분기의 값에 `as` 변환을 추가합니다.

```
let number: Double = if someCondition { 10 } else { 12.34 }
let number = if someCondition { 10 as Double } else { 12.34 }
```

결과 빌더(result builder) 내에서 조건 표현식은 변수나 상수의 초기값으로만 사용할 수 있습니다. 이것은 결과 빌더 내에서 `if`나 `switch`를 변수나 상수 선언이 아닌 위치에서 사용하면 그 코드는 분기 구문으로 이해하고 결과 빌더의 메서드 중 하나가 해당 코드를 변환한다는 의미입니다.

조건 표현식의 분기 중 하나가 오류를 발생하더라도 `try` 표현식에 조건 표현식을 작성하면 안됩니다.

Grammar of a conditional expression

conditional-expression → if-expression | switch-expression

if-expression → `if` condition-list `{` statement `}` if-expression-tail if-expression-tail → `else` if-expression
if-expression-tail → `else` `{` statement `}`

switch-expression → `switch` expression `{` switch-expression-cases `}` switch-expression-cases → switch-expression-case switch-expression-cases? switch-expression-case → case-label statement switch-expression-case → default-label statement

클로저 표현식 (Closure Expression)

*클로저 표현식(closure expression)*은 클로저를 생성하는 표현식이며 다른 프로그래밍 언어에서 *람다(lambda)*나 *익명 함수(anonymous function)*라고 알고 있습니다. 함수 선언과 같이 클로저는 구문을 포함하고 둘러싸인 범위에서 상수와 변수를 캡처합니다. 형식은 다음과 같습니다:

```
{ (<#parameters#>) -> <#return type#> in
   <#statements#>
}
```

*매개변수(parameters)*는 함수 선언 (Function Declaration)에서 설명했듯이 함수 선언의 매개변수 형식과 동일합니다.

클로저 표현식에 `throws`나 `async`를 명시하면 해당 클로저가 오류를 던지거나 비동기 작업을 수행하는 것을 나타냅니다.

```
{ (<#parameters#>) async throws -> <#return type#> in
   <#statements#>
}
```

클로저 본문에 `throws` 구문이나 `try` 표현식이 포함되어 있고 `do` 구문 내에 오류를 처리하지 않는 경우에 이 클로저는 오류를 던지는 것으로 간주됩니다. 클로저가 하나의 오류 타입만 던지면, 클로저는 해당 오류 타입을 던지는 것으로 간주됩니다; 그렇지 않으면 `any Error`를 던지는 것으로 간주됩니다. 마찬가지로 `await` 표현식이 포함되어 있다면, 비동기로 간주됩니다.

클로저를 보다 간결하게 작성할 수 있는 몇 가지 특별한 형식이 있습니다:

- 클로저는 매개변수의 타입, 반환 타입 또는 둘 다 생략할 수 있습니다. 매개변수 이름과 타입 모두 생략하는 경우 구문 전에 `in` 키워드도 생략합니다. 생략한 타입을 추론할 수 없으면 컴파일 시 오류가 발생합니다.
- 클로저는 매개변수 이름을 생략할 수 있습니다. 그러면 매개변수는 암시적으로 `$` 다음에 위치가 붙어 이름이 붙여집니다: `$0`, `$1`, `$2`, 등.
- 단일 표현식으로만 구성된 클로저는 해당 표현식의 값을 반환하는 것으로 간주됩니다. 이 표현식의 내용은 주변 표현식에 대한 타입 추론을 수행할 때도 고려됩니다.

다음의 클로저 표현식은 동일합니다:

```
myFunction { (x: Int, y: Int) -> Int in
    return x + y
}

myFunction { x, y in
    return x + y
}

myFunction { return $0 + $1 }

myFunction { $0 + $1 }
```

함수에 인자로 클로저를 전달하는 것에 대한 정보는 함수 호출 표현식 (Function Call Expression)을 참고바랍니다.

클로저 표현식은 함수 호출의 일부로 클로저를 즉시 사용할 때와 같이 변수나 상수에 저장하지 않고 사용할 수 있습니다. 위 코드에서 `myFunction`에 전달된 클로저 표현식은 즉시 사용되는 예시입니다. 결과적으로 클로저 표현식은 탈출(escaping)인지 비탈출(nonescaping)인지 여부는 주변의 컨텍스트에 의해 결정됩니다. 클로저 표현식은 즉시 호출되거나 비탈출 함수 인자로 전달되면 비탈출 클로저입니다. 그렇지 않으면 클로저 표현식은 탈출 클로저입니다.

탈출 클로저에 대한 자세한 내용은 탈출 클로저 (Escaping Closures)를 참고바랍니다.

캡처 목록 (Capture Lists)

기본적으로 클로저 표현식은 클로저를 감싼 범위의 상수와 변수를 강한 참조로 캡처합니다. *캡처 목록(capture list)*을 사용하여 클로저에서 값을 어떻게 캡처할지 명시적으로 제어할 수 있습니다

캡처 목록은 대괄호 안에 콤마로 구분하여 표현식을 나열해 작성하고 매개변수 목록 앞에 작성합니다. 캡처 목록을 사용하면 매개변수 이름, 매개변수 타입, 반환 타입을 생략하더라도 `in` 키워드를 작성해야 합니다. 캡처 목록의 마지막 표현식에 콤마를 포함할 수도 있습니다.

캡처 목록의 항목은 클로저가 생성될 때 초기화됩니다. 캡처 목록의 각 항목에 대해 상수는 외부 범위에서 이름이 같은 상수나 변수의 값으로 초기화 됩니다. 예를 들어 아래 코드에서 a는 캡처 목록에 포함되지만 b는 포함되지 않으므로 다른 동작을 보여줍니다.

```
var a = 0
var b = 0
let closure = { [a] in
    print(a, b)
}

a = 10
b = 10
closure()
// Prints "0 10".
```

주변 범위의 변수와 클로저 범위의 상수로 a라는 이름으로 두 가지가 있지만, b는 변수로 하나만 존재합니다. 내부 범위의 a는 클로저가 생성될 때 외부 범위의 a 값으로 초기화 되지만, 해당 값은 서로 연결되지 않습니다. 이것은 외부 범위의 a의 값이 변경되더라도 내부 범위의 a의 값은 영향을 받지 않고 클로저 내의 a가 변경되어도 클로저 외부 a의 값은 영향을 받지 않는다는 의미입니다. 반대로 외부 범위의 b는 하나의 변수로만 있으므로 클로저 내부나 외부에서 변경되면 두 위치 모두에 반영됩니다.

이 차이는 캡처된 변수의 타입이 참조 시맨틱(reference semantics)을 가질 경우에는 눈에 띄지 않습니다. 예를 들어 아래 코드에서 x라는 이름의 두 가지가 있는데 하나는 외부 범위의 변수와 또 다른 하나는 내부 범위의 상수이지만 둘 다 참조 의미이기 때문에 동일한 객체를 참조합니다.

```
class SimpleClass {
    var value: Int = 0
}
var x = SimpleClass()
var y = SimpleClass()
let closure = { [x] in
    print(x.value, y.value)
}

x.value = 10
y.value = 10
closure()
// Prints "10 10".
```

표현식의 값의 타입이 클래스라면 표현식의 값을 약한 참조나 미소유 참조로 캡처하기 위해 weak나 unowned로 캡처 목록에 표현식을 표시할 수 있습니다.

```
myFunction { print(self.title) }                    // implicit strong capture
myFunction { [self] in print(self.title) }          // explicit strong capture
myFunction { [weak self] in print(self!.title) }    // weak capture
myFunction { [unowned self] in print(self.title) }  // unowned capture
```

캡처 목록의 명명 값에 임의의 표현식을 바인딩할 수도 있습니다. 표현식은 클로저가 생성될 때 평가되고, 값은 캡처됩니다. 예를 들어:

```
// Weak capture of "self.parent" as "parent"
myFunction { [weak parent = self.parent] in print(parent!.title) }
```

클로저 표현식의 자세한 내용과 예시는 클로저 표현식 (Closure Expressions)을 참고바랍니다. 캡처 목록의 자세한 내용과 예시는 클로저의 강한 순환 참조 해결 (Resolving Strong Reference Cycles for Closures)을 참고바랍니다.

Grammar of a closure expression

closure-expression → { attributes? closure-signature? statements? }

closure-signature → capture-list? closure-parameter-clause `async`? throws-clause? function-result? `in`
closure-signature → capture-list `in`

closure-parameter-clause → `(` `)` | `(` closure-parameter-list `,`? `)` | identifier-list closure-parameter-list → closure-parameter | closure-parameter `,` closure-parameter-list closure-parameter → closure-parameter-name type-annotation? closure-parameter → closure-parameter-name type-annotation `.` `..` closure-parameter-name → identifier

capture-list → `[` capture-list-items `,`? `]` capture-list-items → capture-list-item | capture-list-item `,` capture-list-items capture-list-item → capture-specifier? identifier capture-list-item → capture-specifier? identifier = expression capture-list-item → capture-specifier? self-expression capture-specifier → `weak` | `unowned` | `unowned(safe)` | `unowned(unsafe)`

암시적 멤버 표현식 (Implicit Member Expression)

*암시적 멤버 표현식(implicit member expression)*은 열거형 케이스나 타입 메서드처럼 어떤 타입의 멤버에 접근할 때 타입 추론이 암시된 타입을 결정할 수 있는 문맥에서 사용할 수 있는 축약된 방식입니다. 다음과 같은 형식을 가집니다:

```
.<#member name#>
```

예를 들어:

```
var x = MyEnumeration.someValue
x = .anotherValue
```

타입이 옵셔널로 추론되는 경우에 암시적 멤버 표현식에서 옵셔널이 아닌 타입의 멤버를 사용할 수도 있습니다.

```
var someOptional: MyEnumeration? = .someValue
```

암시적 멤버 표현식 뒤에는 접미 연산자(postfix operator)나 접미 표현식 (Postfix Expressions)에 나열된 다른 접미 구문이 올 수 있습니다. 이것을 *연쇄 암시적 멤버 표현식(chained implicit member expression)*이라 합니다. 연쇄 접미 표현식(chained postfix expression)이 모두 동일한 타입을 갖는 것이 일반적이지만 반드시 그래야 하는 것은 아닙니다. 전체 연쇄 암시적 멤버 표현식이 문맥에서 요구하는 타입으로 변환 가능하면 됩니다. 구체적으로 암시적 타입이 옵셔널인 경우 옵셔널 타입이 아닌 값을 사용할 수 있고, 암시적 타입이 클래스 타입인 경우 해당 하위 클래스 중 하나의 값을 사용할 수 있습니다. 예를 들어:

```
class SomeClass {
    static var shared = SomeClass()
    static var sharedSubclass = SomeSubclass()
    var a = AnotherClass()
}
class SomeSubclass: SomeClass { }
class AnotherClass {
    static var s = SomeClass()
    func f() -> SomeClass { return AnotherClass.s }
}
let x: SomeClass = .shared.a.f()
let y: SomeClass? = .shared
let z: SomeClass = .sharedSubclass
```

위의 코드에서 x의 타입은 문맥에서 암시하는 타입과 정확히 일치하고, y의 타입은 `SomeClass`에서 `SomeClass?`로 변환 가능하며, z의 타입은 `SomeSubclass`에서 `SomeClass`로 변환 가능합니다.

Grammar of an implicit member expression

implicit-member-expression → **.** identifier implicit-member-expression → **.** identifier **.** postfix-expression

괄호 표현식 (Parenthesized Expression)

*괄호 표현식(parenthesized expression)*은 괄호로 둘러싸인 표현식으로 구성됩니다. 연산의 우선순위를 지정할 때, 괄호를 사용하여 표현식을 명시적으로 묶을 수 있습니다. 괄호로 묶이는 것은 표현식의 타입을 변경하지 않습니다 -- 예를 들어 `(1)`의 타입은 단순히 `Int`입니다.

Grammar of a parenthesized expression

parenthesized-expression → (expression)

튜플 표현식 (Tuple Expression)

*튜플 표현식(tuple expression)*은 콤마로 구분된 표현식의 목록을 괄호로 묶어서 구성됩니다. 각 표현식 앞에 식별자를 지정할 수 있으며, 콜론(`:`)으로 구분됩니다. 형식은 다음과 같습니다:

```
(<#identifier 1#>: <#expression 1#>, <#identifier 2#>: <#expression 2#>, <#...#>)
```

튜플 표현식의 각 식별자는 튜플 표현식의 범위 내에서 고유해야 합니다. 중첩된 튜플 표현식에서 동일한 수준의 중첩에서 식별자는 고유해야 합니다. 예를 들어 `(a: 10, a: 20)`은 레이블 `a`가 동일한 수준에서 두 번 나타나므로 유효하지 않습니다. 그러나 `(a: 10, b: (a: 1, x:2))`는 `a`가 두 번 나타나도 외부 튜플과 내부 튜플에서 나타나므로 유효합니다.

튜플 표현식은 표현식을 하나도 포함하지 않거나 두 개 이상의 표현식을 포함할 수 있습니다. 괄호 안의 단일 표현식은 괄호 표현식입니다.

참고

빈 튜플 표현식과 빈 튜플 타입 모두 Swift에서 `()`로 작성합니다. `Void`는 `()`의 타입 별칭이므로, 빈 튜플 타입을 작성하는데 사용할 수 있습니다. 그러나 모든 타입 별칭과 마찬가지로 `Void`는 타입이므로 - 빈 튜플 표현식으로 작성할 수 없습니다.

튜플의 마지막 표현식에 콤마를 포함할 수도 있습니다.

Grammar of a tuple expression

tuple-expression → `(` `)` | `(` tuple-element `,` tuple-element-list `,`? `)` tuple-element-list → tuple-element | tuple-element `,` tuple-element-list tuple-element → expression | identifier `:` expression

와일드카드 표현식 (Wildcard Expression)

*와일드카드 표현식(wildcard expression)*은 할당 중에 값을 명시적으로 무시하는데 사용됩니다. 예를 들어 다음의 할당에서 10은 x에 할당되고 20은 무시됩니다:

```
(x, _) = (10, 20)
// x is 10, and 20 is ignored
```

Grammar of a wildcard expression

wildcard-expression → `_`

매크로-확장 표현식 (Macro-Expansion Expression)

*매크로-확장 표현식(macro-expansion expression)*은 매크로 이름 다음에 매크로의 인자를 콤마로 구분하여 나열하고 괄호로 감싸서 구성합니다. 매크로는 컴파일 때 확장됩니다. 매크로-확장 표현식은 다음의 형식을 가집니다:

```
<#macro name#>(<#macro argument 1#>, <#macro argument 2#>)
```

매크로-확장 표현식은 매크로가 인자를 가지지 않는 경우에 매크로의 이름뒤에 오는 소괄호를 생략합니다.

매크로-확장 표현식은 매개변수의 기본값으로 사용할 수 있습니다. 함수나 메서드 매개변수의 기본값으로 사용할 때, 매크로는 함수를 정의한 위치가 아닌 호출된 소스 코드 위치를 기준으로 평가됩니다. 하지만 기본값이 매크로 뿐만 아니라 다른 코드도 포함하는 더 큰 표현식일 경우에 해당 매크로는 함수를 정의한 위치를 기준으로 평가됩니다.

```
func f(a: Int = #line, b: Int = (#line), c: Int = 100 + #line) {
    print(a, b, c)
}
f()  // Prints "4 1 101".
```

위 함수에서, `a`의 기본값은 단일 매크로 표현식이므로, `f(a:b:c:)`가 호출된 소스 코드 위치에서 해당 매크로가 평가됩니다. 반면에 `b`와 `c`의 값은 매크로를 포함하는 표현식으로 이루어져 있으며 - 이 매크로는 `f(a:b:c:)`가 정의된 소스 코드 위치에서 평가됩니다.

기본값으로 매크로를 사용할 때, 매크로는 확장없이 타입 검사를 하고 다음의 요구사항을 만족해야 합니다:

- 매크로의 접근 수준은 매크로를 사용하는 함수와 같거나 덜 제한적이어야 합니다.
- 매크로는 인자를 받지 않거나, 인자는 문자열 보간이 없는 리터럴이어야 합니다.
- 매크로의 반환 타입은 매개변수의 타입과 일치합니다.

독립 매크로를 호출하려면 매크로 표현식을 사용합니다. 첨부 매크로를 호출하려면, 속성 (Attributes)에서 설명한 커스텀 속성 구문을 사용합니다. 독립 매크로와 첨부 매크로 모두 아래와 같이 확장합니다:

1. Swift는 소스 코드를 분석해서 추상 구문 트리(AST)를 생성합니다.
2. 매크로 구현은 AST 노드를 입력으로 수신하고 해당 매크로에 필요한 변환을 수행합니다.
3. 매크로 구현을 통해 변환된 AST 노드는 본래 AST에 추가됩니다.

각 매크로의 확장은 독립적입니다. 그러나 성능 최적화로 Swift는 매크로를 구현하는 외부 프로세스를 시작하고 여러 매크로를 확장하기 위해 동일한 프로세스를 재사용합니다. 매크로를 구현할 때, 해당 코드는 이전에 확장된 매크로가 무엇인지 또는 현재 시간과 같이 외부 상태에 의존하면 안됩니다.

여러 역할을 가지는 중첩 매크로와 첨부 매크로에 대해 확장 프로세스는 반복됩니다. 중첩 매크로-확장 표현식은 외부에서 안으로 확장됩니다. 예를 들어, 아래 코드에서 `outerMacro(_:)`가 먼저 확장되고 `innerMacro(_:)` 호출이 확장되지 않은 상태로 `outerMacro(_:)`가 입력으로 받는 추상 구문 트리에 나타납니다.

```
#outerMacro(12, #innerMacro(34), "some text")
```

여러 역할을 가지는 첨부 매크로는 각 역할에 대해 한 번씩 확장합니다. 각 확장은 동일한 AST를 입력으로 받습니다. Swift는 생성된 모든 AST 노드를 수집하고 AST에서 적절한 위치에 배치하여 전체 확장을 형성합니다.

Swift에서 매크로의 개요는 매크로 (Macros)를 참고바랍니다.

Grammar of a macro-expansion expression

macro-expansion-expression → # identifier generic-argument-clause? function-call-argument-clause? trailing-closures?

키-경로 표현식 (Key-Path Expression)

*키-경로 표현식(key-path expression)*은 타입의 프로퍼티나 서브스크립트를 참조합니다. 키-경로 표현식은 키-값 관찰(key-value observing)과 같은 동적 프로그래밍 태스크(dynamic programming task)에서 사용합니다. 형식은 다음과 같습니다:

```
\<#type name#>.<#path#>
```

위 형식의 *타입 이름(type name)*은 `String`, `[Int]`, `Set<Int>`와 같은 제네릭 매개변수를 포함한 구체적인 타입 이름입니다.

위 형식의 *경로(path)*는 프로퍼티 이름, 서브스크립트, 옵셔널 체이닝 표현식, 강제 언래핑 표현식으로 구성됩니다. 이러한 각 키-경로 요소는 필요에 따라 어떤 순서로든 여러 번 반복될 수 있습니다.

컴파일 시에 키-경로 표현식은 `KeyPath` 클래스의 인스턴스로 대체됩니다.

키 경로를 사용하여 값에 접근하려면, 모든 타입에서 사용할 수 있는 `subscript(keyPath:)` 서브스크립트에 키 경로를 전달해야 합니다. 예를 들어:

```
struct SomeStructure {
    var someValue: Int
}

let s = SomeStructure(someValue: 12)
let pathToProperty = \SomeStructure.someValue

let value = s[keyPath: pathToProperty]
// value is 12
```

위 형식의 *타입 이름(type name)*은 타입 추론으로 암시된 타입을 결정할 수 있는 문맥에서는 생략할 수 있습니다. 다음 코드는 `\SomeClass.someProperty` 대신에 `\.someProperty`를 사용합니다:

```
class SomeClass: NSObject {
    @objc dynamic var someProperty: Int
    init(someProperty: Int) {
        self.someProperty = someProperty
    }
}

let c = SomeClass(someProperty: 10)
c.observe(\.someProperty) { object, change in
    // ...
}
```

위 형식의 *경로(path)*는 `self`를 참조해 아이덴티티 키 경로(identity key path)(`\.self`)를 생성할 수 있습니다. 아이덴티티 키 경로(identity key path)는 전체 인스턴스를 참조하므로 이를 사용하여 변수에 저장된 모든 데이터를 한 번에 접근하고 변경할 수 있습니다. 예를 들어:

```
var compoundValue = (a: 1, b: 2)
// Equivalent to compoundValue = (a: 10, b: 20)
compoundValue[keyPath: \.self] = (a: 10, b: 20)
```

위 형식의 *경로(path)*는 마침표로 구분된 여러 프로퍼티 이름을 포함할 수 있으므로, 어떤 프로퍼티의 값이 가진 또 다른 프로퍼티를 참조할 수 있습니다. 이 코드는 키 경로 표현식 `\OuterStructure.outer.someValue`을 사용하여 `OuterStructure` 타입의 `outer` 프로퍼티가 가진 `someValue` 프로퍼티에 접근합니다:

```
struct OuterStructure {
    var outer: SomeStructure
    init(someValue: Int) {
        self.outer = SomeStructure(someValue: someValue)
    }
}

let nested = OuterStructure(someValue: 24)
let nestedKeyPath = \OuterStructure.outer.someValue

let nestedValue = nested[keyPath: nestedKeyPath]
// nestedValue is 24
```

위 형식의 *경로(path)*는 대괄호를 사용하여 서브스크립트를 포함할 수 있으며, 이때 서브스크립트의 매개변수 타입은 `Hashable` 프로토콜을 준수해야 합니다. 이 예시는 배열의 두 번째 요소를 접근하기 위해 키 경로로 서브스크립트를 사용합니다:

```
let greetings = ["hello", "hola", "bonjour", "안녕"]
let myGreeting = greetings[keyPath: \[String].[1]]
// myGreeting is 'hola'
```

서브스크립트에서 사용된 값은 명명 값이나 리터럴일 수 있습니다. 값은 값 의미로 키 경로에 캡처됩니다. 다음 코드는 변수 `index`를 키-경로 표현식과 클로저 모두에서 사용하여 `greetings` 배열의 세 번째 요소에 접근합니다. `index`가 수정되면, 키-경로 표현식은 여전히 세 번째 요소를 참조하지만 클로저는 새로운 인덱스를 사용합니다.

```
var index = 2
let path = \[String].[index]
let fn: ([String]) -> String = { strings in strings[index] }

print(greetings[keyPath: path])
// Prints "bonjour".
print(fn(greetings))
// Prints "bonjour".

// Setting 'index' to a new value doesn't affect 'path'
index += 1
print(greetings[keyPath: path])
// Prints "bonjour".

// Because 'fn' closes over 'index', it uses the new value
print(fn(greetings))
// Prints "안녕".
```

위 형식의 *경로(path)*는 옵셔널 체이닝과 강제 언래핑을 사용할 수 있습니다. 이 코드는 키 경로에 옵셔널 체이닝을 사용하여 옵셔널 문자열의 프로퍼티에 접근합니다:

```
let firstGreeting: String? = greetings.first
print(firstGreeting?.count as Any)
// Prints "Optional(5)".

// Do the same thing using a key path.
let count = greetings[keyPath: \[String].first?.count]
print(count as Any)
// Prints "Optional(5)".
```

키 경로의 요소를 조합해 어떤 타입 안에 중첩된 값에도 접근할 수 있습니다. 다음 코드는 이러한 요소를 결합한 키-경로 표현식을 사용하여 배열의 딕셔너리에서 다른 값과 프로퍼티에 접근합니다.

```
let interestingNumbers = ["prime": [2, 3, 5, 7, 11, 13, 17],
                          "triangular": [1, 3, 6, 10, 15, 21, 28],
                          "hexagonal": [1, 6, 15, 28, 45, 66, 91]]
print(interestingNumbers[keyPath: \[String: [Int]].["prime"]] as Any)
// Prints "Optional([2, 3, 5, 7, 11, 13, 17])".
print(interestingNumbers[keyPath: \[String: [Int]].["prime"]![0]])
// Prints "2".
print(interestingNumbers[keyPath: \[String: [Int]].["hexagonal"]!.count])
// Prints "7".
print(interestingNumbers[keyPath: \[String: [Int]].["hexagonal"]!.count.bitWidth])
// Prints "64".
```

키 경로 표현식은 일반적으로 함수나 클로저를 제공하는 곳에서도 사용할 수 있습니다. 구체적으로 루트 타입이 `SomeType`이고 그 경로가 `Value` 타입의 값을 생성하는 키 경로 표현식은 `(SomeType) -> Value` 타입의 함수나 클로저 대신에 사용할 수 있습니다.

```
struct Task {
    var description: String
    var completed: Bool
}
var toDoList = [
    Task(description: "Practice ping-pong.", completed: false),
    Task(description: "Buy a pirate costume.", completed: true),
    Task(description: "Visit Boston in the Fall.", completed: false),
]

// Both approaches below are equivalent.
let descriptions = toDoList.filter(\.completed).map(\.description)
let descriptions2 = toDoList.filter { $0.completed }.map { $0.description }
```

키 경로 표현식의 부작용은 표현식이 평가되는 시점에만 평가됩니다. 예를 들어 키 경로 표현식의 서브스크립트 내에서 함수 호출하는 경우 함수는 표현식을 평가하는 과정에서 한 번만 호출되고 키 경로가 사용될 때마다 호출되지 않습니다.

```
func makeIndex() -> Int {
    print("Made an index")
    return 0
}
// The line below calls makeIndex().
let taskKeyPath = \[Task][makeIndex()]
// Prints "Made an index".

// Using taskKeyPath doesn't call makeIndex() again.
let someTask = toDoList[keyPath: taskKeyPath]
```

Objective-C API와 함께 상호작용하는 코드에서 키 경로를 사용하는 것에 대한 자세한 내용은 Swift에서 Objective-C 런타임 특성 사용 (Using Objective-C Runtime Features in Swift)을 참고바랍니다. 키-값 코딩과 키-값 관찰에 대한 자세한 내용은 키-값 코딩 프로그래밍 가이드 (Key-Value Coding Programming Guide)와 키-값 관찰 프로그래밍 가이드 (Key-Value Observing Programming Guide)를 참고바랍니다.

Grammar of a key-path expression

key-path-expression → `\` type? `.` key-path-components key-path-components → key-path-component | key-path-component `.` key-path-components key-path-component → identifier key-path-postfixes? | key-path-postfixes

key-path-postfixes → key-path-postfix key-path-postfixes? key-path-postfix → `?` | `!` | `self` | `[` function-call-argument-list `]`

Selector 표현식 (Selector Expression)

selector 표현식은 Objective-C에서 메서드나 프로퍼티의 getter나 setter를 참조하는 selector에 접근할 수 있습니다. 형식은 다음과 같습니다:

```
#selector(<#method name#>)
#selector(getter: <#property name#>)
#selector(setter: <#property name#>)
```

위 형식의 *메서드 이름(method name)*과 *프로퍼티 이름(property name)은 Objective-C 런타임에 사용할 수 있는 메서드나 프로퍼티여야 합니다. selector 표현식의 값은 `Selector` 타입의 인스턴스입니다. 예를 들어:

```
class SomeClass: NSObject {
    @objc let property: String

    @objc(doSomethingWithInt:)
    func doSomething(_ x: Int) { }

    init(property: String) {
        self.property = property
    }
}
let selectorForMethod = #selector(SomeClass.doSomething(_:))
let selectorForPropertyGetter = #selector(getter: SomeClass.property)
```

프로퍼티의 getter에 대한 selector를 생성할 때, *프로퍼티 이름(property name)*은 변수나 상수 프로퍼티를 참조할 수 있습니다. 반대로 프로퍼티의 setter에 대한 selector를 생성할 때, *프로퍼티 이름(property name)*은 변수 프로퍼티만 참조할 있습니다.

*메서드 이름(method name)*은 그룹화를 위해 소괄호를 포함할 수 있고, 이름은 같지만 시그니처가 다른 메서드를 구분하기 위해 `as` 연산자를 사용할 수 있습니다. 예를 들어:

```
extension SomeClass {
    @objc(doSomethingWithString:)
    func doSomething(_ x: String) { }
}
let anotherSelector = #selector(SomeClass.doSomething(_:) as (SomeClass) -> (String) -> Void)
```

selector는 런타임이 아닌 컴파일 시 생성되기 때문에, 컴파일러는 메서드나 프로퍼티가 존재하고 Objective-C 런타임에 노출되는지 확인할 수 있습니다.

참고

위 형식의 *메서드 이름(method name)*과 *프로퍼티 이름(property name)*은 표현식이지만 절대 평가되지 않습니다.

Objective-C API와 상호작용하는 Swift 코드에서 selector 사용에 대한 자세한 내용은 Swift에서 Objective-C 런타임 특성 사용 (Using Objective-C Runtime Features in Swift)을 참고바랍니다.

Grammar of a selector expression

selector-expression → `#selector` `(` expression `)` selector-expression → `#selector` `(` `getter:` expression `)`
selector-expression → `#selector` `(` `setter:` expression `)`

키-경로 문자열 표현식 (Key-Path String Expression)

키-경로 문자열 표현식(key-path string expression)은 Objective-C에서 프로퍼티 참조에 사용되는 문자열을 접근할 수 있게 해주며, 이 표현식을 사용하면 KVC(key-value coding)와 KVO(key-value observing) API에서 사용됩니다. 형식은 다음과 같습니다:

```
#keyPath(<#property name#>)
```

위 형식의 *프로퍼티 이름(property name)*은 Objective-C 런타임에서 사용할 수 있는 프로퍼티를 참조해야 합니다. 컴파일 시에 키-경로 문자열 표현식은 문자열 리터럴로 대체됩니다. 예를 들어:

```
class SomeClass: NSObject {
    @objc var someProperty: Int
    init(someProperty: Int) {
        self.someProperty = someProperty
    }
}

let c = SomeClass(someProperty: 12)
let keyPath = #keyPath(SomeClass.someProperty)

if let value = c.value(forKey: keyPath) {
```

```
    print(value)
}
// Prints "12".
```

클래스 내에 키-경로 문자열 표현식을 사용하면, 클래스 이름 없이 프로퍼티 이름 만으로 해당 클래스의 프로퍼티에 참조할 수 있습니다.

```
extension SomeClass {
    func getSomeKeyPath() -> String {
        return #keyPath(someProperty)
    }
}
print(keyPath == c.getSomeKeyPath())
// Prints "true".
```

키 경로 문자열은 런타임이 아닌 컴파일 시에 생성되기 때문에, 컴파일러는 프로퍼티가 존재하고 해당 프로퍼티가 Objective-C 런타임에 노출되는지 확인할 수 있습니다.

Objective-C API와 상호작용하는 Swift 코드에서 키 경로를 사용하는 것에 대한 자세한 내용은 Swift에서 Objective-C 런타임 특성 사용 (Using Objective-C Runtime Features in Swift)을 참고바랍니다. KVC와 KVO에 대한 자세한 내용은 키-값 코딩(KVC) 프로그래밍 가이드 (Key-Value Coding Programming Guide)와 키-값 관찰(KVO) 프로그래밍 가이드 (Key-Value Observing Programming Guide)를 참고바랍니다.

참고

위 형식의 *프로퍼티 이름(property name)*은 표현식이지만 절대 평가되지 않습니다.

Grammar of a key-path string expression

key-path-string-expression → `#keyPath` `(` expression `)`

접미 표현식 (Postfix Expressions)

*접미 표현식(Postfix expressions)*은 표현식에 접미 연산자(postfix operator)나 기타 접미 구문(postfix syntax)을 적용하여 구성됩니다. 문법적으로 모든 기본 표현식(primary expression)은 접미 표현식 입니다.

이러한 연산자의 동작에 대한 자세한 내용은 기본 연산자 (Basic Operators)와 고급 연산자 (Advanced Operators)를 참고바랍니다.

Swift 표준 라이브러리에 의해 제공되는 연산자에 대한 자세한 내용은 연산자 선언 (Operator Declarations)을 참고바랍니다.

Grammar of a postfix expression

postfix-expression → primary-expression postfix-expression → postfix-expression postfix-operator
postfix-expression → function-call-expression postfix-expression → initializer-expression postfix-expression → explicit-member-expression postfix-expression → postfix-self-expression postfix-expression → subscript-expression postfix-expression → forced-value-expression postfix-expression → optional-chaining-expression

함수 호출 표현식 (Function Call Expression)

*함수 호출 표현식(function call expression)*은 함수 이름 다음에 소괄호 안에 콤마로 구분된 인자 목록으로 구성됩니다. 함수 호출 표현식 형식은 다음과 같습니다:

```
<#function name#>(<#argument value 1#>, <#argument value 2#>)
```

위 형식의 *함수 이름(function name)*은 함수 타입 값을 나타내는 표현식이 될 수 있습니다.

함수 정의가 매개변수에 대한 이름을 포함하는 경우, 함수 호출 시 그 이름을 반드시 포함해야 하고 인자값 전에 콜론(`:`)을 붙여 작성합니다. 이러한 종류의 함수 호출 표현식 형식은 다음과 같습니다:

```
<#function name#>(<#argument name 1#>: <#argument value 1#>, <#argument name 2#>: <#argument v
  alue 2#>)
```

괄호 안에 마지막 인자에 콤마를 포함할 수도 있습니다.

함수 호출 표현식은 닫는 소괄호 바로 다음에 클로저 표현식을 작성해 후행 클로저(trailing closures)를 포함할 수 있습니다. 이 후행 클로저는 괄호로 감싼 인자 목록 뒤에 클로저를 작성해 함수의 인자로 이해됩니다. 첫 번째 클로저 표현식은 레이블이 없습니다; 그 다음 모든 추가 클로저 표현식은 인자 레이블이 앞에 옵니다. 아래 예시는 함수 호출의 예시이며, 후행 클로저 구문의 사용 여부와 상관없이 동일합니다:

```
// someFunction takes an integer and a closure as its arguments
someFunction(x: x, f: { $0 == 13 })
someFunction(x: x) { $0 == 13 }

// anotherFunction takes an integer and two closures as its arguments
anotherFunction(x: x, f: { $0 == 13 }, g: { print(99) })
anotherFunction(x: x) { $0 == 13 } g: { print(99) }
```

후행 클로저만 함수의 인자인 경우 소괄호를 생략할 수 있습니다.

```
// someMethod takes a closure as its only argument
myData.someMethod() { $0 == 13 }
myData.someMethod { $0 == 13 }
```

인자에 후행 클로저를 포함하려면 컴파일러는 다음과 같이 왼쪽에서 오른쪽으로 함수의 매개변수를 검사합니다:

후행 클로저	매개변수	동작
레이블 있음	레이블 있음	레이블이 동일하면 클로저는 매개변수와 일치됩니다; 그렇지 않으면 매개변수를 건너뜁니다.
레이블 있음	레이블 없음	매개변수를 건너뜁니다.
레이블 없음	레이블 존재 여부 무관	아래 정의된대로 매개변수가 구조적으로 함수 타입과 유사하면 클로저는 매개변수와 일치됩니다; 그렇지 않으면 매개변수를 건너뜁니다.

후행 클로저는 일치하는 매개변수에 대해 인자로 전달됩니다. 검색 과정 중에 건너뛴 매개변수에는 인자가 전달되지 않습니다 -- 예를 들어 기본 매개변수를 사용할 수 있습니다. 일치하는 항목을 찾은 후에 다음 후행 클로저와 다음 매개변수 검색을 계속 합니다. 일치 과정이 끝나면 모든 후행 클로저는 일치되어야 합니다.

매개변수가 in-out 매개변수가 아니고 매개변수가 다음 중 하나인 경우 함수 타입과 구조적으로 유사합니다:

- `(Bool) -> Int`와 같이 함수 타입인 매개변수
- `@autoclosure () -> ((Bool) -> Int)`와 같이 래핑된 표현식의 타입이 함수 타입인 autoclosure 매개변수
- `((Bool) -> Int)...`와 같이 배열 요소 타입이 함수 타입인 가변 매개변수
- `Optional<(Bool) -> Int>`와 같이 하나 이상의 옵셔널 계층에 래핑된 매개변수
- `(Optional<(Bool) -> Int>)...`와 같이 이러한 허용된 타입을 결합하는 매개변수

후행 클로저는 함수 타입과 구조적으로 유사하지만 함수는 아닌 파리미터와 일치하면 클로저는 필요에 따라 래핑됩니다. 예를 들어 매개변수의 타입이 옵셔널 타입인 경우 클로저는 자동으로 `Optional`로 래핑됩니다.

오른쪽에서 왼쪽으로 매칭을 수행하는 - Swift 5.3 이전 버전에서 코드를 쉽게 마이그레이션 하기위해 - 컴파일러는 왼쪽에서 오른쪽과 오른쪽에서 왼쪽으로 모두 확인합니다. 검색 방향이 다른 결과를 가져오면 오래된 매칭 방법인 오른쪽에서 왼쪽 순서가 사용되고 컴파일러는 경고를 발생시킵니다. Swift의 향후 버전은 항상 왼쪽에서 오른쪽 순서가 사용될 것입니다.

```
typealias Callback = (Int) -> Int
func someFunction(firstClosure: Callback? = nil,
                  secondClosure: Callback? = nil) {
    let first = firstClosure?(10)
    let second = secondClosure?(20)
    print(first ?? "–", second ?? "–")
}

someFunction()  // Prints "– –".
someFunction { return $0 + 100 }  // Ambiguous
someFunction { return $0 } secondClosure: { return $0 }  // Prints "10 20".
```

위의 예시에서 “Ambiguous”로 표시된 함수 호출은 “- 120”을 출력하고 Swift 5.3에서 컴파일러는 경고가 발생합니다. Swift 향후 버전에서는 “110 -“이 출력됩니다.

클래스, 구조체, 열거형 타입은 특별한 이름의 메서드 (Methods with Special Names)에서 설명한대로 여러 메서드 중 하나로 선언하여 함수 호출 구문에 대한 편의 문법(syntactic sugar)을 활성화 할 수 있습니다.

포인터 타입으로 암시적 변환 (Implicit Conversion to a Pointer Type)

함수 호출 표현식에서 인자와 매개변수가 다른 타입을 갖는 경우 컴파일러는 다음 목록의 암시적 변환 중 하나를 적용하여 해당 타입을 일치시키려고 합니다:

- `inout SomeType`은 `UnsafePointer<SomeType>`이나 `UnsafeMutablePointer<SomeType>`이 될 수 있습니다.
- `inout Array<SomeType>`은 `UnsafePointer<SomeType>`이나 `UnsafeMutablePointer<SomeType>`이 될 수 있습니다.
- `Array<SomeType>`은 `UnsafePointer<SomeType>`이 될 수 있습니다.

- `String`은 `UnsafePointer<CChar>`가 될 수 있습니다.

다음 두 가지 함수 호출은 동일합니다:

```
func unsafeFunction(pointer: UnsafePointer<Int>) {
    // ...
}
var myNumber = 1234

unsafeFunction(pointer: &myNumber)
withUnsafePointer(to: myNumber) { unsafeFunction(pointer: $0) }
```

이러한 암시적 변환에 의해 생성된 포인터는 함수 호출 동안에만 유효합니다. 정의되지 않은 동작을 방지하려면, 함수 호출이 끝난 후에 코드가 포인터를 유지하지 않도록 해야합니다.

> **참고**
>
> 암시적으로 배열을 안전하지 않은 포인터(unsafe pointer)로 변환할 때, Swift는 필요에 따라 배열을 변환하거나 복사하여 배열의 저장소가 연속되도록 합니다. 예를 들어 저장소에 대한 API 계약을 작성하지 않은 `NSArray`의 하위 클래스를 `Array`로 브리지했을 때도 이 구문을 사용할 수 있습니다. 이런 암시적 변환 작업을 수행할 필요 없이 배열의 저장소가 이미 연속됨을 보장하고 싶다면, `Array` 대신 `ContiguousArray`를 사용해야 합니다.

`withUnsafePointer(to:)`와 같이 명시적 함수 대신 &을 사용하는 것은 함수가 여러 포인터 인자를 가지는 저수준 C 함수(low-level C functions)를 더 읽기 쉽게 호출할 수 있습니다. 그러나 다른 Swift 코드에서 함수를 호출할 때 &을 사용하는 대신에 안전하지 않은 API를 명시적으로 사용하는 것이 좋습니다.

> **Grammar of a function call expression**
>
> function-call-expression → postfix-expression function-call-argument-clause function-call-expression → postfix-expression function-call-argument-clause? trailing-closures
>
> function-call-argument-clause → **(** **)** | **(** function-call-argument-list **,**? **)** function-call-argument-list → function-call-argument | function-call-argument **,** function-call-argument-list function-call-argument → expression | identifier **:** expression function-call-argument → operator | identifier **:** operator
>
> trailing-closures → closure-expression labeled-trailing-closures? labeled-trailing-closures → labeled-trailing-closure labeled-trailing-closures? labeled-trailing-closure → identifier **:** closure-expression

이니셜라이저 표현식 (Initializer Expression)

*이니셜라이저 표현식(initializer expression)*은 타입의 이니셜라이저에 접근하는 것을 제공합니다. 형식은 다음과 같습니다:

```
<#expression#>.init(<#initializer arguments#>)
```

이니셜라이저 표현식은 함수 호출 표현식 내에서 사용해 새로운 인스턴스를 초기화합니다. 또한 상위 클래스의 이니셜라이저에 위임할 때 사용할 수도 있습니다.

```
class SomeSubClass: SomeSuperClass {
    override init() {
        // subclass initialization goes here
        super.init()
    }
}
```

함수와 같이 이니셜라이저는 값으로 사용할 수 있습니다. 예를 들어:

```
// Type annotation is required because String has multiple initializers.
let initializer: (Int) -> String = String.init
let oneTwoThree = [1, 2, 3].map(initializer).reduce("", +)
print(oneTwoThree)
// Prints "123".
```

이름으로 타입을 지정하면 이니셜라이저 표현식 사용없이 타입의 이니셜라이저에 접근할 수 있습니다. 다른 모든 상황에서는 이니셜라이저 표현식을 사용해야 합니다.

```
let s1 = SomeType.init(data: 3)  // Valid
let s2 = SomeType(data: 1)       // Also valid

let s3 = type(of: someValue).init(data: 7)  // Valid
let s4 = type(of: someValue)(data: 5)       // Error
```

Grammar of an initializer expression

initializer-expression → postfix-expression `.` `init` initializer-expression → postfix-expression `.` `init` `(` argument-names `)`

명시적 멤버 표현식 (Explicit Member Expression)

*명시적 멤버 표현식(explicit member expression)*은 명명 타입, 튜플, 모듈의 멤버에 접근을 허용합니다. 이것은 점(`.`)으로 멤버의 이름과 식별자를 연결하여 구성됩니다.

```
<#expression#>.<#member name#>
```

명명 타입의 멤버는 타입의 선언이나 확장을 통해 정의됩니다. 예를 들어:

```
class SomeClass {
    var someProperty = 42
}
let c = SomeClass()
let y = c.someProperty  // Member access
```

튜플의 멤버는 0을 시작으로 순서대로 정수를 사용하여 암시적으로 명명됩니다. 예를 들어:

```
var t = (10, 20, 30)
t.0 = t.1
// Now t is (20, 20, 30)
```

모듈의 멤버는 해당 모듈의 최상위 선언입니다.

`dynamicMemberLookup` 속성으로 선언된 타입은 속성 (Attributes)에서 설명한대로 런타임 시 조회되는 멤버가 포함됩니다.

이름은 동일하지만 매개변수 이름만 다른 메서드나 이니셜라이저를 구분하기 위해 인자 이름을 소괄호 안에 포함하고 각 인자 이름 뒤에 콜론(`:`)을 붙입니다. 인자에 이름이 없는 경우 언더바(`_`)를 작성합니다. 오버로드된 메서드를 구별하려면 타입 주석을 사용합니다. 예를 들어:

```
class SomeClass {
    func someMethod(x: Int, y: Int) {}
    func someMethod(x: Int, z: Int) {}
    func overloadedMethod(x: Int, y: Int) {}
    func overloadedMethod(x: Int, y: Bool) {}
}
let instance = SomeClass()

let a = instance.someMethod              // Ambiguous
let b = instance.someMethod(x:y:)        // Unambiguous

let d = instance.overloadedMethod        // Ambiguous
let d = instance.overloadedMethod(x:y:)  // Still ambiguous
let d: (Int, Bool) -> Void  = instance.overloadedMethod(x:y:)  // Unambiguous
```

줄 시작에 마침표가 나타나면 암시적 멤버 표현식이 아닌 명시적 멤버 표현식의 부분으로 이해합니다. 예를 들어 다음 목록은 여러 줄로 분할된 메서드 호출 체인을 보여줍니다:

```
let x = [10, 3, 20, 15, 4]
    .sorted()
    .filter { $0 > 5 }
    .map { $0 * 100 }
```

이 여러 줄로 연결된 구문은 컴파일러 제어문과 결합하여 각 메서드가 호출되는 시기를 제어할 수 있습니다. 예를 들어 다음 코드는 iOS에서 다른 필터링 규칙을 사용합니다:

```
let numbers = [10, 20, 33, 43, 50]
#if os(iOS)
    .filter { $0 < 40 }
#else
    .filter { $0 > 25 }
#endif
```

`#if`, `#endif`와 다른 컴파일 지시어 사이의 조건부 컴파일 블록은 접미 표현식을 구성하기 위해 암시적 멤버 표현식 뒤에 접미사를 포함하지 않거나 많은 접미사를 포함할 수 있습니다. 다른 조건부 컴파일러 블록이나 이러한 표현식과 블록을 조합하여 포함할 수도 있습니다.

이 문법은 최상위 코드뿐만 아니라 명시적 멤버 표현식을 작성할 수 있는 모든 곳에서 사용할 수 있습니다.

조건부 컴파일러 블록에서 `#if` 컴파일러 지시어의 분기는 하나 이상의 표현식이 포함되어야 합니다. 다른 분기는 비어있을 수 있습니다.

Grammar of an explicit member expression

explicit-member-expression → postfix-expression `.` decimal-digits explicit-member-expression → postfix-expression `.` identifier generic-argument-clause? explicit-member-expression → postfix-expression `.` identifier `(` argument-names `)` explicit-member-expression → postfix-expression conditional-compilation-block

argument-names → argument-name argument-names? argument-name → identifier `:`

접미 Self 표현식 (Postfix Self Expression)

접미 `self` 표현식(postfix `self` expression)은 표현식이나 타입의 이름 뒤에 `.self`를 붙여 구성됩니다. 형식은 다음과 같습니다:

```
<#expression#>.self
<#type#>.self
```

첫 번째 형식은 위 형식의 *표현식(expression)*의 값으로 평가됩니다. 예를 들어 `x.self`는 x로 평가됩니다.

두 번째 형식은 위 형식의 *타입(type)*의 값으로 평가됩니다. 값으로 타입을 접근하려면 이 형식을 사용합니다. 예를 들어 `SomeClass.self`는 `SomeClass` 타입 자체로 평가되므로 타입을 인자로 요구하는 함수나 메서드에 전달할 수 있습니다.

Grammar of a postfix self expression

postfix-self-expression → postfix-expression `.` `self`

서브스크립트 표현식 (Subscript Expression)

*서브스크립트 표현식(subscript expression)*은 해당 서브스크립트 선언의 getter와 setter를 사용하여 서브스크립트 접근을 제공합니다. 형식은 다음과 같습니다:

```
<#expression#>[<#index expressions#>]
```

서브스크립트 표현식의 값을 평가하기 위해 위 형식의 *표현식(expression)*의 타입에 대한 서브스크립트 getter는 서브스크립트 매개변수로 위 형식의 *인덱스 표현식(index expressions)*을 전달하여 호출합니다. 값을 설정하기 위해서는 서브스크립트 setter는 동일한 방식으로 호출합니다.

서브스크립트 선언에 대한 자세한 내용은 프로토콜 서브스크립트 선언 (Protocol Subscript Declaration)을 참고바랍니다.

Grammar of a subscript expression

subscript-expression → postfix-expression `[` function-call-argument-list `]`

강제-값 표현식 (Forced-Value Expression)

*강제-값 표현식(forced-value expression)*은 `nil` 이 아니라고 확신하는 옵셔널 값을 언래핑합니다. 형식은 다음과 같습니다:

```
<#expression#>!
```

위 형식의 *표현식(expression)*의 값이 `nil`이 아니라면 옵셔널 값은 언래핑되고 옵셔널이 아닌 타입을 반환합니다. 그렇지 않으면 런타임 오류가 발생합니다.

강제-값 표현식의 언래핑된 값은 값 자체를 변경하거나 값의 멤버중 하나에 할당하여 수정할 수 있습니다. 예를 들어:

```
var x: Int? = 0
x! += 1
// x is now 1

var someDictionary = ["a": [1, 2, 3], "b": [10, 20]]
someDictionary["a"]![0] = 100
// someDictionary is now ["a": [100, 2, 3], "b": [10, 20]]
```

Grammar of a forced-value expression

forced-value-expression → postfix-expression `!`

옵셔널-체이닝 표현식 (Optional-Chaining Expression)

*옵셔널-체이닝 표현식(optional-chaining expression)*은 접미 표현식으로 옵셔널 값을 사용하여 간단한 구문을 제공합니다. 형식은 다음과 같습니다:

```
<#expression#>?
```

접미 `?` 연산자는 표현식의 값 변경없이 표현식에서 옵셔널-체이닝 표현식을 만듭니다.

옵셔널-체이닝 표현식은 접미 표현식 내에 나타나야 하고 접미 표현식이 특별한 방법으로 평가되도록 합니다. 옵셔널-체이닝 표현식의 값이 `nil`이면, 접미 표현식의 다른 모든 동작은 무시되고 전체 접미 표현식은 `nil`로 평가됩니다. 옵셔널-체이닝 표현식의 값이 `nil`이 아니면, 옵셔널-체이닝 표현식의 값은 언래핑되고 나머지 접미 표현식을 평가하기 위해 사용됩니다. 두 경우 모두 접미 표현식의 값은 여전히 옵셔널 타입입니다.

옵셔널-체이닝 표현식이 포함된 접미 표현식이 다른 접미 표현식 내에 중첩되었다면 가장 바깥쪽 표현식만 옵셔널 타입을 반환합니다. 아래의 예시에서 `c`가 `nil`이 아니면 해당 값은 언래핑되고 `.property`를 평가하기 위해 사용되고 그 값을 다시 `.performAction()`을 평가하기 위해 사용됩니다. 전체 표현식 `c?.property.performAction()`은 옵셔널 타입의 값을 가집니다.

```
var c: SomeClass?
var result: Bool? = c?.property.performAction()
```

다음의 예시는 옵셔널 체이닝 사용없이 위의 예시의 동작을 보여줍니다.

```
var result: Bool?
if let unwrappedC = c {
    result = unwrappedC.property.performAction()
}
```

옵셔널-체이닝 표현식의 언래핑된 값은 값 자체를 변경하거나 값의 멤버중 하나에 할당을 통해 변경할 수 있습니다. 옵셔널-체이닝 표현식의 값이 `nil`이라면 할당 연산자의 오른쪽의 표현식은 평가되지 않습니다. 예를 들어:

```
func someFunctionWithSideEffects() -> Int {
    return 42  // No actual side effects.
}
var someDictionary = ["a": [1, 2, 3], "b": [10, 20]]

someDictionary["not here"]?[0] = someFunctionWithSideEffects()
// someFunctionWithSideEffects isn't evaluated
// someDictionary is still ["a": [1, 2, 3], "b": [10, 20]]

someDictionary["a"]?[0] = someFunctionWithSideEffects()
// someFunctionWithSideEffects is evaluated and returns 42
// someDictionary is now ["a": [42, 2, 3], "b": [10, 20]]
```

Grammar of an optional-chaining expression

optional-chaining-expression → postfix-expression ?

언어 참조

구문 (Statements)

표현식을 그룹화하고 실행 흐름을 제어합니다.

Swift에서 세 종류의 명령어가 있습니다: 단순 구문(simple statements), 컴파일러 제어 구문(compiler control statements), 제어 흐름 구문(control flow statements)입니다. 단순 구문은 가장 일반적이고 표현식이나 선언으로 구성되어 있습니다. 컴파일러 제어 구문은 프로그램이 컴파일러의 동작 일부를 변경할 수 있고 조건부 컴파일 블록과 라인 제어 구문이 포함됩니다.

제어 흐름 구문은 프로그램에서 실행 흐름을 제어하기 위해 사용됩니다. Swift는 다양한 제어 흐름 구문을 포함하고 있으며, 루프 구문(loop statements), 분기 구문(branch statements), 제어 전송 구문(control transfer statements)이 있습니다. 루프 구문은 코드 블록을 반복적으로 실행하고, 분기 구문은 특정 조건이 일치 할 때만 코드 블록을 실행하고, 제어 전송 구문은 코드의 실행 순서를 변경하는 방법을 제공합니다. 또한 Swift는 새로운 범위를 만들고 오류를 잡고 처리하는 `do` 구문과 현재 범위가 종료되기 직전에 정리 동작을 실행하는 `defer` 구문을 제공합니다.

세미콜론(`;`)은 모든 구문에 선택적으로 나타날 수 있고 같은 줄에 여러 구문을 구분하기 위해 사용할 수 있습니다.

Grammar of a statement

statement → expression `;`? statement → declaration `;`? statement → loop-statement `;`? statement → branch-statement `;`? statement → labeled-statement `;`? statement → control-transfer-statement `;`? statement → defer-statement `;`? statement → do-statement `;`? statement → compiler-control-statement statements → statement statements?

루프 구문 (Loop Statements)

루프 구문(Loop statement)은 루프에 명시한 조건에 따라 특정 코드 블록을 반복적으로 실행되도록 합니다. Swift는 세 가지 루프 구문을 가지고 있습니다: `for-in` 구문, `while` 구문, `repeat-while` 구문입니다.

루프 구문에서 제어 흐름은 `break` 구문과 `continue` 구문에 의해 변경될 수 있고 아래에 중단 구문 (Break Statement)과 계속 구문 (Continue Statement)에 설명되어 있습니다.

Grammar of a loop statement

loop-statement → for-in-statement loop-statement → while-statement loop-statement → repeat-while-statement

For-In 구문 (For-In Statement)

`for-in` 구문은 `Sequence` 프로토콜을 준수하는 컬렉션이나 타입의 각 항목에 대해 한 번씩 코드 블록을 실행합니다.

`for-in` 구문은 다음의 형식을 가집니다:

```
for <#item#> in <#collection#> {
   <#statements#>
}
```

컬렉션 표현식(collection expression)에 대해 `makeIterator()` 메서드가 호출되어 `IteratorProtocol` 프로토콜을 준수하는 타입인 이터레이터(iterator) 타입의 값을 반환합니다. 프로그램은 이터레이터의 `next()` 메서드를 호출하여 루프를 실행합니다. 값이 `nil`을 반환하지 않으면 위 형식의 항목(item) 패턴에 할당되고, 프로그램은 위 형식의 *구문(statements)*을 실행한 다음에 루프의 처음으로 돌아가 위 동작을 계속 실행합니다. 그렇지 않으면 프로그램은 할당이나 위 형식의 구문(statements) 실행을 수행하지 않고 `for-in` 구문 실행을 종료합니다.

Grammar of a for-in statement

for-in-statement → `for` `case`? pattern `in` expression where-clause? code-block

While 구문 (While Statement)

`while` 구문은 조건이 참으로 남아있는 한 반복적으로 코드 블록을 실행합니다.

`while` 구문은 다음의 형식을 가집니다:

```
while <#condition#> {
   <#statements#>
}
```

`while` 구문은 다음과 같이 실행됩니다:

1. 위 형식의 *조건(condition)*은 평가됩니다.

   `true`이면 2번이 계속 실행됩니다. `false`이면 프로그램은 `while` 구문 실행을 종료합니다.

2. 프로그램은 위 형식의 *구문(statements)*을 실행하고 다시 1번으로 돌아갑니다.

위 형식의 *조건(condition)*의 값이 위 형식의 *구문(statements)*이 실행되기 전에 평가되므로, `while` 구문의 위 형식의 *구문(statements)*은 한 번도 실행되지 않거나 여러 번 실행될 수 있습니다.

위 형식의 *조건(condition)*의 값은 `Bool` 타입이거나 `Bool`에 브리지된 타입이어야 합니다. 조건은 옵셔널 바인딩(Optional Binding)에서 설명한대로 옵셔널 바인딩 선언일 수도 있습니다.

Grammar of a while statement

while-statement → `while` condition-list code-block

condition-list → condition | condition `,` condition-list condition → expression | availability-condition | case-condition | optional-binding-condition

case-condition → `case` pattern initializer optional-binding-condition → `let` pattern initializer? | `var` pattern initializer?

Repeat-While 구문 (Repeat-While Statement)

`repeat-while` 구문은 조건이 참인 경우에 한하여 코드 블록을 한 번 이상 실행합니다.

`repeat-while` 구문은 다음의 형식을 가집니다:

```
repeat {
   <#statements#>
} while <#condition#>
```

`repeat-while` 구문은 다음과 같이 실행됩니다:

1. 프로그램은 위 형식의 *구문(statements)*을 실행하고, 2번이 계속 실행됩니다.
2. 위 형식의 *조건(condition)*이 평가됩니다.

   `true`이면 다시 1번으로 돌아갑니다. `false`이면 프로그램은 `repeat-while` 구문 실행을 종료합니다.

위 형식의 *구문(statements)*이 실행된 후에 위 형식의 *조건(condition)*의 값이 평가되기 때문에, `repeat-while` 구문에서 위 형식의 *구문(statements)*은 무조건 한 번은 실행됩니다.

위 형식의 *조건(condition)*의 값은 `Bool` 타입이거나 `Bool`로 브리지된 타입이어야 합니다.

Grammar of a repeat-while statement

repeat-while-statement → `repeat` code-block `while` expression

분기 구문 (Branch Statements)

분기 구문(Branch statement)은 프로그램이 하나 이상의 조건의 값에 따라 코드의 특정 부분을 실행하는 것을 허용합니다. 분기 구문에서 지정된 조건의 값은 프로그램 분기 방법 및 코드의 어떤 블록이 실행되는지 제어합니다. Swift는 세 가지 분기 구문을 가지고 있습니다: `if` 구문, `guard` 구문, `switch` 구문입니다.

`if` 구문이나 `switch` 구문의 제어 흐름은 `break` 구문에 의해 변경될 수 있고 아래 중단 구문 (Break Statement)에 설명되어 있습니다.

Grammar of a branch statement

branch-statement → if-statement branch-statement → guard-statement branch-statement → switch-statement

If 구문 (If Statement)

`if` 구문은 하나 이상의 조건 평가를 기반으로 코드 실행에 사용됩니다.

`if` 구문에는 두 가지 기본 형식이 있습니다. 각 형식은 여는 중괄호와 닫는 중괄호가 필요합니다.

첫 번째 형식은 조건이 참일 때만 코드를 실행하고 다음의 형식을 가지고 있습니다:

```
if <#condition#> {
   <#statements#>
}
```

`if` 구문의 두 번째 형식은 추가로 *else 절(else clause)*을 제공하고 (`else` 키워드 사용) 조건이 참일 때 코드의 한 부분을 실행하고 동일한 조건이 거짓일 때 코드의 다른 부분을 실행하는데 사용됩니다. 하나의 else 절이 있는 경우에 `if` 구문은 다음의 형식을 가집니다:

```
if <#condition#> {
   <#statements to execute if condition is true#>
} else {
   <#statements to execute if condition is false#>
}
```

`if` 구문의 else 절은 둘 이상의 조건을 검사할 수 있는 다른 `if` 구문을 포함할 수 있습니다. 이러한 방법으로 연결된 `if` 구문은 다음의 형식을 가집니다:

```
if <#condition 1#> {
   <#statements to execute if condition 1 is true#>
} else if <#condition 2#> {
   <#statements to execute if condition 2 is true#>
} else {
   <#statements to execute if both conditions are false#>
}
```

`if` 구문의 모든 조건 값은 `Bool` 타입이거나 `Bool`로 브리지된 타입이어야 합니다. 조건은 옵셔널 바인딩 (Optional Binding)에서 설명한대로 옵셔널 바인딩 선언일 수도 있습니다.

Grammar of an if statement

if-statement → `if` condition-list code-block else-clause? else-clause → `else` code-block | `else` if-statement

Guard 구문 (Guard Statement)

`guard` 구문은 하나 이상의 조건이 충족되지 않을 때 범위 밖으로 프로그램 제어를 전달하기 위해 사용됩니다.

`guard` 구문은 다음의 형식을 가집니다:

```
guard <#condition#> else {
   <#statements#>
}
```

`guard` 구문의 모든 조건의 값은 `Bool` 타입이거나 `Bool`로 브리지된 타입이어야 합니다. 조건은 옵셔널 바인딩 (Optional Binding)에서 설명한대로 옵셔널 바인딩 선언일 수도 있습니다.

`guard` 구문 조건에서 옵셔널 바인딩 선언으로 할당된 모든 상수나 변수 값은 guard 구문을 둘러싼 범위 내에서 사용할 수 있습니다.

`guard` 구문의 `else` 절은 필수이고 `Never` 반환 타입을 가진 함수를 호출하거나 다음 구문 중 하나를 사용하여 guard 구문을 둘러싼 범위 밖으로 프로그램 제어를 전달해야 합니다:

- `return`
- `break`
- `continue`

- `throw`

제어 전송 구문은 아래 제어 전송 구문 (Control Transfer Statements)에 설명되어 있습니다. `Never` 반환 타입이 가진 함수에 자세한 내용은 반환하지 않는 함수 (Functions that Never Return)를 참고바랍니다.

Grammar of a guard statement

guard-statement → `guard` condition-list `else` code-block

Switch 구문 (Switch Statement)

`switch` 구문은 제어 표현식의 값에 따라 특정 코드 블록을 실행합니다.

`switch` 구문은 다음의 형식을 가집니다:

```
switch <#control expression#> {
case <#pattern 1#>:
    <#statements#>
case <#pattern 2#> where <#condition#>:
    <#statements#>
case <#pattern 3#> where <#condition#>,
    <#pattern 4#> where <#condition#>:
    <#statements#>
default:
    <#statements#>
}
```

`switch` 구문에서 위 형식의 *제어 표현식(control expression)*은 평가된 다음에 각 케이스에 지정된 위 형식의 패턴과 비교합니다. 일치하는 항목이 있으면, 프로그램은 해당 케이스의 범위 내의 위 형식의 *구문(statements)*을 실행합니다. 각 케이스의 범위는 비어 있을 수 없습니다. 결과적으로 각 케이스 레이블의 콜론(`:`) 다음에 적어도 하나의 구문이 포함되어야 합니다. 일치하는 케이스의 본문에서 코드를 실행하지 않으려면 `break` 구문을 사용해야 합니다.

코드가 분기할 수 있는 표현식의 값은 매우 유연합니다. 예를 들어 정수와 문자와 같은 스칼라 타입의 값을 제외하고 코드는 부동소수점 숫자, 문자열, 튜플, 커스텀 클래스의 인스턴스, 옵셔널을 포함한 모든 타입의 값으로 분기할 수 있습니다. *제어 표현식(control expression)*의 값은 열거형의 케이스의 값과 일치하는지 확인할 수도 있고, 지정된 값 범위에 포함되는지 확인할 수도 있습니다. `switch` 구문에서 값의 이러한 여러 가지 타입을 어떻게 사용하는지에 대한 예시는 제어 흐름 (Control Flow)의 Switch를 참고바랍니다.

`switch` 케이스는 각 패턴 후에 선택적으로 `where` 절을 포함할 수 있습니다. *where 절(where clause)*은 `where` 키워드 다음에 표현식을 붙여 작성하고 케이스의 패턴이 *제어 표현식(control expression)*에 일치하는지 확인하기 전에 추가 조건을 제공하는데 사용합니다. `where` 절이 있는 경우 연관된 케이스 내의 *구문(statements)*은 *제어 표현식(control expression)*의 값이 케이스의 패턴 중 하나와 일치하고 `where` 절의 표현식이 `true`로 평가될 때만 실행됩니다. 예를 들어 아래 예시에서 *제어 표현식(control expression)*은 `(1, 1)`과 같이 동일한 값의 두 요소가 포함된 튜플인 경우에만 케이스가 일치합니다.

```
case let (x, y) where x == y:
```

위의 예시에서 볼 수 있듯이, 케이스의 패턴은 `let` 키워드를 사용하여 상수에 바인딩할 수도 있습니다(`var` 키워드를 사용하여 변수에 바인딩할 수도 있습니다). 이 상수(또는 변수)는 해당 `where` 절과 케이스 범위 내의 나머지 코드에

서 참조할 수 있습니다. 케이스에 제어 표현식과 일치하는 여러 패턴을 포함하는 경우 모든 패턴에는 동일한 상수나 변수 바인딩이 포함되어야 하고 바인딩된 변수나 상수는 케이스의 모든 패턴에서 동일한 타입을 가져야 합니다.

`switch` 구문은 `default` 키워드로 기본 케이스를 포함할 수도 있습니다. 기본 케이스 내의 코드는 제어 표현식이 일치하는 케이스가 없는 경우에만 실행됩니다. `switch` 구문은 오직 하나의 기본 케이스만 포함하고, `switch` 구문의 마지막에 위치해야 합니다.

패턴-일치 연산(pattern-matching operations)의 실제 실행 순서, 특히 케이스 패턴의 평가 순서는 지정되지 않았지만 `switch` 구문의 패턴 일치는 소스 코드가 나타나는 순서대로 평가가 수행되도록 동작합니다. 결과적으로 여러 케이스에 동일한 값으로 평가되는 패턴이 포함되어 제어 표현식의 값과 일치할 수 있는 경우 프로그램은 소스 순서에서 첫 번째로 일치하는 케이스 내의 코드만 실행합니다.

Switch 구문은 완벽해야 함 (Switch Statements Must Be Exhaustive)

Swift에서 제어 표현식의 타입에서 가능한 모든 값은 케이스 패턴 중 적어도 하나의 값과 일치해야 합니다. 이것이 가능하지 않은 경우 (예를 들어 제어 표현식의 타입이 `Int`인 경우), 요구사항을 충족하기 위해 기본 케이스를 포함할 수 있습니다.

향후 열거형 케이스 전환 (Switching Over Future Enumeration Cases)

*비고정 열거형(nonfrozen enumeration)*은 앱을 컴파일하고 출시한 후에도 나중에 새로운 열거형 케이스를 얻을 수 있는 특별한 종류의 열거형입니다. 비고정 열거형을 전환하려면 추가 고려사항이 있습니다. 라이브러리 작성자가 비고정으로 열거형을 표시하면, 새로운 열거형 케이스를 추가할 권한이 있으며 해당 열거형과 상호작용하는 모든 코드는 다시 컴파일되지 않고 향후 케이스를 처리할 수 있어야 합니다. 라이브러리 진화 모드에서 컴파일된 코드, Swift 표준 라이브러리의 코드, Apple 프레임워크용 Swift 오버레이, C 그리고 Objective-C 코드는 비고정 열거형을 선언할 수 있습니다. 고정과 비고정 열거형에 대한 자세한 내용은 frozen을 참고바랍니다.

비고정 열거형 값을 변경할 때, 열거형의 모든 케이스에 해당 switch 케이스가 있더라도 기본 케이스를 포함해야 합니다. 기본 케이스에 `@unknown` 속성을 적용하여 기본 케이스가 향후에 추가될 열거형 케이스에만 일치하도록 나타낼 수 있습니다. Swift는 컴파일 시에 알고있는 열거형 케이스와 일치하는 기본 케이스가 있다면 경고를 발생시킵니다. 향후 이 경고는 라이브러리 작성자가 해당 switch 케이스가 없는 열거형에 새로운 케이스를 추가했음을 알려줍니다.

다음 예시는 Swift 표준 라이브러리의 `Mirror.AncestorRepresentation` 열거형의 세 가지 기존 케이스를 모두 전환합니다. 향후에 추가 케이스를 추가하면, 컴파일러는 새로운 케이스를 고려하기 위해 switch 구문을 업데이트 해야 한다고 경고를 생성합니다.

```
let representation: Mirror.AncestorRepresentation = .generated
switch representation {
case .customized:
    print("Use the nearest ancestor's implementation.")
case .generated:
    print("Generate a default mirror for all ancestor classes.")
case .suppressed:
    print("Suppress the representation of all ancestor classes.")
@unknown default:
    print("Use a representation that was unknown when this code was compiled.")
}
// Prints "Generate a default mirror for all ancestor classes.".
```

실행은 암시적으로 케이스를 통과하지 않음 (Execution Does Not Fall Through Cases Implicitly)

일치한 케이스 내에 코드 실행이 완료되면, 프로그램은 `switch` 구문을 종료합니다. 프로그램 실행은 다음 케이스나 기본 케이스로 계속되거나 "이동(fall through)"되지 않습니다. 이 말은 한 케이스에서 다음 케이스로 계속해서 실행을 하려면, 계속해서 실행되기 원하는 케이스에 `fallthrough` 키워드로 구성된 `fallthrough` 구문을 명시적으로 포함해야 합니다. `fallthrough` 구문에 대한 자세한 내용은 아래의 Fallthrough 구문 (Fallthrough Statement)을 참고바랍니다.

Grammar of a switch statement

switch-statement → `switch` expression `{` switch-cases? `}` switch-cases → switch-case switch-cases?
switch-case → case-label statements switch-case → default-label statements switch-case → conditional-switch-case

case-label → attributes? `case` case-item-list `:` case-item-list → pattern where-clause? | pattern where-clause? `,` case-item-list default-label → attributes? `default :`

where-clause → `where` where-expression where-expression → expression

conditional-switch-case → switch-if-directive-clause switch-elseif-directive-clauses? switch-else-directive-clause? endif-directive switch-if-directive-clause → if-directive compilation-condition switch-cases? switch-elseif-directive-clauses → elseif-directive-clause switch-elseif-directive-clauses?
switch-elseif-directive-clause → elseif-directive compilation-condition switch-cases?
switch-else-directive-clause → else-directive switch-cases?

레이블 구문 (Labeled Statement)

접두사로 루프 구문, `if` 구문, `switch` 구문, `do` 구문 앞에 *구문 레이블(statement label)*을 붙일 수 있으며, 이것은 레이블 이름 뒤에 콜론(`:`)을 붙여 구성합니다. 아래 중단 구문 (Break Statement)과 계속 구문 (Continue Statement)에서 설명한대로 `break`와 `continue` 구문과 함께 구문 레이블을 사용하여 루프 구문이나 `switch` 구문에서 제어 흐름을 어떻게 변경할지 명시적으로 나타낼 수 있습니다.

레이블 구문의 범위는 구문 레이블 다음에 오는 전체 구문입니다. 레이블 구문을 중첩할 수 있지만 각 구문 레이블의 이름은 고유해야 합니다.

구문 레이블을 어떻게 사용하는지 자세한 내용과 예시는 제어 흐름 (Control Flow)의 레이블이 있는 구문 (Labeled Statements)을 참고바랍니다.

Grammar of a labeled statement

labeled-statement → statement-label loop-statement labeled-statement → statement-label if-statement
labeled-statement → statement-label switch-statement labeled-statement → statement-label do-statement

statement-label → label-name `:` label-name → identifier

제어 전송 구문 (Control Transfer Statements)

제어 전송 구문(Control transfer statements)은 프로그램 제어를 한 코드에서 다른 코드로 무조건 전송하여 프로그램의 코드가 실행되는 순서를 변경할 수 있습니다. Swift는 다섯 가지 제어 전송 구문을 가지고 있습니다: `break` 구문, `continue` 구문, `fallthrough` 구문, `return` 구문, `throw` 구문입니다.

Grammar of a control transfer statement

control-transfer-statement → break-statement control-transfer-statement → continue-statement
control-transfer-statement → fallthrough-statement control-transfer-statement → return-statement
control-transfer-statement → throw-statement

중단 구문 (Break Statement)

`break` 구문은 루프, `if` 구문, `switch` 구문의 프로그램 실행을 종료합니다. `break` 구문은 `break` 키워드로만 구성되거나 `break` 키워드 다음에 구문 레이블의 이름으로 구성될 수 있습니다. 형식은 다음과 같습니다.

```
break
break <#label name#>
```

`break` 구문 다음에 구문 레이블의 이름이 오면, 해당 레이블로 명명된 루프, `if` 구문, `switch` 구문의 프로그램 실행을 종료합니다.

`break` 구문 다음에 구문 레이블의 이름이 없으면, `switch` 구문이나 해당 구문이 발생한 가장 안쪽의 루프 구문의 프로그램 실행이 종료됩니다. `if` 구문을 벗어나기 위해 레이블이 없는 `break` 구문을 사용할 수 없습니다.

두 경우 모두 프로그램 제어는 루프나 `switch` 구문 다음 코드의 첫 번째 줄로 전송됩니다.

break 구문을 어떻게 사용하는지에 대한 예시는 제어 흐름 (Control Flow)의 중단 (Break)과 레이블이 있는 구문 (Labeled Statements)을 참고바랍니다.

Grammar of a break statement

break-statement → `break` label-name?

계속 구문 (Continue Statement)

`continue` 구문은 루프 구문의 현재 이터레이션의 프로그램 실행을 종료하지만 루프 구문의 실행을 멈추지 않습니다. `continue` 구문은 `continue` 키워드로만 구성되거나, `continue` 키워드 다음에 구문 레이블의 이름으로 구성될 수 있습니다. 형식은 다음과 같습니다.

```
continue
continue <#label name#>
```

`continue` 구문 다음에 구문 레이블의 이름이 오면, 해당 레이블로 명명된 루프 구문의 현재 이터레이션의 프로그램 실행을 종료합니다.

`continue` 구문 다음에 구문 레이블의 이름이 없으면, 해당 구문이 발생한 가장 안쪽의 루프 구문의 현재 이터레이션의 프로그램 실행이 종료됩니다.

두 경우 모두 프로그램 제어는 둘러싸인 루프 구문의 조건으로 전송됩니다.

`for` 구문에서 증분 표현식은 루프의 본문이 실행된 후에 평가되므로 `continue` 구문 후에도 계속 평가됩니다.

`continue` 구문 사용에 대한 예시는 제어 흐름 (Control Flow)의 계속 (Continue)과 레이블이 있는 구문 (Labeled Statements)을 참고바랍니다.

Grammar of a continue statement

continue-statement → `continue` label-name?

Fallthrough 구문 (Fallthrough Statement)

`fallthrough` 구문은 `fallthrough` 키워드로 구성되고 `switch` 구문의 케이스 블록에서만 발생합니다. `fallthrough` 구문은 `switch` 구문의 한 케이스에서 다음 케이스로 계속해서 프로그램이 실행되도록 합니다. `switch` 구문의 제어 표현식의 값이 케이스 레이블의 패턴과 일치하지 않아도 다음 케이스는 계속 실행됩니다.

`fallthrough` 구문은 케이스 블록의 마지막 구문 뿐만 아니라 `switch` 구문 내 어디든 나타날 수 있지만 마지막 케이스 블록에서는 사용할 수 없습니다. 또한 값 바인딩 패턴이 포함된 케이스 블록으로 제어를 전송할 수도 없습니다.

`switch` 구문에서 `fallthrough` 구문을 사용하는 방법에 대한 예시는 제어 흐름 (Control Flow)의 제어 전송 구문 (Control Transfer Statements)을 참고바랍니다.

Grammar of a fallthrough statement

fallthrough-statement → `fallthrough`

Retrun 구문 (Return Statement)

`return` 구문은 함수나 메서드 정의의 본문에서 발생하고 호출한 함수나 메서드에 반환하기 위해 실행합니다. 프로그램 실행은 함수나 메서드 호출 바로 다음부터 계속해서 실행됩니다.

`return` 구문은 `return` 키워드만으로 구성되거나 `return` 키워드 다음에 표현식으로 구성될 수 있습니다. 형식은 다음과 같습니다.

```
return
return <#expression#>
```

`return` 구문 다음에 표현식이 오면, 표현식의 값은 호출한 함수나 메서드에 반환됩니다. 표현식의 값이 함수나 메서드 선언에 선언된 반환 타입의 값과 일치하지 않으면, 표현식의 값은 호출한 함수나 메서드에 반환되기 전에 반환 타입을 변환합니다.

참고

실패 가능한 이니셜라이저 (Failable Initializers)에서 설명한대로 `return` 구문의 특수한 형식(`return nil`)은 초기화 실패를 나타내기 위해 실패 가능한 이니셜라이저에서 사용할 수 있습니다.

`return` 구문 다음에 표현식이 없으면, 반환값이 없는 함수나 메서드에 반환하기 위해서만 사용될 수 있습니다 (즉, 함수나 메서드의 반환 타입이 `Void`나 `()`인 경우에 해당됩니다).

Grammar of a return statement

return-statement → `return` expression?

Throw 구문 (Throw Statement)

`throw` 구문은 오류를 던지는 함수나 메서드의 본문이나 타입에 `throws` 키워드가 표시된 클로저 표현식의 본문에서 발생합니다.

`throw` 구문은 프로그램이 현재 범위의 실행을 종료하고 둘러싸인 범위로 오류 전파를 시작합니다. 던져진 오류는 `do` 구문의 `catch` 절에 의해 처리될 때까지 계속 전파됩니다.

`throw` 구문은 `throw` 키워드 다음에 표현식으로 구성됩니다. 형식은 다음과 같습니다.

```
throw <#expression#>
```

위 형식의 *표현식(expression)*의 값은 `Error` 프로토콜을 준수하는 타입이어야 합니다. `do` 구문이나 `throw` 구문을 포함하는 함수에 발생하는 오류의 타입을 선언하면, 위 형식의 *표현식(expression)*의 값은 해당 타입의 인스턴스여야 합니다.

`throw` 구문을 어떻게 사용하는지에 대한 예시는 오류 처리 (Error Handling)의 던지는 함수를 이용한 오류 전파 (Propagating Errors Using Throwing Functions)를 참고바랍니다.

Grammar of a throw statement

throw-statement → `throw` expression

Defer 구문 (Defer Statement)

참고

번역자 보충 Swift 6.4에서는 비동기 컨텍스트의 `defer` 본문에서 `await`를 사용할 수 있습니다. 범위를 벗어날 때 해당 작업을 암시적으로 기다립니다. 예제와 격리·취소 조건은 비동기 defer (Asynchronous Defer)를 참고바랍니다.

`defer` 구문은 `defer` 구문이 나타나는 범위의 바깥으로 프로그램 제어가 전송되기 직전에 실행되는 코드에 사용됩니다.

`defer` 구문은 다음의 형식을 가집니다:

```
defer {
    <#statements#>
}
```

`defer` 구문 내의 구문은 프로그램 제어가 어떻게 전송되는 것과 관계없이 실행됩니다. 이것은 예를 들어 `defer` 구문은 파일 닫기와 같은 수동 리소스 관리나 오류가 발생하더라도 수행되어야 하는 작업이 있는 경우에 사용할 수 있습니다.

`defer` 구문의 위 형식의 *구문(statement)*은 `defer` 구문을 둘러싸는 범위의 끝에서 실행됩니다.

```
func f(x: Int) {
  defer { print("First defer") }

  if x < 10 {
    defer { print("Second defer") }
```

```
    print("End of if")
  }

  print("End of function")
}
f(x: 5)
// Prints "End of if".
// Prints "Second defer".
// Prints "End of function".
// Prints "First defer".
```

위의 코드에서 `if` 구문 내의 `defer`는 함수 `f` 내에 선언된 `defer`보다 먼저 실행되는데, 이것은 `if` 구문의 범위가 함수의 범위보다 먼저 끝나기 때문입니다.

같은 범위 안에 여러 개의 `defer` 구문이 있다면, 나타나는 순서의 역순으로 실행됩니다. 범위의 마지막 `defer` 구문을 먼저 실행한다는 것은 해당 마지막 `defer` 구문 내의 구문이 다른 `defer` 구문에 의해 정리될 리소스를 참조할 수 있다는 의미입니다.

```
func f() {
    defer { print("First defer") }
    defer { print("Second defer") }
    print("End of function")
}
f()
// Prints "End of function".
// Prints "Second defer".
// Prints "First defer".
```

`defer` 구문 내의 구문은 `defer` 구문의 밖으로 프로그램 제어를 전송할 수 없습니다.

Grammar of a defer statement

defer-statement → `defer` code-block

Do 구문 (Do Statement)

`do` 구문은 새로운 범위를 도입하기 위해 사용되고 정의된 오류 조건과 일치하는 패턴을 포함하는 하나 이상의 `catch` 절을 포함할 수 있습니다. `do` 구문의 범위에서 선언된 변수와 상수는 해당 범위 내에서만 접근할 수 있습니다.

Swift에서 `do` 구문은 코드 블록을 구분하는데 사용되는 C의 중괄호(`{}`)와 유사하며 런타임에 성능 비용이 발생하지 않습니다.

`do` 구문은 다음의 형식을 가집니다:

```
do {
    try <#expression#>
    <#statements#>
} catch <#pattern 1#> {
    <#statements#>
} catch <#pattern 2#> where <#condition#> {
    <#statements#>
} catch <#pattern 3#>, <#pattern 4#> where <#condition#> {
```

```
    <#statements#>
} catch {
    <#statements#>
}
```

do 구문은 선택적으로 오류의 타입을 지정할 수 있으며, 아래와 같은 형식을 가집니다:

```
do throws(<#type#>) {
    try <#expression#>
} catch <#pattern> {
    <#statements#>
} catch {
    <#statements#>
}
```

do 구문이 throws 절을 포함하면, do 본문은 지정된 *타입(type)*의 오류만 던질 수 있습니다. 위 형식의 *타입(type)*은 Error 프로토콜을 준수하는 구체적인 타입이거나, Error 프로토콜을 준수하는 불투명 타입, 박싱 프로토콜 타입 any Error이어야 합니다. do 구문은 오류의 타입을 지정하지 않으면, Swift는 다음과 같이 오류 타입을 추론합니다:

- do 코드 본문의 모든 throws 구문과 try 표현식이 오류 처리 메커니즘으로 완벽하게 처리되면, Swift는 do 구문이 오류를 던지지 않는다고 추론합니다.
- do 코드 본문이 Never를 던지는 것이 아니라 하나의 오류 타입만을 던지는 코드가 포함되어 있는 경우, Swift는 do 구문이 구체적인 오류 타입을 던진다고 추론합니다.
- do 코드 본문이 하나 이상의 오류 타입을 던지는 경우, Swift는 do 구문은 any Error를 던진다고 추론합니다.

명시적 타입을 가지는 오류 동작의 자세한 내용은, 오류 타입 지정 (Specifying the Error Type)을 참고바랍니다.

do 코드 블록의 구문에서 오류가 발생하면 프로그램 제어는 패턴이 일치하는 첫 번째 catch 절로 전송됩니다. 일치하는 절이 없으면, 오류를 주변 범위로 전파합니다. 오류를 최상위 수준에서 처리되지 않으면, 프로그램 실행은 런타임 오류로 중단됩니다.

switch 구문처럼 컴파일러는 catch 절이 완벽한지 여부를 추론하려고 합니다. 그러한 결정을 내릴 수 있으면 오류가 처리된 것으로 간주합니다. 그렇지 않으면 오류는 포함된 범위 밖으로 전파될 수 있습니다. 이것은 오류는 catch 절에서 처리되거나 포함한 함수가 throws로 선언되어야 한다는 의미입니다.

여러 패턴을 가지는 catch 절은 패턴 중 하나가 일치하는 경우에 오류와 일치합니다. catch 절에 여러 패턴을 포함하는 경우 모든 패턴은 동일한 상수나 변수 바인딩이 포함되어야 하며 각 바인딩된 변수나 상수는 모든 catch 절의 패턴에서 동일한 타입을 가져야 합니다.

오류가 처리되도록 보장하려면 와일드카드 패턴(_)과 같이 모든 오류를 일치하는 패턴으로 catch 절을 사용해야 합니다. catch 절에 패턴을 지정하지 않으면, catch 절은 모든 오류와 일치하고 error라는 이름의 지역 상수에 바인딩 됩니다. catch 절에서 사용할 수 있는 패턴에 대한 자세한 내용은 패턴 (Patterns)을 참고바랍니다.

do 구문과 여러 catch 절을 사용하는 방법에 대한 예시는 오류 처리 (Handling Errors)를 참고바랍니다.

Grammar of a do statement

do-statement → do throws-clause? code-block catch-clauses? catch-clauses → catch-clause catch-clauses? catch-clause → catch catch-pattern-list? code-block catch-pattern-list → catch-pattern | catch-pattern , catch-pattern-list catch-pattern → pattern where-clause?

컴파일러 제어 구문 (Compiler Control Statements)

컴파일러 제어 구문(compiler control statement)은 컴파일러의 동작 측면을 변경할 수 있습니다. Swift는 세 가지 컴파일러 제어 구문을 가지고 있습니다: 조건부 컴파일 블록(conditional compilation block), 라인 제어 구문(line control statement), 컴파일 시점 진단 구문(compile-time diagnostic statement)입니다.

Grammar of a compiler control statement

compiler-control-statement → conditional-compilation-block compiler-control-statement → line-control-statement compiler-control-statement → diagnostic-statement

조건부 컴파일 블록 (Conditional Compilation Block)

조건부 컴파일 블록(conditional compilation block)은 하나 이상의 컴파일러 조건의 값에 따라 코드를 조건부로 컴파일할 수 있습니다.

모든 조건부 컴파일 블록은 `#if` 컴파일 지시문으로 시작하고 `#endif` 컴파일 지시문으로 끝납니다. 간단한 조건부 컴파일 블록은 다음의 형식을 가집니다:

```
#if <#compilation condition#>
    <#statements#>
#endif
```

`if` 구문의 조건과 다르게 위 형식의 *컴파일러 조건(compilation condition)*은 컴파일 시에 평가됩니다. 결과적으로 위 형식의 *구문(statements)*은 컴파일 시에 위 형식의 *컴파일러 조건(compilation condition)*이 `true`로 평가될 때만 컴파일되고 실행됩니다.

위 형식의 *컴파일러 조건(compilation condition)*은 `true`와 `false` Boolean 리터럴(Boolean literals), `–D` 명령줄 플래그를 사용한 식별자, 아래 표에 나열된 플랫폼 조건을 포함할 수 있습니다.

플랫폼 조건	유효 인자
`os()`	`macOS`, `iOS`, `watchOS`, `tvOS`, `visionOS`, `Linux`, `Windows`
`arch()`	`arm`, `arm64`, `i386`, `wasm32`, `x86_64`
`swift()`	>= 또는 < 다음에 버전 번호
`compiler()`	>= 또는 < 다음에 버전 번호
`canImport()`	모듈 이름
`targetEnvironment()`	`simulator`, `macCatalyst`

`swift()`와 `compiler()` 플랫폼 조건에 대한 버전 번호는 버전 번호의 각 부분을 점(`.`)으로 구분하여 주 번호, 선택적 부 번호, 선택적 패치 번호 등으로 구성됩니다. 비교 연산자와 버전 번호 사이에 공백이 없어야 합니다. `compiler()`에 대한 버전은 컴파일러에 전달된 Swift 버전 설정과 상관없이 컴파일러 버전입니다. `swift()`에 대

한 버전은 현재 컴파일되는 언어 버전입니다. 예를 들어 Swift 4.2 모드에서 Swift 5 컴파일러를 사용하여 코드를 컴파일 한다면, 컴파일러 버전은 5이고 언어 버전은 4.2입니다. 이러한 설정으로 다음의 코드는 세가지 메세지를 모두 출력합니다:

```
#if compiler(>=5)
print("Compiled with the Swift 5 compiler or later")
#endif
#if swift(>=4.2)
print("Compiled in Swift 4.2 mode or later")
#endif
#if compiler(>=5) && swift(<5)
print("Compiled with the Swift 5 compiler or later in a Swift mode earlier than 5")
#endif
// Prints "Compiled with the Swift 5 compiler or later".
// Prints "Compiled in Swift 4.2 mode or later".
// Prints "Compiled with the Swift 5 compiler or later in a Swift mode earlier than 5".
```

`canImport()` 플랫폼 조건의 인자는 모든 플랫폼에 존재하지 않을 수 있는 모듈의 이름입니다. 모듈은 이름에 마침표(`.`)를 포함할 수 있습니다. 이 조건은 해당 모듈을 가져올 수 있는지 검사하지만 실제로 가져오지 않습니다. 모듈이 있으면 플랫폼 조건은 `true`를 반환하고; 그렇지 않으면 `false`를 반환합니다.

`targetEnvironment()` 플랫폼 조건은 지정한 환경에 대해 코드가 컴파일 되면 `true`를 반환하고; 그렇지 않으면 `false`를 반환합니다.

> **참고**
>
> `arch(arm)` 플랫폼 조건은 ARM 64 기기에 대해 `true`를 반환하지 않습니다. `arch(i386)` 플랫폼 조건은 32-비트 iOS 시뮬레이터에 대해 코드가 컴파일될 때 `true`를 반환합니다.

논리 연산자 `&&`, `||`, `!`를 사용하여 컴파일 조건을 결합하고 부정할 수 있으며 그룹화를 위한 소괄호를 사용할 수 있습니다. 이러한 연산자는 Boolean 표현식을 결합하는데 사용되는 논리 연산자와 동일한 결합방향과 우선순위를 갖습니다.

`if` 구문과 유사하게 다른 컴파일 조건에 대해 검사하기 위해 여러 조건부 분기를 추가할 수 있습니다. `#elseif` 절을 사용하여 원하는 만큼 분기를 추가할 수 있습니다. `#else` 절을 사용하여 마지막 분기를 추가할 수도 있습니다. 여러 개 분기를 포함하는 조건부 컴파일 블록 형식은 다음과 같습니다:

```
#if <#compilation condition 1#>
    <#statements to compile if compilation condition 1 is true#>
#elseif <#compilation condition 2#>
    <#statements to compile if compilation condition 2 is true#>
#else
    <#statements to compile if both compilation conditions are false#>
#endif
```

> **참고**
>
> 조건부 컴파일 블록의 본문에서 각 구문(statement)은 컴파일 되지 않아도 구문 분석됩니다. 그러나 컴파일 조건이 `swift()`나 `compiler()` 플랫폼 조건을 포함하면 예외가 있습니다: 이 구문은 플랫폼 조건에서 지정한 언어나 컴파일러 버전이 일치하는 경우에만 분석됩니다. 이 예외는 이전 컴파일러가 최신 버전의 Swift에 도입된 구문을 분석하지 않도록 합니다.

조건부 컴파일 블록에서 명시적 멤버 표현식을 래핑하는 방법에 대한 자세한 내용은 명시적 멤버 표현식 (Explicit Member Expression)을 참고바랍니다.

Grammar of a conditional compilation block

conditional-compilation-block → if-directive-clause elseif-directive-clauses? else-directive-clause? endif-directive

if-directive-clause → if-directive compilation-condition statements? elseif-directive-clauses → elseif-directive-clause elseif-directive-clauses? elseif-directive-clause → elseif-directive compilation-condition statements? else-directive-clause → else-directive statements? if-directive → `#if` elseif-directive → `#elseif` else-directive → `#else` endif-directive → `#endif`

compilation-condition → platform-condition compilation-condition → identifier compilation-condition → boolean-literal compilation-condition → `(` compilation-condition `)` compilation-condition → `!` compilation-condition compilation-condition → compilation-condition `&&` compilation-condition compilation-condition → compilation-condition `||` compilation-condition

platform-condition → `os` `(` operating-system `)` platform-condition → `arch` `(` architecture `)` platform-condition → `swift` `(` >= swift-version `)` | `swift` `(` < swift-version `)` platform-condition → `compiler` `(` >= swift-version `)` | `compiler` `(` < swift-version `)` platform-condition → `canImport` `(` import-path `)` platform-condition → `targetEnvironment` `(` environment `)`

operating-system → `macOS` | `iOS` | `watchOS` | `tvOS` | `visionOS` | `Linux` | `Windows` architecture → `arm` | `arm64` | `i386` | `wasm32` | `x86_64` swift-version → decimal-digits swift-version-continuation? swift-version-continuation → `.` decimal-digits swift-version-continuation? environment → `simulator` | `macCatalyst`

라인 제어 구문 (Line Control Statement)

라인 제어 구문(line control statement)은 컴파일 중인 소스 코드의 라인 번호와 파일 이름이 다를 수 있는 라인 번호와 파일 이름을 지정하는데 사용됩니다. 라인 제어 구문을 사용하여 Swift가 진단과 디버깅을 목적으로 사용하는 소스 코드 위치를 변경할 수 있습니다.

라인 제어 구문은 다음의 형식을 가집니다:

```
#sourceLocation(file: <#file path#>, line: <#line number#>)
#sourceLocation()
```

라인 제어 구문의 첫 번째 형식은 라인 제어 구문 다음 줄의 코드부터 `#line`, `#file`, `#fileID`, `#filePath` 리터럴 표현식의 값을 변경합니다. *라인 번호(line number)*는 `#line`의 값을 변경하고, 0보다 큰 모든 정수 리터럴 입니다. *파일 경로(file path)*는 `#file`, `#fileID`, `#filePath`의 값을 변경하고, 문자열 리터럴 입니다. 지정한 문자열은 `#filePath`의 값이 되고, 문자열의 마지막 구성요소는 `#fileID`의 값으로 사용됩니다. `#file`, `#fileID`, `#filePath`의 자세한 내용은 리터럴 표현식 (Literal Expression)을 참고바랍니다.

라인 제어 구문의 두 번째 형식 인 `#sourceLocation()`은 소스 코드 위치를 기본 라인 번호와 파일 경로로 초기화 합니다.

Grammar of a line control statement

line-control-statement → `#sourceLocation ( file:` file-path `, line:` line-number `)`
line-control-statement → `#sourceLocation ( )` line-number → A decimal integer greater than zero
file-path → static-string-literal

컴파일 시점 진단 구문 (Compile-Time Diagnostic Statement)

Swift 5.9 이전에는 컴파일 중에 `#warning`과 `#error` 구문은 진단 정보를 생성했습니다. 이 동작은 이제 Swift 표준 라이브러리에 `warning(_:)`과 `error(_:)` 매크로로 제공됩니다.

가용성 조건 (Availability Condition)

*가용성 조건(availability condition)*은 지정된 플랫폼 인자를 기반으로 런타임에 API의 가용성을 조사하기 위해 `if`, `while`, `guard` 구문의 조건으로 사용됩니다.

가용성 조건은 다음의 형식을 가집니다:

```
if #available(<#platform name#> <#version#>, <#...#>, *) {
    <#statements to execute if the APIs are available#>
} else {
    <#fallback statements to execute if the APIs are unavailable#>
}
```

가용성 조건을 사용하여 런타임에 API가 사용 가능 여부에 따라 코드 블록을 실행합니다. 컴파일러는 코드의 블록이 가능한 API인 경우 가용성 조건의 정보를 사용합니다.

가용성 조건은 콤마로 구분된 플랫폼 이름과 버전을 가집니다. 플랫폼 이름으로 `iOS`, `macOS`, `watchOS`, `tvOS`, `visionOS`를 사용하고 해당 버전 숫자를 포함합니다. `*` 인자는 필수이고 다른 플랫폼에서 가용성 조건으로 보호되는 코드 블록의 본문이 지정한 최소 배포 타겟에서 실행되도록 합니다.

Boolean 조건과 다르게 &&와 || 같은 논리 연산자를 사용하여 가용성 조건을 결합할 수 없습니다. 가용성 조건에 부정을 나타내기 위해 `!`을 사용하는 대신에 다음의 형식을 가지는 불가능성 조건을 사용합니다:

```
if #unavailable(<#platform name#> <#version#>, <#...#>) {
    <#fallback statements to execute if the APIs are unavailable#>
} else {
    <#statements to execute if the APIs are available#>
}
```

`#unavailable` 형식은 조건을 부정하는 구문입니다. 불가능성 조건에서 `*` 인자는 암시적이고 포함해서는 안됩니다. *의 의미는 가용성 조건과 같은 의미를 가집니다.

Grammar of an availability condition

availability-condition → `#available (` availability-arguments `)` availability-condition → `#unavailable (` availability-arguments `)` availability-arguments → availability-argument | availability-argument `,` availability-arguments availability-argument → platform-name platform-version availability-argument → `*`

platform-name → `iOS` | `iOSApplicationExtension` platform-name → `macOS` | `macOSApplicationExtension` platform-name → `macCatalyst` | `macCatalystApplicationExtension` platform-name → `watchOS` | `watchOSApplicationExtension` platform-name → `tvOS` | `tvOSApplicationExtension` platform-name → `visionOS` |

`visionOSApplicationExtension` platform-version → decimal-digits platform-version → decimal-digits **.** decimal-digits platform-version → decimal-digits **.** decimal-digits **.** decimal-digits

언어 참조

선언 (Declarations)

타입, 연산자, 변수 및 기타 이름과 구성을 소개합니다.

*선언(declaration)*은 프로그램에 새로운 이름이나 구성을 도입합니다. 예를 들어 함수와 메서드를 도입하기 위해, 변수와 상수를 도입하기 위해, 열거형, 구조체, 클래스, 프로토콜 타입을 정의하기 위해 선언을 사용합니다. 또한 선언을 사용하여 기존에 명명 타입의 동작을 확장하고 다른 곳에서 선언된 기호를 프로그램으로 가져올 수도 있습니다.

Swift에서 대부분의 선언은 선언과 동시에 구현되거나 초기화된다는 의미에서 정의이기도 합니다. 이 말은 프로토콜은 멤버를 정의하지 않으므로 대부분 프로토콜 멤버는 선언만 있습니다. 편의상 그리고 Swift에서는 해당 구별이 그다지 중요하지 않으므로 *선언(declaration)*은 선언과 정의를 모두 포함합니다.

Grammar of a declaration

declaration → import-declaration declaration → constant-declaration declaration → variable-declaration
declaration → typealias-declaration declaration → function-declaration declaration → enum-declaration
declaration → struct-declaration declaration → class-declaration declaration → actor-declaration
declaration → protocol-declaration declaration → initializer-declaration declaration →
deinitializer-declaration declaration → extension-declaration declaration → subscript-declaration
declaration → macro-declaration declaration → operator-declaration declaration →
precedence-group-declaration

최상위-수준 코드 (Top-Level Code)

Swift 소스 파일의 최상위-수준 코드는 비어있거나 구문, 선언, 표현식으로 구성됩니다. 기본적으로 소스 파일의 최상위-수준에 선언된 변수, 상수, 다른 명명 선언은 동일한 모듈의 일부인 모든 소스 파일 코드에서 접근할 수 있습니다. 접근 제어 수준 (Access Control Levels)에서 설명한대로 접근-수준 수정자(access-level modifier)로 선언을 표시하여 기본 동작을 재정의할 수 있습니다.

최상위-수준 코드는 두 가지 종류가 있습니다: 최상위-수준 선언(top-level declarations)과 실행 가능한 최상위-수준 코드(executable top-level code)입니다. 최상위-수준 선언은 선언으로만 구성되고 모든 Swift 소스 파일에서 허용됩니다. 실행 가능한 최상위-수준 코드는 선언 뿐만 아니라 구문과 표현식을 포함하고 프로그램에 대해 최상위-수준 진입점으로만 허용됩니다.

실행 가능하게 만들기 위해 컴파일 한 Swift 코드는 코드가 파일과 모듈로 구성되는 방식과 상관없이 최상위-수준 진입점을 나타내는 다음 접근방식 중 하나만 포함할 수 있습니다: 최상위-수준 실행 가능한 코드를 포함하는 파일, `main.swift` 파일, `main` 속성, `NSApplicationMain` 속성, `UIApplicationMain` 속성.

Grammar of a top-level declaration

top-level-declaration → statements?

코드 블록 (Code Blocks)

*코드 블록(code block)*은 선언 및 제어 구조에서 구문을 그룹화 하기위해 사용됩니다. 다음의 형식을 가집니다:

```
{
   <#statements#>
}
```

코드 블록 내의 위 형식의 *구문(statements)*은 선언, 표현식, 구문의 다른 종류가 포함되고 소스 코드에 나타나는 순서대로 실행됩니다.

Grammar of a code block

code-block → { statements? }

임포트 선언 (Import Declaration)

*임포트 선언(import declaration)*은 현재 파일 바깥에 선언된 기호에 접근할 수 있게 합니다. 기본 형식은 전체 모듈을 가져옵니다; `import` 키워드 다음에 모듈 이름이 따라오도록 구성됩니다:

```
import <#module#>
```

임포트할 기호에 대한 자세한 제한을 제공하면 모듈이나 하위 모듈 내의 특정 하위 모듈이나 특정 선언을 지정할 수 있습니다. 이런 상세한 형식이 사용하면 임포트한 기호만 (선언한 모듈이 아닌) 현재 범위에서 사용할 수 있습니다.

```
import <#import kind#> <#module#>.<#symbol name#>
import <#module#>.<#submodule#>
```

Grammar of an import declaration

import-declaration → attributes? `import` import-kind? import-path

import-kind → `typealias` | `struct` | `class` | `enum` | `protocol` | `let` | `var` | `func` import-path → identifier | identifier `.` import-path

상수 선언 (Constant Declaration)

*상수 선언(constant declaration)*은 프로그램에 상수값을 도입합니다. 상수 선언은 `let` 키워드를 사용하여 선언하고 다음의 형식을 가집니다:

```
let <#constant name#>: <#type#> = <#expression#>
```

상수 선언은 위 형식의 *상수 이름(constant name)*과 이니셜라이저 *표현식(expression)*의 값 사이의 변경 불가능한 바인딩을 정의합니다; 상수의 값이 설정된 후에는 변경할 수 없습니다. 이 말은 상수가 클래스 객체로 초기화되면, 이 객체 자체는 변경할 수 있지만 이 상수 이름과 참조하는 객체 간의 바인딩은 변경할 수 없습니다.

상수가 전역 범위로 선언되면, 값으로 반드시 초기화되어야 합니다. 상수 선언이 함수나 메서드의 컨텍스트에서 나타나면, 해당 값을 처음 읽기 전에 값을 설정한다는 보장이 있으면 나중에 초기화할 수 있습니다. 컴파일러가 상수의 값을 읽지 않는다는 것을 증명할 수 있으면, 상수에 값을 설정하지 않아도 됩니다. 이 분석을 *확정 초기화(definite initialization)*라고 합니다 - 컴파일러는 읽기 전에 확실하게 값이 설정되어 있다는 것을 증명합니다.

참고

확정 초기화(Definite initialization)는 도메인 지식을 요구하는 증명을 구성할 수 없으며, 조건문에서 상태를 추적하는 것에는 제한이 있습니다. 상수에 항상 값이 있다고 결정할 수 있지만, 컴파일러가 이를 증명할 수 없는 경우에, 값을 설정하는 코드 경로를 단순화하거나, 변수 선언을 사용합니다.

상수 선언이 클래스나 구조체 선언의 컨텍스트에서 나타나면 *상수 프로퍼티(constant property)*로 간주됩니다. 상수 선언은 연산 프로퍼티가 아니므로 getter나 setter를 가지지 않습니다.

상수 선언의 *상수 이름(constant name)*이 튜플 패턴인 경우, 튜플에서 각 항목의 이름은 이니셜라이저 *표현식(expression)*에서 해당 값으로 바인딩 됩니다.

```
let (firstNumber, secondNumber) = (10, 42)
```

이 예시에서 `firstNumber`는 값 `10`에 대한 상수이고, `secondNumber`는 값 `42`에 대한 상수 입니다. 두 상수 모두 독립적으로 사용할 수 있습니다:

```
print("The first number is \(firstNumber).")
// Prints "The first number is 10.".
print("The second number is \(secondNumber).")
// Prints "The second number is 42.".
```

타입 추론 (Type Inference)에서 설명한대로 *상수 이름(constant name)*의 타입을 추론할 수 있을 때 상수 선언의 타입 주석(`:` 타입)은 선택 사항입니다.

상수 타입 프로퍼티(constant type property)를 선언하려면 `static` 선언 수정자로 선언을 표시해야 합니다. 클래스의 상수 타입 프로퍼티는 암시적으로 final 입니다; 하위 클래스에 의한 재정의를 허용하거나 금지하기 위해 `class`나 `final` 선언 수정자로 표시할 수 없습니다. 타입 프로퍼티는 타입 프로퍼티 (Type Properties)에 설명되어 있습니다.

상수에 대한 자세한 내용과 언제 사용해야 하는지에 대한 지침은 상수와 변수 (Constants and Variables)와 저장 프로퍼티 (Stored Properties)를 참고바랍니다.

Grammar of a constant declaration

constant-declaration → attributes? declaration-modifiers? `let` pattern-initializer-list

pattern-initializer-list → pattern-initializer | pattern-initializer `,` pattern-initializer-list pattern-initializer → pattern initializer? initializer → = expression

변수 선언 (Variable Declaration)

*변수 선언(variable declaration)*은 프로그램에 변수 값을 도입하고 `var` 키워드를 사용하여 선언합니다.

변수 선언은 저장 및 연산 변수와 프로퍼티, 저장 변수와 프로퍼티 관찰자, 정적 변수 프로퍼티를 포함하여 여러 종류의 명명된 값과 변경 가능한 값을 선언하는 여러 형식을 가집니다. 사용할 적절한 형식은 변수가 선언되는 범위와 선언하려는 변수의 종류에 따라 다릅니다.

> **참고**
>
> 프로토콜 프로퍼티 선언 (Protocol Property Declaration)에서 설명한대로 프로토콜 선언의 컨텍스트에서도 프로퍼티를 선언할 수 있습니다.

재정의 (Overriding)에서 설명한대로 `override` 선언 수정자로 하위 클래스의 프로퍼티 선언을 표시하여 하위 클래스에서 프로퍼티를 재정의할 수 있습니다.

저장 변수와 변수 저장 프로퍼티 (Stored Variables and Stored Variable Properties)

다음의 형식으로 저장 변수(stored variable)나 변수 저장 프로퍼티(stored variable property)를 선언할 수 있습니다:

```
var <#variable name#>: <#type#> = <#expression#>
```

이러한 형식의 변수 선언은 전역 범위, 함수의 지역 범위나 클래스나 구조체 선언의 컨텍스트에서 정의합니다. 이러한 형식의 변수 선언이 전역 범위나 함수의 지역 범위로 선언되면 *저장 변수(stored variable)*로 참조됩니다. 클래스나 구조체 선언의 컨텍스트에서 선언되면 *변수 저장 프로퍼티(stored variable property)*로 참조됩니다.

이니셜라이저 *표현식(expression)*은 프로토콜 선언에 있을 수 없지만 다른 모든 컨텍스트에서는 이니셜라이저 *표현식(expression)*이 선택 사항입니다. 즉, 이니셜라이저 *표현식(expression)*이 없으면, 변수 선언은 명시적 타입 주석(**:** 타입(type))을 포함해야 합니다.

상수 선언과 마찬가지로 변수 선언이 이니셜라이저 *표현식(expression)*을 생략하는 경우, 변수는 처음 읽기 전에 값을 설정해야 합니다. 또한 상수 선언과 마찬가지로 *변수 이름(variable name)*이 튜플 패턴인 경우, 튜플에서 각 항목에 이름은 이니셜라이저 *표현식(expression)*의 해당 값에 바인딩 됩니다.

이름에서 알 수 있듯이, 저장 변수나 변수 저장 프로퍼티의 값은 메모리에 저장됩니다.

연산 변수와 연산 프로퍼티 (Computed Variables and Computed Properties)

다음의 형식은 연산 변수(computed variable)나 연산 프로퍼티(computed property)를 선언합니다:

```
var <#variable name#>: <#type#> {
   get {
      <#statements#>
   }
   set(<#setter name#>) {
      <#statements#>
   }
}
```

이러한 형식의 변수 선언은 전역 범위, 함수의 지역 범위 또는 클래스, 구조체, 열거형, 확장 선언의 컨텍스트에서 정의합니다. 이러한 형식의 변수 선언을 전역 범위나 함수의 지역 범위로 선언하면 *연산 변수(computed variable)*로 참조합니다. 클래스, 구조체, 확장 선언의 컨텍스트에서 선언하면 *연산 프로퍼티(computed property)*로 참조합니다.

getter는 값을 읽기 위해 사용되고, setter는 값을 작성하기 위해 사용됩니다. setter 절은 선택 사항이며, getter만 필요하다면 읽기 전용 연산 프로퍼티 (Read-Only Computed Properties)에 설명한대로 getter와 setter 절 모두 생략하고 요구된 값을 직접적으로 반환할 수 있습니다. 그러나 setter 절을 제공하면 getter 절도 제공해야 합니다.

위 형식의 *setter 이름(setter name)*과 둘러싸인 소괄호는 선택 사항입니다. setter 이름을 제공하면, setter의 매개변수 이름으로 사용됩니다. setter 이름을 제공하지 않으면, 간략한 Setter 선언 (Shorthand Setter Declaration)에서 설명한대로 setter의 기본 매개변수 이름은 `newValue`입니다.

명명된 저장값과 변수 저장 프로퍼티와 다르게 명명된 연산값이나 연산 프로퍼티의 값은 메모리에 저장되지 않습니다.

연산 프로퍼티의 자세한 내용과 예시는 연산 프로퍼티 (Computed Properties)를 참고바랍니다.

저장 변수 관찰자와 프로퍼티 관찰자 (Stored Variable Observers and Property Observers)

`willSet`과 `didSet` 관찰자와 함께 저장 변수나 프로퍼티를 선언할 수도 있습니다. 괄찰자와 함께 선언한 저장 변수나 프로퍼티는 다음의 형식을 가집니다:

```
var <#variable name#>: <#type#> = <#expression#> {
   willSet(<#setter name#>) {
      <#statements#>
   }
   didSet(<#setter name#>) {
      <#statements#>
   }
}
```

이러한 형식의 변수 선언은 전역 범위, 함수의 지역 범위 또는 클래스나 구조체 선언의 컨텍스트에서 정의합니다. 이러한 형식의 변수 선언이 전역 범위나 함수의 지역 범위로 선언되면, 관찰자는 *저장 변수 관찰자(stored variable observers)*로 참조합니다. 클래스나 구조체 선언의 컨텍스트에서 선언되면, 관찰자는 *프로퍼티 관찰자(property observers)*로 참조합니다.

모든 저장 프로퍼티에 프로퍼티 관찰자를 추가할 수 있습니다. 프로퍼티 관찰자 재정의 (Overriding Property Observers)에서 설명한대로 하위 클래스 내에 프로퍼티 재정의로 저장 프로퍼티나 연산 프로퍼티와 상관없이 모든 상속된 프로퍼티에 프로퍼티 관찰자를 추가할 수도 있습니다.

이니셜라이저 *표현식(expression)*은 클래스나 구조체 선언의 컨텍스트에서 선택 사항이지만 다른 곳에선 필수입니다. 타입이 이니셜라이저 *표현식(expression)*에서 추론될 수 있으면, *타입(type)*주석은 선택 사항입니다. 이 표현식은 프로퍼티의 값을 처음 읽을 때 평가됩니다. 프로퍼티의 초기값을 읽지 않고 재작성하면, 이 표현식은 프로퍼티를 처음 쓰기 전에 평가됩니다.

`willSet`과 `didSet` 관찰자는 변수나 프로퍼티의 값이 설정될 때 관찰하고 적절한 반응을 위한 방법을 제공합니다. 관찰자는 변수나 프로퍼티가 처음 초기화될 때는 호출되지 않습니다. 대신에 값이 초기화 컨텍스트 바깥에서 설정될 때만 호출됩니다.

`willSet` 관찰자는 변수나 프로퍼티의 값이 설정되기 직전에만 호출됩니다. 새로운 값은 상수로 `willSet` 관찰자에 전달되고 `willSet` 절의 구현에서 변경할 수 없습니다. `didSet` 관찰자는 새로운 값이 설정된 후 바로 호출됩니다. `willSet` 관찰자와 달리 변수나 프로퍼티의 이전 값은 여전히 접근이 필요한 경우에 `didSet` 관찰자에 전달됩니다. 이 말은 `didSet` 관찰자 절 내에 변수나 프로퍼티에 값을 할당하면, 할당한 새 값이 방금 설정되어 `willSet` 관찰자에 전달된 값을 대체합니다.

`willSet`과 `didSet` 절에 위 형식의 *setter 이름(setter name)*과 둘러싸인 소괄호는 선택 사항입니다. setter 이름을 제공한다면, 해당 이름은 `willSet`과 `didSet` 관찰자의 매개변수 이름으로 사용됩니다. setter 이름을 제공하

지 않는다면, `willSet` 관찰자의 기본 매개변수 이름은 `newValue`이고 `didSet` 관찰자의 기본 매개변수 이름은 `oldValue`입니다.

`willSet` 절을 제공하면 `didSet` 절은 선택 사항입니다. 마찬가지로 `didSet` 절을 제공하면 `willSet` 절은 선택 사항입니다.

`didSet` 관찰자의 본문이 이전 값을 참조하면, 관찰자 전에 getter가 호출되어 이전 값을 사용 가능하도록 합니다. 그렇지 않으면 상위 클래스의 getter를 호출하지 않고 새 값이 저장됩니다. 아래 예시는 상위 클래스에 정의한 연산 프로퍼티를 보여주고 이 연산 프로퍼티를 하위 클래스에서 관찰자를 추가하여 재정의하는 것을 보여줍니다.

```
class Superclass {
    private var xValue = 12
    var x: Int {
        get { print("Getter was called"); return xValue }
        set { print("Setter was called"); xValue = newValue }
    }
}

// This subclass doesn't refer to oldValue in its observer, so the
// superclass's getter is called only once to print the value.
class New: Superclass {
    override var x: Int {
        didSet { print("New value \(x)") }
    }
}
let new = New()
new.x = 100
// Prints "Setter was called".
// Prints "Getter was called".
// Prints "New value 100".

// This subclass refers to oldValue in its observer, so the superclass's
// getter is called once before the setter, and again to print the value.
class NewAndOld: Superclass {
    override var x: Int {
        didSet { print("Old value \(oldValue) - new value \(x)") }
    }
}
let newAndOld = NewAndOld()
newAndOld.x = 200
// Prints "Getter was called".
// Prints "Setter was called".
// Prints "Getter was called".
// Prints "Old value 12 - new value 200".
```

프로퍼티 관찰자를 사용하는 방법에 대한 자세한 내용과 예시는 프로퍼티 관찰자 (Property Observers)를 참고바랍니다.

타입 변수 프로퍼티 (Type Variable Properties)

타입 변수 프로퍼티(type variable property)를 선언하기 위해 `static` 선언 수정자와 함께 선언에 표시합니다. 클래스는 상위 클래스의 구현을 하위 클래스가 재정의할 수 있도록 `class` 선언 수정자와 함께 타입 연산 프로퍼티를 표시할 수 있습니다. 타입 프로퍼티는 타입 프로퍼티 (Type Properties)에 설명되어 있습니다.

Grammar of a variable declaration

variable-declaration → variable-declaration-head pattern-initializer-list variable-declaration → variable-declaration-head variable-name type-annotation code-block variable-declaration → variable-declaration-head variable-name type-annotation getter-setter-block variable-declaration → variable-declaration-head variable-name type-annotation getter-setter-keyword-block variable-declaration → variable-declaration-head variable-name initializer willSet-didSet-block variable-declaration → variable-declaration-head variable-name type-annotation initializer? willSet-didSet-block

variable-declaration-head → attributes? declaration-modifiers? `var` variable-name → identifier

getter-setter-block → code-block getter-setter-block → { getter-clause setter-clause? } getter-setter-block → { setter-clause getter-clause } getter-clause → attributes? mutation-modifier? `get` code-block setter-clause → attributes? mutation-modifier? `set` setter-name? code-block setter-name → (identifier)

getter-setter-keyword-block → { getter-keyword-clause setter-keyword-clause? } getter-setter-keyword-block → { setter-keyword-clause getter-keyword-clause } getter-keyword-clause → attributes? mutation-modifier? `get` setter-keyword-clause → attributes? mutation-modifier? `set`

willSet-didSet-block → { willSet-clause didSet-clause? } willSet-didSet-block → { didSet-clause willSet-clause? } willSet-clause → attributes? `willSet` setter-name? code-block didSet-clause → attributes? `didSet` setter-name? code-block

타입 별칭 선언 (Type Alias Declaration)

*타입 별칭 선언(type alias declaration)*은 프로그램의 기존 타입에 별칭을 도입합니다. 타입 별칭 선언은 `typealias` 키워드를 사용하여 선언되고 다음의 형식을 가집니다:

```
typealias <#name#> = <#existing type#>
```

타입 별칭이 선언된 후 별칭된 위 형식의 *이름(name)*은 프로그램에서 위 형식의 기존 타입(existing type) 대신에 사용할 수 있습니다. 위 형식의 *기존 타입(existing type)*은 명명 타입이나 복합 타입일 수 있습니다. 타입 별칭은 새로운 타입을 생성하지 않습니다; 간단하게 기존 타입을 참조하도록 이름을 허용합니다.

타입 별칭 선언은 제네릭 매개변수를 사용하여 기존 제네릭 타입에 이름을 제공할 수 있습니다. 타입 별칭은 기존 타입의 제너릭 매개변수 중 일부나 전부에 대해 구체적인 타입을 제공할 수 있습니다. 예를 들어:

```
typealias StringDictionary<Value> = Dictionary<String, Value>

// The following dictionaries have the same type.
var dictionary1: StringDictionary<Int> = [:]
var dictionary2: Dictionary<String, Int> = [:]
```

타입 별칭이 제네릭 매개변수와 함께 선언될 때, 해당 매개변수의 제약조건은 기존 타입의 제네릭 매개변수의 제약조건과 완벽하게 일치해야 합니다. 예를 들어:

```
typealias DictionaryOfInts<Key: Hashable> = Dictionary<Key, Int>
```

타입 별칭과 기존 타입은 서로 바꿔서 사용될 수 있으므로, 타입 별칭은 추가로 제네릭 제약조건을 도입할 수 없습니다.

타입 별칭은 선언에서 모든 제네릭 매개변수를 생략하여 기존 타입의 제네릭 매개변수를 전달할 수 있습니다. 예를 들어 여기 선언한 `Diccionario` 타입 별칭은 `Dictionary`와 동일한 제네릭 매개변수와 제약조건을 가집니다.

```
typealias Diccionario = Dictionary
```

프로토콜 선언 내의 타입 별칭은 자주 사용되는 타입에 더 짧고 편리한 이름을 제공할 수 있습니다. 예를 들어:

```
protocol Sequence {
    associatedtype Iterator: IteratorProtocol
    typealias Element = Iterator.Element
}

func sum<T: Sequence>(_ sequence: T) -> Int where T.Element == Int {
    // ...
}
```

타입 별칭이 없다면, `sum` 함수는 `T.Element` 대신에 `T.Iterator.Element`로 연관 타입을 참조해야 합니다.

더 자세한 내용은 프로토콜 연관 타입 선언 (Protocol Associated Type Declaration)을 참고바랍니다.

Grammar of a type alias declaration

typealias-declaration → attributes? access-level-modifier? `typealias` typealias-name
generic-parameter-clause? typealias-assignment typealias-name → identifier typealias-assignment → = type

함수 선언 (Function Declaration)

*함수 선언(function declaration)*은 프로그램의 함수나 메서드를 도입합니다. 클래스, 구조체, 열거형, 프로토콜의 컨텍스트에 선언된 함수는 *메서드(method)*로 참조됩니다. 함수 선언은 `func` 키워드를 사용하여 선언되고 다음의 형식을 가집니다:

```
func <#function name#>(<#parameters#>) -> <#return type#> {
   <#statements#>
}
```

함수가 `Void`의 반환 타입을 가지면, 반환 타입은 다음과 같이 생략할 수 있습니다:

```
func <#function name#>(<#parameters#>) {
   <#statements#>
}
```

각 매개변수의 타입은 추론될 수 없으므로 반드시 포함되어야 합니다. 매개변수의 타입 앞에 `inout`을 작성하면, 매개변수는 함수의 범위 내에서 수정될 수 있습니다. In-out 매개변수는 아래 In-Out 매개변수 (In-Out Parameters)에서 자세하게 설명되어 있습니다.

함수 선어에서 *구문(statements)*에 단일 표현식만 포함된 경우 해당 표현식의 값을 반환하는 것으로 이해합니다. 이 암시적 반환 구문은 표현식의 타입과 함수의 반환 타입이 `Void`가 아니고 케이스가 없는 `Never`와 같은 열거형이 아닌 경우에만 간주됩니다.

함수는 함수의 반환 타입으로 튜플 타입을 사용하여 여러 값을 반환할 수 있습니다.

함수 정의는 다른 함수 선언 내에 나타날 수 있습니다. 이러한 함수를 *중첩 함수(nested function)*라고 합니다.

중첩 함수는 in-out 매개변수와 같이 절대 탈출하지 않는 값을 캡처하거나 비탈출 함수 인자로 전달된 경우 비탈출 입니다. 그렇지 않으면 중첩 함수는 탈출 함수 입니다.

중첩된 함수에 대한 내용은 중첩 함수 (Nested Functions)를 참고바랍니다.

매개변수 이름 (Parameter Names)

함수 매개변수는 각 매개변수가 여러 형식 중 하나를 갖는 콤마로 구분된 목록입니다. 함수 호출에서 인자의 순서는 함수의 선언 내에 매개변수의 순서와 일치해야 합니다. 매개변수 목록에서 가장 간단한 형식은 다음과 같습니다:

```
<#parameter name#>: <#parameter type#>
```

매개변수는 함수 본문 내에서 사용되는 이름 뿐만 아니라 함수나 메서드 호출할 때 사용되는 인자 레이블이 있습니다. 기본적으로 매개변수 이름은 인자 레이블로도 사용됩니다. 예를 들어:

```
func f(x: Int, y: Int) -> Int { return x + y }
f(x: 1, y: 2) // both x and y are labeled
```

다음의 형식 중 하나로 인자 레이블의 기본 동작을 재정의할 수 있습니다:

```
<#argument label#> <#parameter name#>: <#parameter type#>
_ <#parameter name#>: <#parameter type#>
```

매개변수 이름 전에 이름은 매개변수에 명시적 인자 레이블을 부여하고, 이것은 매개변수 이름과 다를 수 있습니다. 해당 인자는 함수나 메서드 호출에서 주어진 인자 레이블을 사용해야 합니다.

매개변수 이름 전에 언더바(_)는 인자 레이블을 숨깁니다. 해당 인자는 함수나 메서드 호출에서 레이블이 없어야 합니다.

```
func repeatGreeting(_ greeting: String, count n: Int) { /* Greet n times */ }
repeatGreeting("Hello, world!", count: 2) //  count is labeled, greeting is not
```

매개변수 수정자 (Parameter Modifiers)

*매개변수 수정자(parameter modifier)*는 인자가 함수에 전달되는 방식을 변경합니다.

```
<#argument label#> <#parameter name#>: <#parameter modifier#> <#parameter type#>
```

매개변수 수정자를 사용하려면, 인자의 타입 앞에 `inout`, `borrowing`, `consuming`을 작성합니다.

```
func someFunction(a: inout A, b: consuming B, c: C) { ... }
```

In-Out 매개변수 (In-Out Parameters)

기본적으로, Swift에서 함수 인자는 값으로 전달됩니다: 함수 내에서 수정된 것은 호출자에게 보이지 않습니다. in-out 매개변수를 만드려면, `inout` 매개변수 수정자를 적용합니다.

```
func someFunction(a: inout Int) {
    a += 1
}
```

in-out 매개변수를 포함하는 함수를 호출할 때, 인자의 값이 변경될 수 있는 함수 호출인 것을 나타내기위해 in-out 인자는 앰퍼샌드(&)를 앞에 붙여야 합니다.

```
var x = 7
someFunction(a: &x)
print(x)  // Prints "8".
```

In-out 매개변수는 다음과 같이 전달됩니다:

1. 함수가 호출될 때, 인자의 값은 복사됩니다.
2. 함수의 본문 내에서 복사본은 수정됩니다.
3. 함수가 반환될 때, 복사본의 값은 기존 인자에 할당됩니다.

이 동작은 copy-in copy-out이나 call by value 결과라고 합니다. 예를 들어 연산 프로퍼티나 관찰자가 있는 프로퍼티가 in-out 매개변수로 전달되는 경우 getter는 함수 호출의 부분으로 호출되고 setter는 함수 반환의 부분으로 호출됩니다.

최적화로 인자가 메모리의 물리적 주소에 저장된 값인 경우 동일한 메모리 위치가 함수 본문 내부 및 외부에서 모두 사용됩니다. 이런 최적화 동작을 call by reference라고 합니다; 이것은 copy-in copy-out 모델의 모든 요구사항을 충족하는 동시에 복사의 오버헤드를 제거합니다. call-by-reference 최적화에 의존하지 않고 copy-in copy-out에 의해 주어진 모델을 사용하여 작성하면, 최적화에 상관없이 올바르게 작동되도록 합니다.

함수 내에서 기존값이 현재 범위에서 사용가능 하더라도 in-out 인자로 전달된 값은 접근하면 안됩니다. 기존값에 접근하는 것은 값에 대한 동시 접근이며 메모리 독점성을 위반합니다.

```
var someValue: Int
func someFunction(a: inout Int) {
    a += someValue
}

// Error: This causes a runtime exclusivity violation
someFunction(a: &someValue)
```

같은 이유로 여러 개의 in-out 매개변수에 동일한 값을 전달할 수 없습니다.

```
var someValue: Int
func someFunction(a: inout Int, b: inout Int) {
    a += b
    b += 1
}

// Error: Cannot pass the same value to multiple in-out parameters
someFunction(a: &someValue, b: &someValue)
```

메모리 안전성과 메모리 독점성에 대한 자세한 내용은 메모리 안전성 (Memory Safety)을 참고바랍니다.

in-out 매개변수를 캡처하는 클로저나 중첩 함수는 비탈출 이어야 합니다. in-out 매개변수를 변경하지 않고 캡처 해야하는 경우, 캡처 리스트를 사용하여 매개변수를 변경하지 않고 명시적으로 캡처해야 합니다.

```
func someFunction(a: inout Int) -> () -> Int {
    return { [a] in return a + 1 }
}
```

in-out 매개변수를 캡처하고 변경이 필요한 경우, 함수가 반환하기 전에 모든 변경이 완료되었는지 확인하는 멀티 스레드 코드와 같이 명시적으로 지역 복사(local copy)를 사용합니다.

```
func multithreadedFunction(queue: DispatchQueue, x: inout Int) {
    // Make a local copy and manually copy it back.
    var localX = x
    defer { x = localX }

    // Operate on localX asynchronously, then wait before returning.
    queue.async { someMutatingOperation(&localX) }
    queue.sync {}
}
```

in-out 매개변수에 대한 자세한 설명과 예시는 In-Out 매개변수 (In-Out Parameters)를 참고바랍니다.

Borrowing 매개변수와 Consuming 매개변수 (Borrowing and Consuming Parameters)

기본적으로 Swift는 함수 호출 전반에 걸쳐 객체의 수명을 자동으로 관리하고, 필요할 때 값을 복사하는 규칙을 사용합니다. 기본 규칙은 대부분 상황에서 오버헤드를 최소화하도록 설계되어 있습니다 - 더 구체적인 제어를 원하면, `borrowing`이나 `consuming` 매개변수 수정자를 적용할 수 있습니다. 이 경우에 `copy`를 사용해 복사 작업을 명시적으로 표시합니다. 또한 복사 불가능한 타입의 값은 borrowing이나 consuming으로 전달되어야 합니다.

기본 규칙을 사용하는 것과 상관없이, Swift는 객체 수명과 소유권이 모든 상황에서 올바르게 관리되도록 보장합니다. 이 매개변수 수정자는 정확성이 아닌 특정 사용 패턴의 상대적 효율성에만 영향을 줍니다.

`borrowing` 수정자는 함수가 매개변수 값을 유지하지 않음을 나타냅니다. 이 경우에 호출자는 객체의 소유권과 객체의 수명에 대한 책임을 유지합니다. `borrowing`을 사용하면 함수가 객체를 일시적으로만 사용할 때 오버헤드를 최소화합니다.

```
// `isLessThan` does not keep either argument
func isLessThan(lhs: borrowing A, rhs: borrowing A) -> Bool {
    ...
}
```

예를 들어, 전역 변수에 값을 저장하기 위해 함수가 매개변수의 값을 유지해야 하는 경우 - 값을 명시적으로 복사하기 위해 `copy`를 사용합니다.

```
// As above, but this `isLessThan` also wants to record the smallest value
func isLessThan(lhs: borrowing A, rhs: borrowing A) -> Bool {
    if lhs < storedValue {
        storedValue = copy lhs
    } else if rhs < storedValue {
        storedValue = copy rhs
    }
    return lhs < rhs
}
```

반대로, `consuming` 매개변수 수정자는 함수가 값의 소유권을 가지고 있고, 함수가 반환하기 전에 값을 저장하거나 파기하는 책임이 있음을 나타냅니다.

```
// `store` keeps its argument, so mark it `consuming`
func store(a: consuming A) {
    someGlobalVariable = a
}
```

`consuming`을 사용하면 호출자가 함수 호출 후에 더 이상 객체를 사용할 필요가 없을 때 오버헤드를 최소화합니다.

```
// Usually, this is the last thing you do with a value
store(a: value)
```

함수 호출 후에 복사 가능한 객체를 사용하려면, 컴파일러는 자동으로 함수 호출 전에 객체의 복사본을 만듭니다.

```
// The compiler inserts an implicit copy here
store(a: someValue)  // This function consumes someValue
print(someValue)  // This uses the copy of someValue
```

`inout`과 다르게, `borrowing`과 `consuming` 매개변수는 함수를 호출할 때 특별한 표기법이 필요하지 않습니다:

```
func someFunction(a: borrowing A, b: consuming B) { ... }

someFunction(a: someA, b: someB)
```

`borrowing`이나 `consuming`를 명시적으로 사용하는 것은 런타임 소유권 관리의 오버헤드를 더 엄격하게 관리하려는 의도를 나타냅니다. 복사는 예기치 않은 런타임 소유권 동작을 야기할 수 있으므로, 이 수정자 중 하나로 표시된 매개변수는 명시적인 `copy` 키워드를 사용하지 않으면 복사할 수 없습니다:

```
func borrowingFunction1(a: borrowing A) {
    // Error: Cannot implicitly copy a
    // This assignment requires a copy because
    // `a` is only borrowed from the caller.
    someGlobalVariable = a
}

func borrowingFunction2(a: borrowing A) {
    // OK: Explicit copying works
    someGlobalVariable = copy a
}

func consumingFunction1(a: consuming A) {
```

```
    // Error: Cannot implicitly copy a
    // This assignment requires a copy because
    // of the following `print`
    someGlobalVariable = a
    print(a)
}

func consumingFunction2(a: consuming A) {
    // OK: Explicit copying works regardless
    someGlobalVariable = copy a
    print(a)
}

func consumingFunction3(a: consuming A) {
    // OK: No copy needed here because this is the last use
    someGlobalVariable = a
}
```

특별한 종류의 매개변수 (Special Kinds of Parameters)

매개변수는 무시될 수 있고 다음의 형식을 사용하여 가변값을 가지거나 기본값을 제공할 수 있습니다:

```
_ : <#parameter type#>
<#parameter name#>: <#parameter type#>...
<#parameter name#>: <#parameter type#> = <#default argument value#>
```

언더바(_) 매개변수는 명시적으로 무시되고 함수의 본문 내에서 접근할 수 없습니다.

기본 타입 이름 바로 뒤에 세 개의 점(`...`)이 오는 매개변수는 가변 매개변수(variadic parameter)입니다. 가변 매개변수 다음에 오는 매개변수는 인자 레이블이 있어야 합니다. 함수는 여러 개의 가변 매개변수를 가질 수 있습니다. 가변 매개변수는 기본 타입 이름의 요소를 포함하는 배열로 처리합니다. 예를 들어 가변 매개변수 `Int...`는 `[Int]`로 처리됩니다. 가변 매개변수 사용 예시는 가변 매개변수 (Variadic Parameters)를 참고바랍니다.

매개변수 타입 뒤에 등호(=)와 표현식은 주어진 표현식의 기본값을 가지는 것으로 간주됩니다. 함수가 호출될 때 주어진 표현식은 평가됩니다. 함수를 호출할 때 매개변수가 생략되면 기본값이 대신 사용됩니다.

```
func f(x: Int = 42) -> Int { return x }
f()       // Valid, uses default value
f(x: 7)   // Valid, uses the value provided
f(7)      // Invalid, missing argument label
```

특별한 종류의 메서드 (Special Kinds of Methods)

열거형이나 구조체 메서드에서 `self`를 수정하는 메서드는 `mutating` 선언 수정자로 표시되어야 합니다.

상위 클래스 메서드를 재정의한 메서드는 `override` 선언 수정자로 표시되어야 합니다. `override` 수정자 없이 메서드를 재정의하거나 상위 클래스 메서드를 재정의하지 않는 메서드에 `override` 수정자를 사용하면 컴파일 오류가 발생합니다.

타입의 인스턴스가 아닌 타입과 관련된 메서드는 열거형과 구조체의 경우 `static` 선언 수정자로 표시되어야 하고 클래스의 경우 `static`이나 `class` 선언 수정자로 표시되어야 합니다. `class` 선언 수정자로 표시된 클래스 타입 메서드는 하위 클래스 구현에서 재정의할 수 있습니다; `class final`이나 `static`으로 표시된 클래스 타입 메서드는 재정의할 수 없습니다.

특별한 이름의 메서드 (Methods with Special Names)

특별한 이름을 가지는 메서드는 함수 호출 구문을 위한 편의 문법(syntactic sugar)을 가능하게 합니다. 타입을 이러한 메서드 중 하나를 정의하면, 타입의 인스턴스를 함수 호출 구문에서 사용할 수 있습니다. 이런 함수 호출은 해당 인스턴스의 특별한 이름을 가진 메서드 중 하나를 호출합니다.

클래스, 구조체, 열거형 타입은 dynamicCallable에서 설명한대로 `dynamicallyCall(withArguments:)` 메서드나 `dynamicallyCall(withKeywordArguments:)` 메서드를 정의하거나, 아래에서 설명한대로 call-as-function 메서드를 정의하여 함수 호출 구문을 제공할 수 있습니다. 타입이 call-as-function 메서드와 `dynamicCallable` 속성에 의해 사용되는 메서드 중 하나를 모두 정의하면, 컴파일러는 두 메서드를 사용할 수 있는 환경에서 call-as-function 메서드를 우선시합니다.

call-as-function 메서드의 이름은 `callAsFunction()`이나 레이블이 없어도 되지만 추가해도 되는 `callAsFunction(`으로 시작하는 다른 이름도 있습니다 -- 예를 들어 `callAsFunction(_:_:)`와 `callAsFunction(something:)`도 call-as-function 메서드 이름으로 유효합니다.

다음의 함수 호출은 동일합니다:

```
struct CallableStruct {
    var value: Int
    func callAsFunction(_ number: Int, scale: Int) {
        print(scale * (number + value))
    }
}
let callable = CallableStruct(value: 100)
callable(4, scale: 2)
callable.callAsFunction(4, scale: 2)
// Both function calls print 208.
```

call-as-function 메서드와 `dynamicCallable` 속성의 메서드는 타입 시스템에 얼마나 많은 정보를 인코딩하는지와 런타임에 얼마나 많은 동적 동작이 가능한지에 대해 서로 다른 장단점을 가집니다. call-as-function 메서드를 선언할 때, 인자의 수와 인자의 타입과 레이블을 지정합니다. `dynamicCallable` 속성의 메서드는 인자의 배열을 보유하기 위해 사용된 타입만 지정합니다.

call-as-function 메서드나 `dynamicCallable` 속성의 메서드를 정의하는 것은 함수 호출 표현식 외 다른 컨텍스트에서 타입의 인스턴스를 함수처럼 사용할 수 없습니다. 예를 들어:

```
let someFunction1: (Int, Int) -> Void = callable(_:scale:)  // Error
let someFunction2: (Int, Int) -> Void = callable.callAsFunction(_:scale:)
```

`subscript(dynamicMember:)` 서브스크립트는 dynamicMemberLookup에서 설명한대로 멤버 조회를 위한 편의 문법을 활성화 합니다.

오류를 던지는 함수와 메서드 (Throwing Functions and Methods)

오류를 던질 수 있는 함수와 메서드는 `throws` 키워드로 표시되어야 합니다. 이러한 함수와 메서드를 *던지는 함수(throwing functions)*와 *던지는 메서드(throwing methods)*라고 합니다. 형식은 다음과 같습니다:

```
func <#function name#>(<#parameters#>) throws -> <#return type#> {
   <#statements#>
}
```

특정 오류 타입을 던지는 함수는 다음의 형식을 가집니다:

```
func <#function name#>(<#parameters#>) throws(<#error type#>) -> <#return type#> {
   <#statements#>
}
```

오류를 던지는 함수나 메서드 호출은 `try`이나 `try!` 표현식으로 래핑되어야 합니다 (즉, `try`이나 `try!` 연산자의 범위).

함수의 타입에는 오류를 던질 수 있는지와 어떤 타입의 오류를 던지는지 포함합니다. 예를 들어, 이러한 하위 타입 관계는 오류를 던지는 함수가 예상되는 컨텍스트에서 오류를 던지지 않는 함수를 사용할 수 있다는 의미입니다. 던지는 함수의 타입에 대한 자세한 내용은 함수 타입 (Function Type)을 참고바랍니다. 명시적 타입의 오류를 다루는 예시는 오류 타입 지정 (Specifying the Error Type)을 참고바랍니다.

함수가 오류를 던질 수 있는지 여부만으로 함수를 오버로드할 수 없습니다. 이 말은 함수 매개변수가 오류를 던질 수 있는 여부에 따라 함수를 오버로드 할 수는 있습니다.

오류를 던지는 메서드는 오류를 던지지 않는 메서드를 재정의할 수 없고, 오류를 던지는 메서드는 오류를 던지지 않는 메서드에 대한 프로토콜 요구사항을 충족할 수 없습니다. 이 말은 오류를 던지지 않는 메서드는 오류를 던지는 메서드를 재정의할 수 있고, 오류를 던지지 않는 메서드는 오류를 던지는 메서드에 대한 프로토콜 요구사항을 충족할 수 있습니다.

오류를 다시 던지는 함수와 메서드 (Rethrowing Functions and Methods)

함수나 메서드는 함수 타입의 매개변수 중 하나가 오류를 던지는 경우에만 자신도 오류를 던질 수 있음을 나타내기 위해 `rethrows` 키워드로 선언할 수 있습니다. 이러한 함수와 메서드를 *다시 던지는 함수(rethrowing functions)*와 *다시 던지는 메서드(rethrowing methods)*라고 합니다. 다시 던지는 함수와 메서드는 적어도 하나의 던지는 함수 매개변수를 가져야 합니다.

```
func someFunction(callback: () throws -> Void) rethrows {
    try callback()
}
```

오류를 다시 던지는 함수나 메서드는 `catch` 절 안에만 `throw` 구문을 포함할 수 있습니다. 이렇게 하면 `do-catch` 구문 내에서 오류를 던지는 함수를 호출하고 `catch` 절에서 다른 오류를 발생시켜 오류를 처리할 수 있습니다. 또한 `catch` 절은 다시 던지는 함수의 매개변수 중 하나에서 발생한 오류만 처리해야 합니다. 예를 들어 다음은 `catch` 절이 `alwaysThrows()`에서 던져진 오류를 처리하므로 유효하지 않습니다.

```
func alwaysThrows() throws {
    throw SomeError.error
}
func someFunction(callback: () throws -> Void) rethrows {
    do {
        try callback()
        try alwaysThrows()  // Invalid, alwaysThrows() isn't a throwing parameter
    } catch {
        throw AnotherError.error
    }
}
```

오류를 던지는 메서드는 오류를 다시 던지는 메서드를 재정의할 수 없고, 오류를 던지는 메서드는 오류를 다시 던지는 메서드에 대한 프로토콜 요구사항을 충족할 수 없습니다. 이 말은 오류를 다시 던지는 메서드는 오류를 던지는 메서드를 재정의할 수 있고, 오류를 다시 던지는 메서드는 오류를 던지는 메서드에 대한 프로토콜 요구사항을 충족할 수 있습니다.

오류를 다시 던지는 것의 대안은 제네릭 코드에서 특정 오류 타입을 던지는 것입니다. 예를 들어:

```
func someFunction<E: Error>(callback: () throws(E) -> Void) throws(E) {
    try callback()
}
```

오류를 전파하는 이러한 방식은 오류에 대한 타입 정보를 유지합니다. 그러나 `rethrows` 함수와 다르게, 이 방식은 동일한 타입의 오류를 던지도록 함수를 보호하지 않습니다.

비동기 함수와 메서드 (Asynchronous Functions and Methods)

비동기적으로 실행되는 함수와 메서드는 `async` 키워드로 표시되어야 합니다. 이 함수와 메서드를 *비동기 함수(asynchronous functions)*와 *비동기 메서드(asynchronous methods)*라고 합니다. 형식은 다음과 같습니다:

```
func <#function name#>(<#parameters#>) async -> <#return type#> {
   <#statements#>
}
```

비동기 함수나 메서드에 대한 호출은 `await` 표현식으로 래핑되어야 합니다 -- 즉, `await` 연산자 범위 안에 있어야 합니다.

`async` 키워드는 함수 타입의 일부이고, 동기 함수는 비동기 함수의 하위 타입입니다. 결과적으로 비동기 함수 컨텍스트에서 동기 함수를 사용할 수 있습니다. 예를 들어 비동기 메서드를 동기 메서드로 재정의할 수 있으며, 동기 메서드는 비동기 메서드가 필요한 프로토콜 요구사항을 충족할 수 있습니다.

비동기 함수인지 아닌지에 따라 함수를 오버로드할 수 있습니다. 호출 부분에서 컨텍스트에 따라 사용될 오버로드를 결정합니다: 비동기 컨텍스트에서는 비동기 함수가 사용되고, 동기 컨텍스트에서는 동기 함수가 사용됩니다.

비동기 메서드는 동기 메서드를 재정의할 수 없으며, 비동기 메서드는 동기 메서드에 대한 프로토콜 요구사항을 충족할 수 없습니다. 이 말은, 동기 메서드는 비동기 메서드를 재정의할 수 있으며, 동기 메서드는 비동기 메서드에 대한 프로토콜 요구사항을 충족할 수 있습니다.

반환하지 않는 함수 (Functions that Never Return)

Swift는 `Never` 타입을 정의하여 함수나 메서드가 호출자에게 반환하지 않음을 나타냅니다. `Never` 반환 타입이 있는 함수와 메서드는 *비반환(nonreturning)*이라고 합니다. 비반환 함수와 메서드는 복구할 수 없는 오류를 발생하거나 무한으로 계속되는 작업을 시작합니다. 이것은 호출 직후에 실행될 코드가 실행되지 않음을 뜻합니다. 비반환 함수나 메서드인 경우에도 적절한 `catch` 블록으로 프로그램 제어를 전달할 수 있습니다.

비반환 함수나 메서드는 Guard 구문 (Guard Statement)에서 설명한대로 guard 구문의 `else` 절을 끝낼 수 있습니다.

비반환 메서드를 재정의할 수 있지만 새로운 메서드는 반환 타입과 비반환 동작을 유지해야 합니다.

Grammar of a function declaration

function-declaration → function-head function-name generic-parameter-clause? function-signature generic-where-clause? function-body?

function-head → attributes? declaration-modifiers? `func` function-name → identifier | operator

function-signature → parameter-clause `async`? throws-clause? function-result? function-signature → parameter-clause `async`? `rethrows` function-result? function-result → `->` attributes? type function-body → code-block

parameter-clause → `(` `)` | `(` parameter-list `,`? `)` parameter-list → parameter | parameter `,` parameter-list parameter → external-parameter-name? local-parameter-name parameter-type-annotation default-argument-clause? parameter → external-parameter-name? local-parameter-name parameter-type-annotation parameter → external-parameter-name? local-parameter-name parameter-type-annotation `...`

external-parameter-name → identifier local-parameter-name → identifier parameter-type-annotation → `:` attributes? parameter-modifier? type parameter-modifier → `inout` | `borrowing` | `consuming` default-argument-clause → = expression

열거형 선언 (Enumeration Declaration)

*열거형 선언(enumeration declaration)*은 프로그램에 열거형 타입을 도입합니다.

열거형 선언은 두 가지 기본 형식을 가지고 있고 `enum` 키워드를 사용하여 선언합니다. 두 형식 중 하나를 사용하여 선언된 열거형의 본문은 열거형 케이스(enumeration case)라고 불리는 0개 이상의 값과 연산 프로퍼티, 인스턴스 메서드, 타입 메서드, 이니셜라이저, 타입 별칭, 다른 열거형, 구조체, 클래스, 액터 선언을 포함할 수 있습니다. 열거형 선언은 디이니셜라이저나 프로토콜 선언을 포함할 수 없습니다.

열거형 타입은 여러 프로토콜을 채택할 수 있지만 클래스, 구조체, 다른 열거형을 상속할 수 없습니다.

클래스나 구조체와 다르게 열거형 타입은 암시적으로 제공된 기본 이니셜라이저를 가지지 않습니다; 모든 이니셜라이저는 명시적으로 선언해야 합니다. 이니셜라이저는 열거형 내에 다른 이니셜라이저로 위임할 수 있지만, 초기화 과정은 이니셜라이저에서 열거형 케이스 중 하나를 `self`에 할당한 후에 완료되어야만 합니다.

구조체와 비슷하지만 클래스와 달리 열거형은 값 타입입니다; 열거형의 인스턴스는 변수나 상수에 할당될 때나 함수 호출의 인자로 전달될 때 복사됩니다. 값 타입에 대한 자세한 내용은 구조체와 열거형은 값 타입 (Structures and Enumerations Are Value Types)을 참고바랍니다.

확장 선언 (Extension Declaration)에서 설명한대로 확장 선언으로 열거형 타입의 동작을 확장할 수 있습니다.

임의의 타입을 가지는 케이스가 있는 열거형 (Enumerations with Cases of Any Type)

다음의 형식은 임의의 타입을 가지는 열거형 케이스를 포함하는 열거형 타입을 선언합니다:

```
enum <#enumeration name#>: <#adopted protocols#> {
    case <#enumeration case 1#>
    case <#enumeration case 2#>(<#associated value types#>)
}
```

이 형식으로 선언된 열거형을 다른 프로그래밍 언어에서는 *태그된 유니온(discriminated unions)*이라고 합니다.

이 형식에서 각 케이스 블록은 `case` 키워드 다음에 콤마로 구분된 하나 이상의 열거형 케이스로 구성됩니다. 각 케이스의 이름은 고유해야 합니다. 각 케이스는 주어진 타입의 값을 저장하도록 지정할 수도 있습니다. 이러한 타입은 케이스의 이름 바로 다음에 연관값 타입(associated value type) 튜플로 지정합니다.

연관값을 저장하는 열거형 케이스는 지정된 연관값을 사용하여 열거형의 인스턴스를 생성하는 함수로 사용할 수 있습니다. 함수와 마찬가지로 코드에 열거형 케이스에 대한 참조를 가져오고 적용할 수 있습니다.

```
enum Number {
    case integer(Int)
    case real(Double)
}
let f = Number.integer
// f is a function of type (Int) -> Number

// Apply f to create an array of Number instances with integer values
let evenInts: [Number] = [0, 2, 4, 6].map(f)
```

연관값 타입을 사용하는 케이스의 자세한 내용과 예시는 연관값 (Associated Values)을 참고바랍니다.

간접 참조 열거형 (Enumerations with Indirection)

열거형은 재귀적 구조를 가질 수 있습니다. 이 말은 열거형 타입 자체의 인스턴스인 연관값을 사용하는 케이스를 가질 수 있다는 의미입니다. 그러나 열거형 타입의 인스턴스는 값 시맨틱(value semantic)을 가집니다. 이것은 메모리에 고정된 레이아웃을 가지고 있다는 의미입니다. 재귀를 지원하려면 컴파일러는 간접 참조 계층을 삽입해야 합니다.

특정 열거형 케이스에 대한 간접 참조를 사용하려면, `indirect` 선언 수정자로 표시해야 합니다. 간접 케이스는 연관값을 가지고 있어야 합니다.

```
enum Tree<T> {
    case empty
    indirect case node(value: T, left: Tree, right: Tree)
}
```

연관값을 가진 열거형의 모든 케이스에 대해 간접 참조를 사용하려면 전체 열거형을 `indirect` 수정자로 표시합니다 -- 이것은 열거형 케이스에 각각 `indirect` 수정자를 표시해야 될 케이스를 많이 포함하는 경우 편리합니다.

`indirect` 수정자로 표시된 열거형은 연관값을 가진 케이스와 연관값을 가지지 않는 케이스를 혼합해 포함할 수 있습니다. 이 말은 `indirect` 수정자로 표시된 케이스는 포함할 수 없다는 의미입니다.

원시값 타입의 케이스를 가진 열거형 (Enumerations with Cases of a Raw-Value Type)

다음의 형식은 동일한 기본 타입의 열거형 케이스를 포함하는 열거형 타입을 선언합니다:

```
enum <#enumeration name#>: <#raw-value type#>, <#adopted protocols#> {
    case <#enumeration case 1#> = <#raw value 1#>
    case <#enumeration case 2#> = <#raw value 2#>
}
```

이 형식에서 각 케이스 블록은 `case` 키워드 다음에 콤마로 구분된 하나 이상의 열거형 케이스로 구성됩니다. 첫 번째 형식의 케이스와 달리 각 케이스는 동일한 기본 타입의 *원시값(raw value)*이라는 기본값을 가집니다. 이러한 값의 타입은 *원시값 타입(raw-value type)*으로 지정되고 정수, 부동소수점 숫자, 문자열, 단일 문자를 나타내야 합니다. 특히 *원시값 타입(raw-value type)*은 `Equatable` 프로토콜과 다음 프로토콜 중 하나를 준수해야 합니다: 정

수 리터럴에 대한 `ExpressibleByIntegerLiteral`, 부동소수점 리터럴에 대한 `ExpressibleByFloatLiteral`, 여러 문자를 포함하는 문자열 리터럴에 대한 `ExpressibleByStringLiteral`, 단일 문자만 포함하는 문자열 리터럴에 대한 `ExpressibleByUnicodeScalarLiteral` 또는 `ExpressibleByExtendedGraphemeClusterLiteral`. 각 케이스는 고유한 이름을 가지고 고유한 원시값이 할당되어야 합니다.

원시값 타입이 `Int`로 지정되고 케이스에 명시적으로 값을 할당하지 않으면, 이것은 암시적으로 값 `0`, `1`, `2` 등으로 할당합니다. 타입 `Int`의 각 할당되지 않은 케이스는 이전 케이스의 원시값에서 자동으로 증가된 원시값으로 할당됩니다.

```
enum ExampleEnum: Int {
    case a, b, c = 5, d
}
```

위의 예시에서 `ExampleEnum.a`의 원시값은 `0`이고 `ExampleEnum.b`의 값은 `1`입니다. 그리고 `ExampleEnum.c`의 값은 명시적으로 `5`로 설정되어 있으므로 `ExampleEnum.d`의 값은 `5`에서 자동으로 증가되어 `6`입니다.

원시값 타입이 `String`으로 지정되고 명시적으로 케이스에 값을 할당하지 않으면, 각 할당되지 않은 케이스는 암시적으로 케이스의 이름과 동일한 텍스트 문자열로 할당됩니다.

```
enum GamePlayMode: String {
    case cooperative, individual, competitive
}
```

위의 예시에서 `GamePlayMode.cooperative`의 원시값은 `"cooperative"`, `GamePlayMode.individual`의 원시값은 `"individual"`, `GamePlayMode.competitive`의 원시값은 `"competitive"`입니다.

암시적으로 원시값 타입의 케이스를 가지는 열거형은 Swift 표준 라이브러리에 정의된 `RawRepresentable` 프로토콜을 준수합니다. 결과적으로 `rawValue` 프로퍼티를 가지고 있고 `init?(rawValue: RawValue)`인 실패 가능한 이니셜라이저를 가집니다. `ExampleEnum.b.rawValue`처럼 열거형 케이스의 원시값에 접근하기 위해 `rawValue` 프로퍼티를 사용할 수 있습니다. 옵셔널 케이스를 반환하는 `ExampleEnum(rawValue: 5)`와 같이 열거형의 실패 가능한 이니셜라이저를 호출하여 해당 케이스를 찾기 위해 원시값을 사용할 수도 있습니다. 원시값 타입이 있는 케이스의 자세한 내용과 예시는 원시값 (Raw Values)을 참고바랍니다.

열거형 케이스 접근 (Accessing Enumeration Cases)

열거형 타입의 케이스를 참조하기 위해 `EnumerationType.enumerationCase`와 같이 점(`.`) 구문을 사용합니다. 열거형 타입을 컨텍스트에서 추론할 수 있으면 열거형 문법 (Enumeration Syntax)과 암시적 멤버 표현식 (Implicit Member Expression)에서 설명한대로 점은 그대로 유지한 채 생략할 수 있습니다.

열거형 케이스의 값을 확인하려면 스위치 구문에서 열거형 값 일치 (Matching Enumeration Values with a Switch Statement)에서 보았듯이 `switch` 구문을 사용합니다. 열거형 타입은 열거형 케이스 패턴 (Enumeration Case Pattern)에서 설명한대로 `switch` 구문의 케이스 블록에서 열거형 케이스 패턴과 일치됩니다.

Grammar of an enumeration declaration

enum-declaration → attributes? access-level-modifier? union-style-enum enum-declaration → attributes? access-level-modifier? raw-value-style-enum

union-style-enum → **`indirect`**? **`enum`** enum-name generic-parameter-clause? type-inheritance-clause? generic-where-clause? **`{`** union-style-enum-members? **`}`** union-style-enum-members →

union-style-enum-member union-style-enum-members? union-style-enum-member → declaration | union-style-enum-case-clause | compiler-control-statement union-style-enum-case-clause → attributes? `indirect`? `case` union-style-enum-case-list union-style-enum-case-list → union-style-enum-case | union-style-enum-case `,` union-style-enum-case-list union-style-enum-case → enum-case-name tuple-type? enum-name → identifier enum-case-name → identifier

raw-value-style-enum → `enum` enum-name generic-parameter-clause? type-inheritance-clause generic-where-clause? `{` raw-value-style-enum-members `}` raw-value-style-enum-members → raw-value-style-enum-member raw-value-style-enum-members? raw-value-style-enum-member → declaration | raw-value-style-enum-case-clause | compiler-control-statement raw-value-style-enum-case-clause → attributes? `case` raw-value-style-enum-case-list raw-value-style-enum-case-list → raw-value-style-enum-case | raw-value-style-enum-case `,` raw-value-style-enum-case-list raw-value-style-enum-case → enum-case-name raw-value-assignment? raw-value-assignment → = raw-value-literal raw-value-literal → numeric-literal | static-string-literal | boolean-literal

구조체 선언 (Structure Declaration)

*구조체 선언(structure declaration)*은 프로그램에 구조체 타입을 도입합니다. 구조체 선언은 `struct` 키워드를 사용하여 선언하고 아래의 형식을 가집니다:

```
struct <#structure name#>: <#adopted protocols#> {
   <#declarations#>
}
```

구조체의 본문은 0개 이상의 *선언(declarations)*을 포함합니다. 이러한 위 형식의 *선언(declarations)*은 저장 프로퍼티와 연산 프로퍼티, 타입 프로퍼티, 인스턴스 메서드, 타입 메서드, 이니셜라이저, 서브스크립트, 타입 별칭, 다른 구조체, 클래스, 액터, 열거형 선언을 포함할 수 있습니다. 구조체 선언은 디이니셜라이저나 프로토콜 선언을 포함할 수 없습니다. 여러 종류의 선언을 포함한 구조체에 대한 자세한 내용과 예시는 구조체와 클래스 (Structures and Classes)를 참고바랍니다.

구조체 타입은 여러 프로토콜을 채택할 수 있지만 클래스, 열거형, 다른 구조체를 상속할 수 없습니다.

이전에 선언된 구조체의 인스턴스를 생성하는 방법은 세 가지가 있습니다:

- 이니셜라이저 (Initializers)에서 설명한대로 구조체 내 선언된 이니셜라이저 중 하나를 호출합니다.
- 선언된 이니셜라이저가 없는 경우 구조체 타입의 멤버와이즈 이니셜라이저 (Memberwise Initializers for Structure Types)에서 설명한대로 구조체의 멤버별 이니셜라이저를 호출합니다.
- 선언된 이니셜라이저가 없고 구조체 선언의 모든 프로퍼티에 초기값이 주어진 경우 기본 이니셜라이저 (Default Initializers)에서 설명한대로 구조체의 기본 이니셜라이저를 호출합니다.

구조체에 선언된 프로퍼티 초기화 과정은 초기화 (Initialization)에 설명되어 있습니다.

구조체 인스턴스의 프로퍼티는 프로퍼티 접근 (Accessing Properties)에서 설명한대로 점(`.`) 구문을 사용하여 접근할 수 있습니다.

구조체는 값 타입 입니다; 변수나 상수에 할당되거나 함수 호출의 인자로 전달될 때 구조체의 인스턴스는 복사됩니다. 값 타입에 대한 자세한 내용은 구조체와 열거형은 값 타입 (Structures and Enumerations Are Value Types)을 참고바랍니다.

확장 선언 (Extension Declaration)에서 설명한대로 확장 선언으로 구조체 타입의 동작을 확장할 수 있습니다.

Grammar of a structure declaration

struct-declaration → attributes? access-level-modifier? `struct` struct-name generic-parameter-clause? type-inheritance-clause? generic-where-clause? struct-body struct-name → identifier struct-body → { struct-members? }

struct-members → struct-member struct-members? struct-member → declaration | compiler-control-statement

클래스 선언 (Class Declaration)

*클래스 선언(class declaration)*은 프로그램에 클래스 타입을 도입합니다. 클래스 선언은 `class` 키워드를 사용하여 선언하고 다음의 형식을 가집니다:

```
class <#class name#>: <#superclass#>, <#adopted protocols#> {
   <#declarations#>
}
```

클래스의 본문은 0개 이상의 *선언(declarations)*을 포함합니다. 이러한 위 형식의 *선언(declarations)*은 저장 프로퍼티와 연산 프로퍼티, 인스턴스 메서드, 타입 메서드, 이니셜라이저, 하나의 디이니셜라이저, 서브스크립트, 타입 별칭, 다른 클래스, 구조체, 액터, 열거형 선언을 포함할 수 있습니다. 클래스 선언은 프로토콜 선언을 포함할 수 없습니다. 여러 종류의 선언을 포함하는 클래스의 자세한 설명과 예시는 구조체와 클래스 (Structures and Classes)를 참고바랍니다.

클래스 타입은 *상위 클래스(superclass)*로 하나의 부모 클래스만 상속할 수 있지만, 프로토콜은 여러 개 채택할 수 있습니다. *상위 클래스(superclass)*는 위 형식의 *클래스 이름(class name)*과 콜론 다음에 첫 번째로 나타나고 다음으로 *채택된 프로토콜(adopted protocols)*이 나타납니다. 제네릭 클래스(generic class)는 다른 제네릭 클래스와 제네릭이 아닌 클래스를 상속할 수 있지만, 제네릭이 아닌 클래스(nongeneric class)는 다른 제네릭이 아닌 클래스에서만 상속할 수 있습니다. 콜론 뒤에 상위 제네릭 클래스의 이름을 작성할 때, 제네릭 매개변수 절을 포함하는 제네릭 클래스의 전체 이름을 포함해야 합니다.

이니셜라이저 선언 (Initializer Declaration)에서 설명한대로 클래스는 지정 이니셜라이저(designated initializers)와 편의 이니셜라이저(convenience initializers)를 가질 수 있습니다. 클래스의 지정 이니셜라이저는 모든 클래스의 선언된 프로퍼티가 초기화되어야 하고, 상위 클래스의 지정 이니셜라이저를 호출하기 전에 수행되어야 합니다.

클래스는 상위 클래스의 프로퍼티, 메서드, 서브스크립트, 이니셜라이저를 재정의할 수 있습니다. 재정의한 프로퍼티, 메서드, 서브스크립트, 지정 이니셜라이저는 `override` 선언 수정자로 표시되어야 합니다.

하위 클래스가 상위 클래스의 이니셜라이저 구현을 요구하려면 상위 클래스의 이니셜라이저를 `required` 선언 수정자로 표시해야 합니다. 하위 클래스의 해당 이니셜라이저 구현도 `required` 선언 수정자로 표시되어야 합니다.

*상위 클래스(superclass)*에 선언된 프로퍼티와 메서드가 현재 클래스에 의해 상속되더라도 *상위 클래스(superclass)*에 선언된 지정 이니셜라이저는 자동 이니셜라이저 상속 (Automatic Initializer Inheritance)에서 설명한대로 하위 클래스가 조건이 충족될 때만 상속됩니다. Swift 클래스는 범용 기본 클래스는 상속하지 않습니다.

이전에 선언된 클래스의 인스턴스를 생성하는 두 가지 방법이 있습니다:

- 이니셜라이저 (Initializers)에서 설명한대로 클래스 내 선언된 이니셜라이저 중 하나를 호출합니다.

- 선언된 이니셜라이저가 없고 클래스 선언에 모든 프로퍼티에 기본값이 주어진 경우 기본 이니셜라이저 (Default Initializers)에서 설명한대로 클래스의 기본 이니셜라이저를 호출합니다.

프로퍼티 접근 (Accessing Properties)에 설명한대로 점(`.`) 구문으로 클래스 인스턴스의 프로퍼티에 접근할 수 있습니다.

클래스는 참조 타입입니다; 클래스의 인스턴스는 변수나 상수에 할당되거나 함수 호출의 인자로 전달될 때 복사가 아닌 참조됩니다. 참조 타입에 대한 자세한 내용은 클래스는 참조 타입 (Classes Are Reference Types)을 참고바랍니다.

확장 선언 (Extension Declaration)에서 설명한대로 확장 선언으로 클래스 타입의 동작을 확장할 수 있습니다.

Grammar of a class declaration

class-declaration → attributes? access-level-modifier? `final`? `class` class-name generic-parameter-clause? type-inheritance-clause? generic-where-clause? class-body class-declaration → attributes? `final` access-level-modifier? `class` class-name generic-parameter-clause? type-inheritance-clause? generic-where-clause? class-body class-name → identifier class-body → `{` class-members? `}`

class-members → class-member class-members? class-member → declaration | compiler-control-statement

액터 선언 (Actor Declaration)

*액터 선언(actor declaration)*은 프로그램에 액터 타입을 도입됩니다. 액터 선언은 `actor` 키워드를 사용하여 선언되고 다음의 형식을 가집니다:

```
actor <#actor name#>: <#adopted protocols#> {
    <#declarations#>
}
```

액터의 본문에는 0개 이상의 *선언(declarations)*을 포함합니다. 이러한 위 형식의 *선언(declarations)*은 저장 프로퍼티와 연산 프로퍼티, 인스턴스 메서드, 타입 메서드, 이니셜라이저, 하나의 디이니셜라이저, 서브스크립트, 타입 별칭, 다른 클래스, 구조체, 열거형 선언을 포함할 수 있습니다. 여러 종류의 선언을 포함하는 액터의 자세한 설명과 예시는 액터 (Actors)를 참고바랍니다.

액터 타입은 프로토콜을 채택할 수 있지만, 클래스, 열거형, 구조체, 다른 액터를 상속할 수 없습니다. 그러나 `@objc` 속성으로 표시된 액터는 암시적으로 `NSObjectProtocol` 프로토콜을 준수하고 `NSObject`의 하위 타입으로 Objective-C 런타임에 노출됩니다.

이전에 선언된 액터의 인스턴스를 생성하는 두 가지 방법이 있습니다:

- 이니셜라이저 (Initializers)에서 설명한대로 액터 내 선언한 이니셜라이저 중 하나를 호출합니다.
- 선언된 이니셜라이저가 없고 액터 선언의 모든 프로퍼티에 초기값이 제공된 경우 기본 이니셜라이저 (Default Initializers)에서 설명한대로 액터의 기본 이니셜라이저를 호출합니다.

기본적으로 액터의 멤버는 해당 액터와 분리됩니다. 메서드의 본문이나 프로퍼티의 getter와 같은 코드는 해당 액터에서 실행됩니다. 액터 내의 코드는 이미 동일한 액터에서 실행되고 있기 때문에 동기적으로 상호작용할 수 있지만, 액터 외부의 코드는 이 코드가 다른 액터에서 비동기적으로 코드를 실행하고 있음을 나타내기 위해 `await`로 표시해야 합니다. 키 경로는 액터의 격리된 멤버를 참조할 수 없습니다. 액터에 격리된 저장 프로퍼티(Actor-isolated stored properties)는 동기 함수에 in-out 매개변수로 전달할 수 있지만, 비동기 함수에는 전달할 수 없습니다.

액터는 `nonisolated` 키워드로 선언된 비격리 멤버(nonisolated members)를 가질 수도 있습니다. 비격리 멤버는 액터 외부의 코드처럼 실행됩니다: 액터의 격리된 상태와 상호작용할 수 없으며 호출자는 이를 사용할 때 `await`로 표시하지 않습니다.

액터의 멤버는 비격리 멤버이거나 비동기 멤버인 경우에만 `@objc` 속성으로 표시할 수 있습니다.

액터의 선언된 프로퍼티를 초기화하는 과정은 초기화 (Initialization)에 설명되어 있습니다.

액터 인스턴스의 프로퍼티는 프로퍼티 접근 (Accessing Properties)에서 설명한대로 점(`.`) 구문을 사용하여 접근할 수 있습니다.

액터는 참조 타입입니다; 액터의 인스턴스는 변수나 상수에 할당되거나 함수 호출의 인자로 전달될 때 복사되지 않고 참조됩니다. 참조 타입에 대한 자세한 내용은 클래스는 참조 타입 (Classes Are Reference Types)을 참고바랍니다.

확장 선언 (Extension Declaration)에서 설명한대로 확장 선언을 사용하여 액터 타입의 동작을 확장할 수 있습니다.

Grammar of an actor declaration

actor-declaration → attributes? access-level-modifier? `actor` actor-name generic-parameter-clause? type-inheritance-clause? generic-where-clause? actor-body actor-name → identifier actor-body → { actor-members? }

actor-members → actor-member actor-members? actor-member → declaration | compiler-control-statement

프로토콜 선언 (Protocol Declaration)

*프로토콜 선언(protocol declaration)*은 프로그램에 프로토콜 타입을 도입합니다. 프로토콜 선언은 `protocol` 키워드를 사용하여 전역에 선언되고 다음의 형식을 가집니다:

```
protocol <#protocol name#>: <#inherited protocols#> {
   <#protocol member declarations#>
}
```

프로토콜 선언은 전역 범위에서 나타나거나 제네릭이 아닌 타입이나 제네릭이 아닌 함수 내에 중첩될 수 있습니다.

프로토콜의 본문은 0개 이상의 *프로토콜 멤버 선언(protocol member declarations)*을 포함하고, 이것은 프로토콜을 채택하는 모든 타입이 충족해야 하는 준수 요구사항을 설명합니다. 특히 프로토콜은 준수하는 타입이 특정 프로퍼티, 메서드, 이니셜라이저, 서브스크립트를 구현해야 한다고 선언할 수 있습니다. 프로토콜은 *연관 타입(associated types)*이라는 특별한 종류의 타입 별칭도 선언할 수 있으며, 이것은 프로토콜의 다양한 선언의 관계를 명시할 수 있습니다. 프로토콜 선언은 클래스, 구조체, 열거형, 다른 프로토콜 선언을 포함할 수 없습니다. *프로토콜 멤버 선언(protocol member declarations)*은 아래에 자세하게 설명되어 있습니다.

프로토콜 타입은 다른 프로토콜을 상속할 수 있습니다. 프로토콜 타입이 다른 프로토콜을 상속할 때, 다른 프로토콜의 요구사항이 통합되고 현재 프로토콜을 상속하는 모든 타입은 이 모든 요구사항을 준수해야 합니다. 프로토콜 상속을 사용하는 방법에 대한 예시는 프로토콜 상속 (Protocol Inheritance)을 참고바랍니다.

참고

프로토콜 합성 타입 (Protocol Composition Type)과 프로토콜 합성 (Protocol Composition)에서 설명한대로 프로토콜 합성 타입을 사용하여 여러 프로토콜의 준수 요구사항을 통합할 수도 있습니다.

이전에 선언된 타입의 확장 선언을 사용하여 프로토콜을 채택하고 프로토콜 준수를 추가할 수 있습니다. 확장에서 채택된 프로토콜의 모든 요구사항을 구현해야 합니다. 타입이 이미 모든 요구사항을 구현한 경우, 확장 선언의 본문을 비워둘 수 있습니다.

기본적으로 프로토콜을 준수하는 타입은 프로토콜에 선언된 모든 프로퍼티, 메서드, 서브스크립트를 구현해야 합니다. 그러나 이 프로토콜 멤버 선언을 `optional` 선언 수정자로 표시하여 준수하는 타입의 구현이 선택 사항임을 명시할 수 있습니다. `optional` 수정자는 `objc` 속성으로 표시된 멤버와 `objc` 속성으로 표시된 프로토콜의 멤버에만 적용할 수 있습니다. 결과적으로 클래스 타입만 이 옵셔널 멤버 요구사항을 포함한 프로토콜을 채택하고 준수할 수 있습니다. `optional` 선언 수정자 사용에 대한 자세한 내용과 옵셔널 프로토콜 멤버에 접근하는 방법에 대한 자침 -- 예를 들어 타입이 이를 구현하는지 확실하지 않은 경우 -- 는 옵셔널 프로토콜 요구사항 (Optional Protocol Requirements)을 참고바랍니다.

열거형의 케이스는 타입 멤버에 대해 프로토콜 요구사항을 충족할 수 있습니다. 구체적으로 연관값이 없는 열거형 케이스는 `Self` 타입의 get-only 타입 변수에 대한 프로토콜 요구사항을 충족하고, 연관값이 있는 열거형 케이스는 매개변수와 인자 레이블이 케이스의 연관값과 일치하는 `Self`를 반환하는 함수에 대한 프로토콜 요구사항을 충족합니다. 예를 들어:

```
protocol SomeProtocol {
    static var someValue: Self { get }
    static func someFunction(x: Int) -> Self
}
enum MyEnum: SomeProtocol {
    case someValue
    case someFunction(x: Int)
}
```

프로토콜의 채택을 클래스 타입으로만 제한하려면 콜론 뒤에 상속된 프로토콜(inherited protocols) 목록에 `AnyObject` 프로토콜을 추가해야 합니다. 예를 들어 다음의 프로토콜은 클래스 타입에 의해서만 채택될 수 있습니다:

```
protocol SomeProtocol: AnyObject {
    /* Protocol members go here */
}
```

`AnyObject` 요구사항으로 표시된 프로토콜을 상속하는 모든 프로토콜은 마찬가지로 클래스 타입만 채택할 수 있습니다.

> **참고**
>
> `objc` 속성으로 표시된 프로토콜은 해당 프로토콜에 암시적으로 `AnyObject` 요구사항을 적용합니다; 명시적으로 `AnyObject` 요구사항을 프로토콜에 표시할 필요가 없습니다.

위 형식의 상속된 프로토콜(inherited protocols) 목록에 특정 클래스를 지정하여 프로토콜의 채택을 해당 클래스의 하위 클래스로 제한할 수도 있습니다. 예를 들어 다음 프로토콜은 `SomeBaseClass`의 하위 클래스만 채택할 수 있습니다:

```
class SomeBaseClass { /* ... */ }
protocol SomeRefinedProtocol: SomeBaseClass {
    /* Protocol members go here. */
}
```

프로토콜은 명명 타입 이므로 프로토콜을 타입으로 사용 (Protocols as Types)에서 설명한대로 다른 명명 타입과 동일한 위치의 코드에 나타날 수 있습니다. 그러나 프로토콜은 지정하는 요구사항에 대한 구현을 실제로 제공하지 않으므로, 프로토콜의 인스턴스를 생성할 수 없습니다.

위임 (Delegation)에서 설명한대로 프로토콜을 사용하여 클래스나 구조체의 위임이 구현해야 하는 메서드를 선언할 수 있습니다.

Grammar of a protocol declaration

protocol-declaration → attributes? access-level-modifier? `protocol` protocol-name type-inheritance-clause? generic-where-clause? protocol-body protocol-name → identifier protocol-body → { protocol-members? }

protocol-members → protocol-member protocol-members? protocol-member → protocol-member-declaration | compiler-control-statement

protocol-member-declaration → protocol-property-declaration protocol-member-declaration → protocol-method-declaration protocol-member-declaration → protocol-initializer-declaration protocol-member-declaration → protocol-subscript-declaration protocol-member-declaration → protocol-associated-type-declaration protocol-member-declaration → typealias-declaration

프로토콜 프로퍼티 선언 (Protocol Property Declaration)

프로토콜은 프로토콜 선언 본문에 *프로토콜 프로퍼티 선언(protocol property declaration)*을 포함하여 준수하는 타입이 프로퍼티를 구현해야 한다고 선언합니다. 프로토콜 프로퍼티 선언은 변수 선언의 특별한 형식을 가집니다:

```
var <#property name#>: <#type#> { get set }
```

다른 프로토콜 멤버 선언과 마찬가지로 이러한 프로퍼티 선언은 프로토콜을 준수하는 타입에 대한 getter와 setter 요구사항만 선언합니다. 결과적으로 선언된 프로토콜 내에 직접적으로 getter와 setter를 구현하지 않습니다.

getter와 setter 요구사항은 다양한 방법으로 준수하는 타입에서 충족될 수 있습니다. 프로퍼티 선언이 `get`과 `set` 키워드를 모두 포함하면, 준수하는 타입은 변수 저장 프로퍼티나 읽기와 쓰기가 모두 가능한 연산 프로퍼티 (즉, getter와 setter를 모두 구현하는 프로퍼티)로 구현할 수 있습니다. 그러나 해당 프로퍼티 선언은 상수 프로퍼티나 읽기-전용 연산 프로퍼티로 구현될 수 없습니다. 프로퍼티 선언이 `get` 키워드만 포함한다면, 모든 종류의 프로퍼티로 구현할 수 있습니다. 프로토콜의 프로퍼티 요구사항을 구현하는 준수 타입의 예시는 프로퍼티 요구사항 (Property Requirements)을 참고바랍니다.

프로토콜 선언에 타입 프로퍼티 요구사항을 선언하려면 `static` 키워드로 프로퍼티 선언을 표시합니다. 프로토콜을 준수하는 구조체와 열거형은 `static` 키워드로 프로퍼티를 선언하고 프로토콜을 준수하는 클래스는 `static`이나 `class` 키워드로 프로퍼티를 선언합니다. 구조체, 열거형, 클래스에 프로토콜 준수를 추가하는 확장은 확장하는 타입이 사용하는 것과 동일한 키워드를 사용합니다. 타입 프로퍼티 요구사항에 대해 기본 구현을 제공하는 확장은 `static` 키워드를 사용합니다.

변수 선언 (Variable Declaration)도 참고바랍니다.

Grammar of a protocol property declaration

protocol-property-declaration → variable-declaration-head variable-name type-annotation getter-setter-keyword-block

프로토콜 메서드 선언 (Protocol Method Declaration)

프로토콜은 프로토콜 선언 본문에 프로토콜 메서드 선언을 포함하여 준수하는 타입이 메서드를 구현해야 한다고 선언합니다. 프로토콜 메서드 선언은 두 가지를 제외하고 함수 선언과 동일한 형식을 가집니다: 함수 본문과 함수 선언의 일부로 기본 매개변수 값을 제공할 수 없습니다. 프로토콜의 메서드 요구사항을 구현하는 준수 타입에 대한 예시는 메서드 요구사항 (Method Requirements)을 참고바랍니다.

프로토콜 선언에 클래스 메서드나 정적 메서드 요구사항을 선언하려면 `static` 선언 수정자로 메서드 선언을 표시합니다. 프로토콜을 준수하는 구조체와 열거형은 `static` 키워드로 메서드를 선언하고 프로토콜을 준수하는 클래스는 `static`이나 `class` 키워드로 메서드를 선언합니다. 구조체, 열거형, 클래스에 프로토콜 준수를 추가하는 확장은 확장하는 타입이 사용하는 것과 동일한 키워드를 사용합니다. 타입 메서드 요구사항에 대한 기본 구현을 제공하는 확장은 `static` 키워드를 사용합니다.

함수 선언 (Function Declaration)도 참고바랍니다.

Grammar of a protocol method declaration

protocol-method-declaration → function-head function-name generic-parameter-clause? function-signature generic-where-clause?

프로토콜 이니셜라이저 선언 (Protocol Initializer Declaration)

프로토콜은 프로토콜 선언 본문에 프로토콜 이니셜라이저 선언을 포함하여 준수하는 타입이 이니셜라이저를 구현해야 한다고 선언합니다. 프로토콜 이니셜라이저 선언은 이니셜라이저의 본문을 포함하지 않는다는 점을 제외하고 이니셜라이저 선언과 동일한 형식을 가집니다.

준수하는 타입은 실패하지 않는 이니셜라이저(nonfailable initializer)나 `init!` 실패 가능한 이니셜라이저(failable initializer)를 구현하여 실패하지 않는 프로토콜 이니셜라이저 요구사항을 충족할 수 있습니다. 준수하는 타입은 모든 종류의 이니셜라이저 구현으로 실패 가능한 프로토콜 이니셜라이저 요구사항을 충족할 수 있습니다.

클래스가 프로토콜의 이니셜라이저 요구사항을 충족하기 위해 이니셜라이저를 구현할 때, 클래스에 `final` 선언 수정자로 표시되어 있지 않다면 이니셜라이저는 `required` 선언 수정자로 표시되어야 합니다.

이니셜라이저 선언 (Initializer Declaration)도 참고바랍니다.

Grammar of a protocol initializer declaration

protocol-initializer-declaration → initializer-head generic-parameter-clause? parameter-clause throws-clause? generic-where-clause? protocol-initializer-declaration → initializer-head generic-parameter-clause? parameter-clause `rethrows` generic-where-clause?

프로토콜 서브스크립트 선언 (Protocol Subscript Declaration)

프로토콜은 프로토콜 선언 본문에 프로토콜 서브스크립트 선언을 포함하여 준수하는 타입이 서브스크립트를 구현해야 한다고 선언합니다. 프로토콜 서브스크립트 선언은 서브스크립트 선언의 특별한 형식을 가집니다:

```
subscript (<#parameters#>) -> <#return type#> { get set }
```

서브스크립트 선언은 프로토콜을 준수하는 타입에 대한 최소한의 getter와 setter 구현 요구사항만 선언합니다. 서브스크립트 선언에 `get`과 `set` 키워드 모두 포함하면, 준수하는 타입은 getter와 setter 절 모두 구현해야 합니다. 서브

스크립트 선언이 `get` 키워드만 포함하면, 준수하는 타입은 최소한 getter 절은 구현해야 하고 선택적으로 setter 절을 구현할 수 있습니다.

프로토콜 선언에서 정적 서브스크립트 요구사항을 선언하려면 `static` 선언 수정자로 서브스크립트 선언을 표시합니다. 프로토콜은 준수하는 구조체와 열거형은 `static` 키워드로 서브스크립트를 선언하고 프로토콜을 준수하는 클래스는 `static`이나 `class` 키워드로 서브스크립트를 선언합니다. 구조체, 열거형, 클래스에 프로토콜 준수를 추가하는 확장은 확장하는 타입이 사용하는 것과 동일한 키워드를 사용합니다. 정적 서브스크립트 요구사항에 대한 기본 구현을 제공하는 확장은 `static` 키워드를 사용합니다.

서브스크립트 선언 (Subscript Declaration)도 참고바랍니다.

Grammar of a protocol subscript declaration

protocol-subscript-declaration → subscript-head subscript-result generic-where-clause? getter-setter-keyword-block

프로토콜 연관 타입 선언 (Protocol Associated Type Declaration)

프로토콜은 `associatedtype` 키워드를 사용하여 연관 타입을 선언합니다. 연관 타입은 프로토콜 선언의 일부분으로 사용되는 타입에 대한 별칭을 제공합니다. 연관 타입은 제네릭 매개변수 절의 타입 매개변수와 유사하지만 선언된 프로토콜의 `Self`와 연관됩니다. 해당 컨텍스트에서 `Self`는 프로토콜을 준수하는 최종 타입을 참조합니다. 더 자세한 내용과 예시는 연관 타입 (Associated Types)을 참고바랍니다.

프로토콜 선언에 제네릭 `where` 절을 사용하여 연관 타입을 재선언하지 않고 다른 프로토콜에서 상속된 연관 타입에 제약조건을 추가할 수 있습니다. 예를 들어 아래 `SubProtocol`의 선언은 동일합니다:

```
protocol SomeProtocol {
    associatedtype SomeType
}

protocol SubProtocolA: SomeProtocol {
    // This syntax produces a warning.
    associatedtype SomeType: Equatable
}

// This syntax is preferred.
protocol SubProtocolB: SomeProtocol where SomeType: Equatable { }
```

타입 별칭 선언 (Type Alias Declaration)도 참고바랍니다.

Grammar of a protocol associated type declaration

protocol-associated-type-declaration → attributes? access-level-modifier? `associatedtype` typealias-name type-inheritance-clause? typealias-assignment? generic-where-clause?

이니셜라이저 선언 (Initializer Declaration)

*이니셜라이저 선언(Initializer declaration)*은 프로그램에 클래스, 구조체, 열거형에 대한 이니셜라이저를 도입합니다. 이니셜라이저 선언은 `init` 키워드를 사용하여 선언하고 두 개의 기본 형식을 가지고 있습니다.

구조체, 열거형, 클래스 타입은 여러 개의 이니셜라이저를 가질 수 있지만, 클래스 이니셜라이저에 대한 규칙과 관련 동작은 다릅니다. 구조체와 열거형과 다르게 클래스는 두 종류의 이니셜라이저를 가집니다: 초기화 (Initialization)에서 설명한대로 지정 이니셜라이저(designated initializers)와 편의 이니셜라이저(convenience initializers)가 있습니다.

다음 형식은 구조체, 열거형, 클래스의 지정 이니셜라이저에 대한 이니셜라이저를 선언합니다:

```
init(<#parameters#>) {
   <#statements#>
}
```

클래스의 지정 이니셜라이저는 클래스의 모든 프로퍼티를 직접 초기화합니다. 지정 이니셜라이저는 같은 클래스의 다른 이니셜라이저를 호출할 수 없고, 클래스가 상위 클래스를 가지고 있다면 상위 클래스의 지정 이니셜라이저 중 하나를 호출해야 합니다. 클래스가 상위 클래스에서 어떤 프로퍼티를 상속한다면, 상위 클래스의 지정 이니셜라이저 중 하나를 현재 클래스에서 해당 프로퍼티를 설정하거나 수정하기 전에 호출해야 합니다.

지정 이니셜라이저는 클래스 선언의 컨텍스트에서만 선언될 수 있으므로 확장 선언을 사용하는 클래스에 추가할 수 없습니다.

구조체와 열거형에 이니셜라이저는 선언된 다른 이니셜라이저를 호출하여 초기화 과정의 일부나 전체를 위임할 수 있습니다.

클래스에 편의 이니셜라이저를 선언하려면 `convenience` 선언 수정자로 이니셜라이저 선언을 표시합니다.

```
convenience init(<#parameters#>) {
   <#statements#>
}
```

편의 이니셜라이저는 초기화 과정을 다른 편의 이니셜라이저나 클래스의 지정 이니셜라이저 중 하나로 위임할 수 있습니다. 이 말은 초기화 과정은 궁극적으로 클래스의 프로퍼티를 초기화하는 지정 이니셜라이저를 호출로 끝나야 한다는 의미입니다. 편의 이니셜라이저는 상위 클래스의 이니셜라이저를 호출할 수 없습니다.

지정 이니셜라이저와 편의 이니셜라이저에 `required` 선언 수정자로 지정하여 모든 하위 클래스가 이니셜라이저를 구현하도록 요구할 수 있습니다. 해당 이니셜라이저의 하위 클래스의 구현도 `required` 선언 수정자로 표시되어야 합니다.

기본적으로 상위 클래스에 선언된 이니셜라이저는 하위 클래스에 의해 상속되지 않습니다. 이 말은 하위 클래스가 기본값으로 모든 저장 프로퍼티를 초기화하고 자체 이니셜라이저를 정의하지 않으면, 모든 상위 클래스의 이니셜라이저를 상속합니다. 하위 클래스가 모든 상위 클래스의 지정 이니셜라이저를 재정의하면 상위 클래스의 편의 이니셜라이저를 상속합니다.

메서드, 프로퍼티, 서브스크립트와 마찬가지로 `override` 선언 수정자로 재정의된 지정 이니셜라이저를 표시해야 합니다.

> **참고**
>
> `required` 선언 수정자로 이니셜라이저를 표시하면, 하위 클래스에서 필수 이니셜라이저를 재정의할 때 `override` 수정자로 이니셜라이저를 표시하지 않아도 됩니다.

함수와 메서드처럼 이니셜라이저는 오류를 던지거나 다시 던질 수 있습니다. 그리고 함수와 메서드처럼 이니셜라이저의 매개변수 뒤에 `throws`나 `rethrows` 키워드를 사용하여 적절한 동작을 나타낼 수 있습니다. 마찬가지로 이니셜라이저는 비동기일 수 있으며 이것을 나타내기 위해 `async` 키워드를 사용합니다.

여러 타입 선언에서 이니셜라이저의 예시는 초기화 (Initialization)를 참고바랍니다.

실패 가능한 이니셜라이저 (Failable Initializers)

*실패 가능한 이니셜라이저(failable initializer)*는 이니셜라이저가 선언된 타입의 옵셔널 인스턴스나 암시적 언래핑된 옵셔널 인스턴스를 생성하는 이니셜라이저의 타입입니다. 결과적으로 실패 가능한 이니셜라이저는 초기화 실패를 나타내는 `nil`을 반환할 수 있습니다.

옵셔널 인스턴스를 생성하는 실패 가능한 이니셜라이저를 선언하려면 이니셜라이저 선언에서 `init` 키워드 뒤에 물음표를 붙입니다(`init?`). 암시적 언래핑 옵셔널 인스턴스를 생성하는 실패 가능한 이니셜라이저를 선언하려면 이니셜라이저 선언에서 `init` 키워드 뒤에 느낌표를 붙입니다(`init!`). 아래의 예시는 구조체의 옵셔널 인스턴스를 생성하는 실패 가능한 이니셜라이저 `init?`을 보여줍니다.

```
struct SomeStruct {
    let property: String
    // produces an optional instance of 'SomeStruct'
    init?(input: String) {
        if input.isEmpty {
            // discard 'self' and return 'nil'
            return nil
        }
        property = input
    }
}
```

결과의 선택성을 다루는 것을 제외하고 일반 이니셜라이저 호출과 동일하게 `init?`로 실패 가능한 이니셜라이저를 호출합니다.

```
if let actualInstance = SomeStruct(input: "Hello") {
    // do something with the instance of 'SomeStruct'
} else {
    // initialization of 'SomeStruct' failed and the initializer returned 'nil'
}
```

실패 가능한 이니셜라이저는 이니셜라이저 구현의 어느 위치에서든 `nil`을 반환할 수 있습니다.

실패 가능한 이니셜라이저는 모든 종류의 이니셜라이저에 위임할 수 있습니다. 일반 이니셜라이저는 다른 일반 이니셜라이저가나 `init!` 실패 가능한 이니셜라이저로 위임할 수 있습니다. 일반 이니셜라이저는 `super.init()!`처럼 상위 클래스 이니셜라이저의 결과를 강제 언래핑함으로써 `init?` 실패 가능한 이니셜라이저에 위임할 수 있습니다.

초기화 실패는 이니셜라이저 위임으로 전파됩니다. 특히 실패 가능한 이니셜라이저가 실패하고 `nil`을 반환하는 이니셜라이저에 위임하면, 위임된 이니셜라이저도 실패하고 암시적으로 `nil`을 반환합니다. 일반 이니셜라이저가 실패하고 `nil`을 반환하는 `init!` 실패 가능한 이니셜라이저에 위임하면, 런타임 오류가 발생합니다 (`nil` 값을 가지는 옵셔널을 언래핑하기 위해 `!` 연산자를 사용하는 것과 동일합니다).

실패 가능한 지정 이니셜라이저는 하위 클래스에서 모든 종류의 지정 이니셜라이저로 재정의될 수 있습니다. 일반 지정 이니셜라이저는 하위 클래스에서 일반 지정 이니셜라이저로만 재정의될 수 있습니다.

실패 가능한 이니셜라이저에 대한 자세한 내용과 예시는 실패 가능한 이니셜라이저 (Failable Initializers)을 참고바랍니다.

Grammar of an initializer declaration

initializer-declaration → initializer-head generic-parameter-clause? parameter-clause `async`? throws-clause? generic-where-clause? initializer-body initializer-declaration → initializer-head generic-parameter-clause? parameter-clause `async`? `rethrows` generic-where-clause? initializer-body initializer-head → attributes? declaration-modifiers? `init` initializer-head → attributes? declaration-modifiers? `init ?` initializer-head → attributes? declaration-modifiers? `init !` initializer-body → code-block

디이니셜라이저 선언 (Deinitializer Declaration)

*디이니셜라이저 선언(deinitializer declaration)*은 클래스 타입에 대한 디이니셜라이저를 선언합니다. 디이니셜라이저는 매개변수를 가지지 않고 다음의 형식을 가집니다:

```
deinit {
   <#statements#>
}
```

디이니셜라이저는 클래스 객체에 어떠한 참조도 없으면, 클래스 객체가 할당 해제되기 직전에 자동으로 호출됩니다. 디이니셜라이저는 클래스 선언의 본문 내에만 선언될 수 있고 - 클래스의 확장에는 선언될 수 없습니다 - 그리고 각 클래스는 디이니셜라이저를 하나만 가질 수 있습니다.

하위 클래스는 하위 클래스 객체가 할당 해제되기 직전에 암시적으로 호출되는 상위 클래스의 디이니셜라이저를 상속합니다. 하위 클래스 객체는 상속 체인의 모든 디이니셜라이저가 실행을 완료할 때까지 할당 해제되지 않습니다.

디이니셜라이저는 직접 호출할 수 없습니다.

클래스 선언에서 디이니셜라이저가 어떻게 사용되는지에 대한 예시는 소멸 (Deinitialization)를 참고바랍니다.

Grammar of a deinitializer declaration

deinitializer-declaration → attributes? `deinit` code-block

확장 선언 (Extension Declaration)

*확장 선언(extension declaration)*은 기존 타입의 동작을 확장할 수 있습니다. 확장 선언은 `extension` 키워드를 사용하여 선언하고 다음의 형식을 가집니다:

```
extension <#type name#> where <#requirements#> {
   <#declarations#>
}
```

확장 선언의 본문은 0개 이상의 *선언(declarations)*을 포함합니다. 위 형식의 *선언(declaration)은 연산 프로퍼티, 연산 타입 프로퍼티, 인스턴스 메서드, 타입 메서드, 이니셜라이저, 서브스크립트 선언, 클래스, 구조체, 열거형 선언을 포함할 수 있습니다. 확장 선언은 디이니셜라이저나 프로토콜 선언, 저장 프로퍼티, 프로퍼티 관찰자, 다른 확장 선언은 포함할 수 없습니다. 프로토콜 확장에서 선언은 `final`로 표시할 수 없습니다. 여러 종류의 선언을 포함한 확장에 대한 내용과 예시는 확장 (Extensions)을 참고바랍니다.

위 형식의 *타입 이름(type name)*이 클래스, 구조체, 열거형 타입이면, 확장은 해당 타입을 확장합니다. 위 형식의 *타입 이름(type name)*이 프로토콜 타입이면, 확장은 해당 프로토콜을 준수하는 모든 타입을 확장합니다.

제네릭 타입이나 연관 타입이 있는 프로토콜을 확장하는 확장 선언은 위 형식의 *요구사항(requirements)*을 포함할 수 있습니다. 확장된 타입이나 확장된 프로토콜을 준수하는 타입의 인스턴스가 위 형식의 *요구사항(requirements)*을 충족하면 인스턴스는 선언에 지정된 동작을 얻습니다.

확장 선언은 이니셜라이저 선언을 포함할 수 있습니다. 이 말은 확장하려는 타입이 다른 모듈에 정의되어 있으면, 이니셜라이저 선언은 해당 타입의 멤버가 올바르게 초기화되도록 해당 모듈에 이미 정의된 이니셜라이저로 위임해야 합니다.

기존 타입의 프로퍼티, 메서드, 이니셜라이저는 해당 타입의 확장에서 재정의될 수 없습니다.

확장 선언은 아래 형식의 *채택된 프로토콜(adopted protocols)*을 지정하여 기존 클래스, 구조체, 열거형 타입에 프로토콜 준수를 추가할 수 있습니다:

```
extension <#type name#>: <#adopted protocols#> where <#requirements#> {
   <#declarations#>
}
```

확장 선언은 기존 클래스에 클래스 상속을 추가할 수 없으므로 위 형식의 *타입 이름(type name)*과 콜론 뒤에 프로토콜의 리스트만 지정할 수 있습니다.

조건부 준수성 (Conditional Conformance)

요구사항이 충족될 때만 해당 타입의 인스턴스가 프로토콜을 준수하도록 제네릭 타입을 확장하여 조건부로 프로토콜을 준수하게 할 수 있습니다. 확장 선언에 *요구사항(requirements)*을 추가하여 조건부 프로토콜 준수를 추가합니다.

일부 제네릭 컨텍스트에서 재정의된 요구사항이 사용되지 않는 경우 (Overridden Requirements Aren't Used in Some Generic Contexts)

일부 제네릭 컨텍스트에서 조건부 프로토콜 준수에서 동작을 가져오는 타입은 항상 해당 프로토콜 요구사항의 구현을 사용하지 않습니다. 이 동작을 설명하기 위해 다음 예시는 두 프로토콜과 두 프로토콜을 조건부로 준수하는 제네릭 타입을 정의합니다.

```
protocol Loggable {
    func log()
}
extension Loggable {
    func log() {
        print(self)
    }
}

protocol TitledLoggable: Loggable {
    static var logTitle: String { get }
}
extension TitledLoggable {
    func log() {
        print("\(Self.logTitle): \(self)")
    }
}

struct Pair<T>: CustomStringConvertible {
    let first: T
    let second: T
```

```
    var description: String {
        return "(\(first), \(second))"
    }
}

extension Pair: Loggable where T: Loggable { }
extension Pair: TitledLoggable where T: TitledLoggable {
    static var logTitle: String {
        return "Pair of '\(T.logTitle)'"
    }
}

extension String: TitledLoggable {
    static var logTitle: String {
        return "String"
    }
}
```

`Pair` 구조체는 제네릭 타입이 `Loggable`이나 `TitledLoggable`을 각각 준수할 때마다 `Loggable`과 `TitledLoggable`을 준수합니다. 아래 예시에서 `oneAndTwo`는 `Pair<String>`의 인스턴스이고, `String`이 `TitledLoggable`을 준수하기 때문에 해당 인스턴스는 `TitledLoggable`을 준수합니다. `oneAndTwo`에서 `log()` 메서드를 직접 호출하면, 타이틀 문자열을 포함하는 특별한 버전이 사용됩니다.

```
let oneAndTwo = Pair(first: "one", second: "two")
oneAndTwo.log()
// Prints "Pair of 'String': (one, two)".
```

그러나 `oneAndTwo` 제네릭 컨텍스트에서 사용되거나 `Loggable` 프로토콜의 인스턴스로 사용되면, 특별한 버전이 사용되지 않습니다. Swift는 `Pair`가 `Loggable` 준수하는데 필요한 최소 요구사항만 참조하여 `log()`의 어떤 구현을 호출할지 선택합니다. 이러한 이유로 `Loggable` 프로토콜에 의해 제공된 기본 구현이 대신 사용됩니다.

```
func doSomething<T: Loggable>(with x: T) {
    x.log()
}
doSomething(with: oneAndTwo)
// Prints "(one, two)".
```

`log()`가 `doSomething(_:)`에 전달된 인스턴스에서 호출되면, 커스텀된 타이틀은 로깅된 문자열에서 생략됩니다.

프로토콜 준수는 중복되지 않아야 함 (Protocol Conformance Must Not Be Redundant)

구체적인 타입은 특정 프로토콜을 한 번만 준수할 수 있습니다. Swift는 중복 프로토콜 준수를 오류로 표시합니다. 이러한 오류는 두 가지 상황에서 발생할 수 있습니다. 첫 번째 상황은 동일한 프로토콜을 여러 번 명시적으로 준수하지만 요구사항이 다른 경우입니다. 두 번째 상황은 암시적으로 동일한 프로토콜을 여러 번 상속할 때 입니다. 이러한 상황은 아래 섹션에 설명되어 있습니다.

명시적 중복 해결 (Resolving Explicit Redundancy)

구체적인 타입에 여러 확장은 확장의 요구사항이 독립적이더라도 동일한 프로토콜에 대한 준수를 추가할 수 없습니다. 이 제한은 아래 예시에서 설명됩니다. 두 확장 선언은 `Int` 요소가 있는 배열과 `String` 요소가 있는 배열에 대해 `Serializable` 프로토콜의 조건부 준수를 추가합니다.

```
protocol Serializable {
    func serialize() -> Any
}

extension Array: Serializable where Element == Int {
    func serialize() -> Any {
        // implementation
    }
}
extension Array: Serializable where Element == String {
    func serialize() -> Any {
        // implementation
    }
}
// Error: Redundant conformance of 'Array<Element>' to protocol 'Serializable'.
```

여러 구체적인 타입을 기반으로 조건부 준수를 추가하려면, 각 타입이 준수하는 새로운 프로토콜을 생성하고 조건부 준수를 선언할 때 해당 프로토콜을 요구사항으로 사용합니다.

```
protocol SerializableInArray { }
extension Int: SerializableInArray { }
extension String: SerializableInArray { }

extension Array: Serializable where Element: SerializableInArray {
    func serialize() -> Any {
        // implementation
    }
}
```

암시적 중복 해결 (Resolving Implicit Redundancy)

구체적인 타입이 프로토콜을 조건부로 준수하면, 해당 타입은 동일한 요구사항을 가진 모든 부모 프로토콜을 암시적으로 준수합니다.

단일 부모에서 상속하는 두 프로토콜을 조건부로 준수하는 타입이 필요한 경우, 명시적으로 부모 프로토콜에 대한 준수를 선언합니다. 이것은 요구사항이 다른 부모 프로토콜을 암시적으로 두 번 준수하는 것을 방지합니다.

다음 예시는 `TitledLoggable`과 새로운 `MarkedLoggable` 프로토콜을 조건부 준수를 선언할 때 충돌을 막기 위해 `Array`의 조건부 준수를 `Loggable`로 명시적으로 선언합니다.

```
protocol MarkedLoggable: Loggable {
    func markAndLog()
}

extension MarkedLoggable {
    func markAndLog() {
        print("----------")
        log()
    }
}

extension Array: Loggable where Element: Loggable { }
extension Array: TitledLoggable where Element: TitledLoggable {
    static var logTitle: String {
        return "Array of '\(Element.logTitle)'"
    }
}
extension Array: MarkedLoggable where Element: MarkedLoggable { }
```

`Loggable`에 대한 조건부 준수성을 명시적으로 선언하는 확장이 없으면 다른 `Array` 확장이 암시적으로 이러한 선언을 생성하여 오류가 발생합니다:

```
extension Array: Loggable where Element: TitledLoggable { }
extension Array: Loggable where Element: MarkedLoggable { }
// Error: Redundant conformance of 'Array<Element>' to protocol 'Loggable'.
```

Grammar of an extension declaration

extension-declaration → attributes? access-level-modifier? `extension` type-identifier type-inheritance-clause? generic-where-clause? extension-body extension-body → { extension-members? }

extension-members → extension-member extension-members? extension-member → declaration | compiler-control-statement

서브스크립트 선언 (Subscript Declaration)

서브스크립트(subscript) 선언은 특정 타입에 대한 서브스크립트 지원을 추가할 수 있고, 일반적으로 컬렉션, 리스트, 시퀀스의 요소에 접근하기 위한 편리한 구문을 제공하기 위해 사용됩니다. 서브스크립트 선언은 `subscript` 키워드를 사용하여 선언되고 다음의 형식을 가집니다:

```
subscript (<#parameters#>) -> <#return type#> {
   get {
      <#statements#>
   }
   set(<#setter name#>) {
      <#statements#>
   }
}
```

서브스크립트 선언은 클래스, 구조체, 열거형, 확장, 프로토콜 선언의 컨텍스트에서만 나타날 수 있습니다.

위 형식의 *매개변수(parameters)*는 서브스크립트 표현식에서 해당 타입의 요소에 접근하기 위해 사용되는 하나 이상의 인덱스를 지정합니다 (예를 들어 표현식 `object[i]`에서 `i`). 요소에 접근하기 위해 사용되는 인덱스는 모든 타입이 될 수 있지만, 각 매개변수는 각 인덱스의 타입을 지정하기 위해 타입 주석을 포함해야 합니다. 위 형식의 *반환 타입(return type)*은 접근하는 요소의 타입을 지정합니다.

연산 프로퍼티와 마찬가지로 서브스크립트 선언은 접근한 요소의 값을 읽고 쓰기를 제공합니다. getter는 값을 읽기 위해 사용되고, setter는 값을 쓰기 위해 사용됩니다. setter 절은 선택 사항이며, getter만 필요하다면, 모든 절을 생략할 수 있고 요청된 값을 직접 반환할 수 있습니다. 이 말은 setter 절을 제공하면 getter 절을 제공해야 한다는 의미입니다.

위 형식의 *setter 이름(setter name)*과 둘러싸인 소괄호는 선택 사항입니다. setter 이름을 제공하면, setter의 매개변수 이름으로 사용됩니다. setter 이름을 제공하지 않으면, setter의 기본 매개변수 이름은 `newValue`입니다. setter의 매개변수 타입은 위 형식의 *반환 타입(return type)*과 동일합니다.

*매개변수(parameters)*나 *반환 타입(return type)*이 오버로드 하려는 것과 다르면, 선언된 타입의 서브스크립트 선언을 오버로드할 수 있습니다. 상위 클래스에서 상속된 서브스크립트 선언을 재정의할 수도 있습니다. 재정의하면 `override` 선언 수정자로 재정의된 서브스크립트 선언을 표시해야 합니다.

서브스크립트 매개변수는 두 가지를 제외하고 함수 매개변수와 동일한 규칙을 따릅니다. 기본적으로 서브스크립트에서 사용하는 매개변수는 함수, 메서드, 이니셜라이저와 다르게 인자 레이블을 가지지 않습니다. 그러나 함수, 메서드, 이니셜라이저에서 사용하는 동일한 구문을 사용하여 명시적으로 인자 레이블을 제공할 수 있습니다. 또한 서브스크립트는 in-out 매개변수를 가질 수 없습니다. 서브스크립트 매개변수는 특별한 종류의 매개변수 (Special Kinds of Parameters)에서 설명한 구문을 사용하여 기본값을 가질 수 있습니다.

프로토콜 서브스크립트 선언 (Protocol Subscript Declaration)에서 설명한대로 프로토콜 선언의 컨텍스트에서 서브스크립트를 선언할 수도 있습니다.

서브스크립트에 대한 자세한 내용과 서브스크립트 선언의 예시는 서브스크립트 (Subscripts)를 참고바랍니다.

타입 서브스크립트 선언 (Type Subscript Declarations)

타입의 인스턴스가 아닌 타입에 의해 노출되는 서브스크립트를 선언하려면 `static` 선언 수정자로 서브스크립트 선언을 표시합니다. 클래스는 대신 `class` 선언 수정자로 타입 연산 프로퍼티를 표시하여 하위 클래스가 상위 클래스의 구현을 재정의할 수 있습니다. 클래스 선언에서 `static` 키워드는 `class`와 `final` 선언 수정자 모두 선언에 표시하는 것과 동일한 효과를 가집니다.

Grammar of a subscript declaration

subscript-declaration → subscript-head subscript-result generic-where-clause? code-block
subscript-declaration → subscript-head subscript-result generic-where-clause? getter-setter-block
subscript-declaration → subscript-head subscript-result generic-where-clause? getter-setter-keyword-block
subscript-head → attributes? declaration-modifiers? `subscript` generic-parameter-clause? parameter-clause subscript-result → –> attributes? type

매크로 선언 (Macro Declaration)

*매크로 선언(macro declaration)*은 새로운 매크로를 도입합니다. `macro` 키워드로 시작하고 다음의 형식을 가집니다:

```
macro <#name#> = <#macro implementation#>
```

위 형식의 *매크로 구현(macro implementation)*은 또 다른 매크로이며, 매크로의 확장을 수행하는 코드의 위치를 나타냅니다. 매크로 확장을 수행하는 코드는 Swift 코드와 상호작용하기 위해 SwiftSyntax 모듈을 사용하는 별도의 Swift 프로그램 입니다. Swift 표준 라이브러리의 `externalMacro(module:type:)` 매크로를 호출하고, 해당 매크로를 호출할 때 매크로의 구현을 포함하는 타입의 이름과 해당 타입을 포함하는 모듈의 이름을 전달합니다.

매크로는 함수에서 사용하는 동일한 모델에 따라 오버로드될 수 있습니다. 매크로 선언은 파일 범위에서만 나타납니다.

Swift에서 매크로 개요는 매크로 (Macros)를 참고바랍니다.

Grammar of a macro declaration

macro-declaration → macro-head identifier generic-parameter-clause? macro-signature macro-definition? generic-where-clause macro-head → attributes? declaration-modifiers? `macro` macro-signature → parameter-clause macro-function-signature-result? macro-function-signature-result → –> type macro-definition → = expression

연산자 선언 (Operator Declaration)

*연산자 선언(operator declaration)*은 프로그램에 새로운 중위(infix) 연산자, 접두(prefix) 연산자, 접미(postfix) 연산자를 도입하고 `operator` 키워드를 사용하여 선언합니다.

세 가지 다른 고정성(fixity)을 가진 연산자를 선언할 수 있습니다: 중위 연산자, 접두 연산자, 접미 연산자입니다. 연산자의 *고정성(fixity)*은 피연산자에 대한 연산자의 상대적인 위치를 지정합니다.

각 고정성에 대해 연산자 선언의 세 가지 기본 형식이 있습니다. 연산자의 고정성은 `operator` 키워드 전에 `infix`, `prefix`, `postfix` 선언 수정자로 연산자의 선언을 표시하여 지정합니다. 각 형식에서 연산자의 이름은 연산자 (Operators)에서 정의된 연산자 문자만 포함할 수 있습니다.

다음 형식은 새로운 중위 연산자를 선언합니다:

```
infix operator <#operator name#>: <#precedence group#>
```

*중위 연산자(infix operator)*는 표현식 `1 + 2`에서 덧셈 연산자(+)와 같이 두 피연산자 사이에 작성되는 이진 연산자 입니다.

중위 연산자는 선택적으로 우선순위 그룹을 지정할 수 있습니다. 연산자에 대해 우선순위 그룹을 생략하면, Swift는 기본 우선순위인 `DefaultPrecedence`를 사용하고, 이것은 `TernaryPrecedence`보다 약간 높은 우선순위입니다. 더 자세한 내용은 우선순위 그룹 선언 (Precedence Group Declaration)을 참고바랍니다.

다음 형식은 새로운 접두 연산자를 선언합니다:

```
prefix operator <#operator name#>
```

*접두 연산자(prefix operator)*는 표현식 `!a`에서 접두 논리적 NOT 연산자(`!`)와 같은 피연산자 직전에 작성되는 단항 연산자(unary operator)입니다.

접두 연산자 선언은 우선순위를 지정하지 않습니다. 접두 연산자는 비결합적(nonassociative)입니다.

다음 형식은 접미 연산자를 선언합니다:

```
postfix operator <#operator name#>
```

*접미 연산자(postfix operator)*는 표현식 `a!`에서 접미 강제 언래핑 연산자(`!`)와 같이 피연산자 바로 뒤에 작성되는 단항 연산자(unary operator)입니다.

접두 연산자와 같이 접미 연산자 선언은 우선순위를 지정하지 않습니다. 접미 연산자는 비결합적(nonassociative)입니다.

새로운 연산자를 선언한 후에 연산자와 같은 이름을 가지는 정적 메서드를 선언하여 구현합니다. 정적 메서드는 연산자가 값을 인자로 가지는 타입 중 하나의 멤버입니다 -- 예를 들어 `Double`에 `Int`를 곱하는 연산자는 `Double`이나 `Int` 구조체에 정적 메서드로 구현됩니다. 접두 연산자나 접미 연산자를 구현하면, 해당 메서드 선언에 `prefix`나 `postfix` 선언 수정자를 표시해야 합니다. 새로운 연산자를 어떻게 생성하고 구현하는지에 대한 예시는 커스텀 연산자 (Custom Operators)를 참고바랍니다.

Grammar of an operator declaration

operator-declaration → prefix-operator-declaration | postfix-operator-declaration | infix-operator-declaration

prefix-operator-declaration → `prefix` `operator` operator postfix-operator-declaration → `postfix` `operator` operator infix-operator-declaration → `infix` `operator` operator infix-operator-group?

infix-operator-group → `:` precedence-group-name

우선순위 그룹 선언 (Precedence Group Declaration)

*우선순위 그룹 선언(precedence group declaration)*은 프로그램에서 중위 연산자 우선순위에 대한 새로운 그룹화을 도입합니다. 연산자의 우선순위는 그룹화 괄호가 없을 때 연산자가 피연산자와 어떻게 결합하는지 지정합니다.

우선순위 그룹 선언은 다음의 형식을 가집니다:

```
precedencegroup <#precedence group name#> {
    higherThan: <#lower group names#>
    lowerThan: <#higher group names#>
    associativity: <#associativity#>
    assignment: <#assignment#>
}
```

위 형식의 *하위 그룹 이름(lower group names)*과 위 형식의 상위 그룹 이름(higher group names) 목록은 기존 우선순위 그룹에 새로운 우선순위 그룹의 관계를 지정합니다. `lowerThan` 우선순위 그룹 속성은 현재 모듈의 외부에 선언된 우선순위 그룹을 참조하는 데만 사용됩니다. 표현식 `2 + 3 * 5`에서와 같이 두 연산자가 피연산자에 대해 서로 경쟁할 때 상대적으로 우선순위가 높은 연산자가 피연산자와 더 밀접하게 결합합니다.

참고

위 형식의 *하위 그룹 이름(lower group names)*과 위 형식의 *상위 그룹 이름(higher group names)*을 사용하는 서로 관련된 우선순위 그룹은 단일 관계 계층(single relational hierarchy)에 적합해야 하지만 선형 계층(linear hierarchy)을 형성할 필요는 없습니다. 이것은 상대적 우선순위가 정의되지 않은 우선순위 그룹일 수 있다는 의미입니다. 이러한 우선순위 연산자는 그룹화 괄호없이 나란히 사용할 수 없습니다.

Swift는 표준 라이브러리에서 제공된 연산자와 함께 사용할 수많은 우선순위 그룹을 정의합니다. 예를 들어 덧셈(+)과 뺄셈(–) 연산자는 `AdditionPrecedence` 그룹에 속하고 곱셈(*)과 나눗셈(/) 연산자는 `MultiplicationPrecedence` 그룹에 속합니다. Swift 표준 라이브러리에서 제공하는 우선순위 그룹의 전체 목록은 연산자 선언 (Operator Declarations)을 참고바랍니다.

연산자의 *결합방향(associativity)*은 그룹화 괄호가 없을 때 동일한 우선순위 수준을 가진 일련의 연산자가 함께 그룹화되는 방식을 지정합니다. 상황에 맞는 키워드 `left`, `right`, `none` 중 하나를 작성하여 연산자의 결합방향을 지정합니다 -- 결합방향을 생략하면 기본적으로 `none`입니다. 왼쪽 걸합방향(left-associative) 연산자는 왼쪽에서 오른쪽으로 그룹화합니다. 예를 들어 뺄셈 연산자(–)는 표현식 `4 – 5 – 6`을 `(4 – 5) – 6`으로 그룹화하고 –7로 평가되므로 왼쪽 걸합방향(left-associative)입니다. 오른쪽 걸합방향(right-associative) 연산자는 오른쪽에서 왼쪽으로 그룹화하고, `none`의 결합방향으로 지정된 연산자는 전혀 결합되지 않습니다. 동일한 우선순위 수준의 비결합성 연산자는 서로 인접하게 표시될 수 없습니다. 예를 들어 < 연산자는 `none`의 결합방향을 가집니다. 이것은 `1 < 2 < 3`은 유효하지 않은 표현식이라는 의미입니다.

우선순위 그룹의 *할당(assignment)*은 옵셔널 체이닝을 포함하는 연산에서 연산자의 우선순위를 지정합니다. `true`로 설정하면, 해당 우선순위 그룹의 연산자는 Swift 표준 라이브러리의 할당 연산자와 동일한 그룹화 규칙을 옵셔널 체이닝 중에 사용합니다. 그렇지 않으면 `false`로 설정하거나 생략된 경우, 우선순위 그룹에서 연산자는 할당을 수행하지 않는 연산자와 동일한 옵셔널 체이닝 규칙을 따릅니다.

Grammar of a precedence group declaration

precedence-group-declaration → **`precedencegroup`** precedence-group-name **`{`** precedence-group-attributes? **`}`**

precedence-group-attributes → precedence-group-attribute precedence-group-attributes?
precedence-group-attribute → precedence-group-relation precedence-group-attribute →
precedence-group-assignment precedence-group-attribute → precedence-group-associativity

precedence-group-relation → **`higherThan :`** precedence-group-names precedence-group-relation →
`lowerThan :` precedence-group-names

precedence-group-assignment → **`assignment :`** boolean-literal

precedence-group-associativity → **`associativity : left`** precedence-group-associativity →
`associativity : right` precedence-group-associativity → **`associativity : none`**

precedence-group-names → precedence-group-name | precedence-group-name **`,`**
precedence-group-names precedence-group-name → identifier

선언 수정자 (Declaration Modifiers)

*선언 수정자(Declaration modifiers)*는 선언의 동작이나 의미를 수정하는 키워드나 문맥에 맞는 키워드입니다. 선언의 속성이 있는 경우 선언의 속성과 선언을 도입하는 키워드 사이에 적절한 키워드나 문맥에 맞는 키워드를 작성하여 선언 수정자를 지정합니다.

- `class`: 이 수정자를 클래스의 멤버에 적용하여 해당 멤버가 클래스 인스턴스의 멤버가 아니라, 클래스 자체의 멤버임을 나타냅니다. 이 수정자를 가지고 `final` 수정자를 가지지 않는 상위 클래스의 멤버는 하위 클래스에 의해 재정의될 수 있습니다.
- `dynamic`: Objective-C로 표현될 수 있는 클래스의 모든 멤버에 이 수정자를 적용합니다. `dynamic` 수정자로 멤버 선언을 표시하면, 해당 멤버에 대한 접근이 항상 Objective-C 런타임을 사용하여 동적으로 디스패치됩니다. 해당 멤버에 대한 접근은 컴파일러에 의해 인라인되거나 가상화되지 않습니다.

  `dynamic` 수정자로 표시된 선언은 Objective-C 런타임을 사용하여 디스패치되므로 `objc` 속성으로 표시되어야 합니다.
- `final`: 이 수정자를 클래스나 클래스의 프로퍼티, 메서드, 서브스크립트 멤버에 적용합니다. 클래스에 적용되면 해당 클래스를 상속할 수 없음을 나타냅니다. 클래스의 프로퍼티, 메서드, 서브스크립트에 적용되면 클래스 멤버가 모든 하위 클래스에 재정의될 수 없음을 나타냅니다. `final` 속성을 어떻게 사용하는지에 대한 예시는 재정의 방지 (Preventing Overrides)를 참고바랍니다.
- `lazy`: 이 수정자는 클래스나 구조체의 변수 저장 프로퍼티에 적용하여 프로퍼티의 초기값이 프로퍼티에 처음 접근할 때 한 번만 계산되고 저장됨을 나타냅니다. `lazy` 수정자를 어떻게 사용하는지에 대한 예시는 지연 저장 프로퍼티 (Lazy Stored Properties)를 참고바랍니다.
- `optional`: 프로토콜의 프로퍼티, 메서드, 서브스크립트 멤버에 수정자를 적용하여 준수하는 타입에 이 멤버의 구현이 필수가 아님을 나타냅니다.

  `objc` 속성으로 표시된 프로토콜에만 `optional` 수정자를 적용할 수 있습니다. 결과적으로 클래스 타입만 선택적 멤버 요구사항을 포함하는 프로토콜을 채택하고 준수할 수 있습니다. `optional` 수정자를 어떻게 사용하는지에 대한 자세한 내용과 - 예를 들어 준수하는 타입이 이를 구현하는지 확실하지 않을 때와 같이 - 선택적 프로토콜 멤버에 접근하는 방법에 대한 지침은 옵셔널 프로토콜 요구사항 (Optional Protocol Requirements)을 참고바랍니다.
- `required`: 클래스의 지정 이니셜라이저나 편의 이니셜라이저에 이 수정자를 적용하여 모든 하위 클래스가 해당 이니셜라이저를 구현해야 함을 나타냅니다. 하위 클래스의 해당 이니셜라이저의 구현도 `required` 수정자로 표시되어야 합니다.
- `static`: 이 수정자는 구조체, 클래스, 열거형, 프로토콜의 멤버에 적용하여 멤버가 해당 타입의 인스턴스 멤버가 아닌 타입의 멤버 임을 나타냅니다. 클래스 선언의 범위에서 멤버 선언에 `static` 수정자를 작성하는 것은 멤버 선언에 `class`와 `final` 수정자를 작성한 것과 같은 효과를 가집니다. 그러나 클래스의 상수 타입 프로퍼티는 예외입니다: `static`은 해당 선언에 `class`나 `final`을 작성할 수 없으므로 비클래스 의미를 가집니다.
- `unowned`: 이 수정자를 저장 변수, 상수, 저장 프로퍼티에 적용하여 변수나 프로퍼티가 해당 값으로 저장된 객체에 대해 미소유 참조가 있다는 것을 나타냅니다. 객체가 할당 해제된 후에 변수나 프로퍼티에 접근하려고 하면 런타임 오류가 발생합니다. 약한 참조와 마찬가지로 프로퍼티나 값의 타입은 클래스 타입이어야 합니다; 약한 참조와 다르게 타입은 옵셔널이 아닙니다. `unowned` 수정자에 대한 예시와 자세한 내용은 미소유 참조 (Unowned References)를 참고바랍니다.
- `unowned(safe)`: `unowned`의 명시적 표기입니다.
- `unowned(unsafe)`: 이 수정자를 저장 변수, 상수, 저장 프로퍼티에 적용하여 변수나 프로퍼티에 해당 값으로 저장된 객체에 대해 미소유 참조를 가집니다. 객체가 할당 해제된 후에 변수나 프로퍼티에 접근하려고 하면 객체가 사용한 메모리 위치에 접근하는데 이것은 안전하지 않은 메모리(memory-unsafe)동작입니다. 약한 참조와 마찬가지로 프로퍼티나 값의 타입은 클래스 타입이어야 합니다; 약한 참조와 다르게 타입은 옵셔널이 아닙니다. `unowned` 수정자에 대한 예시와 자세한 내용은 미소유 참조 (Unowned References)를 참고바랍니다.

- `weak`: 이 수정자를 저장 변수나 변수 저장 프로퍼티에 적용하면 변수나 프로퍼티에 해당 값으로 저장된 객체에 대해 약한 참조를 가집니다. 변수나 프로퍼티의 타입은 옵셔널 클래스 타입이어야 합니다. 객체가 할당 해제된 후에 변수나 프로퍼티에 접근하려고 하면 해당 값은 `nil`입니다. `weak` 수정자에 대한 예시와 자세한 내용은 약한 참조 (Weak References)를 참고바랍니다.

접근 제어 수준 (Access Control Levels)

Swift는 다섯 가지 접근 제어 수준을 제공합니다: open, public, internal, file private, private입니다. 선언에 접근 수준을 지정하기 위해 접근-수준 수정자 중 하나로 선언을 표시할 수 있습니다. 접근 제어는 접근 제어 (Access Control)에 자세히 설명되어 있습니다.

- `open`: 이 수정자를 선언에 적용하여 선언과 동일한 모듈의 코드에서 선언에 접근하고 하위 클래스화될 수 있음을 나타냅니다. `open` 접근-수준 수정자로 표시된 선언은 해당 선언을 포함하는 모듈을 임포트하는 모듈의 코드에서 접근되고 하위 클래스화될 수 있습니다.
- `public`: 이 수정자를 선언에 적용하여 선언과 동일한 모듈의 코드에서 선언에 접근하고 하위 클래스화될 수 있음을 나타냅니다. `public` 접근-수준 수정자로 표시된 선언은 해당 선언을 포함하는 모듈을 임포트하는 모듈의 코드에서 접근되지만 하위 클래스화될 수 없습니다.
- `package`: 이 수정자를 선언에 적용하여 선언과 동일한 패키지의 코드에 의해서만 접근될 수 있음을 나타냅니다. 패키지는 사용 중인 빌드 시스템에서 정의하는 코드 배포 단위입니다. 빌드 시스템이 코드를 컴파일할 때 Swift 컴파일러에 `–package–name` 플래그를 전달하여 패키지 이름을 지정합니다. 두 모듈이 빌드 시스템이 동일한 패키지 이름을 지정하여 빌드하면 동일한 패키지의 일부입니다.
- `internal`: 이 수정자를 선언에 적용하여 선언과 동일한 모듈의 코드에 의해서만 접근될 수 있음을 나타냅니다. 기본적으로 대부분 선언은 암시적으로 `internal` 접근-수준 수정자로 표시됩니다.
- `fileprivate`: 이 수정자를 선언에 적용하여 선언과 동일한 소스 파일의 코드에서만 선언에 접근될 수 있음을 나타냅니다.
- `private`: 이 수정자를 선언에 적용하여 선언을 직접 둘러싸는 범위 내의 코드에서만 선언에 접근될 수 있음을 나타냅니다.

접근 제어의 목적을 위해 확장은 다음과 같이 동작합니다:

- 동일한 파일에 여러 확장이 있고, 이 확장이 동일한 타입을 확장한다면, 이 모든 확장은 동일한 접근 제어를 가집니다. 확장과 확장하는 타입은 다른 파일에 있을 수 있습니다.
- 확장이 확장하는 타입과 동일한 파일에 있으면, 확장은 확장하는 타입과 동일한 접근 제어를 가집니다.
- 타입의 선언에서 선언된 private 멤버는 해당 타입의 확장에서 접근할 수 있습니다. 한 확장에서 선언된 private 멤버는 다른 확장과 확장된 타입의 선언에서 접근할 수 있습니다.

위의 각 접근-수준 수정자는 선택적으로 괄호로 둘러싸인 `set` 키워드로 구성된 단일 인자를 허용합니다 - 예를 들어 `private(set)`입니다. Getter와 Setter (Getters and Setters)에서 설명한대로 변수나 서브스크립트에 대한 접근 수준보다 작거나 같은 변수나 서브스크립트의 setter에 대한 접근 수준을 지정하려면 이 형식의 접근-수준 수정자를 사용합니다.

Grammar of a declaration modifier

declaration-modifier → `class` | `convenience` | `dynamic` | `final` | `infix` | `lazy` | `optional` | `override` | `postfix` | `prefix` | `required` | `static` | `unowned` | `unowned ( safe )` | `unowned ( unsafe )` | `weak`
declaration-modifier → access-level-modifier declaration-modifier → mutation-modifier

declaration-modifier → actor-isolation-modifier declaration-modifiers → declaration-modifier declaration-modifiers?

access-level-modifier → `private` | `private ( set )` access-level-modifier → `fileprivate` | `fileprivate ( set )` access-level-modifier → `internal` | `internal ( set )` access-level-modifier → `package` | `package ( set )` access-level-modifier → `public` | `public ( set )` access-level-modifier → `open` | `open ( set )`

mutation-modifier → `mutating` | `nonmutating`

actor-isolation-modifier → `nonisolated`

언어 참조

속성 (Attributes)

선언과 타입에 정보를 추가합니다.

Swift에는 두 종류의 속성(attributes)이 있습니다 -- 선언에 적용하는 속성과 타입에 적용하는 속성이 있습니다. 속성은 선언이나 타입에 추가적인 정보를 제공합니다. 예를 들어 함수 선언에 `discardableResult` 속성은 함수가 값을 반환하지만 반환값이 사용되지 않을 때 컴파일러는 경고를 생성하지 않는 것을 나타냅니다.

속성을 지정하려면 `@` 기호 다음에 속성의 이름과 속성이 허용하는 인자를 작성합니다:

```
@<#attribute name#>
@<#attribute name#>(<#attribute arguments#>)
```

어떤 선언 속성은 속성에 대한 추가 정보와 특정 선언에 어떻게 적용하는지 지정하는 인자를 허용합니다. 이러한 *속성 인자(attribute arguments)*는 소괄호로 감싸고 해당 형식이 속한 속성에 의해 정의됩니다.

첨부 매크로와 프로퍼티 래퍼도 속성 문법을 사용합니다. 매크로 확장에 대한 자세한 내용은 매크로-확장 표현식(Macro-Expansion Expression)을 참고바랍니다. 프로퍼티 래퍼에 대한 자세한 내용은 propertyWrapper를 참고바랍니다.

선언 속성 (Declaration Attributes)

> **참고**
>
> **번역자 보충** Swift 6.4의 경고 제어 속성 `@diagnose`는 경고 제어 속성 (Warning Control Attribute)을 참고바랍니다.

선언 속성(declaration attribute)은 선언에만 적용할 수 있습니다.

attached

매크로 선언에 `attached` 속성을 적용합니다. 이 속성의 인자는 매크로의 역할을 나타냅니다. 하나의 매크로가 여러 역할이 있는 경우 각 역할에 대해 `attached` 매크로를 여러 번 적용해야 합니다.

이 속성에 첫 번째 인자는 매크로 역할을 나타냅니다:

- Peer 매크로: 이 속성의 첫 번째 인자로 `peer`를 작성합니다. 이 매크로 구현 타입은 `PeerMacro` 프로토콜을 준수합니다. 이러한 매크로는 매크로가 적용된 선언과 동일한 범위에 새로운 선언을 생성합니다. 예를 들어 구조체의 메서드에 peer 매크로를 적용하면 해당 구조체에 추가로 메서드와 프로퍼티를 정의할 수 있습니다.
- Member 매크로: 이 속성의 첫 번째 인자로 `member`를 작성합니다. 이 매크로 구현 타입은 `MemberMacro` 프로토콜을 준수합니다. 이러한 매크로는 매크로가 적용된 타입이나 확장의 멤버인 새로운 선언을 생성합니다. 예를 들어 구조체 선언에 member 매크로를 적용하면 해당 구조체에 추가로 메서드와 프로퍼티를 정의할 수 있습니다.
- Member 속성: 이 속성의 첫 번째 인자로 `memberAttribute`를 작성합니다. 이 매크로 구현 타입은 `MemberAttributeMacro` 프로토콜을 준수합니다. 이러한 매크로는 매크로가 적용된 타입이나 확장의 멤버에 속성을 추가합니다.

- Accessor 매크로: 이 속성의 첫 번째 인자로 `accessor`를 작성합니다. 이 매크로 구현 타입은 `AccessorMacro` 프로토콜을 준수합니다. 이 매크로는 저장 프로퍼티에 접근자를 추가하여 연산 프로퍼티로 변경합니다.
- Extension 매크로: 이 속성의 첫 번째 인자로 `extension`을 작성합니다. 이 매크로 구현 타입은 `ExtensionMacro` 프로토콜을 준수합니다. 이러한 매크로는 `where` 절로 프로토콜 준수를 추가할 수 있고, 매크로가 적용된 타입의 새로운 멤버 선언을 추가할 수 있습니다. 매크로가 프로토콜 준수를 추가하면, `conformances:` 인자를 포함하고 프로토콜을 지정합니다. 준수 목록은 프로토콜 이름, 준수하는 목록 아이템의 타입 별칭, 복합 프로토콜 준수 목록 아이템을 포함합니다. 중첩 타입에서 확장 매크로는 해당 파일의 최상위 레벨에서 확장합니다. 확장, 타입 별칭, 함수 내에 중첩된 타입에 확장 매크로를 작성하거나 확장 매크로를 사용하여 peer 매크로가 있는 확장을 추가할 수 없습니다.
- Body 매크로: 이 속성의 첫 번째 인자로 `body`를 작성합니다. 이 매크로 구현 타입은 `BodyMacro` 프로토콜을 준수합니다. 이러한 매크로는 매크로가 적용된 함수나 접근자의 본문을 생성하거나 수정하거나 교체할 수 있습니다. 하나의 함수에는 본문 매크로를 최대 한 개만 작성할 수 있습니다. 원래 함수 본문이 제공되면, 문법적으로 올바른 코드여야 하지만, 타입 검사는 매크로가 확장된 후에 수행됩니다.

peer와 member 매크로 역할은 매크로가 생성하는 기호의 이름을 나열하는 `names:` 인자를 요구합니다. accessor 매크로 역할은 매크로가 `willSet`이나 `didSet` 프로퍼티 관찰자를 생성한다면 `names:` 인자를 요구합니다. 프로퍼티 관찰자를 생성하는 accessor 매크로는 프로퍼티 관찰자는 저장 프로퍼티에만 적용가능하므로 다른 accessor 매크로를 추가할 수 없습니다. 매크로가 확장 내에 선언을 추가한다면 확장 매크로 역할도 `names:` 인자를 요구합니다. `names:` 인자가 매크로 선언에 포함되면, 매크로 구현은 해당 목록과 일치하는 이름의 기호로만 생성해야 합니다. 그렇다고 해서 매크로가 목록에 있는 모든 이름에 대한 기호를 생성할 필요는 없습니다. 해당 인자의 값은 다음의 목록 중 하나 이상을 나열할 수 있습니다:

- `named(<#name#>)` 여기서 *이름(name)*은 미리 알려진 고정된 기호 이름입니다.
- `overloaded` 기존 기호와 동일한 이름인 경우입니다.
- `prefixed(<#prefix#>)` 여기서 *접두사(prefix)*는 기호 이름 앞에 붙는 고정된 문자열로 시작하는 이름입니다.
- `suffixed(<#suffix#>)` 여기서 *접미사(suffix)*는 기호 이름 뒤에 붙는 고정된 문자열로 끝나는 이름입니다.
- `arbitrary` 매크로 확장 시점까지 결정할 수 없는 이름입니다.

특수한 경우로 프로퍼티 래퍼(property wrapper)와 유사하게 동작하는 매크로에 대해 `prefixed($)`를 작성할 수 있습니다.

available

이 속성을 적용하여 특정 Swift 언어 버전이나 특정 플랫폼과 운영체제 버전과 관련된 선언의 생명주기를 나타냅니다.

`available` 속성은 두 개 이상의 콤마로 구분된 속성 인자의 목록으로 나타냅니다. 이러한 인자는 다음의 플랫폼이나 언어 이름 중 하나로 시작합니다:

- `iOS`
- `iOSApplicationExtension`
- `macOS`
- `macOSApplicationExtension`

- `macCatalyst`
- `macCatalystApplicationExtension`
- `watchOS`
- `watchOSApplicationExtension`
- `tvOS`
- `tvOSApplicationExtension`
- `visionOS`
- `visionOSApplicationExtension`
- `swift`

또한 별표(`*`)를 사용하여 위에 나열된 모든 플랫폼 이름에 대한 선언의 가용성을 나타낼 수도 있습니다. Swift 버전을 사용하여 지정한 가용 `available` 속성은 별표를 사용할 수 없습니다.

나머지 인자는 순서에 상관없이 나타날 수 있으며 중요한 마일스톤을 포함하여 선언의 생명주기에 대한 추가 정보를 지정할 수 있습니다.

- `unavailable` 인자는 지정된 플랫폼에서 선언을 사용할 수 없음을 나타냅니다. 이 인자는 Swift 버전 가용성을 지정할 때 사용될 수 없습니다.
- `introduced` 인자는 선언이 도입된 플랫폼이나 언어의 첫 번째 버전을 나타냅니다. 다음과 같은 형식을 가집니다:

  ```
  introduced: <#version number#>
  ```

  위 형식의 *버전 번호(version number)*는 마침표로 구분된 하나에서 세 개의 양수로 구성됩니다.
- `deprecated` 인자는 선언이 사용 중단된 플랫폼이나 언어의 첫 번째 버전을 나타냅니다. 다음과 같은 형식을 가집니다:

  ```
  deprecated: <#version number#>
  ```

  옵셔널 *버전 번호(version number)*는 마침표로 구분된 하나에서 세 개의 양수로 구성됩니다. 버전 번호를 생략하면 사용 중단된 시기에 대한 정보를 제공하지 않고 선언이 현재 사용 중단됨을 나타냅니다. 버전 숫자를 생략하면 콜론(`:`)도 생략합니다.
- `obsoleted` 인자는 선언이 폐기된 플랫폼이나 언어의 첫 번째 버전을 나타냅니다. 선언이 폐기되면, 지정된 플랫폼이나 언어에서 제거되고 더 이상 사용이 불가능합니다. 다음과 같은 형식을 가집니다:

  ```
  obsoleted: <#version number#>
  ```

  *버전 번호(version number)*는 마침표로 구분된 하나에서 세 개의 양수로 구성됩니다.
- `noasync` 인자는 선언된 기호가 비동기 컨텍스트에서 직접적으로 사용될 수 없음을 나타냅니다.

Swift 동시성은 잠재적 중단 지점 후에 다른 스레드에서 재개될 수 있으므로, 스레드 로컬 스토리지(thread-local storage), 락(locks), 뮤텍스(mutexes), 세마포어(semaphores)와 같은 요소를 중단 지점을 넘어 사용하면 올바르지 않은 결과를 가져올 수 있습니다.

이런 문제를 피하기 위해 기호의 선언에 `@available(*, noasync)` 속성을 추가합니다:

```
extension pthread_mutex_t {

  @available(*, noasync)
  mutating func lock() {
      pthread_mutex_lock(&self)
  }

  @available(*, noasync)
  mutating func unlock() {
      pthread_mutex_unlock(&self)
  }
}
```

이 속성은 누군가가 비동기 컨텍스트에서 이 기호를 사용하려고 하면 컴파일 시 오류가 발생합니다. 이 기호에 대해 추가적인 정보를 제공하기 위해 `message` 인자를 사용할 수도 있습니다.

```
@available(*, noasync, message: "Migrate locks to Swift concurrency.")
mutating func lock() {
  pthread_mutex_lock(&self)
}
```

잠재적으로 안전하지 않은 기호를 안전한 방식으로 사용함을 보장할 수 있다면, 이것을 동기 함수로 감싸고 비동기 컨텍스트에서 해당 함수를 호출할 수 있습니다.

```

// Provide a synchronous wrapper around methods with a noasync declaration.
extension pthread_mutex_t {
  mutating func withLock(_ operation: () -> ()) {
    self.lock()
    operation()
    self.unlock()
  }
}

func downloadAndStore(key: Int,
                      dataStore: MyKeyedStorage,
                      dataLock: inout pthread_mutex_t) async {
  // Safely call the wrapper in an asynchronous context.
  dataLock.withLock {
    dataStore[key] = downloadContent()
  }
}
```

`noasync` 인자는 대부분 선언에 사용할 수 있습니다; 그러나 디이니셜라이저 선언에는 사용할 수 없습니다. Swift는 동기와 비동기 컨텍스트 모두에서 클래스의 디이니셜라이저를 호출할 수 있어야 합니다.

- `message` 인자는 `deprecated`, `obsoleted`, `noasync`로 표시된 선언 사용에 대한 경고나 오류를 생성할 때, 컴파일러가 표시하는 텍스트 메세지를 제공합니다. 다음과 같은 형식을 가집니다:

```
message: <#message#>
```

위 형식의 *메세지(message)*는 문자열 리터럴로 구성됩니다.

- `renamed` 인자는 이름이 변경된 새로운 선언을 나타내는 텍스트 메세지를 제공합니다. 컴파일러는 이름이 변경된 선언 사용에 대한 오류를 생성할 때, 새로운 이름을 표시합니다. 다음과 같은 형식을 가집니다:

```
renamed: <#new name#>
```

위 형식의 *새로운 이름(new name)*은 문자열 리터럴로 구성됩니다.

프레임워크나 라이브러리의 릴리즈 간에 변경된 선언의 이름을 나타내기 위해 아래와 같이 타입 별칭 선언에 `renamed`와 `unavailable` 인자로 `available` 속성을 적용할 수 있습니다. 이 조합으로 인해 컴파일 시 선언이 변경되었다는 오류가 발생합니다.

```
// First release
protocol MyProtocol {
    // protocol definition
}
```

```
// Subsequent release renames MyProtocol
protocol MyRenamedProtocol {
    // protocol definition
}

@available(*, unavailable, renamed: "MyRenamedProtocol")
typealias MyProtocol = MyRenamedProtocol
```

단일 선언에 여러 개의 `available` 속성을 적용하여 다른 플랫폼과 다른 Swift 버전에서 선언의 가용성을 지정할 수 있습니다. `available` 속성이 적용된 선언은 현재 타겟이 지정한 플랫폼이나 언어 버전과 일치하지 않으면 무시됩니다. 여러 개의 `available` 속성을 사용하는 경우 효과적인 가용성은 플랫폼과 Swift 가용성의 조합입니다.

`available` 속성이 플랫폼이나 언어 이름 인자 외에 `introduced` 인자를 지정하는 경우 아래와 같은 축약 문법을 사용할 수 있습니다:

```
@available(<#platform name#> <#version number#>, *)
@available(swift <#version number#>)
```

`available` 속성에 대한 축약 문법은 여러 플랫폼에 대한 가용성을 간결하게 표현합니다. 두 형식은 동일하지만 가능하면 축약 형식을 선호합니다.

```
@available(iOS 10.0, macOS 10.12, *)
class MyClass {
    // class definition
}
```

Swift 버전 숫자를 사용하여 가용성을 지정하는 `available` 속성은 선언에 플랫폼 가용성을 추가로 지정할 수 없습니다. 대신에 Swift 버전 가용성과 하나 이상의 플랫폼 가용성을 지정하기 위해 분리된 `available` 속성을 사용합니다.

```
@available(swift 3.0.2)
@available(macOS 10.12, *)
struct MyStruct {
    // struct definition
}
```

backDeployed

이 속성을 함수, 메드, 서브스크립트, 연산 프로퍼티에 적용하여 해당 기호 구현의 복사본을 기호를 호출하거나 접근하는 프로그램에 포함시킵니다. 이 속성을 사용하여 운영체제에 포함된 API와 같이 플랫폼의 일부로 제공되는 기호에 주석을 답니다. 이 속성은 기호에 접근하는 프로그램에 해당 구현의 복사본을 포함하여 사용할 수 있게 합니다. 구현 복사본은 *클라이언트로 내보내기(emitting into the client)*라고 합니다.

이 속성은 이 기호를 제공하는 플랫폼의 첫 번째 버전을 나타내는 `before:` 인자를 가지고 있습니다. 이 플랫폼 버전은 `available` 속성에 지정한 플랫폼 버전과 같은 의미를 가집니다. `available` 속성과 다르게 목록은 모든 버전을 참조하는 별표(*)를 포함할 수 없습니다. 예를 들어 아래의 코드를 살펴봅시다:

```
@available(iOS 16, *)
@backDeployed(before: iOS 17)
func someFunction() { /* ... */ }
```

위의 예시에서, iOS SDK는 iOS 17부터 `someFunction()`을 제공합니다. 추가로, SDK는 백 배포(back deployment)를 사용하여 iOS 16에서도 `someFunction()`을 사용할 수 있도록 합니다.

이 함수를 호출하는 코드를 컴파일할 때, Swift는 함수의 구현을 찾는 간접 참조 계층을 삽입합니다. 코드가 이 함수를 포함하는 SDK 버전에서 실행되면, SDK의 구현이 사용됩니다. 그렇지 않으면 호출자에 포함된 복사본이 사용됩니다. 위의 예시에서 `someFunction()` 호출은 iOS 17이상의 버전에서 실행하면, SDK의 구현이 사용되고, iOS 16에서 실행하면 호출자에 포함된 `someFunction()`의 복사본이 사용됩니다.

> **참고**
>
> 호출자의 최소 배포 타겟이 기호를 포함하는 SDK의 처음 버전과 같거나 더 높은 버전이면 컴파일러는 런타임 검사를 최적화하고 SDK의 구현을 직접 호출할 수 있습니다. 이 경우에 백-배포된 기호에 직접 접근하면, 컴파일러가 클라이언트에서 기호의 구현 복사를 생략할 수도 있습니다.

다음 기준을 충족하는 함수, 메서드, 서브스크립트, 연산 프로퍼티는 백 배포될 수 있습니다:

- 선언이 `public`이나 `@usableFromInline`입니다.
- 클래스 인스턴스 메서드와 클래스 타입 메서드의 경우 메서드는 `final`로 표기하고 `@objc`로 표기하지 않습니다.
- 구현은 inlinable에서 설명한대로 인라인 가능한 함수에 대한 요구사항을 충족합니다.

discardableResult

이 속성을 함수나 메서드 선언에 적용하여 값을 반환하는 함수나 메서드의 결과를 사용하지 않고 호출할 때 컴파일러가 경고를 표시하지 않도록 합니다.

dynamicCallable

이 속성을 클래스, 구조체, 열거형, 프로토콜에 적용하여 해당 타입의 인스턴스를 호출 가능한 함수처럼 취급할 수 있습니다. 이 타입은 `dynamicallyCall(withArguments:)` 메서드, `dynamicallyCall(withKeywordArguments:)` 메서드 또는 둘 다 구현해야 합니다.

동적으로 호출 가능한 타입의 인스턴스를 여러 인자를 받는 함수인 것처럼 호출할 수 있습니다.

```
@dynamicCallable
struct TelephoneExchange {
    func dynamicallyCall(withArguments phoneNumber: [Int]) {
        if phoneNumber == [4, 1, 1] {
            print("Get Swift help on forums.swift.org")
        } else {
            print("Unrecognized number")
        }
    }
}

let dial = TelephoneExchange()

// Use a dynamic method call.
dial(4, 1, 1)
// Prints "Get Swift help on forums.swift.org".

dial(8, 6, 7, 5, 3, 0, 9)
// Prints "Unrecognized number".

// Call the underlying method directly.
dial.dynamicallyCall(withArguments: [4, 1, 1])
```

`dynamicallyCall(withArguments:)` 메서드의 선언은 위의 예시에서 `[Int]`와 같이 `ExpressibleByArrayLiteral` 프로토콜을 준수하는 단일 매개변수만 가져야 합니다. 반환 타입은 모든 타입이 가능합니다.

`dynamicallyCall(withKeywordArguments:)` 메서드를 구현하면, 동적 메서드 호출에 레이블을 포함할 수 있습니다.

```
@dynamicCallable
struct Repeater {
    func dynamicallyCall(withKeywordArguments pairs: KeyValuePairs<String, Int>) -> String {
        return pairs
            .map { label, count in
                repeatElement(label, count: count).joined(separator: " ")
            }
            .joined(separator: "\n")
    }
}

let repeatLabels = Repeater()
print(repeatLabels(a: 1, b: 2, c: 3, b: 2, a: 1))
// a
// b b
// c c c
// b b
// a
```

`dynamicallyCall(withKeywordArguments:)` 메서드의 선언은 `ExpressibleByDictionaryLiteral` 프로토콜을 준수하는 단일 매개변수를 가지고 반환 타입은 모든 타입이 가능합니다. 매개변수의 `Key`는 `ExpressibleByStringLiteral`을 준수해야 합니다. 이전 예시는 매개변수 타입으로 `KeyValuePairs`를 사용하므로 호출자는 중복 매개변수 레이블을 포함할 수 있습니다 -- `a`와 `b`는 `repeat`을 호출하는데 여러 번 나타납니다.

`dynamicallyCall` 메서드 모두 구현하면, `dynamicallyCall(withKeywordArguments:)`는 메서드 호출이 키워드 인자를 포함할 때 호출됩니다. 다른 경우에는 `dynamicallyCall(withArguments:)`가 호출됩니다.

동적으로 호출 가능한 인스턴스는 `dynamicallyCall` 메서드 구현 중 하나에서 지정한 타입과 일치하는 인자와 반환값으로만 호출할 수 있습니다. 다음 예시에서 호출은 `KeyValuePairs<String, String>`을 가지는 `dynamicallyCall(withArguments:)`의 구현이 없으므로 컴파일되지 않습니다.

```
repeatLabels(a: "four") // Error
```

dynamicMemberLookup

이 속성을 클래스, 구조체, 열거형, 프로토콜에 적용하여 런타임 시 멤버를 이름으로 찾을 수 있도록 합니다. 이 타입은 `subscript(dynamicMember:)` 서브스크립트를 구현해야 합니다.

명시적 멤버 표현식에서 명명된 멤버에 대한 해당 선언이 없는 경우 표현식은 타입의 `subscript(dynamicMember:)` 서브스크립트를 호출하는 것으로 이해하고 멤버에 대한 정보를 인자로 전달합니다. 서브스크립트는 키 경로나 멤버 이름 중 하나인 매개변수를 사용할 수 있습니다; 두 서브스크립트를 모두 구현하면 키 경로 인자를 가지는 서브스크립트가 사용됩니다.

`subscript(dynamicMember:)`의 구현은 `KeyPath`, `WritableKeyPath`, `ReferenceWritableKeyPath` 타입의 인자를 사용하여 키 경로를 받을 수 있습니다. 대부분 `String`인 `ExpressibleByStringLiteral` 프로토콜을 준수하는 타입의 인자를 사용하여 멤버 이름을 받을 수 있습니다. 서브스크립트의 반환 타입은 모든 타입이 가능합니다.

멤버 이름에 의한 동적 멤버 조회는 다른 언어의 데이터를 Swift로 연결할 때와 같이 컴파일 시에 타입을 확인할 수 없는 데이터에 대한 래퍼 타입을 생성하기 위해 사용될 수 있습니다. 예를 들어:

```
@dynamicMemberLookup
struct DynamicStruct {
    let dictionary = ["someDynamicMember": 325,
                      "someOtherMember": 787]
    subscript(dynamicMember member: String) -> Int {
        return dictionary[member] ?? 1054
    }
}
let s = DynamicStruct()

// Use dynamic member lookup.
let dynamic = s.someDynamicMember
print(dynamic)
// Prints "325".

// Call the underlying subscript directly.
let equivalent = s[dynamicMember: "someDynamicMember"]
print(dynamic == equivalent)
// Prints "true".
```

키 경로에 의한 동적 멤버 조회는 컴파일 시 타입 검사를 제공하는 방법으로 래퍼 타입을 구현하는데 사용될 수 있습니다. 예를 들어:

```
struct Point { var x, y: Int }

@dynamicMemberLookup
struct PassthroughWrapper<Value> {
    var value: Value
    subscript<T>(dynamicMember member: KeyPath<Value, T>) -> T {
        get { return value[keyPath: member] }
    }
}

let point = Point(x: 381, y: 431)
let wrapper = PassthroughWrapper(value: point)
print(wrapper.x)
```

export

이 속성을 함수나 메서드 선언에 적용하여 해당 정의를 클라이언트 모듈로 어떻게 내보낼 지 제어합니다. 내보낼 선언의 어떤 부분을 내보낼지 나타내기 위해 다음 중 하나의 인자를 포함합니다.

- `interface` 인자는 호출 가능한 기호형태로 인터페이스만 클라이언트에 내보내도록 지정합니다. 정의(함수 본문)는 인라인, 최적화, 어떤 목적에도 클라이언트에서 사용할 수 없습니다. 이 인자를 사용하면 클라이언트로부터 구현을 숨길 수 있습니다.
- `implementation` 인자는 정의(함수 본문)만 클라이언트에 내보내도록 지정합니다. 이 함수에 대한 기호는 바이너리에 생성되지 않으며, 클라이언트는 필요한 곳마다 이 정의의 사본을 생성해야 합니다. 이 인자를 사용해 Application Binary Interface(ABI)에 영향없이 새로운 함수나 메서드를 도입할 수 있습니다.

freestanding

`freestanding` 속성을 적용하여 독립 매크로(freestanding macro)를 선언합니다.

frozen

이 속성을 구조체 선언이나 열거형 선언에 적용하여 타입에 대해 만들 수 있는 변경의 종류를 제한합니다. 이 속성은 라이브러리 진화 모드(library evolution mode)로 컴파일할 때만 허용됩니다. 이후 버전의 라이브러리는 열거형의 케이스나 구조체의 저장 인스턴스 프로퍼티를 추가, 제거, 재정렬로 선언을 변경할 수 없습니다. 이러한 변경은 비고정 타입(nonfrozen types)에서 허용되지만 고정 타입(frozen types)에 대해 ABI 호환성을 깨뜨립니다.

라이브러리 진화 모드에서 비고정 구조체와 열거형의 멤버와 상호작용하는 코드는 향후 버전의 라이브러리에서 해당 타입의 일부 멤버를 추가, 제거, 재정렬하더라도 다시 컴파일되지 않고 계속 작업할 수 있는 방식으로 컴파일됩니다. 컴파일러는 런타임에 정보 검색 및 간접 계층 추가와 같은 기술을 사용하여 이를 가능하게 합니다. 구조체나 열거형을 고정(frozen)으로 표시하면 성능을 얻기 위해 이러한 유연성을 포기합니다: 라이브러리의 향후 버전은 타입을 제한적으로 변경할 수 있지만 컴파일러는 타입의 멤버와 상호작용하는 코드에서 추가 최적화를 수행할 수 있습니다.

고정 타입, 고정 구조체의 저장 프로퍼티와 고정 열거형 케이스의 연관값의 타입은 public이거나 `usableFromInline` 속성으로 표시되어야 합니다. 고정 구조체의 프로퍼티는 프로퍼티 관찰자를 가질 수 없고 저장 인스턴스 프로퍼티에 대해 초기값을 제공하는 표현식은 inlinable에서 설명한대로 인라인 가능 함수와 동일한 제한사항을 따릅니다.

커맨드 라인에서 라이브러리 진화 모드를 활성화 하려면 `-enable-library-evolution` 옵션을 Swift 컴파일러에 전달해야 합니다. Xcode에서 가능하게 하려면 Xcode 도움말 (Xcode Help)에서 설명한대로 "Build Libraries for Distribution" 빌드 설정 (`BUILD_LIBRARY_FOR_DISTRIBUTION`)을 Yes로 설정해야 합니다.

고정 열거형에 대한 switch 구문은 향후 열거형 케이스 전환 (Switching Over Future Enumeration Cases)에서 설명한대로 `default` 케이스를 요구하지 않습니다. 고정 열거형을 사용할 때 `default`나 `@unknown default` 케이스를 포함하면 해당 코드는 실행되지 않기 때문에 경고가 생성됩니다.

GKInspectable

이 속성을 적용하여 커스텀 GameplayKit 구성요소 프로퍼티를 SpriteKit 에디터 UI에 노출합니다. 이 속성을 적용하면 `objc` 속성도 암시적으로 포함합니다.

globalActor

이 속성을 액터, 구조체, 열겨헝, final 클래스에 적용합니다. 이 타입은 `shared`라는 정적 프로퍼티를 정의해야 하며, 이 프로퍼티는 액터의 공유 인스턴스를 제공합니다.

전역 액터는 액터 격리의 개념을 코드의 여러 곳 - 여러 타입, 모듈과 같은 - 에 분산된 상태로 만들며 동시성 코드에서 전역 변수를 안전하게 접근할 수 있도록 합니다. 전역 액터가 `shared` 프로퍼티의 값으로 제공하는 액터는 모든 상태의 접근을 직렬화합니다. 또한 모든 코드가 동일한 스레드에서 실행되어야 하는 경우와 같이 동시성 코드의 제약조건을 모델링하는 데 전역 액터를 사용할 수도 있습니다.

전역 액터는 `GlobalActor` 프로토콜을 암시적 준수합니다. 메인 액터는 메인 액터 (The Main Actor)에서 설명한대로 표준 라이브러리에서 제공하는 전역 액터입니다. 대부분 코드에서는 새로운 전역 액터를 정의하는 대신 메인 액터를 사용할 수 있습니다.

inlinable

이 속성을 함수, 메서드, 연산 프로퍼티, 서브스크립트, 편의 이니셜라이저, 디이니셜라이저 선언에 적용하여 해당 선언의 구현을 public 인터페이스 일부로 노출합니다. 컴파일러는 호출 지점에서 인라인 가능 기호에 대한 호출을 기호 구현의 복사본으로 대체할 수 있습니다.

인라인 가능 코드는 모든 모듈에 선언된 `open`과 `public` 기호와 상호작용할 수 있고, 동일한 모듈에 선언되어 `usableFromInline` 속성으로 표시된 `internal` 기호와도 상호작용할 수 있습니다. 인라인 가능 코드는 `private`이나 `fileprivate` 기호와 상호작용할 수 없습니다.

이 속성은 함수 내에 중첩된 선언이나 `fileprivate` 또는 `private` 선언에 적용될 수 없습니다. 인라인 가능 함수 내에 선언된 함수와 클로저는 이 속성으로 표시될 수 없더라도 암시적으로 인라인 가능합니다.

main

이 속성을 구조체, 클래스, 열거형 선언에 적용하여 프로그램 흐름의 최상위 시작점을 포함하는 것을 나타냅니다. 이 타입은 인자가 없고 `Void`를 반환하는 `main` 타입 함수를 제공해야 합니다. 예를 들어:

```
@main
struct MyTopLevel {
    static func main() {
        // Top-level code goes here
    }
}
```

`main` 속성의 요구사항을 설명하는 또 다른 방법은 이 속성을 작성하는 타입이 다음 가상의 프로토콜을 준수하는 타입과 동일한 요구사항을 충족해야 한다는 것입니다:

```
protocol ProvidesMain {
    static func main() throws
}
```

실행 가능하도록 만들기 위해 컴파일한 Swift 코드는 최상위-수준 코드 (Top-Level Code)에서 설명한대로 최상위 시작점을 포함해야 합니다.

nonobjc

이 속성을 메서드, 프로퍼티, 서브스크립트, 이니셜라이저 선언에 적용하여 암시적으로 `objc` 속성을 억제합니다. `nonobjc` 속성은 선언이 Objective-C에서 표현 가능하더라도 컴파일러에게 Objective-C 코드에서 해당 선언을 사용할 수 없도록 합니다.

이 속성을 확장에 적용하면 명시적으로 `objc` 속성으로 표시되지 않은 확장의 모든 멤버에 적용하는 것과 동일한 효과를 가집니다.

`nonobjc` 속성은 `objc` 속성으로 표시된 클래스에서 브리징 메서드에 대한 순환성을 해결하고 `objc` 속성으로 표시된 클래스의 메서드와 이니셜라이저의 오버로딩을 허용하기 위해 사용합니다.

`nonobjc` 속성으로 표시된 메서드는 `objc` 속성으로 표시된 메서드를 재정의할 수 없습니다. 그러나 `objc` 속성으로 표시된 메서드는 `nonobjc` 속성으로 표시된 메서드를 재정의할 수 있습니다. 유사하게 `nonobjc` 속성으로 표시된 메서드는 `objc` 속성으로 표시된 메서드에 대한 프로토콜 요구사항을 충족할 수 없습니다.

NSApplicationMain

> **사용 중단**
>
> 이 속성은 사용 중단되었습니다; 대신 main 속성을 사용합니다. Swift 6에서 이 속성을 사용하면 컴파일 시 오류가 발생합니다.

이 속성을 클래스에 적용하여 앱 델리게이트임을 나타냅니다. 이 속성을 사용하는 것은 `NSApplicationMain(_:_:)` 함수를 호출하는 것과 동일합니다.

이 속성을 사용하지 않으면, `main.swift` 파일에 다음과 같이 `NSApplicationMain(_:_:)` 함수를 호출하는 최상위 코드를 제공해야 합니다:

```
import AppKit
NSApplicationMain(CommandLine.argc, CommandLine.unsafeArgv)
```

실행 파일을 만들기 위해 컴파일한 Swift 코드는 최상위-수준 코드 (Top-Level Code)에서 설명한대로 하나의 최상위 시작점을 포함할 수 있습니다.

NSCopying

이 속성을 클래스의 변수 저장 프로퍼티에 적용합니다. 이 속성은 프로퍼티의 setter가 프로퍼티 값 자체 대신 `copyWithZone(_:)` 메서드에 의해 반환된 프로퍼티 값의 *복사본(copy)*으로 합성되도록 합니다. 프로퍼티의 타입은 `NSCopying` 프로토콜을 준수해야 합니다.

`NSCopying` 속성은 Objective-C `copy` 프로퍼티 속성과 유사하게 동작합니다.

NSManaged

이 속성을 `NSManagedObject`를 상속받는 클래스의 인스턴스 메서드나 변수 저장 프로퍼티에 적용하여 코어 데이터(Core Data)가 관련 엔티티 설명에 따라 런타임 시 해당 구현을 동적으로 제공하는 것을 나타냅니다.
`NSManaged` 속성으로 표시된 프로퍼티의 경우 코어 데이터(Core Data)는 런타임에 저장소도 제공합니다. 이 속성을 적용하면 `objc` 속성도 암시적으로 포함합니다.

objc

이 속성을 Objective-C로 표현될 수 있는 모든 선언에 적용합니다 - 예를 들어 중첩되지 않은 클래스, 프로토콜, 제네릭이 아닌 열거형(정수 원시값 타입으로 제한), 클래스 및 프로토콜의 프로퍼티, 메서드(getter와 setter 포함), 프로토콜의 옵셔널 멤버, 이니셜라이저, 서브스크립트에 적용됩니다. `objc` 속성은 해당 선언이 Objective-C 코드에서 사용 가능함을 컴파일러에게 알려줍니다.

이 속성을 확장에 적용하면 명시적으로 `nonobjc` 속성으로 표시되지 않은 확장의 모든 멤버에 적용하는 것과 동일한 효과를 가집니다.

컴파일러는 암시적으로 Objective-C에 정의된 모든 클래스의 하위 클래스에 `objc` 속성을 추가합니다. 그러나 하위 클래스는 제네릭이 아니어야 하며, 제네릭 클래스를 상속해선 안됩니다. 이러한 기준을 충족하는 하위 클래스에 명시적으로 `objc` 속성을 추가하여 아래에서 설명한대로 Objective-C 이름을 지정할 수 있습니다. `objc` 속성으로 표시된 프로토콜은 이 속성이 표시되지 않은 프로토콜을 상속할 수 없습니다.

`objc` 속성은 다음과 같은 경우에 암시적으로 추가됩니다:

- 선언이 하위 클래스의 재정의이고, 하위 클래스의 선언이 `objc` 속성을 가지고 있습니다.
- 선언이 `objc` 속성을 가지는 프로토콜의 요구사항을 충족합니다.
- 선언이 `IBAction`, `IBSegueAction`, `IBOutlet`, `IBDesignable`, `IBInspectable`, `NSManaged`, `GKInspectable` 속성을 가지고 있습니다.

열거형에 `objc` 속성을 적용하면, 각 열거형 케이스는 열거형 이름과 케이스 이름의 연결로 Objective-C 코드에 노출됩니다. 케이스 이름의 첫 번째 문자는 대문자입니다. 예를 들어 Swift `Planet` 열거형의 `venus`라는 케이스는 Objective-C 코드에서 `PlanetVenus`라는 케이스로 노출됩니다.

`objc` 속성은 선택적으로 단일 속성 인자를 받으며, 이 인자는 식별자로 구성됩니다. 이 식별자는 `objc` 속성을 적용하는 엔티티에 대해 Objective-C에 노출될 이름을 지정합니다. 이 인자를 사용하여 클래스, 열거형, 열거형 케이스, 프로토콜, 메서드, getter, setter, 이니셜라이저의 이름을 지정할 수 있습니다. 클래스, 프로토콜, 열거형에 대해 Objective-C 이름을 지정하는 경우, Objective-C를 사용한 프로그래밍 (Programming with Objective-C)의 규칙 (Conventions)에서 설명한대로 이름에 세 글자 접두사를 포함해야 합니다. 아래 예시는 `ExampleClass`의 `enabled` 프로퍼티에 대한 getter를 프로퍼티 이름 자체 대신 `isEnabled`로 Objective-C 코드에 노출합니다.

```
class ExampleClass: NSObject {
    @objc var enabled: Bool {
        @objc(isEnabled) get {
            // Return the appropriate value
        }
    }
}
```

더 자세한 내용은 Objective-C에 Swift 가져오기 (Importing Swift into Objective-C)를 참고바랍니다.

참고

`objc` 속성의 인자는 해당 선언에 대한 런타임 이름도 변경할 수 있습니다. `NSClassFromString(_:)`과 같이 Objective-C 런타임과 상호작용하는 함수를 호출할 때와 앱의 info.plist 파일에 클래스 이름을 지정할 때 런타임 이름을 사용합니다. 인자를 전달하여 이름을 지정하면, 해당 이름은 Objective-C 코드와 런타임 이름으로 사용됩니다. 인자를 생략하면, Objective-C 코드에서 사용된 이름은 Swift 코드의 이름과 동일하고 런타임 이름은 일반 Swift 컴파일러 이름 변경 규칙을 따릅니다.

objcMembers

이 속성을 클래스 선언에 적용하여 클래스의 모든 Objective-C 호환 멤버, 확장, 하위 클래스, 그리고 모든 확장의 하위 클래스에 암시적으로 `objc` 속성을 적용합니다.

대부분의 코드는 필요한 선언만 노출시키기 위해 대신 `objc` 속성을 사용합니다. 많은 선언의 노출이 필요하다면 `objc` 속성을 가진 확장 내에 그룹화할 수 있습니다. `objcMembers` 속성은 Objective-C 런타임의 인트로스펙션 기능(introspection facilities)을 많이 사용하는 라이브러리에 대한 편의를 위한 것입니다. 필요하지 않은 곳에 `objc` 속성을 적용하면 바이너리 크기를 증가시키고 성능에 부정적인 영향을 미칠 수 있습니다.

preconcurrency

이 속성을 선언에 적용하여 엄격한 동시성 검사를 억제합니다. 다음과 같은 선언에 이 속성을 적용할 수 있습니다:

- 임포트(Imports)
- 구조체(Structures), 클래스(classes), 액터(actors)
- 열거형(Enumerations)과 열거형 케이스(enumeration cases)
- 프로토콜(Protocols)
- 변수(Variables)와 상수(constants)
- 서브스크립트(Subscripts)
- 이니셜라이저(Initializers)
- 함수(Functions)

임포트(import) 선언에서 이 속성은 임포트한 모듈의 타입을 사용하는 코드에서 동시성 검사의 엄격함을 줄입니다. 특히 명시적으로 non-sendable로 표시되지 않은 임포트한 모듈의 타입은 sendable 타입을 요구하는 컨텍스트에서 사용될 수 있습니다.

다른 선언에서 이 속성은 선언되는 기호를 시용하는 코드에 대해 동시성 검사의 엄격성을 감소시킵니다. 최소한의 동시성 검사를 가지는 범위에서 이 기호를 사용하면, `Sendable` 요구사항이나 전역 액터와 같은 해당 기호로 지정된 동시성 관련 제약조건이 검사되지 않습니다.

이 속성은 엄격한 동시성 검사로 코드를 변환하는데 다음과 같이 사용할 수 있습니다:

1. 엄격한 검사를 활성화합니다.
2. 엄격한 검사를 활성화하지 않은 모듈에 대해 임포트 선언에 `preconcurrency` 속성을 표시합니다.
3. 모듈을 엄격한 검사로 변환 후에, `preconcurrency` 속성을 삭제합니다. 컴파일러는 임포트에 있는 `preconcurrency` 속성이 더 이상 영향을 미치지 않고 삭제되어야 하는 위치에 경고를 나타냅니다.

다른 선언의 경우, 엄격한 검사에 대한 변환이 되지 않은 클라이언트가 있는 경우에, 동시성 관련 제약조건을 선언에 추가할 때 `preconcurrency` 속성을 추가합니다. `preconcurrency` 속성은 클라언트가 모두 변환된 후에 삭제합니다.

Objective-C의 선언은 항상 `preconcurrency` 속성이 선언된 것처럼 임포트됩니다.

propertyWrapper

이 속성을 클래스, 구조체, 열거형 선언에 적용하여 프로퍼티 래퍼(protperty wrapper)로 사용합니다. 타입에 이 속성을 적용하면, 타입과 동일한 이름으로 커스텀 속성을 생성합니다. 이 새로운 속성을 클래스, 구조체, 열거형의 프로퍼티에 적용하여 래퍼 타입의 인스턴스로 프로퍼티에 대한 접근을 감쌉니다; 이 속성을 지역 저장 변수 선언에 적용하면 같은 방식으로 변수에 대한 접근을 감쌉니다. 연산 변수, 전역 변수, 상수는 프로퍼티 래퍼를 사용할 수 없습니다.

래퍼는 `wrappedValue` 인스턴스 프로퍼티를 정의해야 합니다. 프로퍼티의 *래핑값(wrapped value)*은 이 프로퍼티의 getter와 setter를 노출하는 값입니다. 대부분의 경우 `wrappedValue`는 연산값이지만, 저장값일 수도 있습니다. 래퍼는 래핑값에 필요한 기본 저장소를 정의하고 관리합니다. 컴파일러는 래핑 프로퍼티의 이름 앞에 언더바(_)로 래퍼 타입의 인스턴스에 대해 저장소를 합성합니다 - 예를 들어 `someProperty`에 대한 래퍼는 `_someProperty`로 저장됩니다. 래퍼에 대한 합성된 저장소는 `private`의 접근 제어 수준을 가집니다.

프로퍼티 래퍼를 가지는 프로퍼티는 `willSet`과 `didSet` 블록을 포함할 수 있지만 컴파일러가 합성한 `get`이나 `set` 블록을 재정의할 수 없습니다.

Swift는 프로퍼티 래퍼의 초기화에 대해 두 가지의 편의 문법(syntactic sugar)을 제공합니다. 래핑값의 정의에서 할당 문법을 사용하여 할당의 오른쪽에 있는 표현식을 프로퍼티 래퍼 이니셜라이저의 `wrappedValue` 매개변수로 전달할 수 있습니다. 또한 프로퍼티에 속성을 적용할 때 인자를 제공할 수도 있으며 이 인자는 프로퍼티 래퍼의 이니셜라이저로 전달됩니다. 예를 들어 아래 코드에서 `SomeStruct`는 `SomeWrapper`가 정의한 각 이니셜라이저를 호출합니다.

```
@propertyWrapper
struct SomeWrapper {
    var wrappedValue: Int
    var someValue: Double
    init() {
        self.wrappedValue = 100
        self.someValue = 12.3
    }
    init(wrappedValue: Int) {
        self.wrappedValue = wrappedValue
        self.someValue = 45.6
    }
    init(wrappedValue value: Int, custom: Double) {
        self.wrappedValue = value
        self.someValue = custom
    }
}

struct SomeStruct {
    // Uses init()
    @SomeWrapper var a: Int

    // Uses init(wrappedValue:)
    @SomeWrapper var b = 10

    // Both use init(wrappedValue:custom:)
    @SomeWrapper(custom: 98.7) var c = 30
    @SomeWrapper(wrappedValue: 30, custom: 98.7) var d
}
```

래핑 프로퍼티의 *투영값(projected value)*은 프로퍼티 래퍼가 추가 기능을 노출하기 위해 사용할 수 있는 두 번째 값입니다. 프로퍼티 래퍼 타입의 작성자는 투영값의 의미를 결정하고 투영값이 노출하는 인터페이스를 정의할 책임이 있습니다. 프로퍼티 래퍼에서 값을 예상하려면, 래퍼 타입에 `projectedValue` 인스턴스 프로퍼티를 정의합니다. 컴파일러는 래핑 프로퍼티의 이름 앞에 달러 기호(`$`)로 투영값에 대한 식별자를 합성합니다 - 예를 들어 `someProperty`에 대한 투영값은 `$someProperty`입니다. 투영값은 기존의 래핑 프로퍼티와 동일한 접근 제어 수준을 가집니다.

```
@propertyWrapper
struct WrapperWithProjection {
    var wrappedValue: Int
    var projectedValue: SomeProjection {
        return SomeProjection(wrapper: self)
    }
}
struct SomeProjection {
    var wrapper: WrapperWithProjection
}

struct SomeStruct {
    @WrapperWithProjection var x = 123
}
let s = SomeStruct()
s.x           // Int value
s.$x          // SomeProjection value
s.$x.wrapper  // WrapperWithProjection value
```

투영값이 원시 식별자로 이름을 지정한 변수를 래핑하는 경우, 해당 값의 합성 저장소와 투영값을 참조할 때도 원시 식별자를 사용합니다. 언더스코어나 달러 기호는 백틱 안에 작성합니다; 예를 들어 someValue. _some raw identifier 는 이름에 공백이 있는 프로퍼티의 합성 저장소를 참조하며, someValue. $some raw identifier 는 해당 프로퍼티의 투영값을 참조합니다.

resultBuilder

이 속성을 클래스, 구조체, 열거형에 적용하여 해당 타입을 결과 빌더(result builder)로 사용합니다. *결과 빌더(result builder)*는 중첩된 데이터 구조를 단계별로 구축하는 타입입니다. 결과 빌더를 사용하여 중첩된 데이터 구조를 자연스럽고 선언적인 방식으로 생성하기 위한 도메인 특화 언어(DSL)를 구현합니다. `resultBuilder` 속성을 어떻게 사용하는지에 대한 예시는 결과 빌더 (Result Builders)를 참고바랍니다.

결과-구축 메서드 (Result-Building Methods)

결과 빌더는 아래에 설명된 정적 메서드를 구현합니다. 결과 빌더의 모든 기능은 정적 메서드를 통해 노출되므로 해당 타입의 인스턴스를 초기화하지 않습니다. 결과 빌더는 `buildBlock(_:)` 메서드를 구현하거나 `buildPartialBlock(first:)`와 `buildPartialBlock(accumulated:next:)` 메서드를 모두 구현해야 합니다. DSL에서 추가 기능을 가능하게 하는 다른 메서드는 선택 사항입니다. 결과 빌더 타입의 선언은 프로토콜 준수를 포함할 필요가 없습니다.

정적 메서드의 설명은 세 가지 타입을 플레이스 홀더로 사용합니다. `Expression` 타입은 결과 빌더의 입력 타입에 대한 플레이스 홀더이고, `Component`는 부분 결과 타입에 대한 플레이스 홀더이며, `FinalResult`는 결과 빌더가 생성하는 결과 타입에 대한 플레이스 홀더입니다. 이러한 타입은 결과 빌더가 사용하는 실제 타입으로 대체해야 합니다. 결과-구축 메서드가 `Expression`이나 `FinalResult`에 대한 타입을 지정하지 않으면 `Component`와 동일한 타입으로 기본 설정됩니다.

결과-구축 메서드는 다음과 같습니다:

- `static func buildBlock(_ components: Component...) -> Component`: 부분 결과의 배열을 단일 부분 결과로 결합합니다.
- `static func buildPartialBlock(first: Component) -> Component`: 첫 번째 컴포넌트로부터 부분 결과 컴포넌트를 구축합니다. 한 번에 하나의 컴포넌트로 블록을 구축하는 것을 지원하기위해 이 메서드와 `buildPartialBlock(accumulated:next:)` 메서드를 모두 구현합니다. `buildBlock(_:)`에 비해, 이 접근은 인자의 다양한 수를 처리하는 일반적인 오버로드의 필요성을 줄여줍니다.
- `static func buildPartialBlock(accumulated: Component, next: Component) -> Component`: 축적된 컴포넌트와 새로운 컴포넌트를 결합하여 부분 결과 컴포넌트를 구축합니다. 한 번에 하나의 컴포넌트로 블록을 구축하는 것을 지원하기위해 이 메서드와 `buildPartialBlock(first:)` 메서드를 모두 구현합니다. `buildBlock(_:)`에 비해, 이 접근은 인자의 다양한 수를 처리하는 일반적인 오버로드의 필요성을 줄여줍니다.

결과 빌더는 위에 나열된 세 가지 블록-구축 메서드 모두 구현할 수 있습니다. 이 경우에 가용성에 따라 호출되는 메서드가 결정됩니다. 기본적으로 Swift는 `buildPartialBlock(first:)`와 `buildPartialBlock(accumulated:next:)` 메서드를 호출합니다. Swift가 대신 `buildBlock(_:)`을 호출하도록 하려면 `buildPartialBlock(first:)`와 `buildPartialBlock(accumulated:next:)`에 지정한 가용성보다 먼저 결과 빌더를 감싸고 있는 선언부에 가용성을 지정해야 합니다.

추가적인 결과-구축 메서드는 다음과 같습니다:

- `static func buildOptional(_ component: Component?) -> Component`: `nil`이 가능한 부분 결과로부터 부분 결과를 구축합니다. `else` 절을 포함하지 않은 `if` 구문을 지원하려면 이 메서드를 구현해야 합니다.
- `static func buildEither(first: Component) -> Component`: 일부 조건에 따라 값이 달라지는 부분 결과를 구축합니다. `switch` 구문과 `else` 절을 포함하는 `if` 구문을 제공하려면 이 메서드와 `buildEither(second:)`를 모두 구현해야 합니다.
- `static func buildEither(second: Component) -> Component`: 일부 조건에 따라 값이 달리지는 부분 결과를 구축합니다. `switch` 구문과 `else` 절을 포함하는 `if` 구문을 제공하려면 이 메서드와 `buildEither(first:)`를 모두 구현해야 합니다.
- `static func buildArray(_ components: [Component]) -> Component`: 부분 결과 배열로부터 부분 결과를 구축합니다. `for` 루프를 지원하려면 이 메서드를 구현해야 합니다.
- `static func buildExpression(_ expression: Expression) -> Component`: 표현식으로부터 부분 결과를 구축합니다. 이 메서드를 구현하여 전처리 (예: 표현식을 내부 타입으로 변환)를 수행하거나 사용하는 곳에서 타입 추론을 위한 추가 정보를 제공하기 위해 구현할 수 있습니다.
- `static func buildFinalResult(_ component: Component) -> FinalResult`: 부분 결과로부터 최종 결과를 구축합니다. 부분과 최종 결과에 대한 다른 타입을 사용하거나 결과를 반환하기 전에 다른 후처리를 수행하는 결과 빌더의 부분으로 이 메서드를 구현할 수 있습니다.
- `static func buildLimitedAvailability(_ component: Component) -> Component`: 타입 정보를 지우는 부분 결과를 구축합니다. 가용성을 검사하는 컴파일러-제어 구문 외부로 타입 정보가 전파되는 것을 막기위해 이 메서드를 구현할 수 있습니다.

예를 들어 아래 코드는 정수의 배열을 구축하는 간단한 결과 빌더를 정의합니다. 이 코드는 타입 별칭으로 `Component`와 `Expression`을 정의하여 아래 예시를 위의 메서드 목록과 쉽게 일치하도록 만듭니다.

```
@resultBuilder
struct ArrayBuilder {
    typealias Component = [Int]
    typealias Expression = Int
    static func buildExpression(_ element: Expression) -> Component {
        return [element]
    }
    static func buildOptional(_ component: Component?) -> Component {
        guard let component = component else { return [] }
        return component
    }
    static func buildEither(first component: Component) -> Component {
        return component
    }
    static func buildEither(second component: Component) -> Component {
        return component
    }
    static func buildArray(_ components: [Component]) -> Component {
        return Array(components.joined())
    }
    static func buildBlock(_ components: Component...) -> Component {
        return Array(components.joined())
    }
}
```

결과 변환 (Result Transformations)

다음 구문 변환은 결과-구축 구문을 사용하는 코드에서 결과 빌더 타입의 정적 메서드를 호출하는 코드로 변환하기 위해 재귀적으로 적용됩니다:

- 결과 빌더가 `buildExpression(_:)` 메서드가 있으면, 각 표현식은 해당 메서드에 대한 호출이 됩니다. 이 변환은 항상 가장 먼저 일어납니다. 예를 들어 다음의 선언은 동등합니다:

```
@ArrayBuilder var builderNumber: [Int] { 10 }
var manualNumber = ArrayBuilder.buildExpression(10)
```

- 할당 구문은 표현식처럼 변환되지만 `()`로 평가되는 것으로 이해됩니다. 할당을 특별히 처리하기 위해 `()` 타입의 인자를 가지는 `buildExpression(_:)` 의 오버로드를 정의할 수 있습니다.

- 가용성 조건을 검사하는 분기 구문은 `buildLimitedAvailability(_:)` 메서드가 구현되어 있으면, 해당 메서드에 대한 호출이 됩니다. `buildLimitedAvailability(_:)`가 구현되어 있지 않다면, 가용성 조건을 검사하는 분기 구문은 다른 분기 구문과 동일한 변환을 사용합니다. 이 변환은 `buildEither(first:)`, `buildEither(second:)`, `buildOptional(_:)`에 대한 호출로 변환되기 전에 발생합니다.

  `buildLimitedAvailability(_:)` 메서드는 어떤 분기를 사용하는지에 따라 변경되는 타입 정보를 지우기 위해 사용합니다. 예를 들어 아래의 `buildEither(first:)`와 `buildEither(second:)` 메서드는 두 분기에 대한 타입 정보를 캡처하는 제네릭 타입을 사용합니다.

```
protocol Drawable {
    func draw() -> String
}
struct Text: Drawable {
    var content: String
    init(_ content: String) { self.content = content }
    func draw() -> String { return content }
}
struct Line<D: Drawable>: Drawable {
    var elements: [D]
    func draw() -> String {
        return elements.map { $0.draw() }.joined(separator: "")
    }
}
struct DrawEither<First: Drawable, Second: Drawable>: Drawable {
    var content: Drawable
    func draw() -> String { return content.draw() }
}

@resultBuilder
struct DrawingBuilder {
    static func buildBlock<D: Drawable>(_ components: D...) -> Line<D> {
        return Line(elements: components)
    }
    static func buildEither<First, Second>(first: First)
        -> DrawEither<First, Second> {
            return DrawEither(content: first)
    }
    static func buildEither<First, Second>(second: Second)
        -> DrawEither<First, Second> {
            return DrawEither(content: second)
    }
}
```

그러나 이 접근방식은 가용성 검사가 있는 코드에서 문제가 발생합니다:

```
@available(macOS 99, *)
struct FutureText: Drawable {
    var content: String
    init(_ content: String) { self.content = content }
    func draw() -> String { return content }
}
@DrawingBuilder var brokenDrawing: Drawable {
    if #available(macOS 99, *) {
        FutureText("Inside.future")  // Problem
    } else {
        Text("Inside.present")
    }
}
// The type of brokenDrawing is Line<DrawEither<Line<FutureText>, Line<Text>>>
```

위의 코드에서 `FutureText`는 `DrawEither` 제네릭 타입의 타입 중 하나이므로 `brokenDrawing`의 타입의 부분으로 나타납니다. 이 타입이 명시적으로 사용되지 않더라도 런타임에 `FutureText`를 사용할 수 없는 경우 프로그램이 충돌할 수 있습니다.

이 문제를 해결하려면 `buildLimitedAvailability(_:)` 메서드를 구현하여 항상 사용가능한 타입을 반환하여 타입 정보를 지워야 합니다. 예를 들어 아래의 코드는 가용성 검사로부터 `AnyDrawable` 값을 구축합니다.

```
struct AnyDrawable: Drawable {
    var content: Drawable
    func draw() -> String { return content.draw() }
}
extension DrawingBuilder {
    static func buildLimitedAvailability(_ content: some Drawable) -> AnyDrawable {
        return AnyDrawable(content: content)
    }
}

@DrawingBuilder var typeErasedDrawing: Drawable {
    if #available(macOS 99, *) {
        FutureText("Inside.future")
    } else {
        Text("Inside.present")
    }
}
// The type of typeErasedDrawing is Line<DrawEither<AnyDrawable, Line<Text>>>
```

- 분기 구문은 `buildEither(first:)`와 `buildEither(second:)` 메서드에 대한 일련의 중첩된 호출이 됩니다. 구문의 조건과 케이스는 이진 트리의 리프 노드에 매핑되고 구문은 루트 노드에서 해당 리프 노드로의 경로를 따라 `buildEither` 메서드에 대한 중첩된 호출이 됩니다.

  예를 들어 세 가지 케이스가 있는 switch 구문을 작성하면, 컴파일러는 세 개의 리프 노드를 가진 이진 트리를 사용합니다. 마찬가지로 루트 노드에서 두 번째 케이스까지의 경로는 “second child”이고 그 다음이 “first child”이므로 해당 케이스는 `buildEither(first: buildEither(second: ... ))`와 같은 중첩된 호출이 됩니다. 다음의 선언은 동일합니다:

```
let someNumber = 19
@ArrayBuilder var builderConditional: [Int] {
    if someNumber < 12 {
        31
    } else if someNumber == 19 {
        32
    } else {
        33
    }
}

var manualConditional: [Int]
if someNumber < 12 {
    let partialResult = ArrayBuilder.buildExpression(31)
    let outerPartialResult = ArrayBuilder.buildEither(first: partialResult)
    manualConditional = ArrayBuilder.buildEither(first: outerPartialResult)
} else if someNumber == 19 {
    let partialResult = ArrayBuilder.buildExpression(32)
    let outerPartialResult = ArrayBuilder.buildEither(second: partialResult)
    manualConditional = ArrayBuilder.buildEither(first: outerPartialResult)
} else {
    let partialResult = ArrayBuilder.buildExpression(33)
    manualConditional = ArrayBuilder.buildEither(second: partialResult)
}
```

- `else` 절이 없는 `if` 구문과 같이 값을 생성하지 않을 수도 있는 분기 구문은 `buildOptional(_:)`에 대한 호출이 됩니다. `if` 구문의 조건이 충족되면, 코드 블록이 변환되고 인자로 전달됩니다; 그렇지 않으면 `buildOptional(_:)`이 `nil`을 인자로 받아 호출됩니다. 예를 들어 다음의 선언은 동일합니다:

```
@ArrayBuilder var builderOptional: [Int] {
    if (someNumber % 2) == 1 { 20 }
}

var partialResult: [Int]? = nil
if (someNumber % 2) == 1 {
    partialResult = ArrayBuilder.buildExpression(20)
}
var manualOptional = ArrayBuilder.buildOptional(partialResult)
```

- 결과 빌더가 `buildPartialBlock(first:)`와 `buildPartialBlock(accumulated:next:)` 메서드를 구현하는 경우, 코드 블록이나 `do` 구문은 해당 메서드를 호출합니다. 블록의 첫 번째 구문은 `buildPartialBlock(first:)` 메서드의 인자로 변환되고, 나머지 구문은 `buildPartialBlock(accumulated:next:)` 메서드에 대한 중첩된 호출이 됩니다. 예를 들어 다음 선언은 동일한 구문입니다:

```
struct DrawBoth<First: Drawable, Second: Drawable>: Drawable {
    var first: First
    var second: Second
    func draw() -> String { return first.draw() + second.draw() }
}

@resultBuilder
struct DrawingPartialBlockBuilder {
    static func buildPartialBlock<D: Drawable>(first: D) -> D {
        return first
```

```
    }
    static func buildPartialBlock<Accumulated: Drawable, Next: Drawable>(
        accumulated: Accumulated, next: Next
    ) -> DrawBoth<Accumulated, Next> {
        return DrawBoth(first: accumulated, second: next)
    }
}

@DrawingPartialBlockBuilder var builderBlock: some Drawable {
    Text("First")
    Line(elements: [Text("Second"), Text("Third")])
    Text("Last")
}

let partialResult1 = DrawingPartialBlockBuilder.buildPartialBlock(first: Text("first"))
let partialResult2 = DrawingPartialBlockBuilder.buildPartialBlock(
    accumulated: partialResult1,
    next: Line(elements: [Text("Second"), Text("Third")])
)
let manualResult = DrawingPartialBlockBuilder.buildPartialBlock(
    accumulated: partialResult2,
    next: Text("Last")
)
```

- 그렇지 않으면 코드 블록이나 do 구문은 `buildBlock(_:)` 메서드에 대한 호출이 됩니다. 블록 내부의 각 구문은 한 번에 하나씩 변환되고 `buildBlock(_:)` 메서드에 대한 인자가 됩니다. 예를 들어 다음의 선언은 동일합니다:

```
@ArrayBuilder var builderBlock: [Int] {
    100
    200
    300
}

var manualBlock = ArrayBuilder.buildBlock(
    ArrayBuilder.buildExpression(100),
    ArrayBuilder.buildExpression(200),
    ArrayBuilder.buildExpression(300)
)
```

- `for` 루프는 임시 변수, `for` 루프, `buildArray(_:)` 메서드에 대한 호출이 됩니다. 새로운 `for` 루프는 시퀀스를 반복하고 각 부분 결과를 해당 배열에 추가합니다. 임시 배열은 `buildArray(_:)` 호출에 인자로 전달됩니다. 예를 들어 다음의 선언은 동일합니다:

```
@ArrayBuilder var builderArray: [Int] {
    for i in 5...7 {
        100 + i
    }
}

var temporary: [[Int]] = []
for i in 5...7 {
    let partialResult = ArrayBuilder.buildExpression(100 + i)
    temporary.append(partialResult)
}
let manualArray = ArrayBuilder.buildArray(temporary)
```

- 결과 빌더가 `buildFinalResult(_:)` 메서드를 가지고 있으면, 최종 결과는 해당 메서드에 대한 호출이 됩니다. 이 변환은 항상 마지막에 일어납니다.

변환 동작은 임시 변수로 설명되지만 결과 빌더를 사용하는 것은 나머지 코드에서 볼 수 있는 새로운 선언을 생성하지 않습니다.

결과 빌더 변환하는 코드에서 `break`, `continue`, `defer`, `guard`, `return` 구문, `while` 구문, `do-catch` 구문을 사용할 수 없습니다.

변환 과정은 코드에서 선언을 변경하지 않으므로 임시 상수와 변수를 사용하여 표현식을 부분적으로 구축할 수 있습니다. 또한 `throw` 구문, 컴파일 시점 진단 구문, `return` 구문이 포함된 클로저도 변경하지 않습니다.

가능하면 변환이 병합됩니다. 예를 들어 표현식 `4 + 5 * 6`은 해당 함수를 여러 번 호출하는 대신 `buildExpression(4 + 5 * 6)`이 됩니다. 마찬가지로 중첩된 분기 구문은 `buildEither` 메서드에 대한 단일 이진 호출 트리가 됩니다.

커스텀 결과-빌더 속성 (Custom Result-Builder Attributes)

결과 빌더 타입을 생성하면 동일한 이름의 커스텀 속성을 생성합니다. 이 속성은 다음 위치에 적용할 수 있습니다:

- 함수 선언에 적용하면, 결과 빌더는 함수의 본문을 구축합니다.
- getter를 포함하는 변수나 서브스크립트 선언에 적용하면, 결과 빌더는 getter의 본문을 구축합니다.
- 함수 선언의 매개변수에 적용하면, 결과 빌더는 해당 인자로 전달되는 클로저의 본문을 구축합니다.

결과 빌더 속성을 적용하는 것은 ABI 호환성에 영향을 주지 않습니다. 매개변수에 결과 빌더 속성을 적용하는 것은 해당 속성을 함수의 인터페이스의 부분으로 만들고 이것은 소스 호환성에 영향을 줄 수 있습니다.

requires_stored_property_inits

이 속성을 클래스 선언에 적용하여 클래스 내의 모든 저장 프로퍼티가 정의의 일부로 기본값을 제공하도록 요구합니다. 이 속성은 `NSManagedObject`를 상속하는 모든 클래스에 대해 추론됩니다.

testable

이 속성을 `import` 선언에 적용하여 모듈의 코드를 테스트하기 쉽게 접근 제어를 변경하여 해당 모듈을 가져옵니다. 임포트한 모듈에서 `internal` 접근-수준 수정자로 표시된 엔티티는 `public` 접근-수준 수정자로 선언된 것처럼 가져옵니다. `internal`이나 `public` 접근-수준 수정자로 표시된 클래스와 클래스 멤버는 `open` 접근-수준 수정자로 선언된 것처럼 가져옵니다. 임포트한 모듈은 테스트가 활성화된 상태로 컴파일되어야 합니다.

UIApplicationMain

사용 중단

이 속성은 사용 중단되었습니다; 대신 main 속성을 사용합니다. Swift 6에서 이 속성을 사용하면 컴파일 시 오류가 발생합니다.

이 속성을 클래스에 적용하여 앱 델리게이트임을 나타냅니다. 이 속성을 사용하는 것은 `UIApplicationMain` 함수를 호출하는 것과 이 클래스의 이름을 델리게이트 클래스의 이름으로 전달하는 것과 동일합니다.

이 속성을 사용하지 않으면, `UIApplicationMain(_:_:_:_:)` 함수를 호출하는 최상위 수준의 코드가 포함된 `main.swift` 파일을 제공해야 합니다. 예를 들어 주 클래스로 `UIApplication`의 커스텀 하위 클래스를 사용하면 이 속성을 사용하는 것 대신 `UIApplicationMain(_:_:_:_:)` 함수를 호출해야 합니다.

실행 파일을 만들기 위해 컴파일 한 Swift 코드는 최상위-수준 코드 (Top-Level Code)에서 설명한대로 하나의 최상위-수준 시작점을 포함해야 합니다.

unchecked

이 속성을 타입 선언의 채택된 프로토콜 목록의 일부로 프로토콜 타입에 적용하여 해당 프로토콜의 요구사항 적용을 끕니다.

지원되는 유일한 프로토콜은 Sendable입니다.

usableFromInline

이 속성을 함수, 메서드, 연산 프로퍼티, 서브스크립트, 이니셜라이저, 디이니셜라이저 선언에 적용하여 해당 선언과 동일한 모듈에 정의된 인라인 가능 코드에서 해당 기호를 사용할 수 있도록 허용합니다. 선언은 `internal` 접근 수준 수정자를 가지고 있어야 합니다. `usableFromInline`으로 표시된 구조체나 클래스는 프로퍼티에 대해 public이나 `usableFromInline`인 타입만 사용할 수 있습니다. `usableFromInline`으로 표시된 열거형은 케이스의 원시값과 연관값에 대해 public이나 `usableFromInline`인 타입만 사용할 수 있습니다.

`public` 접근 수준 수정자와 같이 이 속성은 모듈의 공개 인터페이스의 부분으로 선언을 노출합니다. `public`과 다르게 컴파일러는 `usableFromInline`으로 표시된 선언이 선언의 기호를 내보내지더라도 모듈 외부의 코드에서 이름으로 참조되는 것을 허용하지 않습니다. 그러나 모듈 외부의 코드는 런타임 동작을 사용하여 선언의 기호와 상호작용할 수 있습니다.

`inlinable` 속성으로 표시된 선언은 암시적으로 인라인 가능 코드에서 사용 가능합니다. `inlinable`이나 `usableFromInline` 중 하나를 `internal` 선언에 적용할 수 있지만 두 속성 모두 적용하는 것은 오류가 발생합니다.

warn_unqualified_access

이 속성을 최상위 함수, 인스턴스 메서드, 클래스나 정정 메서드에 적용하여 모듈 이름, 타입 이름, 인스턴스 변수나 상수와 같은 선행 한정자 없이 해당 함수나 메서드 사용될 때 경고를 발생시킵니다. 이 속성은 동일한 범위에서 접근할 수 있는 동일한 이름을 가지는 함수 간의 모호성을 줄이는데 도움을 주기 위해 사용됩니다.

예를 들어 Swift 표준 라이브러리는 최상위 `min(_:_:)` 함수와 비교 가능한 요소가 있는 시퀀스에 대한 `min()` 메서드 모두 포함합니다. 시퀀스 메서드는 `Sequence` 확장 내에서 어느 하나를 사용하려할 때 혼동을 줄이기 위해 `warn_unqualified_access` 속성으로 선언됩니다.

인터페이스 빌더에서 사용되는 선언 속성 (Declaration Attributes Used by Interface Builder)

인터페이스 빌더(Interface Builder) 속성은 Xcode와 동기화하기 위해 인터페이스 빌더에서 사용하는 선언 속성입니다. Swift는 다음의 인터페이스 빌더 속성을 제공합니다: `IBAction`, `IBSegueAction`, `IBOutlet`, `IBDesignable`, `IBInspectable`. 이 속성은 개념적으로 Objective-C와 동일합니다.

`IBOutlet`과 `IBInspectable` 속성은 클래스의 프로퍼티 선언에 적용합니다. `IBAction`과 `IBSegueAction` 속성은 클래스의 메서드 선언에 적용하고 `IBDesignable` 속성은 클래스 선언에 적용합니다.

`IBAction`, `IBSegueAction`, `IBOutlet`, `IBDesignable`, `IBInspectable` 속성을 적용하는 것은 `objc` 속성도 암시적으로 포함합니다.

타입 속성 (Type Attributes)

타입 속성은 타입에만 적용할 수 있습니다.

autoclosure

이 속성을 적용하여 인자를 받지 않는 클로저로 표현식을 자동으로 감싸서 해당 표현식의 평가를 지연시킵니다. 이 속성은 함수나 메서드 선언의 매개변수 타입에 적용하여 인자를 받지 않고 표현식의 타입과 같은 값을 반환하는 함수 타입의 매개변수에 사용합니다. `autoclosure` 속성을 어떻게 사용하는지에 대한 예시는 자동 클로저 (Autoclosures)와 함수 타입 (Function Type)을 참고바랍니다.

convention

이 속성을 함수의 타입에 적용하여 호출 규약(calling conventions)을 나타냅니다.

`convention` 속성은 항상 다음의 인자 중 하나와 함께 사용됩니다:

- `swift` 인자는 Swift 함수 참조를 나타냅니다. 이것은 Swift에서 함수 값에 대한 표준 호출 규약입니다.
- `block` 인자는 Objective-C 호환 블록 참조를 나타냅니다. 함수 값은 블록 객체에 대한 참조로 표현되며 이 객체는 자체 호출 함수를 객체 내에 내장하는 `id`-호환 Objective-C 객체입니다. 호출 함수는 C 호출 규약을 사용합니다.
- `c` 인자는 C 함수 참조를 나타냅니다. 함수 값은 어떤 컨텍스트를 전달하지 않으며, C 호출 규약을 사용합니다.

몇 가지 예외를 제외하고 어떤 호출 규약을 가진 함수라도 다른 호출 규약이 필요한 함수에 사용할 수 있습니다. 제네릭이 아닌 전역 함수, 지역 변수를 캡처하지 않는 지역 함수, 지역 변수를 캡처하지 않는 클로저는 C 호출 규약으로 변환될 수 있습니다. 다른 Swift 함수는 C 호출 규약으로 변환될 수 없습니다. Objective-C 블록 호출 규약을 가지는 함수는 C 호출 규약으로 변환될 수 없습니다.

escaping

이 속성을 함수나 메서드 선언의 매개변수 타입에 적용하여 해당 매개변수의 값이 나중에 실행하기 위해 저장될 수 있음을 나타냅니다. 이는 값이 호출 수명보다 오래 지속될 수 있음을 의미합니다. `escaping` 타입 속성을 가지는 함수 타입 매개변수는 프로퍼티나 메서드에 대해 `self.`를 명시적으로 사용해야 합니다. `escaping` 속성을 어떻게 사용하는지에 대한 예시는 탈출 클로저 (Escaping Closures)를 참고바랍니다.

Sendable

이 속성을 함수 타입에 적용하여 함수나 클로저가 Sendable을 나타냅니다. 함수 타입에 이 속성을 적용하는 것은 비함수(non-function) 타입이 Sendable 프로토콜을 준수하는 것과 같은 의미를 가집니다.

이 속성은 함수나 클로저가 Sendable 값을 기대하는 컨텍스트에서 사용되고 함수나 클로저가 Sendable 요구사항을 충족하는 경우 추론됩니다.

Sendable 함수 타입은 Sendable 하지 않은 함수 타입의 하위 타입입니다.

스위치 케이스 속성 (Switch Case Attributes)

스위치 케이스 속성은 스위치 케이스에만 적용할 수 있습니다.

unknown

이 속성을 스위치 케이스에 적용하여 코드가 컴파일될 때 알려진 열거형의 어떤 케이스와도 일치하지 않는 것으로 예상됨을 나타냅니다. `unknown` 속성을 어떻게 사용하는지에 대한 예시는 향후 열거형 케이스 전환 (Switching Over Future Enumeration Cases)을 참고바랍니다.

Grammar of an attribute

attribute → @ attribute-name attribute-argument-clause? attribute-name → identifier
attribute-argument-clause → (balanced-tokens?) attributes → attribute attributes?

balanced-tokens → balanced-token balanced-tokens? balanced-token → (balanced-tokens?)
balanced-token → [balanced-tokens?] balanced-token → { balanced-tokens? } balanced-token → Any identifier, keyword, literal, or operator balanced-token → Any punctuation except (,), [,], {, or }

언어 참조

패턴 (Patterns)

값을 일치시키고 분해합니다.

*패턴(pattern)*은 단일 값이나 복합값의 구조를 나타냅니다. 예를 들어 튜플 `(1, 2)`의 구조는 콤마로 구분된 두 요소의 목록입니다. 패턴은 특정값이 아닌 값의 구조를 나타내기 때문에 다양한 값과 일치시킬 수 있습니다. 예를 들어 패턴 `(x, y)`는 튜플 `(1, 2)`와 서로 다른 두 요소를 가진 튜플과 일치합니다. 패턴을 값과 일치시키는 것 외에도 복합값의 일부나 전체를 추출하고 각 부분을 상수나 변수 이름으로 바인딩할 수 있습니다.

Swift에는 두 가지 기본 패턴이 있습니다: 하나는 모든 종류의 값과 일치하는 패턴이고 다른 하나는 런타임에 지정된 값과 일치하는데 실패할 수 있는 패턴입니다.

패턴의 첫 번째 종류는 단순 변수, 상수, 옵셔널 바인딩에서 값을 분해하는데 사용됩니다. 여기에는 와일드카드 패턴, 식별자 패턴, 이를 포함하는 모든 값 바인딩이나 튜플 패턴이 포함됩니다. 이러한 패턴에 타입 주석을 지정하여 특정 타입의 값만 일치하도록 제한할 수 있습니다.

패턴의 두 번째 종류는 전체 패턴 일치에 사용되며 일치시키려는 값이 런타임에 없을 수 있습니다. 여기에는 열거형 케이스 패턴, 옵셔널 패턴, 표현식 패턴, 타입-캐스팅 패턴을 포함합니다. 이러한 패턴은 `switch` 구문의 케이스 레이블, `do` 구문의 `catch` 절, `if`, `while`, `guard`, `for-in` 구문의 케이스 조건에서 사용합니다.

Grammar of a pattern

pattern → wildcard-pattern type-annotation? pattern → identifier-pattern type-annotation? pattern → value-binding-pattern pattern → tuple-pattern type-annotation? pattern → enum-case-pattern pattern → optional-pattern pattern → type-casting-pattern pattern → expression-pattern

와일드카드 패턴 (Wildcard Pattern)

*와일드카드 패턴(wildcard pattern)*은 어떤 값이든 일치시키고 무시하며 언더바(_)로 구성됩니다. 일치하는 값에 대해 신경쓰지 않을 경우에 와일드카드 패턴을 사용합니다. 예를 들어 다음의 코드는 닫힌 범위 `1...3`을 반복하고, 각 반복마다 범위의 현재값을 무시합니다:

```
for _ in 1...3 {
    // Do something three times.
}
```

Grammar of a wildcard pattern

wildcard-pattern → _

식별자 패턴 (Identifier Pattern)

*식별자 패턴(identifier pattern)*은 모든 값과 일치하고 일치하는 값을 변수나 상수 이름으로 바인딩합니다. 예를 들어 다음의 상수 선언에서 `someValue`는 `Int` 타입의 `42` 값이 일치하는 식별자 패턴입니다:

```
let someValue = 42
```

일치가 성공하면 값 42는 상수 이름 `someValue`에 바인딩(할당)됩니다.

변수나 상수 선언의 왼쪽 패턴이 식별자 패턴이면 식별자 패턴은 암시적으로 값-바인딩 패턴(value-binding pattern)의 하위 패턴입니다.

Grammar of an identifier pattern

identifier-pattern → identifier

값-바인딩 패턴 (Value-Binding Pattern)

*값-바인딩 패턴(value-binding pattern)*은 일치한 값을 변수나 상수 이름에 바인딩합니다. 상수의 이름에 일치되는 값을 바인딩하는 값-바인딩 패턴은 `let` 키워드로 시작합니다; 변수의 이름에 바인딩하면 `var` 키워드로 시작합니다.

값-바인딩 패턴 내의 식별자 패턴은 일치하는 값을 새로운 명명된 변수나 상수로 바인딩합니다. 예를 들어 튜플의 요소를 분해하고 각 요소의 값을 해당 식별자 패턴에 바인딩할 수 있습니다.

```
let point = (3, 2)
switch point {
    // Bind x and y to the elements of point.
case let (x, y):
    print("The point is at (\(x), \(y)).")
}
// Prints "The point is at (3, 2).".
```

위의 예시에서 `let`은 튜플 패턴 `(x, y)`에서 각 식별자 패턴에 적용합니다. 이 동작으로 인해 `switch` 케이스 `case let (x, y):`와 `case (let x, let y):`은 동일한 값에 일치됩니다.

Grammar of a value-binding pattern

value-binding-pattern → `var` pattern | `let` pattern

튜플 패턴 (Tuple Pattern)

*튜플 패턴(tuple pattern)*은 소괄호로 묶인 0개 이상의 패턴의 콤마로 구분된 목록입니다. 튜플 패턴은 해당 튜플 타입의 값에 일치됩니다.

타입 주석을 사용하여 튜플 패턴이 특정 종류의 튜플 타입에만 일치하도록 제한할 수 있습니다. 예를 들어 상수 선언 `let (x, y): (Int, Int) = (1, 2)`에서 튜플 패턴 `(x, y): (Int, Int)`은 두 요소 모두 `Int` 타입의 튜플 타입만 일치됩니다.

튜플 패턴이 `for-in` 구문이나 변수나 상수 선언에서 패턴으로 사용되면, 와일드카드 패턴, 식별자 패턴, 옵셔널 패턴, 이를 포함하는 다른 튜플 패턴만 포함할 수 있습니다. 예를 들어 다음 코드는 튜플 패턴 `(x, 0)`에서 요소 `0`은 표현식 패턴이므로 유효하지 않습니다:

```
let points = [(0, 0), (1, 0), (1, 1), (2, 0), (2, 1)]
// This code isn't valid.
for (x, 0) in points {
    /* ... */
}
```

단일 요소를 포함하는 튜플 패턴을 둘러싼 소괄호는 아무런 효과가 없습니다. 이 패턴은 단일 요소의 타입 값에 일치됩니다. 예를 들어 다음은 동일합니다:

```
let a = 2        // a: Int = 2
let (a) = 2      // a: Int = 2
let (a): Int = 2 // a: Int = 2
```

Grammar of a tuple pattern

tuple-pattern → `(` tuple-pattern-element-list? `)` tuple-pattern-element-list → tuple-pattern-element | tuple-pattern-element `,` tuple-pattern-element-list tuple-pattern-element → pattern | identifier `:` pattern

열거형 케이스 패턴 (Enumeration Case Pattern)

*열거형 케이스 패턴(enumeration case pattern)*은 기존 열거형 타입의 케이스와 일치합니다. 열거형 케이스 패턴은 `switch` 구문 케이스 레이블과 `if`, `while`, `guard`, `for-in` 구문의 케이스 조건에 나타납니다.

일치시키려는 열거형 케이스에 연관값이 있는 경우, 해당 열거형 케이스 패턴은 각 연관값에 대한 요소를 포함하는 튜플 패턴을 지정해야 합니다. 연관값을 포함하는 열거형 케이스를 일치시키기 위해 `switch` 구문을 사용하는 예시는 연관값 (Associated Values)을 참고바랍니다.

열거형 케이스 패턴은 옵셔널로 래핑된 케이스의 값과도 일치합니다. 이 간략한 구문으로 옵셔널 패턴을 생략할 수 있습니다. `Optional`은 열거형으로 구현되므로 `.none`과 `.some`은 열거형 타입의 케이스와 동일한 스위치에 나타날 수 있습니다.

```
enum SomeEnum { case left, right }
let x: SomeEnum? = .left
switch x {
case .left:
    print("Turn left")
case .right:
    print("Turn right")
case nil:
    print("Keep going straight")
}
// Prints "Turn left".
```

Grammar of an enumeration case pattern

enum-case-pattern → type-identifier? `.` enum-case-name tuple-pattern?

옵셔널 패턴 (Optional Pattern)

*옵셔널 패턴(optional pattern)*은 `Optional<Wrapped>` 열거형의 `some(Wrapped)` 케이스에 래핑된 값과 일치합니다. 옵셔널 패턴은 식별자 패턴 바로 뒤에 물음표가 오는 것으로 구성되며 열거형 케이스 패턴과 동일한 위치에 나타납니다.

옵셔널 패턴은 `Optional` 열거형 케이스 패턴에 대한 편의 문법이므로 다음은 동일합니다:

```
let someOptional: Int? = 42
// Match using an enumeration case pattern.
if case .some(let x) = someOptional {
    print(x)
}

// Match using an optional pattern.
if case let x? = someOptional {
    print(x)
}
```

옵셔널 패턴은 `for-in` 구문에서 옵셔널 값의 배열을 반복하는 편리한 방법을 제공하여, `nil`이 아닌 요소에 대해서만 루프의 본문을 실행합니다.

```
let arrayOfOptionalInts: [Int?] = [nil, 2, 3, nil, 5]
// Match only non-nil values.
for case let number? in arrayOfOptionalInts {
    print("Found a \(number)")
}
// Found a 2
// Found a 3
// Found a 5
```

Grammar of an optional pattern

optional-pattern → identifier-pattern ?

타입-캐스팅 패턴 (Type-Casting Patterns)

타입-캐스팅 패턴(type-casting pattern)은 두 가지가 있으며, `is` 패턴과 `as` 패턴이 있습니다. `is` 패턴은 `switch` 구문 케이스 레이블에서만 나타납니다. `is`와 `as` 패턴은 다음의 형식을 가집니다:

```
is <#type#>
<#pattern#> as <#type#>
```

`is` 패턴은 런타임에 해당 값의 타입이 `is` 패턴의 오른편에 지정한 타입과 일치하거나 해당 타입의 하위 클래스가 일치하면 값과 일치합니다. `is` 패턴은 타입 캐스팅을 동작하지만 반환된 타입을 버린다는 것에서 `is` 연산자와 유사하게 동작합니다.

`as` 패턴은 런타임에 해당 값의 타입이 `as` 패턴의 오른편에 지정한 타입과 일치하거나 해당 타입의 하위 클래스가 일치하면 값과 일치합니다. 일치가 성공하면, 일치된 값은 `as` 패턴의 오른편에 지정한 *타입(type)*으로 캐스팅되고, 왼편의 *패턴(pattern)*에 일치시킵니다.

참고

번역자 주 영문 원본에서 캐스팅 대상을 오른편의 패턴이라고 설명한 부분을 정정했습니다. 문법은 패턴 `as` 타입입니다.

`is`와 `as` 패턴으로 값을 일치시키기 위해 `switch` 구문을 사용하는 예시는 Any와 AnyObject에 대한 타입 캐스팅 (Type Casting for Any and AnyObject)을 참고바랍니다.

Grammar of a type casting pattern

type-casting-pattern → is-pattern | as-pattern is-pattern → `is` type as-pattern → pattern `as` type

표현식 패턴 (Expression Pattern)

*표현식 패턴(expression pattern)*은 표현식의 값을 나타냅니다. 표현식 패턴은 `switch` 구문 케이스 레이블에서만 나타납니다.

표현식 패턴으로 표현되는 표현식은 Swift 표준 라이브러리의 패턴 일치 연산자(~=)를 사용하여 입력 표현식의 값과 비교합니다. 일치가 성공하면 ~= 연산자는 `true`를 반환합니다. 기본적으로 ~= 연산자는 == 연산자를 사용하여 동일한 타입의 두 값을 비교합니다. 다음 예시에서 보듯이 범위 내에 값이 포함되는지 검사하기 위해 값의 범위로 값을 일치시킬 수도 있습니다.

```
let point = (1, 2)
switch point {
case (0, 0):
    print("(0, 0) is at the origin.")
case (-2...2, -2...2):
    print("(\(point.0), \(point.1)) is near the origin.")
default:
    print("The point is at (\(point.0), \(point.1)).")
}
// Prints "(1, 2) is near the origin.".
```

~= 연산자를 오버로드하여 커스텀 표현식 일치 동작을 제공할 수 있습니다. 예를 들어 위의 예시에서 `point` 표현식을 문자열로 비교하기 위해 다시 작성할 수 있습니다.

```
// Overload the ~= operator to match a string with an integer.
func ~= (pattern: String, value: Int) -> Bool {
    return pattern == "\(value)"
}
switch point {
case ("0", "0"):
    print("(0, 0) is at the origin.")
default:
    print("The point is at (\(point.0), \(point.1)).")
}
// Prints "The point is at (1, 2).".
```

Grammar of an expression pattern

expression-pattern → expression

언어 참조

제네릭 매개변수와 인자 (Generic Parameters and Arguments)

선언을 일반화하여 구체적인 타입을 추상화합니다.

이 챕터에서 제네릭 타입, 함수, 이니셜라이저에 대한 매개변수와 인자를 설명합니다. 제네릭 타입, 함수, 서브스크립트, 이니셜라이저를 선언할 때 해당 제네릭 타입, 함수, 이니셜라이저가 동작할 수 있는 타입 매개변수를 지정합니다. 이러한 타입 매개변수는 제네릭 타입의 인스턴스가 생성되거나 제네릭 함수 또는 이니셜라이저가 호출될 때 실제 구체적인 타입 인자에 의해 대체되는 플레이스 홀더 역할을 합니다.

Swift의 제네릭에 대한 개요는 제네릭 (Generics)을 참고바랍니다.

제네릭 매개변수 절 (Generic Parameter Clause)

*제네릭 매개변수 절(generic parameter clause)*은 제네릭 타입이나 함수의 타입 매개변수와 해당 매개변수에 대한 관련 제약조건과 요구사항을 지정합니다. 제네릭 매개변수 절은 꺾쇠 괄호(<>)로 둘러싸여 있고 다음의 형식을 가집니다:

```
<<#generic parameter list#>>
```

위 형식의 *제네릭 매개변수 목록(generic parameter list)*은 콤마로 구분된 제네릭 매개변수의 목록이고 각 매개변수는 다음의 형식을 가집니다:

```
<#type parameter#>: <#constraint#>
```

제네릭 매개변수는 *타입 매개변수(type parameter)*와 선택 사항인 *제약조건(constraint)*으로 구성됩니다. 위 형식의 *타입 매개변수(type parameter)*는 단순히 플레이스 홀더 타입의 이름입니다 (예를 들어 `T`, `U`, `V`, `Key`, `Value` 등). 함수나 이니셜라이저의 시그니처를 포함하여 타입, 함수, 이니셜라이저 선언의 나머지 부분에서 타입 매개변수와 모든 연관 타입에 접근할 수 있습니다.

위 형식의 *제약조건(constraint)*은 타입 매개변수가 특정 클래스를 상속하거나 프로토콜 또는 프로토콜 합성을 준수해야 함을 지정합니다. 예를 들어 아래 제네릭 함수에서 제네릭 매개변수 `T: Comparable`은 타입 매개변수 `T`를 대신하는 모든 타입 인자는 `Comparable` 프로토콜을 준수해야 함을 나타냅니다.

```
func simpleMax<T: Comparable>(_ x: T, _ y: T) -> T {
    if x < y {
        return y
    }
    return x
}
```

예를 들어 `Int`와 `Double`은 `Comparable` 프로토콜을 준수하므로 이 함수는 두 타입의 인자를 허용합니다. 제네릭 타입과 다르게 제네릭 함수나 이니셜라이저를 사용할 때는 제네릭 인자 절을 지정하지 않습니다. 대신 타입 인자는 함수나 이니셜라이저에 전달되는 인자의 타입으로부터 추론됩니다.

```
simpleMax(17, 42) // T is inferred to be Int
simpleMax(3.14159, 2.71828) // T is inferred to be Double
```

제네릭 매개변수 목록의 마지막 제네릭 매개변수는 콤마를 포함할 수도 있습니다.

정수 제네릭 매개변수 (Integer Generic Parameters)

*정수 제네릭 매개변수(integer generic parameter)*는 타입이 아닌 정수값을 위한 플레이스홀더 역할을 합니다. 이것은 다음과 같은 형식을 가집니다:

```
let <#type parameter#>: <#type>
```

타입은 Swift 표준 라이브러리의 `Int` 타입이거나 `Int`로 해석되는 타입 별칭이나 제네릭 타입이어야 합니다.

정수 제네릭 매개변수로 제공하는 값은 정수 리터럴이거나 이를 감싸는 제네릭 컨텍스트에 있는 다른 정수 제네릭 매개변수여야 합니다. 예를 들어:

```
struct SomeStruct<let x: Int> { }
let a: SomeStruct<2>  // OK: integer literal

struct AnotherStruct<let x: Int, T, each U> {
    let b: SomeStruct<x>  // OK: another integer generic parameter

    static let c = 42
    let d: SomeStruct<c>  // Error: Can't use a constant.

    let e: SomeStruct<T>  // Error: Can't use a generic type parameter.
    let f: SomeStruct<U>  // Error: Can't use a parameter pack.
}
```

타입으로 선언된 정수 제네릭 매개변수의 값은 해당 타입과 동일한 접근 수준을 가지는 정적 상수 멤버로 접근할 수 있습니다. 함수에 선언된 정수 제네릭 매개변수의 값은 함수 내에서 상수로 접근할 수 있습니다. 표현식에서 사용될 때, 이 상수는 `Int` 타입을 가집니다.

```
print(a.x)  // Prints "4".
```

정수 제네릭 매개변수의 값은 타입을 초기화하거나 함수를 호출할 때 사용하는 인자의 타입으로 추론될 수 있습니다.

```
struct AnotherStruct<let y: Int> {
    var s: SomeStruct<y>
}
func someFunction<let z: Int>(s: SomeStruct<z>) {
    print(z)
}

let s1 = SomeStruct<12>()
let s2 = AnotherStruct(s: s1)  // AnotherStruct.y is inferred to be 12.
someFunction(s: s1)  // Prints "12".
```

제네릭 Where 절 (Generic Where Clauses)

타입 매개변수와 연관 타입에 요구사항을 추가로 지정하려면 타입이나 함수의 본문의 시작 중괄호 직전에 제네릭 `where` 절을 포함할 수 있습니다. 제네릭 `where` 절은 `where` 키워드로 구성되고 다음에 콤마로 구분된 하나 이상의 *요구사항(requirements)*의 목록으로 구성됩니다.

```
where <#requirements#>
```

제네릭 `where` 절에서 *요구사항(requirements)*은 타입 매개변수가 클래스를 상속하거나 프로토콜 또는 프로토콜 합성을 준수해야 함을 지정합니다. 제네릭 `where` 절은 타입 매개변수에 대한 간단 제약조건 (예를 들어 `<T: Comparable>`은 `<T> where T: Comparable`과 동일)을 표현하는 편의 문법을 제공하지만, 타입 매개변수와 연관 타입에 더 복잡한 제약조건을 제공하기 위해 사용할 수 있습니다. 예를 들어 타입 매개변수의 연관 타입을 프로토콜을 준수하도록 제한할 수 있습니다. 예를 들어 `<S: Sequence> where S.Iterator.Element: Equatable`은 `S`가 `Sequence` 프로토콜을 준수하고 연관 타입 `S.Iterator.Element`는 `Equatable` 프로토콜을 준수하도록 지정합니다. 이 제약조건은 시퀀스의 각 요소가 비교 가능함을 보장합니다. 정수 제네릭 매개변수는 프로토콜이나 상위 클래스의 요구사항을 가질 수 없습니다.

== 연산자를 사용하여 두 타입이 동일하다는 요구사항을 지정할 수도 있습니다. 예를 들어 `<S1: Sequence, S2: Sequence> where S1.Iterator.Element == S2.Iterator.Element`은 `S1`과 `S2`가 `Sequence` 프로토콜을 준수하고 두 시퀀스의 요소가 같은 타입이어야 한다는 제약조건을 나타냅니다. 정수 제네릭 매개변수의 경우 == 연산자는 해당 매개변수 값의 요구사항을 지정합니다. 두 정수 제네릭 매개변수가 동일한 값을 가져야 한다고 요구할 수도 있고, 정수 제네릭 매개변수가 특정 정수값을 가져야 한다고 요구할 수 있습니다.

타입 매개변수를 대체하는 모든 타입 인자는 타입 매개변수에 있는 모든 제약조건과 요구사항을 충족해야 합니다.

제네릭 `where` 절은 타입 매개변수를 포함하는 선언의 일부로 나타나거나 타입 매개변수를 포함하는 선언 내부에 중첩된 선언의 일부로 나타날 수 있습니다. 중첩된 선언에 대한 제네릭 `where` 절은 둘러싸는 선언의 타입 매개변수를 참조할 수 있습니다; 그러나 `where` 절의 요구사항은 작성된 선언에만 적용됩니다.

둘러싸는 선언도 `where` 절을 가지고 있으면, 두 절의 요구사항은 결합됩니다. 아래 예시에서 `startsWithZero()`는 `Element`가 `SomeProtocol`과 `Numeric` 모두 준수하는 경우에만 가능합니다.

```
extension Collection where Element: SomeProtocol {
    func startsWithZero() -> Bool where Element: Numeric {
        return first == .zero
    }
}
```

타입 매개변수에 다른 제약조건, 요구사항 또는 둘 다 제공하여 제네릭 함수나 이니셜라이저를 오버로드할 수 있습니다. 오버로드된 제네릭 함수나 이니셜라이저를 호출할 때 컴파일러는 이러한 제약조건을 사용하여 호출할 오버로드된 함수나 이니셜라이저를 결정합니다.

제네릭 `where` 절에 대한 자세한 정보와 제네릭 함수 선언의 예시를 보려면 제네릭 Where 절 (Generic Where Clauses)을 참고바랍니다.

Grammar of a generic parameter clause

generic-parameter-clause → < generic-parameter-list **,**? > generic-parameter-list → generic-parameter | generic-parameter **,** generic-parameter-list generic-parameter → type-name generic-parameter →

type-name : type-identifier generic-parameter → type-name : protocol-composition-type
generic-parameter → let type-name : type \

generic-where-clause → where requirement-list requirement-list → requirement | requirement ,
requirement-list requirement → conformance-requirement | same-type-requirement

conformance-requirement → type-identifier : type-identifier conformance-requirement → type-identifier :
protocol-composition-type same-type-requirement → type-identifier == type same-type-requirement →
type-identifier == signed-integer-literal

제네릭 인자 절 (Generic Argument Clause)

*제네릭 인자 절(generic argument clause)*은 제네릭 타입의 타입 인자를 지정합니다. 제네릭 인자 절은 꺾쇠 괄호(<>)로 둘러싸여져 있고 다음의 형식을 가집니다:

```
<<#generic argument list#>>
```

*제네릭 인자 목록(generic argument list)*는 타입 인자를 콤마로 구분한 목록입니다. *타입 인자(type argument)*는 제네릭 타입의 제네릭 매개변수 절에서 해당 타입 매개변수를 대체하는 실제 구체 타입의 이름을 의미합니다 - 또는 정수 제네릭 매개변수의 경우에는 해당 정수 제네릭 매개변수를 대체하는 정수값이 타입 인자가 됩니다. 그 결과는 해당 제네릭 타입의 특수 버전이 됩니다. 아래 예시는 Swift 표준 라이브러리의 제네릭 딕셔너리 타입의 단순화한 버전을 보여줍니다.

```
struct Dictionary<Key: Hashable, Value>: Collection, ExpressibleByDictionaryLiteral {
    /* ... */
}
```

제네릭 `Dictionary` 타입의 특수한 버전인 `Dictionary<String, Int>`는 제네릭 매개변수 `Key: Hashable`과 `Value`를 구체적인 타입 인자 `String`과 `Int`로 대체하여 형성됩니다. 각 타입 인자는 제네릭 `where` 절에 지정된 추가 요구사항을 포함하여 자신이 대체하는 제네릭 매개변수의 모든 제약조건을 충족해야 합니다. 위의 예시에서 `Key` 타입 매개변수는 `Hashable` 프로토콜을 준수하도록 제약되므로 `String`도 `Hashable` 프로토콜을 준수해야 합니다.

타입 매개변수 자체를 제네릭 타입의 특수한 버전인 타입 인자로 대체할 수도 있습니다(적절한 제약조건과 요구사항을 충족하는 경우). 예를 들어 `Array<Element>`의 타입 매개변수 `Element`를 배열의 특수한 버전인 `Array<Int>`로 대체하여 요소가 정수인 배열을 형성할 수 있습니다.

```
let arrayOfArrays: Array<Array<Int>> = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

제네릭 매개변수 목록의 마지막 타입 인자는 콤마를 포함할 수도 있습니다.

제네릭 매개변수 절 (Generic Parameter Clause)에서 언급했듯이 제네릭 함수나 이니셜라이저의 타입 인자로 지정하기 위해 제네릭 인자 절을 사용하지 않습니다.

Grammar of a generic argument clause

generic-argument-clause → < generic-argument-list ,? > generic-argument-list → generic-argument |
generic-argument , generic-argument-list generic-argument → type | signed-integer-literal

언어 참조

문법 요약 (Summary of the Grammar)

전체 공식 문법을 읽어보세요.

어휘 구조 (Lexical Structure)

Grammar of whitespace

whitespace → whitespace-item whitespace? whitespace-item → line-break whitespace-item → inline-space whitespace-item → comment whitespace-item → multiline-comment whitespace-item → U+0000, U+000B, or U+000C

line-break → U+000A line-break → U+000D line-break → U+000D followed by U+000A

inline-spaces → inline-space inline-spaces? inline-space → U+0009 or U+0020

comment → `//` comment-text line-break multiline-comment → `/*` multiline-comment-text `*/`

comment-text → comment-text-item comment-text? comment-text-item → Any Unicode scalar value except U+000A or U+000D

multiline-comment-text → multiline-comment-text-item multiline-comment-text? multiline-comment-text-item → multiline-comment multiline-comment-text-item → comment-text-item multiline-comment-text-item → Any Unicode scalar value except `/*` or `*/`

Grammar of an identifier

identifier → identifier-head identifier-characters? identifier → raw-identifier identifier → implicit-parameter-name identifier → property-wrapper-projection identifier-list → identifier | identifier `,` identifier-list

identifier-head → Upper- or lowercase letter A through Z identifier-head → `_` identifier-head → U+00A8, U+00AA, U+00AD, U+00AF, U+00B2-U+00B5, or U+00B7-U+00BA identifier-head → U+00BC-U+00BE, U+00C0-U+00D6, U+00D8-U+00F6, or U+00F8-U+00FF identifier-head → U+0100-U+02FF, U+0370-U+167F, U+1681-U+180D, or U+180F-U+1DBF identifier-head → U+1E00-U+1FFF identifier-head → U+200B-U+200D, U+202A-U+202E, U+203F-U+2040, U+2054, or U+2060-U+206F identifier-head → U+2070-U+20CF, U+2100-U+218F, U+2460-U+24FF, or U+2776-U+2793 identifier-head → U+2C00-U+2DFF or U+2E80-U+2FFF identifier-head → U+3004-U+3007, U+3021-U+302F, U+3031-U+303F, or U+3040-U+D7FF identifier-head → U+F900-U+FD3D, U+FD40-U+FDCF, U+FDF0-U+FE1F, or U+FE30-U+FE44 identifier-head → U+FE47-U+FFFD identifier-head → U+10000-U+1FFFD, U+20000-U+2FFFD, U+30000-U+3FFFD, or U+40000-U+4FFFD identifier-head → U+50000-U+5FFFD, U+60000-U+6FFFD, U+70000-U+7FFFD, or U+80000-U+8FFFD identifier-head → U+90000-U+9FFFD, U+A0000-U+AFFFD, U+B0000-U+BFFFD, or U+C0000-U+CFFFD identifier-head → U+D0000-U+DFFFD or U+E0000-U+EFFFD

identifier-character → decimal-digit identifier-character → U+0300-U+036F, U+1DC0-U+1DFF, U+20D0-U+20FF, or U+FE20-U+FE2F identifier-character → identifier-head identifier-characters → identifier-character identifier-characters?

implicit-parameter-name → `$` decimal-digits property-wrapper-projection → `$` identifier-characters

raw-identifier → ` raw-identifier-characters ` raw-identifier-character → Any Unicode scalar value except `, \, U+0000, U+000A, or U+000D raw-identifier-characters → raw-identifier-character raw-identifier-characters?

Grammar of a literal

literal → numeric-literal | string-literal | regular-expression-literal | boolean-literal | nil-literal

numeric-literal → signed-integer-literal | signed-floating-point-literal boolean-literal → `true` | `false` nil-literal → `nil`

Grammar of an integer literal

signed-integer-literal → –? integer-literal integer-literal → binary-literal integer-literal → octal-literal integer-literal → decimal-literal integer-literal → hexadecimal-literal

binary-literal → `0b` binary-digit binary-literal-characters? binary-digit → Digit 0 or 1 binary-literal-character → binary-digit | _ binary-literal-characters → binary-literal-character binary-literal-characters?

octal-literal → `0o` octal-digit octal-literal-characters? octal-digit → Digit 0 through 7 octal-literal-character → octal-digit | _ octal-literal-characters → octal-literal-character octal-literal-characters?

decimal-literal → decimal-digit decimal-literal-characters? decimal-digit → Digit 0 through 9 decimal-digits → decimal-digit decimal-digits? decimal-literal-character → decimal-digit | _ decimal-literal-characters → decimal-literal-character decimal-literal-characters?

hexadecimal-literal → `0x` hexadecimal-digit hexadecimal-literal-characters? hexadecimal-digit → Digit 0 through 9, a through f, or A through F hexadecimal-literal-character → hexadecimal-digit | _ hexadecimal-literal-characters → hexadecimal-literal-character hexadecimal-literal-characters?

Grammar of a floating-point literal

signed-floating-point-literal → –? floating-point-literal floating-point-literal → decimal-literal decimal-fraction? decimal-exponent? floating-point-literal → hexadecimal-literal hexadecimal-fraction? hexadecimal-exponent

decimal-fraction → `.` decimal-literal decimal-exponent → floating-point-e sign? decimal-literal

hexadecimal-fraction → `.` hexadecimal-digit hexadecimal-literal-characters? hexadecimal-exponent → floating-point-p sign? decimal-literal

floating-point-e → e | E floating-point-p → p | P sign → + | –

Grammar of a string literal

string-literal → static-string-literal | interpolated-string-literal

string-literal-opening-delimiter → extended-string-literal-delimiter? " string-literal-closing-delimiter → " extended-string-literal-delimiter?

static-string-literal → string-literal-opening-delimiter quoted-text? string-literal-closing-delimiter static-string-literal → multiline-string-literal-opening-delimiter multiline-quoted-text? multiline-string-literal-closing-delimiter

multiline-string-literal-opening-delimiter → extended-string-literal-delimiter? """
multiline-string-literal-closing-delimiter → """ extended-string-literal-delimiter?
extended-string-literal-delimiter → # extended-string-literal-delimiter?

quoted-text → quoted-text-item quoted-text? quoted-text-item → escaped-character quoted-text-item → Any Unicode scalar value except ", \, U+000A, or U+000D

multiline-quoted-text → multiline-quoted-text-item multiline-quoted-text? multiline-quoted-text-item → escaped-character multiline-quoted-text-item → Any Unicode scalar value except \
multiline-quoted-text-item → escaped-newline

interpolated-string-literal → string-literal-opening-delimiter interpolated-text? string-literal-closing-delimiter
interpolated-string-literal → multiline-string-literal-opening-delimiter multiline-interpolated-text? multiline-string-literal-closing-delimiter

interpolated-text → interpolated-text-item interpolated-text? interpolated-text-item → \(expression) | quoted-text-item

multiline-interpolated-text → multiline-interpolated-text-item multiline-interpolated-text?
multiline-interpolated-text-item → \(expression) | multiline-quoted-text-item

escape-sequence → \ extended-string-literal-delimiter escaped-character → escape-sequence 0 | escape-sequence \ | escape-sequence t | escape-sequence n | escape-sequence r | escape-sequence " | escape-sequence ' escaped-character → escape-sequence u { unicode-scalar-digits } unicode-scalar-digits → Between one and eight hexadecimal digits

escaped-newline → escape-sequence inline-spaces? line-break

Grammar of a regular expression literal

regular-expression-literal → regular-expression-literal-opening-delimiter regular-expression regular-expression-literal-closing-delimiter regular-expression → Any regular expression

regular-expression-literal-opening-delimiter → extended-regular-expression-literal-delimiter? /
regular-expression-literal-closing-delimiter → / extended-regular-expression-literal-delimiter?

extended-regular-expression-literal-delimiter → # extended-regular-expression-literal-delimiter?

Grammar of operators

operator → operator-head operator-characters? operator → dot-operator-head dot-operator-characters

operator-head → / | = | – | + | ! | * | % | < | > | & | | | ^ | ~ | ? operator-head → U+00A1-U+00A7 operator-head → U+00A9 or U+00AB operator-head → U+00AC or U+00AE operator-head → U+00B0-U+00B1 operator-head → U+00B6, U+00BB, U+00BF, U+00D7, or U+00F7 operator-head → U+2016-U+2017 operator-head → U+2020-U+2027 operator-head → U+2030-U+203E operator-head → U+2041-U+2053 operator-head → U+2055-U+205E operator-head → U+2190-U+23FF operator-head → U+2500-U+2775 operator-head → U+2794-U+2BFF operator-head → U+2E00-U+2E7F operator-head → U+3001-U+3003 operator-head → U+3008-U+3020 operator-head → U+3030

operator-character → operator-head operator-character → U+0300-U+036F operator-character → U+1DC0-U+1DFF operator-character → U+20D0-U+20FF operator-character → U+FE00-U+FE0F operator-character → U+FE20-U+FE2F operator-character → U+E0100-U+E01EF operator-characters → operator-character operator-characters?

dot-operator-head → . dot-operator-character → . | operator-character dot-operator-characters → dot-operator-character dot-operator-characters?

infix-operator → operator prefix-operator → operator postfix-operator → operator

타입 (Types)

Grammar of a type

type → function-type type → array-type type → dictionary-type type → type-identifier type → tuple-type type → optional-type type → implicitly-unwrapped-optional-type type → protocol-composition-type type → opaque-type type → boxed-protocol-type type → metatype-type type → any-type type → self-type type → (type)

Grammar of a type annotation

type-annotation → : attributes? type

Grammar of a type identifier

type-identifier → type-name generic-argument-clause? | type-name generic-argument-clause? . type-identifier type-name → identifier

Grammar of a tuple type

tuple-type → ( ) | (tuple-type-element , tuple-type-element-list) tuple-type-element-list → tuple-type-element | tuple-type-element , tuple-type-element-list tuple-type-element → element-name type-annotation | type element-name → identifier

Grammar of a function type

function-type → attributes? function-type-argument-clause `async`? throws-clause? –> type

function-type-argument-clause → ( ) function-type-argument-clause → (function-type-argument-list ... ?)

function-type-argument-list → function-type-argument | function-type-argument , function-type-argument-list function-type-argument → attributes? parameter-modifier? type | argument-label type-annotation argument-label → identifier

throws-clause → `throws` | `throws` (type)

Grammar of an array type

array-type → [type]

Grammar of a dictionary type

dictionary-type → [type : type]

Grammar of an optional type

optional-type → type `?`

Grammar of an implicitly unwrapped optional type

implicitly-unwrapped-optional-type → type `!`

Grammar of a protocol composition type

protocol-composition-type → type-identifier & protocol-composition-continuation
protocol-composition-continuation → type-identifier | protocol-composition-type

Grammar of an opaque type

opaque-type → `some` type

Grammar of a boxed protocol type

boxed-protocol-type → `any` type

Grammar of a metatype type

metatype-type → type `.` `Type` | type `.` `Protocol`

Grammar of an Any type

any-type → `Any`

Grammar of a Self type

self-type → `Self`

Grammar of a type inheritance clause

type-inheritance-clause → `:` type-inheritance-list type-inheritance-list → attributes? ~? type-identifier | attributes? ~? type-identifier `,` type-inheritance-list

표현식 (Expressions)

Grammar of an expression

expression → try-operator? await-operator? prefix-expression infix-expressions?

Grammar of a prefix expression

prefix-expression → prefix-operator? postfix-expression prefix-expression → in-out-expression

Grammar of an in-out expression

in-out-expression → & primary-expression

Grammar of a try expression

try-operator → `try` | `try ?` | `try !`

Grammar of an await expression

await-operator → `await`

Grammar of an infix expression

infix-expression → infix-operator prefix-expression infix-expression → assignment-operator try-operator? await-operator? prefix-expression infix-expression → conditional-operator try-operator? await-operator? prefix-expression infix-expression → type-casting-operator infix-expressions → infix-expression infix-expressions?

Grammar of an assignment operator

assignment-operator → =

Grammar of a conditional operator

conditional-operator → ? expression `:`

Grammar of a type-casting operator

type-casting-operator → `is` type type-casting-operator → `as` type type-casting-operator → `as ?` type type-casting-operator → `as !` type

Grammar of a primary expression

primary-expression → identifier generic-argument-clause? primary-expression → literal-expression primary-expression → self-expression primary-expression → superclass-expression primary-expression → conditional-expression primary-expression → closure-expression primary-expression → parenthesized-expression primary-expression → tuple-expression primary-expression → implicit-member-expression primary-expression → wildcard-expression primary-expression → macro-expansion-expression primary-expression → key-path-expression primary-expression → selector-expression primary-expression → key-path-string-expression

Grammar of a literal expression

literal-expression → literal literal-expression → array-literal | dictionary-literal | playground-literal

array-literal → `[` array-literal-items? `,`? `]` array-literal-items → array-literal-item | array-literal-item `,` array-literal-items array-literal-item → expression

dictionary-literal → `[` dictionary-literal-items `,`? `]` | `[` `:` `]` dictionary-literal-items → dictionary-literal-item | dictionary-literal-item `,` dictionary-literal-items dictionary-literal-item → expression `:` expression

playground-literal → `#colorLiteral` `(` `red` `:` expression `,` `green` `:` expression `,` `blue` `:` expression `,` `alpha` `:` expression `)` playground-literal → `#fileLiteral` `(` `resourceName` `:` expression `)` playground-literal → `#imageLiteral` `(` `resourceName` `:` expression `)`

Grammar of a self expression

self-expression → `self` | self-method-expression | self-subscript-expression | self-initializer-expression

self-method-expression → `self` `.` identifier self-subscript-expression → `self` `[` function-call-argument-list `]` self-initializer-expression → `self` `.` `init`

Grammar of a superclass expression

superclass-expression → superclass-method-expression | superclass-subscript-expression | superclass-initializer-expression

superclass-method-expression → `super` `.` identifier superclass-subscript-expression → `super` `[` function-call-argument-list `]` superclass-initializer-expression → `super` `.` `init`

Grammar of a conditional expression

conditional-expression → if-expression | switch-expression

if-expression → `if` condition-list `{` statement `}` if-expression-tail if-expression-tail → `else` if-expression if-expression-tail → `else` `{` statement `}`

switch-expression → `switch` expression `{` switch-expression-cases `}` switch-expression-cases → switch-expression-case switch-expression-cases? switch-expression-case → case-label statement switch-expression-case → default-label statement

Grammar of a closure expression

closure-expression → `{` attributes? closure-signature? statements? `}`

closure-signature → capture-list? closure-parameter-clause `async`? throws-clause? function-result? `in` closure-signature → capture-list `in`

closure-parameter-clause → `(` `)` | `(` closure-parameter-list `,`? `)` | identifier-list closure-parameter-list → closure-parameter | closure-parameter `,` closure-parameter-list closure-parameter → closure-parameter-name type-annotation? closure-parameter → closure-parameter-name type-annotation `.` `..` closure-parameter-name → identifier

capture-list → `[` capture-list-items `,`? `]` capture-list-items → capture-list-item | capture-list-item `,` capture-list-items capture-list-item → capture-specifier? identifier capture-list-item → capture-specifier? identifier = expression capture-list-item → capture-specifier? self-expression capture-specifier → `weak` | `unowned` | `unowned(safe)` | `unowned(unsafe)`

Grammar of an implicit member expression

implicit-member-expression → `.` identifier implicit-member-expression → `.` identifier `.` postfix-expression

Grammar of a parenthesized expression

parenthesized-expression → (expression)

Grammar of a tuple expression

tuple-expression → ( ) | (tuple-element , tuple-element-list ,?) tuple-element-list → tuple-element | tuple-element , tuple-element-list tuple-element → expression | identifier : expression

Grammar of a wildcard expression

wildcard-expression → _

Grammar of a macro-expansion expression

macro-expansion-expression → # identifier generic-argument-clause? function-call-argument-clause? trailing-closures?

Grammar of a key-path expression

key-path-expression → \ type? . key-path-components key-path-components → key-path-component | key-path-component . key-path-components key-path-component → identifier key-path-postfixes? | key-path-postfixes

key-path-postfixes → key-path-postfix key-path-postfixes? key-path-postfix → ? | ! | `self` | [function-call-argument-list]

Grammar of a selector expression

selector-expression → `#selector` (expression) selector-expression → `#selector` (`getter:` expression) selector-expression → `#selector` (`setter:` expression)

Grammar of a key-path string expression

key-path-string-expression → `#keyPath` (expression)

Grammar of a postfix expression

postfix-expression → primary-expression postfix-expression → postfix-expression postfix-operator postfix-expression → function-call-expression postfix-expression → initializer-expression postfix-expression → explicit-member-expression postfix-expression → postfix-self-expression postfix-expression → subscript-expression postfix-expression → forced-value-expression postfix-expression → optional-chaining-expression

Grammar of a function call expression

function-call-expression → postfix-expression function-call-argument-clause function-call-expression → postfix-expression function-call-argument-clause? trailing-closures

function-call-argument-clause → ( ) | (function-call-argument-list ,?) function-call-argument-list → function-call-argument | function-call-argument , function-call-argument-list function-call-argument → expression | identifier : expression function-call-argument → operator | identifier : operator

trailing-closures → closure-expression labeled-trailing-closures? labeled-trailing-closures → labeled-trailing-closure labeled-trailing-closures? labeled-trailing-closure → identifier : closure-expression

Grammar of an initializer expression

initializer-expression → postfix-expression . `init` initializer-expression → postfix-expression . `init` (argument-names)

Grammar of an explicit member expression

explicit-member-expression → postfix-expression . decimal-digits explicit-member-expression → postfix-expression . identifier generic-argument-clause? explicit-member-expression → postfix-expression . identifier (argument-names) explicit-member-expression → postfix-expression conditional-compilation-block

argument-names → argument-name argument-names? argument-name → identifier :

Grammar of a postfix self expression

postfix-self-expression → postfix-expression . `self`

Grammar of a subscript expression

subscript-expression → postfix-expression [function-call-argument-list]

Grammar of a forced-value expression

forced-value-expression → postfix-expression !

Grammar of an optional-chaining expression

optional-chaining-expression → postfix-expression ?

구문 (Statements)

Grammar of a statement

statement → expression ;? statement → declaration ;? statement → loop-statement ;? statement → branch-statement ;? statement → labeled-statement ;? statement → control-transfer-statement ;? statement → defer-statement ;? statement → do-statement ;? statement → compiler-control-statement statements → statement statements?

Grammar of a loop statement

loop-statement → for-in-statement loop-statement → while-statement loop-statement → repeat-while-statement

Grammar of a for-in statement

for-in-statement → `for` `case`? pattern `in` expression where-clause? code-block

Grammar of a while statement

while-statement → `while` condition-list code-block

condition-list → condition | condition `,` condition-list condition → expression | availability-condition | case-condition | optional-binding-condition

case-condition → `case` pattern initializer optional-binding-condition → `let` pattern initializer? | `var` pattern initializer?

Grammar of a repeat-while statement

repeat-while-statement → `repeat` code-block `while` expression

Grammar of a branch statement

branch-statement → if-statement branch-statement → guard-statement branch-statement → switch-statement

Grammar of an if statement

if-statement → `if` condition-list code-block else-clause? else-clause → `else` code-block | `else` if-statement

Grammar of a guard statement

guard-statement → `guard` condition-list `else` code-block

Grammar of a switch statement

switch-statement → `switch` expression `{` switch-cases? `}` switch-cases → switch-case switch-cases? switch-case → case-label statements switch-case → default-label statements switch-case → conditional-switch-case

case-label → attributes? `case` case-item-list `:` case-item-list → pattern where-clause? | pattern where-clause? `,` case-item-list default-label → attributes? `default` `:`

where-clause → `where` where-expression where-expression → expression

conditional-switch-case → switch-if-directive-clause switch-elseif-directive-clauses? switch-else-directive-clause? endif-directive switch-if-directive-clause → if-directive compilation-condition switch-cases? switch-elseif-directive-clauses → elseif-directive-clause switch-elseif-directive-clauses? switch-elseif-directive-clause → elseif-directive compilation-condition switch-cases? switch-else-directive-clause → else-directive switch-cases?

Grammar of a labeled statement

labeled-statement → statement-label loop-statement labeled-statement → statement-label if-statement
labeled-statement → statement-label switch-statement labeled-statement → statement-label do-statement

statement-label → label-name `:` label-name → identifier

Grammar of a control transfer statement

control-transfer-statement → break-statement control-transfer-statement → continue-statement
control-transfer-statement → fallthrough-statement control-transfer-statement → return-statement
control-transfer-statement → throw-statement

Grammar of a break statement

break-statement → `break` label-name?

Grammar of a continue statement

continue-statement → `continue` label-name?

Grammar of a fallthrough statement

fallthrough-statement → `fallthrough`

Grammar of a return statement

return-statement → `return` expression?

Grammar of a throw statement

throw-statement → `throw` expression

Grammar of a defer statement

defer-statement → `defer` code-block

Grammar of a do statement

do-statement → `do` throws-clause? code-block catch-clauses? catch-clauses → catch-clause catch-clauses?
catch-clause → `catch` catch-pattern-list? code-block catch-pattern-list → catch-pattern | catch-pattern `,`
catch-pattern-list catch-pattern → pattern where-clause?

Grammar of a compiler control statement

compiler-control-statement → conditional-compilation-block compiler-control-statement →
line-control-statement compiler-control-statement → diagnostic-statement

Grammar of a conditional compilation block

conditional-compilation-block → if-directive-clause elseif-directive-clauses? else-directive-clause? endif-directive

if-directive-clause → if-directive compilation-condition statements? elseif-directive-clauses → elseif-directive-clause elseif-directive-clauses? elseif-directive-clause → elseif-directive compilation-condition statements? else-directive-clause → else-directive statements? if-directive → `#if` elseif-directive → `#elseif` else-directive → `#else` endif-directive → `#endif`

compilation-condition → platform-condition compilation-condition → identifier compilation-condition → boolean-literal compilation-condition → `(` compilation-condition `)` compilation-condition → `!` compilation-condition compilation-condition → compilation-condition `&&` compilation-condition compilation-condition → compilation-condition `||` compilation-condition

platform-condition → `os` `(` operating-system `)` platform-condition → `arch` `(` architecture `)` platform-condition → `swift` `(` `>=` swift-version `)` | `swift` `(` `<` swift-version `)` platform-condition → `compiler` `(` `>=` swift-version `)` | `compiler` `(` `<` swift-version `)` platform-condition → `canImport` `(` import-path `)` platform-condition → `targetEnvironment` `(` environment `)`

operating-system → `macOS` | `iOS` | `watchOS` | `tvOS` | `visionOS` | `Linux` | `Windows` architecture → `arm` | `arm64` | `i386` | `wasm32` | `x86_64` swift-version → decimal-digits swift-version-continuation? swift-version-continuation → `.` decimal-digits swift-version-continuation? environment → `simulator` | `macCatalyst`

Grammar of a line control statement

line-control-statement → `#sourceLocation` `(` `file:` file-path `,` `line:` line-number `)` line-control-statement → `#sourceLocation` `(` `)` line-number → A decimal integer greater than zero file-path → static-string-literal

Grammar of an availability condition

availability-condition → `#available` `(` availability-arguments `)` availability-condition → `#unavailable` `(` availability-arguments `)` availability-arguments → availability-argument | availability-argument `,` availability-arguments availability-argument → platform-name platform-version availability-argument → `*`

platform-name → `iOS` | `iOSApplicationExtension` platform-name → `macOS` | `macOSApplicationExtension` platform-name → `macCatalyst` | `macCatalystApplicationExtension` platform-name → `watchOS` | `watchOSApplicationExtension` platform-name → `tvOS` | `tvOSApplicationExtension` platform-name → `visionOS` | `visionOSApplicationExtension` platform-version → decimal-digits platform-version → decimal-digits `.` decimal-digits platform-version → decimal-digits `.` decimal-digits `.` decimal-digits

선언 (Declarations)

Grammar of a declaration

declaration → import-declaration declaration → constant-declaration declaration → variable-declaration declaration → typealias-declaration declaration → function-declaration declaration → enum-declaration declaration → struct-declaration declaration → class-declaration declaration → actor-declaration declaration → protocol-declaration declaration → initializer-declaration declaration →

deinitializer-declaration declaration → extension-declaration declaration → subscript-declaration declaration → macro-declaration declaration → operator-declaration declaration → precedence-group-declaration

Grammar of a top-level declaration

top-level-declaration → statements?

Grammar of a code block

code-block → { statements? }

Grammar of an import declaration

import-declaration → attributes? `import` import-kind? import-path

import-kind → `typealias` | `struct` | `class` | `enum` | `protocol` | `let` | `var` | `func` import-path → identifier | identifier `.` import-path

Grammar of a constant declaration

constant-declaration → attributes? declaration-modifiers? `let` pattern-initializer-list

pattern-initializer-list → pattern-initializer | pattern-initializer `,` pattern-initializer-list pattern-initializer → pattern initializer? initializer → = expression

Grammar of a variable declaration

variable-declaration → variable-declaration-head pattern-initializer-list variable-declaration → variable-declaration-head variable-name type-annotation code-block variable-declaration → variable-declaration-head variable-name type-annotation getter-setter-block variable-declaration → variable-declaration-head variable-name type-annotation getter-setter-keyword-block variable-declaration → variable-declaration-head variable-name initializer willSet-didSet-block variable-declaration → variable-declaration-head variable-name type-annotation initializer? willSet-didSet-block

variable-declaration-head → attributes? declaration-modifiers? `var` variable-name → identifier

getter-setter-block → code-block getter-setter-block → { getter-clause setter-clause? } getter-setter-block → { setter-clause getter-clause } getter-clause → attributes? mutation-modifier? `get` code-block setter-clause → attributes? mutation-modifier? `set` setter-name? code-block setter-name → (identifier)

getter-setter-keyword-block → { getter-keyword-clause setter-keyword-clause? } getter-setter-keyword-block → { setter-keyword-clause getter-keyword-clause } getter-keyword-clause → attributes? mutation-modifier? `get` setter-keyword-clause → attributes? mutation-modifier? `set`

willSet-didSet-block → { willSet-clause didSet-clause? } willSet-didSet-block → { didSet-clause willSet-clause? } willSet-clause → attributes? `willSet` setter-name? code-block didSet-clause → attributes? `didSet` setter-name? code-block

Grammar of a type alias declaration

typealias-declaration → attributes? access-level-modifier? `typealias` typealias-name generic-parameter-clause? typealias-assignment typealias-name → identifier typealias-assignment → = type

Grammar of a function declaration

function-declaration → function-head function-name generic-parameter-clause? function-signature generic-where-clause? function-body?

function-head → attributes? declaration-modifiers? `func` function-name → identifier | operator

function-signature → parameter-clause `async`? throws-clause? function-result? function-signature → parameter-clause `async`? `rethrows` function-result? function-result → `->` attributes? type function-body → code-block

parameter-clause → `(` `)` | `(` parameter-list `,`? `)` parameter-list → parameter | parameter `,` parameter-list parameter → external-parameter-name? local-parameter-name parameter-type-annotation default-argument-clause? parameter → external-parameter-name? local-parameter-name parameter-type-annotation parameter → external-parameter-name? local-parameter-name parameter-type-annotation `...`

external-parameter-name → identifier local-parameter-name → identifier parameter-type-annotation → `:` attributes? parameter-modifier? type parameter-modifier → `inout` | `borrowing` | `consuming` default-argument-clause → = expression

Grammar of an enumeration declaration

enum-declaration → attributes? access-level-modifier? union-style-enum enum-declaration → attributes? access-level-modifier? raw-value-style-enum

union-style-enum → `indirect`? `enum` enum-name generic-parameter-clause? type-inheritance-clause? generic-where-clause? `{` union-style-enum-members? `}` union-style-enum-members → union-style-enum-member union-style-enum-members? union-style-enum-member → declaration | union-style-enum-case-clause | compiler-control-statement union-style-enum-case-clause → attributes? `indirect`? `case` union-style-enum-case-list union-style-enum-case-list → union-style-enum-case | union-style-enum-case `,` union-style-enum-case-list union-style-enum-case → enum-case-name tuple-type? enum-name → identifier enum-case-name → identifier

raw-value-style-enum → `enum` enum-name generic-parameter-clause? type-inheritance-clause generic-where-clause? `{` raw-value-style-enum-members `}` raw-value-style-enum-members → raw-value-style-enum-member raw-value-style-enum-members? raw-value-style-enum-member → declaration | raw-value-style-enum-case-clause | compiler-control-statement raw-value-style-enum-case-clause → attributes? `case` raw-value-style-enum-case-list raw-value-style-enum-case-list → raw-value-style-enum-case | raw-value-style-enum-case `,` raw-value-style-enum-case-list raw-value-style-enum-case → enum-case-name raw-value-assignment? raw-value-assignment → = raw-value-literal raw-value-literal → numeric-literal | static-string-literal | boolean-literal

Grammar of a structure declaration

struct-declaration → attributes? access-level-modifier? `struct` struct-name generic-parameter-clause? type-inheritance-clause? generic-where-clause? struct-body struct-name → identifier struct-body → `{` struct-members? `}`

struct-members → struct-member struct-members? struct-member → declaration | compiler-control-statement

Grammar of a class declaration

class-declaration → attributes? access-level-modifier? `final`? `class` class-name generic-parameter-clause? type-inheritance-clause? generic-where-clause? class-body class-declaration → attributes? `final` access-level-modifier? `class` class-name generic-parameter-clause? type-inheritance-clause? generic-where-clause? class-body class-name → identifier class-body → { class-members? }

class-members → class-member class-members? class-member → declaration | compiler-control-statement

Grammar of an actor declaration

actor-declaration → attributes? access-level-modifier? `actor` actor-name generic-parameter-clause? type-inheritance-clause? generic-where-clause? actor-body actor-name → identifier actor-body → { actor-members? }

actor-members → actor-member actor-members? actor-member → declaration | compiler-control-statement

Grammar of a protocol declaration

protocol-declaration → attributes? access-level-modifier? `protocol` protocol-name type-inheritance-clause? generic-where-clause? protocol-body protocol-name → identifier protocol-body → { protocol-members? }

protocol-members → protocol-member protocol-members? protocol-member → protocol-member-declaration | compiler-control-statement

protocol-member-declaration → protocol-property-declaration protocol-member-declaration → protocol-method-declaration protocol-member-declaration → protocol-initializer-declaration protocol-member-declaration → protocol-subscript-declaration protocol-member-declaration → protocol-associated-type-declaration protocol-member-declaration → typealias-declaration

Grammar of a protocol property declaration

protocol-property-declaration → variable-declaration-head variable-name type-annotation getter-setter-keyword-block

Grammar of a protocol method declaration

protocol-method-declaration → function-head function-name generic-parameter-clause? function-signature generic-where-clause?

Grammar of a protocol initializer declaration

protocol-initializer-declaration → initializer-head generic-parameter-clause? parameter-clause throws-clause? generic-where-clause? protocol-initializer-declaration → initializer-head generic-parameter-clause? parameter-clause `rethrows` generic-where-clause?

Grammar of a protocol subscript declaration

protocol-subscript-declaration → subscript-head subscript-result generic-where-clause? getter-setter-keyword-block

Grammar of a protocol associated type declaration

protocol-associated-type-declaration → attributes? access-level-modifier? `associatedtype` typealias-name type-inheritance-clause? typealias-assignment? generic-where-clause?

Grammar of an initializer declaration

initializer-declaration → initializer-head generic-parameter-clause? parameter-clause `async`? throws-clause? generic-where-clause? initializer-body initializer-declaration → initializer-head generic-parameter-clause? parameter-clause `async`? `rethrows` generic-where-clause? initializer-body initializer-head → attributes? declaration-modifiers? `init` initializer-head → attributes? declaration-modifiers? `init ?` initializer-head → attributes? declaration-modifiers? `init !` initializer-body → code-block

Grammar of a deinitializer declaration

deinitializer-declaration → attributes? `deinit` code-block

Grammar of an extension declaration

extension-declaration → attributes? access-level-modifier? `extension` type-identifier type-inheritance-clause? generic-where-clause? extension-body extension-body → `{` extension-members? `}`

extension-members → extension-member extension-members? extension-member → declaration | compiler-control-statement

Grammar of a subscript declaration

subscript-declaration → subscript-head subscript-result generic-where-clause? code-block
subscript-declaration → subscript-head subscript-result generic-where-clause? getter-setter-block
subscript-declaration → subscript-head subscript-result generic-where-clause? getter-setter-keyword-block
subscript-head → attributes? declaration-modifiers? `subscript` generic-parameter-clause? parameter-clause subscript-result → –> attributes? type

Grammar of a macro declaration

macro-declaration → macro-head identifier generic-parameter-clause? macro-signature macro-definition? generic-where-clause macro-head → attributes? declaration-modifiers? `macro` macro-signature → parameter-clause macro-function-signature-result? macro-function-signature-result → –> type
macro-definition → = expression

Grammar of an operator declaration

operator-declaration → prefix-operator-declaration | postfix-operator-declaration | infix-operator-declaration

prefix-operator-declaration → `prefix operator` operator postfix-operator-declaration → `postfix operator` operator infix-operator-declaration → `infix operator` operator infix-operator-group?

infix-operator-group → `:` precedence-group-name

Grammar of a precedence group declaration

precedence-group-declaration → `precedencegroup` precedence-group-name `{` precedence-group-attributes? `}`

precedence-group-attributes → precedence-group-attribute precedence-group-attributes?
precedence-group-attribute → precedence-group-relation precedence-group-attribute → precedence-group-assignment precedence-group-attribute → precedence-group-associativity

precedence-group-relation → `higherThan` `:` precedence-group-names precedence-group-relation → `lowerThan` `:` precedence-group-names

precedence-group-assignment → `assignment` `:` boolean-literal

precedence-group-associativity → `associativity` `:` `left` precedence-group-associativity → `associativity` `:` `right` precedence-group-associativity → `associativity` `:` `none`

precedence-group-names → precedence-group-name | precedence-group-name `,` precedence-group-names precedence-group-name → identifier

Grammar of a declaration modifier

declaration-modifier → `class` | `convenience` | `dynamic` | `final` | `infix` | `lazy` | `optional` | `override` | `postfix` | `prefix` | `required` | `static` | `unowned` | `unowned` `(` `safe` `)` | `unowned` `(` `unsafe` `)` | `weak`
declaration-modifier → access-level-modifier declaration-modifier → mutation-modifier
declaration-modifier → actor-isolation-modifier declaration-modifiers → declaration-modifier declaration-modifiers?

access-level-modifier → `private` | `private` `(` `set` `)` access-level-modifier → `fileprivate` | `fileprivate` `(` `set` `)` access-level-modifier → `internal` | `internal` `(` `set` `)` access-level-modifier → `package` | `package` `(` `set` `)` access-level-modifier → `public` | `public` `(` `set` `)` access-level-modifier → `open` | `open` `(` `set` `)`

mutation-modifier → `mutating` | `nonmutating`

actor-isolation-modifier → `nonisolated`

속성 (Attributes)

Grammar of an attribute

attribute → `@` attribute-name attribute-argument-clause? attribute-name → identifier
attribute-argument-clause → `(` balanced-tokens? `)` attributes → attribute attributes?

balanced-tokens → balanced-token balanced-tokens? balanced-token → `(` balanced-tokens? `)`
balanced-token → `[` balanced-tokens? `]` balanced-token → `{` balanced-tokens? `}` balanced-token → Any identifier, keyword, literal, or operator balanced-token → Any punctuation except `(`, `)`, `[`, `]`, `{`, or `}`

패턴 (Patterns)

Grammar of a pattern

pattern → wildcard-pattern type-annotation? pattern → identifier-pattern type-annotation? pattern → value-binding-pattern pattern → tuple-pattern type-annotation? pattern → enum-case-pattern pattern → optional-pattern pattern → type-casting-pattern pattern → expression-pattern

Grammar of a wildcard pattern

wildcard-pattern → _

Grammar of an identifier pattern

identifier-pattern → identifier

Grammar of a value-binding pattern

value-binding-pattern → `var` pattern | `let` pattern

Grammar of a tuple pattern

tuple-pattern → (tuple-pattern-element-list?) tuple-pattern-element-list → tuple-pattern-element | tuple-pattern-element , tuple-pattern-element-list tuple-pattern-element → pattern | identifier : pattern

Grammar of an enumeration case pattern

enum-case-pattern → type-identifier? . enum-case-name tuple-pattern?

Grammar of an optional pattern

optional-pattern → identifier-pattern ?

Grammar of a type casting pattern

type-casting-pattern → is-pattern | as-pattern is-pattern → `is` type as-pattern → pattern `as` type

Grammar of an expression pattern

expression-pattern → expression

제네릭 매개변수와 인자 (Generic Parameters and Arguments)

Grammar of a generic parameter clause

generic-parameter-clause → < generic-parameter-list ,? > generic-parameter-list → generic-parameter | generic-parameter , generic-parameter-list generic-parameter → type-name generic-parameter → type-name : type-identifier generic-parameter → type-name : protocol-composition-type generic-parameter → `let` type-name : type \

generic-where-clause → `where` requirement-list requirement-list → requirement | requirement `,` requirement-list requirement → conformance-requirement | same-type-requirement

conformance-requirement → type-identifier `:` type-identifier conformance-requirement → type-identifier `:` protocol-composition-type same-type-requirement → type-identifier == type same-type-requirement → type-identifier == signed-integer-literal

Grammar of a generic argument clause

generic-argument-clause → < generic-argument-list `,`? > generic-argument-list → generic-argument | generic-argument `,` generic-argument-list generic-argument → type | signed-integer-literal

개정 이력

Document Revision History

최근 변경사항을 확인할 수 있습니다.

Revision History

2026-10-13(화)

- Swift 6.4 기반으로 문서 업데이트.
- 클래스 전용 프로토콜 (Class-Only Protocols)와 프로토콜 선언 (Protocol Declaration)에 상위 클래스 제약 설명 추가.
- 식별자 (Identifiers)에 원시 식별자 설명 추가.
- attached에 본문 매크로 역할 설명 추가.
- 태스크 취소 예제, 옵셔널 바인딩 타입 설명, 메모리 접근 충돌 조건과 인스턴스 해제 조건 수정.
- 번역, API 이름, 문법 요약, 문서 링크와 오탈자 수정.
- Swift 6.4 번역자 보충 (Swift 6.4 Translation Notes)에 최신 언어 기능과 영문 원본의 기술적 정정을 별도로 정리.

2026-04-24(금)

- Swift 6.3 기반으로 문서 업데이트.

2026-01-02(금)

- Swift 6.2.3 기반으로 문서 업데이트.
- 클라이언트 모듈로 정의를 내보내는 것에 대한 내용에 대한 export 섹션 추가.

2026-01-02(금)

- Swift 6.2.1 기반으로 문서 업데이트.
- 숫자 값을 가지는 제네릭 매개변수에 대한 정보를 담은 정수 제네릭 매개변수 (Integer Generic Parameters) 섹션 추가.
- 정수와 부동소수점 숫자 사이의 결정에 대한 내용에 대한 정수 (Integers)와 부동소수점 숫자 (Floating-Point Numbers) 섹션 추가.

2025-09-19(금)

- Swift 6.2 기반으로 문서 업데이트.
- 여러 오탈자와 오역 수정.
- Swift가 방지해주는 이슈에 대한 내용을 메모리 안전성 (Memory Safety)에 추가.

- `if case` 문법에 대한 내용을 패턴 (Patterns)에 추가.
- 메인 액터, 격리, 글로벌 액터에 대한 내용을 동시성 (Concurrency)에 추가.
- 명시적으로 준수성을 작성하지 않고 일반 프로토콜 준수하는 것과 암시적 준수 억제에 대한 내용을 프로토콜 암시적 준수 (Implicit Conformance to a Protocol)에 추가.
- 일반 프로토콜의 준수성을 요구하는 제너릭 제약 조건에 대한 내용을 암시적 제약조건 (Implicit Constraints)에 추가.

2025-04-02(수)

- Swift 6.1 기반으로 문서 업데이트.
- 불투명 매개변수 타입 (Opaque Parameter Types) 섹션을 추가하여 `some`을 제너릭의 간단한 문법으로 사용하는 방법에 대한 정보를 포함.
- available 섹션에 `noasync` 인자에 대한 정보를 추가.

2024-09-19(목)

- Swift 6 기반으로 문서 업데이트.
- 오탈자 수정.

2024-07-02(화)

- Swift 6 Beta 기반으로 문서 업데이트.
- 엄격한 동시성 검사로 변환을 위한 정보를 preconcurrency 섹션에 추가.
- 특정 타입의 오류 발생에 대한 내용을 오류 타입 지정 (Specifying the Error Type) 섹션에 추가.
- 접근 제어 (Access Control) 챕터에 패키지-수준 접근에 대한 내용 추가.

2024-03-08(금)

- Swift 5.10 기반으로 문서 작업
- 오탈자 수정

2024-02-25(일)

- Swift 5.10 Beta 기반으로 문서 작업
- 중첩된 프로토콜에 대한 내용을 위임 (Delegation) 섹션에 추가.
- UIApplicationMain과 NSApplicationMain 섹션에 더이상 사용하지 않는 정보 추가.

2023-12-12(화)

- Swift 5.9.2 기반으로 문서 작업
- 태스크(task), 태스크 그룹(task group), 태스크 취소(task cancellation)에 대한 내용을 동시성 (Concurrency)에 추가했습니다.
- 기존 Swift Package에 매크로 구현에 대한 내용을 매크로 (Macros)에 추가했습니다.

- Conformance 매크로를 대신해 Extension 매크로에 대한 내용을 attached에 업데이트 했습니다.

2023-12-03(일)

- Swift 5.9.2 Beta 기반으로 문서 작업
- `borrowing`과 `consuming` 수정자에 대한 정보를 매개변수 수정자 (Parameter Modifiers) 섹션에 추가했습니다.
- 상수와 변수 선언 (Declaring Constants and Variables) 에 상수를 선언한 후에 값을 설정에 대한 정보를 추가했습니다.
- 백 배포(back deployment)에 대한 정보를 backDeployed 섹션에 추가했습니다.

2023-09-30(토)

- Swift 5.9 기반으로 문서 작업
- 기본 (The Basics)에 옵셔널에 대한 내용을 추가
- Swift 둘러보기 (A Swift Tour)에 동시성 예시 추가
- 결과 변환 (Result Transformations) 섹션에 `buildPartialBlock(first:)`와 `buildPartialBlock(accumulated:next:)` 메서드에 대한 내용 추가
- available과 조건부 컴파일 블록 (Conditional Compilation Block)에서 플랫폼 목록에 visionOS 추가

2023-06-19(월)

- Swift 5.9 Beta 기반으로 문서 작업
- 제어 흐름 (Control Flow) 챕터와 조건 표현식 (Conditional Expression) 섹션에 `if`와 `switch` 표현식에 대한 내용을 추가
- 컴파일 때 코드를 생성하는 것에 대한 매크로 (Macros) 챕터 추가
- 불투명 타입과 박싱 프로토콜 타입 (Opaque and Boxed Protocol Types) 챕터에 박싱 프로토콜 타입(boxed protocol type)에 대한 내용을 추가
- `buildPartialBlock(first:)`와 `buildPartialBlock(accumulated:next:)` 메서드에 대한 내용을 결과-구축 메서드 (Result-Building Methods) 섹션에 추가
- Swift-DocC 적용한 TSPLK (The Swift Programming Language Korean) 페이지 오픈

참고

GitBook과 Swift-DocC 둘 다 운영할 계획입니다.

2023-04-04(화)

- Swift 5.8 기반으로 문서 작업
- 오류 처리 외의 `defer`를 표시하는 연기된 동작 (Deferred Actions) 추가
- 오탈자 수정

2022-09-13(화)

- Swift 5.7 정식 릴리즈에 따른 오탈자 수정

2022-06-29(수)

- 오탈자 수정
- Swift 5.7 기반으로 문서 작업 완료
- 액터(actor)와 태스크(task) 간의 데이터 전송에 대한 내용을 Sendable 타입 (Sendable Types) 섹션에 추가하고 `@Sendable` 과 `@unchecked` 속성에 대한 내용을 Sendable과 unchecked 섹션에 추가하였습니다.
- 정규 표현식 생성에 대한 내용을 정규 표현식 리터럴 (Regular Expression Literals) 섹션에 추가하였습니다.
- `if-let` 형식에 대한 내용을 옵셔널 바인딩 (Optional Binding) 섹션에 추가하였습니다.
- `#unavailable`에 대한 내용을 API 가용성 검사 (Checking API Availability) 섹션에 추가하였습니다.

2022-03-17(목)

- Swift 5.6 정식 릴리즈에 따른 오탈자 수정

2022-01-29(토)

- Swift 5.6 기반으로 문서 작업 완료
- 연결된 메서드 호출과 다른 접미사 표현식과 관련된 `#if` 사용에 대한 정보로 명시적 멤버 표현식 (Explicit Member Expression)을 업데이트
- 이미지 파일 업데이트

2021-10-08(금)

- 프로퍼티 (Properties) 변경 사항 수정
- 동시성 (Concurrency) 변경 사항 수정

2021-08-14(토)

- Swift 5.5 기반으로 문서 작업 완료
- 동시성 (Concurrency) 추가
- 오탈자 수정

2021-04-10(토)

- WELCOME TO SWIFT 번역 완료
- LANGUAGE REFERENCE 번역 완료

2021-02-20(토)

- Swift 5.4 기반으로 문서 작업 완료

2020-10-23(금)

- Swift 5.3 기반으로 문서 작업 완료

번역자 보충

Swift 6.4 번역자 보충 (Swift 6.4 Translation Notes)

최신 언어 기능과 영문 원본의 기술적 정정을 확인합니다.

이 페이지는 Swift 6.4 기준의 번역자 보충 자료입니다. 영문 책에 아직 설명되지 않은 기능과 현재 컴파일러의 동작을 설명하기 위해 필요한 정정을 담고 있습니다. 각 절의 공식 Swift Evolution 제안을 함께 참고할 수 있습니다. 책에 설명이 추가된 버전과 언어 기능이 도입된 버전은 서로 다를 수 있습니다.

비동기 defer (Asynchronous Defer)

Swift 6.4에서는 비동기 컨텍스트의 `defer` 본문에서 `await`로 비동기 함수를 호출할 수 있습니다. 현재 범위를 벗어날 때 지연된 작업을 암시적으로 기다리고, 해당 작업이 완료된 후에 함수가 반환됩니다. 여러 `defer`의 실행 순서는 기존과 같이 선언 순서의 역순입니다.

```
func releaseResource() async {
    // Release a resource asynchronously.
}

func performWork() async {
    defer {
        await releaseResource()
    }
    print("Working")
}
```

`await`를 포함하는 `defer`를 동기 함수에 작성할 수는 없습니다. 클로저처럼 비동기 여부를 추론할 수 있는 컨텍스트에서는 `defer`의 `await`도 클로저가 비동기임을 추론하는 데 사용됩니다. `defer`는 둘러싸는 범위의 액터 격리를 상속합니다.

`defer` 본문에 작성한 호출의 `await`와 별도로, 범위를 벗어나면서 지연된 작업을 기다리는 암시적 중단 지점이 생깁니다. 따라서 모든 중단 지점이 호출 위치의 명시적 `await`에만 있다는 설명은 이 경우를 포함하도록 한정해야 합니다. `defer` 자체가 태스크의 취소 상태를 없애 주지는 않습니다.

자세한 내용은 SE-0493을 참고바랍니다.

some과 any의 옵셔널 구문 (Optional some and any Syntax)

Swift 6.4에서는 단일 프로토콜의 불투명 타입이나 박싱 프로토콜 타입을 옵셔널로 작성할 때 바깥쪽 괄호를 생략할 수 있습니다. `some P?`는 `(some P)?`와 같고, `any P?`는 `(any P)?`와 같습니다. 기존의 괄호를 사용하는 구문도 계속 유효합니다.

```
protocol Named {
    var name: String { get }
}

struct Person: Named {
    let name: String
}

let person: any Named? = Person(name: "Taylor")

func makePerson() -> some Named? {
    Optional(Person(name: "Taylor"))
}
```

프로토콜 합성을 사용할 때는 합성 자체를 괄호로 묶습니다. `any (P & Q)?`와 `(any P & Q)?`는 유효하지만, `any P & Q?`는 유효하지 않습니다.

자세한 내용은 SE-0521을 참고바랍니다.

경고 제어 속성 (Warning Control Attribute)

`@diagnose`는 특정 선언의 시그니처와 어휘적 범위에서 지정한 경고 그룹의 진단 방식을 조정합니다. 첫 번째 인자는 경고 그룹 이름이며, `as:` 인자는 `error`, `warning`, `ignored` 중 하나입니다. 선택적인 `reason:` 인자에는 문자열 보간이 없는 문자열 리터럴을 작성합니다.

```
@available(*, deprecated, message: "Use currentAPI() instead.")
func legacyAPI() {}

@diagnose(DeprecatedDeclaration, as: ignored, reason: "Compatibility bridge")
func compatibilityBridge() {
    legacyAPI()
}
```

이 속성은 지정한 그룹의 경고에만 영향을 줍니다. 컴파일 오류를 무시하도록 바꾸는 기능은 아닙니다. 중첩된 선언은 둘러싸는 범위의 설정을 상속하며, 해당 경고 그룹에 별도 `@diagnose`를 작성하여 설정을 조정할 수 있습니다.

SE-0522는 Swift 6.4에 구현되었습니다. 사용하는 툴체인이 실험적 기능 활성화를 요구하는 경우에는 `-enable-experimental-feature SourceWarningControl`을 사용합니다.

자세한 내용은 SE-0522을 참고바랍니다.

모듈 선택자 (Module Selectors)

`ModuleName::Name` 구문을 사용하면 지정한 모듈에 속하는 선언을 명확하게 참조할 수 있습니다. 여러 모듈에 같은 이름의 선언이 있거나, 모듈과 같은 이름의 타입이 모듈 이름을 가리는 경우에 유용합니다.

```
let value: Swift::Int = 42
```

점(`.`)을 사용하는 일반적인 멤버 접근과 달리, `::` 앞의 이름은 모듈을 선택하는 데 사용됩니다. 이 기능의 Swift Evolution 구현 버전은 Swift 6.3이며, Swift 6.4 릴리스 소개에서도 다루고 있습니다.

자세한 내용은 SE-0491을 참고바랍니다.

정리 작업의 취소 차단 (Cancellation Shields for Cleanup)

태스크 취소는 협조적으로 처리됩니다. 필수 정리 작업이 내부에서 취소 여부를 검사하고 실행을 건너뛰는 경우, `withTaskCancellationShield`를 사용하여 그 작업이 둘러싸는 태스크의 취소 상태를 관찰하지 않도록 보호할 수 있습니다. 이 API에는 동기 클로저와 비동기 클로저를 받는 형태가 있습니다.

```
func flushMetrics() async {
    guard !Task.isCancelled else { return }
    print("Flushing metrics")
}

func processResource() async {
    defer {
        await withTaskCancellationShield {
            await flushMetrics()
        }
    }
    print("Processing resource")
}
```

차단 범위에서는 `Task.isCancelled`와 `Task.checkCancellation()`이 둘러싸는 태스크의 취소를 관찰하지 않습니다. 이 범위에서 생성한 구조화된 자식 태스크에도 둘러싸는 태스크의 취소가 자동으로 전파되지 않습니다. 그러나 태스크의 취소 자체를 취소하거나 되돌리는 것은 아니며, 차단 범위를 벗어나면 원래 취소 상태를 다시 관찰합니다. 자식 태스크나 태스크 그룹을 명시적으로 취소하는 동작까지 막지는 않습니다.

자세한 내용은 SE-0504를 참고바랍니다.

박싱 프로토콜과 제네릭 호출 (Boxed Protocols and Generic Calls)

박싱 프로토콜 타입 자체가 해당 프로토콜을 준수하지 않는다는 사실과 그 값을 제네릭 함수에 인자로 전달할 수 있는지는 구분해야 합니다. Swift는 지원하는 호출 위치에서 existential을 암시적으로 열어, 박스에 저장된 구체적인 타입을 제네릭 매개변수에 바인딩할 수 있습니다.

따라서 불투명 타입과 박싱 프로토콜 타입 (Opaque and Boxed Protocol Types)의 `protoFlip(_:)` 예제에서 `protoFlip(protoFlip(smallTriangle))` 호출은 현재 Swift 6.4의 Swift 6 언어 모드에서 유효합니다. 반면 서로 다른 박스의 구체 타입이 같다고 보장할 수 없는 연산에는 여전히 제약이 있습니다. 모든 existential 사용이 모든 제네릭 컨텍스트에서 가능한 것은 아닙니다.

연관 타입이 있는 프로토콜도 명시적 박싱 프로토콜 타입인 `any Container`로 반환할 수 있습니다. `some Container`를 반환하는 경우와 비교하면, 반환값의 구체적인 타입과 연관 타입 정보를 보존하는 정도가 다릅니다.

자세한 내용은 SE-0352와 SE-0309를 참고바랍니다.

원시 식별자의 형식 문법 (Formal Grammar for Raw Identifiers)

현재 영문 원본의 `raw-identifier-character` 생산 규칙은 원시 식별자에 포함할 수 없는 문자를 일부만 나열합니다. 식별자 (Identifiers) 본문의 금지 문자 목록과 공백만 또는 연산자 문자만으로 된 이름을 허용하지 않는 조건을 함께 적용해야 합니다. 문법 요약의 짧은 생산 규칙만으로 본문에서 금지한 탭이나 유니코드 공백이 허용된다고 해석하면 안 됩니다.

자세한 내용은 SE-0451을 참고바랍니다.

연락처

연락처

오역이나 오타 발견 시 아래 연락처로 연락 바랍니다.

Contact Point

- Email: bbiguduk@gmail.com

원본 라이선스

Apache License Version 2.0, January 2004 http://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:

(a) You must give any other recipients of the Work or Derivative Works a copy of this License; and

(b) You must cause any modified files to carry prominent notices stating that You changed the files; and

(c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.

7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.

8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright [yyyy] [name of copyright owner]

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Runtime Library Exception to the Apache 2.0 License:

As an exception, if you use this Software to compile your source code and portions of this Software are embedded into the binary product as a result, you may redistribute such product without providing attribution as would otherwise be required by Sections 4(a), 4(b) and 4(d) of the License.